

Recipe For Quantitative Trading With Machine Learning

Daniel Bloch

6th of November 2023

The copyright to this computer software and documentation is the property of Quant Finance Ltd. It may be used and/or copied only with the written consent of the company or in accordance with the terms and conditions stipulated in the agreement/contract under which the material has been supplied.

*Copyright © 2023 Quant Finance Ltd
Quantitative Analytics, London*

Recipe For Quantitative Trading With Machine Learning

Daniel BLOCH *
QUANT FINANCE
Working Paper

31st of August 2018
Revised date 6th of November 2023
Revised version: 1.3.0

Abstract

On one hand, financial time series are multifractal, thus exhibiting non-Gaussian distribution, the presence of extreme values (outliers), and long-range dependent dynamics. On the other hand, machine learning (ML) models are processes relying heavily on statistical models and methodologies, but treated as black box models due to their inability to explicitly know the relations established between explanatory variables (input) and dependent variables (output). However, when forecasting market returns, or generating autonomous patterns, it is crucial to know the statistical properties of the time series produced by the ML model under consideration.

We first present some results on using recurrent neural networks to forecast market returns. We then discuss forecasting stock price directions and propose an algorithm for simultaneously forecasting both future price levels and asset price directions. We also discuss autonomous pattern generation and focus on learning traditional shapes defined by stock analysts. Then, we present complex network analysis to predict stock price fluctuation patterns. At last, we take into consideration the statistical characteristics of financial time series and present a recipe for forecasting both market returns and their directions. We choose to reverse the causality and propose a solution consisting in deciding upon the framework by defining how the model should be specified before beginning to analyse the actual data. We should define the framework by properly formulating model hypotheses which make financial or economic sense, and then carefully determining the number of dependent variables in a regression model, or the number of factors and components in a stochastic model.

Based on these observations, we use the multifractal formalism (MF) as a framework for testing the capacity of an ML model to reproduce some non-overlapping statistical properties of the time series. First we define a set of theoretical models with distinct statistical characteristics as well as a set of ML models with the ability to reproduce the properties of the models in the former set. We then train each individual ML model to replicate their respective theoretical model and use ensemble methods to combine these now calibrated ML models to form a meta-model. However, the weights of the latter are statistically computed as a function of the Hurst exponent. As a result, the meta-model is dynamically recombined based on the changing properties of the financial series over time. Once we have identified an ensemble of ML models with specific non-overlapping statistical properties, we can train each constituent model to learn a large number of patterns or technical indicators. We then devise a trading algorithm with strategies accounting for a specific level of Hurst exponent.

Machine Learning, Recurrent Neural Networks, Associative Reservoir Computing, Multifractal Formalism, Forecasting Returns And Directions, Trading Algorithm

*dan@quantfin.eu

1 Introduction

The accumulating evidence against the efficiency of the market (the efficient market hypothesis (EMH)) has caused a resurgence of interest in the claims of Technical Analysis (TA) (see Appendix (11)) as the belief that the distribution of price dynamics being totally random is now being questioned (see Appendix (10)). In general, the frequency distribution of returns is a fat-tailed, high-peaked distribution existing at many different investment horizons. Further, the standard deviation of returns increases at a faster rate than the square root of time. As a result, correlations is stochastic, and volatility is highly unstable. Taking into consideration these observations, Mandelbrot [1960] [1963a] [1963] described the financial market as a system with fat tails, stable distributions and persistence. While investing involves the study of the basic market fundamentals, which may take several years to be reflected in the market, trading involves the study of technical factors governing short-term market movements together with the behaviour of the market. As a result, trading is riskier than long-term investing, but it offers opportunities for greater profits (see Hill et al. [2000]). There exists a large body of work on the mathematical analysis of the behaviour of stock prices, stock markets and successful strategies for trading in these environments (see Section 2.1).

Assuming that the historical data in the markets forms appropriate indications about the market future performance, TA relies on indicators designed to help a trader determine whether current behaviour is indicative of a particular trend, together with the timing of a potential future trade. However, there is no better indicator, so that the indicators should be combined in order to provide different perspectives. Further, a technical indicator always need to be applied to a time window, and the problem of determining the best time window can be the solution to an optimisation problem (see Fernandez-Blanco et al. [2008]), which requires sophisticated search and examination methods.

Machine learning is about programming computers to learn and to improve automatically with experience (see A. Samuel [1959]). While computers can not yet learn as well as people, algorithms have been devised that are effective for certain types of learning tasks, and a theoretical understanding of learning has emerged. Computer programs developed, exhibiting useful types of learning, especially in speech recognition and data mining (see Mitchell [1997]). Thus, new techniques combining elements of learning, evolution and adaptation from the field of Computational Intelligence developed, aiming at generating profitable portfolios by using technical analysis indicators in an automated way. In particular, subjects such as Neural Networks (see Appendix (15.1.1)), Swarm Intelligence, Fuzzy Systems and Evolutionary Computation can be applied to financial markets in a variety of ways such as predicting the future movement of stock's price or optimising a collection of investment assets (funds and portfolios). These techniques assume that there exist patterns in stock returns and that they can be exploited by analysis of the history of stock prices, returns, and other key indicators (see Schwager [1996]). With the fast increase of technology in computer science, new techniques can be applied to financial markets in view of developing applications capable of automatically manage a portfolio. Consequently, there is substantial interest and possible incentive in developing automated programs that would trade in the market much like a technical trader would, and have it relatively autonomous. Mechanical trading systems (MTS) founded on technical analysis developed. It is a mathematically defined algorithm designed to help the user make objective trading decisions based on historically reoccurring events.

Supervised learning algorithms have been used to perform regression tasks as well as classification tasks, that is, predicting continuous values in the former and class labels in the latter. One can then build portfolios in a multi-asset classification/forecasting task (see Steiner et al. [1997]), and devise systematic timing strategies deployed in a trading system. If our goal is to forecast returns, or generate autonomous patterns, from a financial time series by learning some of its behaviour in the hope it repeats in the future, we should concentrate on series exhibiting long range dependence (LRD) rather than short time memory. Thus, it is crucial to understand the statistical properties of financial time series and concentrate on the latter. Further, machine learning (ML) is not a black box, but a process relying heavily on statistical models and methodologies (see Appendices (12) and (15.1.1)). It is inherently a multidisciplinary field drawing results from artificial intelligence, probability and statistics, computational complexity theory, control theory, information theory, philosophy, psychology, neurobiology, and other fields. In general, Recurrent Neural Networks (RNN) are designed to reproduce transient processes and must be modified (with feedback) to capture LRD properties. Hence, it would make sense to first design an ML model with the desired statistical properties, and then train it to

recognise as many technical indicators (or patterns) as possible.

We present financial markets in Section (2) and discuss the properties of asset returns, in particular the long range dependence (LRD) and the difficulties of measuring it empirically. Then, we introduce neural networks and focus on artificial neural networks (ANN) and recurrent neural networks (RNN). We briefly discuss the learning process and consider prediction tasks for (1) chaotic series prediction (regression problem), (2) autonomous pattern generation (classification problem). In Section (3), we test neural networks on the Mackey-Glass signal and several real-time financial time series in order to gain some insight on their capability of reproducing the dynamics of financial time series. We choose to model transient processes with Reservoir Computing (RC), which is an approach designed to train and analyse RNNs in a fast and efficient way (see Jaeger [2001]). There is no learning in the recurrent connections, only between the State units and the Output units, and adaptation is based on a simple associative learning mechanism (see Dominey et al. [2000]). RC models with output feedback were proposed to generate financial time series with interesting properties since feeding the output signal back into the reservoir allows to tackle prediction tasks. In order to improve the accuracy of the forecast returns, we also consider an online algorithm behaving as an adaptive filter throughout the running.

In Section (3.3), we repeat the analysis using Associative Reservoir Computing. Given that an associative connection between two entities is bidirectional, if the forward relation is many-to-one, the reverse relation (one-to-many) is ambiguous. This is always the case if there exists several causes resulting in the same effect. More formally, if there is a non-injective relation between inputs and outputs, there is no unique solution to the inverse problem. This problem is typical of financial time series and traditional feed-forward models can not represent multiple solutions to a mapping without additional knowledge. In the case of persistent processes we consider solving the ambiguous problem displayed by market returns by using the Associative Reservoir Computing Theory (ARC) to shape multi-stable attractor dynamics that reflect the multiplicity of solutions (see Reinhart et al. [2009]). To do so, we overwrite the reservoir forward representation by a joint representation involving input and output. That is, to learn a mapping from output to input we rely on a feedback matrix to access the latter, and define a matrix mapping the reservoir states to the input layer. We then test the model on the Mackey-Glass signal and FX time series.

The prediction of financial time series is a highly challenging task since they are multifractal. It requires models to capture subtle functional relationships among the empirical data even though the underlying relationships are unknown or hard to describe. Fortunately, such relationships are not required as far as predicting future trend is concerned. This is because the direction of an underlying asset refers to the movement of the asset price or the trend of fluctuation in the underlying asset in the future. Therefore, the change of a feature over time is more important than the absolute value of a feature. Further, forecasting the trends results in networks that are less complex than when forecasting the price levels. We describe in Section (4) several algorithms for forecasting the directions of the underlying asset. While accuracy is a good measure for the general ability of a network, it must be combined with return metrics taking into account the magnitude of the trends that a system discovers. One solution is to consider a framework for simultaneously forecasting both future price levels and asset price directions. We describe in Section (5) an algorithm to forecast both market returns and their directions. To conclude, we draw conclusions on the results and define a set of rules we want to follow when building our trading algorithm. Since stock prices consist of several patterns, which appear repeatedly at time intervals, it is logical to focus on learning traditional shapes defined by stock analysts. In Section (6), we discuss autonomous pattern generation and their prediction in view of forecasting price trends, where the patterns are distinctive formations of market prices. It is now well known that supervised learning methods are inadequate for multi-step prediction-learning problems, and that temporal difference (TD) are far preferable. Thus, we also introduce temporal differences (TD) procedures and discuss how to learn to predict financial patterns, which are multi-step prediction-learning problems. Further, we show how to use TD procedures to infer information about the magnitude of future market returns.

In Section (7), we use the multifractal formalism (MF) as a framework for testing the capacity of an ML model to reproduce some statistical properties of time series. We consider a set $\mathcal{T}$ of theoretical models $\mathcal{T}_m$, $m = 1, \dots, M$,

having non-overlapping pre-defined statistical properties. We also consider a set $\mathcal{M}$ of ML models $\mathcal{M}_m$, $m = 1, \dots, M$, each one of them capable of reproducing one theoretical model from the set $\mathcal{T}$. We train each ML model $\mathcal{M}_m$ to approximate its associated theoretical model $\mathcal{T}_m$ (calibration), and use ensemble methods to combine the calibrated ML models $\mathcal{M}_{calib,m}$, obtaining a meta-model. However, the weights of the latter are statistically computed as a function of the Hurst exponent. As a result, the meta-model is dynamically recombined based on the changing properties of the time series over time. Once we have identified an ensemble of ML models with specific non-overlapping statistical properties we can train each model to learn a large number of patterns or technical indicators. We then devise a trading algorithm with strategies accounting for a specific level of Hurst exponent.

2 Presentation

Machine learning (ML) can give computers the ability to learn without being explicitly programmed (see A. Samuel [1959]). It focuses on designing, developing and analysing methods giving a computer the ability to follow a systematic process leading to producing results in difficult tasks that classical algorithms could not produce. As such, it is a general method for solving optimisation problems (see Appendix (15.1.1)).

Asset returns and their properties are discussed in Appendix (9). We are going to consider forecasting asset returns as well as their directions with machine learning, and use the results to devise quantitative trading strategies. In the former, prediction task is directly applied to the market returns, while in the latter, data mining is performed on data sets and classification is privileged among the different possible tasks (see Appendix (12)). We are going to briefly introduce the prediction tasks and discuss training and exploitation of the model. We will then present some results on the Mackey-Glass signal and FX market returns.

2.1 Market returns

The stock market is an organised and regulated financial market where securities such as bonds, notes, and shares are bought and sold at prices governed by the forces of demand and supply. This demand coupled with advances in trading technology has opened up the markets for investors to perform quantitative trading with minimum risk. The price of an asset as a function of time constitutes a financial time series. High-frequency financial data are observations on financial variables taken daily or at a finer time scale, and are often irregularly spaced over time. We call the tick time scale the finest time scale and denote it t_{tick} . It corresponds to transaction-by-transaction data on trades and quotes. These ticks are irregularly spaced time series with random daily numbers of observations. In general, for FX quotes data, the date and time are expressed in conventional format as *dd.mm.yyyy* and *HH : MM : SS*, respectively. Consequently, a lot of attention had been devoted to the analysis and prediction of future values and trends of the financial stock markets (see Hargreaves et al. [2013]). Following notation in Appendix (9), we let the underlying process $(S_t)_{t \geq 0}$ be a one-dimensional Ito process valued in the open subset D . If the d -period gross return on a security is just $1 + R_{t-d,t}$, then the continuously compounded return, or logarithmic return, is defined in Equation (9.12). We are now going to discuss the properties of asset returns, test for trend and mean-reversion, and briefly introduce the multifractal market.

2.1.1 Assessing the properties of asset returns

2.1.1.1 Some empirical results With the availability of new sources of financial data, the 1990s saw a surge of interest in whether or not there is long-range dependence (LRD) in asset returns. The concepts of self-similarity, scaling, fractional processes were used to assess market returns. For instance, various authors used statistical methods of inference to estimate the tail index without assuming a particular shape of the entire distribution (Jansen et al. [1991], Loretan et al. [1994], Lux [1996], Dacorogna et al. [2001]). The tail index, α , was found to be in the range of 3 to 4, giving weight to the stability of the tail behaviour under time aggregation. As a result, it was then assumed that the unconditional distribution of returns converged toward the Gaussian distribution, but was distinctly different from it at the daily (and higher) frequencies. Hence, the non-normal shape of the distribution motivated

the quest for the best non-stable characterisation at intermediate levels of aggregation. While the dependency of long lasting autocorrelation was subject to debate for raw (signed) returns, it was plainly visible in absolute returns, squared returns, or any other measure of the extent of fluctuations (volatility). Still using statistical tests, other authors studied long term dependence with more or less success on finding deviations from the null hypothesis of short memory for raw asset returns, but strongly significant evidence of long memory in squared or absolute returns (see Lo [1991], Ding et al. [1993], Longin [1996]).

Even though it has been assumed little evidence of fractional integration in stock returns, long memory has been identified in the first differences of many economic series a long time ago. For instance, Mandelbrot [1971] argued against martingale models and Maheswaran et al. [1993] suggested potential applications in finance for processes lying outside the class of semimartingales. Econophysics developed to study the herd behaviour of financial markets via return fluctuations, leading to a better understanding of the scaling properties based on methods and approaches in scientific fields. To measure the multifractals of dynamical dissipative systems, the generalised dimension and the spectrum have effectively been used to calculate the trajectory of chaotic attractors that may be classified by the type and number of the unstable periodic orbits. Even though a time series can be tested for correlation in many different ways (see Taqqu et al. [1995]), some attempts at computing these statistical quantities emerged from the box-counting method, while others extended the R/S analysis (see Mandelbrot et al. [1979]). The moment-scaling properties of financial returns have been the object of a growing physics literature confirming that multiscaling was exhibited by many financial time series (see Vandewalle et al. [1998], Schmitt et al. [1999], Pasquini et al. [2000]). Since scaling analysis and multifractal analysis developed, various authors performed empirical analysis to identify anomalous scaling in financial data (see Calvet et al. [2002]).

Since the absence of long range dependence (LRD) in returns is still compatible with its presence in absolute returns (or volatility), several authors suggested models such as the Fractionally Integrated GARCH models, where returns have no autocorrelation but their amplitudes have LRD (see Baillie [1996]). Many continuous multifractal models, such as the multifractal model of asset returns (MMAR), have been proposed to capture the thick tails and long-memory volatility persistence exhibited in the financial time series (see Mandelbrot et al. [1997]). Such models are consistent with economic equilibrium, implying uncorrelated returns and semimartingale prices, thus precluding arbitrage in a standard two-asset economy. Returns have a finite variance, and their highest finite moment can take any value greater than 2. However, the distribution does not need to converge to a Gaussian distribution at low frequencies and never converges to a Gaussian at high frequencies, thus capturing the distributional nonlinearities observed in financial series. These multifractal models have long memory in the absolute value of returns, but the returns themselves have a white spectrum. That is, there is long memory in volatility, but absence of correlation in returns. Subsequent literature moved from the more combinatorial style of the multifractal model of asset returns (MMAR) to iterative causal models of similar design principles, such as the Markov-switching multifractal (MSM) model proposed by Calvet et al. [2004] and the multifractal random walk (MRW) introduced by Bacry et al. [2001], constituting the second generation of multifractal models.

Examining the stability of the Hurst exponent, H , on financial time series on the basis of characteristic values, such as rescaled ranges or fluctuations analysis, some authors (see Vandewalle et al. [1998c], Costa et al. [2003], Cajueiro et al. [2004], Grech [2005]) observed that the values of H could be significantly higher or lower for a specific scale. Using detrended fluctuation analysis (DFA), or detrended moving average (DMA), to analyse asset returns on different markets, various authors observed that the Hurst exponent would change over time indicating multifractal process. They found that the exponent values and the range over which the power law holds varied drastically from one underlying asset to another one, obtaining three categories, the persistent behaviour, the antipersistent behaviour, and the strictly random one. In addition, they showed in some stocks and some exchange rate that the Hurst exponent was changing with time with successive persistent and antipersistent sequences (see Costa et al. [2003], Kim et al. [2004]). Several authors proposed to use methods of long-range analysis such as DFA or DMA to determine the local correlation degree of the series by calculating the local scaling exponent over partially overlapping subsets of the

analysed series (see Vandewalle et al. [1998c], Costa et al. [2003], Cajueiro et al. [2004], Carbone et al. [2004], Matos et al. [2008]).

Later, measuring multifractality with either DFA, DMA, or wavelet analysis, and computing the local Hurst exponent on sliding windows, a large number of studies confirmed multifractality in stock market indices, commodities and FX markets, such as Matia et al. [2003], Kim et al. [2004], Matos et al. [2004], Norouzzadeh et al. [2005]. Further studies confirmed multifractality in stock market indices such as Zunino et al. [2007] [2008], Yuan et al. [2009], Wang et al. [2009], Barunik et al. [2012], Lye et al. [2012], Kristoufek et al. [2013], Niere [2013], to name but a few. Other studies confirmed multifractality on exchange rates such as Norouzzadeh et al. [2006], Wang et al. [2011b], Barunik et al. [2012], Oh et al. [2012], while some confirmed multifractality on interest rates such as Cajueiro et al. [2007], Lye et al. [2012], as well as on commodity such as Matia et al. [2003], Wang et al. [2011].

Then methods for the multifractal characterisation of nonstationary time series were developed based on the generalisation of DFA, such as the MFDFA by Kantelhardt et al. [2002]. Consequently, the multifractal properties as a measure of efficiency (or inefficiency) of financial markets were extensively studied in stock market indices, foreign exchange, commodities, traded volume and interest rates (see Matia et al. [2003], Ho et al. [2004], Moyano et al. [2006], Zunino et al. [2008], Stosic et al. [2014]). For instance, Zunino et al. [2008] used MFDFA to analyse the multifractality degree of a collection of developed and emerging stock market indices. Gu et al. [2010] analysed the return time series of the Shanghai Stock Exchange Composite (SSEC) Index with the one-dimensional MFDMA model within the time period from January 2003 to April 2008, and confirmed that the series exhibits multifractal nature, not caused by fat-tailedness of the return distribution. Lye et al. [2012] used MFDFA coupled with the rolling window approach to scrutinise the dynamics of weak form efficiency of Malaysian sectoral stock market, and showed that it was adversely affected by both Asian and global financial crises.

It was also shown that observable in the dynamics of financial markets have a richer multifractality for emerging markets than mature one. As a rule, the presence of multifractality signals time series exhibiting a complex behaviour with long-range time correlations manifested on different intrinsic time scales. Considering an artificial multifractal process and daily records of the *S&P* 500 index gathered over a period of 50 years, and using multifractal detrended fluctuation analysis (MFDFA) and multifractal diffusion entropy analysis (MFDEA), Jizba et al. [2012] showed that the latter possesses highly nonlinear, and long-ranged, interactions which is the manifestation of a number of interlocked driving dynamics operating at different time scales each with its own scaling function. Such a behaviour typically points to the presence of recurrent economic cycles, crises, large fluctuations (spikes or sudden jumps), and other non-linear phenomena that are out of reach of more conventional multivariate methods (see Mantegna et al. [2000]).

2.1.1.2 The difficulties of measuring empirically LRD We saw in Section (2.1.1.1) that financial systems were assumed to display scaling properties similar to those of systems in statistical physics, such that the existence of long-term correlation could be empirically assessed using

1. fractal analysis
2. statistical self-similarity analysis (using various statistical methods)

However, these statistical approaches are based on the moment properties of stochastic processes and must be restricted to second-order stationary processes. In addition, fractal properties are only verified in the infinitesimal limit. Nonetheless, a large number of statistical studies on market prices checked for the existence of long-range and short-range power law correlation in financial data. Most of these studies did not find temporal correlations present in the system for price changes, but they did for absolute price changes, the average of absolute price changes, the square root of the variance, and the interquartile range of the distribution of price changes. It is well known that it is difficult to test for a particular parametric model, such as α -stable Levy processes or fractional Brownian motions, since self-similarity can have very different origins. For instance, studies estimating the tail exponent directly from the tail

observations tends to overestimate the characteristic exponent α . Also, the true tail behaviour of L-stable laws is only visible for extremely large data sets, leading to strongly misleading results when the sample is not large enough, such as the rejection of the L-stable regime (see McCulloch [1997]). Thus, we should use high-frequency data and consider the most outlying observations when estimating the tail index. In addition, if the time series of asset returns possess the two features of heavy tails and long range dependence, then many standard statistical tests will fail to work (see Resnick et al. [1999]). Also, Resnick et al. [2000] give examples of such processes where sample autocorrelations converge to random values as the sample size grows. Further, we have seen that slow decay of sample autocorrelation functions could indicate non-stationarity, making it hard to distinguish between stationary long memory models and other non-stationary models. Mikosch et al. [2003] observed that the non-stationarity of returns may generate spurious effects easily mistaken for LRD in the volatility. Granger et al. [2004] considered the interaction of LRD with non-stationarity by combining an underlying long memory process with occasional structural breaks. More generally:

- statistical analysis applies to processes with stationary increments
- estimators are vulnerable to trends in data, periodicity, large fluctuations, etc.
- estimators of Hurst exponent works for very long, or infinite, time series
- in the time domain, at high lags only a few samples are available
- in the frequency domain, at frequencies near zero, measurement errors are largest

Yet, stock returns and FX rates suffer from systematic effects mainly due to the periodicity of human activities, and can not be considered as processes with stationary increments. Clegg [2006] showed that LRD was a very difficult property to measure in real life because the data must be measured at high lags/low frequencies where fewer readings are available, and all estimators are vulnerable to trends in the data, periodicity and other sources of corruption.

When eliminating these problems, academics found correlations present in systems and highlighted the multifractal nature of financial time series. It was shown that self-similar models were too restrictive to explain financial time series, as they were unable to capture more fully the complex dynamics of the series. While all these models assume that the response times are trial-independent random variables, financial series exhibit multifractal scaling behaviour since a multitude of scaling exponents are necessary to fully describe the series. Simply put, financial time series exhibit

- non-Gaussian distribution,
- the presence of extreme values (outliers),
- long-range dependent dynamics.

These characteristics of financial time series indicate:

1. that the variations can not be exclusively described by the scaling of the variance alone, but that the scaling of higher order statistical moments must be considered.
2. intermittent changes in the magnitude of response time variation, which might be due to feedback effects, or, changes in investor's behaviour. These changes provide temporal modulation of both the width and shape of the response time distribution, and consequently, temporal modulation of the scaling exponent.

2.1.2 Testing for trend and mean-reversion

Bloch [2014] used the effective local Holder exponents (ELHE) (see Appendix (10.4.3)) to analyse markets for multifractality and obtained highly random local Holder exponents, emphasising the strong multifractal nature of financial data. We present some results on the equity and the FX markets.

2.1.2.1 The equity market We consider the price of "Google US" from August 22nd, 2012 to September 5th, 2014, and apply the same techniques as above to detect trends and mean reverting behaviours on a close-to-close strategy (we trade only at the close of the market everyday).

The Figure (1) represents Google prices from August 22nd, 2014 to September 5th, 2014. We have also plotted its trend computed using a wavelet denoising method. We observe that the stock prices and the trend coincides in most of the period, except in few time intervals, where it is strongly pushed away and then pulled back. For example, the prices fluctuates around the trend just before 2013, in January 2014 and more significantly between March and May 2014. This phenomena suggest to adapt a trend-following strategy when the price process coincides with the local trend, and on the contrary, we want to apply a mean reverting strategy when the price fluctuates around the trend. To do so, we need a indicator that allows us to take a decision systematically. We observe that the local Hurst exponent coincides exactly with the time interval where we observed strong fluctuations around the trend of the price process. Indeed, just before 2013, we observe a local minimum on the local Hurst exponent corresponding exactly with the fluctuations. The same observations may be made in January 2014. More significantly, between February and May 2014, the local Hurst exponent dropped from 0.55 to 0.35, and again, this phenomena exactly corresponds to the huge fluctuations interval observed in the price process. In addition, the global minimum of the local Hurst exponent occurs exactly when the price jumped from 450 to 500 in August 2013. In that period of time, the price is indeed highly mean reverting, since the difference between the trend (blue) and the actual process (black) is high. Hence, we can infer with a high probability that the process is more likely to come back to its trend. The autocorrelation is thus highly negative and the local Hurst exponent is subsequently close to zero.

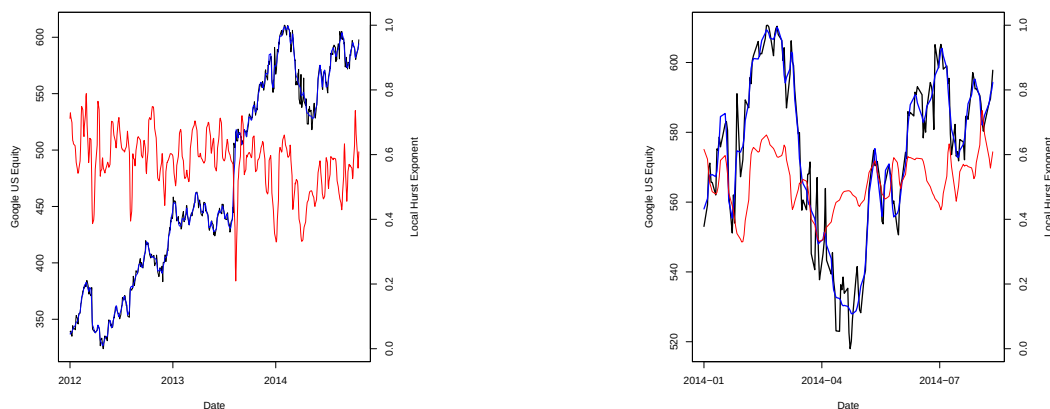


Figure 1: Google Equity Price from August 22nd, 2012 to September 5th, 2014 (black line) its trend (blue line) and its corresponding denoised Local Hurst exponent by MFDFA method (red line) and a zoom between January 2014 and August 2014

2.1.2.2 The FX market The high liquidity in the FX market allows us to define trading strategies at a high-frequency. As illustrated in the figure below, the data is very noisy on this time scale, making it highly difficult to trade directly on the currency pair. One solution could be to use wavelet analysis to extract the trend of the currency pair for different time scales and then build a strategy considering those market imperfections on each time scaling.

Figure (2) represents the EUR / GBP currency pair from August 31st to September 5th. We plotted its trend showing its moves of scale 4 (which represents 4 minutes) and its trend for the moves of scale 2 (which depicts 1 minute). While we observe that the two trends are roughly equal in most of the period, we can also see that the red line sometimes slightly fluctuates around the blue line. This is the case on the September 1st at around 10:00, the September 2nd at around 10:00, the September 3rd at around 10:00 and on the September 4th at around 14:00. In those periods,

when the red line is below the blue line, the red line is most likely going to increase in the next ticks, and conversely, when the red line is above the blue line, it is most likely going to decrease in the next ticks. This phenomena exactly corresponds to a mean-reverting behaviour of the red process around its local mean, modelled by a longer trend (blue process). Further, this behaviour also coincides exactly with a drop in the local Hurst exponent. Therefore, we can propose a simple trading strategy:

1. When the local Hurst exponent is under a fixed threshold (typically 0.3 here), we compute the two trends (blue and red) of the EUR/GBP currency pair and we check the relative position of the blue line on the red line. If the red line is under the blue line, we buy Euros since its value will most likely increase. On the contrary, if the red line is above the blue line, we short sell Euros since its value will most likely decrease.
2. When the local Hurst exponent is over the fixed threshold, we buy and hold Euros if the trend is upward. On the contrary, we sell Euros if the trend is downward.

Obviously, to get an exact idea of the viability of this strategy in practice, many other factors have to be taken into account, such as taxes, transaction costs, technological means and the total traded amount, since the moves in the price are very low in this frequency (around 1/100 euros per tick).

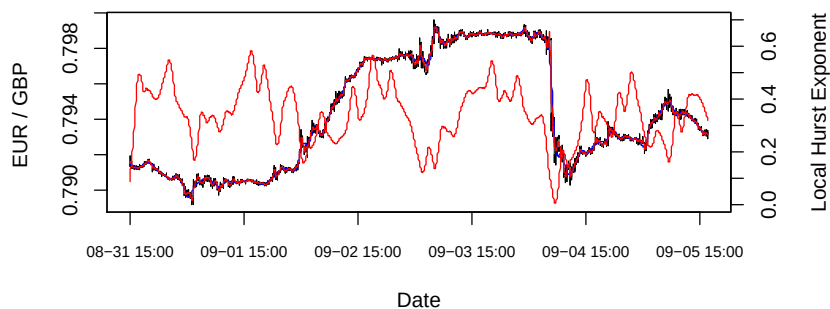


Figure 2: Tick-by-tick (20sec) EUR / GBP currency pair from August 31st, 2014 to September 5th, 2014 (grey line), its trend on different scale (red and blue lines) and its corresponding denoised Local Hurst exponent by MFDFA method (thin red line).

2.1.3 A multifractal market

We used in Section (2.1.2) the effective local Holder exponents (ELHE) (see Appendix (10.4.3)) to analyse markets for multifractality. Highly random local Holder exponents were obtained, emphasising the strong multifractal nature of financial data. Even the denoised effective exponents obtained with wavelet transform are random with abrupt change in values happening continuously, and extremely fast. That is, the denoised local Holder exponents oscillate around a level of Hurst exponent with a succession of small and large amplitudes (similar to spikes), and is capable of sudden, or abrupt, change to a different level of Hurst exponent in presence of large extreme price fluctuations related to market crashes. Various authors already suggested that financial markets could be described as a system of a number of coupled oscillators, subject to stochastic regulation (see Matia *et al.* [2003]). The source of multifractality of financial markets were shown to be due to nonlinear phase ordering as well as the essential contribution of extreme events resulting in fat tails of the event distribution. Even though the width of the multifractal spectra is capable of indicating the presence of large shocks, the oscillating nature of the local Holder exponent characterise the continuously changing dynamics of the response time distribution. Comparing the behaviour of the stock market to that of a healthy heartbeat dominated by two antagonistic systems (sympathetic and parasympathetic) the pessimists who sell stocks and the optimists who buy stocks, leading to a continuously varying number of active agents (degrees of freedom), Struzik

[2003] showed that both systems were at work at any moment in time. Hence, the multifractal spectra can detect a change of level of the Hurst exponent (contribution to fat tails), but it can not be used to identify the intermittent changes in the magnitude of response time variation due to, for example, feedback effects. Similarly to outliers, the latter requires a methodology capable of determining the statistical nature of the non-stationary process both globally and locally, such as the effective local Holder exponent.

2.2 Neural networks

2.2.1 An overview

Among the different networks existing, the artificial neural networks (ANNs) and the artificial recurrent neural networks (RNNs) are computational models designed by more or less detailed analogy with biological brain modules. The former is presented in Appendix (15.1.1) and the latter is introduced in Appendix (16.4.1). ANNs, formalised by McCulloch et al. [1943], are a class of generalised nonlinear nonparametric models inspired by studies of the human brain. They get their intelligence from learning process, giving them the capability of auto-adaptability, association, and memory to perform certain tasks. The backpropagation network, which is the most popular and the most widely implemented neural network in the financial industry, is based on a multi-layered feedforward topology with supervised learning (see Refenes et al. [1997]). The network is fully connected, with every node in the lower layer linked to every node in the next higher layer via weight values. The learning of the backpropagation neural network is based on an error minimisation procedure where the weights are modified according to an error function comparing the neural network output with the training targets. We evaluate the derivatives of the error function with respect to the weights which are used to compute the adjustment to be made to the weights. The simplest such techniques involves gradient descent. In the case of time series prediction, a set of input-target pairs is created, forming the training samples. Every time a new time series is generated, new observations are added to the set and the oldest ones are dropped out. Alternatively, artificial recurrent neural networks (RNNs) are a class of feedback artificial neural network architecture that uses iterative loops to store information, which is inspired by the cyclical connectivity of neurons in the brain. The existence of cycles allows RNNs to develop a self-sustained temporal activation dynamics along its recurrent connection pathways, even in the absence of input, making them a dynamical system. This feature of the RNNs make them attractive for processing serially correlated time series. Unlike the ANNs and traditional time series models, the RNNs are flexible enough, and avoids taking the size of sliding windows into account because they can decide what to store and what to ignore during the learning process.

In real-time data, training neural networks involves the optimisation of non-convex objective functions, which makes the computation of the gradient descent challenging and in turn leads to a costly or even infeasible learning process. One solution is to randomly assign a subset of the networks' weights, so that the resulting optimisation task can be formulated as a linear least-squares problem. Numerous experimental results indicate that such randomised models can match, or even improve, adaptable ones, with a number of favourable benefits, such as, simplicity of implementation, faster learning with less intervention from human beings, possibility of leveraging over all linear regression and classification algorithms. Such improvements in standard RNNs design were proposed independently by Maass et al. [2002] under the name of Liquid State Machines and Jaeger [2001] under the name of Echo State Networks (ESN). Over time, these types of models became known as Reservoir Computing (RC). RC assumes that supervised adaptation of all inter-connection weights are not necessary, and only training a memoryless supervised readout from it is sufficient to obtain good results. Thus, it avoids the shortcomings of the gradient-descent training of RNNs. RC is based on the computational separation between a dynamic reservoir and a recurrence-free readout (see details in Appendix (16.6)).

2.2.2 Modelling changing environment

2.2.2.1 Multiple characteristic timescales In general, the dynamics of RNN models evolve according to precise timing, while real-world time series are characterised by multiple seasonalities (or patterns). Substantial literature

exists on modelling different temporal resolutions (multiple characteristic timescales), especially in presence of long-term relations, among which the Long Short Term Memory (LSTM) became a reference (see Hochreiter et al. [1997]). Several variants have been proposed to improve the simultaneous learning of long and short time relationships (see Cho et al. [2014]). Other methods developed based on the idea that temporal relationships are hierarchically structured, so that RNNs should be organised accordingly (see El Hiji et al. [1995]). It led some models to be implemented by stacking multiple recurrent layers (see Graves [2013], Chung et al. [2015]). In Reservoir Computing, characterised by fading memory preventing to model slow periodicities, solutions were proposed to slow down the dynamics of the reservoir by considering digital bandpass filters. The latter encourages the decoupling of the dynamics within a single reservoir by identifying particular frequency-domain characteristics. A large amount of filters are initialised with random cutoff frequencies to encode a wide range of timescales in the recurrent layer, providing the correct timings to capture the target dynamics. Alternatively, the filters can be manually tuned based on the information of the frequency spectrum of the desired response signal (see Siewert et al. [2007], Wyffels et al. [2008]).

An important development in machine learning was the use of ensemble models, which simultaneously combine different types of classifiers. The idea being that classifiers of different types and/or using different data exhibit distinct strengths and weaknesses. As a result, it has been assumed that combining multiple classifiers could improve prediction accuracy (see Ho et al. [1994]). Jaeger [2004] discussed the fact that a single reservoir could not train a multiple superimposed oscillator (MSO). We know from classic kernel-based signal representation (Fourier and wavelet transform) that an individual kernel can represent an almost unique portion of the target signal. Further, a single reservoir can reproduce the behaviour of a sine-wave generator (see Jaeger [2001]). We can therefore construct an ESN with multiple reservoirs, each of which having a different configuration, to generate multiple sine waves of different frequencies. The problem being to define the grouping of neurons into several separate reservoir in a non-redundant manner.

Wiestra et al. [2005] explained that all the neurons in the same reservoir are coupled, when the task requires the simultaneous existence of multiple decoupled internal states. One solution was to consider an evolutionary-based reservoir design method, called Evolino. Xu et al. [2007] proposed the decoupled echo state network (DESN) involving the use of lateral inhibition. It is a structure making use of multiple reservoirs competing with each other through inhibitory connections and combining the internal states of all the reservoirs to form the output signal. Bianchi et al. [2017] proposed a network made of several recurrent groups of neurons trained to separately adapt to each timescale of the input signal. The recurrent layer of the network is randomly generated, but the connections must form a structure with K groups of bandpass neurons where each group $\mathcal{C}_k$ contains N neurons strongly connected to the other neurons in $\mathcal{C}_k$. Further, a small amount of connections between neurons belonging to different groups are also generated. The bandpass parameters are learned via backpropagation.

In most of these methods a large number of parameters must be learned in order to adapt the network to process the right time scales. It results in a long training time with the possibility of overfitting data. In general, these parameters are retrieved through an analysis in the frequency domain or by a priori knowledge of the input data.

2.2.2.2 Long range dependence In general, memory or persistent internal states can not be modelled by fading memory systems. However, the latter can be enlarged through feedback from trained readouts. Maass et al. [2005a] [2005b] presented a computational theory characterising the gain in computational power achieved through feedback in dynamical systems with fading memory. As a result, anti-persistent systems using feedback can obtain universal computational capabilities similar to those of persistent systems. It leads to high-dimensional attractor-based models capable of capturing the characteristics of persistent signal response.

In the field of autonomous robot locomotion, stable pattern generation and robustness against perturbations is necessary. For instance, learning rhythmic stable motions in a controllable way is usually modelled with central patterns generators (CPG). In this framework, the output feedback can generate a suitable slow dynamics with high precision. As a result, RC models with output feedback were proposed to generate CPGs with interesting properties. In order

to properly feedback the forecast and to modify appropriately the output weights if needed, one needs to implement an online algorithm. That is, when the reservoir is trained online, the model is adapted to recognising patterns and predicting chaotic time series (see Wyffels et al. [2009], Antonik et al. [2016]).

Reinhart et al. [2008] [2009] provided a new approach for training offline, or online, algorithms based on the transient/attractor concepts called Associative Reservoir Computing (ARC). As an example, an associative connection between two entities is bidirectional. The process denotes the recall of one entity from the other. Further, Reinhart [2011] introduced the concept of forward and inverse models in RC models. If the forward relation is many-to-one, the reverse relation becomes ambiguous, that is, one-to-many. Thus, there is a loss of information. One solution is to use additional information that is integrated over time. It can be modelled with an adaptive dynamical system to facilitate bidirectional and continuous association. The author pointed out the necessity of having a feedback in bidirectional association context. The main idea being to solve the ambiguous inverse problems by letting the model remain in an attractor network by teaching the reservoir states to stay stable. At every time step, the algorithm makes the entire network (input, reservoir and output) converge, thus eliminating transient effects. It is very important when one considers an output feedback matrix as it might add transient effect. Since feedback of erroneous outputs into the network can lead to error amplification, the concept of output feedback dynamics is formalised and an output feedback stability criterion is proposed.

2.2.2.3 The variable weight neural network In conventional neural networks, connection weights are fixed. That is, the characteristics of the neural network is invariant (the invariant type neural network). The adaptive ability of neural networks to an unknown environment depends on its generalisation capability. However, there exists a limit of generalisation ability in invariant type neural networks. Yasuda et al. [2006] proposed the neural network with variable connection weights, which changes its connection weights and its characteristic according to the changing environment. Analysing an obstacle avoidance problem for an electric-powered wheelchair, they split the weights in two terms where the first term are basic quantities giving the basic motion of obstacle avoidance and the second terms adjust the connection weights in order to adapt the neural network to the change of environment in the vicinity of the wheelchair. The latter are calculated by using another neural network, whose inputs are outputs of PSD sensors. As a result, the law producing avoidance orders, changes according to the current condition of the obstacle in the vicinity of the wheelchair. Thus, this mechanism adapts the characteristics of neural network to the changing environment at every moment and corrects the generation algorithm of obstacle avoidance operation appropriately.

Lam et al. [2014] presented the variable weight neural network (VWNN), allowing its weights to be changed in operation according to the characteristic of the network inputs. Analysing the problem of surface material recognition and epilepsy seizure phases recognition, they demonstrated its ability to adapt to different characteristics of input data resulting in better performance than with fixed weights neural networks. A VWNN consists of two traditional neural networks, namely tuned and tuning neural networks. The former is the one which actually classifies the input data, while the latter provides the weights to the tuned neural network according to the characteristic of the input data. The VWNN works on the principle that the connection weights are function to the external input signal. Thus, in theory, the VWNN can be viewed as an infinite number of traditional neural networks with fixed weights.

2.2.3 Interpreting neural networks

Neural networks (NNs) are based on predefined equations or formulae (see Appendix (15.1)) and have the capability of modelling complex statistical interactions between features for automatic feature learning. In order to perform a task, data scientists create an architecture (topology) whose (meta)-parameters are able (or not) to provide correct answers when some new input data is presented. These parameters are the key to their knowledge (see Palmer et al. [2002]), which depends on their generalisation ability.

Given this singular way of learning, NNs have been treated as black box models for their lack of interpretability compared with classical statistical models. That is, their inability to know in an explicit way the relations established between explanatory variables (input) and dependent variables (output).

Since the recent applications of NNs in the industry are intended to make critical decisions, it has become paramount

to understand how these models make predictions. To do so, academics focused on explaining individual feature importance and determining the contribution of explanatory variables. One need first to discover (detect) the interactions and then interpret them, which is not an easy task. The methods used for detecting interactions from NNs are usually statistical methods. In general, they rely on the assumption that the response signal is stationary with independent and identically distributed increments. Further, these tests rely on the idea that NNs are capable of successfully modelling complex statistical interactions.

2.2.3.1 Explaining the input feature importance Several approaches have been proposed to interpret NNs, classified as follows:

1. direct interpretation: explaining individual feature importance (Ross et al. [2017]), developing attention-based models, which illustrate where neural networks focus during inference, providing model-specific visualisations, such as feature map and gate activation visualisations (see Yosinski et al. [2015]).
2. indirect interpretation: post-hoc interpretation of feature importance (see Ribeiro et al. [2016]), knowledge distillation to simpler interpretable models.

The discovery of interactions (or statistical interaction detection) has become an essential part of the literature on neural networks. Two general approaches exist:

1. to conduct individual tests for each combination of features. Examples are ANOVA and Additive Groves (see Sorokina et al. [2008]). Both methods are computationally expensive in dimension higher than three.
2. to pre-specify all interaction forms of interest, then use Lasso to simultaneously select which are important. They are quick at selecting interactions but require specifying all interaction terms of interest (see Sun [1999]).

Interpreting NNs is far more difficult, but new methods have recently developed for traditional feedforward form and deep architectures. Methods such as feature map visualisation, de-convolution, saliency maps have been especially important to the vision community for understanding how convolutional networks represent images. In the case of long short-term memory networks (LSTMs), a research direction has studied multiplicative interactions in the unique gating equations of LSTMs to extract relationships between variables across a sequence (see Arras et al. [2017], Murdoch et al. [2018]). Tsang et al. [2018] proposed the Neural Interaction Detection (NID), which detects statistical interactions of any order or form captured by a feed-forward neural network, by examining its weight matrices. Top-K true interactions are determined from interaction rankings by using a special form of generalised additive model, which accounts for interactions of variable order.

2.2.3.2 Determining the contribution of explanatory variables Several methods have been proposed to assess the relative importance (contribution) of each explanatory variable: methods from the family of sensitivity analyses (SA) (perturb method, profile method) (see Cortez et al. [2013]), numeric sensitivity analyses (NSA) computing the slope between input and output (partial derivative method (PaD)) and even the curvature effects (second order derivatives), and methods specific to the topology of NNs (connection weights method). In general, an optimal NN architecture is selected and several variable contribution methods are applied. However, when applying several methods to optimal or suboptimal NN architecture, the importance ranking of variables differs from method to method (high variability), indicating their inherent instability. As a result, there is no consensus on which model is the best for determining the contribution of variables. The difficulty of choosing a single optimal NN model led some authors to consider a set of good-performing NN models called neural network committee (NNC) (see Cao et al. [2008]). To mitigate these variabilities, De Ona et al. [2014] suggested to apply the methods on a set of NNs with the same architecture and trained with identical learning algorithm, activation functions, momentum value and learning ratio. The calculation of the average values of the relative importance of variables was used to determine the contribution of variables. The variables Frequency, Speed, Punctuality and Proximity are classified as the most important by all methods.

2.3 Optimisation for machine learning

In machine learning, given the samples (x_i, y_i) , $i = 1, 2, \dots, m$, of m pairs of input vector $x_i \in \mathbb{R}^{N_i}$ and output vectors $y_i \in \mathbb{R}^{N_o}$, a task consist in learning a functional relation between the input x_i and the desired output y_i , for the training set.

The learning can be understood as finding a mapping $h(\cdot, w)$ such that $h(x_i, w) \approx y_i$ with w an N_w dimensional vector of parameters to be learned. Thus, the goal of machine learning is to minimise the expected loss (also called risk function)

$$\hat{L}(h) = E[L(h(X), Y)]$$

However, we do not know $P(X, Y)$ and can not estimate it. One solution is to consider Empirical Risk minimisation. In general, the relation between the input x_i and the desired output y_i can either be solved by a linear model as

$$\hat{y}_i = W^\top x_i$$

where $W \in \mathbb{R}^{N_i \times N_o}$ is a weight matrix, or by a nonlinear model

$$\hat{y}_i = f(W^\top x_i)$$

where $f(\cdot)$ is the activation function.

The training of neural networks (NNs) depends on the adaptation of free network parameters, the weight values. Thus, it is an optimisation task where the result is to find optimal weight set of the network that reduce an error function E . It can be formulated as the minimisation of an error function in the space of connection weights. In the case of our loss function the minimisation problem is

$$\min_W \hat{L}(h, W)$$

In practice, the function f in the above minimisation problem is not given explicitly but only implicitly through some examples. While convex problems are theoretically and algorithmically much more tractable, the complexity of non-convex problems can grow enormously (NP-hard). Global optimisation addresses the problems of non-convex optimisation.

The methods for training neural networks (NNs) are backpropagation (BP), Levenberg-Marquadt (LM), Quasi-Newton (QN). In general, the error backpropagation method (EBP), based on gradient method, is preferred. However, the objective function describing the neural networks training problem is a multi-modal function, so that the algorithms based on gradient methods can easily be stuck in local extremes. To avoid this problem one can consider using a global search optimisation (see Appendix (20)).

Over time, the relationship between mathematical programming models (MP) and machine learning models have been increasingly coupled. Methods developed for introducing constraints into the learning model in view of adding domain knowledge to enforce non-negativity and sparsity in dimensionality reduction methods. Further, methods were developed for solving existing machine learning models more efficiently. As data set size grows, MP models became inadequate and methods were developed to exploit the properties of learning problems in machine learning. In turn, machine learning has motivated advances in mathematical programming: the optimisation problems arising from large scale machine learning and data mining far exceed the size of the problem typically reported in the mathematical programming literature.

Generally, an ANN model is specified by its topology, node characteristics and the training algorithms. There are a variety of ANNs with various topologies, node properties and training algorithms. See for instance Appendix (16.6) in the case of Reservoir Computing. These characteristics (or features) must be considered when designing a neural network model, leading to large families of models. The data scientist must select an appropriate family of models

and massages the data into a format amenable to modelling. Then the model is typically trained (Training) by solving a core optimisation problem that optimises the variables (or parameters) of the model with respect to the selected loss function and possibly some regularisation function. The efficiency of an optimisation method often depends on the choosing of a number of behavioural parameters (meta parameters). In that case, the mapping becomes

$$h(x_i, w, \theta) \approx y_i$$

with θ an N_θ dimensional vector of parameters to be learned. Hence, in the process of model selection and validation (), the core optimisation problem may be solved many times. It led researchers to combine mathematical programming with machine learning. The interaction of state-of-the-art machine learning and mathematical programming has been called Large Scale Optimization and Machine Learning leading to improved machine learning models as well as new uses of mathematical programming in machine learning.

One solution consists in performing an additional optimisation aiming at minimising the RMSE on these meta parameters Θ . It can be done on the validation sample. In that case, the minimisation problem is

$$\min_{\Theta} \hat{L}(h, W^*, \Theta)$$

Pedersen et al. [2008b] proposed an overlaying optimisation method known as meta-optimisation. The idea is to have an optimisation method act as an overlaying meta-optimiser, trying to find the best performing behavioural parameters for another optimisation method (see Pedersen et al. [2008a]).

2.4 The learning process

2.4.1 Prediction tasks

The strength of neural networks (NNs) compared to other techniques is their high capacity for classification, prediction and their low failure tolerance. In financial markets, NNs are generally applied for both prediction tasks and classification tasks (see Appendix (12)). It can be summarised as follows:

- 1+ Chaotic series prediction: performing time series forecast.

In machine learning, time series forecasting is called regression problem. In the academic literature, chaotic systems are generally described by Mackey-Glass equations (see Mackey et al. [1977]). It is generated from the following non-linear time delay differential equation:

$$\frac{dx(t)}{dt} = \frac{ax(t-\tau)}{1+x^n(t-\tau)} - bx(t)$$

where a , b , n and τ are real numbers, and τ is the delay parameter. Depending on the set of parameters chosen, the Mackey-Glass signal exhibits various characteristics. For instance, Figure (3) below demonstrates a particular behaviour, which is at the same time very chaotic. Similarly to a financial time series, we observe: an identified behaviour (trending or mean-reverting) together with an unidentified one (chaos). Examples are next period price change and next period actual price (opening, high, low and closing prices), which concerns the change in price with respect to the next period and the numerical value for the actual next period price, respectively.

- 2+ Autonomous pattern generation: classification problem.

A pattern is a short sequence of randomly chosen real numbers that is repeated periodically to form an infinite time series. In general, the length of the sequence may be set from 1 up to the number of neurons N . In finance, patterns are distinctive formations of market prices (see Definition (11.1)). For example, the next period direction concerns the direction of the price movement with respect to the next period. In that case we classify each record as Up (+1) or Down (−1). The results are generally obtained by using only historic prices and

technical indicators (TIs), which are mathematical transformations of market returns (see Appendix (11)). The most common TIs are the simple moving average (SMA), exponential moving average (EMA), relative strength index (RSI), rate of change (ROC), moving average convergence / divergence (MACD), William's oscillator and average true range (ATR).

In general, academics and practitioners focus on case (1+), which implies making a statement about the true dynamics (denoted f) followed by market returns. While in case (2+) we only consider part of the probability space, this approach was originally ignored by academics. However, due to its simplicity of use and the fact that traders are concerned with the movements of the underlying, this approach has recently gained momentum. From Computational Learning, an approximation model (the hypothesis, denoted h) must be devised and trained on the full measurement space (case (1+)) or part of the space (case (2+)). In order to assess if the function h is approximately correct, a measure of error must be considered (the cost function).

This process requires sophisticated search and examination methods (optimisation algorithms) such as machine learning.

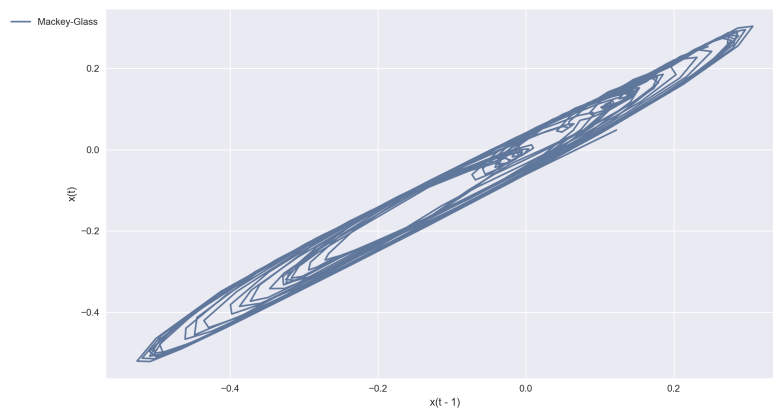


Figure 3: Mackey-Glass Behaviour for $(a, b, n, \tau, x_0) = (0.2, 0.01, 10, 17, \text{random})$

2.4.2 The data

We follow the notation in Appendix (13).

2.4.2.1 Time series forecasting We let $\mathcal{X}$ and $\mathcal{Y}$ be the input and output spaces, $\mathcal{D}$ the probability distribution, D the data set (sample), H the set of hypotheses, $I(\cdot)$ the indicator function. We let x be an instance or feature vector. For example, we consider the training data set $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$ where the instances x_i are independent and identically distributed (i.i.d.) and $y_i = f(x_i)$ where f is the ground-truth target function. That is, x_i is a vector and y_i takes value in $\mathbb{R}$.

2.4.2.2 Binary classification We consider the data set $D = \{(x_1, y_1), \dots, (x_l, y_l)\}$ for the instances x_i and $y_i = f(x_i)$, where f is the ground-truth target function. In the case of a binary classification task, we let y_i takes values in the set $\{-1, +1\}$ representing down and up movement, respectively. We let the instance space $\mathcal{X}$ be the space of market returns and we let the label space $\mathcal{Y}$ be the space of market directions. Given the instance x , we suppose that the learner h makes the prediction $\hat{f} = h(x) \in \{-1, +1\}$, which we call the signal.

2.4.2.3 The time scales The model signal $\hat{f}_k$ corresponds either to a forecast of the market price, or, to a direction forecast of the market price observed at fixed time intervals $t_k = k \times \delta$, where $k = 1, \dots, N_k$ is an integer and $\delta = \Delta t$ is a fixed fraction of time. We will suppose that the model forecast last for the period $\tau = n \times u$, where u is a fixed unit of time (say in second) and $n = 1, 2, \dots$ is an integer.

2.4.2.4 Measuring returns We are interested in the forecast for the learner h for a fixed period d . Thus, to validate the forecast, we are interested in the k th log-return $r_{L,d}(k) = r_L(k, k+d)$ for the d period defined in Equation (9.12). We have two time scales, the one given by the model signals and the market prices observed at tick time levels.

2.4.3 Features representation

We let $x_i(t)$, $i = 1, \dots, n$, be the i th feature at time t and $X_t = (x_1(t), \dots, x_n(t))$ be the vector of all features at time t . Given the training data set $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$, the feature matrix is denoted by

$$M = (X_1, \dots, X_m)^\top$$

Following notation in Appendix (9.1), the difference between two consecutive features from date t to date $(t+1)$ is $x_i(t+1) - x_i(t)$. More generally,

$$\nabla_d x_i(t) = x_i(t) - x_i(t-d)$$

with period d (temporal correlation). In vector notation, we get

$$\nabla_d X_t = (\nabla_d x_1(t), \dots, \nabla_d x_n(t))^\top$$

and

$$\nabla_d M = (\nabla_d X_1, \dots, \nabla_d X_m)$$

The difference in features are normalised by setting $N(\nabla_d x_i(t)) = \frac{x_i(t) - x_i(t-d)}{x_i(t-d)}$. The problem is defined as predicting the sign of difference between two features.

2.4.4 Training and exploitation of the model

When forecasting time series or, generating autonomous patterns, we consider the following three steps:

1. Training: it consists in determining the model parameters that minimise the cost function.
2. Optimisation, and
3. Adaptability.

We split the available data into 3 samples: Training, Validation and Testing. It is called sample hashing (see an example in Figure (4)). Optimisation is performed in the validation set. Testing is the sample where the calibrated model is assessed for generalisation. That is, the model is ran without any exhaustive filter nor complementary optimisation. The learning process is as follows:

- In the case of pattern generation: During the training phase, the reservoir computer receives the pattern signal as input and is trained to predict the next value of the pattern from the current one. The length of the training input sequence is measured in terms of the pattern length, that is, how many times the whole pattern is presented to the reservoir computer. The duration of the training process depends on the length of the pattern. For instance, short patterns are successfully learnt after 100 repetitions, while long patterns need to be shown up to $50k$ times. The RMSE is used to evaluate the training phase. After the training phase, the reservoir input is switched from the training sequence to the reservoir output signal, and the system is left running autonomously.

- In the case of series prediction, training and test phases are the same as for pattern generation. During training, the reservoir receives the time series and is trained to perform a one-step ahead prediction, while in the test phase the reservoir receives its own output and the system runs autonomously.

Based on the classification task chosen, several possibilities exist for training and exploiting models:

- 1* training on transient process → exploitation on transient process
- 2* training on attractor-based model → exploitation on attractor-based
- 3* training on attractor-based model → exploitation on transient process
- 4* training on attractor-based model → exploitation on transient process

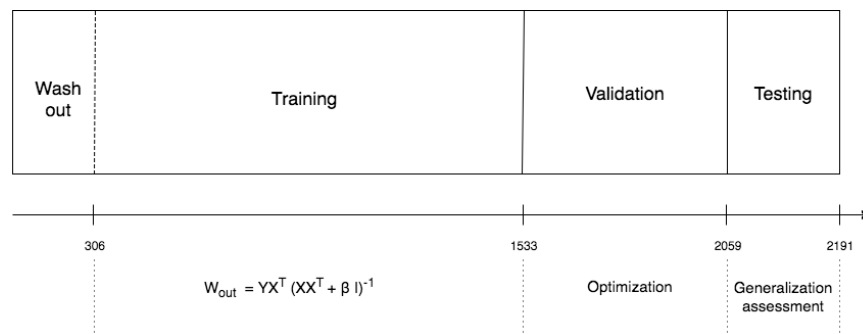


Figure 4: Sample hashing: Training sample is *washed – out* at 20% meaning that the Ridge regression is only performed on the 80% remaining. By doing this, we get rid of the reservoir initial transient.

2.4.5 The framework

2.4.5.1 The input variables When selecting the right input variables for regression problems, Krollner et al. [2010] showed that academics and practitioners rely on some form on lagged index data (case (1+) in Section (2.4)) rather than considering autonomous pattern generation (case (2+)). Examples of articles that focus on forecasting market returns are Zeng et al. [2006], Zhang et al. [2007], Zhu et al. [2008], Lu et al. [2009], Liao et al. [2010]. Based on some well known techniques combining forecasts (see Armstrong [2001]), several studies showed that combining forecasts (using NN models) could greatly enhance the forecast effectiveness (see Wang et al. [2008], Chitra et al. [2010]).

When it comes to classifying financial time series, practitioners considered the usage of features that are not inherent to the time series under study. It includes any added information beyond the values of the given time series. For instance, traders and academics have used technical analysis (see Appendix (11.2)) as well as certain fundamental valuation metrics to improve results (see Gencay [1997], Metghalchi et al. [2007]). Further, some statisticians showed that technical indicators have predictive power and found a good measure of support for their effectiveness (see Chen et al. [2009], Metghalchi et al. [2012] [2015]). As a result, these strategies have been incorporated in machine learning models to forecast future trend direction (case (2+)). The results obtained are achieved using only historic prices and technical indicators (see Kim et al. [2006], Bekiros et al. [2008], Kara et al. [2011], Dingli et al. [2017]). Some authors also used technical indicators to forecast both future price levels and asset price directions (see Patel et al. [2015]). Note, one can directly use continuous technical indicators (actual time series) or one can choose to represent them as trend deterministic data (discrete in nature). Van den Poel et al. [2016] combined in some way continuous technical indicators (see Appendix (11.2)), technical trading rules (see Appendix (11.5.1)) and past returns to become input to predictor models. They proposed a predictive model to predict future stock price direction accurately.

2.4.5.2 The forecasting horizon The different forecasting outputs are computed for the following periods: daily, weekly, monthly, quarterly and yearly. However, an increasing number of articles consider intra-day high frequency data, which requires highly liquid markets. In that case the data comprises the date and time, open-high-low-close price levels and the volume traded for each underlying asset. One of the main reason for considering intraday trading horizon is the notion of self-destruction of predictable patterns in stock prices (see Timmermann et al. [2004]). For the data to be consistent, historical prices and/or volume are usually adjusted to reflect spin-offs, stock splits/consolidations, stock dividend/bonus and right offerings/entitlement.

2.4.5.3 Alignment In general, we use one market (such as NASDAQ) as a basis for data alignment and missing data in other data sources is replaced by linear interpolation.

2.4.5.4 Evaluation methods Most evaluation methods of the ML models are benchmark models where the proposed improved version is compared against the original one using a forecast error as an evaluation metric. Examples of benchmark models are buy and hold, random walk, statistical techniques and other machine learning techniques. We consider a simulated market with trading period T and initial capital C . In a buy and hold strategy, we suppose that we use all the money to buy stocks on the first day and sell the stocks at the end. To do so, we force all possessed stocks to be cashed at the end of the trading period. The profit is depend on the trend during this testing period,

$$PnL = C \frac{(V_T - V_0)}{V_0}$$

where V_t is the value of the portfolio at time t .

We can also assume a trending strategy where we observe the value of the underlying at time t , denoted $x(t)$. If $x(t) > x(t-1)$, then we assume that the value of the underlying at time $t+1$ will be higher than the one at time t . Whenever the prediction is rising, we buy at most Δ shares of stocks. Otherwise, we sell all stocks we have. This model performs well in a trending market, but losses a lot in a mean-reverting market.

2.4.5.5 The measures The measures used when forecasting financial time series with machine learning models can roughly be divided into three main groups: Error-based, Return-based, and Accuracy-based. Here, error is used when forecasting market prices or returns, while accuracy is used in a directional sense. That is, correctness corresponds to the forecasted variable actually moving in the same direction as the forecast. The most popular measures are accuracy, annual return, and root mean squared error. Note, accuracy does not convey any information on profitability, and simulations must be performed. Thus, we should favour performance metrics demonstrating the return capabilities of an algorithm with respect to its field of forecasting or classification.

Linear measures are measurements in one direction. For example length, width, or height of objects, and distance between two points are linear measures. Linearity also express the idea that the model possesses the property of additivity and homogeneity, meaning that a change in one variable causes a proportional change in another variable. An example of a linear measure is the root-mean-square deviation (RMSD), or, root-mean-square error (RMSE), which is used to measure the differences between values predicted by a model or an estimator and the values observed.

In mathematics and science, a nonlinear system is a system in which the change of the output is not proportional to the change of the input. Systems can be defined as nonlinear, regardless of whether known linear functions appear in the equations. Nonlinearity measures are a means of quantifying the size of nonlinearity in the Input/Output-behaviour of nonlinear systems (more than one dimension). Methods that efficiently compute nonlinearity measures are based on convex optimisation.

Note, linear measures do not account for the non-linearity of RNN models (see Bellgard et al. [1999]). One way around is to assess the performance of a model using trading simulation.

2.4.5.6 Analysing models Meta-analysis is the statistical analysis of a large collection of results from individual studies for the purpose of integrating the findings. In finance, the lack of standardised testing metrics and standard testing datasets makes it difficult to form a meaningful metaregression analysis. One solution is to evaluate algorithm

classes based on their relative rank in subgroup experiments. Ryll et al. [2019] proposed an average-over-all approach, where for a given algorithm type, a single rank score between 0 and 1 is computed. They argued that pairwise rank analysis based on the same scoring system uncovers meaningful performance differences between algorithm classes.

2.5 Using neural networks to model returns

2.5.1 An overview

Among the various techniques for classifying and forecasting financial time series, fundamental and technical analysis are the most popular ones (see Hu et al. [2015]) (see Appendix (11)). Even though statistical procedures are widely used for patterns recognition, the effectiveness of these methods relies both on model's assumptions and prior knowledge on data properties. To remedy these pitfalls, several classifiers developed, using various data mining and computational intelligence methods such as rule induction, fuzzy rule induction, decision trees, neural networks etc. In 1990, Kimoto et al. [1990] applied a modular neural network machine learning algorithm to predict the movement of stock index of Tokyo Stock Exchange. Schoneburg [1990] used neural networks to predict stock prices. BaBa et al. [1992] used neural networks to build an intelligent forecasting system of stock prices. Cheng [1994] used neural network for forecasting the price-volume relationship in the Taiwan stock market. Tsibouris et al. [1995] tested the efficient markets hypothesis with gradient descent algorithms. Since ANN developed further, it has been widely applied to stock analysis. Kuo [1998] considered fuzzy neural networks for building a decision support system for predicting the stock market. Nowadays, the best recognised tools in the currency markets is the artificial neural networks (ANNs), supported by numerous empirical studies (see Ahmed et al. [2010]). Similarly, Krollner et al. [2010] provided a survey on forecasting financial time series with machine learning and found that ANNs were the dominant technique in that field. The foremost reason for using ANNs is that there is some nonlinear aspect to the forecasting problem under consideration, taking the form of a complex nonlinear relationship between the independent and dependent variables. The characteristics of financial time series, such as equity stock or currency markets, are influenced by the psychology of traders (behavioural finance) and are strongly non-linear and hardly predictable (see Maknickiene et al. [2011]).

One advantage in using an ANNs is that the researcher does not need to know a priori the type of functional relationship existing between the independent and dependent variables (see Darbellay et al. [2000]). A vast literature demonstrated that neural networks perform better than conventional statistic approaches in financial forecasting (see Refenes et al. [1994], Adya et al. [1998], Abu-Mostafa et al. [2001]). For many financial forecasting problems, classification models work better than point prediction. Further, Qi [2001] argued that due to the continually changing nature of financial relationships, ANNs are more likely to outperform traditional techniques when the input data is kept as current as possible. This is done by recursive modeling, where the researcher adds new observations and drops the oldest ones each time a new time series forecast is made (sliding window). Olson et al. [2003] compared neural network forecasts of one-year ahead Canadian stock returns with the forecasts obtained by using ordinary least squares (OLS) and logistic regression techniques and showed that the backpropagation algorithm outperformed the best regression alternatives for both point estimation and in classifying firms expected to have either high or low returns. This superiority of the NNs translated into greater profitability using various trading rules. Using data from four major stock market indexes, Fok et al. [2008] compared linear regression and neural network backpropagation by testing their forecasting performances. They showed that the latter had better prediction accuracy. Ballings et al. [2015] compared ensemble methods such as Random Forest, AdaBoost, Kernel Factory, against single classifier models such as NNs, logistic regression, SVM, and K-nearest neighbour using data from 5767 European companies. The performance measures were a five time two-fold cross-validation and area under the curve (AUC) to predict long term stock price direction and reported the Random Forest as the best algorithm.

Lin et al. [2009] used Echo State Network to forecast the next closing price. Huang et al. [2009] combined support vector regression (SVR) with a self-organising feature map (SOFM) and feature selection based on filtering for predicting the next day price of the Taiwan index futures. Lee [2009] developed a prediction model on the basis

of an SVM with a hybrid feature selection method to find the best input features. The F-score was used as a selection criterion. Cogo [2015] considered several ANN-based variable selection models for predicting one-day ahead price of the S&P 500 index. The architecture is a three layer network with ten neurons in the hidden layer. Three algorithms are tested for training the networks on one hundred days. The best results are obtained with the Levenberg-Marquardt algorithm where the predicted price mimic fairly well the patterns of the actual one for the first fifty days. The ANN trained with gradient descent training algorithm exhibited poor performances. The author also investigated the impact of the number of neurons in the hidden layer in the prediction process. The sizes tested were 20, 50, and 150 neurons and the author concluded that with one hidden layer the increased number of neurons resulted in lower predictive power. Liu et al. [2016] investigated logistic regression, Gaussian discriminant analysis (GDA), Naive Bayes, and SVM to forecast the movement of the S&P 500 index. After optimising the features selection for different models, the SVM model with a Radial Basis Function (RBF) kernel obtained a rate of accuracy of 62.5% for future market trend. Zhong et al. [2017a] used deep ANN to forecast daily stock market return using dimensionality reduction. Bao et al. [2017] considered deep nonlinear NN topologies to predict financial time series. Di Persio et al. [2017] compared a basic multi-layer RNN, LSTM, and GRU on forecasting Google stock price movements (difference in price) on different time horizons: the next day, the next 5 days, the 10 days. They found LSTM to be the better algorithm. The accuracy for next day is about 66%, and for 5 days horizon it is about 72%. Yang et al. [2017] proposed an ensemble of multi-layer feedforward networks to predict the Chinese stock. Three networks are trained with backpropagation and combined with the bagging approach (see Appendix (13.2)). Zhang et al. [2018] proposed a stock price trend prediction system to predict both the price movement and its interval of growth (or decline) rate within predefined prediction intervals. A Random Forest was trained to classify clips of stocks into four classes (up, down, flat, and unknown) according to the shapes of their close prices. They outperformed existing prediction methods in terms of accuracy and return per trade. Abbas [2018] used a deep learning model (with four hidden layers) to predict stock prices (returns) for major stock indices. He considered short term prediction with a frequency of the minute. The data consisted of over 40,000 minutes of S&P 500 index. The idea was to find emerging trends in minute. The training was done by batches and a fully conducted batch is an epoch. After ten epochs, the MSE on the training set was 0.038.

2.5.2 Combining neural networks with evolutionary algorithms

Some of the disadvantages of NNs are long training time, unwanted convergence to local instead of global optimal solution, and large number of parameters (see Lee et al. [1991]). One way forward is to combine ANN with another algorithm such as evolutionary computing tools that can take care of a specific problem (see Appendix (20)). Many research papers have appeared in the literature using evolutionary computing tools, such as genetic algorithm (GA) (see Montana et al. [1989], Nair et al. [2011], Contreras et al. [2016]), particle swarm optimization (PSO) (see Kennedy et al. [1995] Jordehi et al. [2013]), bacterial foraging optimization (BFO) (see Jhankal et al. [2011]), and Adaptive bacterial foraging optimization (ABFO) in developing forecasting models.

Inspired by the pattern exhibited by bacterial foraging behaviour, Helstrom et al. [1998] proposed an evolutionary computation technique called Bacterial foraging optimization (BFO). It analyses how the run-length unit parameter of BFO controls the exploration of the whole search space and the exploitation of the promising areas. Shin et al. [1998] proposed data mining approach using genetic algorithms (GA) to solve the knowledge acquisition problems that are inherent in constructing and maintaining rule-based applications for stock market. Kim et al. [2000] used GA to improve the learning algorithm, and also to reduce the complexity of the feature space. Kim et al. [2004b] developed a feature transformation method using genetic algorithms. It reduces the dimensionality of the feature space and removes irrelevant factors involved in stock price prediction. Jamous et al. [2016] considered particle swarm optimization with center mass (PSOCOM) technique to develop an efficient forecasting model. Rani et al. [2017] presented an overview on optimisation techniques and hybridisation to improve ANN performance (see details in Appendix (20)).

2.5.3 Study

Bahrammirzaee [2010] presented a study of applications of computational intelligence to financial data and concluded that artificial intelligence algorithms generally possess a higher accuracy than comparable statistical methods. Similarly, Cavalcante et al. [2016] presented a study of applications of computational intelligence to financial data in studies from 2009-2015. Makridakis et al. [2018] evaluated the forecasting performance of ML against statistical models across multiple forecasting horizons using a large subset of 1045 monthly time series used in the M3 Competition. Statistical models were found to be better across both accuracy measures used and for all forecasting horizons examined. Shah et al. [2019] presented a review of methods for predicting stock prices. Ryll et al. [2019] studied over 150 related articles on applying machine learning to financial market forecasting. They classified the different algorithms and conducted rank analyses to assess the comparative performance of the different algorithm classes. They found significant evidence of the outperformance of machine learning against traditional stochastic models. Further, on average, recurrent neural networks outperform feed forward neural networks as well as support vector machines, implying the existence of exploitable temporal dependencies in financial time series across multiple asset classes and geographies.

2.5.4 Generative networks

Zhou et al. [2018] proposed a GAN model (see Appendix (19)) using long short-term memory (LSTM) and a convolutional neural network (CNN) for adversarial training to forecast high-frequency stock market. The method of rolling partition training and testing set is used to analyse the effect of the model update cycle on the prediction performance. Basic technical index data was used as input variable. They improved the prediction accuracy of the price direction and reduced forecast error between real prices and forecast ones.

Zhang et al. [2019] proposed a GAN model with the multi-layer perceptron (MLP) as the discriminator D and the long short-term memory (LSTM) as the generator G for forecasting the closing price of stocks. The latter mine the data distributions of stocks from given data in the stock market and generate data with the same distributions, while the former aims to discriminate the real stock data and generated data. The daily S&P 500 index was tested to predict the next day closing price. Twenty years of daily data together with 7 financial factors (high, low, open, close price, volume, turnover rate and Ma5) were used to predict the future closing price. These factors (technical analysis) occurred to be very valuable and significant in the forecasting process. Several statistical indicators of performance such as MAE, RMSE, MAPE were chosen, and the results were found to be competitive compared with other methods. In fact they were very comparable to those obtained with LSTM, slightly lower.

Koshiyama et al. [2019] used a conditional GAN (cGAN) model for trading strategies calibration (or fine-tuning) and aggregation (portfolio allocation). The data set was made of 579 assets (mainly equity). The model was tested against an ensemble scheme and model validation methods, and the cGAN model was found to outperform the baseline models.

2.5.5 Towards reservoir computing

Miller [2018] argued that DNN models, such as autoregressive Wavenet and Transformer, are replacing RNNs on a diverse set of tasks. This is based on several studies (see Bai et al. [2018]) showing that feedforward networks could match the results of the best recurrent models on benchmark sequence tasks such as language modelling, machine translation, music generation. Miller et al. [2019] proved that stable recurrent neural networks (meaning the gradients can not explode) were well approximated by feedforward networks for the purpose of both inference and training by gradient descent. However, feedforward networks make a strong conditional independence assumption. This is not the case of market returns which are multifractal. Further, the origin of this discussion derived from the vanishing and the exploding gradient problems when training recurrent neural networks (see Appendix (16.4.3)). The problem disappear when considering Reservoir Computing (RC). Thus, it would make sense to use RC when forecasting market returns.

2.5.6 Reinforcement learning for predicting patterns

Since financial patterns are distinctive formations of market prices, forecasting them is a multi-step prediction-learning problem (see Appendix (16.1)). However, as discussed above, research focussed on predicting market prices (regression problem) and their directions (classification problem) as if it was a single-step prediction-learning problem by using supervised learning methods. Sutton [1988] showed that temporal difference (TD) methods were more efficient than supervised learning methods when the data sequences were generated by a dynamical system. In a multi-step problem, we are concerned with experience coming in observation-outcome sequences of the form $x_1, x_2, \dots, x_m, z$, where each x_t is a vector of observable input at time t in the sequence, and $z \in \mathbb{R}$ is the outcome of that sequence. In supervised learning, each sequence of observations and its outcome is represented by a sequence of observation-outcome pair of the form (x_t, z) at time t . As such, we can not capture the transition probabilities of the dynamical system driving the successive relative changes in market prices. TD algorithms are at the heart of reinforcement learning, solving the prediction problem, but they do not provide an optimal policy. It uses the evaluation step in the learning process, and not the improvement step.

3 Forecasting returns

It is now well known that market returns are multifractal, exhibiting trends that repeat over time, and some, following sinusoidal shape (see Lo et al. [2002]). As a result, future prices and trends can be predicted based on past prices. Thus, price prediction is closely related to time series analysis and trend modelling. Some examples include stochastic process theory, spectral analysis, wavelet analysis, among others. Some popular stochastic models in finance are the autoregressive integrated moving average (ARIMA) and the generalised autoregressive conditional heteroscedasticity (GARCH) (see Box et al. [1976]). With the increasing volume of data stored, soft computing methods developed, such as fuzzy logic and neural networks (NNs). Gately [1995] introduced ANN to the financial industry. Logically, hybrid models between stochastic models and networks developed. Zhang [2003] proposed to combine ARIMA and ANN to model time series of market prices. The results outperformed both models separately. This model was improved by Khashei et al. [2010]. New and different hybrid models developed. For instance, Guresen et al. [2011] combined a GARCH and EGARCH model with different ANN structures. Adding to purely statistical analysis, some authors incorporated risk analysis by considering technical analysis (TA) (see Appendix (11)). For instance, Witkowska et al. [2005] combined TA with ANN models to forecast the WIG20 index on the Warsaw Stock Exchange. One can also combine TA and fundamental analyses with ANN for forecasting stock prices (see Faustryjak et al. [2018]). Paluch et al. [2014] proposed to combine technical indicators with fractal indicators. See Appendix (11.4) for a description of fractal indicators. Then, Paluch et al. [2015] [2016] [2018a] considered hybrid ANN-based models with technical and fractal analysis for forecasting closing prices on the Warsaw stock exchange (WSE). The system analysed prices trends and chose assets that will achieve the highest expected profit the next day. The system was tested offline on historical data from 70 companies over a period of six months. The prediction was correct 60% of the time, and a profit of 17.2% was generated, while the WIG20 index made -0.99% and the mWIG40 made 5.69% . Paluch et al. [2018b] proposed a hybrid model combining technical and fractal analysis with ANN to predict close prices in the WSE. The system comprised new fractal analysis indicators.

We are briefly going to present the hybrid method proposed by Paluch et al. [2018b] as an example of an ANN model to forecast closing prices. This is because ANNs are not designed to forecast serially correlated time series. As discussed in Section (2.5), recurrent neural networks outperform traditional artificial neural networks as well as support vector machines, implying the existence of exploitable temporal dependencies in financial time series across multiple asset classes and geographies. Hence, we are going to focus on RNN to forecast market returns. In order to gain some insight on the capability of neural networks to reproduce the dynamics of financial time series, we need to test some of the properties of recurrent neural networks introduced in Section (2.5) as well as some of their characteristic for modelling changing environment presented in Section (2.2.2). Following Section (2.4), we will test neural networks on the Mackey-Glass signal as well as several real-time financial time series. Among the different ML models presented in Section (2.2), we are going to concentrate on random projection networks

(RPN) with non-linear spatio-temporal projections generating transient and attractor dynamics (see Wojcik [2018], Andras [2018]). We will focus on the Associative Reservoir Computing (ARC) (see Reinhart et al. [2008] [2009b]) as it is stable and fast, and, unifies the RPN models. The detailed implementation of the model is given in Appendix (17.1). The model will be tested both with offline and online algorithms. We will also test a method for teaching the reservoir how to remain consistently stable while feeding it with noisy input.

3.1 Neural models for price forecasting

Paluch et al. [2018b] proposed a hybrid model combining technical analysis (TA) and fractal analysis (FA) (see Appendix (11)) with ANN to forecast the closing price of an asset for the next day. The structure of the model is displayed in Figure (5). In the first module (in blue), the TA and FA indicators are first computed. They are then used as input in the ANN, which output the closing price for the next day (time $t = k + 1$). Feed-forward ANNs of the MLP type were considered. Three kind of hybrid models were tested: TA with ANN, FA with ANN, and TA plus FA combined with ANN.

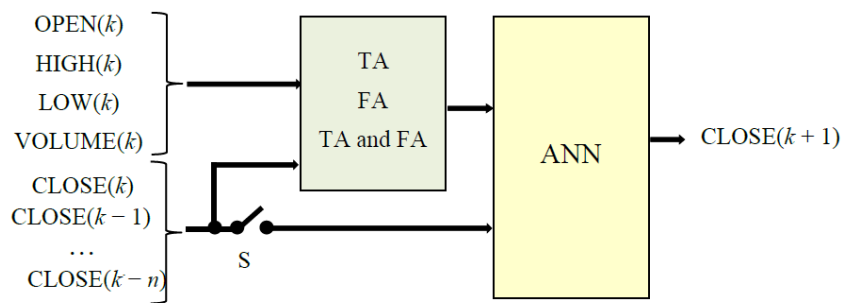


Figure 5: Structure of hybrid model

As discussed in Section (4), technical indicators (TI) give trend indications and are used for forecasting the directions of the underlying asset, and not the next day closing prices. Hence, the authors introduced a moving time window method based on current and past close market prices, which was applied to all three hybrid models. These prices are entered as input to the ANN. It corresponds to the closed switch S in Figure (5).

Different two-layer MLP structures were tested with different number of inputs and neurons in the hidden layer. A combination of input data was tested. The MSE was used as criterion for selection of the best model. The benchmark model was a purely ANN-based model with the moving time window method (past close prices). Data consisted of historical data for five companies listed on the Warsaw Stock Exchange. The structures with closed switch S (past close prices) produced the best results. Hybrid models performed better than the benchmark. The FA with ANN and TA plus FA with ANN were marginally better than TA with ANN, and generated very similar results.

When forecasting market returns it is more judicious to consider a recurrent neural network rather than a neural network. The latter will be discussed when forecasting the directions of the underlying asset.

3.2 The associative reservoir computing

Using a particular implementation of the ARC (see Appendix (17.1)), we reproduce below some results on the Mackey-Glass signal and FX time series obtained by a student of ours who did his MSc thesis at QFL (see Bouizi [2017]).

3.2.1 Using Offline algorithms

3.2.1.1 The Mackey-Glass series In the academic literature, it is found that pattern generation can be easily implemented and produces very good results. However, the Mackey-Glass series prediction task is more complex and

requires a large reservoir and more elaborate training algorithm (see Wyffels et al. [2009], Antonik et al. [2016]). We display the forecasting of the Mackey-Glass time series in Figure (6). Testing and parameters optimisation through the GEN optimisation process.

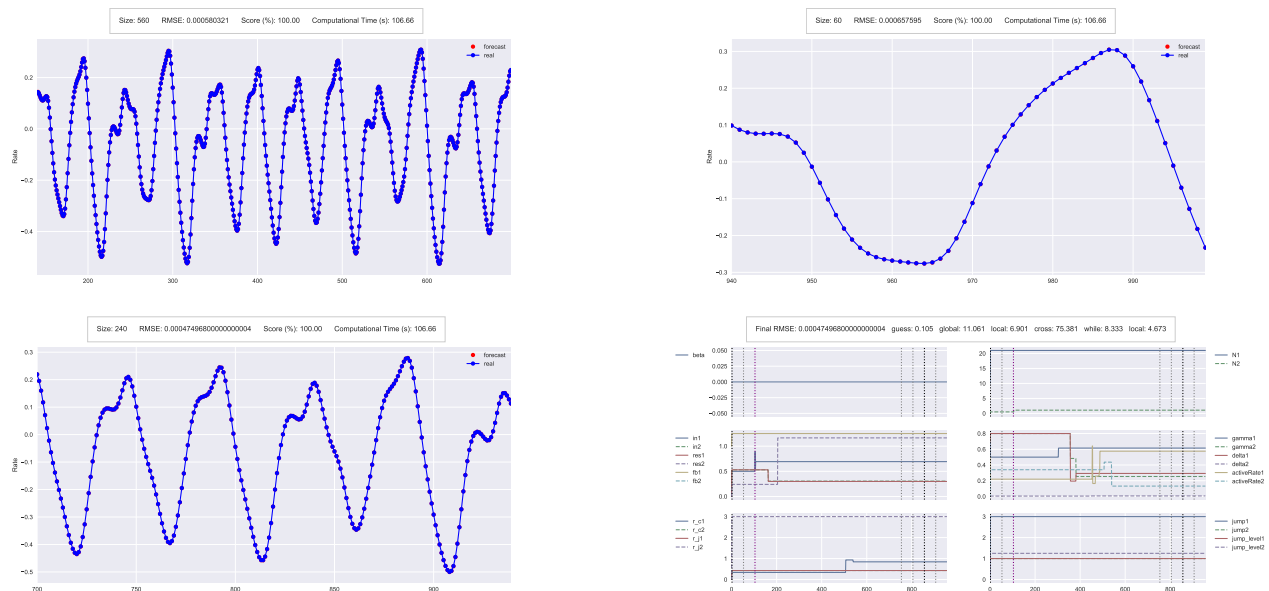


Figure 6: Mackey-Glass time series forecasting. On the left hand side from top to bottom: Training and Validation samples. On the right hand side from top to bottom: Testing and parameters optimisation through the GEN optimisation process. A black dotted vertical line indicates the beginning of a global or *while* loop optimisation. A gray one corresponds to a local one and a purple one indicates a cross optimisation.

Method	RMSE	Ranking
Neural tree (see Chen et al. [2004])	0.0069	5
PNN (see Tan [1995])	0.0059	4
NN + Wavelet Denoising (see Tan [2009])	0.0028	3
NN + Wavelet Packet Denoising (see Tan [2009])	0.0024	2
ESN offline	<u>0.00065</u>	1

Table 1: Comparison of our results with other research papers on the testing sample.

From Table (1) above, we can conclude that our method is very accurate at forecasting the Mackey-Glass time series and clearly outperforms classical methods. It is important to note that GEN optimisation method (see Appendix (17.1.3)) has reduced the RMSE from 0.00545518 to 0.000474968 on the validation sample. One of the reason for getting better results is that we optimise a large number of parameters in a proper manner without overfitting the data and in a relatively short period of time (approximately 00:01:36).

Remark 3.1 *It is not surprising to get good accuracy as this signal exhibits clear information without any noise. In this experiment, we only assess the ability of our neural network to incorporate its behaviour and to reproduce it step by step as redundant information comes in.*

3.2.1.2 Financial time series Following the same approach on financial time series, we reproduce some results on the FX markets. We perform a GEN optimisation (see Appendix (17.1.3)) with offline training using the following parameters: $(globalMax, crossMax, localMax, whileMax) = (20, 20, 20, 0)$. Considering Remark (17.2), we choose the following setting $whileMax = 0$. We study the specific FX pair: EURGBP from 2010-01-01 to 2017-05-01. From Figure (7) below, we observe good results on the training and validation samples with a relatively low RMSE and a good score (almost 60%). However, Figure (7) suggests that testing is not satisfying, with a high RMSE at 0.0440 and a score below 50%.

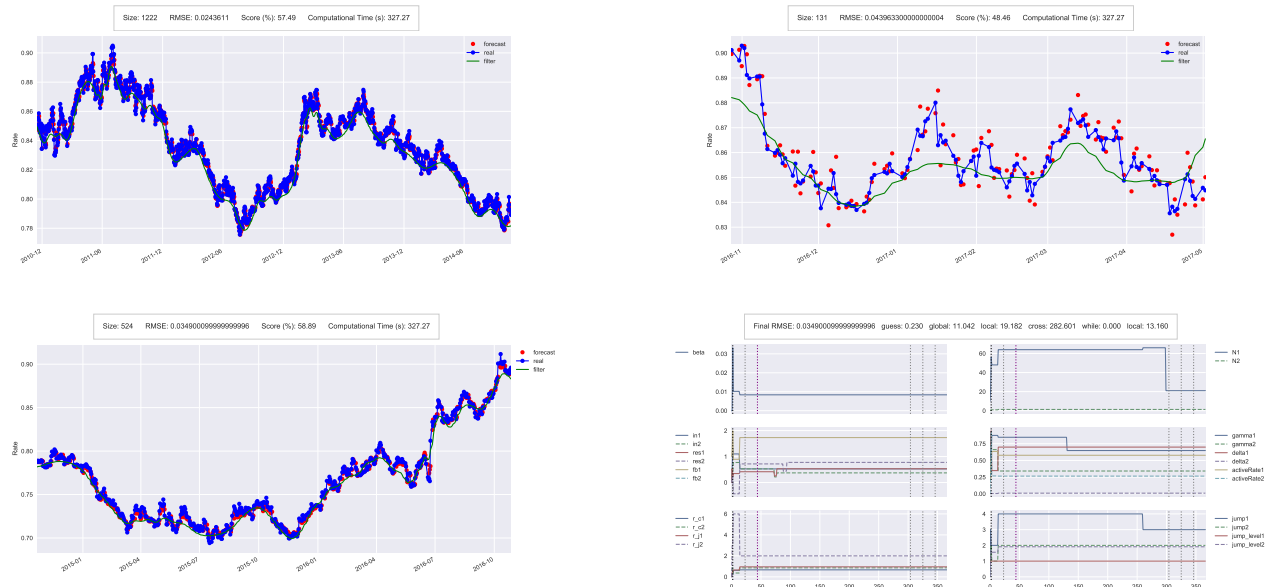


Figure 7: EURGBP time series forecasting. On the left hand side from top to bottom: Training and Validation samples. The filter (green curve) is used as input on these samples. On the right hand side from top to bottom: Testing and parameters optimisation through the GEN optimisation process. It is important to precise that the filter (green curve) is not used during testing and therefore is purely informative.

So far, we have worked with a single FX pair. In order to perform a broader study, we consider a pool of instruments from the same market and focus on two cases: one with filtered data and one with non-filtered data. We have gathered our results and computed the mean of the RMSE and Score for each sample.

	Filtered Input			Non-Filtered Input		
	Training	Validation	Testing	Training	Validation	Testing
RMSE	0.0199	0.0282	0.0290	0.0241	0.0336	0.0237
Score	0.6207	0.6692	0.5045	0.4860	0.5196	0.5126

Table 2: Comparison between filtered input and non-filtered input results on 71 FX pairs.

From Table (2), one can clearly observe that we get better learning when using filtered input data. However, we can not use a denoising technique when testing the data as it would result in data-snooping bias. Thus, we are left with using market data, which results in poor performance (slightly above 50%). Intuitively, we have trained a reservoir with filtered data. As long as the new input data is relatively smooth, the reservoir can produce good predictions.

However, in presence of noise the reservoir state is too much disrupted and gets too far away from a stable state. When using non-filtered data, the table shows that there is no learning. It corresponds to case (3) above where the network is trained with an attractor-based model but exploitation is ran on a transient process.

3.2.2 Using online algorithms

3.2.2.1 The Mackey-Glass series In the case of the Mackey-Glass time series discussed in Section (3.2.1), we have shown that the offline algorithm was very good at calibrating and predicting this signal. However, the performance was downgraded when adding a noisy component to it (equivalent to financial time series).

Both academics and practitioners proposed to smooth financial data with filtering methods. For instance, wavelets were considered as they can represent any signal in a smooth manner and, as such, provide a certain stability (see Appendix (10.2)). Further, it was suggested that they give information about the past and future at any time t in the training and validation samples, teaching the model how to behave. A good overview of the application of wavelets in economics and finance is given by Ramsey [1999].

However, as explained by Bloch [2014], even though the discrete wavelet transform (DWT) developed by Mallat [1989] is ideal for data compression, it can not simply relate information at a given time point at the different scales. Further, it is not possible to have shift invariance (we cannot use information from the future). One can get around this problem by means of a redundant, or, non-decimated wavelet transform, such as the a trous algorithm.

Still, for simplicity of exposition, we consider the following framework:

$$\forall t \in \mathbb{T} \quad f(t) = f^*(t) + \epsilon(t) \quad (3.1)$$

where ϵ is a Gaussian white noise with an appropriate constant width. We also suppose that there exists a wavelet $\hat{f}^*$ capable of approximating f^* . We want to show that learning with an online algorithm produces better approximation results than with an offline algorithm. We consider the Mackey-Glass signal presented in Section (2.4.1). In the training and validation samples, f^* is used as target y in the Algorithm (21). Again, we cannot use it in the testing sample as it would result in data-snooping bias. Therefore, the green signal is only informative. The signal which is used in the testing sample by Algorithm (21) is the one coloured in magenta in Figure (10). It is the result of Algorithm (1) applied at every time t in the testing sample (from the beginning of validation).

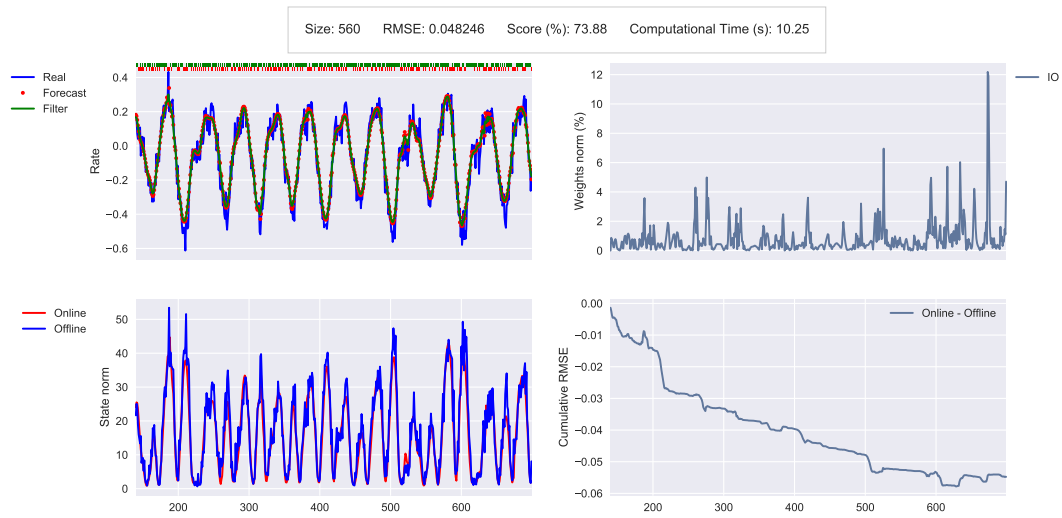


Figure 8: Noised Mackey-Glass online forecasting on the training sample. Top Left: Original, target filtered signals and forecasts. We have also plotted the local accuracy forecast (+1 if real slope is well forecast, -1 if not). Bottom Left: Reservoir states L2-norm from offline and online algorithms. Top Right: Normalised output weights evolution $\frac{\|W_{out}(t) - W_{out}(t-1)\|}{W_{out}(0)}$, where $W_{out}(0)$ is the offline matrix calibrated on the training sample. Bottom Right: error $cumulRMSE_{on} - cumulRMSE_{off}$.

From Figure (8) above, one can see that the denoised signal using Algorithm (1) is very smooth and has the same shape as the original Mack-Glass time series. Further, the learning Algorithm (21) leads to very good prediction of the green signal. This good learning is guaranteed by the combination of smooth states (plot on the bottom left, better visualisation on Figure (10)) and the adaptive filtering technique refining the output weights at each point in time (plot on the top right). This technique results in a very good approximation of the filtered signal and clearly outperforms the offline algorithm. Overall, we get a very good RMSE at 0.048246 and score at 73.88%. Note that we have used only a few iteration in this simulation: $(globalMax, crossMax, localMax) = (1, 1, 1)$. It indicates that a lot of reservoirs can be appropriate for our task, adding more flexibility. Also it is very computationally efficient as a good learning can be produced in less than 11 sec.



Figure 9: Noised Mackey-Glass online forecasting on the validation sample. See Figure (8) for an advanced description.

From Figure (9) above, we see that the good prediction performance is preserved on the validation sample and that the algorithm produces roughly the same results than the one obtained on the training sample.

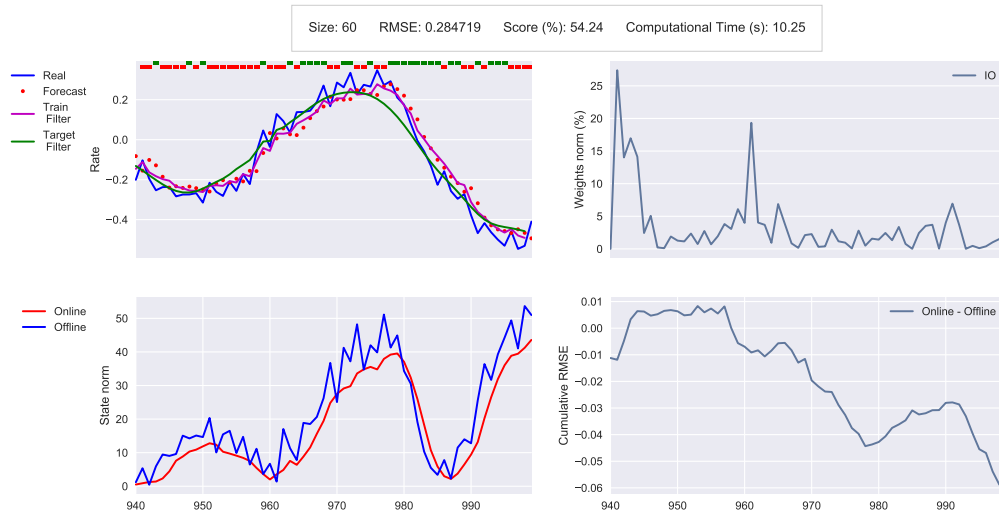


Figure 10: Noised Mackey-Glass online forecasting on the testing sample. See Figure (8) for an advanced description. We have added the filtered used by the algorithm in magenta. The green one is informative only.

Figure (10) shows that the magenta signal produces good results as the states remain stable. However, the results of the online algorithm against the offline one are not as good. Further, the output weights are still changing (from 5% to 25%), indicating that the online algorithm cannot forecast constant parameters in time. This is undesirable as we would like our reservoir to have good predictive properties when only feeding it with the new filtered input.

3.2.2.2 Financial time series We now assess the performance of the online algorithm on market data. Again, we use our pool of 71 FX pairs from 2010-01-01 to 2017-05-01.

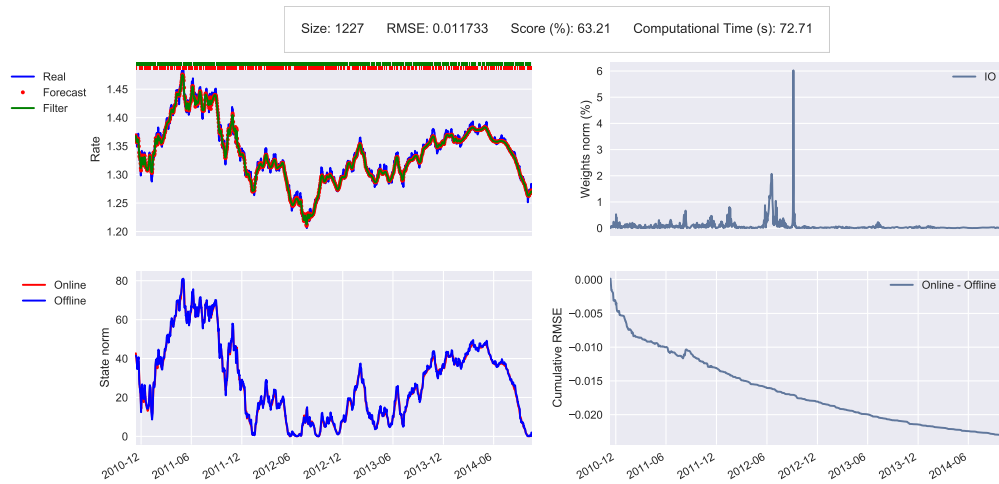


Figure 11: EURUSD online forecasting on the training sample.

Figure (11) above demonstrates good learning and good performance of the online algorithm compared to the offline one. However the output matrix is sometimes unstable. For instance, a peak of nearly 6% has occurred between 2012-06 and 2012-12, which is well above the average amplitude of the relative output matrix variation. It often happens during regime shift. Further, when we look at EURGBP on the 23rd of June, 2016 (Brexit), we see that the adaptive filtering has produced a peak of 4%. Overall, there is good learning with a RMSE at 0.011733 and a Score at 63.21%. The algorithm ran in approximately one minute.

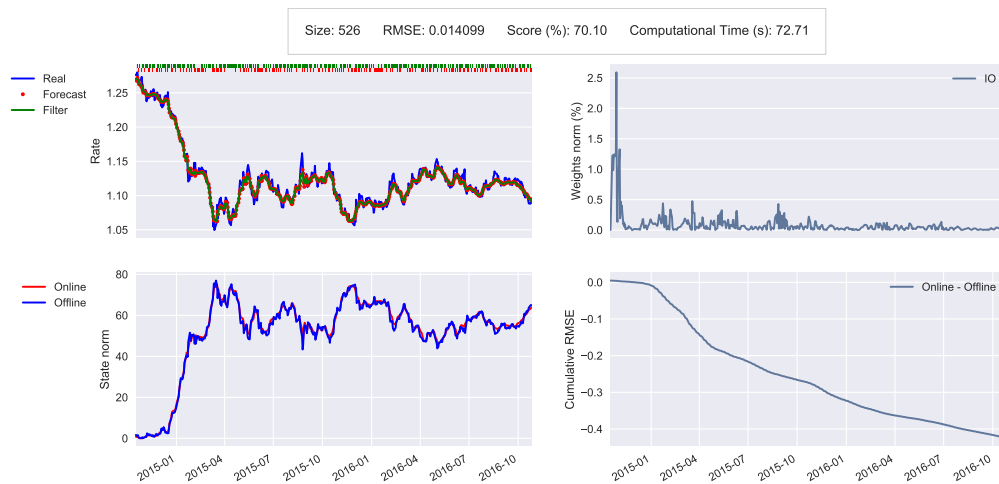


Figure 12: EURUSD online forecasting on the validation sample.

From Figure (12) above, we note that good performance is still maintained on the validation sample.

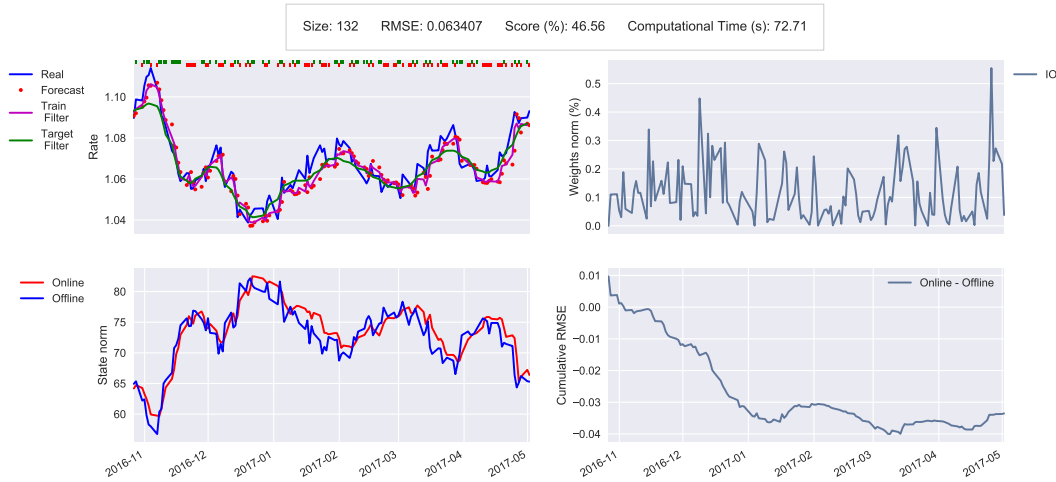


Figure 13: EURUSD online forecasting on the testing sample.

On the testing sample, the performance is degraded as the filter we used has no-longer information about the future. Therefore, it does not move like the green one. Nonetheless, the states norm is still smoother than the one obtained from the offline algorithm, which is desired when learning time series. Further, we observe that the output weights are varying a lot, although with small amplitude (most of the time below 0.5%). Thus, we conclude that the forecast has not reach the nearest stable point (which would correspond to the next value of the filtered series we use).

	Training	Validation	Testing
RMSE	0.0213	0.0314	0.0813
Score	0.6078	0.6694	0.4910

Table 3: Global assessment of the online algorithm on the pool of 71 FX pairs from 2015-01-01 to 2017-05-01.

In Table (3), we report the results on the same FX pool as the one used previously. Our results demonstrate good learning on the training and validation samples with low RMSEs and high scores. However, some issues remain on the testing sample. One could argue that remaining as close as possible to the filtered series (magenta) in the testing is crucial, while not interfering with the calibrated model by modifying some matrix (particularly the output weights).

3.3 Towards dynamically stable reservoirs

3.3.1 The proposed methods

So far, we have seen that Reservoir Computing (RC) models with offline training were very efficient for learning RNNs, since only the output weights were changed, making training extremely fast. However, the results provided in Section (3.2.1) are not satisfactory when exploiting the models with real data. In addition, even with an online algorithm, the results in Section (3.2.2) clearly indicate that the models fail to reproduce the patterns, or forecast chaotic series, in presence of noise. More generally, we have seen that the reservoir states must be smooth (in the sense of its norm) in order to learn the data and produce good predictions.

We presented in Appendix (17.4.2) two methods for teaching the reservoir how to remain consistently stable while feeding it with noisy input. The first one suppose we can observe stable states on the training sample and use it to obtain the output weight matrix in presence of noisy data. The second one consist in working only with filtered inputs and to directly teach the reservoir to predict the filtered data. Indeed, this filter can be seen as an indicator of the initial dataset. One could then argue that getting a good prediction of the filtered data could lead to good real time series forecasting if one can control the forecasting error.

3.3.2 Some results

To illustrate the second method (learning with filtered inputs), we consider the FX pairs previously introduced together with Algorithm (22). We observe that for some thresholds this algorithm does not converge as the states can settle in another attractor. In general it happens on the testing sample where W_{out} and W_{rec} are no-longer trained. We suspect that a more accurate choice of threshold is needed for some instruments. We have gathered the results of our experiment in the table below.

	Training	Validation	Testing
RMSE	0.0236	0.0340	0.0740
Score	0.5852	0.6205	0.5174

Table 4: Global assessment of the ARC algorithm on 71 FX pairs from 2015-01-01 to 2017-05-01.

RMSE are stable both on the training and validation samples. Regarding the score, it is also relatively stable on the training and validation samples and the hit ratio is around 52%. As a particular example, we consider the case of CADSGD presented below.

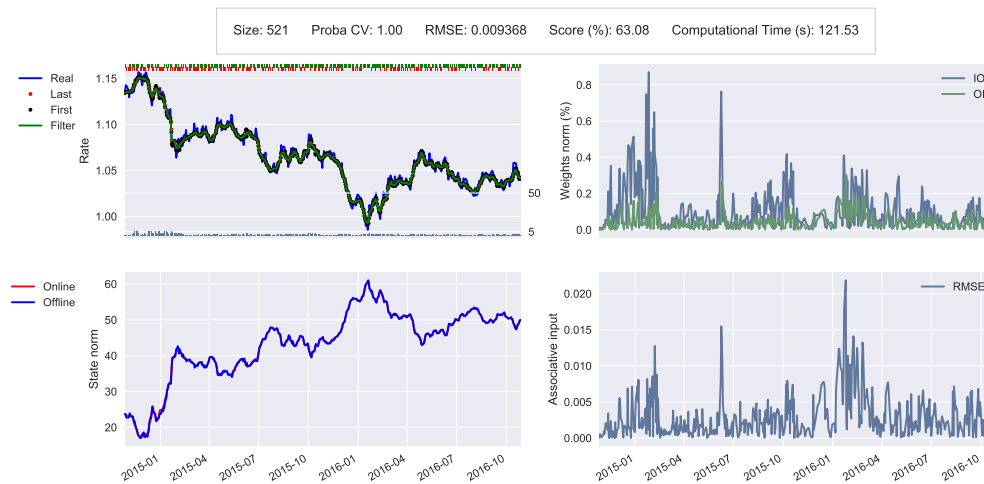


Figure 14: CADSGD online forecasting on the validation sample.

We note that the reservoir is learning very well to calibrate the filtered signal. There are very few iterations in Algorithm (22), which indicates reservoir stability. Furthermore, the weights norm graph shows that only small variation of W_{out} and W_{rec} are needed to learn to predict well (below 0.4% on the right half part of the graph). Also the

associative input absolute error is very small at the end of the validation sample (below 0.007). This shows that the reservoir has properly learned the matrices W_{out} and W_{rec} to forecast the filtered signal, and therefore to predict well the slopes of the real financial time series. Consequently, it makes sense to suppose that our model is now constant in the testing sample (W_{out} and W_{rec} are no-longer updated with Algorithm (21)).

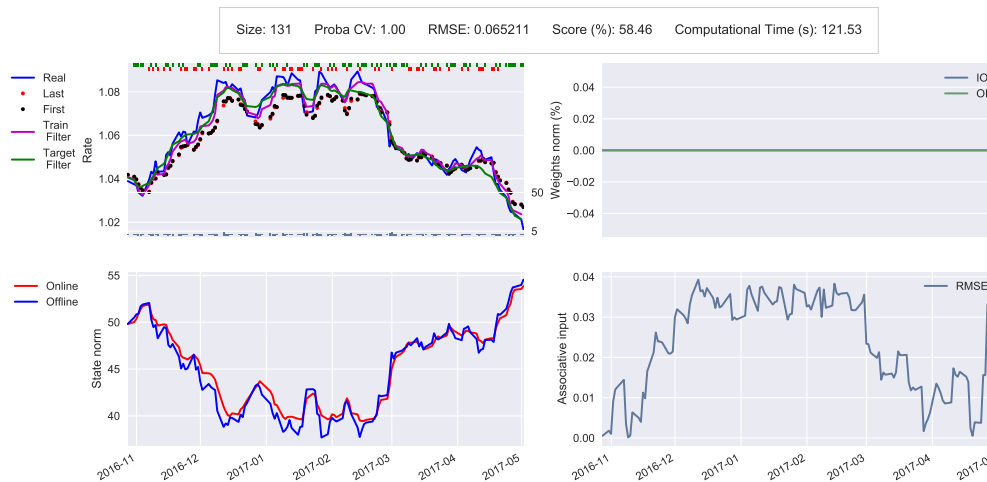


Figure 15: CADSGD online forecasting on the testing sample.

Note, it is important to recall that the model is constant in this sample, meaning that the output and recurrent matrices are not updated. That is, we let the model make prediction only by input feeding and use of Algorithm (22). The bottom bar plot on the top left graph indicates that the reservoir needs only a few iteration to converge to an attractor (always below 5), attesting prediction robustness. The reservoir reproduces well the shape of the train filter (magenta) and stay stable (bottom left figure) at the same time. Secondly, one can observe that the model produces good predictions at the beginning of the testing sample. It can be inferred as follows:

- The top bar plot on the left top graph.
- The associative input absolute error which is relatively small.

Further, by comparing the associative input with the real one at time t , we have some indications about the model convergence to the attractor it should predict without waiting for time $t + 1$.

Lastly, we note that the magenta series is very correlated with the green series, which explains why the forecasts are good. One solution could be to find appropriate filter thresholds to correlate them artificially on the validation sample and see if the correlation is maintained on the testing sample. It is important to point out that there is optimisation to be done for this issue as a too high or too low threshold would both lead to high correlation but poor performance. However, there may exist some thresholds leading to good learning and good enough correlation.

4 Forecasting directions

We are going to describe an algorithm for forecasting the directions of the underlying asset. We will then discuss adaptive decision trading systems that can be applied to all stocks.

4.1 Introduction

4.1.1 An overview

As discussed in Section (3), the prediction of financial time series is a highly challenging task since they are multifractal, thus exhibiting non-Gaussian distribution, the presence of extreme values, and long-range dependent dynamics. It requires models to capture subtle functional relationships among the empirical data even though the underlying relationships are unknown or hard to describe. Fortunately, such relationships are not required as far as predicting future trend is concerned. This is because the direction of an underlying asset refers to the movement of the asset price or the trend of fluctuation in the underlying asset in the future. Therefore, the change of a feature over time is more important than the absolute value of a feature. Further, forecasting the trends results in networks that are less complex than when forecasting the price levels. Precise forecasting of the trends of the underlying asset can be extremely advantageous for investors.

Machine learning (ML) models, such as support vector machine (SVM) and reinforcement learning, have been extensively studied when forecasting the market price directions (see Moody et al. [2001], Huang et al. [2005]). Kim [2003] used SVM to predict the direction of the daily stock price movements on the Korean market, obtaining a hit rate of 56%. Academics also examined Random Forests, K-nearest neighbour, and Gaussian naive Bayes before considering to more complex models such as ANNs and Deep NNs. In an empirical study, Gerlein et al. [2016] showed that models such as $C4.5$, K^* , logistic model tree, were in the range of 48% – 54%. However, the performance of a stock market predictor heavily depends on the correlation between the data used for training and the current input (feature) for prediction.

Several studies explored the relationship between some initial input variables (also called factors or predictors) and the closing price of the considered underlying asset. The features are based on fundamental and technical factors. For instance, Kaastra et al. [1996] analysed the importance of knowing the input variables when forecasting financial time series. It is generally assumed that the input variables have a predictive power. However, the forecasting power of networks is quite sensitive to the capacity of identifying the right input variables (features, other correlated underlying assets, technical indicators) that will best explain the future variations of the underlying asset. Given the increasing processing power of computers, academics are now trying all kind of explanatory variables, combining them in a very large input set, hoping that ML will identify some relationships with the process to be forecasted. Further, some text-mining based methods developed to automatically analyse fundamental factors such as financial status, development strategies, economic conditions psychology of investors (see Ruiz et al. [2012], DingEtAl15). Schumaker et al. [2009] used text-mining technique to forecast, obtaining a hit rate of 57%.

The paradigm has been shifted from identifying the best statistical model to identifying the best input variables, considering the ML model as a black box which will always learn the relationship $Y = f(X)$ between the input and output. The motto is to let the data speak for themselves. In addition, when the forecasting results are not satisfactory, the data scientists will occasionally change the black box $f(\cdot)$ and repeat the tests. There is no-longer a need to analyse the data and scientifically propose a predictive model capable of accounting for the properties of the data at hand. One wonder where the race will end.

4.1.2 The input variables

4.1.2.1 Fundamental factors In the early days, the features selected for the inputs of the networks were mostly derived from the data within the same market under concern, leaving out important information. One solution is to consider sentiment analysis, which infer the attitudes of several key figures or successful analysts in the markets, and then interpolate the general sentiment of investors. However, this approach is strongly biased towards people opinions. Another solution is to consider the use of global stock data as the input features to the networks. Since markets are interconnected, Shen et al. [2012] proposed to consider spatial correlation and combine stock markets, commodity prices and foreign currency data as potential features. Further, one can also consider the temporal correlation between

the closing prices of the markets that stop trading right before the considered market, or at the beginning of that market. In the latter, we can calculate the autocorrelation and cross-correlation of different market trends as a measure for selecting input features.

We can also consider the correlation of market trends in longer term. In general, prediction accuracy increases when time span becomes longer.

4.1.2.2 Technical indicators Over the last twenty years, traders and academics have been using technical analysis as well as certain fundamental valuation metrics to improve results when forecasting trends (see Gencay [1997], Metghalchi et al. [2007]). The theoretical basis of technical indicators (TIs) is the concept of momentum, where a momentum oscillator is used to measure the velocity or rate of change of price over time (see Appendix (11.2)). Further, some economists and statisticians showed that technical indicators have predictive power and found a good measure of support for their effectiveness (see Chen et al. [2009], Metghalchi et al. [2012] [2015]). These properties led some data scientists to incorporate such trading strategies in machine learning in order to forecast future price directions (see Tanaka-Yamawaki et al. [2007], Lee et al. [2007], Senol et al. [2008], Kara et al. [2011], Patel et al. [2015], Qiu et al. [2016]).

4.1.3 Selection and representation

4.1.3.1 Selecting the technical indicators Given the large number of technical indicators, some practitioners proposed to use a specific model for selecting the best TIs to enter the network. Kim et al. [2000] used neural network modified by genetic algorithm (GA). Kwong [2004] used GA to get a dynamic mechanism for selecting appropriate factors from fundamental data and TIs. Mitra [2009] combined signals generated from moving averages of different window size using ANN models. Huang et al. [2009] combined support vector regression (SVR) with a self-organising feature map (SOFM) and feature selection based on filtering for predicting the next day price of the Taiwan index futures. Lee [2009] developed a prediction model on the basis of an SVM with a hybrid feature selection method to find the best input features. Chavan et al. [2013] surveyed different input parameters that could be used for stock market prediction with ANN and concluded that hybridised parameters like combination of technical and fundamental variables gave better prediction accuracy. Adebisi et al. [2012] also combined the technical and fundamental variables using ANN models. Chandwani et al. [2014] combined both technical and fundamental parameters to forecast price directions on the Indian stock market. They integrated genetic algorithm (GA) with two classification techniques, the ANN and SVM, where GA is used to select the set of the most informative input features from among all the technical and financial indicators. They showed that GA significantly increased the accuracy of ANN. However, forecasting trends inherently perform better in trending markets (bull or bear markets) than in markets that might display volatility and be trendless over the examination period.

4.1.3.2 Representing the technical indicators There are two main approaches when using technical indicators:

1. one can either directly use the continuous technical indicators (actual time series), or
2. one can choose to represent them as trend deterministic data (discrete in nature) (see Appendix (11.5.3)).

4.2 The predictive algorithm

We are going to present some methods for forecasting the underlying directions. We will also show that we need a feature selection algorithm to select the features that contribute most to the accuracy of prediction using different machine learning algorithms. The direction measures used to assess the predictive power of the algorithm are defined in Appendix (9.4.2).

4.2.1 Selecting the input variables

When designing neural networks we need to select the most relevant input variables with best forecasting power to reduce the number of inputs used to train the networks and improve accuracy. Since both input and output variables are noisy, variable selection models developed that provide appropriate inputs by a trial and error process (see Deboeck [1994]). They decide on the selection of the inputs to be used and their re-sampling frequency. For instance, sensitivity analysis is used to determine the importance of the inputs to the neural network. It consists in ranking each input based on the contribution to the neural network and eliminating the bottom of the list. That is, one selects a set of data as input, train a network until an acceptable level of performance is achieved, apply sensitivity analysis and eliminate all the variables below a predefined threshold. Alternatively, one can determine the impact that setting an input to its average value has on the overall performance of the ANN. If the network is not using an input to discriminate between possible outcomes, fixing that input to its average value should actually improve the overall performance.

4.2.2 Representing the input variables

There are two main approaches when using technical indicators. When selecting the input variables of the predictive model, we can:

1. Directly use continuous technical indicators (actual time series): Kara et al. [2011] proposed a method in the case where the inputs are the continuous technical indicators. The values of all technical indicators are normalised in the range between $[-1, +1]$ so that larger value of one indicator do not overwhelm the smaller valued indicator. Given a (one hidden layer) neural network (NN), the output can be a single neuron with a sigmoid transfer function, which results in a continuous value output between 0 and 1 (see Figure (16)). A threshold of $\frac{1}{2}$ is used to determine the up and down direction prediction, where the former occurs for a value greater than $\frac{1}{2}$ and the latter occurs for a value smaller than $\frac{1}{2}$.
2. Choose to represent them as trend deterministic data (discrete in nature): In the case where we choose to represent the indicators as trend deterministic data (TDD), Patel et al. [2015] proposed the Trend Deterministic Data Preparation Layer (TDDPL) (see Figure (17)). It extracts trend related information from each of the technical indicators by converting continuous-valued technical parameters to discrete value (up +1, down -1), representing the trend (see details in Appendix (11.5.3)). The idea being that if a prediction model is given direct continuous-valued input, it will try to establish relation between the values at different time steps, which is not required as far as predicting future trend is considered. What matter is how its value (up +1, down -1) has changed with respect to previous time step rather than the absolute value of change. Each input parameters in its discrete form indicates a possible up or down trend determined based on its inherent property. Thus, when using TDDPL, we are directly inputting the trend based such that prediction models only have to determine co-relation between the input trends and output trend (which is easier than determining relation within serially correlated continuous time series).

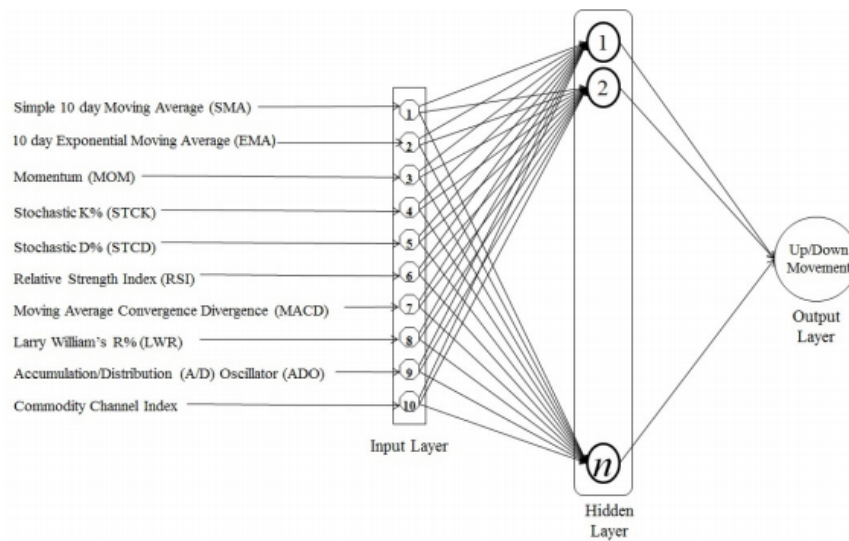


Figure 16: A three-layer neural network with continuous indicators

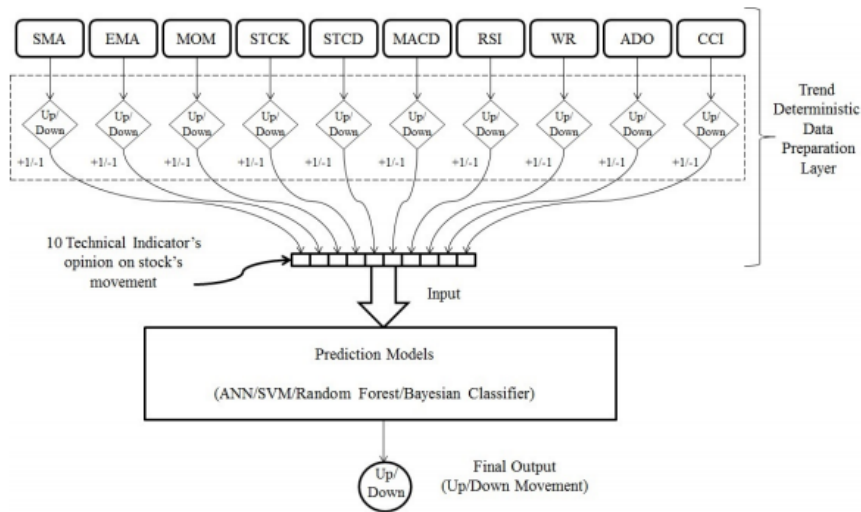


Figure 17: Predicting with trend deterministic data

4.2.3 The algorithm

The predictive model proposed by Van den Poel et al. [2016] is made of two layers, namely the Feature Engineering (selection) layer and the Prediction layer. This is similar to stacking described in Appendix (13.5). Rather than using the Deterministic Data Preparation Layer (DDPL) proposed by Patel et al. [2015], the authors chose to combine the continuous indicators with some technical trading rules which give signals based on criteria for identifying market entry and exit points (see Appendix (11.5.1)). The Feature Selection layer devises the trading rules for each technical indicator. The input of the model are the return values corresponding to the dependent variable along with the continuous-valued technical indicators (see Appendix (11.2)), single technical trading rules and complex trading

rules based on two-way and three-way combinations of individual indicators (see Appendix (11.3)), constituting the independent ones. Thus, the only difference with DDPL is the use of complex trading rules which greatly increases the number of predictors. First, using the generated data, they derive a set of simple and complex trading rules for each of the technical indicator employed, resulting in the discrete values 1, -1, or 0, representing buy, sell or hold signals, respectively. We denote the direction triplet as $(1, -1, 0)$ for each predictor. These signals serve as inputs to the second layer, which employs Random Forest for feature learning and classification. The return values and the predictors are used for learning purposes (similar to Figure (17)). It produces an estimated direction triplet $(1, -1, 0)$.

4.2.4 Strategy and returns

When testing their model, Van den Poel et al. [2016] considered one year of data sampled with a fixed period of time $\delta = 1$ minute. The training sample is eight-month long and the testing sample covers a four-month period. The trading horizon is a day and PnL is determined per minute ($d = 1$ minute) with no reinvestment. The strategy is as follows: when a buy signal is triggered the trader takes a long position in the underlying asset until a sell signal is generated. Then the trader unwind (sell) is position and goes short until the next buy signal or the end of day (if no position is held overnight). In case of two consecutive buy or sell signals, the first one is considered and the other ignored. The returns are determined by applying the trading strategy to the generated signals. We get the mapping

$$(1, -1, 0) \Rightarrow ((+R, -R), (+R, -R), 0) = ((1, 0), (0, 1), 0)$$

where 1 indicates a positive return and 0 a negative return.

The machine learning (ML) model is trained to recognise valuable signals from invaluable ones so that whenever the predictive probability on a positive return exceeds a cutoff value the trade is executed. The overall return generated by the model is then calculated by adding up the returns of the performed trades.

4.3 A hybrid algorithm

The mathematical foundations of modern statistics were laid in the 17th century with the development of the probability theory by Cardano, Pascal and de Fermat. The modern field of statistics emerged in the late 19th and early 20th century, where Galton and Karl Pearson transformed statistics into a rigorous mathematical discipline used for analysis. It was further developed by Fisher in the 1920s and refined by Egon Pearson and Neyman in the 1930s. Since that time, statistics is a mathematical body of science that pertains to the collection, analysis, interpretation or explanation, and presentation of data. It is concerned with the use of data in the context of uncertainty and decision making in the face of uncertainty. It therefore makes sense to combine statistics with machine learning.

Since machine learning algorithms do not require any assumptions about the data, hybrid algorithms developed combining big data analytic techniques with machine learning algorithms. Multivariate analysis consider dimensionality reduction technologies, such as principal component analysis (PCA), fuzzy robust principal component analysis (FR-PCA) and kernel-based principal component analysis (KPCA), among others. Zhong et al. [2017a] presented a study of dimensionality reduction with an application to predict the daily return direction of the SPDR S&P 500 ETF (ticker symbol: SPY) using ANN classifiers. They showed that PCA-based ANN classifiers produced the best predictor of the ETF daily return direction over various datasets transformed using PCA. Zhong et al. [2017b] performed a comprehensive data mining procedure, including both cluster and classification mining, to forecast the ETF daily return direction. They showed that PCA-based ANN classifiers lead to significantly higher accuracy than three different PCA-based logistic regression models.

Zhong et al. [2019] proposed a classification mining procedure based on hybrid machine learning algorithms. They studied deep neural networks (DNNs) and traditional artificial neural networks (ANNs), acting as classifiers, in the task of predicting daily return directions of the SPDR S&P 500 ETF based on 60 financial and economic features taken as input. The models were tested on untransformed dataset, along with two datasets transformed via principal

component analysis (PCA). The principal components can be written as the linear combinations of all the factors with the corresponding coefficients equalling the elements of the eigenvectors (see Appendix (10.1.4)).

Network with a large number of hidden layers have been developed to extract the classification rules from the trained neural networks. For instance, the multilayer feed-forward network is most commonly used for pattern recognition, including classification, in data mining. In their terminology, ANNs have up to ten layers, while DNNs have more than ten layers. The authors considered multilayer feed-forward neural networks specifically designed for classification problems (see Chiang et al. [2016], Kim et al. [2016]). These networks are used for pattern recognition, including classification, in data mining.

The data is preprocessed to account for missing values, outliers, and it is normalised. The data is then transformed using PCA. Once the models have been trained, the aim is to recognise the different patterns of new data during the testing stage. Here, direction forecast (up or down) are more likely to be on the right side of the trade, leading to forecasting models with high classification accuracy. The difference between the predicted class and the true class is measured by the mean squared error (MSE). Successful direction forecasts have a prediction accuracy closer to 60%. The model is best trained in the sense that the validation phase achieves its lowest MSE with the trained model. The simulation results showed that the DNNs using two PCA-represented datasets produce significantly higher classification accuracy than those using the entire untransformed dataset.

4.4 The evaluation metrics

Evaluation measures for classifiers are presented in Appendix (12.4). When it comes to forecasting directions, we need to express the evaluation metrics in terms of market returns and signals. To do so, we let $R_{k,up}$ and $R_{k,down}$ denote an up movement and down movement of the market return at time t_k , respectively. They are given by

$$R_{k,up} = I(r_{L,d}(k) > 0) \text{ and } R_{k,down} = I(r_{L,d}(k) < 0)$$

while a neutral position is $R_{i,neutral} = I(r_{L,d}(k) = 0)$. We also let $M_k = I(r_{L,d}(k) \neq 0)$ be a move and define the k th signal up and down as follows:

$$\text{signal}_{k,up} = I(\hat{f}_k = +1) \text{ and } \text{signal}_{k,down} = I(\hat{f}_k = -1)$$

The k th trade return $R_{k,trade}$ is given by

$$R_{k,trade} = \text{signal}_{k,up} \times r_{L,d}(k) - \text{signal}_{k,down} \times r_{L,d}(k)$$

A win trade is given by

$$\text{win}_k = I(\text{signal}_{k,up} \times R_{k,up} + \text{signal}_{k,down} \times R_{k,down})$$

and a loose trade is given by

$$\text{lose}_k = I(\text{signal}_{k,up} \times R_{k,down} + \text{signal}_{k,down} \times R_{k,up})$$

The union of both a win and loose trade is

$$\text{wl}_k = \text{win}_k \cup \text{lose}_k$$

We denote $N_{sig,up} = \sum_{k=1}^{N_k} \text{signal}_{k,up}$, $N_{sig,down} = \sum_{k=1}^{N_k} \text{signal}_{k,down}$ the number of signal up and down, respectively, and $N_{sig} = N_{sig,up} + N_{sig,down}$. We let R_{trade} be the vector of trade returns with k th element $R_{k,trade}$, and we denote R_{trade}^{win} the vector of trade returns for which all elements have $\text{win} = 1$ and R_{trade}^{loss} the vector of trade returns for which all elements have $\text{loss} = 1$. We also denote R_{trade}^{wl} the vector of trade returns for which all elements have

$wl = 1$. We let N_h be the number of hours during the trading period and define the total number of possible forecast for the model as

$$N_f = N_h \times \frac{60 \times 60}{u}$$

where u is the unit of time. We can then present a few metrics

$$\begin{aligned} \text{accuracy} &= \frac{\sum_{k=1}^{N_k} (\text{signal}_{k,up} \times R_{k,up} + \text{signal}_{k,down} \times R_{k,down})}{\sum_{k=1}^{N_k} (\text{signal}_{k,up} \times M_k + \text{signal}_{k,down} \times M_k)} \\ \text{accuracy up} &= \frac{\sum_{k=1}^{N_k} \text{signal}_{k,up} \times R_{k,up}}{\sum_{k=1}^{N_k} \text{signal}_{k,up} \times M_k} \\ \text{accuracy down} &= \frac{\sum_{k=1}^{N_k} \text{signal}_{k,down} \times R_{k,down}}{\sum_{k=1}^{N_k} \text{signal}_{k,down} \times M_k} \\ \text{hit} &= \frac{1}{N_{sig}} \sum_{k=1}^{N_k} (\text{signal}_{k,up} \times R_{k,up} + \text{signal}_{k,down} \times R_{k,down}) \\ \text{hit up} &= \frac{1}{N_{sig,up}} \sum_{k=1}^{N_k} \text{signal}_{k,up} \times R_{k,up} \\ \text{hit down} &= \frac{1}{N_{sig,down}} \sum_{k=1}^{N_k} \text{signal}_{k,down} \times R_{k,down} \\ \text{coverage} &= \frac{1}{N_f} \sum_{k=1}^{N_k} (\text{signal}_{k,up} + \text{signal}_{k,down}) \\ \text{average win} &= \text{median}(R_{trade}^{win}) \\ \text{average loss} &= \text{median}(R_{trade}^{loss}) \\ \text{average total PnL} &= \text{average win} \times \text{accuracy} + \text{average loss} \times (1 - \text{accuracy}) \\ \text{standard dev win} &= \text{std}(R_{trade}^{win}) \\ \text{standard dev loss} &= \text{std}(R_{trade}^{loss}) \\ \text{mean total PnL} &= \text{mean}(R_{trade}^{wl}) \\ \text{standard dev total PnL} &= \text{std}(R_{trade}^{wl}) \\ \text{Sharpe ratio} &= \frac{\text{mean total PnL}}{\text{standard dev total PnL}} \end{aligned}$$

4.5 Adaptive decision trading systems

Trading systems are often highly dependent on preselected inputs and model parameters that are expected to continue providing trading information well after the initial training or back-tested model development period. However, given the endlessly changing futures market (multifractal market), one must consider a decision support systems (DSS), or rule change trading system (RCTS), based on numerous trading rules generated by some statistical and probability analysis to cover diverse market conditions.

Some authors tried to find one model that could be applied to all stocks. That is, describe the relationship among stock index variables and determine the best representative variables and best theory models for stock indices. They must optimise system performance based on a specific timeframe and desired output. For instance, Samaras et al. [2004]

proposed a multi-criteria decision support system that aims at presenting an evaluation of stocks on the Athens Stock Exchange on the basis of fundamental analysis. Chavarnakul et al. [2009] used technical analysis (see Appendix (11)), along with employing a neural network as a decision maker, for stock trading. Chang et al. [2009] developed a three-stage system for stock trading prediction, called CBDWNN. These stages are: (1) screening out the potential stocks and the important influential factors; (2) using a backpropagation neural network to predict the buy/sell points of stock price; and (3) adopting case-based dynamic windows to further improve the forecasting results.

In general, a model fitting one security well may poorly perform another one. We must therefore consider an adaptive decision trading system (ADTS). For instance, Chiang et al. [2016] proposed a system that adapts both the inputs and the prediction model based on the desired output. This is an adaptive decision support system model, where multiple inputs and modelling techniques are utilised. It adaptively creates a new prediction model for every single security. Potential variables facilitating the best network performance are selected and used in the model as inputs. For instance, the closing price and volume of the stock index for the last trading day together with technical measurements are considered. The output layer has one node, which is the stock movement direction. As discussed above, the output from the neural network is standardised between zero and one and split in two, leading to three scenarios: buy, hold, and sell. Given that with N potential variables, the different possible combination of inputs for the neural network will reach $2^N - 1$, the authors decreased the size of the neural network used to five variables, hence five inputs. In that setting, the number of combination for inputs is

$$C = \sum_{i=1}^5 C_N^i$$

where C_r^n ¹ is the number of combination of r objects taken from n different objects. There are four basic modules: the data pre-processing module, the neural network initialisation module, the neural networking training module, and the output module. The learning algorithm minimise the mean square error (MSE) between the estimated and real output. Direction prediction correctness (DPC) is obtained by calculating the percentage of correct predictions during the testing period. In order to assess the potential benefit of the model, simulated returns following the trading signal output are produced which are compared against a buy-and-hold strategy. Note, higher prediction accuracy does not always guarantee higher return. Thus, forecasting the direction alone is not sufficient and must be combined with some criteria on the generated returns when determining the final predictive model. During real market transactions, the model with the highest return is used as the final adoption model for the next prediction. Since daily movements are usually small, carrying more noise, direction predictions near 55% are sufficient to generate profit. The system provided significantly higher return than the buy-and-hold strategy.

As discussed above, since neural networks have difficulty catching-up with the small changes, one should consider denoising the data with PCA or wavelet transformation before providing it to the network. The resulting trends of these data are clearer. Using the same system on this smoothed data, one can get higher returns. The system can ignores small fluctuations and remains effective in detecting larger trends, reducing the transaction fees.

In practice, trading rules are difficult to apply in the future due to endless changes in the market due to its multifractal nature. To account for changing market conditions, Kim et al. [2016] proposed a rule change trading system (RCTS) based on previous trading results. The system uses rough set analysis to extract a number of trading rules in order to cover diverse market situations and then change the trading rules according to trading performance. Rough set theory (applies to discrete data) is used for stock market analysis, market timing, and trading rule generation. Many reduction methods exist to discretise the data, such as manual reducers, genetic algorithms, the Johnson algorithm, and the dynamic reduct method. To facilitate the rule change mechanism, a genetic algorithm is employed with the objective function of maximising the payoff ratio to determine the thresholds of market timing for both buying and selling in the futures market.

¹ $C_r^n = \frac{n!}{(n-r)!r!}$

5 Forecasting returns and directions

We are going to describe an algorithm for forecasting the both the price level and the directions of the underlying asset. At last, we will draw conclusions on the results and define a set of rules we want to follow when building our trading algorithm.

5.1 Improving the algorithm

5.1.1 Discussion

We have seen in Section (4.5) that accuracy could be a good measure for the general ability of a network, but that it does not convey any information on profitability. Thus, we should favour return metrics taking into account the magnitude of the trends that a system discovers (see Kim et al. [2016]). For instance, when forecasting directions, the data should be denoised and the trading decisions should be combined with some criteria on the generated returns. As a result, the model with the highest return should be selected as the final adoption model for the next prediction. However, while this is a good choice, nothing guarantees that the past will repeat itself. One solution is to consider a model that also forecast the price level for the next period.

We are now going to describe a framework for simultaneously forecasting both future price levels and asset price directions by considering an optimum combination of

- technical indicators (see Appendix (11.2)),
- trading rules, and
- trading strategies (see Appendix (11.5)).

To do so, we will combine the predictive model proposed by Van den Poel et al. [2016] with the proposed forecasting model presented in Section (3), which we will enlarge by incorporating the trading strategies proposed by Metghalchi et al. [2018].

5.1.2 Description

We assume that we can denoise the data with PCA, wavelet decomposition, or other filtering tools.

5.1.2.1 Selection layer We are now going to explain how to enlarge the algorithm presented in Section (4.2) by incorporating the trading strategies proposed by Metghalchi et al. [2018]. At each time step we will select the most efficient trading strategy among the list of strategy given in Appendix (11.5.2). To do so, we propose the Meta Buy/Sell Signal (MBSS) inferred from all the predictors generated by the selection layer. We are going to build a level of confidence around the MBSS and use it to select the most efficient strategy. As discussed above, at a given time t , the inputs of the latter are all the independent variables and the output is the direction triplet $(1, -1, 0)$ for each independent variable. In order to get a meta buy/sell decision signal, we choose to combine all the direction triplets with a majority voting rule (or any other rule) (see Appendix (11.3)). We can then build an algorithm that estimates the likelihood of the majority voting rule and, accordingly, select the best trading strategy. Since the learning process happens within the training sample (see Section (2.4.4)), everything is known and we can let the boolean $\alpha(t)$ be true (T) if the estimated direction is correct and false (F) otherwise. Assuming N predictors (or independent variables), we let $n(t)$ be the number of predictors that gives $\alpha(t) = T$. We can therefore estimate n^* as the minimum n that gives the maximum number of true flags over the training sample. Thus, when validating and testing the data at a given time t_+ , we compute all direction triplets, apply the majority voting rule, and infer $n(t_+)$. Then, the trading rule is as follows:

$$\begin{cases} \text{if } n(t_+) << n^* \text{ use cash} \\ \text{if } n(t_+) < n^* \text{ use long \setminus cash} \\ \text{if } n(t_+) > n^* \text{ use long \setminus short} \\ \text{if } n(t_+) >> n^* \text{ use leverage \setminus short} \end{cases}$$

5.1.2.2 Prediction layer Since machine learning models can learn more than one task, we can use a neural network (or combine two neural networks) to learn both the future price levels and the asset price directions. It is a simple matter to modify the above algorithm to forecast price levels or returns. When discussing continuous technical indicators as input variables, Kara et al. [2011], Patel et al. [2015], among others, considered a three layer ANN model, which does not account for serially correlated time series. However, we have already presented a framework in Section (3) that forecast market returns and highlighted its good predictive power on denoised financial time series. Since the continuous technical indicators are mostly filtered financial time series, we can use a recurrent neural network (RNN) model and consider continuous technical indicators to forecast future directions (see Figure (17)). The time dependent direction measures used to assess the predictive power of the algorithm are defined in Appendix (9.4.3).

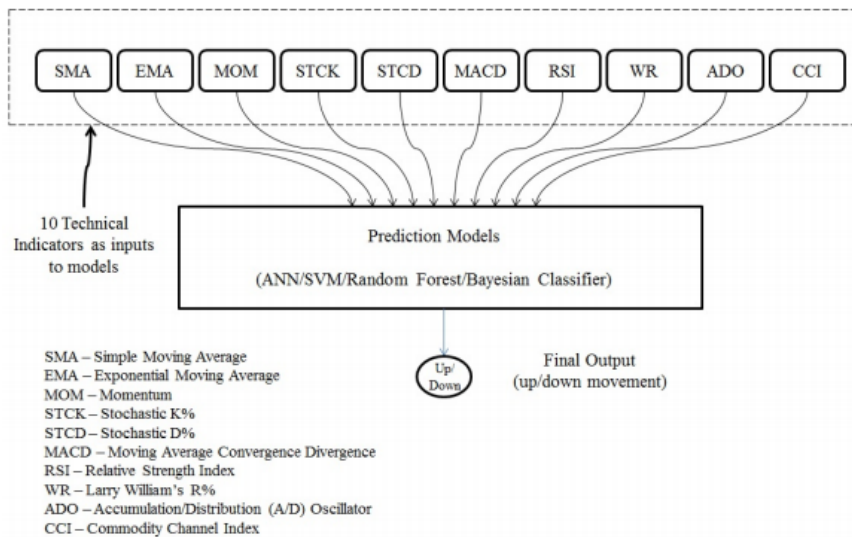


Figure 18: Predicting with trend continuous data

5.1.2.3 The algorithm The steps of the algorithm are as follows:

- Preprocess the data
- Denoise the data with PCA, wavelet decomposition, or other filtering tools
- Create the technical indicators in time and space
- Select the best indicators (or Cluster the indicators and stack them)
- Build a model to forecast returns
- Build a model to forecast directions
- Combine the two previous models, or build a model that simultaneously forecast both returns and directions

- Define a set of trading rules
- Define trading strategies and implement an adaptive decision trading system (ADTS)

5.2 Generative networks

We are now going to present a few methods using a generative adversarial network (GAN) models (see Appendix (19)) to forecast the market prices and their directions.

5.2.1 Combining LSTM and CNN

Zhou et al. [2018] proposed a GAN model using long short-term memory (LSTM) and a convolutional neural network (CNN) for adversarial training to forecast high-frequency stock market. The method of rolling partition training and testing set is used to analyse the effect of the model update cycle on the prediction performance. Basic technical index data was used as input variable.

We let X_t be a vector of basic indicators and Y_t be the closing price at time t . We let $\mathcal{T} = \{1, 2, \dots, T\}$ be a set of one-minute interval, where T is the maximum time lag set to 242. Historical data is denoted by $X = \{X_1, X_2, \dots, X_T\}$ and $Y = \{Y_1, Y_2, \dots, Y_T\}$, the goal being to predict Y_{T+1} . The generative model G is the LSTM model, and the output is $\hat{Y}_{T+1} = G(X)$. The discriminative model D is the CNN model performing convolution operations on the one-dimensional input series to estimate the probability of that series to come from the dataset $\bar{Y} = \{Y, Y_{T+1}\}$ or to be produced by G , $\hat{Y} = \{Y, \hat{Y}_{T+1}\}$. This is because a trader makes a forecast with model G and then estimate the probability of success of his forecast based on previous observed prices. The training of the pair (G, D) minibatches of size one are considered (SGD). The process consists of two alternated steps:

1. Training G : Classifying $\bar{Y}$ into class 1 and $\hat{Y}$ into class 0, the adversarial loss for G is

$$L_{adv}^G(\hat{Y}) = L_{sce}(D(\hat{Y}), 1)$$

where L_{sce} is the sigmoid cross-entropy loss given by

$$L_{sce}(A, B) = - \sum_i B_i \log(f(A_i)) + (1 - B_i) \log(1 - f(A_i))$$

where $f(\cdot)$ is a sigmoid function. In practice, minimising the adversarial loss alone is not sufficient to get accurate predictions, and G must also decrease the forecast error, that is, the loss

$$L_p(\bar{Y}, \hat{Y}) = \|\bar{Y} - \hat{Y}\|_p$$

for $p = 1$ or $p = 2$. Further, since stock price direction is crucial, we can define the direction prediction loss function as

$$L_{dpl}(\bar{Y}, \hat{Y}) = |sgn(\hat{Y}_{T+1} - Y_T) - sgn(Y_{T+1} - Y_T)|$$

Combining all these loss functions with parameters λ_{adv} , λ_p , and λ_{dpl} we get the final loss function

$$L_G(X, Y) = \lambda_{adv} L_{adv}^G(\hat{Y}) + \lambda_p L_p(\bar{Y}, \hat{Y}) + \lambda_{dpl} L_{dpl}(\bar{Y}, \hat{Y})$$

Then, one SGD iteration is performed on G to minimise L_G , while keeping the weights of D fixed.

2. Training D : We keep the weights of G fixed and perform one SGD step on D to minimise the target loss

$$\begin{aligned} L_D(X, Y) &= L_{adv}(\bar{Y}, \hat{Y}) \\ &= L_{sce}(D(\hat{Y}), 0) + L_{sce}(D(\bar{Y}), 1) \end{aligned}$$

The model was tested on 42 stocks on the Chinese market from Jan16 to Dec16. 13 basic technical indicators were chosen for the input data X at each time. The dataset is split into training (M days) and testing (N days) set and a rolling segmentation was applied on a daily basis. Here, $M \in \{10, 20, 60\}$ and $N \in \{5, 10, 20\}$. Three baseline methods were selected for comparison: $ARIMA(1, 1, 1) - GARCH(1, 1)$, a fitted ARIMA model to forecast stock prices and a GARCH model to forecast volatility. Then, an ANN is used and an SVM. Standard evaluation metrics are used such as the RMSRE and the direction prediction accuracy (DPA). The DPA of GAN-FD ranges around 64.6% to 72.2% and its RMSRE is between 0.48% to 1.49% when $N = 5$, and the DPA slumps to 52% to 62.7% and the RMSRE increases between 1.21% to 4.96% when $N = 20$. Thus, one need to reduce the update cycle N and regularly revise the model parameters to adapt to market.

5.3 Conclusions

While a simplified representation of reality can either be descriptive or predictive in nature, or both, financial models are predictive to forecast unknown or future values based on current or known values, using mathematical equations or set of rules. However, the forecasting power of a model is limited by the appropriateness of the inputs and assumptions made, so that one must identify the sources of model risk to understand these limitations (see Appendix (11)).

5.3.1 Forecasting returns

5.3.1.1 Failure to capture noisy data We saw in Section (3) that then forecasting price levels (case (1+)) in Section (2.4.1)), most articles did not report good results when testing data. This is because the financial time series are multi-fractal and transient. Thus, returns are filtered (denoised) and used to train ML models, which are then exploited on transient process (case (3*)) in Section (2.4.4)). A large literature has been written describing case (3*). However, good results are reported on the training set, but when exploiting the data, the hit ratio collapses in the range [48%, 55%] (see results in Section (3)). Unless the training set resemble the filtered data, the learning process is not preserved (see Lin et al. [2009]). Recognising that returns could not be directly forecast, Dablemont et al. [2003] first smoothed the rough data by projecting them onto a functional basis (for example cubic splines), and then forecast the future transactions splines with an empirical functional analysis of the past of series.

Consequently, ML models perform well on smoothed time series (with Hurst exponent $H > \frac{1}{2}$), but have great difficulties forecasting a series with $H \leq \frac{1}{2}$.

5.3.1.2 Overfitting the data In general, academics and practitioners consider a single machine learning (ML) model to explain market returns from different asset classes and different underlying. Usually, different topologies are tried against the data and several sets of meta-data are calibrated to the returns and a score function is applied to select the set that best fit the data (see Lukosevicius et al. [2009]). Increasing the number of model parameters for the sake of fitting a finite sample is bound to overfitting the data, explaining the poor out-of-sample behaviour. As a result, the model lose its predictive power. One solution, discussed in Section (4.5), is to consider an adaptive decision trading system (ADTS) which adaptively creates a new prediction model for every single security.

5.3.2 Capturing trend

Forecasting financial time series is characterised by data intensity, noise, non-stationarity, unstructured nature, high degree of uncertainty, and hidden relationships. Thus, predicting their future values is a highly challenging task,

which requires models capable of capturing subtle functional relationships between values at different time steps. Such relationships are not required as far as predicting future trend is concerned. Trend deterministic data is the statistical indication of whether the assets are over-bought or over-sold and as such is value independent. Thus, prediction models have to determine co-relation between the input trends and the output trend, which is a simpler task than forecasting market returns.

5.3.3 Common mistakes

Some of the mistakes commonly made by academics using ML in finance are:

- to ignore the properties of financial returns
- to ignore the statistical characteristic of the ML model considered
- to use the same ML model on all financial returns (including $H \leq \frac{1}{2}$)
- to train the model on a small data set (few thousands points)
- to trade at every sample point
- to ignore trading rules (See Appendix (11.5.1))
- to ignore trading strategies (See Appendix (11.5.2))
- to ignore risk management when validating and testing a model

5.3.4 Defining principles

We can infer some useful information from the results obtained in Sections (3) and (3.3.2):

- ML models can only reproduce certain stochastic processes (smoothed ones)
- it would make sense to use ML to forecasting time series when $H > \frac{1}{2}$
- when $H < \frac{1}{2}$, it would make sense to use ML
 1. for autonomous pattern generations by forecasting directions (see Section (4.2))
 2. to forecasting filtered time series, or equivalently technical indicators (see Appendix 11.5)

From these inferences, we can therefore the devise the following principles:

Result 1 Principles

1. when $0 < H \leq \frac{1}{2}$ we cannot forecast market returns, but we can consider
 - (a) autonomous pattern generations (see Section (4.2))
 - (b) forecasting technical indicators (see Appendix 11.5)
2. when $H > \frac{1}{2}$ we can still consider autonomous pattern, but forecasting both returns and directions (see Section (5.1)).
3. when $H \gg \frac{1}{2}$ we consider time series prediction.

6 Predicting patterns

Even if the market is weakly efficient, in the sense that on average local irregularities naturally disappear, we can still build a system to identify these hidden short-term irregularities when they occur and generate profitable trades based on defined trading strategies (see Plummer [1993]). We are going to discuss autonomous pattern generation (case [2+] in Section (2.4.1)) and their prediction in view of forecasting price trends, where the patterns are distinctive formations of market prices (see Definition (11.1)).

6.1 Overview

The stock market provides a lot of financial information. In order to make their trading decisions, day traders use various types of market data, commonly referred to as Level I and Level II market data. Level 3 is the highest level of quotes provided by a trading service and gives the institution the ability to enter quotes, execute orders and send information. All three levels of quotes build on top of each other. Basic market data is known as Level I data (bid / ask price, bid / ask size, last price, last size), while Level II (also known as the order book) provides more information (highest / lowest bid / ask price, highest / lowest bid / ask size,). When using big data we call these market data features (or variables). It consists of date, type, completion price, completion amount, opening price, high price, low price, price just before, accumulated completion amount, accumulated completion price.

As discussed in Section (2.5), machine learning (ML) has been used to analyse and predict stock prices. For instance, Cao et al. [2001] used SVM to determine whether the new pattern data belongs to a certain category (or types) of shapes. Others, such as Kimoto et al. [1990], Kohara et al. [1997] used ANNs to predict daily stock prices in the case of complex relationships of the variables. However, stock prices consist of several patterns which appear repeatedly at time intervals (see Appendix (11.1)). Examples of such patterns are consolidation, cup with handle, double bottom, saucer. The objective is to use observed features to predict the next stock price pattern graph. Several studies focused on learning traditional shapes defined by stock analysts. For instance, Liu et al. [2007] proposed a method based on different time window sizes and applied wavelet analysis for identifying and extracting chart patterns from denoised market data. Jeon et al. [2017] presented a methodology for finding the optimal historical dataset with similar patterns according to some algorithms for each stock. Then an ANN model is trained to predict the best stock price. They aggregated tick-by-tick data to transform non-continuous data into continuous one. A hierarchical clustering algorithm is used to sort out patterns and a stepwise regression to select the important features. Then an ANN was proposed to predict market prices. Zhang et al. [2018] proposed a stock price trend prediction system to predict both the price movement and its interval of growth (or decline) rate within predefined prediction intervals. A Random Forest was trained to classify clips of stocks into four classes (up, down, flat, and unknown) according to the shapes of their close prices. They outperformed existing prediction methods in terms of accuracy and return per trade.

It was argued that trend prediction (see Section (4)) is the prediction of stock price fluctuation patterns (see Teixeira et al. [2010]). Thus, by analysing the properties of stock price fluctuations, and being able of predicting them, we can forecast the stock price movement. Complex network analysis explain stock market behaviour from a systematic perspective. This can be achieved with a stock price volatility pattern network (SPVPN). For instance, Huang et al. [2010] used these networks to assess the properties of market prices on the Shanghai Stock Exchange (SSE) and predict their trends. The results showed that the networks followed the basic fluctuations exhibited during different periods. Wang et al. [2016] used these networks to study the properties of crude oil and gasoline in different periods. Using these properties they predicted oil and gasoline stock price patterns. However, most studies on SPVPN focused on univariate time series. Cao et al. [2019] proposed a pattern network construction method for multivariate stock time series.

Since estimating financial patterns is concerned with inferring some knowledge about the future trajectory of market returns, forecasting patterns is a multi-step prediction-learning problem (see Appendix (16.1)). Moreover, single-step prediction problems generated by a dynamical system are more naturally viewed as multi-step problems. Sutton

[1988] showed that temporal difference (TD) methods were more efficient than supervised learning methods when the data sequences were generated by a dynamical system. It is therefore logical to use TD methods rather than supervised learning when forecasting patterns. Lee [2001] proposed a reinforcement learning model by combining a $TD(0)$ algorithm with ANN to learn the values of states corresponding to a stock price trend at a given time. The state-values represent the expected cumulative reward. The performance of the method was evaluated on the Korean stock market. We modify the TD-procedures and propose some methods for predicting financial patterns and assessing their probabilities.

6.2 Predicting shapes

As discussed in Appendix (11.1), it is assumed that different chart patterns reveal different market trends. For example, a head-and-shoulder top pattern (see Figure (28)) will most likely rise before coming back to the same level. The main idea is to use a model to identify chart patterns on the historical time series of an underlying asset. These extracted patterns from market data are used as training data in a neural network. Once the model has learned these shapes, it can then be used to predict chart patterns. By successfully predicting chart patterns, a system can implement the right trading strategy and generate profit.

6.2.1 Some examples

6.2.1.1 Chart pattern prediction Liu et al. [2007] used different time window sizes, $W = \{w_1, \dots, w_m\}$, to locate the occurrence of a particular pattern, and applied wavelet multiresolution (MR) analysis and radial basis function networks (RBFN) for identifying market irregularities. The trading data contains a set of time data, $T = \{t_1, t_2, \dots, t_n\}$, which is divided into temporary subset T_j , for $j = 1, 2, \dots$. A set P is defined, where $P \subset T$, which contains the time ranges for previously identified wave patterns. Given a template of chart patterns, the trading data is analysed to see if it is a predefined pattern. Since the data is noisy, it can not directly be used for comparison. Both the input data and the template have been denoised with wavelet decomposition (see Appendix (10.2)). Given the window size w_i , wavelet analysis is applied to the subset T_j . The process goes from coarse to fine scales. If a predefined wave pattern is identified we add T_j to P , otherwise the process is repeated with finer scale. Eventually, all identified wave patterns are stored in P .

Pattern matching (recognition) was performed with a radial basis function network (RBFN) with p nodes in the hidden layer (since simple MR matching was not efficient). Different RBFNs were used to recognise different patterns at different resolution level. The input of the network is the wavelet transformed values in a particular resolution. The input layer consists of past network outputs (feedback effect) and the variables explaining the prediction problem. Given the past observation sequence $\{X_t, X_{t-1}, \dots, X_{t-n+1}\}$ of size n and the explanatory variables $\{V_{t,1}, \dots, V_{t,m}\}$, we want to predict the sequence $\{X_{t+1}, \dots, X_{t+k}\}$, with predictive step $k > 0$.

Since there was not enough patterns extracted from the HK stock market (only 64), the authors used the radial deformation technique to generate a total 308 patterns. A quarter of the training set was extracted as validation set. Once the appropriate settings were obtained, ten different stocks over a ten year period were tested for identification and extraction of the chart patterns. The RBFN was on average 81% accurate. However, the accuracy of the recognition process was heavily dependent on the resolution level. The problem becomes that of identifying the right resolution level.

Note, this test was performed on denoised data with high wavelet resolution. Further, once the chart patterns have been extracted, it remains to test for the prediction of chart patterns in real market environment, assess its exploitation by a trading strategy, and measure the potential return generated by a trading system.

6.2.1.2 Automated price predicting system Jeon et al. [2017] proposed an automated system for short-term prediction of stock prices. The daily stock prices are predicted. The system includes five steps:

1. Preprocessing step: The data from the KOSPI started on August 2014 and ended on July 2015. The tick-by-tick data is aggregated at five minute intervals to get continuous data.

2. Searching similar patterns: The length of a pattern is one day and patterns are generated at five minute intervals with a sliding window. There are twelve patterns in an hour. Similar patterns are found by comparing historical patterns with the current pattern. Find all similar patterns for one year using hierarchical clustering algorithm.
3. Feature selection: Given all features, use a stepwise regression to remove insignificant variables. It uses a bidirectional elimination approach. It repeatedly add and remove a variable among all the variables, and conduct regression analysis with the remainder. Use R-square as a selection measure.
4. Prediction model: Use ANN to generate the final prediction model. Five hidden layers were used. The output is a predicted data set.
5. Verify the validity of the model using RMSE as a measure of accuracy. The actual data was compared against two data values predicted by the model.

6.2.2 Automatic data processing system

Since stock markets carry out huge amount of transactions every day (for instance high frequency trading), unsupervised pattern recognition is becoming more important with increasing processing power of computers (see Wu et al. [2014]). Zhang et al. [2018] proposed an automatic data processing system to predict the trend of stock prices. It uses unsupervised pattern recognition methods to automatically generate training samples from raw transaction data. All patterns are extracted from the stock market and the interested ones are transformed into training samples. Multiple learning models are trained to meet the prediction goals. The trading frequency chosen is the day and the forecast duration range from 10 days to 60 days.

As discussed in Appendix ((11.1)), any stock chart may exhibit countless different pattern combinations. To simplify the search space, the authors focussed on four classes. The four classes considered are: Up (trend), Down (trend), Flat, Unknown (noisy mean-reverting). There are two subclasses for the Up/Down, the growth (decline) rate (changes of close prices) (1) within the range $[10, 30]\%$ and (2) greater than 30% . Note, relative return with respect to a benchmark is also considered. It leads to selecting the AbsoluteMode and the RelativeMode.

There are three duration periods:

1. The longest period is the Pattern Duration (PD) (clip): the expected pattern.
2. The Model Duration (MD): all data points are used as training sample. MD is $\frac{2}{3}$ of PD.
3. The smallest period is the Test Duration (TD): all data points form an input instance whose labels and their probabilities will be predicted.

Using a sliding window of predefined size moving with a one-day step, the raw data is divided into pieces (clips) which are analysed to match predefined patterns. It is based on the (simple) shape of the close prices over a fixed duration period. The shape of a clip determines the class label of its corresponding training sample. Heuristic algorithms based on geometry are used to recognise the defined patterns. The training samples are generated according to the recognised patterns. The method generates as many samples as possible to cover all possible cases. Then ML models (Random Forest) are trained on these samples.

Observing data at the start of the trading period, the prediction interface predicts the probability of forming a predefined shape over a fixed duration period. For a given stock, it provides the probability of going up or down with its associated probability of increase or decrease of the price. The user specifies the historical period under study as well as the prediction period, and the model provides the prediction results for every stock. Sorting these probabilities, the user will know the stocks having the highest probability of increase or decrease and generating the highest returns. In essence, the training sample is a subset of the clip, and the ML model is trained to learn the latter from the former. Thus, rather than learning S_t from S_{t-1} , the system learns a shape covering the data points $\{S_t, S_{t+1}, \dots, S_{t+d}\}$ from the sample $\{S_{t-k}, S_{t-k+1}, \dots, S_{t-1}\}$, $d, k \in \mathbb{N}$, for a clip of size $d + k + 1$.

In practice most clips belong to the class Unknown so that we get an imbalanced learning problem. To go around, the authors balanced the number of samples for all classes using the undersampling technique. The features of the training samples (close, open, high and low) were extended by adding multiple technical indices of each trade day. The feature selection method is based on the Forward Sequential Search method, which select one among all the candidates to the current state.

The system was tested on 495 stocks on the Chinese market from January 2010 till the end of September 2016. Assuming several fixed prediction durations, 70 learning models were considered on 70 data sets. The data was split in 30/70 predict / train. Excluding feature selection, the average accuracy of all models was 67.5%. The system performed better for predicting trend in long term duration in the range of 30 to 40 days. The performances were improved by adding technical indices in the set of features. The system can be refined by adding the subclasses in a two-stage prediction scheme. In that setting, the performances decreased to an average performance greater than 60%.

6.3 Predicting stock price fluctuations

Since the behaviour of hidden short-term irregularities is non-linear, researchers developed neural networks and other wavelet frameworks to capture these irregularities (or fluctuations) to forecast market directions. Some studies on predicting stock price fluctuations focused on assessing volatility patterns of single stocks (see Moghaddam et al. [2016]). However, Kwon et al. [2005] showed that price fluctuations in a single stock were not isolated, but were influenced by the trends of multiple related stocks. Complex network analysis explain stock market behaviour from a systematic perspective. Several studies proposed different methods for building complex networks from time series, such as visibility graphs, recurrence networks (see Marwan et al. [2009], Donner et al. [2010]), correlation networks, pattern networks (see Wang et al. [2016], [2017]), and K-neighbours networks. The symbolic pattern networks reflect well the degree of correlation and direction of primitive elements in a complex system. In a stock price volatility pattern network (SPVPN), each volatility pattern is regarded as a network node, and the relationship between patterns is regarded as a connection between nodes. One infer the properties of stock price fluctuations from the topological properties of the network. Most studies on SPVPN focused on univariate time series.

6.3.1 Complex network methods

Cao et al. [2019] considered complex network methods to analyse the stock price fluctuation patterns of the main US indices. They proposed a pattern network construction method for multivariate stock time series. The volatility patterns analysed (every thirty days) are: the average strength, average shortest path length, average degree centrality, and closeness centrality. First, the price volatility patterns of these indices were combined using a coarse-grained method. Then, these price volatility combination patterns of the main US indices were used as nodes of the network, and the frequencies and directions of the conversion of the patterns were used as the weights and directions of the network connections. Then, the directed weighted networks were constructed. The result showed that stock price volatility pattern networks (SPVPN) were reflecting special periods (of sharp fluctuations) on the stock market, and could be used to identify such periods. We will see in Section (7) that it makes more sense to use the Hurst exponent to reveal the properties of stock price fluctuations. Thus, these variables (SPVPN) for each combination pattern are used as input variables for K-nearest neighbours (KNN) and SVM algorithms to predict the next day volatility patterns of a single stock.

The authors used daily closing prices for each index and computed one-day return, five-day return, and five-day volatility V which they rolled on a sliding window with length thirty days and a one-day time step. This way, we get a pattern network every thirty days. They computed the average five-day volatility and obtained the average volatility for the overall market V' . From the sign of the five-day return and its volatility they classified each stock into four patterns S_i , $i = 1, 2, 3, 4$, based on the relation between V and V' . The patterns of each stock index were combined. Assuming three indices, we get 64 combination modes. The daily combination patterns are the nodes of the networks and the edges and weights are determined in time order. Then, the authors computed the characteristic variables for every network:

- the average degree centrality $\rho = \frac{1}{N^2} \sum_{i,j}^N a_{ij}$ where N is the number of nodes and a_{ij} is the value of the adjacency matrix of an undirected network,
- the average strength $S = \frac{1}{N} \sum_{i,j}^N w_{ij}$ where w_{ij} is the directed edge from node i to node j ,
- the average shortest path length $L = \frac{1}{N(N-1)} \sum_{i,j}^N d_{ij}$ where d_{ij} is the distance from node i to node j ,
- the closeness centrality $\frac{1}{N} \sum_{i=1}^N \frac{n-1}{N-1} \frac{n-1}{\sum_{j=1}^{n-1} l_{ij}}$ where l_{ij} is the weighted shortest path and n is the number of nodes reachable from node i .

These normalised variables are learned by two ML models: the KNN and SVM. The authors first train the models and identify the best model by performing cross-validation. Then we can perform the next-day pattern prediction for each stock index. They reported an accuracy greater than 70% for predicting the next day price patterns. They predicted stock trends by using a classification algorithm and combining the characteristics of the multistock price network.

6.4 Using TD to predict patterns

We discussed in Section (3) how to perform a one-point forecast to estimate the level of return, and we presented in Section (4) different ways of estimating the next direction of market returns. However, in order to devise trading strategies, traders are more interested in estimating financial patterns (see Appendix (11.1)). That is, inferring some knowledge about the future trajectory of market returns. Forecasting patterns is a multi-step prediction-learning problem (see Appendix (16.1)). Moreover, single-step prediction problems generated by a dynamical system are more naturally viewed as multi-step problems. Sutton [1988] showed that temporal difference (TD) methods were more efficient than supervised learning methods when the data sequences were generated by a dynamical system. It is therefore logical to use TD methods rather than supervised learning when forecasting patterns. Lee [2001] proposed a reinforcement learning model by combining a $TD(0)$ algorithm with ANN to learn the values of states corresponding to a stock price trend at a given time. The state-values represent the expected cumulative reward. The performance of the method was evaluated on the Korean stock market.

We briefly discuss the approach proposed by Lee [2001] and, following Sutton [1988], propose some methods for predicting financial patterns and assessing their probabilities.

6.4.1 Notation

6.4.1.1 Returns We are using notation in Appendix (11.2) and let C_t be the closing price at time t , O_t is the opening price, L_t is the low price, H_t the high price, and V_t is the volume of the trade at time t . The immediate reward at time t is given by

$$r_t = 100 \cdot \frac{C_t - C_{t-1}}{C_{t-1}}$$

These market returns can be modelled as follows: We let $\{r_t; t \in T\}$ be a real-valued random process for the market return, and $\{t_1, \dots, t_n\} \subset T$, where $t_1 < t_2 < \dots < t_n$. Further, we assume that $\{r_t\}$ is either serially uncorrelated or with minor lower order serial correlations, but it is dependent. We assume that r_t follows the dynamics

$$r_t = \mu_t + a_t \tag{6.2}$$

where a_t is the (stochastic) shock or mean-corrected return (or innovation) of an asset return². The conditional mean and conditional variance of r_t , given the filtration $\mathcal{F}_{t-1}$, are defined by

²since $a_t = X_t - \mu_t$

$$\begin{aligned}\mu_t &= E[r_t | \mathcal{F}_{t-1}] = G(\mathcal{F}_{t-1}) \\ \sigma_t^2 &= \text{Var}(r_t | \mathcal{F}_{t-1}) = E[(r_t - \mu_t)^2 | \mathcal{F}_{t-1}] = H(\mathcal{F}_{t-1})\end{aligned}$$

where G and H are well defined functions with $H(\cdot) > 0$. The model for μ_t is the mean equation for r_t , and the model for σ_t^2 is the volatility equation for r_t . Both the functions G and H could be non-linear. For simplicity of exposition we first consider the linear case.

We standardise the return by subtracting its expected value $E[r]$ and dividing the difference by its standard deviation $\sigma(r)$, getting

$$\hat{r} = \frac{r - E[r]}{\sigma(r)}$$

In our case, we subtract the population mean μ_r from an individual return value r_i and then divide the difference by the population standard deviation σ_r . That is,

$$\hat{r} = \frac{r_i - \mu_r}{\sigma_r}$$

6.4.1.2 TD procedures We let the data sequences be generated by absorbing Markov processes, where the next state depends only on the current state. They are defined by a set of terminal states T , a set of nonterminal states N , and a set of transition probabilities p_{ij} , $i \in N, j \in N \cup T$, where each p_{ij} is the probability of a transition from state i to state j , given that the process is in state i . Since it is absorbing, all sequences eventually terminate. Given an initial state q_1 , the absorbing Markov process generates the sequence $q_1, q_2, \dots, q_{m+1}$, where $q_{m+1} \in T$. The learning algorithms do not know about the state sequence, but only of a related observation-outcome sequence $x_1, x_2, \dots, x_m, z$. We follow the notation in Appendix (16.2) and consider a multi-step problem with experience coming in observation-outcome sequences of the form $e = (x_1, x_2, \dots, x_m, z)$, where each x_t is a vector of observable input at time t in the sequence, and $z \in \mathbb{R}$ is the outcome of that sequence. A training set consists of several sequences $\{x_{1,p}, x_{2,p}, \dots, x_{m,p}, z_p\}$, for $p \in [1, P]$. For each such sequence, the learner produces a sequence of predictions $P_1, P_2, \dots, P_m$, each of which being an estimate of z . For simplicity, it is assumed that P_t is a function of x_t and a vector of weights w , that is, $P_t = P(x_t, w)$. The $TD(\lambda)$ procedure in Equation (16.99) is used to update the weights. Following Sutton, we can use a linear prediction $P_t = w \cdot x_t$, or, a much complex supervised learning such as a deep neural network. Since the relationship between input and output in stock prices is highly nonlinear, we should consider a multilayer ANN to represent the prediction $P_t = P(x_t, w)$. As discussed in Appendix (16), the update equation is not changed but the gradient term $\nabla_w P_t(x_t, w)$ is more complex as it is computed via backpropagation. Thus, in the rest of this section P_t can be a linear function or a multilayer ANN.

6.4.2 Stock price prediction

Lee [2001] proposed a reinforcement learning model by combining a $TD(0)$ algorithm with ANN to learn the values of states corresponding to a stock price trend at a given time. The state-values represent the expected cumulative reward. Note, while the author discussed reinforcement learning for predicting stock prices, only the evaluation part is considered, not the improvement part to obtain the optimal policy. As a result, the author is only considering TD-methods for predicting financial patterns.

We let $\mathcal{S}$ be the continuous state space and $\mathcal{A}$ be the action space. At each time step t , we denote s_t the current state, and a_t the action chosen by the agent. The reward between times t and $t + 1$ is $r_{t+1} = r(s_t, a_t)$, and the succeeding state is $s_{t+1} = \delta(s_t, a_t)$. The task of the agent is to learn a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ for selecting the next action based on the observed state, that is, $\pi(s_t) = a_t$. The value-function is given by

$$V_{\pi}(s_t) = \sum_{k=0}^{\infty} \gamma^k r_{t+1+k}$$

where γ , $0 < \gamma < 1$, is the discount factor. By varying the discount factor we can consider short or long-term market return predictions. The optimal policy π^* is given by

$$\pi^* = \arg \max_{\pi} V_{\pi}(s), \forall s$$

which is solved by iteration policy (IP). It is a two step process consisting of the evaluation of the value-function and the improvement of the policy.

The author considered the $TD(0)$ -algorithm to evolve the value function, getting

$$V(s_t) \leftarrow V(s_t) + \alpha \sigma_t$$

where the TD error is

$$\sigma_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

and the target value is $r_{t+1} + \gamma V(s_{t+1})$.

The state of the system is a vector with a fixed number of real-valued components corresponding to a raw daily data or a derived data. It is denoted by

$$s = \phi_s = (\phi_{s,1}, \dots, \phi_{s,n})$$

The value-function is represented by a functional form $V(s, a; \theta) \approx V^*(s, a)$, which is a V-network where θ is the function parameters. The update rule is

$$\theta_{t+1} \leftarrow \theta_t + \Delta \theta_t$$

where

$$\Delta \theta_t = \alpha \sigma_t \nabla_{\theta_t} V_t(s_t)$$

If we follow Sutton [1988], the value function is a linear function given by $V_t(s_t) = \theta_t \cdot \phi_{s_t} = \sum_{i=1}^n \theta_{i,t} \phi_{i,s_t}$, and the gradient simplifies to $\nabla_{\theta_t} V_t(s_t) = \phi_{s_t}$. However, since the relationship between input and output in stock prices is highly nonlinear, the author considered a multilayer ANN to represent the value-function. As discussed in Appendix (16), the update equation is not changed but the gradient term $\nabla_{\theta_t} V_t(s_t)$ is more complex as it is computed via backpropagation.

The method was tested on data from the Korean stock market, with a training period of two years and a testing period of one year. The training was done on-line. Returns were classified into 16 distinct grades, and the average returns are truncated after 1, 5, 10 and 20 days. The root mean squared (RMS) for 1 and 20 days were not as good as the ones for the other days.

6.4.3 Predicting trends

A simple way of using TD methods to predict financial patterns is to follow the example on bounded random walk given in Appendix (16.2.3). The walk is a symmetric process bounded above and below by a barrier level, b_{up} and b_{down} , respectively. The aim is to find the conditional probability of a walk reaching a barrier level, given that it was in each of the other states.

We assume that the market return in Equation (6.2) is composed of a deterministic drift and some noise, a random walk. We are interested in assessing simple upward trends in market returns. We consider the sequences e defined above where the vector x_t contains the standardised market return $\hat{r}$ at time t , and the outcome $z = b_{up}$ is an up barrier

level which is above the initial return value. We want to find the conditional expectation of return reaching the up barrier, given that it was in each of the other states. That is, $E[z | x_k]$, for $k = 1, \dots, m$. The outcome is $z = 0$ for the return ending near b_{down} and $z = 1$ if it ends near b_{up} . The neighbourhood of b_{down} and b_{up} is defined below. Thus, we are actually finding the conditional probability of return reaching the up barrier, given that it was in each of the other states. In our setting, the noise is $\epsilon \sim N(0, 1)$, so that we can choose b_{up} and b_{down} as $\pm x\% \cdot 1$, respectively, for a given percentage. The choice of that percentage will depend on the frequency of trading and the length of the pattern one desire to estimate.

Since market return is a continuous process, we discretise space between $[b_{down}, b_{up}]$ into N steps as follows $\Delta = \frac{b_{up} - b_{down}}{N}$, where $b_{down} + N\Delta = b_{up}$. We are in state i if the standardised continuous return $\hat{r}$ is in the range $[b_{down} + (i - \frac{1}{2})\Delta, b_{down} + (i + \frac{1}{2})\Delta)$, for $i = 1, \dots, N - 1$. If $\hat{r}$ is in the range $[b_{down}, b_{down} + \frac{1}{2}\Delta)$ we have reach b_{down} , and if $\hat{r}$ is in the range $[b_{down} + (N - \frac{1}{2})\Delta, b_{down} + N\Delta]$ we have reach b_{up} . For each nonterminal state i , we associate an observation vector X_i of size $N - 1$. If the return is in state i at time t , then $x_t = X_i$. For example $X_3 = (0, 0, 1, 0, \dots, 0)^\top$. We follow the repeat presentations training paradigm, where each training set is presented repeatedly to each learning procedure until the latter no-longer produce any significant changes in the weight vector. Once we have learnt to predict the conditional probabilities for reaching the up barrier we can assess the market for upward trends and devise a trading strategy accordingly. We can do the same for a down barrier b_{down} and assess the market for downward trends. This way, we have a tool for predicting upward and downward trends.

Similarly to the example in Appendix (16.2.3), we can use Monte Carlo simulations to generate sequences of returns and train 100 sets, each consisting of 10 sequences, and use the root mean square (RMS) error as a measure of performance.

Without proof of convergence, we can repeat the method on simulated market returns in Equation (6.2) with more complex functions G and H . We can also try it on real market returns. One possible strategy in the case of upward trend is as follows:

- Choose a frequency of trading and the length of the pattern one desire to estimate.
- Compute the conditional probabilities $P_{model}(x_k, w)$, $k = 1, \dots, m$, of reaching the up barrier for a random walk.
- Learn to predict the conditional probabilities of reaching the up barrier for real market returns.
- Move along a sequence and, at time t , compute the prediction $P_{market}(x_t, w)$.
- If $P_{market}(x_t, w) > P_{model}(x_t, w)$, we are capturing some upward trend. The market Hurst coefficient is larger than $\frac{1}{2}$. Depending on the value $Diff(t) = P_{market}(x_t, w) - P_{model}(x_t, w)$, we can get more or less aggressive in getting long the market. Our strategy at time t can also depends on $P_{t-1}, P_{t-2}, \dots$
- If $Diff(t) \approx 0$, we can choose not to trade.

The same can be done for downward trend.

6.4.4 Predicting values and inferring patterns

More generally, we consider the sequences e defined above where the vector x_t contains the standardised market return $\hat{r}$ at time t , and the outcome $z \in \mathbb{R}$ is a return value at future time $T > m$. We want to find the conditional expectation of return being equal to z , given that it was in each of the other states. That is, $E[z | x_k]$, for $k = 1, \dots, m$. For each such sequence, the learner produces a sequence of predictions $P_1, P_2, \dots, P_m$, each of which being an estimate of z . These predictions are the forward price of market returns for the fixed maturity T . As we move in time along the sequence, we refine our prediction of z based on the path taken by market returns. When at time t of the sequence, we have already predicted $P_{t-1}, P_{t-2}, \dots$. It gives us some information about the market pattern we are in.

As discussed in Section (6.2), different chart patterns reveal different market trends. The main idea is to use a model to identify chart patterns on the historical time series of an underlying asset and use these patterns as training data in a

neural network. Once the model has learned these shapes, it can then be used to predict chart patterns. That is, rather than learning S_t from S_{t-1} , the system learns a shape covering the data points $\{S_t, S_{t+1}, \dots, S_{t+d}\}$ from the sample $\{S_{t-k}, S_{t-k+1}, \dots, S_{t-1}\}$, $d, k \in \mathbb{N}$, for a clip of size $d + k + 1$. By successfully predicting chart patterns, a system can implement the right trading strategy and generate profit.

Rather than using neural networks to learn some defined patterns, this problem can be solved much more efficiently with TD procedures. We consider the training set consisting of sequences of the form $\{x_1, x_2, \dots, x_m, z_p\}$, for $p \in [1, P]$. We assume that z_1 is $\mathcal{F}_{T_1}$ -measurable, z_2 is $\mathcal{F}_{T_2}$ -measurable, and so on, with $m < T_1 < T_2 < \dots < T_P$. For each such sequence, the learner produces a sequence of predictions $P_{1,p}, P_{2,p}, \dots, P_{m,p}$, each of which being an estimate of z_p . Thus, we get all the conditional expectation $E[z_p | x_k]$, $k = 1, \dots, m$, $p = 1, \dots, P$. This way, the system learns a shape covering the data points $\{z_1, \dots, z_P\}$.

One possible strategy is as follows:

- Choose a frequency of trading and the length of the pattern one desire to estimate.
- Compute the conditional expectations $P_{k,p} = P(x_k, w)$, $k = 1, \dots, m$, $p = 1, \dots, P$ of reaching the outcomes z_p , $p = 1, \dots, P$.
- Learn to predict the conditional expectations of reaching the outcomes z_p , $p = 1, \dots, P$.
- Move along a sequence and, at time t , compute the predictions $P_{t,p} = P(x_t, w)$, $p = 1, \dots, P$.
- If $P_{t,p} = P(x_t, w)$ form a recognisable pattern we can get more or less aggressive in implement the right trading strategy. Our strategy at time t can also depends on $P_{t-1}, P_{t-2}, \dots$.
- If $P_{t,p}$, $p = 1, \dots, P$, do not form a pattern we can choose not to trade.

6.4.5 Predicting cumulative returns

We saw in Section (5) that when forecasting directions, the data should be denoised and the trading decisions should be combined with some criteria on the generated returns. Further, we have discussed how to predict financial patterns. We are now going to show how to use TD procedures to infer information about the magnitude of future market returns.

6.4.5.1 Description As discussed in Appendix (16.2.6), in addition to predicting the final outcome z of a sequence, we can also use TD procedures to predict the cumulative outcomes. We assume that each step of a sequence incur a cost, and want to predict, at time t , the expected total cost of the remaining sequence. In finance, we are interested in predicting the cumulative returns of a sequence to define proper trading strategies. Here, the costs are replaced by returns, which become known as the sequence evolves. We use the observation vector received at each step to predict the total cumulative return after that step. We want P_t to predict the remaining cumulative return given the t -th observation. We use all the equations in Appendix (16.2.6) with c_{t+1} replaced by r_{t+1} , the return incurred between times t and $t + 1$.

We want P_t to equal the expected value of $z_t = \sum_{k=t}^m r_{k+1}$, where m is the number of observation vectors in the sequence. The prediction error can be expressed in terms of temporal differences as

$$z_t - P_t = \sum_{k=t}^m r_{k+1} - P_t = \sum_{k=t}^m \sigma_k$$

where

$$\sigma_k = r_{k+1} + P_{k+1} - P_k$$

and $P_{m+1} = 0$. Following the steps to derive the $TD(\lambda)$ family of procedures in Appendix (16.2.2), the learning rule for the cumulative $TD(\lambda)$ family is given by

$$\Delta w_t = \alpha \sigma_t \sum_{k=1}^t \lambda^{t-k} \nabla_w P_k$$

When P_k is a linear function, this is the Adaptive Heuristic Critic model introduced by Sutton [1984], with costs replaced by returns.

One possible strategy is as follows:

- Choose the length of the pattern one desire to estimate.
- Compute the estimated cumulative returns of the sequence $P(x_k, w)$, $k = 1, \dots, m$.
- Learn to predict the estimated cumulative returns.
- Move along a sequence and, if $P(x_t, w) > \epsilon$, where ϵ is a threshold, we are capturing some upward trend. Depending on the magnitude of $P(x_t, w)$, we can get more or less aggressive in getting long the market. Our strategy at time t can also depends on previous predictions $P_{t-1}, P_{t-2}, \dots$
- If $P(x_t, w) < \epsilon$, we can choose not to trade.

6.4.5.2 Combining cumulative returns and probabilities We can combine a model that estimates the probabilities of reaching an up barrier with a model that estimate cumulative returns. We let $P(x_t, w_b)$ be the estimated probability of reaching the barrier b_{up} , and $P(x_t, w_r)$ is the prediction of the cumulative returns. One possible strategy is as follows:

- Choose the length of the pattern one desire to estimate.
- Compute the conditional probabilities $P_{model}(x_k, w_b)$, $k = 1, \dots, m$, of reaching the up barrier for a random walk.
- Learn to predict the conditional probabilities of reaching the up barrier for real market returns.
- Move along a sequence and, at time t , compute the prediction $P_{market}(x_t, w_b)$.
- If $P_{market}(x_t, w_b) > P_{model}(x_t, w_b)$, we are capturing some upward trend. Depending on the value $Diff(t) = P_{market}(x_t, w_b) - P_{model}(x_t, w_b)$, we can get some indication on the likelihood of the trend.
- Compute the estimated cumulative returns of the sequence $P(x_k, w_r)$, $k = 1, \dots, m$.
- Learn to predict the estimated cumulative returns.
- Move along a sequence and, if $P(x_t, w_r) > \epsilon$, where ϵ is a threshold, we are capturing some upward trend. We get some indication on the magnitude of future returns.
- Depending on the the likelihood of the trend and the magnitude of future returns, we can get more or less aggressive in getting long the market.
- If $Diff(t) \approx 0$ and $P(x_t, w_r) < \epsilon$, we choose not to trade.

7 The recipe

7.1 Defining the framework

7.1.1 Problems and solutions

We have seen in Section (2.2.3) that neural networks (NNs) were treated as black box models due to their inability to explicitly know the relations established between explanatory variables (input) and dependent variables (output). Given the increasing processing power of computers, we saw in Section (4) that academics are now trying all kind of explanatory variables, combining them in a very large input set, hoping that ML networks will identify some relationships with the process to be forecasted. The paradigm has been shifted from identifying the best statistical model to identifying the best input variables, considering the ML model as a black box which will always learn the relationship $Y = f(X)$ between the input and output. In addition, when the forecasting results are not satisfactory, the data scientists will occasionally change the black box $f(\cdot)$ and repeat the tests.

While the discovery of interactions has become an essential part of the literature on neural networks, their interpretation is far more difficult. Some statistical methods have been proposed to solve the problem, but they rely on the existence of an optimal solution for the network and assume stationarity of the signal response.

Nowadays, the discovery of interactions and their interpretation is especially useful for scientific discoveries and hypothesis validation. For instance, doctors may want to know what interactions are accounted for in risk prediction models, in order to compare against known interactions from existing medical literature. In finance, we are interested in knowing the statistical properties of the time series produced by the ML model under consideration. This is because we need to make sure that the chosen ML model is capable of reproducing the characteristics of the desired series.

The results in Sections (3) on forecasting market returns (case [1+]) suggest that a single RNN does not have the capacity of capturing the information inherent to neither noisy chaotic series nor to financial data submitted during the training (and testing) process. These conclusions can be explained by the fact that financial time series are multifractal, thus exhibiting non-Gaussian distribution, the presence of extreme values (outliers), and long-range dependent dynamics (see Section (2.1)). Since financial markets can follow different behaviours over time, financial series should not be modelled by a single process, but preferably by a succession of different processes. For instance, Dablemont et al. [2003] assumed that a single model could not capture the dynamics of the whole series and splitted the historical series into clusters. They generated a specific local neural model for each of clusters, which were combined in a probabilistic way according to the distribution of the series in the past. Further, several studies showed that combining forecasts (using NN models) could greatly enhance the forecast effectiveness (see Wang et al. [2008], Chitra et al. [2010]).

As discussed in Section (4), some academics showed that technical indicators with their associated trading rules have predictive power and found a good measure of support for their effectiveness (see Appendix (11.7)). As a result, some authors considered forecasting future trend direction (case [2+]) using classification networks (See Section (4.2)). The results obtained are achieved using only historic prices and technical indicators with their associated trading rules. Recently, some authors designed trading strategies to use in association with each trading rule. Accounting for risk and transaction costs, these strategies proved very profitable, beating the Buy and Hold strategy. Consequently, we should not only consider trading rules but go further and incorporate trading strategies in the learning process. To this end, we proposed the Meta Buy/Sell Signal (MBSS) in Section (5.1).

As discussed in Section (6), even though stock prices are multifractal, they also consist of several patterns which appear repeatedly at time intervals. As a result, some authors proposed to use observed features to predict the next stock price pattern graph. Recognising that there is no single time scale that works for all analytical purposes, Liu et al. [2007] proposed a method based on different time window sizes and applied wavelet analysis for identifying and extracting chart patterns from denoised market data. Different radial basis function networks (RBFNs) were used to recognise the different patterns at different resolution level. Zhang et al. [2018] considered hierarchical clustering algorithms to sort out (classify) patterns and a stepwise regression to select the important features. Then an ML model

was trained to predict market prices. Other authors, such as Cao et al. [2019], proposed to analyse the properties of stock price fluctuations and tried to predict them in view of forecasting the stock price movements. They considered complex network analysis to assess the properties of market prices and predict their trends.

Rather than fitting an ML model to some external data and figuring out its statistical properties through some statistical interpretation methods, we inverse the causal relation by first generating large data from a priori theoretical model and then fitting an ML model to this data. This way we know the statistical properties of our ML model. We can then train a series of ML models to reproduce a set of theoretical models with distinct statistical characteristics and combine the trained (calibrated) models with ensemble methods.

As discussed in Section (2.2), in the case where real-world time series are characterised by multiple seasonalities, solutions were developed to train a multiple superimposed oscillator (MSO). Bandpass filters decoupling the signal in the frequency space were proposed, as well as networks made of multiple reservoirs trained to separately adapt to each timescale of the input signal. In each case, the parameters for adapting the network to the right time scales require analysis in the frequency domain or a priori knowledge of the input data.

While frequency analysis can be applied to decompose periodic time series, multifractal analysis must be used to decompose market returns (see Appendix (10)). Motivated by the recent research on MSOs, we propose to use multifractal analysis to decompose the target signal with the aim of capturing its statistical properties. We build a general framework and define a meta-model with the desired statistical properties. We then train the model to recognise patterns, or technical indicators, and devise a trading algorithm.

7.1.2 The framework

We propose to decide upon the framework by defining how the model should be specified before beginning to analyse the actual data. We define the framework by properly formulating model hypotheses which make financial or economic sense, and then carefully determining the number of dependent variables in a regression model, or the number of factors and components in a stochastic model.

We consider the multifractal formalism (MF) which provides a scale invariant mechanism for the analysis and generation of complex signals that fit well with the observed experimental properties of fully developed turbulence (FDT) as well as other physical systems ranging from natural images to heartbeat dynamics or econometric signals. In addition, the formalism also allows to highlight relevant dynamical features of the systems under study, and theoretical models can be devised to fit the observed multifractal properties.

As discussed in Section (5.3), rather than using a single model (or blindly stacking models³) to fit a small sample of market returns, and risk overfitting the data, we want to find models generalising potential future data. Further, we know that neural networks with fixed weights have great difficulty accounting for changing environment. We must therefore construct a framework where we can generate unlimited data from a priori distributions, formulate an hypothesis, build a model and test its adaptive ability to changing environments. In the case of poor performance, we would modify the hypothesis or formulate a new one, resulting in a modified (or new) model until a level of acceptance is reached.

Based on these observations, we use the multifractal formalism to devise a recipe for quantitative trading with machine learning which is independent from the choice of a model. Similarly to physics, we can use the multifractal formalism (MF) as a framework for testing the capacity of an ML model to reproduce some statistical properties of time series. To do so, we consider a set $\mathcal{T}$ of theoretical models $\mathcal{T}_m$, $m = 1, \dots, M$, having non-overlapping pre-defined statistical properties. We also consider a set $\mathcal{M}$ of ML models $\mathcal{M}_m$, $m = 1, \dots, M$, each one of them capable of reproducing one theoretical model from the set $\mathcal{T}$. We train each ML model $\mathcal{M}_m$ to approximate its associated theoretical model $\mathcal{T}_m$ (calibration), and use ensemble methods to combine the calibrated ML models $\mathcal{M}_{calib,m}$, obtaining a meta-model.

³In general, stacking produces small gains with a lot of added complexity. The difficulty being to devise the combining function.

However, the weights of the latter are statistically computed as a function of the Hurst exponent. As a result, the meta-model is dynamically recombined based on the changing properties of the time series over time.

Once we have identified an ensemble of ML models with specific non-overlapping statistical properties, we want to use the algorithm described Section (5) to forecast both the market returns and their directions. We still need to fit the external input signal and possibly learn some patterns, or, technical indicators with their associated trading rules. We consider a meta-set where the market returns are the dependent variables, while the continuous-valued technical indicators with single technical trading rules as well as complex trading rules are the independent variables. Since we can associate different trading strategies to each trading rules, we use the Meta Buy/Sell Signal (MBSS) presented in Section (5.1) to identify the best trading strategy under the constraints of risk and transaction costs.

Recall from Section (2), a reservoir can always be split into sub-reservoirs so that different tasks can be learned. Thus, we leave a number of variable neurons, or free sub-reservoirs $\mathcal{R}_{free}$, dedicated to fitting the external input signal (financial time series) as well as learning a large number of patterns or technical indicators (see Section (4.2)).

7.2 The algorithm

We are now going to build our ML meta-model to satisfies certain pre-defined statistical properties taken from the multifractal formalism and, following the principles in Result (1), we are going to devise our trading algorithm. For simplicity of exposition, we let the set of theoretical models $\mathcal{T}$ be generated from fractional Brownian motion (fBm) with successive Hurst exponent. In practice, they can be generated from any fractal or multifractal theoretical model. Further, we consider a time-dependent Hurst exponent, but we can also consider a time and scale Hurst exponent (see Appendix (10.5)).

7.2.1 Defining an ensemble ML models

Inspired by the ensemble methods (see Definition (13.1)), our approach consists in devising several ML models capable of reproducing some pre-defined statistical properties (such as the Hurst exponent H), training them, and recombining them. We consider M models $\mathcal{M}_1, \dots, \mathcal{M}_M$ with corresponding predictions $\hat{f}_1(x), \dots, \hat{f}_M(x)$ with instance x . For each model $\mathcal{M}_m$, $m = 1, \dots, M$, we define its associated band of Hurst exponent $[h_{m-1}, h_m]$ where $h_0 = 0 < h_1 < \dots < h_M = 1$ and set $\Delta_m = h_m - h_{m-1}$. We let $H_{mid,m} = \frac{\Delta_m}{2}$ be the mid Hurst exponent of the band value.

The method is as follows:

1. For each $m = 1, \dots, M$, generate a large set of data from a theoretical model $\mathcal{T}_m$ in the set $\mathcal{T}$ having some pre-defined statistical properties (using the multifractal formalism). For instance, the data can be generated from a fractional Brownian motion(fBm) with $H_{mid,m}$.
2. Devise an ML model $\mathcal{M}_m$, $m = 1, \dots, M$, capable of reproducing this theoretical model $\mathcal{T}_m$.
3. Train the model to get optimum network.
4. Repeat steps (1 – 3), obtaining an ensemble of M ML models, each of them having non-overlapping statistical properties.

While there are several choices for building the ML model in step (2) above (see Section (2)), we will start by considering

- Reservoir computing (RC) to characterise transient processes
- Associative reservoir computing (ARC) to characterise persistent processes

Other choices are possible, as long as the models can reproduce transient and persistent processes.

We apply model stacking and combine the models to obtain a meta-model. While we could solve the minimisation problem in Equation (13.51) to find the weights of the meta-model, we feel that it would result in over-fitting the data. Alternatively, we are going to statistically compute the weights by making them a function of the Hurst exponent, $\omega_m = \varphi(H)$. The prediction of the meta-model is (see Equation (13.52)) defined by

$$\hat{f}_{avecv}(x) = \sum_{m=1}^M \omega_m \hat{f}_m^{(cv)}(x)$$

One solution for specifying the weight ω_m is given by

$$\omega_m = \begin{cases} 1 & \text{if } H \in [h_{m-1}, h_m] \\ 0 & \text{otherwise} \end{cases}$$

where $[h_{m-1}, h_m]$ is the band of Hurst exponent for model $\mathcal{M}_m$. This is the indicatrice function I_A for the event $A = H \in [h_{m-1}, h_m]$. To handle events around the boundary of the band of Hurst exponent, we could also use a hat function. For instance, given the hat function centred around x_j and taking values in $[x_{j-1}, x_{j+1}]$,

$$\Lambda(x) = \begin{cases} \frac{(x-x_{j-1})}{(x_j-x_{j-1})} & \text{if } x_{j-1} \leq x < x_j \\ \frac{(x_{j+1}-x)}{(x_{j+1}-x_j)} & \text{if } x_j \leq x < x_{j+1} \\ 0 & \text{otherwise} \end{cases}$$

we can set $x_{j-1} = h_{m-1}$, $x_{j+1} = h_m$ and $x_j = H_{mid,m}$ so that the hat function is centred around $H_{mid,m}$. Then,

$$\omega_m = \Lambda(H) \text{ and } \begin{cases} \omega_{m-1} = 1 - \omega_m & \text{if } m > 1 \text{ and } h_{m-1} \leq H < H_{mid,m} \\ \omega_{m+1} = 1 - \omega_m & \text{if } m < M \text{ and } H_{mid,m} \leq H < h_m \end{cases}$$

Note, more sophisticated function $\varphi(\cdot)$ can be used.

7.2.2 The learning algorithm

In the first part of the recipe we have identified an ensemble of ML models with specific non-overlapping statistical properties. We are still left with some free sub-reservoirs to fit the external input signal and learn a large number of patterns or technical indicators. The former is the basis of all learning algorithm and we refer the readers to Appendix (17) for more details in our chosen model.

We are now going to explain how to train the remaining sub-reservoirs to learn a large number of patterns or technical indicators (see Appendix (11)). We follow the predictive model described in Section (5.1), which is made of two layers, namely the Feature Engineering layer and the Prediction layer. First, using the generated data, we derive a set of simple and complex trading rules for each of the technical indicator employed, resulting in the discrete values 1, -1, or 0, representing buy, sell or hold signals, respectively. These signals serve as inputs to the second layer, which employs the neural network for feature learning and classification.

We let $\mathcal{R}_{free,m}$ be a set of free reservoir for the m -th calibrated ML model. The meta-model is going to be combined on the fly as new external input reveals. Prior to this, we need to teach each set $\mathcal{R}_{free,m}$ to recognise patterns or technical indicators.

The method is as follows:

1. Pick a calibrated ML model $\mathcal{M}_{calib,m}$, $m = 1, \dots, M$, and associated set $\mathcal{R}_{free,m}$, from our ensemble of models.
2. Teach the model (free sub-reservoirs) to identify:
 - (a) either autonomous patterns corresponding to specific strategies (discrete technical indicators), or
 - (b) technical indicators (continuous technical indicators).

3. Repeat steps (1 – 2) with other calibrated models from the ensemble.

At the end of the learning process we have an ensemble of calibrated ML models $\mathcal{M}_{calib,m}$, $m = 1, \dots, m$, with non-overlapping statistical properties and associated sub-reservoirs $\mathcal{R}_{calib,m}$, each of them capable recognising patterns or technical indicators. We are left with fitting the remaining free reservoir to external input data. At last, as time evolves, we statistically estimate the Hurst exponent of the desired time series and use it to combine our meta-model. The latter is used in the trading strategy of our choice.

7.2.3 The trading algorithm

Once we have an ensemble of calibrated ML models capable of recognising a large number of patterns, or forecasting technical indicators, we devise a trading algorithm with strategies accounting for a specific level of Hurst exponent. We can then validate and exploit our approach with real market data.

We let the coefficient H be a function of time and denote H_d the Hurst exponent computed at discrete time t_d . The trading algorithm is as follows:

1. At time t_d , statistically analyse the market returns, identifying the Hurst coefficient H_d , or time-space Hurst exponents. Build the meta-model by computing the weights ω_m as described above. Then proceed as follows:
 - (a) if $H_d \leq \frac{1}{5}$, then:
 - either do not trade, or
 - forecast the asset directions using technical indicators.
 - predict market patterns using pattern recognition.
 - (b) if $\frac{1}{5} < H_d < \frac{3}{5}$, forecast both market returns and their directions.
 - (c) if $H_d > \frac{3}{5}$, then forecast directly the market returns.
2. Then use the MBSS to identify the best trading strategy and execute it.
3. Increment time and go to step (1) to account for the fact that the statistical properties of the signal response might have been modified.

Appendices

8 Some random sampling

For details see text book by Rice [1995].

We assume that the population is of size N and that associated with each member of the population is a numerical value of interest denoted by $x_1, x_2, \dots, x_N$. The variable x_i may be a numerical variable such as age or weight, or it may take on the value 1 or 0 to denote the presence or absence of some characteristic. The latter is called the dichotomous case.

8.1 The sample moments

In general, the population variance of a finite population of size N is given by

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2 \text{ with } \mu = \frac{1}{N} \sum_{i=1}^N x_i$$

where μ is the population mean. In the dichotomous case μ equals the proportion p of individuals in the population having the particular characteristic. The population total is

$$\tau = \sum_{i=1}^N x_i = N\mu$$

In many practical situations, the true variance of a population is not known a priori and must be computed somehow. When dealing with extremely large populations, it is not possible to count every object in the population and one must estimate the variance of a population from a sample. We take a sample with replacement of n values $X_1, \dots, X_n$ from the population, where $n < N$ and such that X_i is a random variable. X_i is the value of the i th member of the sample, and x_i is that of the i th member of the population. The joint distribution of the X_i is determined by that of the x_i . We let $\xi_1, \dots, \xi_m$ be the elements of a vector corresponding to the possible values of x_i . Since each member of the population is equally likely to be in the sample, we get

$$P(X_i = \xi_j) = \frac{n_j}{N}$$

The sample mean is

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

Theorem 8.1 *With simple random sampling* $E[\bar{X}] = \mu$

We say that an estimate is unbiased if its expectation equals the quantity we wish to estimate.

$$\text{Mean squared error} = \text{variance} + (\text{biased})^2$$

Since $\bar{X}$ is unbiased, its mean square error is equal to its variance.

Lemma 8.1 *With simple random sampling*

$$\text{Cov}(X_i, X_j) = \begin{cases} \sigma^2 & \text{if } i = j \\ -\frac{\sigma^2}{(N-1)} & \text{if } i \neq j \end{cases}$$

It shows that X_i and X_j are not independent of each other for $i \neq j$, but that the covariance is very small for large values of N .

Theorem 8.2 *With simple random sampling*

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n} \left(1 - \frac{n-1}{N-1}\right)$$

Note, the ratio $\frac{n}{N}$ is called the sampling fraction and

$$p_c = \left(1 - \frac{n-1}{N-1}\right)$$

is the finite population correction. If the sampling fraction is very small we get the approximation

$$\sigma_{\bar{X}} \approx \frac{\sigma}{\sqrt{n}}$$

We estimate the variance on the basis of this sample as

$$\hat{\sigma}^2 = \sigma_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2 \text{ with } \bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

This is the biased sample variance. The unbiased sample variance is

$$\bar{\sigma}_n^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2 \text{ with } \bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

While the first one may be seen as the variance of the sample considered as a population (over n), the second one is the unbiased estimator of the population variance (over N) where $(n-1)$ is the Bessel's correction. Put another way, we get

$$E[\sigma_n^2] = \frac{n-1}{n} \frac{N}{N-1} \sigma^2 \text{ and } E[\bar{\sigma}_n^2] = \sigma^2$$

That is, the unbiased estimate of σ^2 may be obtained by multiplying σ_n^2 by the factor $\frac{n}{n-1} \frac{N-1}{N}$. If the population is large relative to n , the dominant bias is due to the term $\frac{n-1}{n}$. In general one set n between 50 and 180 days. Being a function of random variables, the sample variance is itself a random variable, and it is natural to study its distribution. In the case that y_i are independent observations from a normal distribution, Cochran's theorem shows that $\bar{\sigma}_n^2$ follows a scaled chi-squared distribution

$$(n-1) \frac{\bar{\sigma}_n^2}{\sigma^2} \approx \chi_{n-1}^2$$

If the conditions of the law of large numbers hold for the squared observations, $\bar{\sigma}_n^2$ is a consistent estimator of σ^2 and the variance of the estimator tends asymptotically to zero. The obtained standard deviation $\tilde{\sigma}_N$ is an estimator of σ called the historical volatility.

Corollary 1 *An unbiased estimate of $\text{Var}(\bar{X})$ is*

$$s_{\bar{X}}^2 = \frac{\hat{\sigma}^2}{n} \frac{n}{n-1} \frac{N-1}{N} \frac{N-n}{N-1} = \frac{s^2}{n} \left(1 - \frac{n}{N}\right)$$

where

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$$

One can compute the sample mean and sample variance recursively as follow. Starting from the identities

$$(n+1)\bar{X}_{n+1} = n\bar{X}_n + X_{n+1}$$

and

$$(n+1)(\sigma_{n+1}^2 + (\bar{X}_{n+1})^2) = n(\sigma_n^2 + (\bar{X}_n)^2) + X_{n+1}^2$$

we get the recursive sample mean

$$\bar{X}_{n+1} = \bar{X}_n + \frac{X_{n+1} - \bar{X}_n}{n+1} \quad (8.3)$$

and the recursive sample variance

$$\sigma_{n+1}^2 = \sigma_n^2 + (\bar{X}_n)^2 - (\bar{X}_{n+1})^2 + \frac{X_{n+1}^2 - \sigma_n^2 - (\bar{X}_n)^2}{n+1} \quad (8.4)$$

such that σ_{n+1}^2 only depends on σ_n^2 , $\bar{X}_n$, $\bar{X}_{n+1}$ and X_{n+1} . If we normalise the data with a constant K such that the i th value is $\frac{X_i}{K}$, we let $\tilde{X} = \frac{1}{n} \sum_{i=1}^n \frac{X_i}{K}$ be the sample mean and $\tilde{\sigma}_n^2 = \frac{1}{n} \sum_{i=1}^n (\frac{X_i}{K} - \tilde{X})^2$ be the sample variance. Then, the recursive sample mean becomes

$$\tilde{X}_{n+1} = \tilde{X}_n + \frac{1}{n+1} \left(\frac{X_{n+1}}{K} - \tilde{X}_n \right) \quad (8.5)$$

and the recursive sample variance becomes

$$\tilde{\sigma}_{n+1}^2 = \tilde{\sigma}_n^2 + (\tilde{X}_n)^2 - (\tilde{X}_{n+1})^2 + \frac{1}{n+1} \left(\frac{X_{n+1}^2}{K^2} - \tilde{\sigma}_n^2 - (\tilde{X}_n)^2 \right) \quad (8.6)$$

8.2 Estimation of a ratio

We consider the estimation of a ratio, and assume that for each member of a population, two values, x and y , may be measured. The ratio of interest is

$$r = \frac{\sum_{i=1}^N y_i}{\sum_{i=1}^N x_i} = \frac{\mu_y}{\mu_x}$$

Assuming that a sample is drawn consisting of the pairs (X_i, Y_i) , the natural estimate of r is $R = \frac{\bar{Y}}{\bar{X}}$. Since R is a nonlinear function of the random variables $\bar{X}$ and $\bar{Y}$, there is no closed form for $E[R]$ and $Var(R)$ and we must approximate them by using $Var(\bar{X})$, $Var(\bar{Y})$, and $Cov(\bar{X}, \bar{Y})$. We define the the population covariance of x and y as

$$\sigma_{xy} = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_x)(y_i - \mu_y)$$

One can show that

$$Cov(\bar{X}, \bar{Y}) = \frac{\sigma_{xy}}{n} p_c$$

Theorem 8.3 *With simple random sampling, the approximate variance of $R = \frac{\bar{Y}}{\bar{X}}$ is*

$$Var(R) \approx \frac{1}{\mu_x^2} (r^2 \sigma_X^2 + \sigma_Y^2 - 2r\sigma_{XY}) = \frac{1}{n} p_c \frac{1}{\mu_x^2} (r^2 \sigma_x^2 + \sigma_y^2 - 2r\sigma_{xy})$$

The population correlation coefficient given by

$$\rho = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$$

is a measure of the strength of the linear relationship between the x and the y values in the population. We can express the variance in the above theorem in terms of ρ as

$$\text{Var}(R) \approx \frac{1}{n} p_c \frac{1}{\mu_x^2} (r^2 \sigma_x^2 + \sigma_y^2 - 2r \rho \sigma_x \sigma_y)$$

so that strong correlation of the same sign as r decreases the variance. Further, for small μ_x we get large variance, since small values of $\bar{X}$ in the ratio $R = \frac{\bar{Y}}{\bar{X}}$ cause R to fluctuate wildly.

Theorem 8.4 *With simple random sampling, the expectation of R is given approximately by*

$$E[R] \approx r + \frac{1}{n} p_c \frac{1}{\mu_x^2} (r^2 \sigma_x^2 - \rho \sigma_x \sigma_y)$$

so that strong correlation of the same sign as r decreases the bias, and the bias is large if μ_x is small. In addition, the bias is of the order $\frac{1}{n}$, so its contribution to the mean squared error is of the order $\frac{1}{n^2}$ while the contribution of the variance is of order $\frac{1}{n}$. Hence, for large sample, the bias is negligible compared to the standard error of the estimate. For large samples, truncating the Taylor Series after the linear term provides a good approximation, since the deviations $\bar{X} - \mu_x$ and $\bar{Y} - \mu_y$ are likely to be small. To this order of approximation, R is expressed as a linear combination of $\bar{X}$ and $\bar{Y}$, and an argument based on the central limit theorem can be used to show that R is approximately normally distributed, and confidence intervals can be formed for r by using the normal distribution. In order to estimate the standard error of R , we substitute R for r in the formula of the above theorem where the x and y population variances are estimated by s_x^2 and s_y^2 . The population covariance is estimated by

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y}) = \frac{1}{n-1} \left(\sum_{i=1}^n X_i Y_i - n \bar{X} \bar{Y} \right)$$

and the population correlation is estimated by

$$\hat{\rho} = \frac{s_{xy}}{s_x s_y}$$

The estimated variance of R is thus

$$s_R^2 = \frac{1}{n} p_c \frac{1}{\bar{X}^2} (R^2 s_x^2 + s_y^2 - 2R s_{xy})$$

and the approximate $100(1 - \alpha)\%$ confidence interval for r is $R \pm z(\frac{\alpha}{2}) s_R$.

8.3 Stratified random sampling

The population is partitioned into subpopulations, or strata, which are then independently sampled. We are interested in obtaining information about each of a number of natural subpopulations in addition to information about the population as a whole. It guarantees a prescribed number of observations from each subpopulation, whereas the use of a simple random sample can result in underrepresentation of some subpopulations. Further, the stratified sample mean can be considerably more precise than the mean of a simple random sample, especially if there is considerable variation between strata.

We will denote by N_l , where $l = 1, \dots, L$ the population sizes in the L strata such that $N_1 + N_2 + \dots + N_L = N$ the total population size. The population mean and variance of the l th stratum are denoted by μ_l and σ_l^2 (unknown). The overall population mean can be expressed in terms of the μ_l as follows

$$\mu = \frac{1}{N} \sum_{l=1}^L \sum_{i=1}^{N_l} x_{il} = \frac{1}{N} \sum_{l=1}^L N_l \mu_l = \sum_{l=1}^L W_l \mu_l$$

where x_{il} denotes the i th population value in the l th stratum and $W_l = \frac{N_l}{N}$ is the fraction of the population contained in the l th stratum.

Within each stratum a simple random sample of size n_l is taken. The sample mean in stratum l is denoted by

$$\bar{X}_l = \frac{1}{n_l} \sum_{i=1}^{n_l} X_{il}$$

where X_{il} denotes the i th observation in the l th stratum. By analogy with the previous calculation we get

$$\bar{X}_s = \sum_{l=1}^L \frac{N_l \bar{X}_l}{N} = \sum_{l=1}^L W_l \bar{X}_l$$

Theorem 8.5 *The stratified estimate, $\bar{X}_s$ of the population mean is unbiased. That is, $E[\bar{X}_s] = \mu$.*

Since we assume that the samples from different strata are independent of one another and that within each stratum a simple random sample is taken, the variance of $\bar{X}_s$ can easily be calculate.

Theorem 8.6 *The variance of the stratified sample mean is given by*

$$Var(\bar{X}_s) = \sum_{l=1}^L W_l^2 \frac{1}{n_l} \left(1 - \frac{n_l - 1}{N_l - 1}\right) \sigma_l^2 \quad (8.7)$$

Note, $\frac{n_l - 1}{N_l - 1}$ represents the finite population corrections. If the sampling fractions within all strata are small ($\frac{n_l - 1}{N_l - 1} \ll 1$), we get the approximation

$$Var(\bar{X}_s) \approx \sum_{l=1}^L \frac{W_l^2}{n_l} \sigma_l^2 \quad (8.8)$$

The estimate of σ_l^2 is given by

$$S_l^2 = \frac{1}{n_l - 1} \sum_{i=1}^{n_l} (X_{il} - \bar{X}_l)^2$$

and $Var(\bar{X}_s)$ is estimated by

$$S_{\bar{X}_s}^2 = \sum_{l=1}^L W_l^2 \frac{1}{n_l} \left(1 - \frac{n_l - 1}{N_l - 1}\right) S_l^2$$

The question that naturally arises is how to choose $n_1, \dots, n_L$ to minimise $Var(\bar{X}_s)$ subject to the constraint $n_1 + \dots + n_L = n$. Ignoring the finite population correction within each stratum we get the Neyman allocation.

Theorem 8.7 *The sample sizes $n_1, \dots, n_L$ that minimise $Var(\bar{X}_s)$ subject to the constraint $n_1 + \dots + n_L = n$ are given by*

$$n_l = n \frac{W_l \sigma_l}{\sum_{k=1}^L W_k \sigma_k}$$

where $l = 1, \dots, L$.

This theorem shows that those strata for which $W_l\sigma_l$ is large should be sampled heavily. If W_l is large, the stratum contains a large fraction of the population. If σ_l is large, the population values in the stratum are quite variable, and a relatively large sample size must be used. Substituting the optimal values of n_l into the Equation (8.7) we get the following corollary.

Corollary 2 Denoting by $\bar{X}_{so}$ the stratified estimate using the optimal Neyman allocation and neglecting the finite population correction, we get

$$Var(\bar{X}_{so}) = \frac{1}{n} \left(\sum_{l=1}^L W_l \sigma_l \right)^2$$

The optimal allocations depend on the individual variances of the strata, which generally will not be known. A simple alternative method of allocation is to use the same sampling fraction in each stratum

$$\frac{n_1}{N_1} = \frac{n_2}{N_2} = \dots = \frac{n_L}{N_L}$$

which holds if

$$n_l = n \frac{N_l}{N} = n W_l \quad (8.9)$$

$l = 1, \dots, L$. This method is called the Proportional allocation. The estimate of the population mean based on proportional allocation is

$$\bar{X}_{sp} = \sum_{l=1}^L W_l \bar{X}_l = \frac{1}{n} \sum_{l=1}^L \sum_{i=1}^{n_l} X_{il}$$

since $\frac{W_l}{n_l} = \frac{1}{n}$. This estimate is simply the unweighted mean of the sample values.

Theorem 8.8 With stratified sampling based on proportional allocation, ignoring the finite population correction, we get

$$Var(\bar{X}_{sp}) = \frac{1}{n} \sum_{l=1}^L W_l \sigma_l^2$$

We can compare $Var(\bar{X}_{so})$ and $Var(\bar{X}_{sp})$ to define when optimal allocation is substantially better than proportional allocation.

Theorem 8.9 With stratified random sampling, the difference between the variance of the estimate of the population mean based on proportional allocation and the variance of that estimate based on optimal allocation is, ignoring the finite population correction,

$$Var(\bar{X}_{sp}) - Var(\bar{X}_{so}) = \frac{1}{n} \sum_{l=1}^L W_l (\sigma_l - \bar{\sigma})^2$$

where

$$\bar{\sigma} = \sum_{l=1}^L W_l \sigma_l$$

As a result, if the variances of the strata are all the same, proportional allocation yields the same results as optimal allocation. The more variable these variances are, the better it is to use optimal allocation.

We can also compare the variance under simple random sampling with the variance under proportional allocation. Neglecting the finite population correction, the variance under simple random sampling is $Var(\bar{X}) = \frac{\sigma^2}{n}$. We first need a relationship between the overall population variance σ^2 and the strata variances σ_l^2 . The overall population variance may be expressed as

$$\sigma^2 = \frac{1}{N} \sum_{l=1}^L \sum_{i=1}^{N_L} (x_{il} - \mu)^2$$

Further, using the relation

$$(x_{il} - \mu)^2 = (x_{il} - \mu_l)^2 + 2(x_{il} - \mu_l)(\mu_l - \mu) + (\mu_l - \mu)^2$$

and realising that when both sides are summed over l , the middle term on the right-hand side becomes zero, we have

$$\sum_{i=1}^{N_L} (x_{il} - \mu)^2 = \sum_{i=1}^{N_L} (x_{il} - \mu_l)^2 + N_L(\mu_l - \mu)^2 = N_L\sigma_l^2 + N_L(\mu_l - \mu)^2$$

Dividing both sides by N and summing over l , we have

$$\sigma^2 = \sum_{l=1}^L W_l\sigma_l^2 + \sum_{l=1}^L W_l(\mu_l - \mu)^2$$

Substituting this expression for σ^2 into $Var(\bar{X})$ and using the formula for $Var(\bar{X}_{sp})$ completes the proof of the following theorem.

Theorem 8.10 *The difference between the variance of the mean of a simple random sample and the variance of the mean of a stratified random sample based on proportional allocation is, neglecting the finite population correction,*

$$Var(\bar{X}) - Var(\bar{X}_{sp}) = \frac{1}{n} \sum_{l=1}^L W_l(\mu_l - \mu)^2$$

Thus, stratified random sampling with proportional allocation always gives a smaller variance than does simple random sampling, providing that the finite population correction is ignored. Typically, stratified random sampling can result in substantial increases in precision for populations containing values that vary greatly in size.

In order to construct the optimal number of strata, the population values themselves (which are unknown) would have to be used. Stratification must therefore be done on the basis of some related variable that is known or on the results of earlier samples.

According to the Neyman-Pearson Paradigm, a decision as to whether or not to reject H_0 in favour of H_A is made on the basis of $T(X)$, where X denotes the sample values and $T(X)$ is a statistic.

The statistical properties of the methods are relevant if it is reasonable to model the data stochastically. There exists methods that are sample analogues of the cumulative distribution function of a random variable. It is useful in displaying the distribution of sample values.

Given $x_1, \dots, x_n$ a batch of numbers, the empirical cumulative distribution function (ecdf) is defined as

$$F_n(x) = \frac{1}{n}(\text{no } x_i \leq x)$$

where $F_n(x)$ gives the proportion of the data less than or equal to x . It is a step function with a jump of height $\frac{1}{n}$ at each point x_i . The ecdf is to a sample what the cumulative distribution is to a random variable. We now consider some of the elementary statistical properties of the ecdf when $X_1, \dots, X_n$ is a random sample from a continuous distribution function F . We choose to express F_n as

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n I_{(-\infty, x]}(X_i)$$

The random variables $I_{(-\infty, x]}(X_i)$ are independent Bernoulli random variables

$$I_{(-\infty, x]}(X_i) = \begin{cases} 1 & \text{with probability } F(x) \\ 0 & \text{with probability } 1 - F(x) \end{cases}$$

Thus, $nF_n(x)$ is a binomial random variable with the first two moments being

$$\begin{aligned} E[F_n(x)] &= F(x) \\ \text{Var}(F_n(x)) &= \frac{1}{n} F(x)[1 - F(x)] \end{aligned}$$

8.4 Geometric mean

The use of a geometric mean normalises the ranges being averaged, so that no range dominates the weighting, and a given percentage change in any of the properties has the same effect on the geometric mean. The geometric mean applies only to positive numbers. It is also often used for a set of numbers whose values are meant to be multiplied together or are exponential in nature, such as data on the growth of the human population or interest rates of a financial investment. The geometric mean of a data set $\{a_1, a_2, \dots, a_n\}$ is given by

$$\left(\prod_{i=1}^n a_i \right)^{\frac{1}{n}}$$

The geometric mean of a data set is less than the data set's arithmetic mean unless all members of the data set are equal, in which case the geometric and arithmetic means are equal. By using logarithmic identities to transform the formula, the multiplications can be expressed as a sum and the power as a multiplication

$$\left(\prod_{i=1}^n a_i \right)^{\frac{1}{n}} = e^{\frac{1}{n} \sum_{i=1}^n \ln a_i}$$

This is sometimes called the log-average. It is simply computing the arithmetic mean of the logarithm-transformed values of a_i (i.e., the arithmetic mean on the log scale) and then using the exponentiation to return the computation to the original scale.

9 The importance of asset returns

It is important to understand the properties of the market returns in order to devise their dynamics. We are first going to define market returns and recall some statistical tools necessary to model asset returns.

9.1 Asset returns and their characteristics

9.1.1 Defining financial returns

We consider the probability space $(\Omega, \mathcal{F}, \mathbb{P})$ where $\mathcal{F}_t$ is a right continuous filtration including all $\mathbb{P}$ negligible sets in $\mathcal{F}$. For simplicity, we let the market be complete and assume that there exists an equivalent martingale measure $\mathbb{Q}$ as defined in the diffusion model by Harrison et al. [1979] [1981]. In the presence of continuous dividend yield, that unique probability measure equivalent to $\mathbb{P}$ is such that the discounted stock price plus the cumulated dividends are martingale when the riskless asset is the numeraire. In a general setting, we let the underlying process $(S_t)_{t \geq 0}$ be a one-dimensional Ito process valued in the open subset D .

9.1.1.1 Asset returns Even though the price of an asset as a function of time constitutes a financial time series, it generally shows exponential behaviour in time, making it difficult to manipulate. Return series are easier to handle than price series due to their more attractive statistical properties and to the fact that they represent a complete and scale-free summary of the investment opportunity (see Campbell et al. [1997]).

Expected returns need to be viewed over some time horizon, in some base currency, and using one of many possible averaging and compounded methods. Holding the asset for one period of time, from date t to date $(t+1)$, would result in a simple gross return

$$1 + R_{t,t+1} = \frac{S_{t+1}}{S_t} \quad (9.10)$$

where the corresponding one-period simple net return, or simple return, $R_{t,t+1}$, is given by

$$R_{t,t+1} = \frac{S_{t+1}}{S_t} - 1 = \frac{S_{t+1} - S_t}{S_t}$$

More generally, we let

$$R_{t-d,t} = \frac{\nabla_d S_t + D_{t-d,t}}{S_{t-d}}$$

be the discrete return of the underlying process where $\nabla_d S_t = S_t - S_{t-d}$ with period d and where $D_{t-d,t}$ is the dividend over the period $[t-d, t]$. For simplicity we will only consider dividend-adjusted prices with discrete dividend-adjusted returns $R_{t-d,t} = \frac{\nabla_d S_t}{S_{t-d}}$. Hence, holding the asset for d periods, between the dates $t-d$ and t , gives a d -period simple gross return

$$\begin{aligned} 1 + R_{t-d,t} = \frac{S_t}{S_{t-d}} &= \frac{S_t}{S_{t-1}} \times \frac{S_{t-1}}{S_{t-2}} \times \cdots \times \frac{S_{t-d+1}}{S_{t-d}} \\ &= (1 + R_{t-1,t})(1 + R_{t-2,t-1}) \cdots (1 + R_{t-d,t-d+1}) = \prod_{j=1}^d (1 + R_{t-j,t-j+1}) \end{aligned} \quad (9.11)$$

so that the d -period simple gross return is just the product of the d one-period simple gross returns, which is called a compound return. In that setting, the price of the underlying asset becomes

$$S_t = S_{t-d} \prod_{j=1}^d (1 + R_{t-j,t-j+1})$$

Holding the asset for d years, we get the gross return

$$(1 + R_{t-d,t}^A)^d = \frac{S_t}{S_{t-d}}$$

where $R_{t-d,t}^A$ is the annualised (average) return defined as

$$\begin{aligned} R_{t-d,t}^A &= \left(\frac{S_t}{S_{t-d}} \right)^{\frac{1}{d}} - 1 \\ &= \left(\prod_{j=1}^d (1 + R_{t-j,t-j+1}) \right)^{\frac{1}{d}} - 1 \end{aligned}$$

It is the geometric mean of the d one-period simple gross returns involved, and can be computed (see Appendix (??)) by

$$R_{t-d,t}^A = e^{\frac{1}{d} \sum_{j=1}^d \ln(1 + R_{t-j,t-j+1})} - 1$$

It is simply the arithmetic mean of the logarithm returns $\ln(1 + R_{t-j,t-j+1})$, for $j = 1, \dots, d$, which is then exponentiated to return the computation to the original scale. As it is easier to compute arithmetic average than geometric mean, and since the one-period returns tend to be small, one can use a first-order Taylor expansion⁴ to approximate the annualised (average) return as

$$R_{t-d,t}^A \approx \frac{1}{d} \sum_{j=1}^d R_{t-j,t-j+1}$$

Note, the arithmetic mean of two successive returns of +50% and -50% is 0%, but the geometric mean is -13% since $[(1 + 0.5)(1 - 0.5)]^{\frac{1}{2}} = 0.87$ with $d = 2$ periods. While some financial theory requires arithmetic mean as inputs (single-period Markowitz or mean-variance optimisation, single-period CAPM), most investors are interested in wealth compounding which is better captured by geometric means.

We now introduce the continuously compounded interest rates. In general, the net asset value A of continuous compounding is

$$A = Ce^{r \times n}$$

where r is the interest rate per annum, C is the initial capital, and n is the number of years. We call $e^{r \times n}$ the compounding factor. Similarly,

$$C = Ae^{-r \times n}$$

is referred to as the present value of an asset that is worth A dollars n years from now, assuming that the continuously compounded interest rate is r per annum. We call $e^{-r \times n}$ the discount factor.

If the d -period gross return on a security is just $1 + R_{t-d,t}$, then the continuously compounded return, or logarithmic return, is

$$r_L(t-d, t) = \ln(1 + R_{t-d,t}) = L_t - L_{t-d} \quad (9.12)$$

where $L_t = \ln S_t$. Note, on a daily basis we get $R_t = R_{t-1,t}$ and $r_L(t) = \ln(1 + R_t)$.

The change in log price is the yield or return, with continuous compounding, from holding the security from trading day $t-1$ to trading day t . As a result, the price of the underlying asset becomes

$$S_t = S_{t-1} e^{r_L(t)}$$

Further, the continuously compounded return $r_L(t)$ has the property that the log-return, between the price at time t_1 and at time t_n , is given by the sum of the $r_L(t)$ between t_1 and t_n , that is,

⁴since $\log(1+x) \approx x$ for $|x| \leq 1$

$$\log \frac{S_{t_n}}{S_{t_1}} = \sum_{i=1}^n r_L(t_i)$$

which implies that

$$S_{t_n} = S_{t_1} e^{\sum_{i=1}^n r_L(t_i)}$$

As a result, if the $r_L(t)$ are independent random variables with finite mean and variance, the central limit theorem implies that for very large n , the summand in the above equation is normally distributed. Hence, we would get a log-normal distribution for S_{t_n} given S_{t_1} . In addition, the variability of simple price changes, for a given security, is an increasing function of the price level of the security, whereas this is not necessarily the case with the change in log price. Then, given Equation (9.11), the logarithmic returns for d-period, defined in Equation (9.12), become

$$r_L(t-d, t) = r_L(t) + r_L(t-1) + r_L(t-2) + \cdots + r_L(t-d+1) \quad (9.13)$$

so that the continuously compounded multiperiod return is simply the sum of continuously compounded one-period returns. Note, statistical properties of log returns are more tractable. Moreover, in the cross section approach aggregation is done across the individual returns.

Remark 9.1 Simple returns $R_{t-d,t}$ are additive across assets but not over time (see Equation (9.11)), whereas continuously compounded returns $r_L(t-d, t)$ are additive over time but not across assets (see Equation (9.13)).

9.1.1.2 The percent returns versus the logarithm returns In the financial industry, most measures of returns and indices use change of returns $R_t = R_{t-1,t}$ defined as $\frac{S_t - S_{t-1}}{S_{t-1}}$ where S_t is the price of a series at time t . However, some investors and researchers prefer to use returns based on logarithms of prices $r_t = \log \frac{S_t}{S_{t-1}}$ or compound returns. As discussed in Section (9.1.1.1), continuous time generalisations of discrete time results are easier, and returns over more than one day are simple functions of a single day return. In order to compare change and compound returns, Longerstaey et al. [1995a] compared kernel estimates of the probability density function for both returns. As opposed to the histogram of the data, this approach spreads the frequency represented by each observation along the horizontal axis according to a chosen distribution function, called the kernel, and chosen to be the normal distribution. They also compared daily volatility estimates for both types of returns based on an exponential weighting scheme. They concluded that the volatility forecasts were very similar. They used that methodology to compute the volatility for change returns and then replaced the change returns with logarithm returns. The same analysis was repeated on correlation by changing the inputs from change returns to logarithm returns. They also used monthly time series and found little difference between the two volatility and correlation series. Note, while the one month volatility and correlation estimators based on change and logarithm returns do not coincide, the difference between their point estimates is negligible.

9.1.2 Portfolio returns

We consider a portfolio consisting of N instruments, and let $r_L^i(t)$ and $R_i(t)$, for $i = 1, 2, \dots, N$, be respectively the continuously compounded and percent returns. We assign weights ω_i to the i th instrument in the portfolio together with a condition of no short sales $\sum_{i=1}^N \omega_i = 1$ (it is the percentage of the portfolio's value invested in that asset). We let P_0 be the initial value of the portfolio, and P_1 be the price after one period, then by using discrete compounding, we derive the usual expression for a portfolio return as

$$P_1 = \omega_1 P_0 (1 + R_1) + \omega_2 P_0 (1 + R_2) + \cdots + \omega_N P_0 (1 + R_N) = \sum_{i=1}^N \omega_i P_0 (1 + R_i)$$

We let $R_p(1) = \frac{P_1 - P_0}{P_0}$ be the return of the portfolio for the first period and replace P_1 with its value. We repeat the process at periods $t = 2, 3, \dots$ to get the portfolio at time t as

$$R_p(t) = \omega_1 R_1(t) + \omega_2 R_2(t) + \dots + \omega_N R_N(t) = \sum_{i=1}^N \omega_i R_i(t)$$

Hence, the simple net return of a portfolio consisting of N assets is a weighted average of the simple net returns of the assets involved. However, the continuously compounded returns of a portfolio do not have the above convenient property. The portfolio of returns satisfies

$$P_1 = \omega_1 P_0 e^{r_1} + \omega_2 P_0 e^{r_2} + \dots + \omega_N P_0 e^{r_N}$$

and setting $r_p = \log \frac{P_1}{P_0}$, we get

$$r_p = \log (\omega_1 P_0 e^{r_1} + \omega_2 P_0 e^{r_2} + \dots + \omega_N P_0 e^{r_N})$$

Nonetheless, RiskMetrics (see Longestaeay et al. [1996]) uses logarithmic returns as the basis in all computations and the assumption that simple returns R_i are all small in magnitude. As a result, the portfolio return becomes a weighted average of logarithmic returns

$$r_p(t) \approx \sum_{i=1}^N \omega_i r_L^i(t)$$

since $\log(1+x) \approx x$ for $|x| \leq 1$.

9.1.3 Modelling returns: The random walk

We are interested in characterising the future changes in the portfolio of returns, described in Section (9.1.2), by forecasting each component of the portfolio using only past changes of market prices. To do so, we need to model

1. the temporal dynamics of returns
2. the distribution of returns at any point in time

Traditionally, to get tractable statistical properties of asset returns, financial markets assume that simple returns $\{R_{it}|t = 1, \dots, T\}$ (see Equation (9.10)) are independently and identically distributed as normal with fixed mean and variance. However, while the lower bound of a simple return is -1 , normal distribution may assume any value in the real line (no lower bound). Further, if we assume that R_{it} is normally distributed, then the multi-period simple return $R_{it}(k)$ is no-longer normally distributed. At last, the normality assumption is not supported by many empirical asset returns which tend to have positive excess kurtosis. Still, the random walk is one of the widely used class of models to characterise the development of price returns.

In order to guarantee non-negativity of prices, we model the logarithm price L_t as a random walk with independent and identically distributed (iid) normally distributed changes with mean μ and variance σ^2 . It is given by

$$L_t = \mu + L_{t-1} + \sigma \epsilon_t, \epsilon_t \sim iidN(0, 1)$$

The use of logarithm prices, implies that the model has continuously compounded returns, that is,

$$r_t = L_t - L_{t-1} = \mu + \sigma \epsilon_t$$

with mean and variance

$$E[r_t] = \mu, Var(r_t) = \sigma^2$$

Hence, an expression for prices can be derived as

$$S_t = S_{t-1}e^{\mu + \sigma\epsilon_t}$$

and S_t follows the log-normal distribution. Hence, the mean and variance of simple returns become

$$E[R_t] = e^{\mu + \frac{1}{2}\sigma^2} - 1, \text{Var}(R_t) = e^{2\mu + \sigma^2}(e^{\sigma^2} - 1)$$

which can be used in forecasting asset returns. There is no lower bound for r_t , and the lower bound for R_t is satisfied using $1 + R_t = e^{r_t}$. Assuming that logarithmic price changes are i.i.d. implies that

- at each point in time t the log price changes are identically distributed with mean μ and variance σ^2 implying homoskedasticity (unchanging prices over time).
- log price changes are statistically independent of each other over time (the values of returns sampled at different points are completely unrelated).

However, the lognormal assumption is not consistent with all the properties of historical stock returns. The above models assume a constant variance in price changes, which in practice is flawed in most financial time series data. We can relax this assumption to let the variance vary with time in the modified model

$$L_t = \mu + L_{t-1} + \sigma_t\epsilon_t, \epsilon_t \sim N(0, 1)$$

These random walk models imply certain movement of financial prices over time.

9.2 An introduction to statistical models

9.2.1 The framework

In order to model returns with classic statistical models some statistical properties must be assumed such as stationarity or weak stationarity.

We let r_t be the log return of an asset at time t , and assume that $\{r_t\}$ is either serially uncorrelated or with minor lower order serial correlations, but it is dependent. We assume that r_t follows the dynamics

$$r_t = \mu_t + a_t \tag{9.14}$$

where a_t is the (stochastic) shock or mean-corrected return (or innovation) of an asset return⁵. The conditional mean and conditional variance of r_t , given the filtration $\mathcal{F}_{t-1}$, are defined by

$$\begin{aligned} \mu_t &= E[r_t|\mathcal{F}_{t-1}] = G(\mathcal{F}_{t-1}) \\ \sigma_t^2 &= \text{Var}(r_t|\mathcal{F}_{t-1}) = E[(r_t - \mu_t)^2|\mathcal{F}_{t-1}] = H(\mathcal{F}_{t-1}) \end{aligned}$$

where G and H are well defined functions with $H(\cdot) > 0$. The model for μ_t is the mean equation for r_t , and the model for σ_t^2 is the volatility equation for r_t .

Remark 9.2 Some authors use h_t to denote the conditional variance of r_t , in which case the shock becomes $a_t = \sqrt{h_t}\epsilon_t$.

⁵since $a_t = r_t - \mu_t$

Both the functions G and H could be non-linear. For simplicity of exposition we first consider the linear case. We will then briefly present some statistical models when H is non-linear. Since we assumed that the serial dependence of a stock return series was weak, if it exists at all, μ_t should be simple and we can assume that r_t follows a simple time series model such as a stationary $ARMA(p, q)$ model. In that case, the filtration is given by $\mathcal{F}_{t-1} = \{r_{t-1}, r_{t-2}, \dots, r_{t-p}\}$ with p -lags and the drift satisfies

$$\mu_t = \phi_0 + \sum_{i=1}^p \phi_i r_{t-i} - \sum_{i=1}^q \theta_i a_{t-i} \quad (9.15)$$

where ϕ_i and θ_i are real numbers. The function H is constant, that is, $H(\mathcal{F}_{t-1}) = \sigma^2$. The parameters p and q are non-negative integers, and the order (p, q) of the ARMA model may depend on the frequency of the return series.

9.2.2 The need to forecast volatility

9.2.2.1 Presentation When visualising financial time series, one can observe heteroskedasticity⁶, with periods of high volatility and periods of low volatility, corresponding to periods of high and low risks, respectively. We also observe returns having very high absolute value compared with their mean, suggesting fat tail distribution for returns, with large events having a larger probability to appear when compared to returns drawn from a Gaussian distribution. Hence, besides the return series, we must also consider the volatility process and the behaviour of extreme returns of an asset (the large positive or negative returns). The negative extreme returns are important in risk management, whereas positive extreme returns are critical to holding a short position. Volatility is important in risk management as it provides a simple approach to calculating the value at risk (VaR) of a financial position. Further, modelling the volatility of a time series can improve the efficiency in parameter estimation and the accuracy in interval forecast. As returns may vary substantially over time and appear in clusters, the volatility process is concerned with the evolution of conditional variance of the return over time. Users must make sure that volatilities and correlations are predictable and that their forecasts incorporate the most useful information available. As the forecasts are based on historical data, the estimators must be flexible enough to account for changing market conditions. One simple approach is to assume that returns are governed by the random walk model, and that the sample standard deviation $\hat{\sigma}_N$ or the sample variance $\hat{\sigma}_N^2$ of returns for N periods of time can be used as a simple forecast of volatility of returns, r_t , over the future period $[t + 1, t + h]$ for some positive integer h . However, volatility has some specific characteristics such as

- volatility clusters: volatility may be high for certain time periods and low for other periods.
- continuity: volatility jumps are rare.
- mean-reversion: volatility does not diverge to infinity, it varies within some fixed range so that it is often stationary.
- volatility reacts differently to a big price increase or a big price drop.

These properties play an important role in the development of volatility models. As a result, there is a large literature on econometric models available for modelling the volatility of an asset return, called the conditional heteroscedastic (CH) models. Some univariate volatility models include the autoregressive conditional heteroscedastic (ARCH) model of Engle [1982], the generalised ARCH (GARCH) model of Bollerslev [1986], the exponential GARCH (EGARCH) of Nelson [1991], the stochastic volatility (SV) models and many more. Tsay [2002] discussed the advantages and weaknesses of each volatility model and showed some applications of the models. Unfortunately, stock volatility is not directly observable from returns as in the case of daily volatility where there is only one observation in a trading day. Even though one can use intraday data to estimate daily volatility, accuracy is difficult to obtain. The unobservability of volatility makes it difficult to evaluate the forecasting performance of CH models and heuristics must be developed to estimate volatility on small samples.

⁶Heteroskedastic means that a time series has a non-constant variance through time.

9.2.2.2 A first decomposition As risk is mainly given by the probability of large negative returns in the forthcoming period, risk evaluation is closely related to time series forecasts. The desired quantity is a forecast for the probability distribution (pdf) $\tilde{p}(r)$ of the possible returns r over the risk horizon ΔT . This problem is generally decomposed into forecasts for the mean and variance of the return probability distribution

$$r_{\Delta T} = \mu_{\Delta T} + a_{\Delta T}$$

with

$$a_{\Delta T} = \sigma_{\Delta T} \epsilon$$

where the return $r_{\Delta T}$ over the period ΔT is a random variable, $\mu_{\Delta T}$ is the forecast for the mean return, and $\sigma_{\Delta T}$ is the volatility forecast. The term $a_{\Delta T} = r_{\Delta T} - \mu_{\Delta T}$ is the mean-corrected asset return. The residual ϵ , which corresponds to the unpredictable part, is a random variable distributed according to a pdf $p_{\Delta T}(\epsilon)$. The standard assumption is to let $\epsilon(t)$ be an independent and identically distributed (iid) random variable. In general, a risk methodology will set the mean μ to zero and concentrate on σ and $p(\epsilon)$. To validate the methodology, we set

$$\epsilon = \frac{r - \mu}{\sigma}$$

compute the right hand side on historical data, and obtain a time series for the residual. We can then check that ϵ is independent and distributed according to $p(\epsilon)$. For instance, we can test that ϵ is uncorrelated, and that given a risk threshold α (say, 95%), the number of exceedance behaves as expected. However, when the horizon period ΔT increases, it becomes very difficult to perform back testing due to the lack of data. Alternatively, we can consider a process to model the returns with a time increment δt of one day, computing the forecasts using conditional averages. We can then relate daily data with forecasts at any time horizon, and the forecasts depend only on the process parameters, which are independent of ΔT and are consistent across risk horizon. The quality of the volatility forecasts is the major determinant factor for a risk methodology. The residuals can then be computed and their properties studied.

9.2.3 The structure of volatility models

9.2.3.1 Presentation The above heuristics being poor estimates of the future volatility, one must rely on proper volatility models such as the conditional heteroscedastic (CH) models. Since the early 80s, volatility clustering spawned a large literature on a new class of stochastic processes capturing the dependency of second moments in a phenomenological way. As the lognormal assumption is not consistent with all the properties of historical stock returns, Engle [1982] first introduced the autoregressive conditional heteroscedasticity model (ARCH) which has been generalised to GARCH by Bollerslev [1986].

We consider the log return r_t to follow the $ARMA(p, q)$ model described in the above Appendix. The excess kurtosis values, measuring deviation from the normality of the returns, are indicative of the long-tailed nature of the process. Hence, one can then compute and plot the autocorrelation functions for the returns process r_t as well as the autocorrelation functions for the squared returns r_t^2 .

1. If the securities exhibit a significant positive autocorrelation at lag one and higher lags as well, then large (small) returns tend to be followed by large (small) returns of the same sign. That is, there are trends in the return series. This is evidence against the weakly efficient market hypothesis which asserts that all historical information is fully reflected in prices, implying that historical prices contain no information that could be used to earn a trading profit above that which could be attained with a naive buy-and-hold strategy which implies further that returns should be uncorrelated. In this case, the autocorrelation function would suggest that an autoregressive model should capture much of the behaviour of the returns.
2. The autocorrelation in the squared returns process would suggest that large (small) absolute returns tend to follow each other. That is, large (small) returns are followed by large (small) returns of unpredictable sign. It implies that the returns series exhibits volatility clustering where large (small) returns form clusters throughout

the series. As a result, the variance of a return conditioned on past returns is a monotonic function of the past returns, and hence the conditional variance is heteroskedastic and should be properly modelled.

The conditional heteroscedastic (CH) models are capable of dealing with this conditional heteroskedasticity. The variance in the model described in Equation (9.14) becomes

$$\sigma_t^2 = \text{Var}(r_t | \mathcal{F}_{t-1}) = \text{Var}(a_t | \mathcal{F}_{t-1})$$

Since the way in which σ_t^2 evolves over time differentiate one volatility model from another, the CH models are concerned with the evolution of the volatility. Hence, modelling conditional heteroscedasticity (CH) amounts to augmenting a dynamic equation to a time series model to govern the time evolution of the conditional variance of the shock. We distinguish two types or groups of CH models, the first one using an exact function to govern the evolution of σ_t^2 , and the second one using a stochastic equation to describe σ_t^2 . For instance, the (G)ARCH model belongs to the former, and the stochastic volatility (SV) model belongs to the latter. In general, we estimate the conditional mean and variance equations jointly in empirical studies.

9.2.3.2 Benchmark volatility models The ARCH model, which is the first model providing a systematic framework for volatility modelling, states that

1. the mean-corrected asset return a_t is serially uncorrelated, but dependent
2. the dependence of a_t can be described by a simple quadratic function of its lagged values.

Specifically, setting $\mu_t = 0$ for simplicity, an $ARCH(p)$ model assumes that

$$r_t = h_t^{\frac{1}{2}} \epsilon_t, h_t = \alpha_0 + \alpha_1 r_{t-1}^2 + \dots + \alpha_p r_{t-p}^2$$

where $\{\epsilon_t\}$ is a sequence of i.i.d. random variables with mean zero and variance 1, $\alpha_0 > 0$, and $\alpha_i \geq 0$ for $i > 0$. In practice, ϵ_t follows the standard normal or a standardised Student-t distribution. Generalising the ARCH model, the main idea behind (G)ARCH models is to consider asset returns as a mixture of normal distributions with the current variance being driven by a deterministic difference equation

$$r_t = h_t^{\frac{1}{2}} \epsilon_t \text{ with } \epsilon_t \sim N(0, 1) \quad (9.16)$$

and

$$h_t = \alpha_0 + \sum_{i=1}^p \alpha_i r_{t-i}^2 + \sum_{j=1}^q \beta_j h_{t-j}, \alpha_0 > 0, \alpha_i, \beta_j > 0 \quad (9.17)$$

where $\alpha_0 > 0$, $\alpha_i \geq 0$, $\beta_j \geq 0$, and $\sum_{i=1}^{\max(p,q)} (\alpha_i + \beta_i) < 1$. The latter constraint on $\alpha_i + \beta_i$ implies that the unconditional variance of r_t is finite, whereas its conditional variance h_t evolves over time. In general, empirical applications find the $GARCH(1, 1)$ model with $p = q = 1$ to be sufficient to model financial time series

$$h_t = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta_1 h_{t-1}, \alpha_0 > 0, \alpha_1, \beta_1 > 0 \quad (9.18)$$

When estimated, the sum of the parameters $\alpha_1 + \beta_1$ turns out to be close to the non-stationary case, that is, mostly only a constraint on the parameters prevents them from exceeding 1 in their sum, which would lead to non-stationary behaviour. Different extensions of GARCH were developed in the literature with the objective of better capturing the financial stylised facts. Among them are the Exponential GARCH (EGARCH) model proposed by Nelson [1991] accounting for asymmetric behaviour of returns, the Threshold GARCH (TGARCH) model of Rabemananjara et al. [1993] taking into account the leverage effects, the regime switching GARCH (RS-GARCH) developed by Cai [1994], and the Integrated GARCH (IGARCH) introduced by Engle et al allowing for capturing high persistence observed in returns time series. Nelson [1990] showed that Ito diffusion or jump-diffusion processes could be obtained as a

continuous time limit of discrete GARCH sequences. In order to capture stochastic shocks to the variance process, Taylor [1986] introduced the class of stochastic volatility (SV) models whose instantaneous variance is driven by

$$\ln(h_t) = k + \phi \ln(h_{t-1}) + \tau \xi_t \text{ with } \xi_t \sim N(0, 1) \quad (9.19)$$

This approach has been refined and extended in many ways. The SV process is more flexible than the GARCH model, providing more mixing due to the co-existence of shocks to volatility and return innovations. However, one drawback of the GARCH models and extension to Equation (9.19) is their implied exponential decay of the autocorrelations of measures of volatility which is in contrast to the very long autocorrelation discussed in Appendix (10). Both the GARCH and the baseline SV model are only characterised by short-term rather than long-term dependence. In order to capture long memory effects, the GARCH and SV models were expanded by allowing for an infinite number of lagged volatility terms instead of the limited number of lags present in Equations (9.17) and (9.19). To obtain a compact characterisation of the long memory feature, a fractional differencing operator was used in both extensions, leading to the fractionally integrated GARCH (FIGARCH) model introduced by Baillie et al. [1996], and the long-memory stochastic volatility model of Breidt et al. [1998]. As an intermediate approach, Dacorogna et al. [1998] proposed the heterogeneous ARCH (HARCH) model, considering returns at different time aggregation levels as determinants of the dynamic law governing current volatility. In this model, we need to replace Equations (9.17) with

$$h_t = c_0 + \sum_{j=1}^n c_j r_{t,t-\Delta t_j}^2$$

where $r_{t,t-\Delta t_j} = \ln(p_t) - \ln(p_{t-\Delta t_j})$ are returns computed over different frequencies. This model was motivated by the finding that volatility on fine time scales can be explained to a larger extent by coarse-grained volatility than vice versa (see Muller et al. [1997]). Thus, the right hand side of the above equation covers local volatility at various lower frequencies than the time step of the underlying data ($\Delta t_j = 2, 3, \dots$). Note, multifractal models have a closely related structure but model the hierarchy of volatility components in a multiplicative rather than additive format.

9.3 Generalised nonlinear nonparametric models

9.3.1 Presentation

Among the various techniques to forecast and classify financial time series, we saw in Appendix (11) that fundamental and technical analysis were the most popular ones. Even though statistical procedures were widely used for pattern recognition, the effectiveness of these methods relies both on model's assumptions and prior knowledge on data properties. To remedy these pitfalls, several classifiers developed, using various data mining and computational intelligence methods such as rule induction, fuzzy rule induction, decision trees, neural networks etc. For instance, the best recognised tools in the currency markets is the artificial neural networks (ANNs), supported by numerous empirical studies (see Ahmed et al. [2010]). Among the different networks existing, the artificial neural networks (ANNs) and the artificial recurrent neural networks (RNNs) are computational models designed by more or less detailed analogy with biological brain modules. The former is presented in Section (15.1) and the latter is introduced in Section (16.4.1). The foremost reason for using ANNs is that there is some nonlinear aspect to the forecasting problem under consideration, taking the form of a complex nonlinear relationship between the independent and dependent variables. The characteristics of financial time series, such as equity stock or currency markets, are influenced by the psychology of traders (behavioural finance) and are strongly non-linear and hardly predictable (see Maknickiene et al. [2011]).

9.3.2 Describing the models

Given a time series $\{x_t, x_{t-1}, \dots\}$, we aim to learn from the known data to predict future values. That is, we want to estimate x at some future time

$$\hat{x}_{t+h} = f(x_t, x_{t-1}, \dots, x_{t-p}, z_1, z_2, \dots, z_q)$$

where h is the horizon prediction, and z_q is the q th other explanatory variable. Before training the network, the input data should be normalised into the interval $[b_L, b_H]$, with b_L and b_H being lower and upper bonds, by using the equation

$$\hat{x}_i = (b_H - b_L) \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)} + b_L \text{ for } i = 1, \dots, N \quad (9.20)$$

where $\min(x_i)$ is the minimal value of all x_i , $\max(x_i)$ is the maximal value of all x_i , and $\hat{x}_i$ is the normalised value of x_i , respectively. Note, we sometime need to perform operations on the normalised data within a sample window. When comparing the results among different windows we must always convert them back into the original order of magnitude using

$$x_i = [\max(x_i) - \min(x_i)] \frac{\hat{x}_i - b_L}{(b_H - b_L)} + \min(x_i) \text{ for } i = 1, \dots, N$$

As discussed in Appendix (9.2), the statistical approach to forecasting time series involves the construction of stochastic models to predict the value $\hat{x}_{t+h}$ given previous observations. One approach is to use an $ARMA(p, q)$ model, but it lacks the ability to capture the nonlinear features of financial time series. Another approach is to extend the linear models to nonlinear ones. For example, we can generalise an $AR(p)$ model to become a *nonlinear autoregressive* (NAR) model (see Connor et al. [1994])

$$x_t = h(x_{t-1}, x_{t-2}, \dots, x_{t-p}) + e_t, \quad (9.21)$$

where h is an unknown smooth function. Assuming that $E(e_t | x_{t-1}, x_{t-2}, \dots) = 0$ and e_t has finite second moment, then the minimum mean square error optimal predictor of x_t with the knowledge of $x_{t-1}, \dots, x_{t-p}$ is the conditional mean:

$$\hat{x}_t = E(x_t | x_{t-1}, \dots, x_{t-p}) = h(x_{t-1}, \dots, x_{t-p}), t > p$$

Alternatively, as discussed above we can use ANNs (see Appendix (15.1.1)). Lapedes [1987] showed that feedforward networks are NAR models for time series prediction. The predictor is given by the approximation of h as follow

$$\hat{x}_t = \hat{h}(x_{t-1}, \dots, x_{t-p})$$

with

$$\hat{h}(x_{t-1}, \dots, x_{t-p}) = \sum_{i=1}^{N_h} w_i f\left(\sum_{j=1}^p w_{ij} x_{t-j} + b_i\right) \quad (9.22)$$

Figure 19 shows the general architecture of the NAR as an extension of the $AR(p)$ in a feedforward network. The weights, $\{w_i\}$ and $\{w_{ij}\}$ in Equation (9.22) can be seen as knobs defining the input-output function, $\hat{h}$, of the network. N_h is the number of neurons in the second layer, f is a monotonic, smooth, and bounded function, b_i is a bias. The weights of the network, $\{w_i\}$ and $\{w_{ij}\}$ are estimated through supervised learning process given a time series sample $x_0, x_1, \dots, x_n$, and the input-target pairs in the training set are $((x_{t-1}, x_{t-2}, \dots, x_{t-p}), x_t)$, where $p > 0, t \geq p$.

More generally, to illustrate the nonlinearity of ANNs, we consider a multilayer network (see Appendix (15.1.5)) with $p = 1, \dots, P$ input-output training pairs where $p = P$ is the most recent observation, and assume a set of artificial neurons $\{f_{i,j,l}\}_{k=0}^K$ where the multilayer subscript $k = 0$ corresponds to the set of inputs $\{x_i\}_{i=1}^{N_0}$ and $k = K$ corresponds to the set of outputs $\{y_i\}_{i=1}^{N_K}$. For simplicity of exposition, we focus on a system with one hidden layer where the subscripts i and j represent the nodes on the input ($(K - 2)$ layer) and the $(K - 1)$ th hidden layer, and the subscript s represent the nodes on the k th layer, respectively. In that setting, the output $O_{s,K}^p(x, w)$ can be expressed as

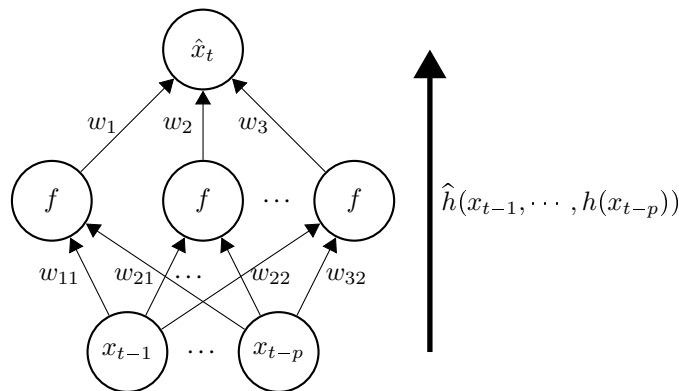


Figure 19: Feedforward network for nonlinear autoregressive model.

$$O_{s,K}^p(x, w) = g_{N_K} \left(\sum_{j=1}^{N_{K-1}} g_{N_{K-1}} \left(\sum_{i=1}^{N_{K-2}} O_{i,K-2}^p(x, w) w_{i,j}^{k-2} + b_{j,K-1} \right) w_{j,s}^{k-1} + b_{s,K} \right), s = 1, \dots, N_K \quad (9.23)$$

where $O_{i,K-2}^p(x, w) = x_{p,i}$ is the i th element of the vector input, and $g_{N_K}(\cdot)$ and $g_{N_{K-1}}(\cdot)$ are two possibly different functions. In the special case where $g_{N_K}(\cdot)$ is a linear function and $g_{N_{K-1}}(\cdot)$ is a sigmoid function, the system simplifies to

$$O_{s,K}^p(x, w) = \sum_{j=1}^{N_{K-1}} g_{N_{K-1}} \left(\sum_{i=1}^{N_{K-2}} O_{i,K-2}^p(x, w) w_{i,j}^{k-2} + b_{j,K-1} \right) w_{j,s}^{k-1} + b_{s,K}, s = 1, \dots, N_K$$

which can be compared to the ordinary least squares (OLS) regression

$$y_s^p = \alpha + \sum_{i=1}^{N_{K-2}} \beta_i x_{p,i} + \epsilon_s, s = 1, \dots, N_K$$

Hence, we see that the OLS regressor variables y_s^p are put through a transformation, or squashing function, given by $g_{N_{K-1}}(\cdot)$. By iterating within sample and examining the error terms, the neural network modify the input weights $\{w_{i,j}^{k-2}\}$ and the output weights $\{w_{j,s}^{k-1}\}$ to minimise within-sample measure for point prediction models. It can also maximise the classification rate of correct predictions for classification models.

It is not difficult to see that Equation (9.23) corresponds to the rate dynamics in Equation (9.14) with nonlinear G function. That is,

$$y = g_K \left(a + \sum_{i=1}^p \phi_i x_i + \sum_{j=1}^q \theta_j g_{K-1} \left(\alpha_j + \sum_{i \rightarrow j} \omega_{ij} x_i \right) \right)$$

where $x_1, \dots, x_p$ are input values, y is the output, for the q -hidden nodes. If the activation function g_K is linear, the feedforward neural network (FFNN) becomes an autoregressive moving average model with nonlinear moving average part. Hence, the FFNN (with one hidden layer) is a generalisation of $ARMA(p, q)$ models.

9.4 Accounting for time and earning profits

In traditional time series forecasting, the criteria for assessing model performance are error functions based on the goodness of fit of target and predicted values. Since neural networks are similar to conventional regression estima-

tors, except for their nonlinearity, error functions are also used to judge the goodness of fit of the model. However, normalised mean square error (NMSE) and other error functions are not the most appropriate measures in finance.

9.4.1 Time factor

For instance, weighting all data equally (ordinary least squares) are less accurate than discounted least squares, which weight the most recent data more heavily (see Makridakis et al. [1982]). Incorporating time factor to neural networks, Refenes et al. [1997] proposed the discounted least squares (DLS) neural network model. Assuming a feed-forward network with n input and a single output unit ($m = 1$) with k hidden layers, we let the ordinary least-squares criterion of the network be given by

$$E_{LS} = \frac{1}{2P} \sum_{p=1}^P (O_p - d_p)^2$$

where O_p and d_p denote the p th output and target value, respectively.

Remark 9.3 *In general, when considering a training set consisting of sequential data, or time-varying input, we represent them as $\{(\bar{x}_1, \bar{d}_1), \dots, (\bar{x}_T, \bar{d}_T)\}$ with T ordered pairs. To be consistent with the articles described we keep the P ordered pairs, and we let $p = P$ be the most recent observation.*

Refenes et al. [1997] proposed a simple modification to the error backpropagation procedure taking into account gradually changing input-output pairs. The procedure is based on the principle of discounted least-squares whereby learning is biased towards more recent observations with long term effects experiencing exponential decay through time. The cumulative error calculated by the DLS procedure is given by

$$E_{DLS} = \frac{1}{2P} \sum_{p=1}^P \phi(p)(O_p - d_p)^2$$

where $\phi(p)$ is an adjustment of the contribution of observation p to the overall error. They chose the simple sigmoidal decay function

$$\phi_{DLS}(p) = \frac{1}{1 + e^{a-bp}}$$

where $b = \frac{2a}{P}$ and a is the discount rate. The learning rule is derived in the usual way by repeatedly changing the weights by an amount proportional to

$$\frac{\partial E_{DLS}}{\partial W} = \phi_{DLS}(p) \frac{\partial E_{LS}}{\partial W}$$

since the discount factor $\phi(p)$ is a function of the recency of the observation which is independent of the actual order of pattern presentation within the learning process. Hence, the algorithm is not affected by randomising the order in which patterns are presented and can be applied to both batch and stochastic update rules. Using a sine wave with changing amplitude and a non-trivial application in exposure analysis of stock returns to multiple factors, Refenes et al. [1997] compared the performance of the two cost functions on a backpropagation network with a single hidden layer of 12 units and batch update. They showed that DLS was a more efficient procedure for weakly non-stationary data series.

9.4.2 Direction measures

Further, we are usually more interested in earning profits than in the quality of the forecast itself. When classifying returns, it is possible to use a direction measure to test the number of times a prediction neural network predicts the direction of the predicted return movement correctly. Merton [1981] proposed a modified direction given by

$$\text{Modified Direction} = \frac{\# \text{ of correct up predictions}}{\# \text{ of times index up}} + \frac{\# \text{ of correct down predictions}}{\# \text{ of times index down}} - 1$$

In general, direction is defined as the number of times the prediction followed the up and down movement of the underlying stock/index. Harvey et al. [2002] and Castiglione et al. [2001] proposed the following direction measure

$$\xi = \frac{1}{|T|} \sum_{t \in T} \mathcal{H}(R_t \hat{R}_t) + 1 - (|R_t| + |\hat{R}_t|)$$

where $R_t = \frac{P_t}{P_{t-1}}$ is the percentage return on the underlying at time step $t \in T$, $\hat{R}_t$ is the forecast percentage return, T is the number of days in the validation period, and $\mathcal{H}(\cdot)$ is the Heaviside function. Alternatively, we can write the direction measure as

$$\xi = \frac{1}{|T|} \sum_{t \in T} \mathcal{H}(\Delta P_t \Delta \hat{P}_t) + 1 - (|\Delta P_t| + |\Delta \hat{P}_{t+1}|)$$

where $\Delta P_t = P_t - P_{t-1}$ is the price change at step $t \in T$ and P_{t-1} is assumed to be known. These two direction measures provide a summary of how well the predicted time series and the actual ones move together at any point in time. However, we want to implement measurement errors taking into account the simultaneous behaviour of trend and magnitude within the trend. Hence, to reflect this point when evaluating the performance of a forecasting model, Yao et al. [1996] used the correctness of trend and a paper profit. They proposed a profit based adjusted weight factor for backpropagation network training by adding a factor containing profit, direction, and time information to the error function.

9.4.3 Time dependent direction profit

In order to take profit gain into account, Caldwell [1995] proposed a weighted directional symmetry (WDS) function which penalise more heavily the incorrectly predicted directions than the correct ones. The cumulative error function is given by

$$E_{WDS} = \frac{100}{P} \sum_{p=1}^P \phi(p) |O_p - d_p|$$

with

$$\phi(p) = \begin{cases} g & \text{if } \Delta d_p \times \Delta O_p \leq 0 \\ h & \text{otherwise} \end{cases}$$

where $\Delta d_p = d_p - d_{p-1}$ and $\Delta O_p = O_p - O_{p-1}$, and g and h are constants or some function of d_p . The values $g = 1.5$ and $h = 0.5$ were suggested. Yao et al. [1996] argued that the profit driven procedure was not only a function of the direction, but also of the amount of change. That is, the penalty on WDS weights should be increased in case of a wrongly forecasted direction for a big change of values, and the penalty should be further reduced if the direction is correctly forecasted for a big change in value. They proposed a modified directional profit (DP) adjustment factor defined as

$$\phi_{DP}(p) = \begin{cases} a_1 & \text{if } \Delta d_p \times \Delta O_p > 0 \text{ and } |\Delta d_p| \leq \sigma \\ a_2 & \text{if } \Delta d_p \times \Delta O_p > 0 \text{ and } |\Delta d_p| > \sigma \\ a_3 & \text{if } \Delta d_p \times \Delta O_p < 0 \text{ and } |\Delta d_p| \leq \sigma \\ a_4 & \text{if } \Delta d_p \times \Delta O_p < 0 \text{ and } |\Delta d_p| > \sigma \end{cases}$$

where σ is a threshold for the changes in sample values generally taken as the standard deviation of the training set

$$\sigma^2 = \frac{1}{P} \sum_{p=1}^P (d_p - \mu_d)^2$$

where μ_d is the mean of the target series. The values $a_1 = 0.5$, $a_2 = 0.8$, $a_3 = 1.2$, and $a_4 = 1.5$ were suggested. While the DLS model focus on a time factor $\phi_{DLS}(p)$ and the DP model focus on a profit factor $\phi_{DP}(p)$, Yao et al. [1998] [2000] combined the two approaches obtaining a time dependent directional profit (TDP) model given by

$$\phi_{TDP}(p) = \phi_{DLS}(p) \times \phi_{DP}(p)$$

The three models, DLS, DP, and TDP were tested and compared to the benchmarked OLS model on four major Asian stock indices and the US Dow Jones Industrials Index (DJ). They considered a one hidden layer network with learning rate $\eta = 0.25$ and momentum rate $\gamma = 0.9$. The structure of the neural network was $30 - 10 - 1$, that is, 30 input, 10 nodes in the hidden layer and one output. Thus, 30 consecutive days of data are fed to the network to forecast the index of the following day. Considering a paper profit measure for the model performance, they showed that the preference of DLS over OLS was of 60%, and that the preference of DP over OLS was of 80%. In this combined model, they obtained up to 27.6% excess annual return above the OLS model with at least 2.8% excess annual return in the worst case. Lu Dang Khoa et al. [2006] argued that big changes in prices between two consecutive periods are rare to occur, leading to change between two consecutive prices smaller than the standard deviation of the price. They replaced the inequality $|\Delta d_p| > \sigma$ with $\frac{\Delta d_p}{d_{p-1}} > a_5$ and obtained the following adjustment factor

$$\phi_{DPN}(p) = \begin{cases} a_1 & \text{if } \Delta d_p \times \Delta O_p > 0 \text{ and } \frac{\Delta d_p}{d_{p-1}} < a_5 \\ a_2 & \text{if } \Delta d_p \times \Delta O_p > 0 \text{ and } \frac{\Delta d_p}{d_{p-1}} > a_5 \\ a_3 & \text{if } \Delta d_p \times \Delta O_p < 0 \text{ and } \frac{\Delta d_p}{d_{p-1}} < a_5 \\ a_4 & \text{if } \Delta d_p \times \Delta O_p < 0 \text{ and } \frac{\Delta d_p}{d_{p-1}} > a_5 \end{cases}$$

with $a_5 = 0.05$. They used that model to predict the S&P 500 stock index one month in the future without much improvements compared to the traditional FFN algorithm. They considered a mixed of input based on fundamental factors such as the prices of oil, the interest rates, inflation, and technical indicators such as volatility, relative strength index, directional index etc.

10 Statistical properties of financial time series

10.1 Optimisation problem

10.1.1 Characterisation in terms of eigenvalues

We are now going to redefine positive semidefinite matrices using slightly different notations.

Definition 10.1 A real symmetric $n \times n$ matrix M is positive semidefinite (denoted $M \geq 0$) if $z^\top M z \geq 0$ for all $z \in \mathbb{R}^n$. It is positive definite (denoted $M > 0$) if $z^\top M z > 0$ for all nonzero $z \in \mathbb{R}^n$

Example:

As an example, we consider any random vector $X \in \mathbb{R}^n$, and let $\mu = E[X]$ and $S = E[(X - \mu)(X - \mu)^\top]$ be its mean and covariance, respectively. Then, $S \geq 0$ because for any $z \in \mathbb{R}^n$, we have

$$z^\top S z = z^\top E[(X - \mu)(X - \mu)^\top] z = E[(z^\top (X - \mu))((X - \mu)^\top z)] = E[(z \cdot (X - \mu))^2] \geq 0$$

Further, (semi)definiteness is easily characterised in terms of eigenvalues.

Theorem 10.1 Let M be a real symmetric $n \times n$ matrix. Then,

1. M is positive semidefinite if and only if all its eigenvalues $\lambda_i \geq 0$.
2. M is positive definite if and only if all its eigenvalues $\lambda_i > 0$.

We will only prove the case (1). We let $\lambda_1, \dots, \lambda_n$ be the eigenvalues of M , with corresponding eigenvectors $u_1, \dots, u_n$. If we suppose that $M \geq 0$, then for all i , $\lambda_i = u_i^\top M u_i \geq 0$. Conversely, suppose that all the $\lambda_i \geq 0$, then for any $z \in \mathbb{R}^n$, we have

$$z^\top M z = z^\top \left(\sum_{i=1}^n \lambda_i u_i u_i^\top \right) z = \sum_{i=1}^n \lambda_i (z \cdot u_i)^2 \geq 0$$

10.1.2 Quadratic optimisation problem

Eigenvalues constitute the optimal solution of a very basic quadratic optimisation problem.

Theorem 10.2 *Rayleigh quotient theorem*

Let M be a real symmetric $n \times n$ matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ and corresponding eigenvectors $u_1, \dots, u_n$. Then,

$$\begin{aligned} \max_{\|z\|=1} z^\top M z &= \max_{z \neq 0} \frac{z^\top M z}{z^\top z} = \lambda_1 \\ \min_{\|z\|=1} z^\top M z &= \min_{z \neq 0} \frac{z^\top M z}{z^\top z} = \lambda_n \end{aligned}$$

and these are realised at $z = u_1$ and $z = u_n$, respectively.

Proof: We use the spectral decomposition $M = Q \Lambda Q^\top$ to prove that result. We have

$$\begin{aligned} \max_{z \neq 0} \frac{z^\top M z}{z^\top z} &= \max_{z \neq 0} \frac{z^\top Q \Lambda Q^\top z}{z^\top Q Q^\top z} \text{ since } Q Q^\top = I \\ &= \max_{y \neq 0} \frac{y^\top \Lambda y}{y^\top y} \text{ writing } y = Q^\top z \\ &= \max_{y \neq 0} \frac{\lambda_1 y_1^2 + \dots + \lambda_n y_n^2}{y_1^2 + \dots + y_n^2} \leq \lambda_1 \end{aligned}$$

We obtain equality in the last step when $y = e_1$, that is, $z = Q e_1 = u_1$. The argument for the minimum is identical. End of proof.

Theorem (10.2) tells us the direction of maximum variance is u_1 , and that of minimum variance is u_n .

Example:

Following our previous example, $z^\top S z$ represents the variance of X in the direction z . That is,

$$\text{Var}(z^\top X) = E[(z^\top (X - \mu))^2] = E[z^\top (X - \mu)(X - \mu)^\top z] = z^\top S z$$

10.1.3 Dimensionality reduction

We are now interested in the k -dimensional subspace of $\mathbb{R}^n$ having the most variance. We think of a linear projection from $\mathbb{R}^n$ to $\mathbb{R}^k$ as a function $x \rightarrow P^\top x$, where $P^\top$ is a $k \times n$ matrix with $P^\top P = I_k$. That is, the rows of the projection matrix are orthogonal. When a random vector $X \in \mathbb{R}^n$ is subjected to such a projection, the resulting k -dimensional vector has covariance

$$\text{Cov}(P^\top X) = E[P^\top (X - \mu)(X - \mu)^\top P] = P^\top S P$$

In order to summarise the variance with a single number (rather than a matrix), we use the trace of the matrix and we write $\text{Var}(P^\top X) = \text{Tr}(P^\top S P)$. This is also equal to $E\|P^\top X - P^\top \mu\|^2$. We can now determine the projection $P^\top$ maximising this variance.

Theorem 10.3 *Let M be a real symmetric $n \times n$ matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ and corresponding eigenvectors $u_1, \dots, u_n$. Pick any $k \leq n$. Then,*

$$\begin{aligned} \max_{P \in \mathbb{R}^{n \times k}, P^\top P = I} \text{Tr}(P^\top M P) &= \lambda_1 + \dots + \lambda_k \\ \min_{P \in \mathbb{R}^{n \times k}, P^\top P = I} \text{Tr}(P^\top M P) &= \lambda_{n-k+1} + \dots + \lambda_n \end{aligned}$$

These are realised when the columns of P span the k -dimensional subspace by $\{u_1, \dots, u_k\}$ and $\{u_{n-k+1}, \dots, u_n\}$, respectively.

Proof: We prove this result for the maximum, the other case being symmetric. Let $p_1, \dots, p_k$ denote the columns of P . Then,

$$\text{Tr}(P^\top M P) = \sum_{i=1}^k p_i^\top M p_i = \sum_{i=1}^k p_i^\top \left(\sum_{j=1}^n \lambda_j u_j u_j^\top \right) p_i = \sum_{j=1}^n \lambda_j \sum_{i=1}^k (p_i \cdot u_j)^2$$

which we will show is at most $\lambda_1 + \dots + \lambda_k$. We let z_j be defined by $\sum_{i=1}^k (p_i \cdot u_j)^2$, which is nonnegative. We can show that $\sum_{j=1}^n z_j = k$ and that each $z_j \leq 1$. First, we have

$$\sum_j z_j = \sum_{i=1}^k \sum_{j=1}^n (p_i \cdot u_j)^2 = \sum_{i=1}^k \sum_{j=1}^n p_i^\top u_j u_j^\top p_i = \sum_{i=1}^k p_i^\top Q Q^\top p_i = \sum_{i=1}^k \|p_i\|^2 = k$$

since $\sum_{j=1}^n u_j u_j^\top = Q Q^\top$. To find an upper bound for z_j , we expand the k orthogonal vectors $p_1, \dots, p_k$ to a full orthonormal basis $p_1, \dots, p_n$ of $\mathbb{R}^n$. Then,

$$z_j = \sum_{i=1}^k (p_i \cdot u_j)^2 \leq \sum_{i=1}^n (p_i \cdot u_j)^2 = \sum_{i=1}^n u_j^\top p_i p_i^\top u_j = \|u_j\|^2 = 1$$

Thus,

$$\text{Tr}(P^\top M P) = \sum_{j=1}^n \lambda_j z_j \leq \lambda_1 + \dots + \lambda_k$$

and equality holds when $p_1, \dots, p_k$ span space as $u_1, \dots, u_k$.
End of proof.

10.1.4 The PCA

Principal component analysis (PCA) is a statistical procedure that uses an orthogonal transformation to convert a set of observations of possibly correlated variables into a set of values of linearly uncorrelated variables called principal components. PCA aims at constructing a low-dimensional representation of the data while maintaining the maximal variance and covariance structure of the data (see Jolliffe [1986]). It is the simplest of the true eigenvector-based multivariate analyses. If a multivariate dataset is visualised as a set of coordinates in a high-dimensional data space (1

axis per variable), PCA can supply the user with a lower-dimensional picture, a projection of this object when viewed from its most informative viewpoint. It is closely related to factor analysis, which incorporates more domain specific assumptions about the underlying structure and solves eigenvectors of a slightly different matrix.

10.1.4.1 Description Let $X \in \mathbb{R}^n$ be a random vector. We want to find the single direction capturing as much as possible of the variance of X . That is, we want the direction $p \in \mathbb{R}^n$ such that $\|p\| = 1$, so as to maximise $\text{Var}(p^\top X)$.

Theorem 10.4 *The solution to this optimisation problem is to make p the principal eigenvector of $\text{Cov}(X)$.*

Again, we let $\mu = E[X]$ and $S = \text{Cov}(X)$. For any $p \in \mathbb{R}^n$, the projection $p^\top X$ has mean $E[p^\top X] = p^\top \mu$ and variance

$$\text{Var}(p^\top X) = E[(p^\top X - p^\top \mu)^2] = E[p^\top (X - \mu)(X - \mu)^\top p] = p^\top S p$$

By Theorem (10.2), this is maximised, over all unit-length p , when p is the principal eigenvector of S .

Likewise, the k -dimensional subspace capturing as much as possible of the variance of X is simply the subspace spanned by the top k eigenvectors of $\text{Cov}(X)$. If we call them $u_1, \dots, u_k$, the projection onto these eigenvectors is called Principal Component Analysis (PCA). It can be used to reduce the dimension of the data from n to k as follow

- compute the mean μ and covariance matrix S of the data X .
- compute the top k eigenvectors $u_1, \dots, u_k$ of S .
- project $X \rightarrow P^\top X$, where $P^\top$ is the $k \times n$ matrix whose rows are $u_1, \dots, u_k$.

10.1.4.2 Affine approximation Another property of PCA is that it is the k -dimensional affine subspace best approximating X , in the sense that the expected squared distance from X to the subspace is minimised. Formally, a k -dimensional affine subspace is given by a displacement $p_0 \in \mathbb{R}^n$ and a set of orthonormal basis vectors $p_1, \dots, p_k \in \mathbb{R}^n$. The subspace itself is then $\{p_0 + \alpha_1 p_1 + \dots + \alpha_k p_k : \alpha_i \in \mathbb{R}\}$. The projection of $X \in \mathbb{R}^n$ onto this subspace is $P^\top X + p_0$, where $P^\top$ is the $k \times n$ matrix whose rows are $p_1, \dots, p_k$. Thus, the expected squared distance from X to this subspace is $E\|X - (P^\top X + p_0)\|^2$. We can find the subspace for which this is minimised.

Theorem 10.5 *Let μ and S denote the mean and covariance of X , respectively. The solution of this optimisation problem is to choose $p_1, \dots, p_k$ to be the top k eigenvectors of S and to set $p_0 = (I - P^\top)\mu$.*

Fixing any matrix P , the choice p_0 minimising $E\|X - (P^\top X + p_0)\|^2$ is $p_0 = E[X - P^\top X] = (I - P^\top)\mu$. The cost function for optimising P is

$$E\|X - (P^\top X + p_0)\|^2 = E\|(I - P^\top)(X - \mu)\|^2 = E\|(X - \mu)\|^2 - E\|P^\top(X - \mu)\|^2$$

where the second step is an invocation of the Pythagorean theorem. Hence, we need to maximise $E\|P^\top(X - \mu)\|^2 = \text{Var}(P^\top X)$, and we know the solution of that optimisation problem.

We now want to find the k -dimensional projection minimising the expected distortion in interpoint distances. Formally, we want to find the $k \times n$ projection matrix $P^\top$ (with $P^\top P = I_k$), such that, for i.i.d. random vectors X and Y , the expected squared distortion $E[\|X - Y\|^2 - E\|P^\top X - P^\top Y\|^2]$ is minimised.

Theorem 10.6 *The solution to this problem is to make the rows of $P^\top$ the top k eigenvectors of $\text{Cov}(X)$.*

We want to maximise

$$E\|P^\top X - P^\top Y\|^2 = 2E\|P^\top X - P^\top \mu\|^2 = 2\text{Var}(P^\top X)$$

and we are back to our original problem.

Note, this is not the same as finding the linear transformation $P^\top$ minimising $E\|X - Y\|^2 - E\|P^\top X - P^\top Y\|^2$.

10.1.4.3 k-means clustering We can now introduce the notion of k-means clustering. Suppose that for random vector $X \in \mathbb{R}^n$, the optimal k-means centers are $\mu_1^*, \dots, \mu_k^*$ with cost

$$C_{opt} = E\|X - (\text{nearest } \mu_i^*)\|^2$$

Instead, we want to estimate the quality of projecting X into the k-dimensional PCA subspace, and find the best k centers $\mu_1, \dots, \mu_k$ in that subspace.

Theorem 10.7

$$\text{cost}(\mu_1, \dots, \mu_k) \leq 2C_{opt}$$

Without loss of generality, $E[X] = 0$ and the PCA mapping is $X \rightarrow P^\top X$. Since $\mu_1, \dots, \mu_k$ are the best centers for $P^\top X$, it follows that

$$E\|P^\top X - (\text{nearest } \mu_i)\|^2 \leq E\|P^\top(X - (\text{nearest } \mu_i^*))\|^2 \leq E\|X - (\text{nearest } \mu_i^*)\|^2 = C_{opt}$$

Let $X \rightarrow A^\top X$ denotes the projection onto the subspace $\mu_1^*, \dots, \mu_k^*$. From our earlier result on approximating affine subspaces, we know that $E\|X - P^\top X\|^2 \leq E\|X - A^\top X\|^2$. Thus,

$$\begin{aligned} \text{cost}(\mu_1, \dots, \mu_k) &= E\|X - (\text{nearest } \mu_i)\|^2 \\ &= E\|P^\top X - (\text{nearest } \mu_i)\|^2 + E\|X - P^\top X\|^2 \text{ Pythagorean theorem} \\ &\leq C_{opt} + E\|X - A^\top X\|^2 \\ &\leq C_{opt} + E\|X - (\text{nearest } \mu_i^*)\|^2 = 2C_{opt} \end{aligned}$$

10.2 Introduction to wavelet theory

Fitting financial time series is a difficult task as they are not stationary and contain noise. It is possible to get rid of this noise by using a filter. However, when denoising a signal we need to preserve the relevant multi-scaled information, which can be done with wavelets.

We present a short introduction to wavelets and provide some examples on denoising techniques.

10.2.1 Some definitions

Wavelet theory overcomes some problems linked to Fourier theory (such as stationarity) by generalising the decomposition to a time-frequency domain. The word wavelet (ondelette) were used by Morlet and Grossmann in the early 1980s. Started with the work of Haar in the early 20th century, Zweig discovered of the continuous wavelet transform in 1975, which were extended by Goupillaud, Grossmann and Morlet. Stromberg introduced discrete wavelets (1983) and Daubechies presented the orthogonal wavelets with compact support (1988). Wavelets were first used in physics to characterise the Brownian motion and have been widely used in diverse tasks related to financial time series analysis.

Definition 10.2 Wavelet

A wavelet is a square integrable function $\psi_{s,\tau} : t \in \mathbb{R} \rightarrow \mathbb{R}$, where $(s, \tau) \in \mathbb{R}^{*+} \times \mathbb{R}$, such that $\psi_{s,\tau} \in L^2(\mathbb{R})$ and $\int_{-\infty}^{+\infty} \psi_{s,\tau}(t)dt = 0$.

Remark 10.1 Wavelets derived from a mother wavelet Ψ in the following way: $\forall t \in \mathbb{R}, \psi_{s,\tau}(t) = \frac{1}{\sqrt{s}}\Psi(\frac{t-\tau}{s})$. In that sense, a mother wavelet generates a subgroup Λ of the affine transformation group of $\mathbb{R}^n$.

There are many wavelet families, each of them gathering wavelets having common properties.

10.2.1.1 Continuous Wavelet As in Fourier theory, both continuous and discrete wavelet transforms exist. We first define the continuous wavelet transform (CWT) of a signal f as follows:

$$\forall (s, \tau) \in \mathbb{R}^{*+} \times \mathbb{R} \quad g(s, \tau) = \int_{-\infty}^{+\infty} f(t) \psi_{s, \tau}^*(t) dt$$

$*$ stands for the complex conjugate, τ is a translation factor and s is a dilatation factor. To recover f from g , the following equation is used:

$$\forall t \in \mathbb{R} \quad f(t) = \frac{1}{C} \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} \frac{1}{|s|^2} g(s, \tau) \psi_{s, \tau}(t) ds d\tau$$

where $C = \int_{-\infty}^{+\infty} \frac{|\hat{\Psi}(x)|}{|x|} dx$, $\hat{\Psi}$ being the Fourier transform of the mother wavelet Ψ .

An example of such wavelet is the Mexican hat wavelet. The continuous wavelet transform defines a priori an infinite number of coefficients which cannot be used in practice. For that reason, we will only make use of discrete wavelet transform (DWT) introduced below.

10.2.1.2 Discrete Wavelet Consider a discrete time set $\mathbb{T} = \llbracket 0, T-1 \rrbracket$. We define a discrete wavelet on this set as follows:

$$\forall t \in \mathbb{T} \quad \psi_{m, n}(t) = s_0^{-m/2} \psi(s_0^{-m} t - n\tau_0)$$

where s_0 and τ_0 are constant. Then, the scaling and wavelet functions of DWT are defined as follows:

$$\begin{aligned} \phi(2^m t) &= \sum_{i=1}^n h_{m+1}(i) \phi(2^{m+1} t - i) \\ \psi(2^m t) &= \sum_{i=1}^n g_{m+1}(i) \phi(2^{m+1} t - i) \end{aligned}$$

where $(h_{m+1}(i))_{i \in \llbracket 1, n \rrbracket}$ and $(g_{m+1}(i))_{i \in \llbracket 1, n \rrbracket}$ are respectively the coefficient sequences of ϕ and ψ . From these functions, a discrete signal x can be written as:

$$x(t) = \sum_{i=1}^n \lambda_{m-1}(i) \phi(2^{m-1} t - i) + \sum_{i=1}^n \gamma_{m-1}(i) \psi(2^{m-1} t - i)$$

If $\{\psi_{m, n} : m, n \in \mathbb{Z}\}$ defines an orthonormal basis of $L^2(\mathbb{R})$, the following formula is used in order to recover the signal x .

$$\forall t \in \mathbb{T} \quad x(t) = \sum_{m \in \mathbb{Z}} \sum_{n \in \mathbb{Z}} \langle x, \psi_{m, n} \rangle \cdot \psi_{m, n}(t)$$

Remark 10.2 Orthonormality is an interesting property but not necessarily required for wavelets' construction. Compact support is also an useful property wavelets have as they can calibrate data efficiently, though it is not required neither.

Remark 10.3 Using a set of wavelets having the orthonormal property makes it easier to recover a discrete signal from its coefficients $(\langle x, \psi_{m, n} \rangle)_{m, n}$. This is the case of Haar wavelets for example.

Wavelets take care of temporal characteristics of the regular factors described earlier as they are defined in a two dimensional domain: frequency and time resolution combinaison. Wavelets are indeed small waves located at different times and therefore give significant information about both time and frequency domains. This new degree of freedom provides the ability to capture trends in a local fashion.

10.2.2 Decomposition schemes

We introduce three decomposition schemes standing for different approaches of the wavelet transform introduced earlier.

10.2.2.1 Discrete Wavelet Transform DWT is the most basic wavelet decomposition, Figure (20) shows how it can be done. H, L and H', L' respectively stands for high-pass, low-pass filters for wavelet decomposition and reconstruction. In the decomposition phase, L and H remove respectively the higher frequencies and the lower frequencies to give an approximation and a detail of signal x . The result is then downsampled by 2. The right part of the figure denotes the reverse process and is known as the reconstruction phase.

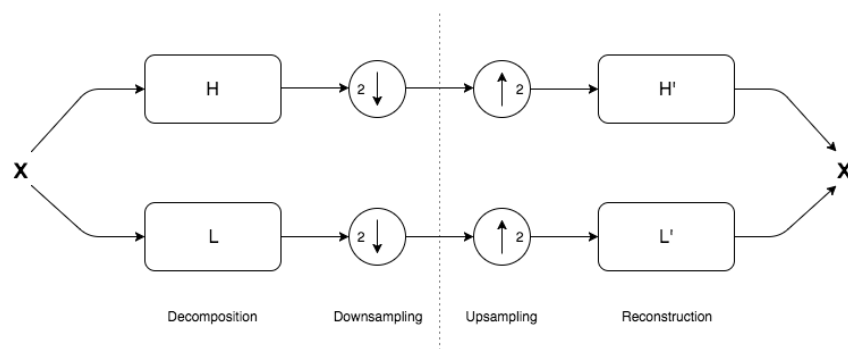


Figure 20: DWT Filter Scheme

More generally, we can repeat this process n times by a cascade algorithm which decomposes and downsamples the latter approximation. The following figure describes the process. A signal x can then be decomposed at level n in the following fashion:

$$x(t) = A_n(t) + \sum_{k=1}^n D_k(t) \quad (10.24)$$

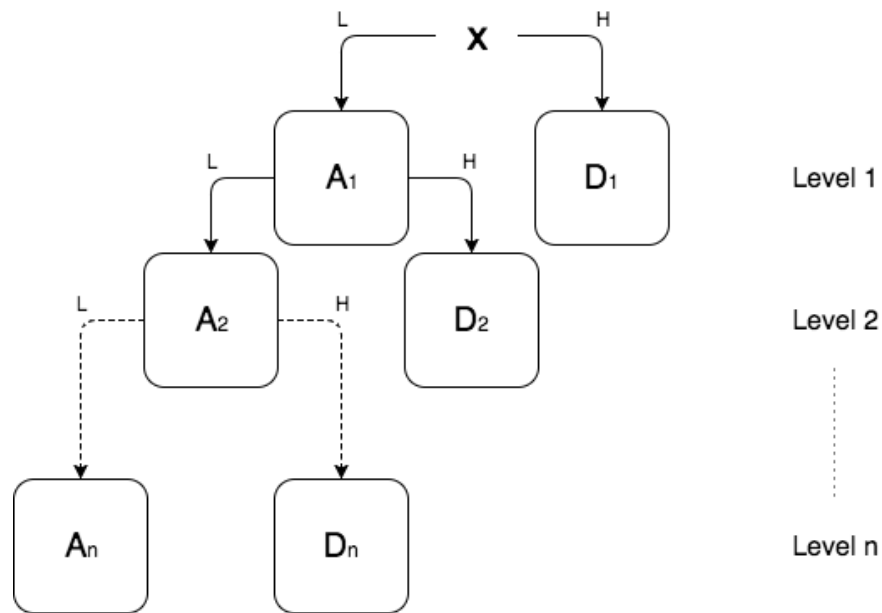


Figure 21: DWT Decomposition Tree

The main drawback of DWT is to be time-variant. For example, if we consider a periodic signal x with period τ , the DWT of x on $\llbracket t - \tau, t \rrbracket$ is in general not the DWT of x on $\llbracket t, t + \tau \rrbracket$ due to the downsampling. This property being desirable, researchers have developed a new type of wavelet transform called stationary wavelet transform (SWT) which guarantees time-invariance.

10.2.2.2 Stationary Wavelet Transform In the DWT framework the downsampling meant choosing even or odd indexed elements at every decomposition step. However, in general, we could choose between even and odd at every decomposition which would lead to many possible combinaisons. The cornerstone of SWT is to average these denoised signals. We choose 0 to be coinciding with an even downsampling and 1 with an odd one.

Definition 10.3 ϵ -decimated DWT

Let N be a decomposition level of a signal x and $\epsilon = (\epsilon_1, \dots, \epsilon_N)$ a sequence of integers such that $\forall n \in \llbracket 1, N \rrbracket$, $\epsilon_n \in \{0, 1\}$. We say that the DWT $\chi = (A_N, D_N, \dots, D_1)$ is ϵ -decimated DWT if $\forall n \in \llbracket 1, N \rrbracket$, ϵ_n corresponds to the downsampling choice of χ at time n .

It is possible to calculate all the ϵ -decimated DWTs for a given finite signal x by computing χ for every possible sequence ϵ . However, doing so is very computationally inefficient as 2^N DWT need to be done for a given decomposition level N . Another way to do this is to avoid downsampling and perform an upsampling on the original filters H and L .

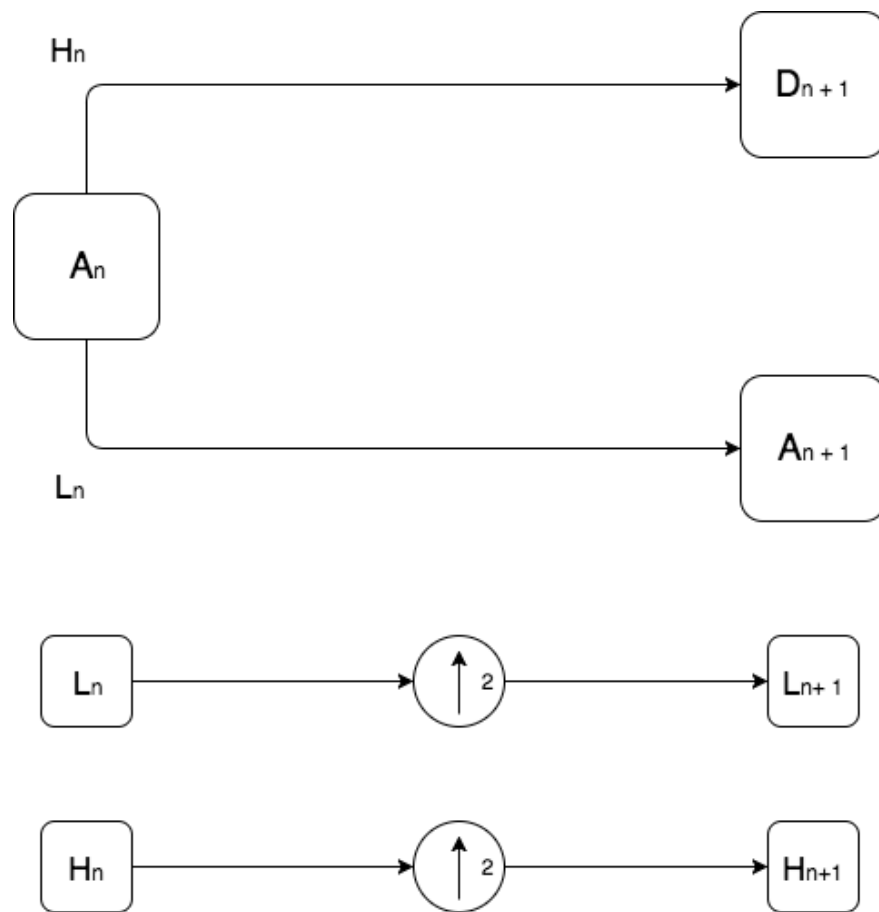


Figure 22: SWT calculation at level n

By following this process we end up with a time-invariant wavelet transform χ . In the next part, we describe a more general decomposition which is worth being investigated.

10.2.2.3 Wavelet Packet Transform WPT can be seen as an extension of DWT. The latter decomposes only the approximation coefficients while WPT decomposes both the approximation and detail coefficients. This leads to more complex and somehow more flexible decomposition. Figure (23) describes its process:

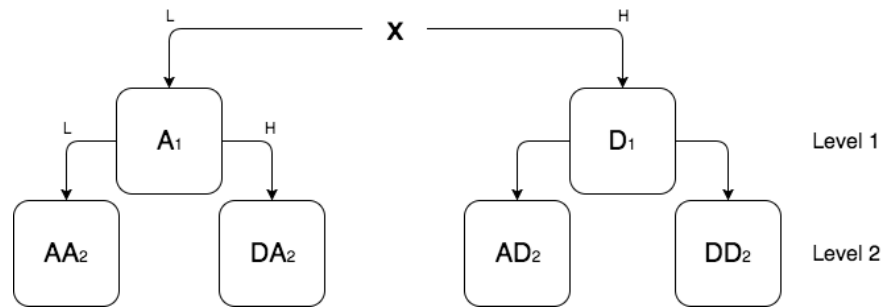


Figure 23: WPT Decomposition Tree

However, this process leads to 2^N coefficients for a given decomposition level N and thus can be more cumbersome in terms of storage and computation processing.

10.2.3 Denoising techniques

In this section we explain how DWT, SWT and WPT can be used to denoise a given signal. We say that a signal is noised when it suffers from unwanted or unknown modifications. Although we can have an intuitive understanding of noise, particularly in financial time series, there cannot be clear definition of it. Consequently, researchers often model it. A common model used in the research papers is the Gaussian white noise which has the characteristic to have the same power spectral density at every bandwidth frequency.

Definition 10.4 Power spectral density

Let x a signal and X its Fourier transform. We define the power spectral density (PSD) of the signal x as $\Gamma_x = |X|^2 * T$.

Definition 10.5 Gaussian white noise

A signal x is called a Gaussian white noise process if x is a stationary Gaussian process such that its mean $\mu_x = 0$ and its PSD is constant.

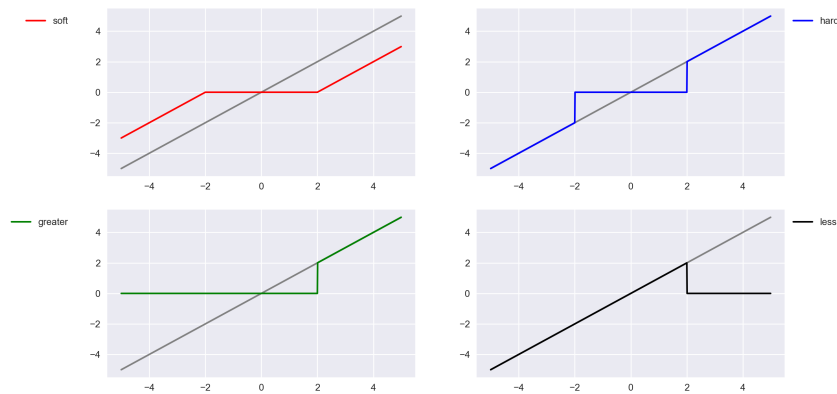
From theses definition we state our goal: from a noised signal x defined on a domain D and carrying a Gaussian white noise ϵ , we wish to find the true signal x^* such that:

$$\forall t \in D \quad x(t) = x^*(t) + \epsilon(t) \quad (10.25)$$

In the previous section, we have introduced the concept of wavelet decomposition leading to a set of approximation and detail coefficients. As the noise in a signal is generally contained in the detail ones, a common procedure consists in setting the smallest coefficients to zero. Given a threshold $\lambda \in \mathbb{R}^+$, we sum up the many possibilities below.

soft	hard
$x \rightarrow \begin{cases} x - \lambda & \text{if } x > \lambda \\ 0 & \text{if } x \leq \lambda \\ x + \lambda & \text{if } x < -\lambda \end{cases}$	$x \rightarrow \begin{cases} 0 & \text{if } x < \lambda \\ x & \text{otherwise} \end{cases}$
greater	less
$x \rightarrow \begin{cases} 0 & \text{if } x < \lambda \\ x & \text{otherwise} \end{cases}$	$x \rightarrow \begin{cases} 0 & \text{if } x > \lambda \\ x & \text{otherwise} \end{cases}$

Table 5: Threshold Function Definitions

Figure 24: Thresholding functions ($\lambda = 2$)

It is a well-known fact that soft thresholding provides smoother results when comparing to other ones. However, hard thresholding provides better edge preservation.

Algorithm (1) defines our filtering process for Discrete Wavelet Transform and has been used in our experiments to denoise a signal.

Filtering processes for other wavelet decomposition methods are very similar, the general idea being thresholding detail coefficients.

Algorithm 1 DWT filtering process

Require: get noised signal x **Require:** set wavelet name wlt **Require:** set decomposition level n **Require:** set thresholding function f and level λ 1: $[A(n), D_n, D_{n-1}, \dots, D_1] \leftarrow \text{decomposition}(x, wlt, n)$ 2: **for** $i = 1$ **to** n **do**3: $D_i \leftarrow f(D_i, \lambda)$ 4: **end for**5: $x^* \leftarrow \text{reconstruction}([A, D], wlt)$ 6: **return** x^*

10.3 Fractal time series

A self-similar process is characterised by its Hurst coefficient H (see Hurst [1951]). The Brownian motion, the L-stable process (see Mandelbrot [1963]), and the fractional Brownian motion (fBm) are the main examples of self-affine processes in finance. The value of the Hurst Exponent varies between 0 and 1, with $H = \frac{1}{2}$ implying a random walk or an independent process (see Figure 39). Note, for $H \neq \frac{1}{2}$ we can observe two phenomenon

1. the Noah effect: rare events, fat tail distributions
2. the Joseph effect: observe long stretches of time when the process is above or below the mean

These phenomenon correspond respectively to

1. $0 < H < \frac{1}{2}$, short term memory: we have anti-persistence (or ergodicity) where the process covers less distance than a random walk. It is often referred as mean reverting, meaning that if the system has been up in the previous period, it is more likely to be down in the next period. The converse being true. Anti-persistent time series exhibit higher noise and more volatility than a random series because it consists of frequent reversals.
2. $\frac{1}{2} < H \leq 1$ long term memory, or long range dependence (LRD): we have persistence (or trend-reinforcing) where the process covers more distance than a random walk. A persistent series has long memory effects such that the trend at a particular point in time affects the remainder of the time series. If the series has been up (down) in the last period, then the chances are that it will continue to be positive (negative) in the next period.

Figure (40) illustrates the role of the Hurst exponent. If our goal is to forecast returns from a time series by learning some of its behaviour in the hope it repeats in the future, it does not make much sense to consider returns in the case (1). We are left with focusing on momentum (case (2)).

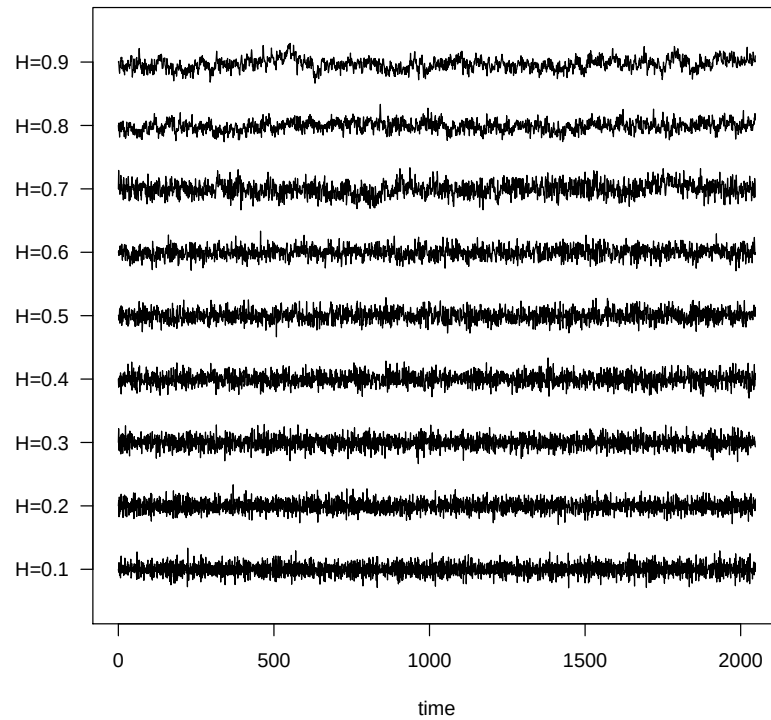
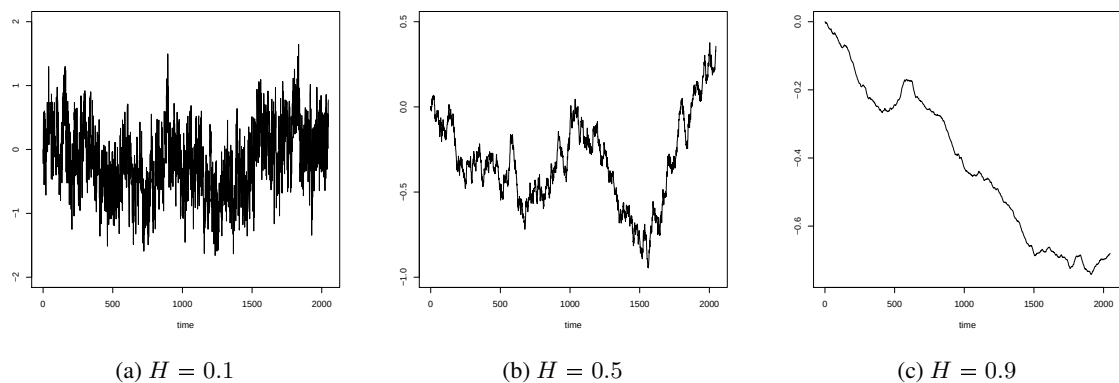


Figure 25: Range of Hurst exponents between 0.1 and 0.9

Figure 26: Cumulative observations for different values of H

The fractal dimension D of a time series measures how jagged the time series is. Mandelbrot showed that the Hurst Exponent, H , is related to the fractal dimension D for a self-similar surface in n -dimensional space by the relation

$$D = n + 1 - H \quad (10.26)$$

where $0 \leq H \leq 1$. While the fractal dimension of a line is 1.0 and the fractal dimension of a geometric plane is 2.0, the fractal dimension of a random walk is half way between the line and the plane at 1.5. A value of $\frac{1}{2} < H \leq 1$ results in a fractal dimension closer to a line, so that a persistent time series would result in a smoother, less jagged line than a random walk. Similarly, for $0 < H < \frac{1}{2}$, a time series is more jagged than a random series, or has more reversals.

The Hurst Exponent can be estimated by

$$H = \frac{\log(\frac{R}{S})}{\log(T)}$$

where T is the duration of the sample data, and $\frac{R}{S}$ is the corresponding value of the rescaled range.

Persistent series are fractional Brownian motion, or biased random walk with the strength of the bias depending on how far H is above $\frac{1}{2}$. For $H \neq \frac{1}{2}$ each observation carry a long-term memory, theoretically lasting forever. A system exhibiting Hurst statistics is the result of a long stream of interconnected events where time is important as the future depends on the past.

Since the development of the rescaled range analysis (R/S), a large number of fractal quantification methods called Wavelet Analysis (WA) and Detrending Methods (DMs) have been proposed to accomplish accurate and fast estimates of the Hurst exponent H in order to investigate correlations at different scales in dimension one. While the R/S analysis makes the rough approximation that the trend is the average in each segment under consideration, the former uses spectral analysis, and the latter considers more advanced methods to detrend the series. Examples of the latter are detrended fluctuation analysis (DFA) (see Peng et al. [1994]) and detrending moving average analysis (DMA). There exists many more methods for estimating the Hurst exponent such as the aggregated variance, the differenced variance, absolute values, boxed periodigram, Higuchi, modified periodigram, Whittle and local Whittle methods (see Fox et al. [1986]).

Wavelet Transforms can be used to characterise the scaling properties of self-similar fractal and multifractal objects (see Muzy et al. [1991]). For instance, Abry et al. [1993] [1998] proposed a methodology for estimating the Hurst exponent.

10.4 Multifractal time series

10.4.1 Multifractal analysis

As explained by Kantelhardt et al. [2002], many series do not exhibit a simple monofractal scaling behaviour described with a single scaling exponent. Alternatively, the scaling behaviour may be more complicated, so that different scaling exponents are required for different part of the series. For instance, the scaling behaviour in the first half of the series differ from that in the second half. In complex system, such different scaling behaviour can be observed for many interwoven fractal subsets of the time series, in which case a multitude of scaling exponents is required for a full description of the scaling behaviour, and a multifractal analysis must be applied. Thus, multifractality, or anomalous scaling, allows for a richer variation of the behaviour of a process across different scales. This variability of scaling laws can be translated into the variability of the Hurst index, which is no-longer constant. Abry et al. [2002] and Wendt [2008] gave a synthetic overview of the concepts of scale invariance, scaling analysis, and multifractal analysis.

To do so, we need to adopt a more local viewpoint and examine the regularity of realised paths (of stochastic process X) around a given instant. Contrary to Scaling Analysis (SA), which concentrate on the scaling exponent $\tau(q)$, the multifractal analysis (MFA) studies how the (pointwise) local regularity of X fluctuates in time (or space). Local Holder regularity describes the regularity of sample paths of stochastic processes by means of a local comparison against a power law function and is therefore closely related to scaling in the limit of small scales. The exponent of this power law, $h(t)$, is called the (local) Holder exponent and depends on both time and the sample path of X .

Multifractal situations happen when the Holder exponent is no-longer unique, but vary from point to point. More precisely, when the regularity $h(t)$ is itself a highly irregular function of t , possibly even a random process rather than

a constant or a fixed deterministic function, the process X is said to be multifractal. The aim of multifractal analysis (MA) is to provide a description of the collection of Holder exponents h of the function f .

However, since the Holder exponent may jump from one point to another, it was understood that describing them for each time step was not of much importance. Hence, researchers focused on a global description of the regularity of the function of f in form of Multifractal Spectrum (also called the singularity spectrum) reflecting the size of the set of points for which the Holder exponent takes a certain value h . The measure of size most commonly used is the Hausdorff dimension which gives rise to the Hausdorff spectrum (called the multifractal spectrum). It describes the collection of Holder exponents $h(t)$ by mapping to each value of h the Hausdorff dimension $D(h)$ of the collection of points t_i at which $h_f(t_i) = h$. The main idea being that the relative frequency of the local exponents can be represented by a renormalised density called the multifractal spectrum.

These multifractal signals can only be described in terms of an infinite set of exponents and their density distribution. The characterisation of a multifractal process, or measure, by a distribution of local Holder exponents underlines its heterogeneous nature with alternating calm and turbulent phases.

Since scaling analysis and multifractal analysis developed, various authors performed empirical analysis to identify anomalous scaling in financial data. For instance, Calvet et al. [2002] investigated the MMAR model, which predicts that the moments of returns vary as a power law of the time horizon, and confirmed this property for Deutsche mark/US dollar exchange rates and several equity series. Kantelhardt et al. [2002] proposed the multifractal detrended fluctuation analysis (MF-DFA) as a generalisation of the detrended fluctuation analysis (DFA), with the advantage that it can be used for non-stationary multifractal data. Gu et al. [2010] extended the DMA method to multifractal detrending moving average (MFDMA) in order to analyse multifractal time series and multifractal surfaces.

Considering wavelet decomposition tools, Mallat et al. [1990], Muzy et al. [1991], and Arneodo et al. [1991] generalised the multifractal formalism to singular signals. DFA and its variants are methods used for analysing the behaviour of the average fluctuations of the data at different scales after removing the local trends. Further, wavelet transforms (WT) are well adapted to evaluate typical self-similarity properties. As a result, some authors (see Murguía et al. [2009], Manimaran et al. [2009]) merged the two approaches obtaining powerful tools for quantifying the scaling properties of the fluctuations. All multifractal analyses of a response time series follow a step-wise procedure, which was described by Ihlen [2013].

10.4.2 Interpretation

The generalised Hurst exponent $h(q)$ is a decreasing function of q for multifractal time series, and a constant for monofractal processes. The scaling exponent $h(q)$ is only one of several types of scaling exponents used to parametrise the multifractal structure of time series. Multifractal series are also described by the singularity spectrum $f(\alpha)$ through the Legendre transform, where $f(\alpha)$ denotes the fractal dimension of the series subset characterised by the singularity strength, or Holder exponent, α . The spectrum describes the distribution of the Holder exponents. For monofractal signals, the singularity spectrum produces a single point in the $f(\alpha)$ plane, whereas multifractal processes yield a humped function (see Figure 27). The plot of the singularity strength (exponent) $\alpha(q)$ versus the singularity spectrum $f(\alpha)$ is referred to as the Multifractal Spectrum. The multifractal time series has multifractal exponent $\tau(q)$ with a curved q -dependency, and a decreasing singularity exponent $\alpha(q)$. That is, the resulting multifractal spectrum is a large arc where the difference between the maximum and minimum $\alpha(q)$ are called the multifractal spectrum width (or degree). The Hurst exponent defined by the monofractal DFA represents the average fractal structure of the time series and is closely related to the central tendency of multifractal spectrum. The deviation from average fractal structure for segments with large and small fluctuations is represented by the multifractal spectrum width. Thus, each average fractal structure in the continuum of Hurst exponents has a new continuum of multifractal spectrum widths representing the deviations from the average fractal structure. Moreover, the shape of the multifractal spectrum does not have to be symmetric, it can have either a left or a right truncation originating from a levelling of the q -order Hurst exponent for positive or negative q 's, respectively. The spectrum will have a long left tail when the time series have a multifractal structure being insensitive to the local fluctuations with small magnitudes, but it will have long right tail when the structure is insensitive to the local fluctuations with large magnitudes. Consequently, the width and shape of

the multifractal spectrum can classify a wide range of different scale invariant structures of time series.

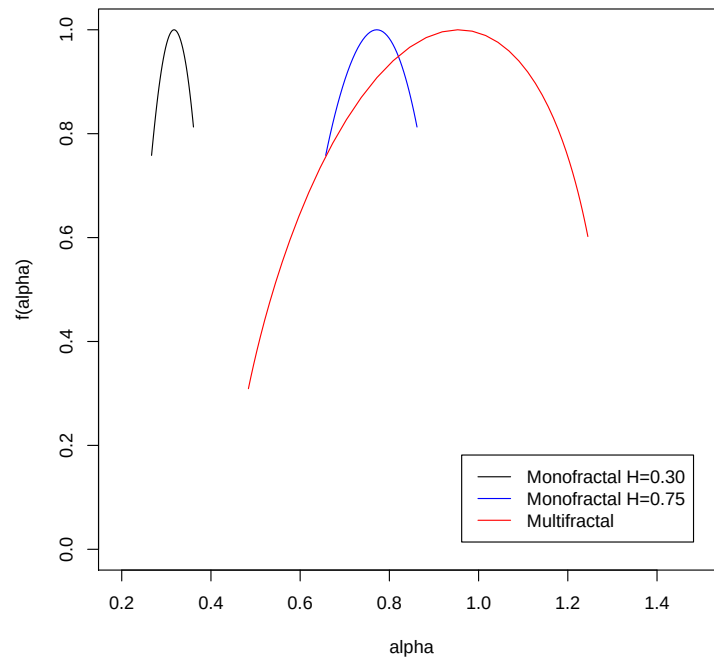


Figure 27: Multifractal spectrum $(\alpha(q), f(\alpha))$. The multifractal spectrum is the log-transformation of the normalised probability distribution of local Hurst exponents. Therefore, when the width of the spectrum is wide (red), the series is multifractal. On the contrary, when the width of the spectrum is small (black and blue), the series is monofractal. In this case, the peak is reached on the Hurst exponent.

10.4.3 Local Holder exponent estimation methods

There are cases when local information about scaling provides more relevant information than the global spectrum. It generally happens for time series where the scaling properties are non-stationary, due to intrinsic changes in the signal scaling characteristics, or even boundary effects. Until recently, the estimation of both local singularity and their spectra were very unstable and prone to gross numerical errors. However, using the wavelet transform multiscale decomposition, Struzik [1999] proposed stable procedures for both the local exponent and its global spectrum. This is the effective local Holder exponent (ELHE), which is a model-based approximation of the local scaling exponent. Later, Turiel et al. [2006] defined a scale-dependent measure with the help of a continuous wavelet transformation called the gradient modulus wavelet projection (GMWP).

We now briefly introduce the fluctuation analysis. We saw above that some researchers tried to estimate the local Hurst exponent at time t as the Hurst exponent computed on a sliding window of size n from $t - n + 1$ till t . However, all methods estimating the Hurst exponent work well only in a large size window, and are therefore not capable of detecting local jumps. Following Struzik [2000], who combined a multiplicative cascade model with the wavelet transform modulus maxima (WTMM) method to estimate the local Hurst exponent, Ihlen [2012] detailed a method that retrieve the local Hurst exponent from a small window size using detrended fluctuation analysis (DFA). That is, rather than using the WTMM tree for defining the partition function-based multifractal formalism, he considered the

DFA model, obtaining a good estimate of the local Hurst exponent.

Bloch [2014] tested for the validity of the local Hurst exponent observed at time t and computed as the Hurst exponent on a sliding window against the one computed with the local detrended analysis (LDA). The former is called "Local" Hurst exponent with double quotes and the latter Local Hurst exponent without any quote. The inability for the "Local" Hurst exponent computed on a sliding window to detect large local changes of fractal structure was easily explained. Further, the equity and FX markets were successfully tested for multifractality.

10.5 Applications to finance

10.5.1 Introducing time-dependent Hurst exponent

Even though the Hurst exponent can only be estimated together with a confidence interval, the stability of the Hurst exponent is related to the fact that a process is self-similar only when the coefficient H is well defined. Examining the stability of H on financial time series on the basis of characteristic values, such as rescaled ranges or fluctuations analysis, some authors (see Vandewalle et al. [1998c], Costa et al. [2003], Cajueiro et al. [2004], Grech [2005]) observed that the values of H could be significantly higher or lower for a specific scale. For example, using the DFA method, Costa et al. [2003] analysed the behaviour of the Sao Paulo stock exchange Ibovespa index covering over 30 years of data from 1968 till 2001, amounting to 8,209 trading days. For the complete time series they obtained $H = 0.6$, indicating persistence in the Ibovespa index. They also computed the local Hurst exponent with a three year sliding window, and found that $H(t)$ varied considerably over time. They found that the local exponent was always greater than $\frac{1}{2}$ before 1990, and then rapidly dropped towards $\frac{1}{2}$ and stayed there afterwards. They interpreted this phenomena as a consequence of the economic plan adopted in March 1990, corresponding to structural reforms in the Brazilian economy. Separate analysis of the two periods gave $H = 0.63$ for the former and $H \approx \frac{1}{2}$ for the latter, indicating multifractality of the process.

Several authors proposed to use methods of long-range analysis such as DFA or DMA to determine the local correlation degree of the series by calculating the local scaling exponent over partially overlapping subsets of the analysed series (see Vandewalle et al. [1998c], Costa et al. [2003], Cajueiro et al. [2004], Carbone et al. [2004], Matos et al. [2008]). The proposed technique examines the local Hurst exponent $H(t)$ around a given instant of time. The ability of DFA and DMA algorithm to perform such analysis relies on the local scaling properties of the scale-dependent measure. Following this method, we can calculate the exponent, $H(t)$, without any a priori assumption on the stochastic process and on the probability distribution function of the random variables entering the process. As a result, one can deduce trading strategies from these instabilities of the Hurst exponent H , such as optimal investment horizon (see Lo [1991]). However, the existence of such optimal investment horizons contradicts both EMH and FMH. In the former, it implies potential predictability of the market, while in the latter it implies that self-similar structure of the market breaks down, turning the market into a spiral. Consequently, these authors introduced time-dependent Hurst exponent for which the examination of different scales solely is not possible. In that setting, we can focus on significant changes in values of Hurst exponent as such changes imply significant shift of rescaled ranges or fluctuations at either low or high scales.

10.5.2 Time and scale Hurst exponent

We saw above that one way of generalising the global behaviour of the Hurst exponent was to estimate the local Hurst exponent $H(t)$ for fixed size windows, N_s , in view of studying its time dependency. Exploring the dichotomy between emerging and mature markets, Matos et al. [2008] considered the local Hurst exponent $H(t, N_s)$ as a function of both time t and scale N_s . To do so, they applied DFA on the interval $(t - \frac{N_s}{2}, t + \frac{N_s}{2})$ in order to recover a power of the scale inside each sub-interval. The technique called time and scale Hurst exponent (TSH) is performed with maximum scale $N_{s_{max}} = \frac{T}{4}$. In that framework $H(t, N_s)$ is the focus of the analysis, and the DFA or DMA are implementation details. Its goal is to provide a map of the evolution of markets to maturity, including significant episodes (major events affecting markets), but also gradual changes in response times. Considering daily data for a set of worldwide market indices from America, Asia, Africa, Europe and Oceania, they observed several notable features

- They can distinguish mature markets by the stability of H values around $\frac{1}{2}$, and emergent markets by the stability of H values above $\frac{1}{2}$.
- For some periods, a phase transition occurs, sometimes observable across all scales, sometimes across partial scales only. This is reflected by spikes occurring for both lower and larger scales.
- They expected smooth variations of H for large scales since it takes more data into account leading to greater robustness to sudden changes of the data. It was confirmed empirically.
- Some markets evolve in time, where they observed a shift from emergent to mature features. The other markets slowly decreased in the values of the Hurst exponent.
- Significant events, causing marked change in the Hurst exponent behaviour was seen in almost all markets.

We see that studying the Hurst behaviour for multiple time and scale intervals gives more refined details on the time series. Matos et al. proposed a refined classification of financial markets in three states

- mature markets: they have H around $\frac{1}{2}$. The presence of regions with higher values of H is limited to small periods and well defined both in time and scale.
- emergent markets: they have H well above $\frac{1}{2}$. The presence of regions with values of H around $\frac{1}{2}$ is well defined both in time and in scale.
- hybrid markets: unlike the two other cases, the distinction between the mature and emergent phases is not well determined, with behaviour seemingly mixing at all scales.

11 Introducing technical analysis

11.1 Defining pattern and technical analysis

11.1.1 Pattern analysis

In general, market prices consist of several patterns which appear repeatedly at time intervals. As told by Mandelbrot [1982], the question of the predictability of financial markets is an old one, as financial newspapers have always presented analysis of charts claiming that they could predict the future from the geometry of those charts. It was the start of what was called pattern analysis (PA).

Definition 11.1 *Patterns*

Patterns are the distinctive formations created by the movements of security prices on a chart. A pattern is identified by a line that connects common price points, such as closing prices or highs or lows, during a specific period of time. Chartists seek to identify patterns as a way to anticipate the future direction of a security's price. Trading patterns can occur at any point or measure in time. While price patterns may be simple to detect in hindsight, spotting them in real time is a much larger challenge.

The simplest example of a pattern is the next period direction, which concerns the direction of the price movement with respect to the next period. In that case we classify each record as Up or Down. Patterns can be based on seconds, minutes, hours, days, months or even ticks and can be applied to bar, candlestick and line charts.

The most basic form of chart pattern is a trend line.

- Trend lines: A trend can often be found by establishing a line chart. A trend line is the line formed between a high and a low. If that line is going up, the trend is up. If the trend line is sloping downward, the trend is down. Trend lines are the foundation for most chart patterns. They are also useful for finding support and resistance levels, which can also be discovered through pattern recognition. A line of support is a level a stock price won't go under; a line of resistance is a point the stock won't go above.

- Pattern types: There are two basic types of patterns: continuation and reversal.
 1. Continuation patterns identify opportunities for traders to continue with the trend. These are retracements or temporary consolidation patterns. The most common continuation patterns include ascending and descending triangles, flag patterns, pennant patterns and symmetrical triangles.
 2. The opposite of continuation patterns is reversal patterns. These are employed to find favourable opportunities to base a trade on the reversal of a trend. In other words, reversal patterns seek to unearth where trends have ended. Common reversal patterns are double tops and bottoms, head-and-shoulders patterns and triple tops and bottoms.

Thomas [2000] identified 47 different chart patterns revealing different market trends. Examples of such patterns are consolidation, cup with handle, head-and-shoulders, double bottom, saucer. Figure (28) display typical chart patterns. However, any stock chart may exhibit countless different pattern combinations, making it very difficult identify and predict.

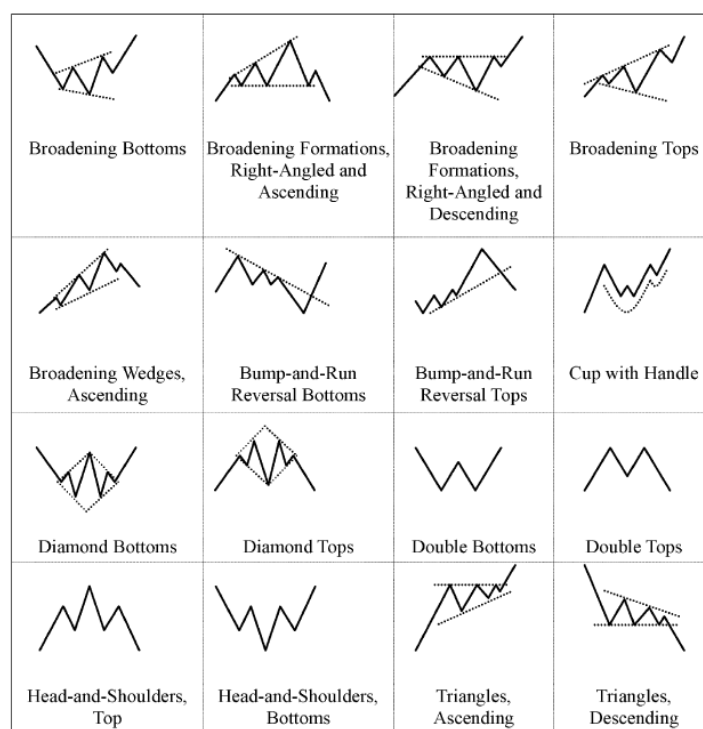


Figure 28: Chart patterns

Since chart pattern identification is highly subjective, pattern analysis led to the development of technical indicators defined as mathematical transformations of market returns. Investopedia describes these indicators as any class of metrics whose value is derived from generic price activity. Each class of indicator focuses on a different aspect of market activity. Technical indicators evaluate price levels, volume, direction and momentum amongst others. The library TA-Lib, designed in the form of an open source API, provides over 200 technical indicators.

11.1.2 Technical analysis

The accumulating evidence against the efficiency of the market has caused a resurgence of interest in the claims of technical analysis as the belief that the distribution of price dynamics being totally random is now being questioned.

Mandelbrot [1963] was the first to demonstrate that empirical data are distinctly non-Gaussian, exhibiting excess kurtosis and higher probability mass in the center and in their tails than the normal distribution. Since this seminal article, a large body of work developed on the mathematical analysis of the behaviour of stock prices, stock markets and successful strategies for trading in these environments. As a result, there are two primary types of stock analysis: fundamental and technical.

1. Fundamental analysis looks at the specifics of a company's business, conducting research on earnings projections, balance sheets, price-to-book ratios and much more.
2. Technical analysis is mostly involved with pattern recognition, regardless of performance. These patterns are then used to uncover pricing trends.

Fundamental analysis can help determine what to buy, while technical analysis can help determine when to buy. Technical analysts use chart patterns to find trends in the movement of a company's stock price. While investing involves the study of the basic market fundamentals, which may take several years to be reflected in the market, trading involves the study of technical factors governing short-term market movements together with the behaviour of the market. Consequently, trading is riskier than long-term investing, but it offers opportunities for greater profits (see Hill et al. [2000]). Recent research finds that the vast majority of fund managers use technical analysis and it is preferred to fundamental analysis (see Menkhoff [2010]).

Technical analysis is about market traders studying market price history with the view of predicting future price changes in order to enhance trading profitability (see Lo et al. [2010]). Technical trading rules involve the use of technical analysis to design indicators that help a trader determine whether current behaviour is indicative of a particular trend, together with the timing of a potential future trade. As a result, in order to apply Technical Analysis, which tries to analyse the securities past performance in view of evaluating possible future investments, we must assume that the historical data in the markets forms appropriate indications about the market future performance. Technical Analysis relies on three principles (see Murphy [1999])

1. market action discounts everything (all information available in the market)
2. prices move in trends or are contrarian (prices are not random)
3. history tends to repeat itself

Hence, by analysing financial data and studying charts, we can anticipate which way the market is most likely to go. That is, even though we do not know when we pick a specific stock if its price is going to rise or fall, we can use technical indicators to give us a future perspective on its behaviour in order to determine the best choice when building a portfolio. Technical indicators try to capture the behaviour and investment psychology in order to determine if a stock is under or overvalued. For instance, in order to classify each stock within the market, we can employ a set of rules based on technical indicators applied to the asset's prices, their volumes, and/or other financial factors. Based on entry/exit signals and other plot characteristics, we can define different rules allowing us to score the distinct stocks within the market and subsequently pick the best securities according to the indicator employed. However, there are several problems occurring when using technical indicators. There is no better indicator, so that the indicators should be combined in order to provide different perspectives. Further, a technical indicator always need to be applied to a time window, and determining the best time window is a complex task. For instance, the problem of determining the best time window can be the solution to an optimisation problem (see Fernandez-Blanco et al. [2008]).

11.1.3 Mechanical trading systems

New techniques combining elements of learning, evolution and adaptation from the field of Computational Intelligence developed, aiming at generating profitable portfolios by using technical analysis indicators in an automated way. In particular, subjects such as Neural Networks, Swarm Intelligence, Fuzzy Systems and Evolutionary Computation

can be applied to financial markets in a variety of ways such as predicting the future movement of stock's price or optimising a collection of investment assets (funds and portfolios). These techniques assume that there exist patterns in stock returns and that they can be exploited by analysis of the history of stock prices, returns, and other key indicators (see Schwager [1996]). With the fast increase of technology in computer science, new techniques can be applied to financial markets in view of developing applications capable of automatically manage a portfolio. Consequently, there is substantial interest and possible incentive in developing automated programs that would trade in the market much like a technical trader would, and have it relatively autonomous. A mechanical trading systems (MTS), founded on technical analysis, is a mathematically defined algorithm designed to help the user make objective trading decisions based on historically reoccurring events. Some of the reasons why a trader should use a trading systems are

- continuous and simultaneous multimarket analysis
- elimination of human emotions
- back test and verification capabilities

Mechanical system traders assume that the trending nature of the markets can be understood through the use of mathematical formulas. For instance, properly filtering time series by removing the noise (congestion), they recover the trend which is analysed in view of inferring trading signals. Assuming that assets are in a continuous state of flux, a single system can profitably trade many markets allowing a trader to be exposed to different markets without fully understanding the nuances of all the individual markets. Since MTS can be verified and analysed with accuracy through back testing, they are very popular. Commodity Trading Advisors (CTA) use systems due to their ease of use, non-emotional factor, and their ability to be used as a foundation for an entire trading platform. Everything being mathematically defined, a CTA can demonstrate a hypothetical track record based on the different needs of his client and customise a specific trading plan. However, one has to make sure the the system is not over-fitting the data in the back test. A system must have robust parameters, that is, parameters not curve fitted to historical data. Hence, one must understand the logic and mathematics behind a system. Unlike actual performance records, simulated results do not represent actual trading. As a rule of thumb, one should expect only one half of the total profit and twice the maximum draw down of a hypothetical track record.

11.2 Presenting a few trading indicators

Any mechanical trading system (MTS) must have some consistent method or trigger for entering and exiting the market based on some type of indicator or mathematical statistics with price forecasting capability. Anything that indicates what the future may hold is an indicator. In general, time series analysis is used to examine data for repeating patterns by collecting data on a system over time in order to analyse a system or to predict future trends. For instance, researchers consider serial correlation and run-tests to test if there are significant price patterns (see Damodaran [2003]).

Most system traders and developers spend 90% of their time developing entry and exit technique, and the rest of their time is dedicated to the decision process determining profitability. Some of the most popular indicators include moving average (MA), weighted moving average (WMA), rate of change, momentum, stochastic (Lane 1950), Relative Strength Index (RSI) (see Wilder [1978]), moving average convergence divergence (MACD) (see Appel [1999]), Donchian breakout, Bollinger bands (BB) (see Bollinger [1992]), Keltner bands (see Keltner [1960]), Commodity Channel Index (CCI) (see Lambert [1980]), Chaikin accumulation/distribution (ChaikinAD), triple smoothed exponential oscillator (TRIX). However, indicators can not stand alone, and should be used in concert with other ideas and logic. There is a large list of indicators with price forecasting capability and we are now going to describe a few of them. For more details we refer the readers to Murphy [1999], Hill et al. [2000], Brabazon [2000], among other.

11.2.1 Stable levels

Technical Analysis tries to derive profitable buy and sell signals by isolating upward and downward price trends from oscillations around a stable level (see Schulmeister [2005]). We let C_t be the closing price at time t , O_t is the opening

price, L_t is the low price, H_t the high price, and V_t is the volume of the trade at time t . LL_n is the lowest low and HH_n is the highest high in the look back period with n discrete events. Some of these levels are as follows:

- Momentum: $C_t - C_{t-n}$ where n is the number of discrete events in the look back period.
- Support and resistance: the former is a price level that a share has reached but not fallen below, the latter is a price level that the market has reached but has not risen above. A breach of either of these levels is sometimes considered to indicate a significant change in a share's price, with the level breached then forming a new support or resistance level respectively.
- Retracing: it assumes that prices trends will eventually tend to reverse to visit a support or resistance level. Thus, attempting to predict a future price.
- UpDown Length: It is the number of consecutive days that a security price has either closed up (higher than previous day) or closed down (lower than previous days). Closing up values represented in positive numbers and closing down is represented with negative numbers. If a security closes at the same price on back to back days, the UpDown Length is 0.
- Stochastic oscillators: In the 50s Lane introduced an oscillating type of indicator called Stochastics that compares the current market close to the range of prices over a specific time period, indicating when a market is overbought or oversold (see Lane [1984]). The stochastic oscillator is a momentum indicator that uses support and resistance levels. It is based on the assumption that when an up-trend/down-trend approaches a turning point, the closing prices start moving away from the high/low price of a specific range. The number generated by this indicator is a percentage in the range $[0, 100]$ such that for a reading of 70 or more it indicates that the close is near the high of the range. A reading of 30 or below indicates the close is near the low of the range. Note, in general the values of the indicator are smoothed with a moving average (MA).
 1. Stochastic $K\%$ oscillators: $\frac{C_t - LL_n}{HH_n - LL_n} \times 100$ where C_t is the current price, LL_n is the lowest price in the last n days and HH_n is the highest price in the last n days. A value nearer 0 is considered to indicate a market which is oversold (which will tend to rise) and a value near 100 indicates a market which is overbought.
 2. Stochastic $D\%$ oscillators: It is the moving average of the stochastic $K\%$ oscillators. The formula is $\frac{1}{n} \sum_{i=0}^{n-1} K_{t-i}\%$, where $K_t\%$ is the above oscillator.
 3. Stochastic slow $D\%$ oscillators: $\frac{1}{n} \sum_{i=0}^{n-1} D_{t-i}\%$ where $D_t\%$ is the above oscillator.
- Larry William $R\%$ (LWR): $\frac{HH_n - C_t}{HH_n - LL_n} \times (-100)$
- Accumulation / Distribution (A/D) oscillator: $\frac{(H_t - O_t) + (C_t - L_t)}{2(H_t - L_t)} \times 100$
- Moving average indicator: the simple moving average (SMA), or just MA, is

$$MA_t(n) = \frac{1}{n} \sum_{i=0}^{n-1} C_{t-i} \quad (11.27)$$

A buy signal is generated when a share's price exceeds the moving average and a sell signal when the moving average exceeds the share price. A variation of this indicator is the moving average convergence divergence (MACD) oscillator calculated by taking the difference of a short run and a long run moving averages of differing length. If the result is positive, this is taken as a signal that the market is trending upward.

- Disparity in n days: $\frac{C_t}{MA(n)} \times 100$
- Bias in n days: $BIAS(n) = \frac{C_t - MA(n)}{MA(n)} \times 100$

- OSCP price oscillator: $\frac{MA(short) - MA(long)}{MA(short)}$
- Average return in the last n days: $ASY(n) = \frac{1}{n} \sum_{i=1}^n SY_{t-i+1}$, where $SY_t = \ln \frac{C_t}{C_{t-1}} \times 100$
- Ratio of the number of rising periods over the n day period: $PSY(n) = \frac{1}{n} A \times 100$, where A is the number of rising days in the last n days.
- Exponential moving average (EMA) indicator: It gives more weight to the most recent periods, making it more reliable than the SMA and a better representation of the recent performance of the security. Hence, it can be used to create a better moving average strategy. It is given by the formula

$$EMA(k)_t = EMA(k)_{t-1} + \alpha \times (C_t - EMA(k)_{t-1}) \quad (11.28)$$

where $\alpha = \frac{2}{k+1}$ is a smoothing factor and k is the time period of k -day EMA. Note, it can be rewritten as

$$EMA(k)_t = \alpha \times C_t + (1 - \alpha)EMA(k)_{t-1} \quad (11.29)$$

The weightage to the most recent data is greater for a shorter period EMA than for a longer period. Each term in the moving average period has an exponentially greater weightage than its preceding term, making the EMA faster to react than the SMA.

- Variable Moving Average (VMA) indicator: In 1991 Tushar Chande suggested that the performance of an EMA could be improved by using a Volatility Index (VI) to adjust the smoothing period when market conditions change. Volatility is the measure of how quickly or slowly prices change over time. The volatility index shows the market's volatility predictions for the next 30 days. The main idea is to slow down the average when prices are in the consolidation period to avoid unavailing trading signals and to speed up the average when the market is trending so as to make the most out of the trending prices. Its formula is obtained by modifying the smoothing factor α in the EMA in Equation (11.29), getting

$$VMA(k)_t = \hat{\alpha} \times C_t + (1 - \hat{\alpha})VMA(k)_{t-1}$$

where $\hat{\alpha} = VI * \alpha$ and VI is a measure of volatility or trend strength.

- On Balance Volume (OBV) indicator: it is based on a cumulative total volume and is intended to relate price and volume in the stock market (see Granville [1976]). The idea is that volume is higher on days where the price move is in the dominant direction, for example in a strong uptrend there is more volume on up days than down days. The formula is as follows:

$$OBV(k)_t = OBV(k)_{t-1} + \theta \times V_t \quad (11.30)$$

where

$$\theta = \begin{cases} +1 & \text{if } C_t \geq C_{t-1} \\ -1 & \text{if } C_t < C_{t-1} \end{cases}$$

11.2.2 Technical indicators

We now describe more complex technical indicators:

- The Chande momentum oscillator (CMO) is a technical momentum indicator (see Chande et al. [1994]). It measures momentum on both up and down days. It is created by calculating the difference between the sum of all recent gains and the sum of all recent losses and then dividing the result by the sum of all price movement over the period.

We consider $U_t(n) = \sum_{i=0}^{n_u-1} UP_{t-i}$ (UP_t is upward price changes) where n_u is the number of up periods

characterised by the close being higher than the previous close, that is, $UP_t = C_t - C_{t-1} > 0$.

We also consider $D_t(n) = \sum_{i=0}^{n_d-1} DW_{t-i}$ (DW_t is downward price changes) where n_d is the number of down periods characterised by the close being lower than the previous close, that is, $DW_t = C_{t-1} - C_t > 0$.

The formula is given by

$$CMO = \frac{U_t(n) - D_t(n)}{U_t(n) + D_t(n)} \times 100$$

Note, $U_t(n) + D_t(n)$ is the sum of all price movement over a given time period. The oscillator is in the range $[-100, +100]$.

- Commodity channel index (CCI): It is an oscillator originally introduced by Donald Lambert [1980] used for identifying cyclical trends. CCI measures a security's variation from the statistical mean, fitting into the momentum category of oscillators. It is calculated as the difference between the typical price of the underlying and its simple moving average, divided by the mean absolute deviation of the typical price. It is an unbounded indicator so that the index is usually scaled by an inverse factor of 0.015 to ensure that approximately 70 to 80 percent of CCI values would fall between -100 and $+100$. The four steps to compute the formula are as follow:

1. Compute the Typical Price

$$M_t = \frac{H_t + L_t + C_t}{3}$$

2. Compute the simple moving average (SMA) price of the last n Typical prices

$$SM_t(n) = \frac{1}{n} \sum_{i=1}^n M_{t-i+1}$$

3. Compute the mean absolute deviation of the last n Typical prices

$$D_t(n) = \frac{1}{n} \sum_{i=1}^n |M_{t-i+1} - SM_t(n)|$$

4. Compute the CCI

$$CCI = \frac{M_t - SM_t(n)}{0.015 \times D_t(n)}$$

The percentage of CCI values falling in the range $[-100, +100]$ will depend on the number of periods used. A shorter CCI will be more volatile with a smaller percentage of values in that range. Conversely, the more periods used to calculate the CCI, the higher the percentage of values in that range.

- Wilder [1978] developed an indicator that measures the velocity of directional movement in prices, making it a momentum⁷ oscillator. It provides a relative evaluation of the strength of a security's recent price performance. As such, it can be used to evaluate overvalued or undervalued conditions in the price of that security. The Relative Strength Index (RSI) is a ratio of the upward price movement to the total price movement over a given period of time (days). The average gain (AU) and average loss (AD) are calculated using an n -period smoothed or modified moving average (SMMA or MMA,) which is an exponentially smoothed Moving Average with $\alpha = \frac{1}{\text{period}}$. Note, one can also use a standard exponential moving average (EMA) as the average instead of the SMMA. Default smoothing period is 14 periods (or days), $\alpha = \frac{1}{14}$. It can be lowered to increase sensitivity or raised to decrease sensitivity. Wilder originally formulated the calculation of the moving average as:

⁷Momentum is the rate of the rise or fall in price.

$$\text{newval} = \frac{1}{\text{period}} (\text{prevval} \times (\text{period} - 1) + \text{newdata})$$

A variation called Cutler's RSI (RSI) is based on a simple moving average (SMA) of U and D , instead of the exponential average above. Cutler found that since Wilder used a smoothed moving average to calculate RSI, the value of the RSI depended upon where in the data file his calculations started. Cutler termed this Data Length Dependency. His RSI is not data length dependent, and returns consistent results regardless of the length of, or the starting point within a data file. Losses are expressed as positive values, not negative values. That is, periods with price losses are counted as zero in the calculations of average gain, and periods with price increases are counted as zero in the calculations of average loss. The formula is given by

$$RSI = \frac{AU(n)}{AU(n) + AD(n)} \times 100 \quad (11.31)$$

where

- $AU(n) = \frac{1}{n_u} \sum_{i=0}^{n_u-1} UP_{t-i} = SMA(U, n)$ (UP_t is upward price changes) is the average of n_u days up closes. Up periods are characterised by the close being higher than the previous close. That is, $UP_t = C_t - C_{t-1} > 0$.
- $AD(n) = \frac{1}{n_d} \sum_{i=0}^{n_d-1} DW_{t-i} = SMA(D, n)$ (DW_t is downward price changes) is the average of n_d days down closes. Down periods are characterised by the close being lower than the previous close. That is, $DW_t = C_{t-1} - C_t > 0$.

The ratio takes values in the range $[0, 100]$. When $AD = 0$ the RSI is 100 (only going up) and when $AU = 0$ the RSI is 0. Traditionally, an RSI reading of 70 or above indicates an overbought situation, and a reading of 30 or below indicates an oversold condition.

The formula is sometime written as

$$RSI = 100 - \frac{100}{(1 + RS(n))} \quad (11.32)$$

where

$$RS(n) = \frac{AU(n)}{AD(n)} = \frac{SMA(U, n)}{SMA(D, n)}$$

is the average gain over the average loss. If we choose the EMA as the average, we get

$$RS(n) = \frac{EMA(U, n)}{EMA(D, n)} \quad (11.33)$$

In general the formula will need at least 250 data points. A plot of RS looks exactly the same as a plot of RSI. The normalisation step makes it easier to identify extremes because RSI is range bound.

- The Donchian channel (or breakout) is an envelope indicator involving two lines that are plotted above and below the market. The top line represents the highest high of n days back (the last n days) (or weeks), and conversely the bottom line represents the lowest low of n days back (the last n days). This centreline is simply the mean of the upper and lower values. While it is an indicator of volatility of a market price, it is used for providing signals for long and short positions. If a security trades above its highest n periods high, then a long is established, otherwise if it trades below the lowest n periods, a short is established. Alternatively, when the day's high penetrates the highest high of four weeks back and selling when the day's low penetrates the lowest low of four weeks back. The channels are wider when there are heavy price fluctuations and narrow when prices are relatively flat.

- The Moving Average Crossover involves two or more moving averages usually consisting of a longer-term and shorter-term average. When the short-term MA crosses from below the long-term MA, it usually indicates a buying opportunity, and selling opportunities occur when the shorter-term MA crosses from above the longer-term MA. Moving averages can be calculated as simple, exponential, and weighted average. Exponential and weighted MAs tend to skew the moving averages toward the most recent prices, increasing volatility.
- In the 70s, Appel [1974] developed another price oscillator called the moving average convergence divergence (MACD). The **MACD line** is the difference between a fast (short term) EMA and a slow (long term) EMA of the closing price of a particular security. The formula is given by

$$DIFF_t = EMA(n_S)_t - EMA(n_L)_t \quad (11.34)$$

where n_S is the number of observations in the short period, n_L is the number of observations in the long period, and $EMA(k)_t$ is given in Equation (11.28). Practically, it is calculated by subtracting the value of a 26-period exponential moving average ($EMA(26)_t$) from a 12-period exponential MA $EMA(12)_t$. It corresponds to a Double Moving Average Crossover (DMAC) on EMA, where $EMA(12)_t$ is the fast average and $EMA(26)_t$ is the slow average.

In general, it is compared with its 9-day exponential moving average, which is called **the signal line**

$$SL_t = EMA_{DIFF_t}(9) \quad (11.35)$$

In that case, it corresponds to a Triple Moving Average Crossover (DMAC) on EMA, where $EMA(9)_t$ is the fast average, $EMA(12)_t$ is the medium average, and $EMA(26)_t$ is the slow average. The intersection of these two signals indicates a buying signal if the MACD line comes from the bottom (below), and a selling signal if this line comes from the top (above). The formula is given by

$$MACD_t = DIFF_t - SL_t$$

We buy if $MACD_t > 0$ and sell if $MACD_t < 0$.

Variants exist. For instance, it can be derived from three different exponentially smoothed moving averages (see Appel et al. [2008]). It is plotted as two different lines, the first line (**the MACD line**) being the difference between the two MAs (long-term and short-term MAs), and the second line (**the signal or trigger line**) being an exponentially smoothed MA of the MACD line. The formula is given by

$$MACD(n; n_S, n_L)_t = MACD(n; n_S, n_L)_{t-1} + \alpha(DIFF_t - MACD(n; n_S, n_L)_{t-1}) \quad (11.36)$$

with smoothing factor $\alpha = \frac{2}{n+1}$.

Remark 11.1 By analogy with the EMA formula in Equation (11.28), the MACD line $DIFF_t$ plays the role of the closing price C_t and the signal $MACD(n; n_S, n_L)_{t-1}$ plays the role of the EMA $EMA(n)_{t-1}$. That is, the signal line is the EMA of the MACD line. Hence, the MACD is an EMA applied to the MACD line instead of the price.

The above formula can be rewritten as

$$MACD(n; n_S, n_L)_t = \alpha DIFF_t + (1 - \alpha)MACD(n; n_S, n_L)_{t-1}$$

Note, the difference (or divergence) between the MACD line and the signal line,

$$Hist_t = (DIFF_t - MACD(n; n_S, n_L)_{t-1}) \quad (11.37)$$

is shown as a bar graph. It is called the MACD histogram. The purpose being to try to eliminate the lag associated with MA type systems. This is done by anticipating the MA crossover and taking action before the

actual crossover. The system buys when the MACD line crosses the signal line from below and sells when the MACD line crosses the signal line from above.

- The rate of change (ROC) momentum involves the analysis of the rate of price change rather than the price level. The speed of price movement and the rate at which prices are moving up or down provide clues to the amount of strength the bulls or bears have at a given point in time. It is calculated by dividing the change in the days' closing price C_t , over n periods, by the closing price n number of days (or weeks) ago C_{t-n} , and then multiplying the ratio by 100. The formula is

$$\begin{aligned} ROC &= \left(\frac{\text{Today's close} - \text{close } n \text{ periods ago}}{\text{close } n \text{ periods ago}} \right) \times 100 \\ &= \frac{C_t - C_{t-n}}{C_{t-n}} \times 100 \end{aligned}$$

It is sometimes given by the difference in prices

$$ROC = \text{Today's close} - \text{close } n \text{ periods ago}$$

Generally, the ROC is calculated based on 14-periods for input n , but can be modified to any trader preferred period. When prices are rising or advancing, ROC values remain above the Zero Line (positive) and when they are falling or declining, they remain below the Zero Line (negative). Note, it is not bounded to a set range since there is no limit to how far a security can advance in price even if there is a limit to how far it can decline.

- Connors RSI (CRSI): The indicator is a composite of three separate components where Wilder's RSI is used in two of the indicator's three components. The RSI, UpDown Length, and Rate-of-Change (ROC), combine to form a momentum oscillator. CRSI outputs a value in the range $[0, 100]$, which is then used to identify short-term overbought and oversold conditions. A value over 90 should be considered overbought and a value under 10 should be considered oversold. Note, have a tendency to produce false signals.
- As an example of channel trading, Keltner [1960] proposed a system called the 10-day moving average rule using a constant width channel to time buy-sell signals with the following rules
 1. compute the daily average price $\frac{high+low+close}{3}$.
 2. compute a 10-day average of the daily average price.
 3. compute a 10-day average of the daily range.
 4. add and subtract the daily average range from the 10-day moving average to form a band or channel.
 5. buy when the market penetrates the upper band and sell when it breaks the lower band.

While this system is buying on the strength and selling on weakness, some practitioners have modified the rules as follow

1. instead of buying at the upper band, you sell and vice versa.
 2. the number of days are changed to a three-day average with bands around that average.
- The Bollinger bands (BB), also called alpha-beta bands, focuses on price and volatility. Assuming prices are Gaussian $N(\mu, \sigma)$, it is a measure of deviation from the mean. Given $P(t, T)$ the price seen at time t for the maturity $T = t + \Delta t$, we get

$$P(t, T) = \mu\Delta t \pm \alpha \times \sigma\sqrt{\Delta t}$$

where α is the number of standard deviation. It gives the dispersion for the price distribution. Note, in financial models, prices are log-normally distributed and returns are normally distributed.

It usually uses 20 or more days in its calculations and does not oscillate around a fixed point (see Bollinger [1992]). It consist of three lines (bands), where the middle line is a simple moving average (SMA) and the outside lines are plus or minus two (that number can vary) standard deviations above and below the SMA. Note, the first two steps are the same as in the CCI. The formula is as follows:

1. Compute the Typical Price

$$M_t = \frac{H_t + L_t + C_t}{3}$$

2. Compute the simple moving average (SMA) price of the last n Typical prices

$$SM_t(n) = \frac{1}{n} \sum_{i=1}^n M_{t-i+1}$$

3. Compute the (daily) standard deviation⁸ of the last n Typical prices

$$SD_t(n) = \sqrt{\frac{1}{n} \sum_{i=1}^n (M_{t-i+1} - SM_t(n))^2}$$

4. Calculate the upper band

$$\text{TopBand} = SM_t(n) + \alpha \times SD_t(n) \quad (11.38)$$

where $\alpha = 2$ (two standard deviations).

5. Calculate the middle band

$$\text{MidBand} = SM_t(n)$$

6. Calculate the bottom band

$$\text{BotBand} = SM_t(n) - \alpha \times SD_t(n) \quad (11.39)$$

Note, one can use a 20-day MA with one and two standard deviations above and below the MA. In the Bollinger bands (BB), looking at the chart we can deduce trend, volatility, and overbought/oversold conditions. A market above one standard deviation is overbought, and it becomes extremely overbought above two standard deviation. That is, most underlyings will pullback to the average even in strongly trending markets. Further, with narrow bands we should buy volatility (calls and puts) and when the bands are widening we should sell volatility. Bollinger [1992] also described two other indicators, the $B\%$ (in relation to the stochastic oscillator $K\%$). The former tells where the prices are within the bands, and it can be above or below one and zero. The formula is

$$B\% = \frac{C_t - \text{BotBand}}{\text{TopBand} - \text{BotBand}}$$

Harrington [2005] normalised this equation to map the bands to $[0, 100]$ instead of $[0, 1]$

$$NB\% = 200 \times B\% - 100$$

⁸The standard deviation measures how spread out prices are from the average. It quantifies the volatility of the price series.

In the $B\%$, the 100% line is the **TopBand** and the -100% line is the **BotBand** of the Bollinger bands. Recall, the bandwidth measures the width of the band as a percentage of the moving average

$$BW = \frac{\text{TopBand} - \text{BotBand}}{\text{MidBand}}$$

When the bands narrow drastically, a sharp expansion in volatility usually occurs in the very near future.

- The Bollinger oscillator (BOS) informs whether the market is overbought or oversold. Derived from the Bollinger band, the formula is

$$BOS_t = \frac{M_t - SM_t(n)}{SD_t(n)} \quad (11.40)$$

where M_t is the Typical price. Note, the numerator $M_t - SM_t(n)$ is a Moving Average Crossover between the Typical price and its Moving Average, indicating a positive or negative trend. This quantity is normalised by the standard deviation of the Typical price, which is a measure of dispersion (risk). That is, the trend is risk-adjusted. The absolute value of the BOS is high when the standard deviation is low.

- The Bollinger double bands (BDB): It generates two sets of Bollinger bands, one set using the parameter of one standard deviation and the other using the typical setting of two standard deviations. Given the SMA $SM_t(n)$, the one-SD band is

$$\text{TopBand1} = SM_t(n) + \alpha_1 \times SD_t(n) \text{ and } \text{BotBand1} = SM_t(n) - \alpha_1 \times SD_t(n) \quad (11.41)$$

where α_1 is one-SD, and the two-SD band is

$$\text{TopBand2} = SM_t(n) + \alpha_2 \times SD_t(n) \text{ and } \text{BotBand2} = SM_t(n) - \alpha_2 \times SD_t(n) \quad (11.42)$$

where α_2 is two-SD. While these formulas use a simple moving average (SMA) for the middle band, we can also use other types of moving averages, such as exponential moving averages (EMA) or weighted moving averages (WMA). These alternative moving averages give more weight to recent price data, making them more responsive to changes in price action.

11.2.3 Visualising trends

Geometrical tools have been proposed to visualise and summarise the information produced by the technical indicators described above. Some examples are as follows:

- Considering projected charts to map future market activity, Drummond et al. [1999] introduced the Drummond Geometry (DG) which is both a trend-following and a congestion-action methodology, leading rather than lagging the market. It tries to foretell the most likely scenario that shows the highest probability of occurring in the immediate future and can be custom fitted to one's personality and trading style. The key elements of DG include a combination of the following three basic categories of trading tools and techniques
 1. a series of short-term moving averages
 2. short-term trend lines
 3. multiple time-period overlays
- The concept of Point and Line (PL) reflecting how all of life moves from one extreme to another, flowing back and forth in a cyclical or wave-like manner, is applied to the market. The PLdot (average of the high, low, and close of the last three bars) is a short-term MA based on three bars (or time periods) of data capturing the trend/nontrend activity of the time frame that is being charted. It represent the center of all market activity. The PLdot is very sensitive to trending markets, it is also very quick at registering the change of a market out of congestion (noise) into trend, and it is sensitive to ending trend.

11.3 The limitation of indicators: combining indicators

In general, momentum based indicators fail most of the time because

- none of them is a pure momentum oscillator that measures momentum directly.
- the time period of the calculations is fixed, giving a different picture of market action for different time periods.
- they all mirror the price pattern, so that it may be better trading prices themselves.
- they do not consistently show extremes in prices because they use a constant time period.
- the smoothing mechanism introduces lags and obscures short-term price extremes that are valuable for trading.

As a result, technical analysts do not rely on a single indicator for detecting trend reversals but rather combine the outcome of various technical indicators. Since each indicator has a significant failure rate, the random nature of price change being one reason why indicators fail, Chande et al. [1994] explained how most traders developed several indicators to analyse prices. Traders use multiple indicators to confirm the signal of one indicator with respect to another one, believing that the consensus is more likely to be correct. For instance, one can generate complex trading rules based on two-way and three-way indicator combinations. The number of combinations is given by the binomial coefficient $\binom{n}{r}$ where r is the number of objects that can be drawn from a set containing n distinguishable objects. In order to get a single buy/sell decision, one must combine the different indicators according to some defined decision rules. For example, one can give a larger weight to trade signals than to hold ones. In case of multiple trade signals one solution is to employ majority voting for deriving the final outcome (see Appendix (13)). In the case of three-way indicator, two sell signal and one buy signal results in a sell signal.

However, this may not be a viable approach due to the strong similarities existing between price based momentum indicators.

11.4 Presenting fractal indicators

Since fractal analysis (FA) allows for a precise modelling of the behaviour of market returns, it has gained popularity in market analysis. As discussed in Appendix (14), the implementation of FA requires the estimation of the fractal dimension. We calculate the fractal dimension of a time series by counting the number of circles N of a given, fixed diameter d needed to cover the entire time series. We then increase the diameter and count again the number of circles required to cover the time series. Repeating that process, the number of circles has an exponential relationship. The fractal dimension is given by Equation (14.54).

Ehlers [2005] adapted the box-counting method to financial time series. There, the data chart is covered by N small elements (boxes) of size S . If we change the scale and size of the elements from S_1 to S_2 , the relationship between the number of elements N_1 and N_2 needed to cover the graph with objects of S_1 and S_2 size, respectively, is given by

$$\frac{N_2}{N_1} = \left(\frac{S_1}{S_2}\right)^D$$

with fractal dimension

$$D = \frac{\log\left(\frac{N_1}{N_2}\right)}{\log\left(\frac{S_2}{S_1}\right)}$$

Note, we recover Equation (14.54) if we set $N_2 = S_2 = 1$.

Rather than measuring the fractal dimension of prices by covering the curve with a series of small boxes, assuming the price samples to be uniformly spaced, the author noted that the box count was approximately the average slope of the curve. Thus, he estimated the box count as the highest price during an interval minus the lowest price in that interval, divided by the length of the interval itself. To do so, we divide the considered period of time $2T$ into two

equal intervals of period T , that is, the periods $[0, T]$ and $[T, 2T]$. For each time period, the chart is covered by N elements. The number $N_{i,T}$, $i = 1, 2$, of elements in the period T (averaging), is equal to

$$N_{i,T}(t) = \frac{H_{i,T}(t) - L_{i,T}(t)}{T}$$

where $H_{i,T}(t)$ and $L_{i,T}(t)$ are the highest and lowest prices in the time period T . Further, the number N_{2T} of elements in the total period is equal to

$$N_{2T}(t) = \frac{H_{2T}(t) - L_{2T}(t)}{2T}$$

where $H_{2T}(t)$ and $L_{2T}(t)$ are the highest and lowest prices in the total time period $2T$.

Since we are looking backwards in time, the slope computation of the fractal dimension of the price chart is given by

$$D = \frac{\log\left(\frac{N_{1,T} + N_{2,T}}{N_{2T}}\right)}{\log\left(\frac{2T}{T}\right)} = \frac{\log(N_{1,T} + N_{2,T}) - \log(N_{2T})}{\log(2)}$$

The author used the fractal dimension to define the fractal adaptive moving average (FRAMA). It is derived from the exponential moving average in Equation (11.28) where D is used to dynamically change the α of the EMA. That is,

$$FRAMA(k)_t = FRAMA(k)_{t-1} + \alpha \times (C_t - FRAMA(k)_{t-1})$$

where α is given by

$$\alpha = e^{-4.6(D-1)}$$

In the case of a straight line, we get $D = 1$, then $\alpha = 1$ and $FRAMA(k)_t = C_t$. In the case of a plane, $D = 2$, and we get $\alpha = 0.01$ so that $FRAMA(k)_t \approx FRAMA(k)_{t-1}$. Thus, FRAMA swings between a fast MA ($D = 1$) and a very slow one ($D = 2$). It is useful when price makes a strong saw-toothed movement.

Remark 11.2 It is a rough approximation to estimate the fractal dimension D . It is actually very difficult to estimate D . We refer the readers to Bloch [2014] for a review of existing techniques for estimating D from market data.

Paluch et al. [2018b] extended the approach by proposing new technical indicators with FRAMA. They considered

1. The relative strength with FRAMA (RSI-FRAMA): it is derived from the RSI in Equation (11.32) by replacing EMA with FRAMA. That is,

$$RSIFRAMA = 100 - \frac{100}{(1 + RS(n))}$$

where

$$RS(n) = \frac{FRAMA(U, n)}{FRAMA(D, n)}$$

2. The moving average convergence/divergence with FRAMA (MACD-FRAMA): it is based on the MACD in Equation (11.34) and SL in Equation (11.35). That is,

$$DIFFFRAMA_t = FRAMA(12)_t - FRAMA(26)_t$$

and

$$SLFRAMA_t = FRAMA_{DIFFFRAMA_t}(9)$$

3. The Bollinger oscillator with FRAMA (BOS-FRAMA): It is based on the Bollinger oscillator in Equation (11.40). That is,

$$BOSFRAMA_t = \frac{M_t - FRAMA_t(n)}{SD_t(n)}$$

11.5 Trading with technical indicator

11.5.1 Trading rules

A trader must decide on a specific entry technique, stop point, and take profit. For instance, whenever a buy signal is triggered, the investor takes a long position in the respective share until a sell signal is generated, at which point the trader unwind his position (sell). He could have simultaneously unwind his position and go short. This position is held until the next buy signal. These are trading strategies discussed in the following section. For now we focus on the entry point and stopping point. When trading on a daily basis at market close, it is assumed that one can estimate the asset price that would trigger the buy and sell signal just before the day's close and initiate a conditional limit order at the close of the market to follow various trading rules. In the case of high frequency trading, buying and selling occur until the end of the trading day, since no positions are held overnight.

Given the technical indicators defined in Appendix (11.2), their associated trading rules are as follows:

- In the case of a simple moving average (SMA) and exponential moving average (EMA), a buy signal (+1) is emitted when the asset price moves from below to above the SMA/EMA considered, and a sell signal (−1) is emitted when the asset price penetrates the SMA/EMA from above to below.
- In the case of stochastic oscillators, when the indicator is increasing, the asset prices are likely to go up and vice versa. In the case of an up-trend, prices tend to make higher highs, and the settlement price usually tends to be in the upper end of that time period's trading range. When the momentum starts to slow, the settlement prices will start to retreat from the upper boundaries of the range, causing the stochastic indicator to turn down at or before the final price high. A value nearer 0 is considered to indicate a market which is oversold (which will tend to rise) and a value near 100 indicates a market which is overbought. An alert or set-up is present when the $D\%$ line is in an extreme area and diverging from the price action. The actual signal takes place when the faster $K\%$ line crosses the $D\%$ line.
- In the case of two moving averages, say a 10 days and 20 days MA, when the 10D-MA moves up through the 20D-MA a buy sign is generated and when the 10D-MA moves down through the 20D-MA a sell sign is generated. The logic being that the price will continue to move in that direction so that the value of shares continues to increase or decrease. See details in Appendix (11.6).
- In the case of a moving average convergence divergence (MACD), then $MACD > 0$ represents the rule such that if MACD is greater than zero we choose to be in the market, otherwise we stay out of the market. See details in Appendix (11.6).
- For variants of MACD, we have $MACD > \text{signal line}$ is the trading rule such that if MACD is above the signal line we choose to be in the market, otherwise we stay out of the market.
- Traders and investors use the commodity channel index (CCI) to help identify price reversals, price extremes and trend strength. It is often used for detecting divergences from price trends as an overbought/oversold indicator, and to draw patterns on it and trade according to those patterns. Readings above +100 imply an overbought condition (showing strength), while readings below −100 imply an oversold condition (showing weakness). It means that there is a large probability that the price will correct to more representative levels. Thus, assuming momentum, when movements are above +100 we buy (+1) and when they are below −100 we sell (−1). It becomes a coincident indicator. The position should be closed when the CCI moves back below +100, in the

former. The position should be closed when the CCI moves back above -100 , in the latter. Since about 70 to 80 percent of the CCI values are between $+100$ and -100 , a buy or sell signal will be in force only 20 to 30 percent of the time. Alternatively, traders have also found the CCI valuable for identifying reversals and divergences. Assuming a mean-reverting market, when price movements are above $+100$ we sell (-1) and when they are below -100 we buy ($+1$). It becomes a leading indicator, where the trader look for overbought or oversold conditions.

- Traditional interpretation and usage of the RSI is that RSI values of 70 or above indicate that a security is becoming overbought or overvalued, and therefore, may be primed for a trend reversal or corrective pull-back in price (-1). An RSI reading of 30 or below is commonly interpreted as indicating an oversold or undervalued condition that may signal a trend change or corrective price reversal to the upside ($+1$). Note, the RSI indicator can stay in the overbought region for extended periods while the stock is in an uptrend. Conversely, it may also remain in oversold territory for a long time when the stock is in a downtrend. Sudden large price movements can create false buy or sell signals in the RSI. It is, therefore, best used with refinements to its application or in conjunction with other, confirming technical indicators.

In addition to Wilder's original theories of RSI interpretation, Cardwell and Brown developed several new interpretations of RSI to help determine and confirm trend. Brown [1999] suggested that oscillators do not travel between 0 and 100 and identified a bull market range and a bear market for RSI. RSI tends to fluctuate between 40 and 90 in a bull market (uptrend) with the 40 – 50 zones acting as support. These ranges may vary depending on RSI parameters, strength of trend and volatility of the underlying security. RSI tends to fluctuate between 10 and 60 in a bear market (downtrend) with the 50 – 60 zone acting as resistance. Andrew Cardwell developed positive and negative reversals for RSI, which are the opposite of bearish and bullish divergences. A positive reversal forms when RSI forges a lower low and the security forms a higher low. A negative reversal is the opposite of a positive reversal. RSI forms a higher high, but the security forms a lower high.

- The Rate of Change indicator (ROC) might be used to confirm price moves or detect divergences and might be used as a guide for determining overbought and oversold conditions. If it is positive we choose to be in the market (buy signal $+1$) and, if it is negative, then it is a sell signal (-1). It applies to both ROC formula. For instance we can consider $ROC(30)$, the 30-day rate of change of the asset price.
- Donchian channels (DC) are mainly used to identify the breakout of a stock or any traded entity enabling traders to take either long or short positions. Traders can take a long position ($+1$), if the stock is trading higher than the Donchian channels n period and book their profits/short the stock (-1) if it is trading below the DC channels n period.

The middle band is the average of the upper and lower bands. The middle band in Donchian channels could also be used as a breakout indicator. If the stock rises above the middle band of the DCs, then you can open a long position ($+1$). On the contrary, if the stock is trading below the middle band of the DCs, then a trader can open a short position (-1).

- A security is deemed to be overbought when the Chande momentum oscillator (CMO) is above $+50$ and oversold when it is below -50 . Trend strength can also be measured using the Chande momentum oscillator. The higher the oscillator's value, the stronger the trend, the lower the value, the weaker the trend.

Traders can use the Chande momentum oscillator to spot positive and negative price divergence between the indicator and the underlying security. A negative divergence occurs if the underlying security is trending upward and the Chande momentum oscillator is moving downwards. A positive divergence occurs if price is declining, but the oscillator is rising.

Many technical traders add a 10-period moving average to this oscillator to act as a signal line. The oscillator generates a bullish signal ($+1$) when it crosses above the moving average, and a bearish signal (-1) when it moves below the moving average.

- The Bollinger bands (BB) can be used for determining overbought and oversold levels, as a trend following tool, and for monitoring for breakouts. When the price of an asset is trading near the upper BB (TopBand), it

is considered overbought and may indicate that the price is likely to fall. Conversely, when the price is trading near the lower BB (BotBand), it is considered oversold and may indicate that the price is likely to rise. Hence, a typical system buys (+1) when price reaches the bottom (BotBand) and liquidates (−1) as the price moves up past the MA. The sell side is simply the opposite. BB identifies asset prices that have deviated from the mean. It is assumed that when a price goes beyond two standard deviations it should revert to the MA.

- In the Bollinger double bands (BDB), whenever price holds between the upper TopBand1 and TopBand2 away from mean, the trend is up. We call it the buy zone. Conversely, whenever price holds between the lower BotBand1 and BotBand2 away from mean, the trend is down. We call it the sell zone. If price meanders between the upper TopBand1 and the lower BotBand1, we are in a neutral state (or zone). In general, we take a long position once the price enters the buy zone from below and unwind the position when it leaves that zone from above.

11.5.2 Trading strategies

Accounting for the predictive power of trading rules, Metghalchi et al. [2015] proposed some trading strategies to beat the profitability of the buy and hold (B&H) strategy. Contrary to the B&H strategy, which is only long, the trader can still invest in cash or short the market. The authors detail some actions to be taken when the trader is out of the market. The strategies are as follows:

- 1= the trader will be in the market when a trading rule emits buy signals and be in the money market (cash) when a trading rule emits sell signals (long/money).
- 2= the trader will be in the market when a trading rule emits buy signals and would short the market when the rule emits sell signals (long/short).
- 3= the trader will borrow at the money market rate (cash rate) and double stock investment when a trading rule emits buy signals and be in the money market when the rule emits sell signals (leverage/ money).
- 4= the trader will borrow at the money market rate and double stock investment when a trading rule emits buy signals and would short the market when the rule emits sell signals (leverage/short).

In case of leverage, the total return on buy days is estimated as twice the market return minus the daily money market return.

11.5.3 Discretising technical indicators

Note, one can directly use continuous technical indicators (actual time series) or one can choose to represent them as trend deterministic data (discrete in nature). In the former, for comparison purposes, the values of all technical indicators are normalised in the range $[-1, +1]$ (see Kara et al. [2011]). Since each technical indicator has its own inherent property used by traders to predict up movement or down movement, a mapping function can be used to convert the continuous values to discrete values representing the trend (see Patel et al. [2015]). That is, +1 indicates an up movement and −1 shows a down movement. To do so, we use the trading rules in Appendix (11.5.1). For example, when SMA/EMA at time t is greater than at time $t - 1$, it suggests an up movement (+1) and vice versa for a down movement (−1). In the case of stochastic oscillators, they are clear trend indicators for any asset. MACD, RSI, CCI and A/D oscillators also follow the asset trend.

11.5.4 Trading methodology

Since pattern is defined as a predictable route or movement, all trading systems are pattern recognition systems. For instance, a long-term moving average cross over system uses pattern recognition, the crossover, in its decision to buy or sell. Similarly, an open range breakout is pattern recognition given by the movement from the open to the breakout

point. All systems look for some type of reoccurring event and try to capitalise on it. Hill demonstrated the success of pattern recognition when used as a filter. The system was developed around the idea of a pattern consisting of the last four days' closing prices. A buy or sell signal is not generated unless the range of the past four days' closes is less than the 30-day average true range, indicating that the market has reached a state of rest and any movement, up or down, from this state will most likely result in a significant move.

Eventually, we want to develop an approach into a comprehensive, effective, trading methodology that combines analytical sophistication with tradable rules and principles. One way forward is to consider a multiple time frame approach. A time frame is any regular sampling of prices in a time series, from the smallest such as one minute up to the longest capped out at ten year. The multiple time frame approach has proven to be a fundamental advance in the field of TA allowing for significant improvement in trading results. For instance, if market analysis is coordinated to show the interaction of these time frames, then the trader can monitor what happens when the support and resistance lines of the different time frames coincide. Assuming we are interested in analysing the potential of the integration of time frames, then we need to look at both a higher time frame and a lower time frame.

11.6 Trading with moving averages

Moving averages (MAs) are used to smooth out the short-term price fluctuations of an asset, providing a clearer picture of the underlying trend. MAs are lagging indicators because they are computed by using historical data. A faster moving average (short term or short lookback period) has less lag when compared to a slower moving average (long term or long lookback period). This is illustrated in Figure (29). There are different types of moving averages, including simple moving averages (SMA), exponential moving averages (EMA), and weighted moving average (WMA), each with its own method of calculating the average. The significant difference between the different MAs is the weight assigned to data points in the moving average period. For instance, SMA (see Equation (11.27)) give equal weight to all data points in the chosen period, EMA (see Equation (11.28)) give more weight to recent data points, making them more responsive to recent price changes, WMA also gives greater weight to recent prices, but each price point is assigned a weight based on its position in the period, with the most recent prices given the highest weights. It is a compromise between the SMA and EMA, providing a balance between smoothness and responsiveness. There are other types of MAs such as Smoothed Moving Average (SMMA), the Triangular Moving Average (TMA), and the Volume Weighted Moving Average (VWMA). The TMA applies more weight to data in the middle of the moving average period.

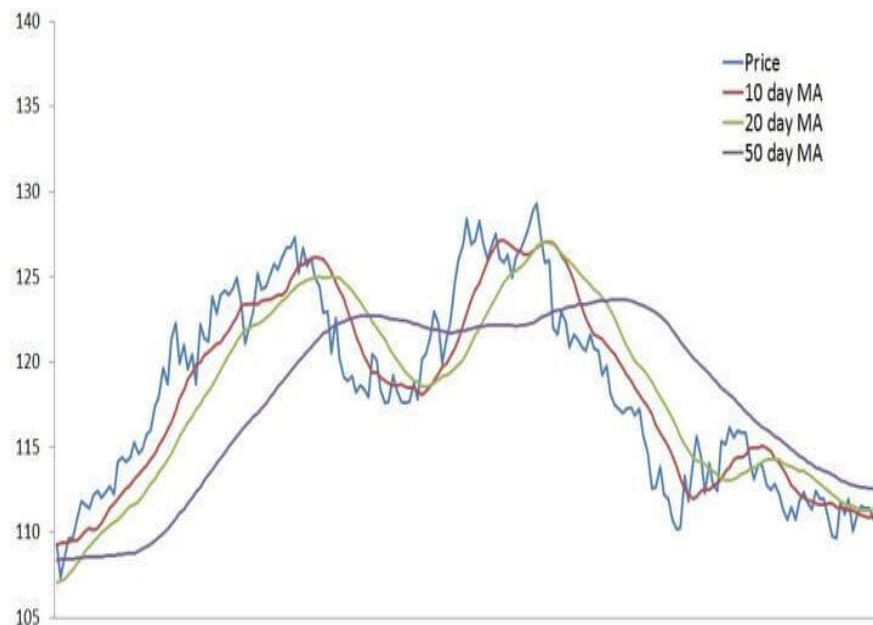


Figure 29: SMA with different speeds.

In moving average trading, each moving average indicator has its own pros and cons. The SMA moves much slower than other averages, keeping trades longer when there are short-lived price movements or price fluctuations. The EMA reacts faster when the price is changing direction. For example, if the price retraces lower, the EMA will start turning down to indicate a change in the trading signal.

11.6.1 Some strategies

This approach is used for identifying potential entry and exit points in the market. In general, we combine the moving averages to obtain moving average crossover (MAC) strategies. A moving average with a short time period will react much faster to price changes than a moving average with a long time period. They are powerful tools for identifying trends and market reversals. As a result, traders can gain insight into the direction of the market and the strength of the trend, making informed decisions in the market. The most common MAC strategies (See Faith [?]) include:

- **Double Moving Average Crossover (DMAC):** This strategy uses two moving averages (MA) of different lengths. A buy signal is generated when the shorter moving average crosses above the longer moving average (a Golden Cross), and a sell signal is generated when the shorter moving average crosses below the longer moving average (a Death Cross).
- **Exponential Moving Average (EMA) Crossover:** This strategy uses exponential moving averages that give more importance to recent price data. When a shorter-term EMA crosses above a longer-term EMA, it generates a buy signal, and when it crosses below, it generates a sell signal.
- **Triple Moving Average Crossover (TMAC):** This strategy uses three moving averages of different lengths (fast, medium, and slow). A buy signal is generated when the fast moving average crosses above both the medium and slow moving averages, and a sell signal is generated when the fast moving average crosses below both the medium and slow moving averages. It is better equipped at dealing with false trading signals than the dual moving average crossover system. The buy signal is generated early in the development of a trend and a sell signal is generated early when a trend ends. When the security begins an uptrend, faster MA (short term) will

begin rising much earlier than the slower MAs (medium and long term). The third MA is used in combination with the other two MAs to confirm or deny the signals they generate, reducing the probability of acting on false signals. For instance, the signal to buy is confirmed when the medium MA crosses above the slow MA. Similarly, the signal to sell is confirmed when the medium MA crosses below the slow MA. When the fast moving average goes above the medium moving average, the system exits its position. The system is out of the market when the relationship between the slow and medium moving averages do not match that between the medium and fast moving averages.

- **Moving Average Ribbon:** This strategy involves placing a large number of moving averages onto the same chart, creating a ribbon effect. Trading signals are generated in a similar manner to the triple MAC system, one must decide the number of crossovers to trigger a buy or sell signal. We buy when the faster MAs cross above the slower MAs and sell when the faster MAs cross below the slower MAs. The ribbon can help traders identify trend strength, potential entry and exit points, and trend reversals. When the ribbon is wide and moving averages are parallel, it indicates a strong trend, while a narrowing ribbon and converging moving averages signal a potential trend reversal. These MAC strategies can be powerful tools for traders to identify trend changes and potential entry and exit points in the market. However, they should be used in conjunction with other technical indicators to improve their effectiveness.
- **Moving Average Convergence Divergence (MACD):** See details in Appendix (11.2.2). The MACD is illustrated in Figure (30). The system buys when the MACD line crosses the signal line from below and sells when the MACD line crosses the signal line from above. It is understood that the trend in the underlying security is about to escalate in the direction of the crossover. One can also consider zero crossover when the slow and fast moving averages of the price curve crossover each other, or when the MACD series changes sign. A change from positive to negative is considered to be a bearish sign while a change from negative to positive is considered as a bullish sign. The zero crossover provides confirmation about a change in trend but it is less reliable in triggering signals than the signal crossover. Alternatively, we can monitor the divergence between the MACD line and the signal line, which can be observed through the histogram. When the histogram starts falling (moves towards the zero line), it indicates that the trend is weakening, this happens when the MACD and signal lines are converging. In any case, when the signal line and MACD line are diverging, or the histogram is rising (moves away from the zero line), it is an indication that the trend is growing stronger.
- **The Bollinger bands (BB)** can be used for determining overbought and oversold levels, as a trend following tool, and for monitoring for breakouts. In general, the outer bands coincide with Support and Resistance. This is illustrated in Figure (31). Some of the strategies include
 - **Overbought and oversold strategy:** When the price of an asset is trading near the upper BB (TopBand), it is considered overbought and may indicate that the price is likely to fall. Conversely, when the price is trading near the lower BB (BotBand), it is considered oversold and may indicate that the price is likely to rise. Hence, a typical system buys (+1) when price reaches the bottom (BotBand) and liquidates (−1) as the price moves up past the MA. Note, if the MACD shows a bullish crossover when the price bounces off the lower Bollinger Band, it could confirm a buy signal. The sell side is simply the opposite. BB identifies asset prices that have deviated from the mean. It is assumed that when a price goes beyond two standard deviations it should revert to the MA. Note, we can adjust your Bollinger Bands settings to 3 standard deviations (or higher) to identify even more overbought/oversold levels to trade-off. At last, some practitioners revert the logic and sell (−1) rather than buy when prices reach the lower band (BotBand), and vice versa with the upper band (TopBand). This is typical of range-bound markets where mean reversion strategies work well, as prices travel between the two bands like a bouncing ball.
 - **Trend following:** First look at the overall direction of price and only then take trade signals that align us with the trend. For example, if the trend is down, only take short positions when the upper band is tagged (or touched). The lower band can still be used as an exit if desired, but a new long position is not opened since that would mean going against the trend.

- Squeeze strategy: A squeeze occurs when the price has been moving aggressively then starts moving sideways in a tight consolidation. In that case, the upper and lower bands get closer together, meaning that the volatility of the asset has decreased. This is called volatility contraction. After a period of consolidation, the price often makes a larger move in either direction (price break), ideally on high volume. When the price breaks through the upper or lower band, the trader buys or sells the asset, respectively.
- In the Bollinger double bands (BDB), whenever price holds between the upper TopBand1 and TopBand2 away from mean, the trend is up. We call it the buy zone. Conversely, whenever price holds between the lower BotBand1 and BotBand2 away from mean, the trend is down. We call it the sell zone. If price meanders between the upper TopBand1 and the lower BotBand1, we are in a neutral state (or zone). The bands adapt dynamically to price expanding and contracting as volatility increases and decreases. Therefore, the bands naturally widen and narrow in sync with price action, creating a very accurate trending envelope. Some of the strategies include
 - Trend following: Take a long position once the price enters the buy zone (crossing the upper TopBand1 from below). Close the long position if the candle on the candlestick charts turns red and more than 75% of its body is below the buy zone.
 - Trend exhaustion: Identify the turn in price (counter-trend). When prices fall out of the trend channel, short the next tag of the upper band.
 - Stop-loss points (above and below): Use the volatility of the market to help set a stop-loss level. Measure the width of the neutral zone and add it to the upper band to get the SL-points.
- The Relative Strength Index (RSI): The RSI can be used to predict the price behaviour of a security, validate trends and trend reversals, identify overbought and oversold securities, provide buy and sell signals. This is illustrated in Figure (32). Some of the strategies include
 - Trend following: During trends, the RSI readings may fall into a band or range. In an uptrend, the RSI tends to stay above 30 and should frequently hit 70. In a downtrend, the indicator frequently hits 30 or below and hardly exceeds 70. Hence, we can create a horizontal trendline between the levels of 30 and 70 when a strong trend is in place to better identify the overall trend and extremes.
 - Overbought or oversold: When the RSI indicator crosses 30 on the RSI chart, we anticipate a bullish market, and when it crosses 70, we anticipate a bearish market. That is, values of 70 or above indicate that a security is becoming overbought or overvalued. It may be primed (ready) for a trend reversal or corrective price pullback. Conversely, a RSI reading of 30 or below indicates an oversold or undervalued condition.
 - Predicting trend lines: The RSI is a leading indicator so that we can use trendlines on the RSI chart and trade its break. When the RSI indicator is rising, an upward trendline is drawn by connecting two or more lows and projecting the line into the future. Similarly, when the RSI indicator is falling, a downward trendline is drawn by connecting two or more highs and projecting the line into the future. A break of an RSI trendline precedes an actual price reversal or continuation in the market. For instance, if the asset price breaks above a downward trendline, it is a signal that the price is about to edge upwards, either as a continuation of an uptrend or as a reversal of an existing downtrend in the market (a bullish divergence). Conversely, if the asset price breaks below an upward trendline, it is a signal that the price is about to edge downwards, either as a continuation of a downtrend or as a reversal of an existing uptrend in the market (a bearish divergence). This is illustrated in Figure (32).
 - RSI divergences: An RSI divergence occurs when price moves in the opposite direction of the RSI. That is, a chart might display a change in momentum before a corresponding change in price. A bullish divergence occurs when the RSI displays an oversold reading followed by a higher low that appears with lower lows in the price. It may indicate rising bullish momentum, and a break above oversold territory could be used to

trigger a new long position. Conversely, a bearish divergence occurs when the RSI creates an overbought reading followed by a lower high that appears with higher highs on the price. This is illustrated in Figure (32).

- Positive-negative RSI reversals: A positive RSI reversal may take place once the RSI indicator reaches a low that is lower than its previous low at the same time that a security's price reaches a low that is higher than its previous low price. We can see this formation as a bullish sign and a buy signal. Conversely, a negative RSI reversal may take place once the RSI indicator reaches a high that is higher than its previous high at the same time that a security's price reaches a lower high. This formation would be a bearish sign and a sell signal.
- RSI swing rejections: The bullish swing rejection has four parts: (1) the RSI falls into oversold territory, (2) the RSI crosses back above 30, (3) the RSI forms another dip without crossing back into oversold territory, (4) the RSI then breaks its most recent high. Conversely, the bearish swing rejection also has four parts: (1) the RSI rises into overbought territory, (2) the RSI crosses back below 70, (3) the RSI forms another high without crossing back into overbought territory, (4) the RSI then breaks its most recent low.
- Combining RSI with MACD: The MACD measures the relationship between two EMAs, while the RSI measures price change momentum in relation to recent price highs and lows. Since the MACD uses moving averages in its computation, it is largely a lagging indicator and can be used to qualify signals generated by the RSI indicator, which is a leading indicator. For instance, if the RSI displays a reading of above 70, which implies overbought conditions in the market, a sell position can be initiated when the MACD series shifts from positive to negative (crossing below the 0 centreline). Conversely, if the RSI displays a reading of below 30, which implies oversold conditions in the market, a buy position can be initiated when the MACD series shifts from negative to positive (crossing above the 0 centreline).
- Combining RSI with Bollinger bands: BB is a volatility indicator whose bands squeeze when there is low volatility but diverge when there is high volatility. Further, periods of consolidation are usually followed by massive breakouts, which makes any Bollinger Bands squeeze a vital time to watch the markets. We also know that a breakout signal will be delivered when there is an RSI overbought or oversold failure swing. For instance, a bearish signal happens when the RSI line rises above 70 and then falls below 70 again. Hence, we can open a sell position when the prices touch the lower Bollinger band *BotBand* in Equation (11.39). Conversely, a bullish signal happens when the RSI line falls below 30 and then rises above 30 again. In that case, we can open a buy position when the prices touch the upper Bollinger band *TopBand* in Equation (11.38).



Figure 30: MACD lines and its histogram.



Figure 31: Bollinger bands.



Figure 32: RSI oscillator.

11.6.2 Trading styles

We need to consider a particular trading style in order to determine the optimal time frame for MAC strategies. For examples:

- **Day trading:** For day traders, who hold positions for a few hours or less, shorter time frames are more suitable. A combination of 5, 8, and 13-bar simple moving averages (SMAs) can be effective for day trading strategies.
- **Swing trading:** Swing traders, who hold positions for a few days to a few weeks, can use moving average crossovers to enter trades. Commonly used time frames for swing trading include 20-day, 50-day, and 200-day SMAs.
- **Long-term investing:** For long-term investors, longer time frames are more appropriate. Commonly used periods for creating moving average lines include 50-day, 100-day, and 200-day moving averages.

Day trading require continuous feedback on short-term price action to make very fast buy and sell decisions. One approach is to use intraday bars wrapped in multiple moving averages, allowing quick analysis that highlights current risks (as well as the most advantageous entries and exits). The 5 – 8 – 13-bar simple moving averages (SMAs) offer relatively strong inputs for day trading strategies. To improve results, one should also look at the slopes of the MAs which reflect subtle shifts in short-term momentum. Increases in observed momentum (acceleration) offer buying opportunities for day traders, while decreases in momentum (deceleration) may signal timely exits. Trend-less markets and periods of high volatility will force the 5 – 8 – 13-bar SMAs into large-scale whipsaws (oscillations), with horizontal orientation and frequent crossovers telling observant traders to stand aside. In that case, the slope tends to near zero, indicating that intraday trending is weak and profitable opportunities are limited. In fact, trading ranges expand in volatile markets and contract in trend-less markets.

When performing day trading with RSI, we choose between 6 and 9 periods, so that more overbought and oversold signals are generated. Ideally, these levels should correspond with support and resistance levels. Thus, in a ranging market, buy orders will be placed off a support area when the RSI reading is below 30, whereas sell orders will be

placed off a resistance area when the RSI reading is above 70.

In trending markets, the RSI setting can be even lower (2 – 6) periods, so that we can identify the best value areas to enter the market in the direction of the dominant trend. For instance, in an uptrend, we will look to enter buy trades after a pullback that will be indicated by an oversold RSI reading. Similarly, in a downtrend, sell trades will be placed after a retracement that will be highlighted by an overbought RSI reading.

Since a lower setting will generate numerous overbought and oversold signals, it is important to combine RSI and other technical indicators, such as moving averages (MAs), in trending markets so as to filter out the best quality signals when day trading. MAs will confirm the trend and also provide possible retracement target levels where RSI overbought and oversold signals can confirm solid trading opportunities.

11.6.3 Some advantages and drawbacks

Moving average trading indicators help us trace the price changes and fluctuations so that we can take the right decisions regarding to the trading position. Using MA indicators, we can identify an uptrend (the price moves above the moving average) or a downtrend (the price moves below the moving average). They are used to predict the price change and the change in the trend of the financial market. Considering other factors such as the length of the trading period, moving average crossover, etc., we can find out the trading positions, the entry points when the prices are strongly trending. Further, MAs can be used as support/resistance points. The MAs indicator are responsive to new and updated information which means better predictions.

Some drawback of moving averages are:

- Lagging indicators: They provide signals after the trend has already begun, which can result in missed opportunities and false signals.
- Whipsaw markets: In markets with choppy price action and no clear trend, MAs can generate a large number of false signals, leading to losses.
- Slow-moving: MAs are slow-moving indicators, which means they may not respond quickly enough to sudden changes in market conditions.
- Not suitable for all market conditions: MAC strategies may work well in trending markets but may be less effective in range-bound or volatile markets (generating multiple trend reversals or trade signals). Adjusting the time frame can help with this problem.

To maximise the effectiveness of MACs, traders should use them in conjunction with other technical indicators and fundamental analysis. This way, they can confirm signals and make informed trading decisions.

11.6.4 Mathematical formulation

Recall, we can use any specific moving averages when trading with MAs. For simplicity, we consider the SMA (see Equation (11.27)) at time t given by

$$MA_t(N) = \frac{1}{N} \sum_{i=0}^{N-1} P(t-i) = \frac{1}{N} \sum_{i=1}^N P(t-(i-1))$$

where $P(t)$ the price at time t . We could as well used the EMA in Equation (11.28).

11.6.4.1 The double moving average crossover In the case of a Double Moving Average Crossover (DMAC), we let N_S be the number of observations in the short period, N_L be the number of observations in the long period, X_b a percentage band. The MA rule is (N_S, N_L, X_b) and some popular ones are $(1, 50, 0)$, $(1, 100, 0)$. At time t , the buy signal is given by

$$MA_t(N_S) > MA_t(N_L) + X_b \rightarrow \text{buy at time } t$$

and the sell signal is reversed, as follows:

$$MA_t(N_S) < MA_t(N_L) - X_b \rightarrow \text{sell at time } t$$

11.6.4.2 The triple moving average crossover In the case of a Triple Moving Average Crossover (TMAC), we let N_S be the short period, N_M the medium period, N_L the long period, X_b a percentage band, and $P(t)$ the price at time t . At time t , the long (buy) signal is given by

$$MA_t(N_S) > MA_t(N_M) + X_b \text{ and } MA_t(N_M) > MA_t(N_L) + X_b \rightarrow \text{buy at time } t \quad (11.43)$$

and we unwind the long signal (we sell) when

$$MA_t(N_S) < MA_t(N_M) - X_b \quad (11.44)$$

The short (sell) signal is reversed, as follows:

$$MA_t(N_S) < MA_t(N_M) - X_b \text{ and } MA_t(N_M) < MA_t(N_L) - X_b \rightarrow \text{sell at time } t \quad (11.45)$$

and we unwind the short signal (we buy) when

$$MA_t(N_S) > MA_t(N_M) + X_b \quad (11.46)$$

11.6.4.3 The moving average convergence divergence The MACD line at time t , denoted $DIFF_t$, is given in Equation (11.34) and the signal line at time t , denoted $MACD(n; n_S, n_L)_t$, is given in Equation (11.36). Practically, we can use the MACD's histogram the same way that we use the MACD itself. The MACD is illustrated in Figure (33). Hence, we consider the MACD histogram (see Equation (11.37)) given by

$$Hist_t = (DIFF_t - MACD(n; n_S, n_L)_t) \quad (11.47)$$

It represents the distance between the line MACD and its signal line.



Figure 33: MACD histogram.

Some possible use of the histogram are as follows:

- Positive or negative crossovers: If MACD line is above the signal line, the histogram will be above the MACD's baseline (or zero line), otherwise if it is below, the histogram will be below the MACD's baseline. Zeroes in the histogram occur when the MACD line crosses the signal line above or below.
- Identify strength: The magnitude of height is directly proportional to the spread between the MACD and signal lines. When the spread between the two lines increases, the histogram becomes taller or deeper, depending on its direction. When the opposite scenario occurs (when the two lines draw closer), the histogram becomes shorter.
- Identify bullish (when MACD rises above the signal line) or bearish (when MACD falls below the signal line) momentum, and possibly identify overbought/oversold signals. MACD functions on a relative basis so that we should focus on the level and direction of the MACD/signal lines compared with preceding price movements in the security at hand. Peaks and troughs in the histogram indicate when a burst of bearish or bullish momentum is losing strength, and the curve is likely to return to its mean.
- Divergence: When MACD forms highs or lows that exceed the corresponding highs and lows on the price, it is called a divergence. A bullish divergence appears when MACD forms two rising lows that correspond with two falling lows on the price. This is a valid bullish signal when the long-term trend is still positive. Similarly, when MACD forms a series of two falling highs that correspond with two rising highs on the price, a bearish divergence has been formed. A bearish divergence appearing during a long-term bearish trend is considered confirmation that the trend is likely to continue.
- Rapid rises or falls: When MACD rises or falls rapidly (the shorter-term moving average pulls away from the longer-term moving average), it is a signal that the security is overbought or oversold and will soon return to normal levels. This analysis can be combined with the RSI or other TIs to verify overbought or oversold conditions.
- Predict change in trend: We can use MACD histogram to predict when a change in trend will occur. In that case, it makes MACD a leading indicator. This is done by computing the slope of the histogram. From the slope

of the histogram, one can get the most common signal. For example, when the current bar is higher than the preceding bar, the slope is up and one can go long. A slant divergence is a slope that is capable of creating a divergence without well-defined peaks/troughs.

For simplicity, we consider the case where we buy (long) the security when the MACD line crosses above the signal line, and sell (short) the security when the MACD line crosses below the signal line. That is, given some ϵ ,

- Go long when the MACD histogram turns up below zero, $Hist_t > \epsilon$.
- Go short when MACD histogram turns down above zero, $Hist_t < \epsilon$.
- Otherwise we don't trade.

11.6.4.4 The Bollinger double bands Given the Bollinger double bands (BDB) defined in Equations (11.41) and (11.42), we consider a mix of trend following and mean-reverting strategy. The buy zone is given by

$$\text{Buy Zone} = [\text{TopBand1}, \text{TopBand2}] \quad (11.48)$$

and sell zone is given by

$$\text{Sell Zone} = [\text{BotBand2}, \text{BotBand1}] \quad (11.49)$$

The BDB strategy becomes

- Go long when the price enters the buy zone. Unwind when the price exit (from above or below) the buy zone.
- Go short when the price enters the sell zone. Unwind when the price exit (from below or above) the sell zone.
- Otherwise we don't trade.

11.6.4.5 The RSI The RSI indicator is given in Equation (11.32). Since the main signal the RSI oscillator defines overbought and oversold price ranges, we consider the overbought or oversold strategy. When the RSI line goes above the level 70 or below the level 30, it signals that market is overbought/oversold, respectively. The rules for opening and closing positions based on the RSI signals are:

- If the RSI line crosses the level 30 from above (or first from above and then from below), a long position (buy) is opened. Exit on opposite indicator's signal. Unwind the position if the RSI line crosses the level 70 from below (or first from below and then from above).
- If the RSI line crosses the level 70 from below (or first from below and then from above), a short position (sell) is opened. Exit on opposite indicator's signal. Unwind the position if the RSI line crosses the level 30 from above (or first from above and then from below).

11.6.5 Trading rules

The Triple Moving Average (TMA) strategy has the following trading rules (See Faith [?]):

1. Long position

- Enter long rules:
 - Open a long position when the $MA_t(N_S)$ crosses over the $MA_t(N_M)$, and
 - Both the $MA_t(N_S)$ and $MA_t(N_M)$ are above the $MA_t(N_L)$.
- Exit long rules:

- Sell the long position when the $MA_t(N_S)$ crosses below the $MA_t(N_M)$.

2. Short position

- Enter short rules:
 - Open a short position when the $MA_t(N_S)$ crosses under the $MA_t(N_M)$, and
 - Both the $MA_t(N_S)$ and $MA_t(N_M)$ are below the $MA_t(N_L)$.
- Exit short rules: Cover a short position when the $MA_t(N_S)$ crosses above the $MA_t(N_M)$.

Note, the TMA strategy only trades when two moving averages are on the same side of the $MA_t(N_L)$. Further, even though the TMA strategy lacks of stop-loss orders, there is an implicit stop. When prices move against our position, sooner or later the short MA crosses the middle MA and the strategy generates its exit signal. However, the exit time is random so that the price level is unknown. Hence, we must impose a global stop loss. At last, the TMAverage strategy underperforms significantly when prices move sideways. One solution is to filter out those low-probability trades (with specific technical indicators).

The template for the TMA strategy is:

1. Define strategy settings
2. Calculate strategy values
3. Determine long trading conditions
4. Code short trading conditions
5. Output strategy data
6. Submit entry orders
7. Submit exit orders

11.6.6 The entry and exit algorithms

We use the strategies defined in Appendix (11.6.4) to implement their entry and exit algorithms.

11.6.6.1 The triple moving average crossover

The entry and exit algorithms for the TMA strategy are:

Algorithm 2 Set entry rule for TMA.

Require: The current time t_j , the stock price S_{t_j} , the window size.

Require: The moving averages $MA_{t_j}(N_S)$, $MA_{t_j}(N_M)$, and $MA_{t_j}(N_L)$

- 1: **if** Equation (11.43) **then**
 - 2: Identify the trade as long: set $BoolLong = \text{True}$ and $BoolTrade = \text{True}$.
 - 3: **else if** Equation (11.45) **then**
 - 4: Identify the trade as short: set $BoolLong = \text{False}$ and $BoolTrade = \text{True}$.
 - 5: **else**
 - 6: Do not trade : set $BoolTrade = \text{False}$.
 - 7: **end if**
 - 8: **return** $BoolTrade, BoolLong$
-

Algorithm 3 Set exit rule for TMA.

Require: The current time t_j , the stock price S_{t_j} , the window size.**Require:** The moving averages $MA_{t_j}(N_S)$, $MA_{t_j}(N_M)$, and $MA_{t_j}(N_L)$ **Require:** BoolTrade, BoolLong

- 1: **if** *BoolLong* And Equation (11.44) **then**
 - 2: Identify the exit time of a long position: set *BoolTrade* = False .
 - 3: **else if** *!BoolLong* And Equation (11.46) **then**
 - 4: Identify the exit time of a short position: set *BoolTrade* = False .
 - 5: **end if**
 - 6: **return** BoolTrade
-

11.6.6.2 The moving average convergence divergence The entry and exit algorithms for the MACD strategy are:

Algorithm 4 Set entry rule for MACD.

Require: The current time t_j , the stock price S_{t_j} , the window size.**Require:** The MACD and signal lines $DIFF_{t_j}$ and $MACD(n; n_S, n_L)_{t_j}$ **Require:** The histogram $Hist_{t_j}$ given in Equation (11.47)

- 1: **if** $Hist_{t_j} > \epsilon$ **then**
 - 2: Identify the trade as long: set *BoolLong* = True and *BoolTrade* = True .
 - 3: **else if** $Hist_{t_j} < \epsilon$ **then**
 - 4: Identify the trade as short: set *BoolLong* = False and *BoolTrade* = True .
 - 5: **else**
 - 6: Do not trade : set *BoolTrade* = False .
 - 7: **end if**
 - 8: **return** BoolTrade, BoolLong
-

Algorithm 5 Set exit rule for MACD.

Require: The current time t_j , the stock price S_{t_j} , the window size.**Require:** The MACD and signal lines $DIFF_{t_j}$ and $MACD(n; n_S, n_L)_{t_j}$ **Require:** The histogram $Hist_{t_j}$ given in Equation (11.47)**Require:** BoolTrade, BoolLong

- 1: **if** *BoolLong* And $Hist_{t_j} < \epsilon$ **then**
 - 2: Identify the exit time of a long position: set *BoolTrade* = False .
 - 3: **else if** *!BoolLong* And $Hist_{t_j} > \epsilon$ **then**
 - 4: Identify the exit time of a short position: set *BoolTrade* = False .
 - 5: **end if**
 - 6: **return** BoolTrade
-

11.6.6.3 The Bollinger double bands The entry and exit algorithms for the BDB strategy are:

Algorithm 6 Set entry rule for BDB.

Require: The current time t_j , the stock price S_{t_j} , the window size.**Require:** The Buy Zone given in Equation (11.48) and the Sell Zone given in Equation (11.49)

- 1: **if** S_{t_j} enters the Buy Zone **then**
 - 2: Identify the trade as long: set $BoolLong = \text{True}$ and $BoolTrade = \text{True}$.
 - 3: **else if** S_{t_j} enters the Sell Zone **then**
 - 4: Identify the trade as short: set $BoolLong = \text{False}$ and $BoolTrade = \text{True}$.
 - 5: **else**
 - 6: Do not trade : set $BoolTrade = \text{False}$.
 - 7: **end if**
 - 8: **return** $BoolTrade, BoolLong$
-

Algorithm 7 Set exit rule for BDB.

Require: The current time t_j , the stock price S_{t_j} , the window size.**Require:** The Buy Zone given in Equation (11.48) and the Sell Zone given in Equation (11.49)**Require:** $BoolTrade, BoolLong$

- 1: **if** $BoolLong$ And S_{t_j} exit the Buy Zone **then**
 - 2: Identify the exit time of a long position: set $BoolTrade = \text{False}$.
 - 3: **else if** $\neg BoolLong$ And S_{t_j} exit the Sell Zone **then**
 - 4: Identify the exit time of a short position: set $BoolTrade = \text{False}$.
 - 5: **end if**
 - 6: **return** $BoolTrade$
-

11.6.6.4 The RSI The entry and exit algorithms for the RSI strategy are:

Algorithm 8 Set entry rule for RSI.

Require: The current time t_j , the stock price S_{t_j} , the window size.**Require:** The RSI indicator given in Equation (11.32)

- 1: **if** RSI line crosses the level 30 from above **then**
 - 2: Identify the trade as long: set $BoolLong = \text{True}$ and $BoolTrade = \text{True}$.
 - 3: **else if** RSI line crosses the level 70 from below **then**
 - 4: Identify the trade as short: set $BoolLong = \text{False}$ and $BoolTrade = \text{True}$.
 - 5: **else**
 - 6: Do not trade : set $BoolTrade = \text{False}$.
 - 7: **end if**
 - 8: **return** $BoolTrade, BoolLong$
-

Algorithm 9 Set exit rule for RSI.

Require: The current time t_j , the stock price S_{t_j} , the window size.**Require:** The RSI indicator given in Equation (11.32)**Require:** BoolTrade, BoolLong

- 1: **if** BoolLong And RSI line crosses the level 70 from below **then**
 - 2: Identify the exit time of a long position: set BoolTrade = False .
 - 3: **else if** !BoolLong And RSI line crosses the level 30 from above **then**
 - 4: Identify the exit time of a short position: set BoolTrade = False .
 - 5: **end if**
 - 6: **return** BoolTrade
-

11.7 Analysing the trading rules

We have defined the moving average indicator in Appendix (11.2). More formally, we let N_S be the short period, N_L the long period, X_b a percentage band, and $P(t)$ the price at time t . The MA rule is (N_S, N_L, X_b) and some popular ones are $(1, 50, 0)$, $(1, 100, 0)$. The buy signal is given by

$$\frac{1}{N_S} \sum_{i=1}^{N_S} P(t - (i - 1)) > \frac{1}{N_L} \sum_{i=1}^{N_L} P(t - (i - 1)) + X_b \rightarrow \text{buy at time } t$$

and the sell signal is reversed, as follows

$$\frac{1}{N_S} \sum_{i=1}^{N_S} P(t - (i - 1)) < \frac{1}{N_L} \sum_{i=1}^{N_L} P(t - (i - 1)) - X_b \rightarrow \text{sell at time } t$$

Sweeney [1986] applied filter rules on ten currencies, and Lukac et al. [1988] applied moving average and other technical rules to twelve US futures markets. They both showed that trading rules have predictive power. Brock et al. [1992] analysed the Dow Jones industrial index for a period of 89 years and reported consistent and positive results about the forecasting power of technical trading rules and in particular the Moving Average (MA) rules.

Since these seminal studies, numerous studies have examined MA rules in various world markets and generally found a good measure of support for their effectiveness (see Chen et al. [2009], Metghalchi et al. [2012]). However, it was observed that the rules tend to vary in their effectiveness over time (see Shynkevich et al. [2012]). Further, adding trading costs and other market frictions several articles found that these adjustments eliminate risk-adjusted excess profits (see Bessembinder et al. [1998], Day et al. [2002]). Some new insights were added to the literature that do not necessarily depend on the ability to make direct trading profits. For instance, Han et al. [2013] showed that applying a moving average strategy to portfolios sorted by volatility substantially outperforms a buy-and-hold strategy.

It is not clear as to why MA rules can predict future price moves. Some explanations considered short term market behaviour while other focused on the long term behaviour. It can be summarised as follows:

1. short term trends occur because of
 - (a) rational factors: the existence of short term market frictions
 - (b) behavioural factors: behavioural biases
2. the existence of long term effects tend to depend on behavioural biases such as under-reaction to news over a substantial period of time or the existence of extended periods where a particular type of market sentiment dominates.

Since the moving average rules draw on relatively long term data to make predictions, it is clear whether short term or long term models are more appropriate. In any case, the poor performance of conventional econometric models (random walk, $AR(1)$, GARCH-M and EGARCH) are not supportive of short term rational factors. Note that an MA rule can be explained by an almost infinite variety of underlying processes.

Hudson et al. [2017] proposed to model MA rule simple trend following rules such as a basic Markov chain. They showed that the ability of the rules to predict market direction is largely based on the existence of very short term trends in the data in which positive/negative daily returns are more likely to be followed by positive/negative returns. They concluded that short term market frictions were the main explanation for the success of the MA rules.

Metghalchi et al. [2018] tested the profitability of four trading rules (MA, RSI, MACD, ROC momentum) (see Appendix (11.5.1)) in the case of the OMX Iceland All-Share Index (market close from 1999 till 2016) and showed that they had predictive power. Then, they designed four trading strategies (see Appendix (11.5.2)) for each trading rule (a total of ten: 5 MA, 1 RSI, 2 MACD, 2 ROC). They obtained very good results for strategy (1) where the average of all 10 trading rules have a mean daily return of 0.0067 and annual return of 16.03%, compared with 0.00001 and 0.37% of the Buy and Hold (B&H) strategy. Strategy (2) has higher daily and annual returns than Strategy (1), but it also has higher risk. For Strategy (3) the rules have higher daily and annual returns than Strategy (1), implying that leveraging on buy days improves returns, but at higher risk (almost the double of Strategy (1)). However, these trading rules imply many times in and out of the market resulting in transaction costs and reduced net returns. In the case of Strategies (2) and (4) they estimated the one-way break-even transaction costs (BEC). The best rules are those with high BEC and low risk. Considering risk and transaction costs, they showed that three trading rules, namely MA200, MA150, and MACD, could beat the B&H strategy for the entire period.

11.8 Risk management

11.8.1 The risk of overfitting

While a simplified representation of reality can either be descriptive or predictive in nature, or both, financial models are predictive to forecast unknown, or future, values based on current or known values using mathematical equations or set of rules. However, the forecasting power of a model is limited by the appropriateness of the inputs and assumptions so that one must identify the sources of model risk to understand these limitations. Model risk generally occurs as a result of incorrect assumptions, model identification or specification errors, inappropriate estimation procedures, or in models used without satisfactory out-of-sample testing. For instance, some models can be very sensitive to small changes in inputs, resulting in big changes in outputs. Further, a model may be overfitted, meaning that it captures the underlying structure or the dynamics in the data as well as random noise. This generally occurs when too many model parameters are used, restricting the degrees of freedom relative to the size of the sample data. It often results in good in-sample fit but poor out-of-sample behaviour. Hence, while an incorrect or misspecified model can be made to fit the available data by systematically searching the parameter space, it does not have a descriptive or predictive power. Familiar examples of such problems include the spurious correlations popularised in the media, where over the past 30 years when the winner of the Super Bowl championship in American football is from a particular league, a leading stock market index historically goes up in the following months. Similar examples are plentiful in economics and the social sciences, where data are often relatively sparse but models and theories to fit the data are relatively prolific. In economic time series prediction, there may be a relatively short time-span of historical data available in conjunction with a large number of economic indicators. One particularly humorous example of this type of prediction was provided by Leinweber who achieved almost perfect prediction of annual values of the *S&P* 500 financial index as a function of annual values from previous years for butter production, cheese production, and sheep populations in Bangladesh and the United States. There are no easy technical solutions to this problem, even though various strategies have been developed. In order to avoid model overfitting and data snooping, one should decide upon the framework by defining how the model should be specified before beginning to analyse the actual data. First, by properly formulating model hypothesis making financial or economic sense, and then carefully determining the number of dependent variables in a regression model, or the number of factors and components in a stochastic model one

can expect avoiding or reducing storytelling and data mining. To increase confidence in a model, true out-of-sample studies of model performance should be conducted after the model has been fully developed. We should also be more comfortable with a model working cross-sectionally and producing similar results in different countries. Note, in a general setting, sampling bootstrapping, and randomisation techniques can be used to evaluate whether a given model has predictive power over a benchmark model (see White [2000]). All forecasting models should be monitored and compared on a regular basis, and deteriorating results from a model or variable should be investigated and understood.

11.8.2 Evaluating trading system performance

We define a successful strategy to be one that maximise the number of profitable days, as well as positive average profits over a substantial period of time, coupled with reasonably consistent behaviour. As a result, while we must look at profit when evaluating trading system performance, we must also look at other statistics such as

- maximum drawdown: the highest point in equity to the subsequent lowest point in equity. It is the largest amount of money the system lost before it recovered.
- longest flat time: the amount of time the system went without making money.
- average drawdown: maximum drawdown is one time occurrence, but the average drawdown takes all of the yearly drawdowns into consideration.
- profit to loss ratio: it represents the magnitude of winning trade dollars to the magnitude of losing trade dollars. As it tells us the ratio of wins to losses, the higher the ratio the better.
- average trade: the amount of profit or loss we can expect on any given trade.
- profit to drawdown ratio: risk in this statistic comes in the form of drawdown, whereas reward is in the form of profit.
- outlier adjusted profit: the probability of monstrous wins and/or losses reoccurring being extremely slim, it should not be included in an overall track record.
- most consecutive losses: it is the total number of losses that occurred consecutively. It gives the user an idea of how many losing trades one may have to go through before a winner occurs.
- Sharpe ratio: it indicates the smoothness of the equity curve. The ratio is calculated by dividing the average monthly or yearly return by the standard deviation of those returns.
- long and short net profit: as a robust system would split the profits between the long trades and the short trades, we need to make sure that the money made is well balanced between both sides.
- percent winning months: it checks the number of winning month out of one year.

12 Introduction to data mining

12.1 From data to information

We refer the readers to textbooks by Hand et al. [2001], Han et al. [2006], among others.

12.1.1 Data

12.1.1.1 Definitions Data is a set of values of qualitative or quantitative variables. In computing, data is information that has been translated into a form that is efficient for movement or processing. A digital computer represents a piece of data as a sequence of symbols drawn from a fixed alphabet. The most common digital computers use a binary alphabet, that is, an alphabet of two characters, typically denoted 0 and 1. Thus, data is information converted into binary digital form. It is acceptable for data to be used as a singular subject or a plural subject. More generally, data, information, knowledge and wisdom are closely related concepts, but each has its own role in relation to the other, and each term has its own meaning. Data is collected and analysed; data only becomes information suitable for making decisions once it has been analysed in some fashion. The concept of data in the context of computing was introduced by Claude Shannon who ushered in binary digital concepts based on applying two-value Boolean logic to electronic circuits. The amount of information content in a data stream may be characterized by its Shannon entropy. Data is often assumed to be the least abstract concept, information the next least, and knowledge the most abstract. Thus, data becomes information by interpretation (see Ilkka [2000]). Generally, the concept of information is closely related to notions of constraint, communication, control, data, form, instruction, knowledge, meaning, mental stimulus, pattern, perception, and representation (see Beynon-Davies [2009]). This leads to the concept of data mining.

12.1.1.2 Data set A data set consists of feature vectors, where each feature vector is a description of an object by using a set of features. For example, in a three-Gaussian data set, a feature vector can be represented by $(value1, value2, cross)$ or $(value1, value2, circle)$, where $(value1, value2)$ are (x, y) -coordinates and *cross* and *circle* are labels. The number of features of a data set is called dimension. Features are also called attributes, a feature vector is also called an instance, and a data is sometime called a sample. Labelled data is a group of samples that have been tagged with one or more labels. Labelling typically takes a set of unlabelled data and augments each piece of that unlabelled data with meaningful tags that are informative. A data class refers to a class that contains only fields and crude methods for accessing them (getters and setters). These are simply containers for data used by other classes.

12.1.2 Data mining

Data mining is the process of discovering patterns in large data sets involving methods at the intersection of machine learning, statistics, and database systems. It is an interdisciplinary subfield of computer science with an overall goal of extracting information (with intelligent method) from a data set and transform the information into a comprehensible structure for further use (see Hastie et al. [2009], Kamber et al. [2011]). The rapid growth and integration of databases provided scientists, engineers, and business people with a vast new resource that can be analysed to make scientific discoveries, optimise industrial systems, and uncover financially valuable patterns. New methods targeted at large data mining problems have been developed. Hand et al. [2001] defined Data Mining (DM) as follows:

Definition 12.1 *Data Mining is the analysis of (large) observational data sets to find unsuspected relationships and to summarise the data in novel ways that are both understandable and useful to the data owner.*

More generally, data mining is the analysis step of the Knowledge Discovery in Databases process, or KDD (see Han et al. [2006]). The goal is the extraction of patterns and knowledge from large amounts of data, not the extraction (mining) of data itself. According to the Cross Industry Standard Process for Data Mining (CRISP-DM), the six phases of the knowledge discovery in databases (KDD) process are

- Business understanding
- Data understanding
- Data preparation
- Modelling

- Evaluation
- Deployment

or a simplified process such as (1) Pre-processing, (2) Data Mining, and (3) Results Validation.

Before data mining algorithms can be used, a target data set must be assembled. A common source for data is a data mart or data warehouse. Pre-processing is essential to analyse the multivariate data sets before data mining. The target set is then cleaned. Data cleaning removes the observations containing noise and those with missing data.

There exists several functionalities to data mining, such as:

1. Data characterisation summarising the general characteristics or features of a target class of data.
2. Data discrimination comparing the general features of target class data objects with the general features of objects from a set of contrasting classes.
3. Association analysis is the discovery of association rules showing attribute value conditions occurring frequently together in a given set of data.
4. Classification is the process of finding a set of models (or functions) that describe and distinguish data classes or concepts, for the purpose of being able to use the model to predict the class of objects whose class label is unknown.

In fact, data mining involves six common classes of tasks (see Sondwale [2015]):

1. Anomaly detection (outlier/change/deviation detection): The identification of unusual data records, that might be interesting or data errors that require further investigation.
2. Association rule learning (dependency modelling): Searches for relationships between variables. For example, a supermarket might gather data on customer purchasing habits. Using association rule learning, the supermarket can determine which products are frequently bought together and use this information for marketing purposes. This is sometimes referred to as market basket analysis.
3. Clustering: is the task of discovering groups and structures in the data that are in some way or another similar, without using known structures in the data.
4. Classification: is the task of generalising known structure to apply to new data. For example, an e-mail program might attempt to classify an e-mail as legitimate or as spam.
5. Regression: attempts to find a function which models the data with the least error that is, for estimating the relationships among data or datasets.
6. Summarisation: providing a more compact representation of the data set, including visualisation and report generation.

12.1.3 Modelling

In general, a model is a predictive model that we want to construct or discover from the data set. One major task of pattern recognition and data mining, within machine learning, is to construct good models from data sets (see Otero et al. [2013]). The process of generating models from data is called learning or training. There are several types of learning, such as

- supervised learning: it consists of a specified set of classes, and example objects labelled with the appropriate class. The goal being to learn from the training objects, enabling novel objects to be identified as belonging to one of the classes.

- unsupervised learning: it is the task of inferring a function that describes the structure of unlabelled data (data that has not been classified or categorised). Among neural network models, the self-organizing map (SOM) and adaptive resonance theory (ART) are commonly used in unsupervised learning algorithms.

The learned model is called a predictor or a hypothesis (or learner). There are several types of label such as categorical or numerical.

1. If the label is categorical, such as shape, the task is also called classification and the learner is also called classifier.
2. However, if the label is numerical, such as x -coordinate, the task is also called regression and the learner is also called fitted regression model.

If the label is categorical, such as shape, the task is called classification and the learner is called a classifier. If the label is numerical, such as x -coordinate, the task is called regression and the learner is called fitted regression model. In this appendix, we will focus on supervised learning, especially classification.

12.2 Classification

12.2.1 Terminology

In the more general problem of pattern recognition, we distinguish

- Classification and clustering, which is the assignment of some sort of output value to a given input value
- Regression, which assigns a real-valued output to each input
- Sequence labelling, which assigns a class to each member of a sequence of values (for example, part of speech tagging, which assigns a part of speech to each word in an input sentence)
- Parsing, which assigns a parse tree to an input sentence, describing the syntactic structure of the sentence

Often, the individual observations are analysed into a set of quantifiable properties, known variously as explanatory variables or features. These properties may variously be

- categorical (such as A , B , AB or O , for blood type),
- ordinal (such as large, medium or small),
- integer-valued (such as the number of occurrences of a particular word in an email), or
- real-valued (such as a measurement of blood pressure).

Terminology across fields is quite varied. In statistics, where classification is often done with logistic regression or a similar procedure, the properties of observations are termed explanatory variables (or independent variables, regressors, etc.), and the categories to be predicted are known as outcomes, which are considered to be possible values of the dependent variable. In machine learning, the observations are often known as instances, the explanatory variables are termed features (grouped into a feature vector), and the possible categories to be predicted are classes.

Class label is the discrete attribute having finite values (dependent variable) whose value you want to predict based on the values of other attributes (features). The class label always takes on a finite number of different values. Classification is a type of problem whereas labelling is a function trying to label an object and classify using the information. For instance, given a set of examples of the form (attribute values, class label), we want to learn a rule that computes the label from the attribute values. For example, in binary classification, we use positive (+1) and negative (−1) to denote the two class labels.

12.2.2 Definitions

In machine learning and statistics, classification is the problem of identifying to which of a set of categories (sub-populations) a new observation belongs, on the basis of a training set of data containing observations (or instances) whose category membership is known (see Alpaydin [2010]). Classification is considered an instance of supervised learning, that is, learning where a training set of correctly identified observations is available. The corresponding unsupervised procedure is known as clustering, and involves grouping data into categories based on some measure of inherent similarity or distance.

Machine learning have been traditionally used for classification. A classification process usually consists of three main stages:

1. In the first stage, data from objects have to be collected for the design of classifiers.
2. In the second stage, feature extraction is performed to extract characteristics from the collected data to be classified such that redundant information is removed and representative information is extracted resulting in reduction of input dimensions and improved classification accuracy.
3. In the third stage, a classifier is designed using the feature data.

Classification can be thought of as two separate problems:

1. binary classification: only two classes are involved (it is a better understood task).
2. multiclass classification: involves assigning an object to one of several classes. Since many classification methods have been developed specifically for binary classification, multiclass classification often requires the combined use of multiple binary classifiers.

12.2.3 The algorithms

Most algorithms describe an individual instance whose category is to be predicted using a feature vector of individual, measurable properties of the instance. Some algorithms work only in terms of discrete data and require that real-valued or integer-valued data be discretised into groups (e.g. less than 5, between 5 and 10, or greater than 10). Other classifiers work by comparing observations to previous observations by means of a similarity or distance function.

In the literature, classification techniques and methods from traditional methods to machine learning methods can be found. Examples of the former are Linear discriminant analysis (LDA), logic based method, statistical approach, or instance-based methods. In the latter, models such as decision trees, SVM, neural networks, Bayesian belief networks, genetic algorithm etc have been considered. The ability to perform classification and be able to learn to classify objects is paramount to the process of decision making.

The process of seeking relationships within a data set involves a number of steps:

1. Model or pattern structure: determining the nature and structure of the representation to be used.
 - (a) a model structure is a global summary of a data set making statements about any point in the full measurement space,
 - (b) pattern structures only make statements about restricted regions of the space spanned by the variables.
2. Score function: deciding how to quantify and compare how well different representations fit the data (choosing a score function).

Score functions judge the quality of a fitted model, and should precisely reflect the utility of a particular predictive model.
3. Optimisation and search method: choosing an algorithmic process optimising the score function.

- (a) optimisation problem: the task of finding the best values of parameters in models
 - (b) combinatorial problem: the task of finding interesting patterns (such as rules) from a large family of potential patterns, and is often accomplished using heuristic search techniques.
4. Data management strategy: deciding what principles of data management are required for implementing the algorithms efficiently.
Data management strategy is about the ways in which the data are stored, indexed, and accessed.

Some examples of classification algorithms include:

- Linear classifiers: Fisher's linear discriminant (see Fisher [1936]), Logistic regression, Naive Bayes classifier (see Binder [1978]), Perceptron
- Support vector machines: Least squares support vector machines
- Quadratic classifiers
- Kernel estimation: k-nearest neighbour
- Boosting (meta-algorithm)
- Decision trees: Random forests
- Neural networks
- Learning vector quantization

The success of classification learning is heavily dependent on the quality of the data provided for training, as the learner only has the input to learn from. On the other hand, we want to avoid overfitting the given data set, and would rather find models or patterns generalising potential future data. Note, even though data mining is an interdisciplinary exercise, it is a process relying heavily on statistical models and methodologies. The main difference being the large size of the data sets to manipulate, requiring sophisticated search and examination methods. Further difficulties arise when there are many variables (curse of dimensionality), and often the data is constantly evolving. Recently, a lot of advances have been made on machine learning strategies mimicking human learning, and we refer the reader to Battula et al. [2013] for more details.

12.3 The challenges of computational learning

While the ability to perform classification and be able to learn to classify objects is paramount to the process of decision making, there exists several types of learning (see Mitchell [1997]), such as

- Supervised learning: learning a function from example data made of pairs of input and correct output.
- Unsupervised learning: learning from patterns without corresponding output values
- Reinforcement learning: learning with no knowledge of an exact output for a given input. Nonetheless, online or delayed feedback on the desirability of the types of behaviour can be used to help adaptation of the learning process.
- Active learning: learning through queries and responses.

More formally, these types of learning are part of what is called inductive learning where conclusions are made from specific instances to more general statements. That is, examples are provided in the form of input-output pairs $[X, f(X)]$ and the learning process consists of finding a function h (called hypothesis) which approximates a set of samples generated by the function f . The search for the function h is formulated in such a way that it can be solved by using search and optimisation algorithms.

Some of the challenges of computational learning can be summarised as follow:

- Identifying a suitable hypothesis can be computationally difficult.
- Since the function f is unknown, it is not easy to tell if the hypothesis h generated by the learning algorithm is a good approximation.
- The choice of a hypothesis space describing the set of hypotheses under consideration is not trivial.

As a result, a simple hypothesis consistent with all observations is more likely to be correct than a complex one. In the case where multiple hypotheses (an Ensemble) are generated, it is possible to combine their predictions with the aim of reducing generalisation error. For instance, boosting works as follow

- Examples in the training set are associated with different weights.
- The weights of incorrectly classified examples are increased, and the learning algorithm generates a new hypothesis from this new weighted training set. The process is repeated with an associated stopping criterion.
- The final hypothesis is a weighted-majority of all the generated hypotheses which can be based on different mixture of expert rules.

The difficult part being to know when to stop the iterative process and how to define a proper measure of error. One way forward is to consider the Probably Approximately Correct (PAC) learning which can be described as follow:

- A hypothesis is called approximately correct if its error insample lies within a small constant of the true error.
- By learning from a sufficient number of examples, one can calculate if a hypothesis has a high probability of being approximately correct.
- There is a connection between the past (seen) and the future (unseen) via an assumption stating that the training and test datasets come from the same probability distribution. It follows from the common sense that non-representative samples do not help learning.

More formally, a concept class C is said to be PAC learnable using a hypothesis class H if there exists a learning algorithm L such that for all concepts in C , for all instance distributions D on an instance space X ,

$$\forall \epsilon, \delta \text{ such that } 0 < \epsilon, \delta < 1$$

when given access to the example set, produces with probability at least $(1 - \delta)$, a hypothesis h from H with error no-more than ϵ . To specify the problem we get a set of instances X , a set of hypotheses H , a set of possible target concepts C , training instances generated by a fixed, unknown probability distribution $\mathcal{D}$ over X , a target value $c(x)$, some training examples $\langle x, c(x) \rangle$, and a hypothesis h estimating c . Then, the error of a hypothesis h satisfies

$$error_D = P_{x \in \mathcal{D}}(c(x) \neq h(x))$$

and the deviation of the true error from the training error satisfies

- Training error: $h(x) \neq c(x)$ over training instances.
- True error: $h(x) \neq c(x)$ over future random instances.

We must now measure the difference between the true error and the training error. Any hypothesis h is consistent when for all training samples,

$$h(x) = c(x)$$

If the hypothesis space H is finite, and $\mathcal{D}$ is a sequence of $m \geq 1$ independent random examples of some target concept c , then for any $0 \leq \epsilon \leq 1$, the probability that $V_{S_{H,D}}$ contains a hypothesis with error greater than ϵ is less than

$$|H|e^{-\epsilon m}$$

and $P(1 \text{ of } |H| \text{ hyps. consistent with } m \text{ exs.}) < |H|e^{-\epsilon m}$. Considering a bounded sample size, for the probability to be at most δ , that is, $|H|e^{-\epsilon m} \leq \delta$, then

$$m \geq \frac{1}{\epsilon} (\ln |H| + \ln \frac{1}{\delta})$$

More can be found on agnostic learning and infinite hypothesis space in books by Mitchell [1997] and Bishop [2006].

12.4 Evaluation and validation

Data mining can unintentionally be misused, and can then produce results which appear to be significant, but which do not actually predict future behaviour and cannot be reproduced on a new sample of data and bear little use. Often this results from investigating too many hypotheses and not performing proper statistical hypothesis testing. Not all patterns found by the data mining algorithms are necessarily valid. It is common for the data mining algorithms to find patterns in the training set which are not present in the general data set. This is called overfitting. To overcome this, the evaluation uses a test set of data on which the data mining algorithm was not trained. The learned patterns are applied to this test set, and the resulting output is compared to the desired output. A number of statistical methods may be used to evaluate the algorithm, such as Precision and Recall which are evaluated from True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). More recently Sensitivity, Specificity, Accuracy and Receiver Operating Characteristic (ROC) curves have been used to evaluate the tradeoff between true- and false-positive rates of classification algorithms.

Sensitivity and Specificity are statistical measures of the performance of a binary classification test, also known in statistics as a classification function:

- Sensitivity (also called the true positive rate, the recall, or probability of detection in some fields) measures the proportion of actual positives that are correctly identified as such (e.g., the percentage of sick people who are correctly identified as having the condition).
- Specificity (also called the true negative rate) measures the proportion of actual negatives that are correctly identified as such (e.g., the percentage of healthy people who are correctly identified as not having the condition).

In the terminology true/false positive/negative, true or false refers to the assigned classification being correct or incorrect, while positive or negative refers to assignment to the positive or the negative category.

In medicine sensitivity, specificity and accuracy are widely used statistics to describe a diagnostic test since they quantify how good and reliable a test is.

- Sensitivity evaluates how good the test is at detecting a positive disease
- Specificity estimates how likely patients without disease can be correctly ruled out
- The ROC curve is a graphic presentation of the relationship between both sensitivity and specificity helping to decide the optimal model through determining the best threshold for the diagnostic test
- Accuracy measures how correct a diagnostic test identifies and excludes a given condition. Accuracy of a diagnostic test can be determined from sensitivity and specificity with the presence of prevalence.

The terms used are true positive (TP), true negative (TN), false negative (FN), and false positive (FP). If a disease is proven present in a patient, the given diagnostic test also indicates the presence of disease, the result of the diagnostic test is considered true positive. Similarly, if a disease is proven absent in a patient, the diagnostic test suggests the disease is absent as well, the test result is true negative (TN). See details in Table (6). Both true positive and true negative suggest a consistent result between the diagnostic test and the proven condition (also called standard of truth). If the diagnostic test indicates the presence of disease in a patient who actually has no such disease, the test result is false positive (FP). Similarly, if the result of the diagnosis test suggests that the disease is absent for a patient with disease for sure, the test result is false negative (FN). Both false positive and false negative indicate that the test results are opposite to the actual condition.

Outcome of the test		Condition as determined by the Standard of Truth		
	Positive	Negative		Row total
Positive	TP	FP		$TP + FP$
Negative	FN	TN		$FN + TN$
Column total	$TP + FN$	$FP + TN$	$N = TP + TN + FP + FN$	

Table 6: Accuracy table

The evaluation measures are described in terms of TP, TN, FN and FP. The Precision and Recall are defined as follows:

$$\begin{aligned} \text{Precision}_+ &= \frac{TP}{TP + FP} \\ \text{Precision}_- &= \frac{TN}{TN + FN} \\ \text{Recall}_+ &= \frac{TP}{TP + FN} \\ \text{Recall}_- &= \frac{TN}{TN + FP} \end{aligned}$$

Precision is the weighted average of precision positive and negative, while Recall is the weighted average of recall positive and negative. The F -measure is given by

$$F\text{-measure} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The value range of these measures is between 0 and 1, where 0 indicates the worst performance, while 1 indicates the best performance.

Sensitivity, Specificity and Accuracy are defined as follows:

$$\begin{aligned} \text{Sensitivity} &= \frac{TP}{TP + FN} \\ \text{Specificity} &= \frac{TN}{TN + FP} \\ \text{Accuracy} &= \frac{TN + TP}{TN + TP + FN + FP} \end{aligned}$$

Thus, sensitivity is the proportion of true positives that are correctly identified by a diagnostic test, showing how good the test is at detecting a disease. The numerical values of sensitivity represents the probability of a diagnostic test identifies patients who do in fact have the disease. The higher the numerical value of sensitivity, the less likely diagnostic test returns false-positive results. Specificity is the proportion of the true negatives correctly identified by a diagnostic test, suggesting how good the test is at identifying normal (negative) condition. The numerical value of specificity represents the probability of a test diagnoses a particular disease without giving false-positive results. Accuracy is the proportion of true results, either true positive or true negative, in a population. It measures the degree of veracity of a diagnostic test on a condition.

Note, accuracy can be determined from sensitivity and specificity, where prevalence is known. Prevalence is the probability of disease in the population at a given time

$$\text{Accuracy} = \text{Sensitivity} \times \text{Prevalence} + \text{Specificity} \times (1 - \text{Prevalence})$$

Note, even if both sensitivity and specificity are high, say 99%, it does not suggest that the accuracy of the test is equally high as well.

For a given diagnostic test, the true positive rate (TPR) against false positive rate (FPR) can be measured, where $TRP = \frac{TP}{TP+FN}$ and $FPR = \frac{FP}{FP+TN}$. Thus, TPR is equivalent to sensitivity and FPR is equivalent to specificity. All possible combinations of TPR and FPR compose a ROC space. One TPR and one FPR together determine a single point in the ROC space, and the position of a point in the ROC space shows the tradeoff between sensitivity and specificity, that is, the increase in sensitivity is accompanied by a decrease in specificity. Thus the location of the point in the ROC space depicts whether the diagnostic classification is good or not. In an ideal situation, a point determined by both TPR and FPR yields a coordinates (0, 1), or we can say that this point falls on the upper left corner of the ROC space. This idea point indicates the diagnostic test has a sensitivity of 100% and specificity of 100%. It is also called perfect classification. Diagnostic test with 50% sensitivity and 50% specificity can be visualised on the diagonal determined by coordinate (0, 0) and coordinates (1, 0). Theoretically, a random guess would give a point along this diagonal. A point predicted by a diagnostic test fall into the area above the diagonal represents a good diagnostic classification, otherwise a bad prediction.

13 Ensemble models

We briefly introduce ensemble models. More details can be found in textbooks by Rokach [2010], Zhou [2012], among others.

We let $\mathcal{X}$ and $\mathcal{Y}$ be the input and output spaces, $\mathcal{D}$ the probability distribution, D the data set (sample), H the set of hypotheses, $I(\cdot)$ the indicator function. We let x be an instance or feature vector. For example, we consider the training data set $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$ where the instances x_i are independent and identically distributed (i.i.d.) and $y_i = f(x_i)$ where f is the ground-truth target function. That is, x_i is a vector and y_i takes value in $\mathbb{R}$. The learning process aim at constructing a learner h minimising the generalisation error

$$\mathcal{E}(h) = E_x[I(h(x) \neq f(x))]$$

We can also let $\mathcal{X}$ be the instance space, $\mathcal{Y}$ be the label space, define the labelled data set as $L = \{(x_1, y_1), \dots, (x_l, y_l)\}$ and consider the binary classification tasks where $\mathcal{Y} = \{-1, +1\}$.

13.1 Introduction

13.1.1 Presentation

Ensemble methods train multiple learners to solve the same problem by combining them. It contains a number of learners called base learners generated from a base learning algorithm (decision tree, neural network etc). The base learners are also referred to as weak learners. A weak learner is just slightly better than random guess, while a strong learner is very close to perfect performance.

Early contributions to ensemble methods are:

- combining classifiers (for pattern recognition working on strong classifiers),
- ensembles of weak learners (machine learning boosting performance from weak to strong, such as, AdaBoost, Bagging, etc), and
- mixture of experts (neural networks considering a divide-and-conquer strategy).

Hansen et al. [1990] found that predictions made by the combination of a set of classifiers are often more accurate than the ones made by the best single classifier. Further, Schapire [1990] proved that weak learners (easy to obtain) can be boosted to strong learners.

Stacking (also called stacked generalisation) proposed by Wolpert [1992] is a popular method for improving results on

data mining competitions. It is a model ensembling technique used to combine information from multiple predictive models to generate a new model (see Figure (34)). It is assumed that the stacked model (also called 2nd-level model) will outperform each of the individual models due to its smoothing nature and ability to highlight each base model where it performs best and discredit the ones where it poorly performs. Thus, stacking is most effective when the base models are significantly different. Some of the advantages of the stacking techniques are to avoid over-fitting due to K fold cross validation and to interpret non-linearity between features due to treating output scores as features.

13.1.2 Definitions

We briefly introduce ensemble methods and then present a few facts on model stacking.

Definition 13.1 *An ensemble is a set of classifiers that learn a target function, and their individual predictions are combined to classify new examples.*

An ensemble is constructed in two steps, generating the base learners, and then combining them. There are two basic ways of aggregating classification methods:

1. Before the fact: Creates solutions to be combined (for example bagging).

Suppose we want to create a robust method for determining if a given image contains a face:

- Problem: It may be very difficult and computationally intensive to create a classifier for that task
- One solution: detect eyes instead of faces
 - Advantage: A lot more efficient
 - Problem: this may produce models with low accuracy
 - Solution: Combine it with other similar classifiers

2. After the fact: Combines existing solutions (for example blending).

Netflix challenge teams merging: Different models are already built. Find an intelligent way of combining them. Given each hypothesis $\hat{f}_1, \dots, \hat{f}_M$ and a new instance x , we can use a linear regression to compute the prediction $g(x)$. That is,

$$\hat{f}_1, \dots, \hat{f}_M \rightarrow g(x) = \sum_{i=1}^M \omega_i \hat{f}_i(x)$$

Choose ω_i to minimise the error on aggregation set.

Each of these methods can be more or less appropriate to particular problems. There are two paradigms of ensemble methods

1. sequential ensemble methods: the base learners are generated sequentially (AdaBoost). It exploits the dependence between the base learners, since the overall performance can be boosted in a residual-decreasing way.
2. parallel ensemble methods: the base learners are generated in parallel (Bagging). It exploits the independence between the base learners, since the error can be reduced dramatically by combining independent base learners.

Some examples of ensemble methods are bagging, boosting, random forests, extra trees. Nowadays, stacking and blending are also ensemble methods (see details in textbook by Zhou [2012]). We now briefly introduce stacking:

Definition 13.2 *Model stacking is a learning method that aims to improve predictive accuracy by combining predictions from multiple models. It is a particular case of ensemble learning.*

It is argued that model stacking leads to better predictions, at the expense of interpretability⁹. In general, it is used as a component of several winning entries in Kaggle competitions.

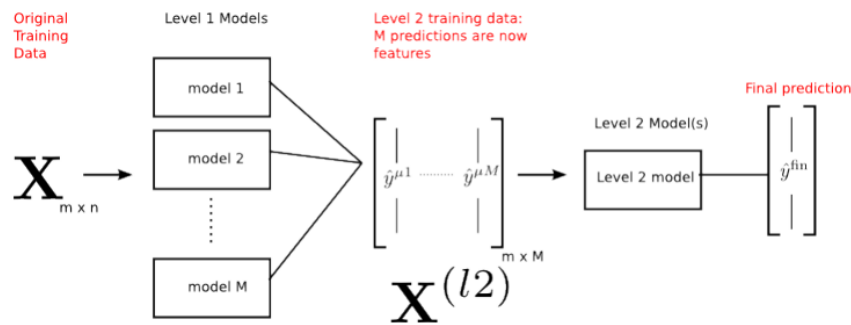


Figure 34: Two levels model stacking

13.1.3 Diversity

We call ensemble diversity the difference among the individual learners. The diversity is crucial to ensemble performance since a gain from combination requires the individual learners to be different. The major problem is that the individual learners are trained for the same task from the same training data, and thus they are usually highly correlated. However, several algorithms assume either independent or less correlated learners. Even if correlation between individual learners is considered, it is generally based on the assumption that the individual learners produce independent estimates of the posterior probabilities, which is not the case in practice. In addition, the problem of generating diverse individual learners is even more challenging if we consider that the individual learners must not be very poor, and otherwise their combination would not improve and could even worsen the performance. For example, when the performance of individual classifiers is quite poor, the added error of simple soft voted ensemble may become arbitrarily large. As a result, it is desired that the individual learners should be accurate and diverse. Nonetheless, combining only accurate learners is often worse than combining some accurate ones together with some relatively weak ones, since complementarity is more important than pure accuracy. In conclusion, the success of ensemble learning lies in achieving a good tradeoff between the individual performance and diversity. Unfortunately, there is no well-accepted formal definition of diversity, but it is crucial in ensemble learning.

13.1.4 Generalisation error

Ensemble methods are based on statistical tools and, as such, rely on error computation and error reduction. For instance, we take binary classification on classes $\{-1, +1\}$ and let f be the truth function. We suppose that each base classifier has an independent generalisation error ϵ , that is, for base classifier h_i

$$P(h_i(x) \neq f(x)) = \epsilon$$

After combining N number of such base classifiers according to

$$H(x) = \text{sgn}\left(\sum_{i=1}^N h_i(x)\right)$$

⁹In mathematical logic, interpretability is a relation between formal theories that expresses the possibility of interpreting or translating one into the other.

the ensemble H makes an error only when at least half of its base classifiers make errors. By Hoeffding inequality, the generalisation error of the ensemble is

$$P(H(x) \neq f(x)) = \sum_{k=0}^{\lfloor \frac{N}{2} \rfloor} \binom{N}{k} (1-\epsilon)^k \epsilon^{N-k} \leq e^{-\frac{1}{2}N(2\epsilon-1)^2}$$

It shows that the generalisation error reduces exponentially to the ensemble size N and ultimately approaches to zero as N approaches to infinity. Even though one can not get absolutely independent base learners generated from the same training data set, base learners with less dependence can be obtained by introducing randomness in the learning process, and a good generalisation ability can be expected by the ensemble. Further, parallel methods are favourable to parallel computing, speeding up the training process.

One can show that the generalisation error of an ensemble depends on a term related to diversity. Thus, one must estimate error decomposition to understand ensemble learning. Examples are the error-ambiguity decomposition and the bias-variance decomposition.

13.2 Some simple models

13.2.1 Decision tree

Decision tree (DT) learning is one of the most popular techniques for classification. The classification model learnt through these techniques is represented as a tree called a decision tree. It consists in a set of tree-structured decision tests working in a divide-and-conquer way as follows:

- Each non-leaf node is associated with a feature test also called a split, which splits data into different subsets according to their different values on the feature test.
- Each leaf node is associated with a label, which will be assigned to instances falling into this node.

When predicting, a series of feature tests is conducted starting from the root node, and the result is obtained when a leaf node is reached. On the other hand, the classification process starts by testing whether the value of the feature y-coordinate is larger than a threshold; if so, the instance is classified as "cross", and otherwise the tree tests whether the feature value of x-coordinate is larger than another threshold; if so, the instance is classified as "cross" and otherwise is classified as "circle". The process is recursive since in each step, a data set is given and a split is selected, then this split is used to divide the data set into subsets, and each subset is considered as the given data set for the next step. Remains to decide how to select the splits. For instance, dividing the training set D into subsets $D_1, \dots, D_k$, Breiman et al. [1984] proposed the CART which uses Gini index for selecting the split maximising the Gini

$$G_{gini}(D; D_1, \dots, D_k) = I(D) - \sum_{i=1}^k \frac{|D_i|}{|D|} I(D_i)$$

where

$$I(D) = 1 - \sum_{y \in \mathcal{Y}} P^2(y|D)$$

Alternatively, Quinlan [1998] proposed to use the information gain criterion in the ID3 algorithm. Given a training set D , the entropy of D is defined as

$$\text{Ent}(D) = - \sum_{y \in \mathcal{Y}} P(y|D) \log P(y|D)$$

If the training set D is divided into subsets $D_1, \dots, D_k$, the entropy may be reduced. The amount of the reduction is the information gain

$$G(D; D_1, \dots, D_k) = \text{Ent}(D) - \sum_{i=1}^k \frac{|D_k|}{|D|} \text{Ent}(D_k)$$

Thus, the feature-value pair which will cause the largest information gain is selected for the split. However, features with a lot of possible values will be favoured, disregarding their relevance to classification. Note, Quinlan [1993] had considered a variant of the information gain called the *C4.5* based on the gain ratio

$$P(D; D_1, \dots, D_k) = G(D; D_1, \dots, D_k) \cdot \left(\sum_{i=1}^k \frac{|D_k|}{|D|} \log \frac{|D_k|}{|D|} \right)^{-1}$$

which takes normalisation on the number of feature values. In that setting, the feature with the highest gain ratio, among features with better-than-average information gains, is selected as the split. Note, *C4.5* and *CART* can deal with numerical features.

To reduce the risk of overfitting, a general strategy is to employ pruning to cut off some tree branches caused by noise or peculiarities of the training set. Pre-pruning tries to prune branches when the tree is being grown, while post-pruning re-examines fully grown trees to decide which branches should be removed. This is achieved with the validation error: for pre-pruning, a branch will not be grown if the validation error will increase by growing the branch; for post-pruning, a branch will be removed if the removal will decrease the validation error.

Trees that are grown very deep tend to learn highly irregular patterns, but they overfit their training sets. That is, they have low bias, but very high variance.

13.2.2 Boosting

Boosting is an algorithm that converts weak learners to strong learners. We suppose the weak learner will work on any data distribution it is given, and take the binary classification task as an example. That is, we want to classify instances as positive and negative. The training instances in space $\mathcal{X}$ are drawn i.i.d. from distribution $\mathcal{D}$, and the truth function is f . Suppose the space X is made of three parts $\mathcal{X}_i$, $i = 1, 2, 3$, each takes $\frac{1}{3}$ amount of the distribution, and a learner working by random guess has 50% classification error on this problem. We want to get an accurate classifier on the problem, but can only have a weak classifier with correct classifications in spaces $\mathcal{X}_1$ and $\mathcal{X}_2$ and has wrong classifications in $\mathcal{X}_3$, thus has $\frac{1}{3}$ classification error. We denote this weak classifier as h_1 . The idea of boosting is to correct the mistakes made by h_1 by deriving a new distribution $\mathcal{D}'$ from $\mathcal{D}$, which makes the mistakes of h_1 more evident. For example, it focuses more on the instances in $\mathcal{X}_3$. Then, we can train a classifier h_2 from $\mathcal{D}'$. Again, suppose h_2 is also a weak classifier, which has correct classifications in $\mathcal{X}_1$ and $\mathcal{X}_3$ and has wrong classifications in $\mathcal{X}_2$. By combining h_1 and h_2 in an appropriate way, the combined classifier will have correct classifications in $\mathcal{X}_1$, and maybe some errors in $\mathcal{X}_2$ and $\mathcal{X}_3$. Again, we derive a new distribution $\mathcal{D}''$ to make the mistakes of the combined classifier more evident, and train a classifier h_3 from the distribution, so that h_3 has correct classifications in $\mathcal{X}_2$ and $\mathcal{X}_3$. Then, by combining h_1 , h_2 and h_3 , we have a perfect classifier, since in each space of $\mathcal{X}_i$, $i = 1, 2, 3$, at least two classifiers make correct classifications. To conclude, boosting works by training a set of learners sequentially and combining them for prediction, where the later learners focus more on the mistakes of the earlier learners. See Algorithm (10).

Algorithm 10 Boosting procedure**Require:** Input: sample distribution $\mathcal{D}$, base learning algorithm L , number of learning rounds N **Require:** $\mathcal{D}_1 = \mathcal{D}$ initialise distribution

- 1: **for** $n = 1$ **to** N **do**
- 2: $h_n = L(\mathcal{D}_n)$ train a weak learner from distribution $\mathcal{D}_n$
- 3: $\epsilon_n = P_{x \sim \mathcal{D}_n}(h_n(x) \neq f(x))$ evaluate the error of h_n
- 4: $\mathcal{D}_{n+1} = \text{Adjust-Distribution}(\mathcal{D}_n, \epsilon_n)$
- 5: **end for**

Require: Output: $H(x) = \text{Combine-Outputs}(\{h_1(x), \dots, h_n(x)\})$

Freund et al. [1997] proposed the AdaBoost to specify Adjust-Distribution and Combine-Outputs. See Algorithm (11).

Algorithm 11 AdaBoost algorithm**Require:** Input: data set $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$, base learning algorithm L , number of learning rounds N **Require:** $\mathcal{D}_1 = \frac{1}{m}$ initialise the weight distribution

- 1: **for** $n = 1$ **to** N **do**
- 2: $h_n = L(D, \mathcal{D}_n)$ train a classifier h_n from D under distribution $\mathcal{D}_n$
- 3: $\epsilon_n = P_{x \sim \mathcal{D}_n}(h_n(x) \neq f(x))$ evaluate the error of h_n
- 4: **if** $\epsilon_n > \frac{1}{2}$ **then**
- 5: break
- 6: **end if**
- 7: $\alpha_n = \frac{1}{2} \ln(\frac{1-\epsilon_n}{\epsilon_n})$ determine the weight of h_n
- 8: $\mathcal{D}_{n+1}(x) = \frac{\mathcal{D}_n(x)}{Z_n} e^{-\alpha_n f(x) h_n(x)}$ update the distribution, where Z_n is a normalisation factor which enables $\mathcal{D}_{n+1}$ to be a distribution
- 9: **end for**

Require: Output: $H(x) = \text{sgn}(\sum_{n=1}^N \alpha_n h_n(x))$

Considering a binary classification on classes $\{-1, +1\}$, Friedman et al. [2000] modified the AdaBoost algorithm by minimising the exponential loss function

$$L_{exp}(h|\mathcal{D}) = E_{x \sim \mathcal{D}}[e^{-f(x)h(x)}]$$

using additive weighted combination of weak learners as

$$H(x) = \sum_{n=1}^N \alpha_n h_n(x)$$

Minimising the exponential loss by H , the partial derivative for every x is zero, that is, $\partial_{H(x)} e^{-f(x)H(x)} = 0$, so that

$$H(x) = \frac{1}{2} \ln \frac{P(f(x) = 1|x)}{P(f(x) = -1|x)}$$

As a result,

$$\text{sgn}(H(x)) = \arg \max_{y \in \{-1, 1\}} P(f(x) = y|x)$$

which implies that $\text{sgn}(H(x))$ achieves the Bayes error rate. The H is obtained by iteratively generating h_n and α_n .

13.2.3 Bagging

Bootstrap AGGREGatING (called Bagging) is composed of bootstrap and aggregation (see Breiman [1996c]). Since the combination of independent base learners will lead to a dramatic decrease of errors, we want to get the base learners as independent as possible. Ideally, given a data set, we want to sample a number of non-overlapped data subsets and then train a base learner from each of the subsets. However, this is in general not possible due to limited data. One solution is to consider data sample manipulation, such as data perturbation. For instance, bagging adopts the bootstrap distribution (see Efron et al. [1993]) for generating different base learners. It applies bootstrap sampling to obtain the data subsets for training the base learners. Given a training data set containing m number of training examples, a sample of m training examples will be generated by sampling with replacement. By applying the process N times, N samples of m training examples are obtained. Then, from each sample a base learner can be trained by applying the base learning algorithm. The outputs of the base learners are aggregated by voting for classification and averaging for regression. If the base learners are able to output confidence values, weighted voting or weighted averaging are often used. For instance, when forecasting instance (feature vector), the algorithm feeds the instance to its base classifiers and collects all of their outputs, and then votes the labels and takes the winner label as the prediction, where ties are broken arbitrarily. See Algorithm (12).

Algorithm 12 Bagging procedure

Require: Input: data set $\{(x_1, y_1), \dots, (x_m, y_m)\}$, base learning algorithm L , number of base learner N

- 1: **for** $n = 1$ **to** N **do**
- 2: $h_n = L(D, \mathcal{D}_{bs})$ $\mathcal{D}_{bs}$ is the bootstrap distribution
- 3: **end for**

Require: Output: $H(x) = \arg \max_{y \in \mathcal{Y}} \sum_{n=1}^N I(h_n(x) = y)$

Note bagging has a large variance reduction effect which is particularly effective with unstable base learners.

13.3 Random forest

Random forests (RF) or random decision forests are an ensemble learning method for classification, regression and other tasks, that operate by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees (see Ho [1995] [1998]). The algorithm was extended by Breiman [2001]. Another extension combines the bagging idea from Breiman and random selection of features, introduced first by Ho and later independently by Amit et al. [1997], in order to construct a collection of decision trees with controlled variance.

The idea of ensemble learning is that a single classifier is not sufficient for determining class of test data since the latter is not able to distinguish between noise and pattern. Thus, random decision forests correct for decision trees' habit of overfitting to their training set. Ho established that forests of trees splitting with oblique hyperplanes can gain accuracy as they grow without suffering from overtraining, as long as the forests are randomly restricted to be sensitive to only selected feature dimensions. He eventually concluded that other splitting methods behave similarly, as long as they are randomly forced to be insensitive to some feature dimensions. Note that this observation of a more complex classifier (a larger forest) getting more accurate nearly monotonically is in sharp contrast to the common belief that the complexity of a classifier can only grow to a certain level of accuracy before being hurt by overfitting.

13.3.1 Tree bagging

The training algorithm for random forests applies the general technique of bootstrap aggregating, or bagging, to tree learners. Given a training set $X = x_1, \dots, x_m$ with responses $Y = y_1, \dots, y_m$, bagging repeatedly (N times) selects a random sample with replacement of the training set and fits trees to these samples.

Algorithm 13 Tree Bagging

```
1: for  $n = 1$  to  $N$  do  
2:   sample, with replacement,  $n$  training examples from  $X, Y$ : called  $X_n, Y_n$   
3:   train a classification or regression tree  $f_n$  on  $X_n, Y_n$   
4: end for
```

After training, predictions for unseen samples x' can be made by averaging the predictions from all the individual regression trees on x'

$$\hat{f} = \frac{1}{N} \sum_{n=1}^N f_n(x')$$

or by taking the majority vote in the case of classification trees.

This bootstrapping procedure leads to better model performance because it decreases the variance of the model, without increasing the bias. This means that while the predictions of a single tree are highly sensitive to noise in its training set, the average of many trees is not, as long as the trees are not correlated. Bootstrap sampling is a way of de-correlating the trees by showing them different training sets. An estimate of the uncertainty of the prediction can be made as the standard deviation of the predictions from all the individual regression trees on x'

$$\sigma^2 = \frac{1}{N-1} \sum_{n=1}^N (f_n(x') - \hat{f})^2$$

An optimal number of trees N can be found using cross-validation, or by observing the out-of-bag error.

13.3.2 The procedure

Random forests (RF) is an extension of bagging incorporating randomised feature selection. During the construction of a component decision tree, at each step of split selection (deterministic), RF first randomly selects a subset of features, and then carries out the conventional split selection procedure within the selected feature subset. That is, RF use a modified tree learning algorithm that selects, at each candidate split in the learning process, a random subset of the features. It is called feature bagging. Typically, for a classification problem with p features, $\sqrt{p}$ features are used in each split.

The parameter K controls the incorporation of randomness. When K equals the total number of features, the constructed decision tree is identical to the traditional deterministic decision tree, but when $K = 1$, a feature will be selected randomly. Breiman [2001] suggested $K = \log p$, the logarithm of the number of features.

Algorithm 14 Random tree algorithm

Require: Input: data set $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$, feature subset size K

```
1:  $N \leftarrow$  create a tree node based on  $D$ 
2: if all instances in the same class then
3:   return  $N$ 
4: end if
5:  $\mathcal{F} \leftarrow$  the set of features that can be split further
6: if  $\mathcal{F}$  is empty then
7:   return  $N$ 
8: end if
9:  $\tilde{\mathcal{F}} \leftarrow$  select  $K$  features from  $\mathcal{F}$  randomly
10:  $N \cdot f \leftarrow$  the feature that has the best split point in  $\tilde{\mathcal{F}}$ 
11:  $N \cdot p \leftarrow$  the best split point on  $N \cdot f$ 
12:  $D_l \leftarrow$  subset of  $D$  with values of  $N \cdot f$  smaller than  $N \cdot p$ 
13:  $D_r \leftarrow$  subset of  $D$  with values of  $N \cdot f$  no smaller than  $N \cdot p$ 
14:  $N_l \leftarrow$  call the process with parameters  $(D_l, K)$ 
15:  $N_r \leftarrow$  call the process with parameters  $(D_r, K)$ 
16: return  $N$ 
```

Require: Output: a random decision tree

Adding one further step of randomisation yields extremely randomized trees, or ExtraTrees. These are trained using bagging and the random subspace method, like in an ordinary random forest, but additionally the top-down splitting in the tree learner is randomised. Instead of computing the locally optimal feature/split combination (based on, information gain or the Gini impurity), for each feature under consideration, a random value is selected for the split. This value is selected from the feature's empirical range (in the tree's training set, that is, the bootstrap sample).

The technique is as follows: The first step in measuring the variable importance in a data set $D_n = \{(X_i, Y_i)\}_{i=1}^n$ is to fit a random forest to the data. During the fitting process the out-of-bag error for each data point is recorded and averaged over the forest (errors on an independent test set can be substituted if bagging is not used during training). To measure the importance of the j th feature after training, the values of the j th feature are permuted among the training data and the out-of-bag error is again computed on this perturbed data set. The importance score for the j th feature is computed by averaging the difference in out-of-bag error before and after the permutation over all trees. The score is normalised by the standard deviation of these differences.

As an example, we consider a random forest that performs sampling with replacement, such that, given N trees that are to be learnt, the results are based on these data set samples. Each tree is learnt using 3 features selected randomly. After the creation of n trees, when testing data is used, the decision which majority of trees come up with is considered as the final output. This approach also avoids problem of overfitting. The implementation is summarised in the Algorithm (15).

Algorithm 15 Random Forest

Require: Input: training set D , number of trees in the ensemble N **Require:** Output: a composite model M_*

```
1: for  $n = 1$  to  $N$  do
2:   create bootstrap sample  $D_n$  by sampling  $D$  with replacement
3:   select three features randomly
4:   use  $D_n$  and randomly select three features to derive tree  $M_n$ 
5: end for
6: return  $M_*$ 
```

13.4 Towards combination methods

Ensemble methods resort to combination to achieve a strong generalisation ability, where the combination method plays a crucial role. Some of the reasons for improvement are

- Statistical issue: high variance
- Computational issue: high computational variance
- Representational issue: high bias

Through combination, the variance as well as the bias of learning algorithms may be reduced.

13.4.1 Averaging

Averaging is the most fundamental combination method for numeric outputs. To illustrate the method, we take regression as an example. Thus, we consider the regression setting and suppose that we estimate N models (learners) $\{\mathcal{M}_1, \dots, \mathcal{M}_N\}$, with $h_i = \mathcal{M}_i$, $i = 1, \dots, N$, on a single data set D leading to corresponding predictions $h_1(x), \dots, h_N(x)$ with instance x (or feature vector) where $h_i(x) = \hat{f}_i(x) \in \mathbb{R}$. In model averaging, we compute the prediction by directly averaging the outputs of individual learners

$$H(x) = \hat{f}_{ave}(x) = \sum_{i=1}^N \omega_i h_i(x) \quad (13.50)$$

where ω_i , $i = 1, \dots, N$, are fixed model weights. There are several ways for choosing the weights ω_i , $i = 1, \dots, N$.

13.4.1.1 Simple averaging The simplest one being to set $\omega_i = \frac{1}{N}$ to get a simple model average. Suppose the underlying true function we try to learn is $f(x)$, and x is sampled according to a distribution $p(x)$. The output of each learner can be written as the true value plus an error item as follows:

$$h_i(x) = f(x) + \epsilon_i(x), \quad i = 1, \dots, N$$

Then, the mean squared error of h_i can be written as

$$\mathcal{E}(h) = \int (h_i(x) - f(x))^2 p(x) dx = \int \epsilon_i^2(x) p(x) dx$$

and the averaged error made by the individual learners is

$$\bar{\mathcal{E}}(h) = \frac{1}{N} \sum_{i=1}^N \int \epsilon_i^2(x) p(x) dx$$

Similarly, the expected error of the combined learner is

$$\mathcal{E}(H) = \int \left(\frac{1}{N} \sum_{i=1}^N h_i(x) - f(x) \right)^2 p(x) dx = \int \left(\frac{1}{N} \sum_{i=1}^N \epsilon_i(x) \right)^2 p(x) dx$$

Thus, one can see that

$$\mathcal{E}(H) \leq \bar{\mathcal{E}}(h)$$

which states that the expected ensemble error will be no larger than the averaged error of the individual learners. In addition, if we assume that the errors ϵ_i , $i = 1, \dots, N$, have zero mean and are uncorrelated

$$\int \epsilon_i p(x) dx = 0 \text{ and } \int \epsilon_i \epsilon_j p(x) dx = 0 \text{ for } i \neq j$$

we get

$$\mathcal{E}(H) = \frac{1}{N} \bar{\mathcal{E}}(h)$$

so that the ensemble error is smaller by a factor of N than the averaged error of the individual learners.

Remark 13.1 *In general, the errors are typically highly correlated since the individual learners are trained on the same problem.*

13.4.1.2 Weighted averaging More generally, assume $\omega_i \geq 0$ and $\sum_{i=1}^N \omega_i = 1$, then

$$\begin{aligned} \mathcal{E}(H) &= \int \left(\sum_{i=1}^N \omega_i h_i(x) - f(x) \right)^2 p(x) dx \\ &= \int \left(\sum_{i=1}^N \omega_i h_i(x) - f(x) \right) \left(\sum_{j=1}^N \omega_j h_j(x) - f(x) \right) p(x) dx \\ &= \sum_{i=1}^N \sum_{j=1}^N \omega_i \omega_j C_{ij} \end{aligned}$$

where

$$C_{ij} = \int (h_i(x) - f(x))(h_j(x) - f(x)) p(x) dx$$

and C is the correlation matrix. Thus, the optimal weights can be solved by

$$\omega = \arg \min_{\omega} \sum_{i=1}^N \sum_{j=1}^N \omega_i \omega_j C_{ij}$$

Applying the Lagrange multiplier method, and assuming the correlation matrix C to be invertible, we get

$$\omega_i = \frac{\sum_{j=1}^N C_{ij}^{-1}}{\sum_{k=1}^N \sum_{j=1}^N C_{kj}^{-1}}$$

which provides a closed-form solution to the optimal weights.

Remark 13.2 *The correlation matrix is usually singular or ill-conditioned, since the errors of the individual learners are typically highly correlated and many individual learners may be similar since they are trained on the same problem.*

Therefore ω_i is generally infeasible, and moreover, it does not guarantee non-negative solutions.

Note, voting, is special cases or variants of weighted averaging. More generally any ensemble method can be regarded as trying a specific way to decide the weights for combining the individual learners, and different ensemble methods can be regarded as different implementations of weighted averaging.

For example, given a training set with m number of training examples, we can choose the coefficients by least squares

$$\min_{\omega_i: i \in [1, N]} \sum_{k=1}^m (y_k - \hat{f}_{ave}(x_k))^2$$

where we may want to impose restrictions on the weights such as non-negativity and a total sum of one. However, the least squares estimation might not work well since the regression models $\{h_1(\cdot), \dots, h_N(\cdot)\}$ were previously estimated based on the same training data.

The minimisation does not take into account the complexity of the individual models, and will tend to overfit by putting too much weight on the most complex models (which will have low training errors).

Remark 13.3 When the data is noisy and insufficient, the estimated weights are often unreliable.

In general, simple averaging is appropriate for combining learners with similar performances, whereas if the individual learners exhibit non-identical strength, weighted averaging with unequal weights may achieve a better performance.

13.4.2 Voting

Voting is the most popular and fundamental combination method for nominal outputs. Thus, to illustrate the method, we consider classification and suppose given a set of N individual classifiers $\{h_1, \dots, h_N\}$. We want to combine the h_i to predict the class label from a set of l possible class labels $\{c_1, \dots, c_l\}$. We assume that for any instance x , the outputs of the classifier h_i are given as an l -dimensional label vector $(h_{i,1}(x), \dots, h_{i,l}(x))^T$, where $h_{i,j}(x)$ is the output of h_i for the class label c_j . The output can take different types of values according to the information provided by the individual classifiers. For instance:

- Crisp label: $h_{i,j}(x) \in \{0, 1\}$ which takes value one if h_i predicts c_j as the class label and zero otherwise.
- Class probability: $h_{i,j}(x) \in [0, 1]$ which is an estimate of the posterior probability $P(c_j|x)$.

For classifiers that produce un-normalised margins, such as SVMs, calibration methods such as Platt scaling or Isotonic Regression can be used to convert such an output to a probability.

13.4.2.1 Majority voting Majority voting (MV) is the most popular voting method. Every classifier votes for one class label, and the final output class label is the one that receives more than half of the votes; if none of the class labels receives more than half of the votes, a rejection option will be given and the combined classifier makes no prediction. The output class label of the ensemble is

$$H(x) = \begin{cases} c_j & \text{if } \sum_{i=1}^N h_{i,j}(x) > \frac{1}{2} \sum_{k=1}^l \sum_{i=1}^N h_{i,k}(x) \\ \text{rejection} & \text{if a tax credit} \end{cases}$$

For example, given a total of N classifiers for a binary classification problem, the ensemble decision will be correct if at least $\lfloor \frac{N}{2} + 1 \rfloor$ classifiers choose the correct class label.

Assume that the outputs of the classifiers are independent and each classifier has an accuracy p ¹⁰. Then, the probability of the ensemble for making a correct decision can be calculated using a binomial distribution (see Hansen et al. [1990]). The probability of obtaining at least $\lfloor \frac{N}{2} + 1 \rfloor$ correct classifiers out of N is

$$P_{mv} = \sum_{k=\lfloor \frac{N}{2} + 1 \rfloor}^N \binom{N}{k} p^k (1-p)^{N-k}$$

Lam et al. [1997] showed that

¹⁰each classifier makes a correct classification at probability p

- if $p > \frac{1}{2}$, then P_{mv} is monotonically increasing in N , and

$$\lim_{N \rightarrow \infty} P_{mv} = 1$$

- if $p < \frac{1}{2}$, then P_{mv} is monotonically decreasing in N , and

$$\lim_{N \rightarrow \infty} P_{mv} = 0$$

- if $p = \frac{1}{2}$ then $P_{mv} = \frac{1}{2}$ for any N

Remark 13.4 This result is obtained based on the assumption that the individual classifiers are statistically independent, but in practice the classifiers are generally highly correlated since they are trained on the same problem.

13.4.2.2 Plurality voting Plurality voting (PV) takes the class label which receives the largest number of votes as the final winner. That is, the output class label of the ensemble is

$$H(x) = c_{\arg \max_j \sum_{i=1}^N h_{i,j}(x)}$$

and ties are broken arbitrarily. Note, plurality voting does not have a reject option. Further, in the case of binary classification, plurality voting coincides with majority voting.

13.4.2.3 Weighted voting If the individual classifiers have unequal performance, we want to give more power to the stronger classifiers in voting; this is realised by weighted voting. The output class label of the ensemble is

$$H(x) = c_{\arg \max_j \sum_{i=1}^N \omega_i h_{i,j}(x)}$$

where ω_i is the weight assigned to the classifier h_i . Again, we want $\omega_i > 0$ and $\sum_{i=1}^N \omega_i = 1$. Further, we need to decide how to obtain the weights. We let $\ell = (\ell_1, \dots, \ell_N)^\top$ be the outputs of the individual classifiers, where ℓ_i is the class label predicted for the instance x by the classifier h_i and p_i denotes the accuracy of h_i . There is a Bayesian optimal discriminant function for the combined output on class label c_j

$$H_j(x) = \log(P(c_j)P(\ell|c_j))$$

Assuming that the outputs of the individual classifiers are conditionally independent

$$P(\ell|c_j) = \prod_{i=1}^N P(\ell_i|c_j)$$

then one can show

$$H_j(x) = \log(P(c_j)) + \sum_{i=1, \ell_i=c_j}^N \log \frac{p_i}{1-p_i} + \sum_{i=1}^N \log(1-p_i)$$

Since $\sum_{i=1}^N \log(1-p_i)$ does not depend on the class label c_j , and $\ell_i = c_j$ can be expressed by the result of $h_{i,j}(x)$, the discriminant function can be simplified to

$$H_j(x) = \log(P(c_j)) + \sum_{i=1}^N h_{i,j}(x) \log \frac{p_i}{1-p_i}$$

The first term on the RHS does not rely on the individual learners, while the second term discloses that the optimal weights for weighted voting satisfy

$$\omega_i \propto \log \frac{p_i}{1 - p_i}$$

which shows that the weights should be in proportion to the performance of the individual learners.

Remark 13.5 *The above formula obtained by assuming independence among the outputs of the individual classifiers, yet this does not hold since all the individual classifiers are trained on the same problem and they are usually highly correlated.*

13.4.2.4 Soft voting Soft voting (SV) is applied to classifiers which produce class probability outputs. The individual classifier h_i outputs a 1-dimensional vector $(h_{i,1}(x), \dots, h_{i,l}(x))^T$ for the instance x , where $h_{i,j}(x) \in [0, 1]$ can be regarded as an estimate of the posterior probability $P(c_j|x)$. In the case of uniform weighting, the simple soft voting average all the individual outputs, getting the final output for class c_j as

$$H_j(x) = \frac{1}{N} \sum_{i=1}^N h_{i,j}(x)$$

On the other hand, the weighted soft voting method can be any of the following three forms:

1. A classifier-specific weight is assigned to each classifier, and the combined output for class c_j is

$$H_j(x) = \sum_{i=1}^N \omega_i h_{i,j}(x)$$

where ω_i is the weight of the classifier h_i .

2. A class-specific weight is assigned to each classifier per class, and the combined output for class c_j is

$$H_j(x) = \sum_{i=1}^N \omega_{i,j} h_{i,j}(x)$$

where $\omega_{i,j}$ is the weight of the classifier h_i for the class c_j .

3. A weight is assigned to each example of each class for each classifier, and the combined output for class c_j is

$$H_j(x) = \sum_{i=1}^N \sum_{k=1}^m \omega_{i,j,k} h_{i,j}(x)$$

where $\omega_{i,j,k}$ is the weight of the instance x_k of the class c_j for the classifier h_i .

Focussing on case (2), we have

$$h_{i,j}(x) = P(c_j|x) + \epsilon_{i,j}(x)$$

where $\epsilon_{i,j}(x)$ is the approximation error. In classification, the target output is given as a class label. In the case of an unbiased estimation the combined output $H_j(x)$ is also unbiased, and we can obtain a variance-minimised unbiased estimation of $H_j(x)$ for $P(c_j|x)$ by estimating the weights. Minimizing the variance of the combined approximation error $\sum_{i=1}^N \omega_{i,j} \epsilon_{i,j}(x)$ under the constraints $\omega_{i,j} \geq 0$ and $\sum_{i=1}^N \omega_{i,j} = 1$, we get the optimisation problem

$$\omega_j = \arg \min_{\omega_j} \sum_{k=1}^m \left(\sum_{i=1}^N \omega_{i,j} h_{i,j}(x) - I(f(x_k) = c_j) \right)^2, j = 1, \dots, l$$

and solve for the weights. In general, soft voting is used for homogeneous ensembles. In the case of heterogeneous ensembles, one must perform a calibration where the class probability outputs are converted to class label outputs by setting $h_{i,j}(x) = 1$ if $h_{i,j}(x) = \max\{h_{i,j}(x)\}$ and 0 otherwise. The voting methods for crisp labels can then be applied.

13.5 Combining by learning

13.5.1 Stacking

The main idea of stacking is that we want to learn whether training data have been properly learned. Thus, a learner is trained to combine the individual learners. The individual learners are called the first-level learners, while the combiner is called the second-level learner, or meta-learner. Given the first-level learners we generate a new data set for training the second-level learner, where the outputs of the first-level learners are regarded as input features while the original labels are still regarded as labels of the new training data.

There are two approaches for combining models: voting and stacking. The differences between the stacking and voting framework are as follow:

- In contrast to stacking, no learning takes place at the meta-level when combining classifiers by a voting scheme.
- Label that is most often assigned to a particular instance is chosen as the correct prediction when using voting.

Practically, stacking is concerned with combining multiple classifiers generated by different learning algorithms $L_1, \dots, L_N$ on a single dataset D , which is composed by a feature vector $d_i = (x_i, y_i)$. The stacking process can be broken into two phases:

1. Generate a set of base-level classifiers $h_1, \dots, h_N$ where $h_n = L_n(D)$.
2. Train a meta-level classifier to combine the outputs of the base-level classifiers.

The implementation is summarised in the Algorithm (16).

Algorithm 16 Stacking procedure

Require: Input: data set $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$, first level learning algorithms $L_1, \dots, L_N$, second level learning algorithm L

Require: Output: a composite model M_*

```

1: for  $n = 1$  to  $N$  do
2:    $h_n = L_n(D)$  train a first-level learner by applying the first-level learning algorithm  $L_n$ 
3: end for
4:  $D' = \emptyset$  generate a new data set
5: for  $i = 1$  to  $m$  do
6:   for  $n = 1$  to  $N$  do
7:      $z_{i,n} = h_n(x_i)$  train a first-level learner by applying the first-level learning algorithm  $L_n$ 
8:   end for
9:    $D' = D' \cup ((z_{i,1}, \dots, z_{i,N}), y_i)$ 
10: end for
11:  $h' = L(D')$  train the second-level learner  $h'$  by applying the second-level learning algorithm  $L$  to the new data set  $D'$ 

```

Require: Output: $H(x) = h'(h_1(x), \dots, h_N(x))$

To avoid overfitting, it is suggested that the instances used for generating the new data set are excluded from the training examples for the first-level learners, and a cross validation or leave-one-out procedure is often recommended. The stacking framework is as follows:

- The training set for the meta-level classifier is generated through a leave-one-out cross validation process

$$\forall i = 1, \dots, m \text{ and } \forall n = 1, \dots, N \text{ then } h_n^i = L_n(D - d_i)$$

- The learned classifiers are then used to generate predictions for $d_i : z_i^n = h_n^i(x)$
- The meta-level dataset consists of examples of the form $((z_i^1, \dots, z_i^N), y_i)$, where the features are the predictions of the base-level classifiers and the class is the correct class of the example in hand.

In the case of k -fold cross-validation, the original training data set D is randomly split into k almost equal parts $D_1, \dots, D_k$. Let D_j and $D_{(-j)} = D \setminus D_j$ to be the test and training sets for the j th fold. Given N learning algorithms, a first-level learner $h_n^{(-j)}$ is obtained by using the n th learning algorithm on $D_{(-j)}$. For x_i in D_j , we let $z_{i,n}$ denotes the output of the learner $h_n^{(-j)}$ on x_i . Then, at the end of the entire cross-validation process, the new data set is generated from the N individual learners as

$$D' = \{(z_{i,1}, \dots, z_{i,N}, y_i)\}_{i=1}^m$$

It is then applied to the second-level learning algorithm and the resulting learner h' is a function of $(z_1, \dots, z_N)$ for y . Similarly to averaging described above, stacking is equivalent to solving the minimisation problem

$$\min_{\omega_m : m \in [1, M]} \sum_{i=1}^N (y_i - \hat{f}_{avecv}(x_i))^2 \quad (13.51)$$

where

$$\hat{f}_{avecv}(x) = \sum_{m=1}^M \omega_m \hat{f}_m^{(cv)}(x) \quad (13.52)$$

and $\hat{f}_m^{(cv)}(\cdot)$ are cross validation predictions. That is, model stacking fits a linear regression model based on constructed predictors derived from different models. We say that the linear regression model is therefore the meta model for the stack.

Remark 13.6 More generally, we have

$$\hat{f}_{avecv}(x) = g(\hat{f}_1^{(cv)}(x), \dots, \hat{f}_M^{(cv)}(x))$$

There is no reason for $g(\cdot)$ to be linear, except for simplicity.

This idea can be generalised and used with any algorithm as a meta model. For instance, it can be applied to classification. Some options are:

- Hard voting classifier: (weighted) majority voting across several models.
- Soft voting classifier: (weighted) probability averages.
- Model stacking: fit a meta model such as a logistic regression using cross validation predictions from multiple models as inputs.

13.5.2 Infinite ensemble

Since most ensemble methods exploit only a small finite subset of hypotheses, Lin et al. [2008] developed an infinite ensemble framework that constructs ensembles with infinite hypotheses. It is based on support vector machines (SVM) and corresponds to learning the combination weights for all possible hypotheses. By embedding infinitely many hypotheses into a kernel, it can be found that the learning problem reduces to an SVM training problem with specific kernels.

Let $\mathcal{H} = \{h_\alpha : \alpha \in \mathcal{C}\}$ denote the hypothesis space, where $\mathcal{C}$ is a measure space. The kernel that embeds $\mathcal{H}$ is defined as

$$K_{\mathcal{H},r}(x_i, x_j) = \int_{\mathcal{C}} \Phi_{x_i}(\alpha) \Phi_{x_j}(\alpha) d\alpha$$

where $\Phi_x(\alpha) = r(\alpha)h_\alpha(x)$ and $r : \mathcal{C} \rightarrow \mathbb{R}_+$ is chosen such that the integral exists for all x_i, x_j . α denotes the parameter of the hypothesis h_α , and $Z(\alpha)$ means that Z depends on α . It has been proved that the kernel is valid. Following SVM, the framework formulates the following (primal) problem:

$$\begin{aligned} \min_{\omega \in L_2(\mathcal{C}), b \in \mathbb{R}, \xi \in \mathbb{R}^m} \quad & \frac{1}{2} \int_{\mathcal{C}} \omega^2(\alpha) d\alpha + C \sum_{i=1}^m \xi_i \\ \text{s.t. } & y_i \left(\int_{\mathcal{C}} \omega(\alpha) r(\alpha) h_\alpha(x) d\alpha + b \right) \geq 1 - \xi_i \\ & \xi_i \geq 0, \forall i = 1, \dots, m \end{aligned}$$

The final classifier obtained from this optimization problem is

$$g(x) = \text{sgn} \left(\int_{\mathcal{C}} \omega(\alpha) r(\alpha) h_\alpha(x) d\alpha + b \right)$$

By using the Lagrangian multiplier method and the kernel trick, the dual problem of the primal problem can be obtained, and the final classifier can be written in terms of the kernel $K_{\mathcal{H},r}$ as

$$g(x) = \text{sgn} \left(\sum_{i=1}^m y_i \lambda_i K_{\mathcal{H},r}(x_i, x) + b \right)$$

where the λ_i are the Lagrange multipliers. By equivalence, the above equation infinite ensemble over $\mathcal{H}$. The learning problem can be reduced to solving an SVM with the kernel $K_{\mathcal{H},r}$ and the final ensemble can be obtained by applying typical SVM solvers.

13.5.3 Other methods

There are many other combination methods available and we are going to briefly present a few of them.

13.5.3.1 The algebraic methods The algebraic methods which is based on a probabilistic framework (see Kittler et al. [1998]). Again, we let $h_{i,j}(x)$ be the class probability of c_j output from h_i . Bayesian decision theory states that given N classifiers, the instance x should be assigned to the class c_j which maximises the posteriori probability $P(c_j | h_{1,j}, \dots, h_{N,j})$. From Bayes theorem, it follows that

$$P(c_j | h_{1,j}, \dots, h_{N,j}) = \frac{P(c_j) P(h_{1,j}, \dots, h_{N,j} | c_j)}{\sum_{i=1}^l P(c_i) P(h_{1,j}, \dots, h_{N,j} | c_i)}$$

where $P(h_{1,j}, \dots, h_{N,j} | c_j)$ is the joint probability distribution of the outputs from the classifiers. Assuming that the outputs are conditionally independent, that is, $P(h_{1,j}, \dots, h_{N,j} | c_j) = \prod_{i=1}^N P(h_{i,j} | c_j)$, it follows that

$$\begin{aligned} P(c_j | h_{1,j}, \dots, h_{N,j}) &= \frac{P(c_j) \prod_{i=1}^N P(h_{i,j} | c_j)}{\sum_{i=1}^l P(c_i) \prod_{k=1}^N P(h_{k,j} | c_i)} \\ &= P^{N-1}(c_j) \prod_{i=1}^N P(c_j | h_{i,j}) \end{aligned}$$

Since $h_{i,j}$ is the probability output, we have $P(c_j|h_{i,j}) = h_{i,j}$. Thus, if all classes are with equal prior, we get the product rule for combination

$$H_j(x) = \prod_{i=1}^N h_{i,j}(x)$$

Following the same approach, Kittler et al. [1998] derived soft voting method, as well as the maximum/minimum/median rules which choose the maximum/minimum/median of the individual outputs as the combined output. For instance, the median rule generates the combined output according to

$$H_j(x) = \text{med}_i(h_{i,j}(x))$$

where $\text{med}(\cdot)$ is the median statistic.

13.5.3.2 Mixture of experts Mixture of experts (ME) works in a divide-and-conquer strategy where a complex task is broken up into several simpler and smaller subtasks, and individual learners (called experts) are trained for different subtasks (see Jacobs et al. [1991], Xu et al. [1995]). Gating is usually employed to combine the experts. The individual learners are generated for different subtasks and there is no need to devote to diversity (see Figure (35)). Thus, the key problem becomes to find the natural division of the task and then derive the overall solution from sub-solutions. To do so, we make the experts local by targeting each expert to a distribution specified by the gating function, rather than the whole original training data distribution.

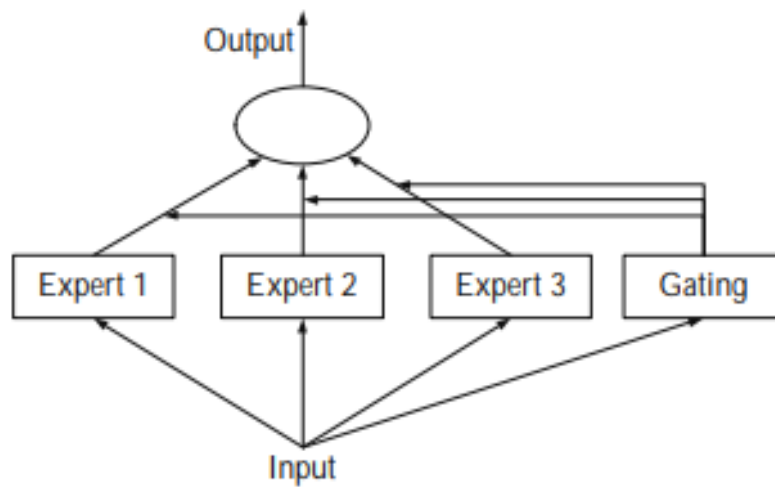


Figure 35: Mixture of experts

As an example, we assume N experts and let the output y be a discrete variable with possible values 0 and 1 for binary classification. Given an input x , each local expert h_i tries to approximate the distribution of y and obtains a local output $h_i(y|x; \theta_i)$ where θ_i is the parameter of the i th expert h_i . The gating function provides a set of coefficients $\pi_i(x; \alpha)$ weighting the contributions of experts where α is the parameter of the gating function. Thus, the final output of the ME is a weighted sum of all the local outputs produced by the experts given by

$$H(y|x; \Psi) = \sum_{i=1}^N \pi_i(x; \alpha) \cdot h_i(y|x; \theta_i)$$

where Ψ includes all unknown parameters. The output of the gating function is often modelled by the softmax function as

$$\pi_i(x; \alpha) = \frac{e^{v_i^\top x}}{\sum_{j=1}^N e^{v_j^\top x}}$$

where v_i is the weight vector of the i th expert in the gating function, and α contains all the elements in the vector v_i . When training, $\pi_i(x; \alpha)$ states the probability of the instance x appearing in the training set of the i th expert h_i . However, when testing, it defines the contribution of h_i to the final prediction.

The training procedure tries to achieve two goals: for given experts, to find the optimal gating function; for given gating function, to train the experts on the distribution specified by the gating function. The unknown parameters are usually estimated by the Expectation Maximisation (EM) algorithm (see Jordan et al. [1995], Xu et al. [1996]). Hierarchical mixture of experts (HME) extends mixture of experts (ME) into a tree structure where the experts are built from multiple levels of experts and gating functions (see Jordan et al. [1992]).

13.6 Diversity measures

A symmetric measure will keep the same when the values of 0 (incorrect) and 1 (correct) in binary classification are swapped.

13.6.1 Pairwise measures

When measuring diversity, the simplest approach is to measure the pairwise similarity/dissimilarity between two learners, and then average all the pairwise measurements to compute the overall diversity. One advantage of pairwise measures is that they can be visualized in $2d$ -plots.

Given the data set $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$, we consider binary classification and define the contingency table (see Table (7)) for two classifiers h_i and h_j , where $a + b + c + d = m$ are non-negative variables showing the numbers of examples satisfying the conditions specified by the corresponding rows and columns. We are going to present a few representative pairwise measures based on these variables.

Classifiers	$h_i = +1$	$h_i = -1$
$h_j = +1$	a	c
$h_j = -1$	b	d

Table 7: Contingency table

The disagreement measure (see Skalak [1996]) between h_i and h_j is defined as the proportion of examples on which two classifiers make different predictions

$$\text{dis}_{ij} = \frac{b + c}{m}$$

where dis_{ij} takes values in $[0, 1]$. The larger the value, the larger the diversity.

The Q -statistic (see Yule [1900]) of h_i and h_j is defined as

$$Q_{ij} = \frac{ad - bc}{ad + bc}$$

where Q_{ij} takes value in the range $[-1, 1]$. It is zero if h_i and h_j are independent, it is positive if h_i and h_j make similar predictions, otherwise it is negative.

The correlation coefficient (see Sneath et al. [1973]) of h_i and h_j is defined as

$$\rho_{ij} = \frac{ad - bc}{\sqrt{(a+b)(a+c)(c+d)(b+d)}}$$

which is classical when measuring the correlation between two binary vectors. Note, ρ_{ij} and Q_{ij} have the same sign, and $|\rho_{ij}| \geq |Q_{ij}|$.

The kappa-statistic (see Cohen [1960]) is defined as

$$\kappa_p = \frac{\Theta_1 - \Theta_2}{1 - \Theta_2}$$

where Θ_1 and Θ_2 are the probabilities that the two classifiers agree and agree by chance, respectively. The probabilities for h_i and h_j , on the data set D , are estimated as follows:

$$\begin{aligned}\Theta_1 &= \frac{a + d}{m} \\ \Theta_2 &= \frac{(a+b)(a+c) + (c+d)(b+d)}{m^2}\end{aligned}$$

Then, $\kappa_p = 1$ if the two classifiers totally agree on D , $\kappa_p = 0$ if they agree by chance, and $\kappa_p < 0$ occurs when the agreement is even less than what is expected by chance. All these measures do not require knowing the classification correctness.

13.6.2 Non-pairwise measures

We now focus on measuring the ensemble diversity directly rather than averaging pairwise measurements. We consider given a set of individual classifiers $\{h_1, \dots, h_N\}$ and a data set $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$ where x_i is an instance and $y_i \in \{-1, +1\}$ is class label.

The Kohavi-Wolpert (KW) variance is based on the bias-variance decomposition of the error of a classifier (see Kohavi et al. [1996]). On an instance x , the variability of the predicted class label y is defined as

$$var_x = \frac{1}{2} \left(1 - \sum_{y \in \{-1, +1\}} P^2(y|x) \right)$$

Kuncheva et al. [2003] modified the variability to measure diversity by considering two classifier outputs: correct (denoted by $\tilde{y} = +1$) and incorrect (denoted by $\tilde{y} = -1$). They estimated $P(\tilde{y} = +1|x)$ and $P(\tilde{y} = -1|x)$ over individual classifiers as follows:

$$P(\tilde{y} = +1|x) = \frac{\rho(x)}{N} \text{ and } P(\tilde{y} = -1|x) = 1 - \frac{\rho(x)}{N}$$

where $\rho(x)$ is the number of individual classifiers that classify x correctly. By substituting these formulas in the KW-variance and averaging over the data set D the $\kappa\omega$ measure is obtained

$$\kappa\omega = \frac{1}{mN^2} \sum_{k=1}^m \rho(x_k)(N - \rho(x_k))$$

where the larger the $\kappa\omega$ measurement, the larger the diversity.

Interrater agreement is a measure of interrater (interclassifier) reliability (see Fleiss [1981]). It can be used to measure the level of agreement within a set of classifiers (see Kuncheva et al. [2003]). It is measured as follows

$$\kappa = 1 - \frac{\frac{1}{N} \sum_{k=1}^m \rho(x_k)(N - \rho(x_k))}{m(N-1)\bar{p}(1-\bar{p})}$$

where $\rho(x_k)$ is the number of classifiers that classify x_k correctly, and

$$\bar{p} = \frac{1}{mN} \sum_{i=1}^N \sum_{k=1}^m I(h_i(x_k) = y_k)$$

is the average accuracy of individual classifiers. Again, $\kappa = 1$ if the classifiers totally agree on D , and $\kappa \leq 0$ if the agreement is even less than what is expected by chance.

Kuncheva et al. [2003] showed that the KW-variance, the averaged disagreement dis_{avg} and the kappa-statistic κ were closely related. The relations are as follows

$$\kappa\omega = \frac{(N-1)}{2N} dis_{avg} \text{ and } \kappa = 1 - \frac{N}{(N-1)\bar{p}(1-\bar{p})} \kappa\omega$$

where $\bar{p}$ is as above.

In entropy, for an instance x_k , the disagreement will be maximised if a tie occurs in the votes of individual classifiers. Cunningham et al. [2000] directly calculated the Shannon's entropy on every instance and averaged them over D for measuring diversity, getting

$$Ent_{cc} = \frac{1}{m} \sum_{k=1}^m \sum_{y \in \{-1, +1\}} -P(y|x_k) \log P(y|x_k)$$

where

$$P(y|x_k) = \frac{1}{N} \sum_{i=1}^N I(h_i(x_k) = y)$$

can be estimated by the proportion of individual classifiers that predict y as the label of x_k . The computation of Ent_{cc} does not require to know the correctness of individual classifiers.

Assuming the correctness of the classifiers known, Shipp et al. [2002] defined their entropy measure as

$$Ent_{sk} = \frac{1}{m} \sum_{k=1}^m \frac{\min(\rho(x_k), N - \rho(x_k))}{N - \lceil \frac{N}{2} \rceil}$$

where $\rho(x_k)$ is the number of individual classifiers that classify x correctly. The value of Ent_{sk} is in the range of $[0, 1]$, where 0 indicates no diversity and 1 indicates the largest diversity. Note, even though this formula does not use the logarithm function, it is easier to handle and faster to calculate.

Assuming that the diversity is maximised when the failure of one classifier is accompanied by the correct prediction of the other, Partridge et al. [1997] defined the generalised diversity as follows

$$gd = 1 - \frac{p(2)}{p(1)}$$

where

$$p(1) = \sum_{i=1}^N \frac{1}{N} p_i \text{ and } p(2) = \sum_{i=1}^N \frac{1}{N} \frac{(i-1)}{(N-1)} p_i$$

where p_i denotes the probability of i randomly chosen classifiers failing on a randomly drawn instance x . The gd value is in the range of $[0, 1]$, and the diversity is minimised when $gd = 0$. The authors also proposed a modified version of the generalised diversity, called coincident failure (CF), given by

$$cfd = \begin{cases} 0 & \text{if } p_0 = 1 \\ \frac{1}{1-p_0} \sum_{i=1}^N \frac{(N-i)}{(N-1)} p_i & \text{if } p_0 < 1 \end{cases}$$

In that setting, $cf d = 0$ if all classifiers give the same predictions simultaneously, and $cf d = 1$ if each classifier makes mistakes on unique instances.

14 Multifractal analysis

14.1 Introducing fractal analysis

14.1.1 A brief history

The Hurst Exponent (H) proposed by Hurst [1951] and tested by Hurst et al. [1965] relates to the autocorrelations of the time series, and the rate at which these decrease as the lag between pairs of values increases. It is used as a measure of long-term memory of time series, classifying the series as a random process, a persistent process, or an anti-persistent process. Hurst measured how a reservoir level (Nile River Dam project) fluctuated around its average level over time, and found that the range of the fluctuation would change, depending on the length of time used for measurement. If the series were random, the range would increase with the square root of time $T^{\frac{1}{2}}$. To standardise the measure over time, he created a dimensionless ratio by dividing the range by the standard deviation of the observation, obtaining the rescaled range analysis (R/S analysis). He found that most natural phenomena follow a biased random walk, that is, a trend with noise which could be measured by how the rescaled range scales with time, or how high the exponent H is above $\frac{1}{2}$.

In 1975 Mandelbrot (see Mandelbrot [1975] [2004]) introduced the term fractal as a geometric shape that is self-similar and has fractional dimensions, leading to the Mandelbrot set in 1977 and 1979. An object for which its topological dimension is lower or equal to its Hausdorff-Besicovitch (H-B) dimension is a fractal. By doing so, he was able to show how visual complexity can be created from simple rules. A more general definition states that a fractal is a shape made of parts similar to the whole in some way (see Feder [1988]). Two examples of fractal shapes are the Sierpinski Triange (see Figure (36)) and Koch Curve (see Figure (37)). The above Figures are bi-dimensional and deterministic fractal. Focusing on financial time series, Figure (38) illustrates a one-dimensional non-deterministic fractal representing thirty returns from the CAC40 Index at different time scales. Without any labels on the $X - Y$ scales, it is impossible to know which one is daily, weekly, or monthly returns. The fractal dimension D measures the smoothness of a surface, or, in our case, the smoothness of a time series. As discussed by Peters [1994], the importance of the fractal dimension of a time series is that it recognises that a process can be somewhere between deterministic (a line with fractal dimension of 1) and random (a fractal dimension of 1.50).

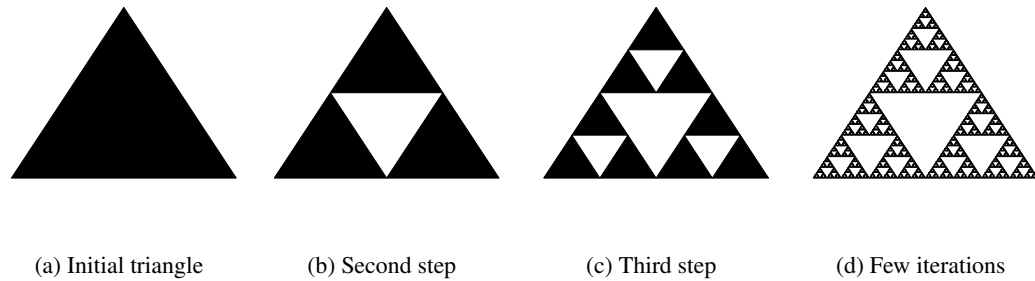


Figure 36: Sierpinski Triangle

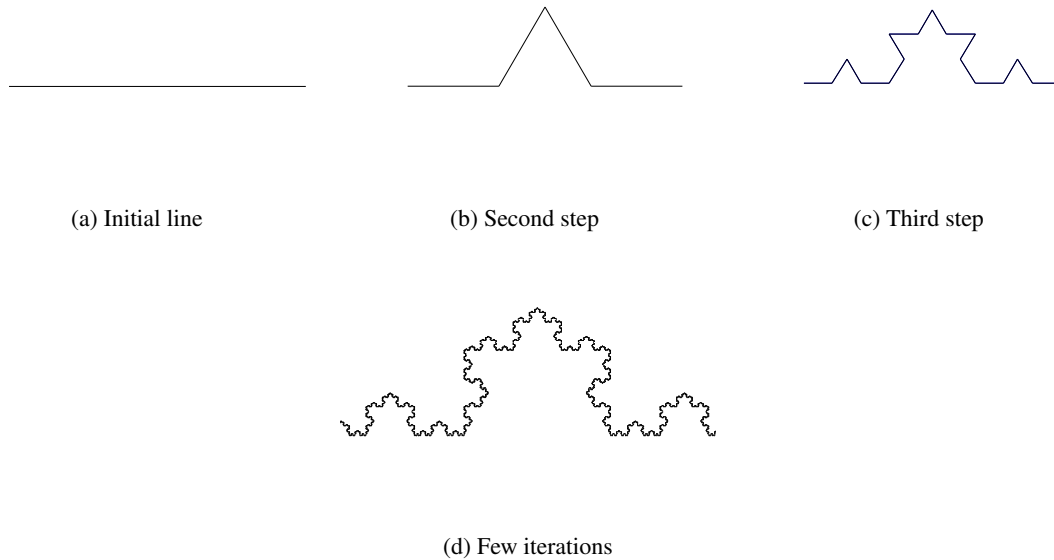


Figure 37: Koch Curve

Fractal analysis is done by conducting rescaled adjusted range analysis (R/S analysis) of time series (see Mandelbrot [?]), where the Hurst Exponent and the fractal dimension of the series are estimated. A major application of the R/S analysis is the study of price fluctuations in financial markets. There, the value of the Hurst Exponent, H , in a time series may be interpreted as an indicator of the irregularity of the price of a commodity, currency or similar quantity. Interval estimation and hypothesis testing for H are central to comparative quantitative analysis.

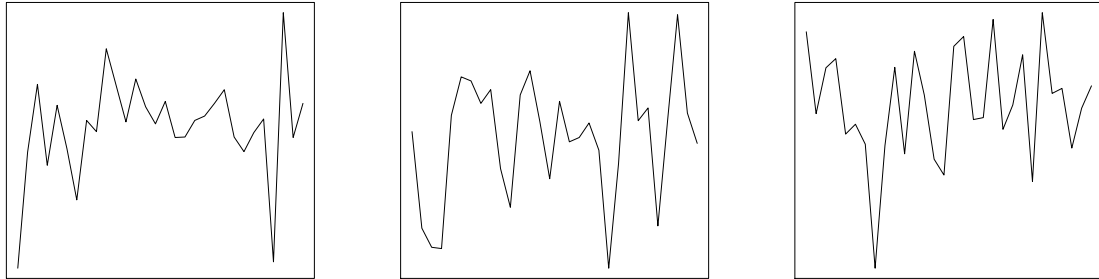


Figure 38: Self-similarity in CAC40 Index returns

14.1.2 Presenting the results

Working on the Nile River Dam Project, Hurst was concerned with the storage capacity of a reservoir, based on an estimate of the water inflow and of the need for water outflow. Studying a 847-year record from 622 A.D. to 1469 A.D., he found that the record was not random, as larger than average overflows were more likely to be followed by more large overflows. Further, the process would abruptly change to a lower than average overflow, and be followed by more lower overflows. This is the characteristics of cycles with nonperiodic length. Working on Brownian motion, Einstein [1908] found that the distance R that a random particle covers increases with the square root of time T used to measure it, that is,

$$R = T^{\frac{1}{2}} \quad (14.53)$$

which is called the T to the one-half rule commonly used in statistics. It is used in financial economics to annualise volatility or standard deviation. However, in a sample data with N elements, we can only use this equation if the time series under consideration is independent of increasing values of N . That is, Equation (14.53) only applies to Brownian motion time series. Hurst generalised the concept to non-Brownian motion by taking into account systems that are not independent. Hurst et al. [1965] remedied this drawback by dividing the range by the standard deviation S of the original observations. They proposed a more general relation by calculating the Rescaled Range as

$$\frac{R}{S} = k \times T^H$$

where k is a constant depending on the time series (see Peters [1994] [1991-96]). The $\frac{R}{S}$ value has a mean of zero, is expressed in terms of local standard deviation S , and scales as we increase the time increment T by a power law value equals to the Hurst Exponent (H). By rescaling the data we can compare diverse phenomena and time periods. The value of the Hurst Exponent varies between 0 and 1, with $H = \frac{1}{2}$ implying a random walk or an independent process (see Figure 39). Figure (40) illustrates the role of the Hurst exponent. For $0 \leq H < \frac{1}{2}$ we have anti-persistence (or ergodicity) where the process covers less distance than a random walk. It is often referred as mean reverting, meaning that if the system has been up in the previous period, it is more likely to be down in the next period. The converse being true. Anti-persistent time series exhibit higher noise and more volatility than a random series because it consists of frequent reversals. For $\frac{1}{2} < H \leq 1$ we have persistence (or trend-reinforcing) where the process covers more distance than a random walk. A persistent series has long memory effects such that the trend at a particular point in time affects the remainder of the time series. If the series has been up (down) in the last period, then the chances are that it will continue to be positive (negative) in the next period.

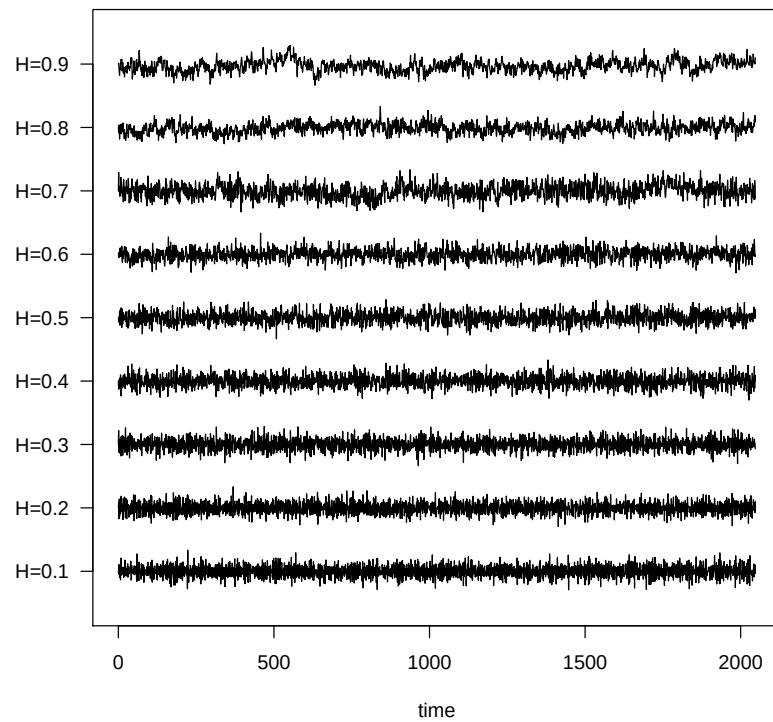
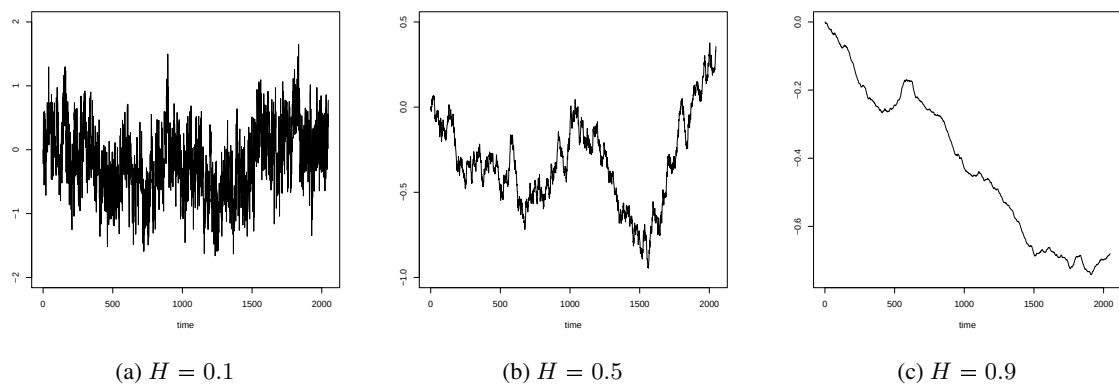


Figure 39: Range of Hurst exponents between 0.1 and 0.9

Figure 40: Cumulative observations for different values of H

The fractal dimension D of a time series measures how jagged the time series is. In order to calculate the fractal dimension of a time series we count the number of circles of a given, fixed diameter needed to cover the entire time

series. We then increase the diameter and count again the number of circles required to cover the time series. Repeating that process, the number of circles has an exponential relationship with the radius of the circle satisfying

$$N \times d^D = 1$$

where N is the number of circles, and d is the diameter. Transforming the above equation ¹¹, the fractal dimension is given by

$$D = \frac{\log(N)}{\log(\frac{1}{d})} \quad (14.54)$$

Thus, the fractal dimension of a time series is a function of scaling in time. Mandelbrot showed that the Hurst Exponent, H , is related to the fractal dimension D for a self-similar surface in n -dimensional space by the relation

$$D = n + 1 - H \quad (14.55)$$

where $0 \leq H \leq 1$. While the fractal dimension of a line is 1.0 and the fractal dimension of a geometric plane is 2.0, the fractal dimension of a random walk is half way between the line and the plane at 1.5. A value of $\frac{1}{2} < H \leq 1$ results in a fractal dimension closer to a line, so that a persistent time series would result in a smoother, less jagged line than a random walk. Similarly, for $0 < H < \frac{1}{2}$, a time series is more jagged than a random series, or has more reversals.

Definition 14.1 *The Hurst Exponent is the measure of the smoothness of fractal time series based on the asymptotic behaviour of the rescaled range of the process.*

The Hurst Exponent can be estimated by

$$H = \frac{\log(\frac{R}{S})}{\log(T)}$$

where T is the duration of the sample data, and $\frac{R}{S}$ is the corresponding value of the rescaled range.

Definition 14.2 *The Rescaled Range is the measure characterising the divergence of time series defined as the range of the mean-centred values for a given duration divided by the standard deviation of that duration.*

Persistent series are fractional Brownian motion, or biased random walk with the strength of the bias depending on how far H is above $\frac{1}{2}$. For $H \neq \frac{1}{2}$ each observation carry a long-term memory, theoretically lasting forever. A system exhibiting Hurst statistics is the result of a long stream of interconnected events where time is important as the future depends on the past. As discussed in Feder [1988], the variance of fractionally integrated processes is proportional to t^{2H} , and the impact of the present on the future can be expressed as a correlation

$$C = 2^{(2H-1)} - 1 \quad (14.56)$$

where C is the correlation measure. For $H = \frac{1}{2}$ we get $C = 0$, and events are random and uncorrelated. The R/S analysis can classify an independent series, no matter what is the shape of the underlying distribution. Note, the correlation C is not related to the Auto Correlation Function (ACF) of Gaussian random variables which assumes Gaussian, or near-Gaussian, properties in the underlying distribution.

¹¹

$$N = (\frac{1}{d})^D$$

14.2 Defining random fractals

Mandelbrot et al. [1968] introduced the fractional Brownian motion (fBm) as a generalisation of the ordinary Brownian motion by considering correlations between the increments of the process. The fBm and its corresponding generalised derivative process, the fractional Gaussian noise (fGn), are the benchmark of the Fractal Market Hypothesis (FMH). This is the only family of processes being Gaussian, self-similar, and endowed with stationary increments.

14.2.1 The fractional Brownian motion

We consider the one dimensional random process $X(t)$ with values $X(t_1), X(t_2), \dots$ such that the increment $X(t_2) - X(t_1)$ has a Gaussian distribution, and the mean square increments have a variance proportional to the time differences,

$$E[|X(t_2) - X(t_1)|^2] \propto |t_2 - t_1|$$

We say that the increments of X are statistically self-similar in the sense that

$$X(t_0 + t) - X(t_0) \text{ and } \frac{1}{\sqrt{r}}(X(t_0 + rt) - X(t_0))$$

have the same finite dimensional joint distribution functions for any t_0 and time scale $r > 0$. The integral of uncorrelated white Gaussian noise W satisfies the above equations, and is given by

$$X(t) = \int_{-\infty}^t W(s)ds$$

where the random variables $W(t)$ are uncorrelated and have the same normal distribution. The fractional Brownian motion (fBm) is a generalisation satisfying

$$\text{Var}(X(t_2) - X(t_1)) \propto |t_2 - t_1|^{2H}$$

for $0 < H < 1$. In the special case $H = \frac{1}{2}$ we recover the Brownian motion. Again, the increments of X are statistically self-similar with parameter H in the sense that

$$X(t_0 + t) - X(t_0) \text{ and } \frac{1}{r^H}(X(t_0 + rt) - X(t_0))$$

have the same finite dimensional joint distribution functions for any t_0 and $r > 0$. Setting $t_0 = 0$, the accelerated fBm $X(rt)$ is properly rescaled by dividing amplitudes by r^H .

More formally, the fractional Brownian motions, $\{B_H(t), t > 0\}$, is a Gaussian process with zero mean and stationary increments whose variance and covariance are given by

$$\begin{aligned} E[B_H^2(t)] &= \sigma^2 t^{2H} \\ E[B_H(s)B_H(t)] &= \frac{1}{2}\sigma^2(s^{2H} + t^{2H} - |t - s|^{2H}), t, s > 0 \end{aligned}$$

where $0 < H < 1, \sigma > 0$. Note, fBm are continuous but non-differentiable (in the classical sense). Note, Mandelbrot et al. [1968] defined the above as self-affinity of increments in distribution and proposed the following definition of self-similarity in distribution

Definition 14.3 A process $(X_t)_{-\infty < t < \infty}$ is called self-similar if

$$X(at) \rightarrow a^H X(t)$$

for a positive factor a and non-negative self-similarity parameter H .

A major consequence of this definition is that the moments of X , provided they exist, behave as power laws of time

$$E[|X(t)|^q] = E[|X(1)|^q] |t|^{qH} \quad (14.57)$$

In the framework of stochastic processes, such laws could only hold in distribution. In this case, Mandelbrot et al. speak of self-affine processes. The self-affinity in distribution is defined as

Definition 14.4 A process $(X_t)_{-\infty < t < \infty}$, $X(0) = 0$, is called self-affine if

$$\{X(ct_1), \dots, X(ct_k)\} \rightarrow \{c^H X(t_1), \dots, c^H X(t_k)\}$$

for non-negative factors c and k , periods $t_1, \dots, t_k$, and a positive self-similarity parameter H .

so that self-affinity in distribution is a special case of self-similarity in distribution. Definition (14.3) determines the dependency structure of the increments of a process obeying such scaling behaviour as well as their higher moments showing hyperbolic decline of their autocorrelations with an exponent depending linearly on H .

14.2.2 The multidimensional fBm

A fractional Brownian function $f(r) : \mathbb{R}^d \rightarrow \mathbb{R}$ is characterised by

- a variance

$$\sigma_H^2 = \langle [f(r + \lambda) - f(r)]^2 \rangle \propto \|\lambda\|^\alpha, \alpha = 2H$$

with $r = (x_1, \dots, x_d)$, $\lambda = (\lambda_1, \dots, \lambda_d)$, and $\|\lambda\| = \sqrt{\lambda_1^2 + \dots + \lambda_d^2}$.

- a power spectrum

$$S_H \propto \|w\|^{-\beta}, \beta = d + 2H$$

with $w = (w_1, \dots, w_d)$ the angular frequency, and $\|w\| = \sqrt{w_1^2 + \dots + w_d^2}$.

- a number of objects N_H of characteristic size ϵ needed to cover the fractal

$$N_H \propto \epsilon^{-D}, D = d + 1 - H$$

where D is the fractal dimension of $f(r)$,

In the special case where $d = 1$, the power spectrum for the fractional Brownian motion becomes

$$S_H \propto \frac{1}{|w|^\beta}$$

with $\beta = 2H + 1$ and $1 < \beta < 3$. Similarly, the power spectrum for the fractional Gaussian noise becomes

$$S_H \propto \frac{1}{|w|^\beta}$$

with $\beta = 2H - 1$ and $-1 < \beta < 1$.

14.2.3 The fractional Gaussian noise

The implications of self-similarity for time series are not based on distributions but on dynamic properties of the time series which are defined by the autocorrelation function $\gamma(k)$ (see Section ()). We denote by $\{W_H(t), t > 0\}$ the process derived from the increment of fBm, namely

$$W_H(t) = B_H(t+1) - B_H(t)$$

Beran [1994] and Embrechts et al. [2002] showed that self-similar processes have autocorrelation function defined exactly and asymptotically in the following two propositions.

Proposition 1 *Let $(X_t)_{t=0}^T$ be self-similar process with $0 < H < 1$ and finite variance $\sigma^2 < \infty$. Then the correlations are given by autocorrelation function*

$$\gamma(k) = E[X_t X_{t+k}] = \frac{(k+1)^{2H} - 2k^{2H} + (k-1)^{2H}}{2}$$

for $k \geq 0$, where H is the Hurst exponent.

Proposition 2 *Let $(X_t)_{t=0}^T$ be self-similar process with $0 < H < 1$ and finite variance $\sigma^2 < \infty$. Then the correlations are asymptotically given by autocorrelation function*

$$\gamma(k) \sim H(2H-1)k^{2H-2}$$

for $k \rightarrow \infty$, where H is the Hurst exponent.

Note, the second Proposition (2) suggests that if $H = \frac{1}{2}$, we have either an independent process, if the first Proposition (1) holds as well, or a short-term dependent process, if that first proposition is not valid. Thus, an independent process has zero correlations at all non-zero lags, while a short-term dependent process has significantly non-zero correlations at low lags but zero correlations at high lags. For $H > \frac{1}{2}$, the process has significantly positive correlations at all lags. It has hyperbolically decaying correlations which are non-summable and $\sum_{k=0}^{\infty} \gamma(k) = \infty$. For $H < \frac{1}{2}$, the process has significantly negative correlations at all lags decaying hyperbolically. However, the correlations are summable, and we get $0 < \sum_{k=0}^{\infty} \gamma(k) < \infty$ (see Embrechts et al. [2002]). The spectral density (Fourier transform of γ) is

$$f(\lambda) = C_H (2 \sin(\frac{\lambda}{2}))^2 \sum_{k=-\infty}^{\infty} \frac{1}{|\lambda + 2\pi k|^{2H+1}} \sim C_H |\lambda|^{1-2H} \text{ as } \lambda \rightarrow 0$$

where C_H is a constant.

14.2.4 The fractal process and its distribution

Following Mandelbrot et al. [1997] we now define fractal process with the separation between multi-fractal and monofractal processes of Lux [2003].

Definition 14.5 *A process $(X_t)_{-\infty < t < \infty}$ is called fractal if*

$$X(ct) \rightarrow M(c)X(t) = c^{H(c)}X(t) \quad (14.58)$$

for a positive factor c and an arbitrary non-negative function $H(c)$. If $H(c) = H$ is a constant function, the process is said to be monofractal. Otherwise, the process is said to be multi-fractal.

The scaling factor $M(c)$ is a random function with possibly different shape for different scales. One moves from one scale to another via multiplication with a random factor $M(c)$. The last equality emphasise the fact that this variability of scaling laws can be translated into variability of the exponent H . Note, a self-affine process is a special case of a self-similar process which is in turn a special case of a fractal process, all connected by the parameter H showing persistent, anti-persistent, short-term dependent and independent processes. Levy distributions are stable distributions. According to Levy, a distribution function, $F(x)$, is stable if, for all $b_1, b_2 > 0$, there also exists $b > 0$ such that

$$F\left(\frac{x}{b_1}\right) \times F\left(\frac{x}{b_2}\right) = F\left(\frac{x}{b}\right)$$

This is a general characteristic of the class of stable distribution. The characteristic functions of F can be expressed in a similar manner

$$f(b_1 t) \times f(b_2 t) = f(bt)$$

so that $f(b_1 t)$, $f(b_2 t)$, and $f(bt)$ all have the same shaped distribution, despite their being products of one another which accounts for their stability. Following Mandelbrot [?], we are now going to define the family of stable distributions, also called fractal because of their self-similar properties.

Definition 14.6 *Stable distributions are determined by characteristic function, natural logarithm of which is defined as*

$$\phi(t) = \ln E[e^{ixt}] = i\delta t - |ct|^\alpha \left(1 - i\beta \frac{t}{|t|} \tan \frac{\pi\alpha}{2}\right)$$

for $\alpha \neq 1$ and

$$\phi(t) = i\delta t - |ct|^\alpha \left(1 + i\beta \frac{2}{\pi} \ln |t|\right)$$

for $\alpha = 1$, where $0 < \alpha \leq 2$ is a characteristic exponent determining peakedness at δ , $\beta \leq |1|$ is a skewness parameter, $-\infty < \delta < \infty$ is a location parameter and $0 \leq c < \infty$ is a scale parameter.

Note, $\frac{t}{|t|}$ is the sign of t . We usually represent the parameters of a stable distribution by $S(x; \alpha, \beta, c, \delta)$. For $\alpha = 2$, $\beta = 0$ we recover the Gaussian distribution with variance equal to $c = 2\sigma^2$. Further, if $\delta = 0$, we get the standardised normal distribution with zero mean and unit variance. Setting $\alpha = 1$, $\beta = 0$, we recover the Cauchy distribution which has infinite variance and mean. The parameter α is crucial for an existence of variance. For $1 < \alpha < 2$, the distribution has infinite or undefined second moment and thus population variance. Moreover, for $0 < \alpha \leq 1$, the distribution has infinite mean as well. Even though there are a number of different ways of measuring the characteristic exponent α (see Fama [1965]), the R/S analysis and spectral analysis offer the most reliable method. Peters [1991-96], Panas [2001] and others showed that the self-similar process is connected to stable distribution of the process by the following relation between the Hurst exponent and the characteristic exponent α

$$\alpha = \frac{1}{H} \tag{14.59}$$

While H measures the fractal dimension of the time trace by the fractal dimension $2 - H$ (see Equation (14.55) with $n = 1$), it is also related to the statistical self-similarity of the process through Equation (14.59). However, $\frac{1}{H}$ measures the fractal dimension of the probability space.

14.3 Defining multifractality

Multifractality or anomalous scaling allows for a richer variation of the behaviour of a process across different scales. Equation (14.58) shows that this variability of scaling laws can be translated into the variability of the Hurst index, which is no-longer constant. Following Abry et al. [2002] and Wendt [2008] who both gave a synthetic overview of the concepts of scale invariance, scaling analysis, and multifractal analysis, we are going to introduce multifractality.

14.3.1 Fractal measures and their singularities

In nonlinear physics, one wants to characterise complicated fractal objects and describe the events occurring on them. For example, this characterisation applies to dynamical systems theory, diffusion-limited aggregation, percolation, to name a few. In general, one describes such events by dividing the object into pieces labelled by index i running from 1 to N . The size of the i th piece is l_i and the event occurring upon it is given by the number M_i . For instance, in the droplet theory of Ising model, the magnetisation has a value of the order of

$$M_i \sim l_i^y$$

where y is a critical index. Since these droplets should fill the entire space, the density of such droplets is

$$\rho(l) \sim \frac{1}{l^d} \quad (14.60)$$

where d is the Euclidean dimension of space. Halsey et al. [1986] considered the case where each M_i is a probability that some event will occur upon the i th piece. They assumed the d -dimensional time series $\{X_i\}_{i=1}^N$ with trajectories lying on a strange attractor of dimension D , $D < d$ and tried to find out how many times N_i would the time series visit the i th box. They defined $p_i = \lim_{N \rightarrow \infty} \frac{N_i}{N}$ and generated the measure on the attractor $d\mu(X)$, since $p_i = \int_{i\text{th box}} d\mu(X)$. That is, p_i is the probability (integrated measure) in the i th box. If a scaling exponent α ¹² is defined by

$$p_i^q \sim l_i^{\alpha q}$$

then α can take on a range of values, corresponding to different regions of the measure. They suggested that the number of times α would take on value between α' and $\alpha' + d\alpha'$ should be of the form

$$d\alpha' \rho(\alpha') l^{-f(\alpha')}$$

where $f(\alpha')$ is a continuous function. It reflects the differing dimensions of the sets upon which the singularities of strength α' may lie. It is roughly equivalent to Equation (14.60) with the dimension d replaced by the fractal dimension $f(\alpha)$ which varies with α . Thus, fractal measures are modelled by interwoven sets of singularities of strength α , each characterised by its own dimension $f(\alpha)$.

The function $f(\alpha)$ must then be related to observable properties of the measure. The most basic property of a strange attractor is its dimension, and the three main ones are the fractional (or similarity) dimension D of the support of the measure (see also box counting dimension), the information dimension σ , and the correlation dimension ν . Hentschel et al. [1983] showed these attractors are characterised by an infinite number of generalised dimension D_q , $q > 0$. Rather than considering droplet theory of Ising model, they let $\{X_i\}_{i=1}^N$ be the points on the attractor covering space with a mesh of d -dimensional cubes of size b^d , and $M(b)$ be the number of cubes containing points of the series. They let $p_\alpha = \frac{N_\alpha}{N}$ be the probability to fall in one of the boxes of type α and defined the rescaling hierarchy based on a box of size l^d on the n th level covered with bins of size b^d where the probability of the i th bin is denoted by $p_i(l, b)$. They showed self-similarity between the box of size l^d on the n th level of the hierarchy and the box of size $((\frac{l}{b})_\alpha)^d$ on the $(n+1)$ th level which is used to calculate the dimensions associated with the natural measure. So far, the exponent ν was related to correlations between pairs of point on the fractal, but Hentschel et al. [1983] considered higher order correlation functions or correlation integrals $C_n(l)$ and showed that

$$C_n(l) \approx C_n(l, b) = \sum_{i \in l} p_i^n(l, b)$$

scales like

¹²Lipschitz-Holder exponent

$$C_n(l, b) = C_n(l) b^{\nu_n}$$

Obtaining D_n for an integer n , they generalised it to an uncountable infinity of quantities D_q with any $q > 0$ defined by

$$D_q = \lim_{l \rightarrow 0} \left(\frac{1}{q-1} \frac{\ln \chi(q)}{\ln l} \right) \quad (14.61)$$

where

$$\chi(q) = \sum_{i \in l} p_i^q \quad (14.62)$$

is a partition function. Hence, the generalised dimensions D_q correspond to the scaling exponents for the q th moments of the measure. Note, D_0 is the fractional (similarity) dimension, D_1 is the information dimension, and D_2 is the correlation dimension. That is, multifractals have multiple dimension in the D_q versus q spectra, but monofractals stay rather flat in that area. Note, the term

$$S_q = \frac{1}{q-1} \ln \chi(q)$$

is called the Renyi entropy representing a one-parametric generalisation of Shannon's entropy (for which it reduces for $q \rightarrow 1$).

Later, Halsey et al. [1986] related the function $f(\alpha)$ to the set of dimensions D_q , and after replacing p_i with its definition above, they obtained

$$\chi(q) = \int d\alpha' \rho(\alpha') l^{-f(\alpha')} l^{q\alpha'} = \int d\alpha' \rho(\alpha') l^{q\alpha' - f(\alpha')}$$

Since l is very small, this integral is dominated by the value of α' making $q\alpha' - f(\alpha')$ smallest, provided that $\rho(\alpha')$ is nonzero. To find the minimum, they replaced α' by $\alpha(q)$, which is defined by the external condition

$$\left. \frac{d}{d\alpha'} [q\alpha' - f(\alpha')] \right|_{\alpha' = \alpha(q)} = 0$$

Differentiating one more time, we have

$$\left. \frac{d^2}{d(\alpha')^2} [q\alpha' - f(\alpha')] \right|_{\alpha' = \alpha(q)} > 0$$

such that $f'(\alpha(q)) = q$ and $f''(\alpha(q)) < q$.

Remark 14.1 As a result of the optimisation process, $\alpha(q)$ is the value for which the expression $q\alpha - f(\alpha)$ is extremal.

Then, from the definition of the generalised fractal dimensions in Equation (14.61), they obtained

$$D_q = \frac{1}{q-1} [q\alpha(q) - f(\alpha(q))] \quad (14.63)$$

so that knowing $f(\alpha)$ and the spectrum of α values, one can define D_q . Alternatively, given D_q , one can get $\alpha(q)$ ¹³ since

¹³ $\frac{d}{dq} [q\alpha(q) - f(\alpha(q))] = \alpha(q) + q \frac{d}{dq} \alpha(q) - \frac{d}{d\alpha} f(\alpha) \frac{d}{dq} \alpha(q) = \alpha(q)$ and $f'(\alpha(q)) = q$

$$\alpha(q) = \frac{d}{dq}(q-1)D_q$$

Consequently, the generalised dimensions D_q provide an alternative description of the singular measure. Generalising the definition of the dimension D_q , they considered a strange set S embedded in a finite portion of d -dimensional Euclidean space, and partitioned the set into some number of disjoint pieces $S_1, \dots, S_N$ in which each piece has a measure p_i and lies within a ball of radius l_i , where each l_i is restricted by $l_i < l$. Then given the partition function

$$\Gamma(q, \tau, \{S_i, l_i\}) = \sum_{i=1}^N \frac{p_i^q}{l_i^\tau}$$

they argued that for large N , it is of the order unity only when $\tau = (q-1)D_q$. Defining

$$\Gamma(q, \tau) = \lim_{l \rightarrow 0} \Gamma(q, \tau, l)$$

and arguing that the function $\tau(q)$ is unique, they defined D_q as

$$(q-1)D_q = \tau(q) \quad (14.64)$$

which allows to recover $\alpha(q)$ and $f(\alpha(q))$. That is, replacing in Equation (14.63), we get

$$\tau(q) = [q\alpha(q) - f(\alpha(q))] \quad (14.65)$$

and

$$\alpha(q) = \frac{d}{dq}\tau(q) \quad (14.66)$$

Note, $\tau(q)$ is the Legendre transform of $f(\alpha)$. Further, this definition of D_q precisely makes D_0 the Hausdorff dimension. They considered some simple examples to illustrate the quantities $\tau(q)$ and gain intuition about $\alpha(q)$ and $f(\alpha)$.

This multifractal formalism (MF), which is very useful in the characterisation of singular measures, accounts for the statistical scaling properties of these measures through the determination of their singularity spectrum $f(\alpha)$ which is intimately related to the generalised fractal dimension D_q . It reflects a deep connection with the thermodynamic formalism (TF) of equilibrium statistical mechanics where $\tau(q)$ and q are conjugate thermodynamic variables to $f(\alpha)$ and α . In fact, the concept of multifractality originated from a general class of multiplicative cascade models introduced by Mandelbrot [1974] in the context of fully developed turbulence (FDT). In the past, the D_q has usually been used on real, or, computer experiments, and the $f(\alpha)$ curves were determined from the Legendre transform of the $\tau(q)$ curves, which involved first smoothing the former and then Legendre transforming. A few years later, Chhabra et al. [1989] proposed a direct evaluation of $f(\alpha)$ without resorting to the intermediate Legendre transform. Denoting the probabilities in the boxes of size l as $p_j(l)$, they constructed a one-parameter family of normalised measure $\mu(q)$ given by

$$\mu_i(q, l) = \frac{p_i^q(l)}{\sum_j p_j^q(l)} \quad (14.67)$$

and they obtained the singularity spectrum as

$$\begin{aligned} f(q) &= \lim_{l \rightarrow 0} \frac{\sum_i \mu_i(q, l) \ln \mu_i(q, l)}{\ln l} \\ \alpha(q) &= \lim_{l \rightarrow 0} \frac{\sum_i \mu_i(q, l) \ln p_i(l)}{\ln l} \end{aligned} \quad (14.68)$$

which provides a relationship between the Hausdorff dimension f and an average singularity strength α as an implicit function of the parameter q . We are now going to discuss how one can estimate the $f(\alpha)$ spectrum based on the local dissipation.

14.3.2 Scaling analysis

As explained in Section (14.3.1), processes with spectra obeying a power law within a given (and sufficiently wide) range of frequencies (scales) are often referred to as $\frac{1}{f}$ processes

$$\Gamma_X(\nu) = C_0 |\nu|^{-\gamma}, \nu_m \leq |\nu| \leq \nu_M$$

where $\Gamma_X(\nu)$ is any standard spectrum estimation procedure, $0 < \gamma < 1$ and C_0 is a positive constant. Two special cases where the scale range is semi-infinite, either at small frequencies, $\nu_m \rightarrow 0$ (equivalently large scales) or at large frequencies, $\nu_M \rightarrow \infty$ (small scales), define long-range dependent processes and monofractal processes. Scale invariance is generally defined as the power law behaviours of (the time average of the q th power of) multiresolution quantities, denoted $T_X(a, t)$ with respect to the analysis scale a , for a given (large) range of scales $a \in (a_m, a_M)$, $\frac{a_M}{a_m} \gg 1$

$$\frac{1}{n_a} \sum_{k=1}^{n_a} |T_X(a, k)|^q \simeq c_q a^{\tau(q)} \quad (14.69)$$

where $T_X(a, k)$ describes the content of X around a time position t , and a scale a , such as a wavelet transform (see Muzy et al. [1991], Abry et al. [1995]). As explained in Section (14.3.1), $\tau(q)$ is a scaling exponent related to the generalised dimension via Equation (14.64). Hence, these quantities can be seen as some kind of spectral estimates.

Definition 14.7 Scaling analysis

The aim of scaling analysis is to validate the existence of power law behaviours as in Equation (14.69), and to measure the scaling exponents $\tau(q)$ that characterise them.

As suggested by Equation (14.69), the analysis and estimation procedures consist in tracking straight lines and estimating slopes in log-log plots. The estimated exponents can then be used for the physical understanding of the data or the system producing them. Note, the central theoretical notion of scale invariance is that of self-similarity, where the statistical information obtained from an object is independent of the scale of observation. However, self-similarity is a very demanding property, as it implies exact invariance to dilatation for all scale factors $a > 0$ of all finite dimensional distributions, therefore involving all statistical orders of the process. In practice, investigation of scale invariance is often restricted to the statistical order 2, or only certain range of scale factors. To be more flexible, one definition of scale invariance is obtained by relaxing the self-similarity property. First, scale invariance has to hold only for a restricted range of scaling factors $a \in (a_m, a_M)$, $\frac{a_M}{a_m} \gg 1$, rather than all $a > 0$. Second, the (single) self-similarity parameter H is replaced with the function $\tau(q)$, called the scaling function or the scaling exponents of $X(t)$.

Definition 14.8 Scale invariant stationary process

Suppose $X(t)$ is a process with stationary increments. Then it is scale invariant if

$$E[|X(at)|^q] = |a|^{\tau(q)} E[|X(t)|^q]$$

for a range of scale $a \in (a_m, a_M)$, $\frac{a_M}{a_m} \gg 1$ and some range of statistical orders q .

which implies for the increment process

$$E[|\delta_\theta X(t)|^q] = |\theta|^{\tau(q)} E[|\delta_1 X(0)|^q], 0 < \theta_m < \theta < \theta_M < \infty$$

where $\delta_\theta X(t) = X(t + \theta) - X(t)$ is a longitudinal velocity increment over a distance θ . The single parameter H is replaced with a function $\tau(q)$, which is in general non-linear ($\tau(q) \neq qH$), and hence represents a whole collection of parameters for the characterisation of the process.

While the work of Frisch et al. [1985] and Halsey et al. [1986], described in Section (14.3.1), showed that there was an infinite hierarchy of exponents allowing for a much more complete representation of the fractal measures, the related ideas of multifractality have only been applied to self-similar sets. Barabasi et al. [1991] extended the concept of multifractality to self-affine fractals, providing a more complete description of fractal surfaces. They investigated the multiscaling properties of the self-affine function $f(X)$ by calculating the q th order height-height correlation function

$$C_q(\Delta t) = \frac{1}{N} \sum_{t=1}^N |X(t, \Delta t)|^q$$

where $X(t, \Delta t) = X(t) - X(t - \Delta t)$, $N \gg 1$ is the number of points over which the average is taken, and only the terms with $|X(t, \Delta t)| > 0$ are considered. They showed that this correlation function exhibited a nontrivial multiscaling behaviour if

$$C_q(\Delta t) \sim (\Delta t)^{qH(q)} \quad (14.70)$$

with $H(q)$ changing continuously with q at least for some region of the q values. As a result, multiscaling in empirical data is typically identified by differences in the scaling behaviour of different (absolute) moments

$$E[|X(t, \Delta t)|^q] = c(q)(\Delta t)^{qH(q)} = c(q)(\Delta t)^{\tau(q)} \quad (14.71)$$

where $c(q)$ and $\tau(q)$ are deterministic functions of the order of the moment q . From Equation 14.71 we can see that $\tau(q) = qH(q)$, which motivates the definition of the generalised Hurst exponent as

$$H(q) = \frac{\tau(q)}{q} \quad (14.72)$$

A similar expression holds for mono-scaling processes such as fBm

$$E[|X(t, \Delta t)|^q] = c^H(\Delta t)^{qH} \quad (14.73)$$

Remark 14.2 *There exists different ways of defining the notion of scale invariance and multifractality in the literature.*

In terms of the behaviour of moments, monofractal, self-affine processes, are characterised by linear scaling, while multifractality (anomalous scaling) is characterised by non-linear (typically concave) shape. Hence, to diagnose multifractality, we can consider the inspection of the empirical scaling behaviour of an ensemble of moments. The traditional approach in the physics literature consists in extracting the self-similarity exponent, $\tau(q)$, from a chain of linear log-log fits of the behaviour of the various moments q for a certain selection of time aggregation steps Δt . We can therefore use regressions to the temporal scaling of moments of powers q

$$\ln E[|X(t, \Delta t)|^q] = a_0 + a_1 \ln(\Delta t) \quad (14.74)$$

and constructs the empirical $\tau(q)$ curve (for a selection of discrete q) from the ensemble of estimated regression coefficients for all q . An alternative and more widespread approach consists in looking directly at the varying scaling coefficients $H(q)$. This is called multifractal analysis which we introduce in Section (14.3.3). While the unique coefficient H quantifies a global scaling property of the underlying process, the multiplicity of such coefficients in multifractal processes, called Holder exponents, can be viewed as local scaling rates governing various patches of a time series, leading to a characteristically heterogeneous (or intermittent) appearance of such series. Focusing on the concept of Holder exponents, multifractality analysis amounts to identifying the range of such exponents rather than a degenerate single H as in the case of monofractal processes.

14.3.3 Multifractal analysis

So far, we focused on global properties such as moments and autocovariance, we are now going to adopt a more local viewpoint and examine the regularity of realised paths around a given instant. Contrary to scaling analysis which concentrate on the scaling exponent $\tau(q)$, the multifractal analysis (MFA) studies how the (pointwise) local regularity of X fluctuates in time (or space). Local Holder regularity describes the regularity of sample paths of stochastic processes by means of a local comparison against a power law function and is therefore closely related to scaling in the limit of small scales. The exponent of this power law, $h(t)$, is called the (local) Holder exponent and depends on both time and the sample path of X .

Definition 14.9 *Pointwise regularity*

A function $f(t)$, $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is $C^\alpha(t_0)$ with $\alpha > 0$, denoted as $f \in C^\alpha(t_0)$, if there exist $C > 0$, $\epsilon > 0$ and a polynomial $P_{t_0}(\theta)$ of order strictly smaller than α such that

$$\text{if } |\theta| \leq \epsilon, |f(t_0 + \theta) - P_{t_0}(\theta)| \leq C|\theta|^\alpha$$

If such polynomial $P_{t_0}(\theta)$ exists, it is unique, and its constant part is always given by $P_{t_0}(0) = f(t_0)$.

Definition 14.10 *Holder exponent*

The Holder exponent $h_f(t_0)$ of f at t_0 is

$$h_f(t_0) = \sup \{ \alpha : f \in C^\alpha(t_0) \}$$

Note, if $P_{t_0}(0) = f(t_0)$ reduces to a constant, the Holder exponent characterises the power law behaviour of the increments at t_0

$$|f(t_0 + \theta) - f(t_0)| \leq C(t_0)|\theta|^{h_f(t_0)}$$

where $C(t_0)$ is called the prefactor at time t_0 . Put another way, the exponent $h_f(t_0)$ quantifies the scaling properties of the process at time t_0 . The Holder exponent generalises the heuristic definition of singularity, so that it is also called the singularity exponent. That is, if f has Holder exponent $h(t_0) = h_f(t_0) < 1$ at t_0 , then f has at t_0 either a cusp-type singularity

$$|f(t_0 + \theta) - f(t_0)| \sim C|\theta|^{h(t_0)} \tag{14.75}$$

or an oscillating singularity

$$|f(t_0 + \theta) - f(t_0)| \sim C|\theta|^{h(t_0)} \sin\left(\frac{1}{|\theta|^\beta}\right) \tag{14.76}$$

with oscillation exponent $\beta > 0$. Conversely, if f has either a cusp or an oscillating singularity at t_0 and $1 > h(t_0)$, then $h(t_0) = h_f(t_0)$ is the Holder exponent of f at t_0 .

Multifractal situations happen when the Holder exponent is no-longer unique, but vary from point to point. More precisely, when the regularity $h(t)$ is itself a highly irregular function of t , possibly even a random process rather than a constant or a fixed deterministic function, the process X is said to be multifractal. The aim of multifractal analysis is to provide a description of the collection of Holder exponents h of the function f . Since the Holder exponent may jump from one point to another, it was understood that describing them for each time step was not of much importance. Hence, researchers focused on a global description of the regularity of the function of f in form of multifractal spectrum (also called the singularity spectrum) reflecting the size of the set of points for which the Holder exponent takes a certain value h . The measure of size most commonly used is the Hausdorff dimension which gives rise to the Hausdorff spectrum (called the multifractal spectrum). It describes the collection of Holder exponents $h(t)$ by mapping to each value of h the Hausdorff dimension $D(h)$ of the collection of points t_i at which $h_f(t_i) = h$. The

main idea being that the relative frequency of the local exponents can be represented by a renormalised density called the multifractal spectrum.

Definition 14.11 *Iso-Holder sets*

The Iso-Holder sets $I_f(h)$ is the collection of points t_i for which the Holder exponent takes a certain value h .

$$I_f(h) = \{t_i | h_f(t_i) = h\}$$

For defining the Hausdorff dimension, we first need to define the Hausdorff measure.

Definition 14.12 *Hausdorff measure*

Let $S \subset \mathbb{R}^d$, $\epsilon > 0$, and let $\gamma_\epsilon(S)$ be ϵ -coverings of S , that is, bounded sets $\{c_n\}_{n \in \mathbb{N}}$ of radius $|c_n| \leq \epsilon$ (maximal distance between two elements of c_n) that cover $S : S \subset \gamma_\epsilon(S)$. Let $C_\epsilon(S)$ be the collection of all ϵ -coverings $\gamma_\epsilon(S)$ of S . The δ -dimensional Hausdorff measure of S is

$$m_\delta(S) = \lim_{\epsilon \rightarrow 0} \inf_{C_\epsilon(S)} \sum_{\gamma_\epsilon(S)} |c_n|^\delta$$

It can be shown that either $m_\delta(S) = 0$ if $\delta < \delta_c$, or $m_\delta(S) = \infty$ if $\delta > \delta_c$. The Hausdorff dimension of S is defined as the critical value δ_c .

Definition 14.13 *Hausdorff dimension*

The Hausdorff dimension $\dim_H(S)$ of $S \subset \mathbb{R}^d$ is given by

$$\dim_H(S) = \inf_{\delta} \{m_\delta(S) = \infty\} = \sup_{\delta} \{m_\delta(S) = 0\}$$

The multifractal spectrum assigns now to each Holder exponent, as a measure of its geometric importance, the Hausdorff dimension of the set of points that share the same exponent h .

Definition 14.14 *Multifractal spectrum*

The multifractal spectrum of a function f is defined as the Hausdorff dimension of the iso-Holder sets $I_f(h)$.

$$D_f(h) = \dim_H(I_f(h))$$

We can now review the definition of monofractal and multifractal processes.

Definition 14.15 *Monofractal function or process*

A function or process $X(t)$ in $\mathbb{R}^d$ for which $h_X(t)$ is a constant, $\forall t : h_X(t) = H$, is called monofractal.

Definition 14.16 *Multifractal function or process*

A function or process $X(t)$ in $\mathbb{R}^d$ is called multifractal if it contains more than one Holder exponent h that is living on a support with non-zero Hausdorff dimension.

All homogeneous processes X with Holder exponents $h_X(x)$ not constant have their exponents which are discontinuous everywhere and hence are highly variable and change widely from point to point. As pointed out by Abry et al. [2002], one of the major consequences of multifractality in processes lies in the fact that quantities called partition functions present power law behaviours in the limit of small scales

$$S_\theta(q) = \sum_{k=1}^{\frac{1}{\theta}} |\delta_\theta X(k\theta)|^q \simeq c(q) |\theta|^{\tau(q)-1}, |\theta| \rightarrow 0$$

which correspond to Equation (14.70) with $N = \frac{1}{\theta}$. For processes with stationary increments, the time averages $\theta S_\theta(q)$ can be seen as estimators for the statistical averages $E[|\delta_\theta X(t)|^q]$ for $t = k\theta$. As a result, the above equation is highly reminiscent of the fundamental Equation (14.57) implied by self-similarity. However, the exponents $\tau(q)$ need not to follow the linear behaviour qH of self-similarity.

14.3.4 The wavelet transform and the thermodynamical formalism

Considering the multifractal spectrum of a process to be the multifractal spectrum of each of his realisations, we can speak of multifractal spectrum for both functions and processes. Hence, multifractal analysis determine the multifractal spectrum which describe a local property, the point-wise regularity of a function, globally through the geometrical importance of different Holder exponents, disregarding any information on their precise geometric repartition. Following Parisi et al. [?], under some assumptions on the homogeneity and isotropy of the statistics of local singularities, it is possible to derive a relation between self-similarity exponents $\tau(q)$ and the singularity spectrum $D(h)$. They showed that the self-similarity exponents $\tau(q)$ could be computed from the the Legendre transformation¹⁴ of the singularity spectrum $D(h)$

$$\tau(q) = \inf_h \{qh + d - D(h)\} \quad (14.77)$$

where d is the dimension of the embedding space. This is to be related to the work on fractal measures by Halsey et al. [1986] and described in Section (14.3.1).

Remark 14.3 It correspond to Equation (14.63) with Holder exponent satisfying $\alpha' = h$, that is, before it has been minimised. Once it has been minimised we can replace h with $h(q)$. We also see that the multifractal spectrum $D(h)$ corresponds to the singularity spectrum $f(\alpha(q))$.

Using this equation, one can then relate statistics and geometry. Note, the singularity spectrum is often estimated by using indirect methods which are numerically more stable, such as the Legendre spectrum. We can invert the equation above since the Legendre spectrum correspond to the Legendre transform of the self-singularity exponents $\tau(q)$. That is, the spectrum of Holder exponents (or multifractal spectrum, or Legendre spectrum) can be obtained by the Legendre transformation of the scaling function $\tau(q)$ as

$$D_L(h) = \inf_q \{qh + d - \tau(q)\}$$

which can be computed numerically since there exists a lot of estimation procedure for the scaling function $\tau(q)$. For example, Calvet et al. [2002] reported the spectrum of multiplicative measures when the random variable V is binomial, Poisson, or Gamma. Note, this method failed to fully characterise the singularity spectrum $D(h)$, since only the strongest singularities are amenable to this analysis. In order to determine the whole singularity spectrum $D(h)$, Muzy et al. [1991] considered the wavelet transform (WT) and established the foundations for a thermodynamical formalism (TF) for singular signals. In that setting, the singularity spectrum $D(h)$ is directly determined from the scaling behaviour of partition functions defined from the wavelet transform modulus maxima (WTMM). Improving on the standard partition multifractal formalism, the wavelet transform modulus maxima (WTMM) is based on wavelet analysis, and involves tracing the maxima lines in the continuous wavelet transform over all scales (see Arneodo et al. [1995]). Later, under the uniform Holder regularity condition for X , Wendt [2008] showed that $D_L(h)$ with $\tau_L(q)$ estimated from the wavelet Leader structure functions

$$S^L(j, q) = \frac{1}{n_j} \sum_{k=1}^{n_j} L_X^q(j, k)$$

where $L_X(j, k)$ are wavelet Leader coefficients, provided a tight upper bound to the multifractal spectrum $D(h)$

¹⁴The Legendre transformation is a mathematical operation transforming a function of a coordinate, $g(x)$, into a new function $h(y)$ whose argument is the derivative of $g(x)$ with respect to x , i.e., $y = \frac{dg(x)}{dx}$.

$$D(h) \leq D_L(h)$$

Remark 14.4 The structure function $S^L(j, q)$ is a special case of the time average of the q th power of multiresolution quantities $T_X(a, t)$ in Equation (14.69), where $T_X(a, t)$ has been replaced with the wavelet Leader coefficients. We will see that other representations of the quantities $T_X(a, t)$ exist, defining alternative models.

The wavelet leader multifractal formalism (WLMF) asserts that this inequality turns into an equality

$$\forall h, D(h) = D_L(h) = \inf_q \{qh + d - \tau(q)\}$$

so that the Legendre spectrum of X can be interpreted in terms of the Holder singularities of X (see Wendt [2008]). One can then compute numerically the Legendre spectrum $D_L(h)$. Hence, the scaling exponents $\tau(q)$ and the multifractal spectrum $D(h)$ are closely related via the Legendre transform. That is, the power law behaviours together with the multifractal spectrum constitute the fundamental relations establishing the connection between scale invariance and multifractality.

15 Introduction to neural networks

15.1 Artificial neural networks

While wavelet analysis enable to decompose complex systems into simpler elements, in order to understand them, we can also gather simple elements to produce a complex system. Networks is one approach among others achieving that goal. They are characterised by a set of interconnected nodes seen as computational units receiving inputs and processing them to obtain an output. The connections between nodes, which can be unidirectional or bidirectional, determine the information flow between them. We obtain a global behaviour of the network, called emergent, since the abilities of the network supercede the one of its elements, making networks a very powerful tool. Since Neural Nets have been widely studied by computer scientists, electronic engineers, biologists, psychologists etc, they have been given many different names such as Artificial Neural Networks (ANNs), Connectionism or Connectionist Models, Multi-layer Perceptrons (MLPs), or Parallel Distributed Processing (PDP) to name a few. However, a small group of classic networks emerged as dominant such as Hopfield Networks (see Hopfield [1982]), Back Propagation (see Rumelhart et al. [1986] [1986b]), Competitive Networks and networks using Spiky Neurons.

Among the different networks existing, the artificial neural networks (ANNs) and the artificial recurrent neural networks (RNNs) are computational models designed by more or less detailed analogy with biological brain modules. They are inspired from natural neurons receiving signals through synapses located on the dendrites, or membrane of the neuron. When the signals received are strong enough (surpass a certain threshold), the neuron is activated and emits a signal through the axon, which might be sent to another synapse, and might activate other neurons. ANN is a high level abstraction of real neurons consisting of inputs (like synapses) multiplied by weights (strength of the respective signals), and computed by a mathematical function determining the activation of the neuron. Another function computes the output of the artificial neuron (sometimes in dependence of a certain threshold). Depending on the weights (which can be negative), the computation of the neuron will be different, such that by adjusting the weights of an artificial neuron we can obtain the output we want for specific inputs. In presence of a large number of neurons we must rely on an algorithm to adjust the weights of the ANN to get the desired output from the network. This process of adjusting the weights is called learning or training. The aim of training the network is to obtain the optimal network minimising the difference between the output and a target, given some input data. Then, the output of the optimal network becomes the optimal predictor. The optimisation process is usually done by gradient descent method, where the weights are updated by an amount being equal to the opposite of the gradient. This way of updating weights is called the standard back propagation.

15.1.1 Presenting ANNs

Artificial neural networks (ANNs) provide a general, practical method for learning real-valued, discrete-valued, and vector-valued functions from examples. The basic architecture of an ANN is a network comprised of nodes (called *neurons* in biology) interacting with each other via incoming and outgoing weighted connections indicating the strength degree of dendrite between neurons. While McCulloch et al. [1943] introduced the first ANN model, most modelling of neural learning networks has been based on synapses of a general type described by Hebb [1949] and Eccles [1953]. Given some input, the network will be activated and this activation signal will be passed throughout the rest of the network through the connections. Generally, an ANN model is specified by its topology, node characteristics and the training algorithms. There are a variety of ANNs with various topologies, node properties and training algorithms. Rosenblatt [1962] introduced the principles of perceptrons. Perceptrons were mainly modelled with neural connections in a forward direction $A \rightarrow B \rightarrow C \rightarrow D$. Consequently, the analysis of networks with strong backward coupling $A \rightleftharpoons B \rightleftharpoons C$ proved intractable. Further, perceptron modelling required synchronous neurons like a conventional digital computer. Hopfield [1982] overcame these difficulties and triggered the research interest in the area by demonstrating the computational capabilities of networks with symmetric connections. The main contribution of the paper was the introduction of the energy function of the network which turns to be their most useful property in the context of optimisation. Two of the most important distinct types of ANNs are feedforward and feedback network. ANNs with cycles are *feedback* networks, which are also referred to as *recurrent* neural networks (RNNs). Those networks with acyclic connections are called *feedforward* neural networks (FNNs). Algorithms such as back-propagation use gradient descent to tune network parameters to best fit a training set of input-output pairs, making the learning process robust to errors in the training data (see Rumelhart et al. [1986] [1986b]). Various successful applications to practical problems developed, such as learning to recognise handwritten characters (see LeCun et al. [1989]) and spoken words (see Lang [1990]), learning to detect fraudulent use of credit cards, drive autonomous vehicles on public highways (see Pomerleau [1993]). Rumelhart et al. [1994] provided a survey of practical applications. Since McCulloch et al. [1943] introduced the first ANN model, various models were developed with different functions, accepted values, topology, learning algorithms, hybrid models where each neuron has a larger set of properties etc. While there is various types of classifiers, neural network based classifiers dominate the literature. Yet, compared to traditional NNs, higher order neural networks (HONNs) have several unique characteristics, including

- stronger approximation with faster convergence property
- greater storage capacity
- higher fault tolerance capability

However, its major drawback is the exponential growth in the number of weights with the increasing order of the network. As a special case of the feedforward HONN, the Pi-Sigma networks (PSNs) introduced by Shin et al. [1991] have the capability of higher order neural networks, but using a smaller number of weights. In order to enhance the learning capability of neural network, many researchers have improved the system by combining other techniques such as fuzzy logic (see Zadeh [1994]), genetic algorithm (see Haralad et al. [1997]).

For simplicity of exposition we are first going to present the Hopfield networks (see Hopfield [1982]) and then briefly describe the back-propagation algorithm proposed by Rumelhart et al. [1986].

15.1.2 The Hopfield networks

15.1.2.1 Description The Hopfield neural network model (see Hopfield [1982]) consists of a fully connected network of units (or neurons). The connections between the units are weighted, such that for any two units i and j , w_{ij} is the weight of the connection between them. The model assumes symmetrical weights ($w_{ij} = w_{ji}$) and in most of the cases zero self-coupling terms ($w_{ii} = 0$). A connection with positive weight is an excitatory connection, while a connection with negative weight is an inhibitory connection. A unit i is characterised by its output (or state) v_i , the activation (or network input) u_i that receives from the other units, and a threshold θ_i . The network state is given by

the output (or state) vector $v = (v_1, v_2, \dots, v_n)$.

Each unit receives input from all the other units and forwards its output to all the other units. The output of a unit is updated by the dynamics of the network. In general, the output behaviour is described as a function of the activation, where the activation depends on the weighted summation of the inputs and the threshold. In general, we follow the dynamics rule from McCulloch et al. [1943], given by

$$v_i(t+1) = \Phi(u_i(t)) = \Phi\left(\sum_{j=1}^n w_{ij}v_j(t) + \theta_i\right) \quad (15.78)$$

where the activation is

$$u_i(t) = w_i \cdot v(t) + \theta_i = \sum_{j=1}^n w_{ij}v_j(t) + \theta_i \quad (15.79)$$

and θ_i is the threshold. The activation function $\Phi(x)$ can take several forms:

- it can be the unit step function (Heaviside function)

$$\Theta(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

in which case we have a discrete Hopfield network with binary states $\{0, 1\}$.

- the sign function

$$\text{sgn}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

in which case the network is discrete but with values $\{1, -1\}$.

- a network with continuous-valued units (analog Hopfield network), where the values fall in the range $[0, 1]$ or $[-1, 1]$ can be obtained with the sigmoid (or logistic) function

$$g_\beta(x) = \frac{1}{1 + e^{-2\beta x}}$$

The parameter β is usually taken as $\beta = \frac{1}{T}$ where T is a virtual temperature. The latter adjusts the sharpness of the sigmoid (or hyperbolic tangent) function and at the limit $T \rightarrow 0$ (absolute temperature) the output becomes discrete. We can also use the hyperbolic tangent function

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Note that

$$\text{sgn}(x) = 2\Theta(x) - 1 \text{ and } \tanh(\beta x) = 2g_\beta(x) - 1$$

so that the $\{0, 1\}$ (or $[0, 1]$) model can be easily transformed to $\{-1, 1\}$ (or $[-1, 1]$) and vice-versa. In the limit $\beta \rightarrow \infty$ the values become discrete.

All the networks above are deterministic in the sense that the next state is an explicit function of the previous state and the characteristics of the network. In the case of the stochastic Hopfield networks the probability of a unit to be in a particular state is drawn from a probability distribution such as Boltzmann or Cauchy. Alternatively, the stochastic network can be viewed as a deterministic network, where the threshold of a unit is variable and is drawn from a probability density.

Note, the updating policy can be

- synchronous, in which case all the units are updated simultaneously at each time step, or
- asynchronous, where either the units are updated in sequence one at each time step (sequential asynchronous update) or at each time step one randomly chosen unit is updated (random asynchronous update).

In summary, the properties that characterize different Hopfield networks are

- Discrete vs Continuous-Valued Units
- Deterministic vs Stochastic Networks
- Synchronous vs Asynchronous Update (Sequential or Random)

15.1.2.2 The energy function The essential ingredient of the learning process is the Hebbian property which is the modification of w_{ij} by correlations like

$$\Delta w_{ij} = [v_i(t)v_j(t)]_{average}$$

where the average is some appropriate calculation over past history. In addition, the Hopfield model has stable limit points. That is, the energy function of a Hopfield network is a function defined over the state space of the network given by

$$\begin{aligned} E(v) &= -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n v_i v_j w_{ij} - \sum_{i=1}^n v_i \theta_i \\ &= -\sum_{i=1}^n v_i \left(\frac{1}{2} \sum_{j=1}^n v_j w_{ij} + \theta_i \right) \\ &= -\sum_{i=1}^n v_i u_i = -u \cdot v \end{aligned} \quad (15.80)$$

Note, this function always decreases (not necessarily monotonically because of the threshold) during the evolution of the network due to the dynamics of the systems given in Equation (15.78). State changes (altering v_i) will continue until a least (local) E is reached. This case is isomorphic with an Ising model.

Thus, the energy function can be viewed as defining an energy landscape and the dynamics can be thought of as the motion of a small sphere on the energy surface under the influence of gravity and friction. Consequently, the network performs as a minimiser of the energy function and will reach a local or global minimum of the function during its evolution.

One can show that whenever a unit i changes state, the energy difference ΔE_i is given by

$$\Delta E_i = -u_i \Delta v_i = -\left(\sum_{j=1}^n w_{ij} v_j + \theta_i \right) \Delta v_i \quad (15.81)$$

where u_i is the activation in Equation (15.79). As an example, we consider the $\{0, 1\}$ network with dynamics given in Equation (15.78). Then

- If the activation is positive, then either $\Delta u_i = 0$ ($1 \rightarrow 1$) or $\Delta u_i = 1$ ($0 \rightarrow 1$) and thus $\Delta E_i \leq 0$.
- If the activation is negative, then either $\Delta u_i = 0$ ($0 \rightarrow 0$) or $\Delta u_i = -1$ ($1 \rightarrow 0$) and thus $\Delta E_i \leq 0$.

The argument is similar for the other models. Note, the main property of the energy function given in Equation (15.80) is held only if the weights of the connections are symmetric and the self-coupling terms are zero or positive. This is the case for the Hopfield network so that its energy function is called Lyapunov function.

15.1.3 The multilayer perceptron

15.1.3.1 Description The *multilayer perceptron* (MLP) is formed in layers, the idea being that multilayer ANN can approximate any continuous function. This algorithm is a layer feed-forward ANN, since the artificial neurons are organised in layers with signals sent forward and with errors propagated backwards. The network receives inputs via neurons in the input layer, and the output of the network is given via neurons on the output layers. There may be one or more intermediate hidden layers. For clarity, we consider a network with three layers, that is, an input layer, a hidden layer, and an output layer. The hidden layer is used for capturing the non-linear relationships among variables. It is illustrated in Figure 41, where the bottom layer is the *input layer*. Then the information is propagated to the middle layer, called the *hidden layer*. Finally, the output layer receives the incoming value. This process is termed the *forward pass*. Note that the S-shaped curve is the squashing function which forces the output of the unit to fall between certain bounded interval. Except for the input layer, the output value of each unit in the other layers should be passed through the squashing function before transferring to the next layer. The back-propagation algorithm uses supervised learning where we provide the algorithm with examples of the inputs and outputs we want the network to compute, and then the error (difference between actual and expected results) is calculated. Defining the network function f_{out} as a particular implementation of a composite function from input to output space, the learning problem consists in finding the optimal combination of weights so that f_{out} approximates a given function f as closely as possible. However, in practice the function f is not given explicitly but only implicitly through some examples.

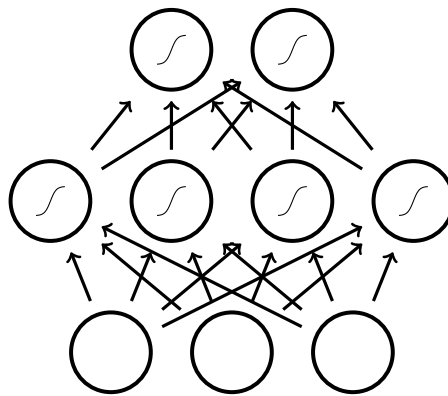


Figure 41: **A multilayer perceptron.** The three layers are input, hidden, and output layers, respectively, from bottom to top. The S-shaped curves in the hidden and output layers are the sigmoid squashing functions applying to the net values.

15.1.3.2 The universal approximation theorem Cybenko [1989] hinted at the universal approximation theorem for sigmoid activation functions. Funahashi [1989] and Hornik et al. [1989] showed that a MLP with one hidden layer and sufficient non-linear units could approximate any continuous function on a compact input domain to arbitrary precision. Therefore, MLP are recognised as universal approximators. Hornik [1991] showed that it is not the specific choice of the activation function, but rather the multilayer feedforward architecture itself which gives neural networks the potential of being universal approximators. The universal approximation theorem can be stated as follows:

Theorem 15.1 *The universal approximation theorem*

Let $\Phi(\cdot)$ be a nonconstant, bounded, and continuous function. Let I_m denotes the m -dimensional unit hypercube $[0, 1]^m$. The space of continuous functions on I_m is denoted by $C(I_m)$. Then, given any $\epsilon > 0$ and any function $f \in C(I_m)$, there exist an integer N , real constants $v_i, b_i \in \mathbb{R}$ and real vectors $w_i \in \mathbb{R}^m$, where $i = 1, \dots, N$, such that we may define:

$$F(x) = \sum_{i=1}^N v_i \Phi(w_i^\top x + b_i)$$

as an approximate realisation of the function f where f is independent of Φ . That is,

$$|F(x) - f(x)| < \epsilon$$

for all $x \in I_m$. In other words, functions of the form $F(x)$ are dense in $C(I_m)$.

Note, the theorem still holds when replacing I_m with any compact subset of $\mathbb{R}^m$. Kurkova [1992] took advantage of techniques developed by Kolmogorov to give a direct proof of the universal approximation capabilities of perceptron type networks with two hidden layers.

15.1.4 Gradient descent and the delta rule

15.1.4.1 The gradient descent rule As explained by Rumelhart et al. [1986] [1986b], one way forward to finding the optimal combination of weights in the multilayer perceptron is to consider the delta rule, which uses gradient descent to search the hypothesis space of possible weight vectors to find the weights best fitting the training examples. The gradient descent provides the basis for the backpropagation algorithm, which can learn networks with many interconnected units.

Given a vector of input $x \in \mathbb{R}^n$ together with a weight vector $w \in \mathbb{R}^n$, we consider the task of training an unthresholded perceptron, that is, a linear unit for which the output O is given by

$$O(x) = w \cdot x$$

A linear unit is the first stage of a perceptron without the threshold. We must specify a measure for the training error of a hypothesis (weight vector), relative to the set $\mathcal{D}$ of training examples. A common measure is to use

$$E(w) = \frac{1}{2} \sum_{p=1}^P (O_p - d_p)^2$$

where d_p is the target output for training example p and O_p is the output of the linear unit for training example p . One can show that under certain conditions, the hypothesis minimising E is also the most probable hypothesis in H given the training data. The entire hypothesis space of possible weight vectors and their associated E values produce an error surface. Given the way in which we defined E , for linear units, this error surface must always be parabolic with a single global minimum.

The gradient descent search determines a weight vector minimising E by starting with an arbitrary initial weight vector, and then repeatedly modifying it in small steps. At each step, the weight vector is altered in the direction that produces the steepest descent along the error surface, until a global minimum error is reached. This direction is found by computing the derivative of E with respect to each component of the vector w , called the gradient of E with respect to w , given by

$$\nabla E(w) = \left[\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right]$$

The gradient specifies the direction that produces the steepest increase in E , and its negative produces the steepest decrease. Hence, the training rule for gradient descent is

$$w \leftarrow w + \Delta w$$

where

$$\Delta w = -\eta \nabla E(w)$$

where $\eta > 0$ is the learning rate determining the step size in the gradient descent search. The negative sign implies that we move the weight vector in the direction that decreases E . Given the definition of $E(w)$ we can easily differentiate the vector of $\frac{\partial E}{\partial w_i}$ derivatives as

$$\frac{\partial E}{\partial w_i} = \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{p=1}^P (O_p - d_p)^2, i = 1, \dots, n$$

which gives after some calculation

$$\frac{\partial E}{\partial w_i} = \sum_{p=1}^P (O_p - d_p) x_{i,p}$$

where $x_{i,p}$ is the single input component x_i for training example p . The weight update rule for gradient descent is

$$\Delta w_i = -\eta \sum_{p=1}^P (O_p - d_p) x_{i,p} = \eta \sum_{p=1}^P (d_p - O_p) x_{i,p}$$

In the case where η is too large, the gradient descent search may overstep the minimum in the error surface rather than settling into it. One solution is to gradually reduce the value of η as the number of gradient descent steps grows. Gradient descent is a strategy for searching through a large or infinite hypothesis space which can be applied whenever

1. the hypothesis space contains continuously parameterised hypotheses.
2. the error can be differentiated with respect to these hypothesis parameters.

The key practical difficulties in applying gradient descent are

1. converging to a local minimum can sometimes be quite slow.
2. if there are multiple local minima in the error surface, then there is no guarantee that the procedure will find the global minimum.

15.1.4.2 The delta rule While the gradient descent training rule computes weight updates after summing over all the training examples in $\mathcal{D}$, the stochastic gradient descent (SGD) approximates this gradient descent search by updating weights incrementally, following the calculation of the error for each individual example. Hence, as we iterate through each training example, we update the weight according to

$$\begin{aligned} \Delta w_i &= \eta (d_p - O_p) \frac{\partial O_p}{\partial w_i} \\ &= \eta (d_p - O_p) x_i \end{aligned} \tag{15.82}$$

where the subscript i represents the i th element for the training example in question. One way of viewing this stochastic gradient descent is to consider a distinct error function $E_p(w)$ defined for each individual training example p as follow

$$E_p(w) = \frac{1}{2} (d_p - O_p)^2$$

where d_p and O_p are the target value and the unit output value for training example p . We therefore iterates over the training examples p in $\mathcal{D}$, at each iteration altering the weights according to the gradient with respect to $E_p(w)$. The sequence of these weight updates provides a reasonable approximation to descending the gradient with respect to the

original error function $E(w)$. This training rule is known as the Widrow-Hoff rule (see Widrow et al. [1960]), or the delta rule, or sometimes the least-mean-square (LMS) rule. Widrow et al. devised the ADALINE (Adaptive Linear Neuron or later Adaptive Linear Element), which is the name of a physical device that implemented a single layer artificial neural network. In the learning phase, the weights are adjusted according to the weighted sum of the inputs. The difference

$$(d_p - O_p)$$

in Equation (15.82) represents the scalar error between the prediction $O_p = w \cdot x$ and the target value d_p . It is multiplied by the observation x_i to determine the weight changes because x_i indicates how the changing weight will affect the error. Note, the delta rule converges only asymptotically toward the minimum error hypothesis, but it converges regardless of whether the training data are linearly separable.

15.1.5 Introducing multilayer networks

While a single perceptron can only express linear decision surfaces, multilayer networks are capable of expressing a rich variety of nonlinear decision surfaces. Since multiple layers of cascaded linear units can only produce linear functions, we need to introduce another unit to represent highly nonlinear functions. That is, we need a unit whose output is a nonlinear function of its inputs, but which is also a differentiable function of its input. We are now going to discuss such a unit and then describe how to apply the gradient descent in the case of multilayer networks.

15.1.5.1 Describing the problem We consider a feedforward network with N_i input units, N_o output units and N_k hidden layers which can exhibit any desired feedforward connection pattern. We are also given a training set $\{(\bar{x}_1, \bar{d}_1), \dots, (\bar{x}_P, \bar{d}_P)\}$ consisting of P ordered pairs of n - and m -dimensional vectors called the input and output patterns, respectively. Note, the training set may consist of sequential data, or time-varying input, which we then represent as $\{(\bar{x}_1, \bar{d}_1), \dots, (\bar{x}_T, \bar{d}_T)\}$ with T ordered pairs. We assume that the primitive functions at each node of the network is continuous and differentiable, and that the weights of the edges are real numbers selected randomly so that the output $\bar{O}_p$ of the network is initially different from the target $\bar{d}_p$. The idea being to minimise the distance between $\bar{O}_p$ and $\bar{d}_p$ for $p = 1, \dots, P$ by using a learning algorithm searching in a large hypothesis space defined by all possible weight values for all the units in the network. There are different measures of error, and for simplicity we let the error function of the network be given by

$$E = \frac{1}{2} \sum_{p=1}^P \|\bar{O}_p - \bar{d}_p\|_2^2$$

which is a sum of L_2 norms. After minimising the function with a training set, we can consider new unknown patterns and use the network to interpolate it. We use the backpropagation algorithm to find a local minimum to the error function by computing recursively the gradient of the error function and correcting the initial weights. Every one of the j output units of the network is connected to a node evaluating the function $\frac{1}{2}(O_{p,j} - d_{p,j})^2$ where $O_{p,j}$ and $d_{p,j}$ denote the j th component of the output vector $\bar{O}_p$ and the target vector $\bar{d}_p$, respectively. The outputs of the additional m nodes are collected at a node which adds them up and gives the sum E_p as its output. The same network extension is built for each pattern $\bar{d}_p$. We can then collect all the quadratic errors and output their sum, obtaining the total error for a given training set $E = \sum_{p=1}^P E_p$. Since E is computed exclusively through composition of the node functions, it is a continuous and differentiable function of the q weights $w_1, \dots, w_q$ in the network. Therefore, we can minimise E by using an iterative process of gradient descent where we need to calculate the gradient

$$\nabla E = \left(\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_q} \right)$$

and each weight is updated with the increment

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} \text{ for } i = 1, \dots, q$$

where η is a learning constant, that is, a proportionality parameter defining the step length of each iteration in the negative gradient direction. Hence, once we have a method for computing the gradient, we can adjust the weights iteratively until we find a minimum of the error function where $\nabla E \approx 0$.

Remark 15.1 *In the case of multilayer networks, the error surface can have multiple local minima. Hence, the minimisation by steepest descent will only produce a local minimum of the error function, and not necessarily the global minimum error.*

15.1.5.2 Describing the algorithm For simplicity of exposition we first describe the algorithm in the case where there is no hidden layer and the training set consists of a single input-output pair ($P = 1$). We assume a set of artificial neurons $\{f_{i,j}\}$, each receiving $\{x_i\}_{i=1}^{N_i}$ input and computing $\{O_j\}_{j=1}^{N_o}$ output, and we focus our attention on one of the weights, $w_{i,j}$, going from input i to neuron j in the network. We let W denote the $N_i \times N_o$ weight matrix with element $w_{i,j}$ at the i th row and the j th column and we let $w(j)$ be the $N_i \times 1$ row vector for the j th column. In the back-propagation algorithm, given the input vector $x = (x_1, \dots, x_{N_i})$, the activation function (also called net_j) for the j th neuron satisfies the weighted sum

$$A_j(x, w) = x \cdot w(j) = \sum_{i=1}^{N_i} x_i w_{i,j}, j = 1, \dots, N_o$$

which only depends on the inputs and the weights $w_{i,j}$ from input i to neuron j . We let the output function (or threshold box, or activation function) be a function g of the activation function, getting

$$O_j(x, w) = g(\tilde{A}_j(x, w)), j = 1, \dots, N_o$$

where $\tilde{A}_j(x, w) = A_j(x, w) + b_j$ and b_j is a bias value. In compact form, the output vector of all units is

$$O(x, w) = g(xW)$$

using the convention that we apply the function $g(\cdot)$ to each component of the argument vector. The simplest output function is the identity function. When using a threshold activation function, if the previous output of the neuron is greater than the threshold of the neuron, the output of the neuron will be one, and zero otherwise. Further, to simplify computation, the threshold can be equated to an extra weight. The error being the difference between the actual and the desired output, it only depends on the weights. Hence, to minimise the error by adjusting the weights, we define the error function for the output of each neuron. The error function for the j th neuron satisfies

$$E_j(x, w, d) = (O_j(x, w) - d_j)^2, j = 1, \dots, N_o$$

where d_j is the j th element the desired target vector $\bar{d}$. In that setting, the total error of the network is simply the sum of the errors of all the neurons in the output layer

$$E(x, w, d) = \frac{1}{2} \|\bar{O}(x, w) - \bar{d}(x)\|_2^2 = \frac{1}{2} \sum_{j=1}^{N_o} E_j(x, w, d)$$

To minimise the error, we will update the weights of the network so that the expected error in the next iteration is lower using the method of gradient descent. Each weight is updated by using the increment

$$\Delta w_{i,j} = w_{i,j}^{l+1} - w_{i,j}^l = -\eta \frac{\partial E}{\partial w_{i,j}^l}$$

where l is the iteration counter, and $\eta \in (0, 1]$ is a learning rate. The size of the adjustment depends on η and on the contribution of the weight to the error of the function. The adjustment will be largest for the weight contributing the most to the error. We repeat the process until the error is minimal.

15.1.5.3 Describing the nonlinear transformation We use the chain rule to compute the gradient of the error function. Since this method requires computation of the gradient of the error function at each iteration step, we must guarantee the continuity and differentiability of the error function. One way forward is to use the sigmoid, or logistic function, a real function $f_c : \mathbb{R} \rightarrow (0, 1)$ defined as

$$f_c(x) = \frac{1}{1 + e^{-cx}}$$

where the constant c can be chosen arbitrarily, and its reciprocal $\frac{1}{c}$ is called the temperature parameter. For $c \rightarrow \infty$ the sigmoid converges to a step function at the origin. Further, $\lim_{x \rightarrow \infty} f_c(x) = 1$, $\lim_{x \rightarrow -\infty} f_c(x) = 0$, and $\lim_{x \rightarrow 0} f_c(x) = \frac{1}{2}$. The first derivative of the sigmoid with respect to x is

$$\frac{d}{dx} f_c(x) = \frac{ce^{-cx}}{(1 + e^{-cx})^2} = cf_c(x)(1 - f_c(x)) \quad (15.83)$$

and the second derivative of the sigmoid with respect to x is

$$\frac{d^2}{dx^2} f_c(x) = cf'_c(x) - 2cf_c(x)f'_c(x)$$

where $f'_c(x) = \frac{d}{dx} f_c(x)$. To get a symmetric output function, we can consider the symmetrical sigmoid $f_s(x)$ defined as

$$f_s(x) = 2f_1(x) - 1 = \frac{1 - e^{-x}}{1 + e^{-x}}$$

which is the *hyperbolic tangent* for the argument $\frac{x}{2}$, written $\tanh(\frac{x}{2})$. An alternative solution is simply to linearly translate the domain of definition of the logistic function in the range $[b_L, b_H]$ where b_L is the lower bound and b_H the upper bound. Hence, the real function $f_{b,c} : \mathbb{R} \rightarrow (b_L, b_H)$ is defined as

$$f_{b,c}(x) = b_L + (b_H - b_L)f_c(x)$$

with $\lim_{x \rightarrow \infty} f_c(x) = b_H$, $\lim_{x \rightarrow -\infty} f_c(x) = b_L$, and $\lim_{x \rightarrow 0} f_c(x) = \frac{1}{2}(b_L + b_H)$. The derivative of this function with respect to x is

$$\frac{d}{dx} f_{b,c}(x) = (b_H - b_L) \frac{d}{dx} f_c(x) = (b_H - b_L)cf_c(x)(1 - f_c(x))$$

Hence, in order to get a symmetric output function we can set $b_L = -1.5$ and $b_H = 1.5$. Example of translated logistic function and its derivative are given in Figure (42) for $b_L = -0.6$ and $b_H = 0.6$ with $c = \frac{1}{p}$ for $p = 0.5, 1, 1.5$. Several other output functions can be used and have been proposed in the back-propagation algorithm. However, smoothed output function can lead to local minima in the error function which would not be there if the Heaviside function had been used. Further, the slope of the sigmoid function given in Equation (15.83) takes values in the range $[0, \frac{1}{4}c]$ where the maximum is reached at $x = 0$. Thus, the steepest slope of the sigmoid function occurs at the origin, and depending on the size of the constant c , the slope can become greater than 1, leading to gradient explosion. In addition, the derivative of the sigmoid function is asymptotically zero for large values of x , such that the gradient will decay much more when the net value is taking large of small values. The decaying or explosion of the gradient are both called the vanishing gradient problem.

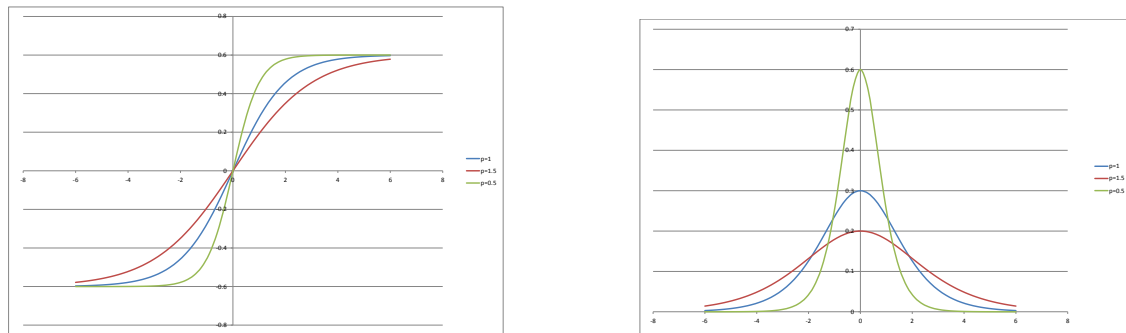


Figure 42: Translated logistic function and its derivative for $b_L = -0.6$ and $b_H = 0.6$.

15.1.5.4 A simple example To explain the method, we first consider a two-layers ANN model, where the most common output function used with a threshold is the sigmoidal function

$$O_j(x, w) = \frac{1}{1 + e^{-A_j(x, w)}}$$

with the limits $\lim_{A_j \rightarrow \infty} = 1$, $\lim_{A_j \rightarrow -\infty} = 0$, and $\lim_{A_j \rightarrow 0} = \frac{1}{2}$ allowing for a smooth transition between the low and high output of the neuron. We compute the gradient of the total error with respect to the weight $w_{i,j}$, noted $\nabla E(w_{i,j})$, where it is understood that the subscripts i and j are fixed integers within $i = 1, \dots, N_i$ and $j = 1, \dots, N_o$. From the linearity of the total error function, the gradient is given by

$$\frac{\partial E}{\partial w_{i,j}} = \frac{1}{2} \sum_{j=1}^{N_o} \frac{\partial E_j}{\partial w_{i,j}}, i = 1, \dots, N_i$$

where the subscripts i and j are fixed integers in the partial derivatives. We first differentiate the total error with respect to the output function

$$\frac{\partial E}{\partial O_j} = (O_j - d_j)$$

and then differentiate the output function with respect to the weights

$$\frac{\partial O_j}{\partial w_{i,j}} = \frac{\partial O_j}{\partial A_j} \frac{\partial A_j}{\partial w_{i,j}} = O_j(1 - O_j)x_i$$

since $\frac{\partial A_j}{\partial w_{i,j}} = x_i$. Putting terms together, the adjustment to each weight becomes

$$\Delta w_{i,j} = -\eta(O_j - d_j)O_j(1 - O_j)x_i = \eta(d_j - O_j)O_j(1 - O_j)x_i$$

We can use the above equation to train an ANN with two layers. Given a training set with p input-output pairs, the error function can be computed by creating p similar networks and adding the outputs of all of them to obtain the total error of the set.

15.1.6 Multi-layer back propagation

In the case of a multilayer network, again we consider a single input-output pair and assume a set of artificial neurons $\{f_{i,j,k}\}_{k=0}^K$ where the multilayer subscript $k = 0$ corresponds to the set of inputs $\{x_i\}_{i=1}^{N_i}$, and the remaining subscript

k corresponds to the set of outputs $\{y_i\}_{i=1}^{N_{o,k}}$, where $N_{o,k}$ is the number of output in the k -th layer. In that setting, we define the output function of the j th node for the k -th layer as

$$O_{j,k}(x, w) = g(\tilde{A}_{j,k-1}(x, w)), j = 1, \dots, N_{o,k}$$

where $\tilde{A}_{j,k-1}(x, w) = A_{j,k-1}(x, w) + b_{j,k}$ with the bias $b_{j,k}$, and we let the corresponding activation function satisfies

$$A_{j,k-1}(x, w) = \sum_{i=1}^{N_{o,k-1}} O_{i,k-1}(x, w) w_{i,j}^{k-1} \text{ and } O_{j,0}(x, w) = A_{j,-1}(x, w) = \sum_{i=1}^{N_i} x_i$$

where $w_{i,j}^{k-1}$ is the weight going from the i th node in layer $k-1$ to the j th node in layer k . Note, since $\frac{d\tilde{A}_{j,k-1}(x, w)}{dA_{j,k-1}(x, w)} = 1$, we get $\frac{\partial O_{j,k}(x, w)}{\partial A_{j,k-1}(x, w)} = \frac{\partial O_{j,k}(x, w)}{\partial \tilde{A}_{j,k-1}(x, w)}$ where $\frac{\partial O_{j,k}(x, w)}{\partial \tilde{A}_{j,k-1}(x, w)} = \frac{\partial}{\partial \tilde{A}_{j,k-1}} g(\tilde{A}_{j,k-1}(x, w))$.

When running backpropagation, the error signal travels from the output layer to the input layer. Given a hidden layer with subscript k , and focusing on neuron with index h , we define the pre-error signal and error signal as

$$\begin{aligned} e_{h,k} &= \frac{\partial}{\partial O_{h,k}} E \\ \delta_{h,k} &= \frac{\partial}{\partial \tilde{A}_{h,k-1}(x, w)} E = \frac{\partial}{\partial net_{h,k}} E \end{aligned}$$

Using these results, we are going to summarise the feedforward neural network by considering the multilayer perceptron (MLP).

15.1.6.1 Forward pass We consider an MLP consisting of K hidden layers with N_i input units, $N_{o,k}$ output units in the k -th hidden layer and $N_{o,K}$ output units in the output layer. In each unit of the hidden layer, or in the output layer, we first calculate the weighted sum of the incoming values, which is referred to as the *net value* of the input unit. The *activation function*, denoted by f , is then applied to the net value. The value of the activation function, y , is the output of the unit. The activation function is required to be bounded, differentiable and monotonous. The two most common functions used in machine learning are the *hyperbolic tangent* and the *logistic sigmoid* functions, which are both nonlinear. This important feature makes it possible for the network to model nonlinear equations. As the number of hidden layers increases, the network can approximate more complex nonlinear functions. Methods using a network with a large number of hidden layers are referred to as *deep learning* network. Since the input domain of the activation function is infinite, while the output domain is finite, the activation functions are also termed as *squashing functions*. Letting h be the index of the first hidden unit, the net value and output function satisfy

$$\begin{aligned} net_{h,1} &= \sum_{i=1}^{N_i} w_{ih} x_i \\ y_h &= f(net_{h,1}) \end{aligned} \quad (15.84)$$

Considering two adjacent layers, denoted by $k-1$ and k , with index h and h' , respectively, the summation and the activation process are similar to that of Equation (15.84), given by

$$\begin{aligned} net_{h',k} &= \sum_{h=1}^{N_{o,k-1}} w_{hh'} y_h \\ y_{h'} &= f(net_{h',k}) \end{aligned} \quad (15.85)$$

The net value and activation function of the output layer are calculated in the same way as those in the hidden layer. We let s be the index ranging over the output layer, and $N_{o,K-1}$ is the number of neurons in the $(K-1)$ -th hidden layer closest to the output layer. We get

$$\begin{aligned} net_{s,K} &= \sum_{h=1}^{N_{o,K-1}} w_{hs} y_h \\ y_s &= f(net_{s,K}) \end{aligned} \quad (15.86)$$

15.1.6.2 Backward pass In general, after a forward pass the network output is not the target input, and we need to measure the distance between the actual output and the target output to serve as a measurement of the network performance. This distance is called the *error function*. Given the input-target pair (x, z) , and the network output y , then the error function is given by

$$E(x, z) = \frac{1}{2} \sum_{s:output} (y_s - z_s)^2, \quad (15.87)$$

where $y = (y_1, \dots, y_s, \dots, y_K)$, $z = (z_1, \dots, z_s, \dots, z_K)$, and $N_{o,K}$ is the number of units in the output layer. As long as the error function is larger than a given tolerance level, we need to modify the weights of the network to further decrease the measure E . The algorithm designed for stepwise updating the weights that minimise the error function is called the *training algorithm*, or *learning algorithm* of the network. It is natural to relate the weights updates to the *gradient*, which is a vector of derivatives of the error function with respect to all the weights. It costs the network certain number of steps to reach a tolerably small error. Putting all the weights in a vector, the weight vector at step n is denoted by w^n , which is then updated by an amount Δw^n . We denote the gradient as $\nabla E = \frac{\partial E}{\partial w^n}$ and apply the *gradient descent* repeatedly using the chain rule. By working backward, from computing the derivative with respect to the output layer to computing the derivatives with respect to all the internal weights, the error is back propagated in the network. This procedure is called the standard *back propagation*, discovered independently by different researchers (see Werbos [1974], Parker [1985], Rumelhart et al. [1986b]). In standard back propagation, the weight update of the n -th step is

$$\Delta w^n = -\eta \frac{\partial E}{\partial w^n}, \quad (15.88)$$

where η is the *learning rate*, a real number between 0 and 1. Following the previous notations, the detailed procedure to calculate the gradient $\frac{\partial E}{\partial w_{ij}}$ is as follows:

1. Calculate the derivatives of the error function with respect to the output units:

$$\frac{\partial E}{\partial y_s} = y_s - z_s \quad (15.89)$$

$$\frac{\partial E}{\partial net_{s,K}} = \frac{\partial E}{\partial y_s} \frac{\partial y_s}{\partial net_{s,K}} \quad (15.90)$$

2. Calculate the derivatives of the error function with respect to the hidden units. Work backward through hidden layers, using the chain rule. To do so, we introduce the *error signal*:

$$\delta_j := \frac{\partial E}{\partial net_j}, \quad (15.91)$$

where j is the index of an arbitrary unit in the network.

- (a) Calculate the error signals of the last hidden layer:

Let j be the index of a unit in the last hidden layer which is the closest to the output layer. Since the error E depends on hidden unit j only through its connection to the output units, we have

$$\begin{aligned}\delta_j &= \frac{\partial E}{\partial y_j} \frac{\partial y_j}{\partial \text{net}_{j,K-1}} \\ &= \frac{\partial y_j}{\partial \text{net}_{j,K-1}} \sum_{s=1}^{N_{o,K}} \frac{\partial E}{\partial \text{net}_{s,K}} \frac{\partial \text{net}_{s,K}}{\partial y_j}\end{aligned}\quad (15.92)$$

From Equation (15.85) and (15.86), and given the definition of the error signal in Equation (15.91), we have,

$$\delta_j = f'(\text{net}_{j,K-1}) \sum_{s=1}^{N_{o,K}} \delta_s w_{js} \quad (15.93)$$

- (b) Calculate the error signals for hidden layers before the last hidden layer:

The error signals for the hidden units in hidden layer k ($k < K$) are based on the calculation of the error signal of the $(k+1)$ -th layer due to the chain rule. That is to say, except for the units of the last hidden layer, the error signal of each unit in other hidden unit can be computed recursively as

$$\delta_i = f'(\text{net}_{i,k}) \sum_{j=1}^{N_{o,k+1}} \delta_j w_{ij} \quad (15.94)$$

where i and j are indicators of unit in the hidden layers k and $k+1$, respectively, before the last hidden layer.

Having computed all the error signals for all the hidden units as well as the one for the output units, the derivatives of error function with respect to the weights w_{ij} can be written as:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial \text{net}_{j,k+1}} \frac{\partial \text{net}_{j,k+1}}{\partial w_{ij}} = \delta_j y_i \quad (15.95)$$

Then, the change of weight from unit i to unit j is given by

$$\Delta w_{ij} = -\alpha \delta_j y_i \quad (15.96)$$

Thus, introducing the concept of error signal makes the back propagation process more comprehensible, as only the error signal propagates backward through the network. The changing amount of the weight is proportional to the product of the error signal of its destination unit and the output of its source unit. To summarise, the forward pass calculates the network output, while the backward pass updates the weights to minimise the error. The method used in backward pass is called back propagation, and a forward pass together with a backward pass is regarded as one *loop*. The training process continue until some *stopping criteria* is met, such as the error function is small enough, or, it stops to decrease after a certain number of loops.

16 Temporal difference learning

16.1 Overview

In a prediction-learning problem¹⁵, a learning agent performs a sequence of actions, obtaining some rewards or penalties (punishments). The temporal credit assignment problem is to assign credit or blame to the situations and actions

¹⁵Using past experience with an incompletely known system to predict its future behaviour.

involved. However, it is generally difficult to perform when the reinforcement delay is long and the environment is stochastic. The most important learning paradigm has been supervised learning paradigm (SLP) (see Section (??)), where the learner needs to associate pairs of items. When presented with just the first item, the learner recalls the second one. Any prediction problem can be cast in a SLP by taking the first item to be the input of a prediction function, and the second item to be the actual outcome. That is, $y = f(x)$ for the pair (x, y) . However, Sutton [1988] argued that these methods are inadequate and that temporal difference (TD) are far preferable.

Nowadays, the most popular procedure for temporal credit assignment is temporal difference (TD). While conventional prediction-learning methods, such as supervised learning methods (see Section (15.1)), are driven by the error between predicted and actual outcomes, TD methods are driven by the error between temporally successive predictions. Several TD algorithms have been proposed (see Samuel [1959], Hampson [1983], Barto et al. [1983], Holland [1986]). Sutton [1988] proved their convergence and optimality for special cases and related them to supervised learning methods.

We can distinguish between two kinds of prediction-learning problem:

1. Single-step: all information about the correctness of each prediction is revealed at once. It is a single point forecast. For example, predict next day stock prices based on today's prices.
2. Multi-step: correctness is not revealed until more than one step after the prediction is made. Only partial information relevant to its correctness is revealed at each step. For example, predict X -days ahead weather forecast.

In type [1], the data comes in observation-outcome pairs, which is ideally suited for pairwise supervised learning approach. TD methods are more concerned with type [2] prediction problems since type [1] is a special case. Nonetheless, many single-step prediction problems are more naturally viewed as multi-step problems. This is because patterns are not unrelated, but are part of a series of related patterns, all providing information about the same classification. Sutton [1988] showed that TD methods were more efficient than supervised learning methods when the data sequences were generated by a dynamical system.

TD methods were originally used to predict the probability of winning a game for each possible board situation. As an example, we consider the learning problem of predicting the winning probability of a tic-tac-toe by collecting all the training instances. Given the training instance in Figure (43), suppose that the estimated winning probabilities from states A , B and C are 0.8, 0.3, and 0.6, respectively. The player started at state A moved to B and then to C , and won the game.

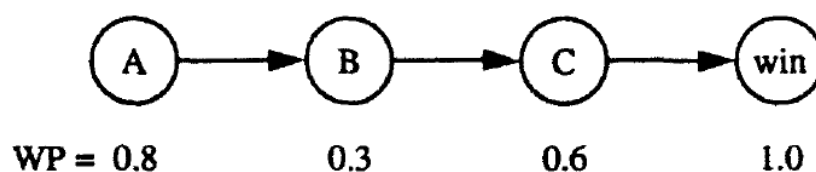


Figure 43: A sequence of state transitions with estimated probabilities in a game play.

In that setting, the delta rule (see Section (15.1.4)) would increase the WP (see Equation (15.82)) from states A , B , and C by $(1 - 0.8)\alpha$, $(1 - 0.3)\alpha$, and $(1 - 0.6)\alpha$, where α is a learning rate. Here, the learning changes are proportional to the differences between the current predictions and the actual outcome (in this case 1). Storing the WPs in a lookup table, and sufficient training data, the delta rule would eventually converge to the true WP from every state.

On the other hand, $TD(0)$ minimises the error (TD error) between successive predictions. Let Δ_s be the learning change to the WP from a state s . $TD(0)$ makes the following learning changes:

$$\Delta_A = (0.3 - 0.8)\alpha, \Delta_B = (0.6 - 0.3)\alpha, \Delta_C = (1.0 - 0.6)\alpha$$

Sutton [1988] made three assumptions to prove convergence of $TD(0)$ to the optimal estimates:

1. sufficient training data
2. repeated presentations of training data
3. the target system is linear

The author also showed that on multi-step prediction problems outperformed supervised learning. The family of TD methods is denoted $TD(\lambda)$, where $0 \leq \lambda \leq 1$ is called the recency factor. Then, $TD(0)$ is the difference between two successive predictions, and $TD(\lambda)$ is the sum of $TD(0)$ errors exponentially weighted by λ . In our previous example, we get the following learning

$$\begin{aligned}\Delta_A &= [(0.3 - 0.8) + (0.6 - 0.3)\lambda + (1.0 - 0.6)\lambda^2]\alpha \\ \Delta_b &= [(0.6 - 0.3) + (1.0 - 0.6)\lambda]\alpha \\ \Delta_C &= (1.0 - 0.6)\alpha\end{aligned}$$

Dayan [?] proved convergence of $TD(\lambda)$. Note, $TD(1)$ makes the same learning changes as the supervised learning method. As λ approaches 1, $TD(\lambda)$ behaves more like supervised learning. Further, $TD(\lambda)$ not only makes prediction changes based on information from the immediately successive state, but also information from all the future states. Thus, $TD(\lambda)$ results in faster learning than $TD(0)$ at the early learning stage.

16.2 The TD family of learning procedures

The data sequences are generated by absorbing Markov processes, where the next state depends only on the current state. They are defined by a set of terminal states T , a set of nonterminal states N , and a set of transition probabilities p_{ij} , $i \in N, j \in N \cup T$, where each p_{ij} is the probability of a transition from state i to state j , given that the process is in state i . Since it is absorbing, all sequences eventually terminate. Given an initial state q_1 , the absorbing Markov process generates the sequence $q_1, q_2, \dots, q_{m+1}$, where $q_{m+1} \in T$. The learning algorithms do not know about the state sequence, but only of a related observation-outcome sequence $x_1, x_2, \dots, x_m, z$.

16.2.1 The $TD(1)$ procedure

Consider a multi-step problem with experience coming in observation-outcome sequences of the form $x_1, x_2, \dots, x_m, z$, where each x_t is a vector of observable input at time t in the sequence, and $z \in \mathbb{R}$ is the outcome of that sequence. Note, z only depends on the terminal state q_{m+1} , that is, z is $\mathcal{F}_T$ -measurable. A training set consists of several sequences $\{x_{1,p}, x_{2,p}, \dots, x_{m,p}, z_p\}$, for $p \in [1, P]$. For each such sequence, the learner produces a sequence of predictions $P_1, P_2, \dots, P_m$, each of which being an estimate of z . For simplicity, it is assumed that P_t is a function of x_t and a vector of weights w , that is, $P_t = P(x_t, w)$.

Remark 16.1 While $P_t = P(x_t, w)$ is a function of x_t and w , possibly a perceptron or a two-layers network, it is not a multi-layer network.

All learning procedures can be expressed as rules for updating w . We want to find rules for updating the vector w . It is assumed that w does not change during a sequence. For each observation, an increment to w , denoted Δw_t is computed. After a complete sequence has been run, w is modified as follows:

$$w \leftarrow w + \sum_{t=1}^m \Delta w_t$$

In supervised learning (see Section (15.1)), each sequence of observations and its outcome is represented by a sequence of observation-outcome pairs $(x_1, z), (x_2, z), \dots, (x_m, z)$. In the delta rule (see Section (15.1.4)), the increment due to time t depends on the error between P_t and z and the gradient of P_t with respect to w given by

$$\Delta w_t = \alpha(z - P_t) \nabla_w P_t \quad (16.97)$$

where α is the learning rate. Note, this is Equation (15.82) where $(z - P_t)$ represents the scalar error between the prediction P_t and the target value z . In the special case where P is a linear function of x and w , $P_t = w^\top x_t$, then we recover the Widrow-Hoff rule

$$\Delta w_t = \alpha(z - w^\top x_t) x_t$$

In the case of backpropagation (see Section (15.1.3)), P_t is computed by a multi-layer network and is a nonlinear function of x_t and w . Here, Equation (16.97) still applies, but a complex process is used to compute the gradient $\nabla_w P_t$. Note, all Δw_t depend critically on z , and can only be determined at the end of the sequence, once z is known. Thus, all observations and predictions made during the sequence must be remembered until its end, when all the Δw_t are computed. Thus, Equation (16.97) can not be computed incrementally. On the other hand, there is a TD method, called $TD(1)$, producing the same result, but which can be computed incrementally. To do so, we represent the error $z - P_t$ as a change in prediction:

$$z - P_t = \sum_{k=t}^m (P_{k+1} - P_k) \text{ where } P_{m+1} = z$$

We can therefore combine the above equations as

$$\begin{aligned} w \leftarrow w + \sum_{t=1}^m \alpha(z - P_t) \nabla_w P_t &= w + \sum_{t=1}^m \alpha \sum_{k=t}^m (P_{k+1} - P_k) \nabla_w P_t \\ &= w + \sum_{k=1}^m \alpha \sum_{t=1}^k (P_{k+1} - P_k) \nabla_w P_t \\ &= w + \sum_{t=1}^m \alpha (P_{t+1} - P_t) \sum_{k=1}^t \nabla_w P_k \end{aligned}$$

Thus, the rule to use becomes

$$\Delta w_t = \alpha(P_{t+1} - P_t) \sum_{k=1}^t \nabla_w P_k \quad (16.98)$$

which can be computed incrementally because each Δw_t only depends the difference between P_{t+1} and P_t , and the sum of past gradients. Hence, we only need to compute one increment to w on each time step.

16.2.2 The $TD(\lambda)$ procedure

So far, the predictions in Equation (16.98) are altered to an equal extent. The $TD(\lambda)$ procedure permits to make greater alterations to more recent predictions. It is an exponential weighting with recency where the alterations to the predictions of observation vectors occurring k steps in the past are weighted by λ^k for $0 \leq \lambda \leq 1$. It is given by

$$\Delta w_t = \alpha(P_{t+1} - P_t) \sum_{k=1}^t \lambda^{t-k} \nabla_w P_k \quad (16.99)$$

In the case $\lambda = 1$ we recover Equation (16.98), hence the name $TD(1)$. One advantage of exponential weighting is that it can be computed incrementally. We denote $e_t = \sum_{k=1}^t \lambda^{t-k} \nabla_w P_k$ and write

$$\begin{aligned} e_{t+1} &= \sum_{k=1}^{t+1} \lambda^{t+1-k} \nabla_w P_k \\ &= \nabla_w P_{t+1} + \sum_{k=1}^t \lambda^{t+1-k} \nabla_w P_k \\ &= \nabla_w P_{t+1} + \lambda e_t \end{aligned}$$

In the special case where $\lambda = 0$, the weight increment is only determined by its effect on the prediction associated with the most recent observation:

$$\Delta w_t = \alpha (P_{t+1} - P_t) \nabla_w P_t$$

This is Equation (16.97) with z replaced by P_{t+1} . Both procedures use the same learning mechanism, but with different errors.

Sutton [1988] proved convergence of the linear $TD(0)$ procedure. He devised an asymptotic convergence theorem for $TD(0)$ when presented with new data sequences (generated by absorbing Markov processes), and a theorem that $TD(0)$ converges under repeated presentations of optimal estimates.

16.2.3 A random-walk example

As discussed in Appendix (16.1), Sutton [1988] showed that TD methods were more efficient than supervised learning methods when the data sequences were generated by a dynamical system. As an example, the author considered a bounded random walk, that is, a random walk taking steps to the right or to the left with equal probability, until a boundary is reached. This is illustrated in Figure (44), where the walk starts in state D and ends if either edge state A or G is reached. A typical walk (path) might be $DCDEFG$.

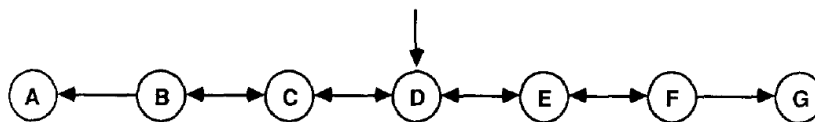


Figure 44: A bounded random walk

We want to find the conditional probability of a walk reaching state G , given that it is in each of the other states. The outcome is $z = 0$ for a walk ending in state A and $z = 1$ if it ends in state G . For each nonterminal state i , we associate an observation vector X_i . If the walk is in state i at time t , then $x_t = X_i$. For example, $X_D = (0, 0, 1, 0, 0)^\top$. The sequence of the above typical walk becomes $X_D, X_C, X_D, X_E, X_F, 1$. Thus, at time t and state i , for a linear prediction $P_t = w \cdot x_t$, we simply get the value of the i th component of w , denoted w_i .

The author tested linear supervised learning against TD methods. To get statistically reliable results, 100 training sets, each consisting of 10 sequences, were constructed for use by all learning procedures. The $TD(\lambda)$ procedure in Equation (16.99) was used with $\lambda = 0, 0.1, 0.3, 0.5, 0.7, 0.9, 1$. The true conditional probabilities for right-side termination are $\frac{1}{6}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}$ and $\frac{5}{6}$ for states B, C, D, E and F , respectively. Two experiments were set:

1. Δw are accumulated over sequences and only used to update the weight vector after the complete presentation of a training set. Each training set are presented repeatedly to each learning procedure until the latter no-longer produce any significant changes in the weight vector. This is the repeat presentations training paradigm.

- Each training set is presented only once to each procedure rather than repeatedly until convergence. Weight updates are performed after each sequence, rather than after each complete training set. The learning rate parameter α is varied within a range of values. Weight vector is initialised to $\frac{1}{2}$.

The root mean squared (RMS) error was used as a measure of performances. In set [1], averaging over the training sets, performance improved as λ was decreased from 1 to 0. This is illustrated in Figure (45).

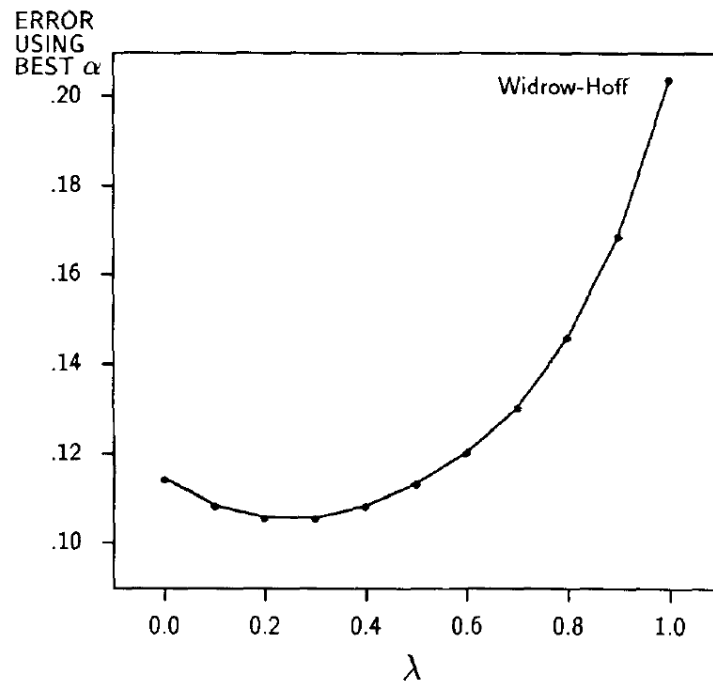


Figure 45: Average error for random walk in case [1]

The Widrow-Hoff procedure, $\lambda = 1$, only minimises error on the training set, but it does not minimise error for future experience. In fact, Sutton proved that $TD(0)$ converges to the optimal estimates for matching future experience, those consistent with the maximum-likelihood estimate of the underlying Markov process. In set [2], WH produced the worst estimates. This is illustrated in Figure (46).

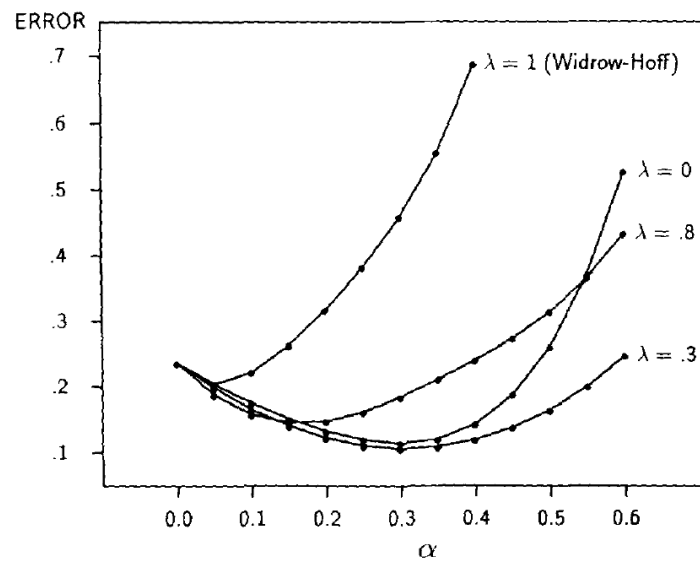


Figure 46: Average error for random walk in case [2]

The author also displayed the best error level obtained for each λ value, and the result was not $\lambda = 0$, but it was near 0.3. This is illustrated in Figure (47).

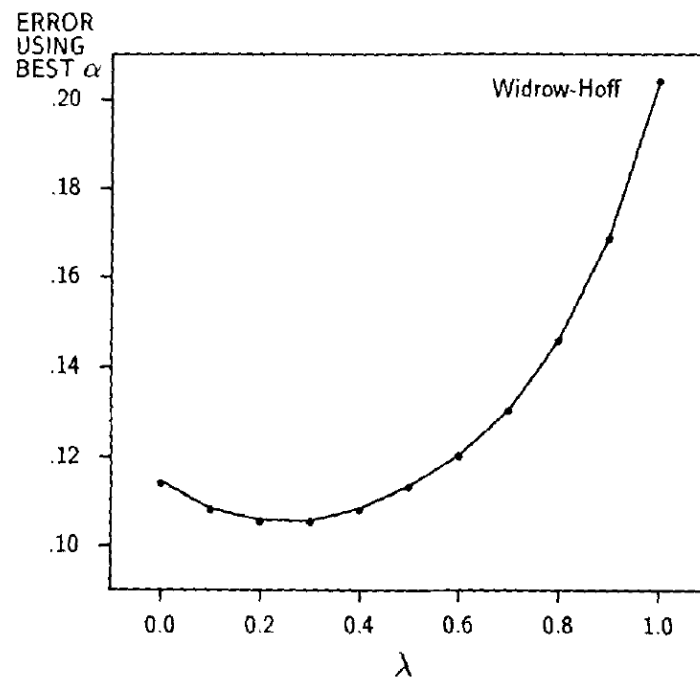


Figure 47: Average error for random walk in case [3]

This is because $TD(0)$ is slow at propagating prediction levels back along a sequence. There is no problem if the

sequence is repeatedly presented (case [1]). One solution is to work backwards through the sequences. In the sequence $X_D, X_E, X_F, 1$, first update the prediction for F in light of the 1, then the prediction for E is updated based on F , and so on. Thus, the effect of 1 is back propagated to the start of the sequence with only a single presentation. However, it no-longer has an incremental implementation. But if learning is done offline from an existing database, working backward should produce the best predictions.

16.2.4 TD as a gradient descent

Sutton [1988] also explained the fact that TD methods could be viewed as gradient descent in the space of the modifiable parameters (weights). Their goal is to minimise an overall error measure $J(w)$ over the space of weights by repeatedly incrementing the weight vector in an approximation to the direction in which $J(w)$ decreases the most steeply. We let $\hat{\nabla}_w J(w)$ be the approximated gradient, then the rule is

$$\Delta w_t = -\alpha \hat{\nabla}_w J(w)$$

In a multi-step prediction problem where $P_t = P(x_t, w)$ approximates $E[z | x_t]$, a natural error measure is the expected value of the square of the difference between these two quantities. That is,

$$J(w) = E_X[(E[z | X] - P(X, w))^2]$$

where X represents observation vectors. Here, $J(w)$ measures the error for a weight vector averaged over all observation vectors. However, at each time step, we get information about a single observation vector. Thus, the author defined a pre-observation error measure $Q(w, X)$ with the property that

$$E_X[Q(w, X)] = J(w)$$

where

$$Q(w, X) = (E[z | X] - P(X, w))^2$$

At each time step, the weight increments are then determined by using $\nabla_w Q(w, x_t)$, relying on

$$E_X[\nabla_w Q(w, X)] = \nabla_w J(w)$$

so that the overall effect of the equation for Δw_t can be approximated over many steps using small α as follows:

$$\begin{aligned} \Delta w_t &= -\alpha \nabla_w Q(w, x_t) \\ &= 2\alpha (E[z | x_t] - P(x_t, w)) \nabla_w P(x_t, w) \end{aligned} \tag{16.100}$$

Note, the conditional expectation $E[z | x_t]$ is not directly known and must be estimated:

- if $E[z | x_t]$ is approximated by z (the outcome), then we get the classical supervised learning procedure in Equation (16.97).
- if $E[z | x_t]$ is approximated by $P(x_{t+1}, w)$ (the following prediction), then we get the extreme TD method, $TD(0)$.

Remark 16.2 The key point is that the real goal is for each prediction to match the expected value of the subsequent outcome, not the actual outcome occurring in the training set. That is, the actual outcome of a sequence is often not the best estimate of its expected value.

16.2.5 Intra-sequence weight update

We assumed in Section (16.2.1) that the vector of weights w did not change during a sequence, but only after a complete sequence was run. However, since each observation of a sequence generates an increment to the weight vector, Sutton [1988] discussed how to update it immediately after each observation. In order for the update rule that only changes in prediction due to x (and not w) are effective in causing weight alterations, the author proposed the following update:

$$w_{t+1} = w_t + \alpha(P(x_{t+1}, w_t) - P(x_t, w_t)) \sum_{k=1}^t \lambda^{t-k} \nabla_w P(x_k, w_t) \quad (16.101)$$

This approach was used by Samuel [1959] in the checker player and Sutton [1988] in the AHC-learning (Adaptive Heuristic Critic).

16.2.6 Predicting cumulative outcomes

Rather than predicting the final outcome z of a sequence, we can also predict cumulative outcomes. We assume that each step of a sequence incurs a cost, and want to predict, at time t , the expected total cost of the remaining sequence. Here, the costs become known as the sequence evolves. We use the observation vector received at each step to predict the total cumulative cost after that step. We want P_t to predict the remaining cumulative cost given the t -th observation. We let c_{t+1} be the cost incurred between times t and $t+1$ and $\bar{c}_{ij}$ be the expected value of the cost incurred on transition from state i to state j .

16.2.6.1 Finite predictions We assume that the costs are elapsed time. We want P_t to equal the expected value of $z_t = \sum_{k=t}^m c_{k+1}$, where m is the number of observation vectors in the sequence. The prediction error (as defined in Equation (15.82)) can be expressed in terms of temporal differences as

$$z_t - P_t = \sum_{k=t}^m c_{k+1} - P_t = \sum_{k=t}^m \sigma_k$$

where

$$\sigma_k = c_{k+1} + P_{k+1} - P_k$$

and $P_{m+1} = 0$. Following the steps to derive the $TD(\lambda)$ family of procedures in Appendix (16.2.2), the learning rule for the cumulative $TD(\lambda)$ family is given by

$$\Delta w_t = \alpha \sigma_t \sum_{k=1}^t \lambda^{t-k} \nabla_w P_k$$

The ideal prediction for each state $i \in N$ is the expected value of the cumulative sum of the costs $E[z_t | x_t = X_i]$. It is computed, as in the $TD(\lambda)$ family, with $\bar{z}$ replaced by $\bar{c}$, so that convergence holds.

16.2.6.2 Infinite discounted predictions So far we have considered prediction problems with definite outcomes. In the case of infinite-horizon prediction problems we need to include some discounting. If a process generates costs c_{t+1} at each transition from t to $t+1$, we want P_t to predict the discounted sum

$$z_t = \sum_{k=0}^{\infty} \gamma^k c_{t+k+1}$$

with discounted rate parameter γ , $0 \leq \gamma < 1$. We want to find P_t to be equal to z_t and derive a recursive equation describing the relationship between temporally successive predictions. So, if the predictions are accurate, we get

$$\begin{aligned}P_t &= \sum_{k=0}^{\infty} \gamma^k c_{t+k+1} \\&= c_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k c_{t+k+2} \\&= c_{t+1} + \gamma P_{t+1}\end{aligned}$$

The TD error is the difference between the two sides of the equation,

$$\sigma_t = (c_{t+1} + \gamma P_{t+1}) - P_t$$

In his Adaptive Heuristic Critic, Sutton [1984] used this error in a learning rule identical to $TD(\lambda)$ given by

$$\Delta w_t = \alpha \sigma_t \sum_{k=1}^t \lambda^{t-k} \nabla_w P_k \quad (16.102)$$

where P_t is the linear form $w \cdot x_t$ and $\nabla_w P_t = x_t$. Hence, the AHC-model is best understood as using the linear TD method for predicting discounted cumulative outcomes.

16.3 Combining supervised learning and TD procedures

Both backpropagation (see Section (15.1.3)) and TD methods are designed to accurately assign credit.

- Backpropagation: it decides which part of a network to change in order to improve convergence to the output by reducing the overall error of a loss function. It addresses a structural credit-assignment problem.
- TD methods: they decide how each output of a temporal sequence of outputs should be changed. They address a temporal credit-assignment problem.

Both methods are perfectly compatible and can easily be combined. While Sutton [1988] discussed the linear case for TD methods, they are applicable for predictions $P(x_t, w)$ formed by nonlinear functions. The key requirement being that the gradient $\nabla_w P_t$ is computable, which is the case with backpropagation. Anderson [1986] [1987] implemented such a combination of backpropagation and TD method for solving a nonlinear broomstick-balancing task and the Towers of Hanoi problem.

16.4 Recurrent neural networks

16.4.1 Presentation

16.4.1.1 An overview *Recurrent Neural Networks* (RNNs) are extension of *Feedforward Neural Networks* (FNNs) discussed in Section (15.1.6), where we allow connections among hidden units associated with time delay. Thus, RNNs can retain information about the past, and, as such, discover temporal correlations between events occurring far in the past. While the feedforward neural networks have no cycles for connections, a recurrent neural network has feedback connection, meaning that the nodes of the network have cyclical connections between them (see Figure 48a). The existence of cycles allows RNNs to develop a self-sustained temporal activation dynamics along its recurrent connection pathways, even in the absence of input, making them a dynamical system.

As shown in Figure 48b, RNNs can be visualised by unfolding the RNNs along the whole input sequence. That is, in the case of time series (notion of time), the RNN are unfolded through time (see Figure (49)). The unfolded graph has no cycles, which is the same as FNNs, so that the forward pass and backward pass of MLP can be applied (see Section (15.1.3)). Recall from Section (15.1.3) that the MLP is an approximator for nonlinear functions, and since RNN can

be unfolded to a deep feedforward network, it has the advantage of the MLP, making it a better approximator. There are two main classes of RNNs, (1) the first one being characterised by an energy-minimising stochastic dynamics and symmetric connections (see Taylor et al. [2007]), and (2) the second featuring a deterministic update dynamics and directed connections. We are going to concentrate on the latter.

The biggest feature of recurrent neural network is that it has a sparsely connected hidden layer called reservoir, which enables RNNs to have short term memory that captures information about what has been calculated so far. Usually, this recurrent part of the structure is essential in learning complex patterns. In fact, RNNs have proved to be an attractive form for modelling non-linearity due to their ability to approximate any dynamical system with arbitrary precision (see Siegelmann et al. [1991]). As discussed in Section (15.1.3), Funahashi et al. [1993] showed that under mild and general assumptions, RNNs are universal approximations of dynamical systems. Indeed, as an equivalent theory of universal approximation for MLPs, it is said that a single hidden layer RNN with sufficient hidden units can approximate any sequence to arbitrary accuracy (see Hammer [2000]). Thus, the RNN is a natural generalisation of feedforward neural networks to sequences (see). Some applications include motion capture data (see Sutskever [2008]), text (see Sutskever et al. [2011]), music (see Boulanger-Lewandowski et al. [2012]), among others.

For simplicity of exposition, we are going to focus on a simple RNN with one self-connected hidden layer, as shown in Figure 48a. As trivial the difference in topology between FNNs and RNNs may seem, the advantage of the RNNs over the FNNs is profound. The latter can only map a limited number of inputs to a limited number of outputs, while a simple RNN can map the entire time series to some outputs. This is significant for time series prediction, as the long history can be fed to the network, and the existence of the recursive connection allows the network to memorise information of previous time steps, which is useful considering that financial time series are mostly serially correlated.

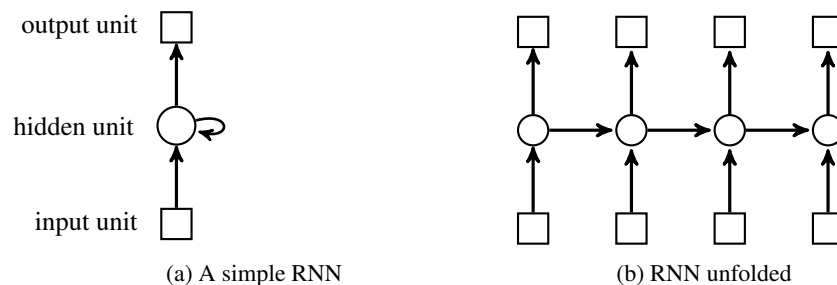


Figure 48: Recurrent Neural Network

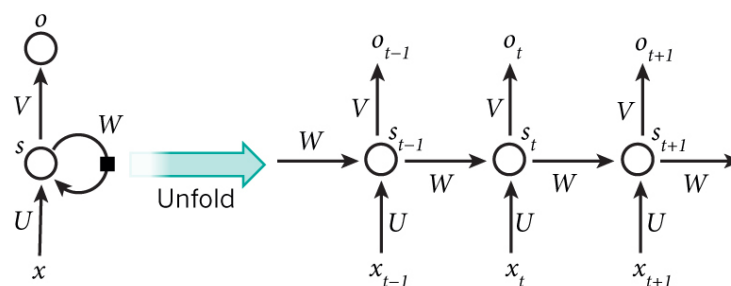


Figure 49: Recurrent Neural Network in time

16.4.2 The algorithm

16.4.2.1 Forward pass Following the process described in Section (15.1.6) for the MLP, we now consider a sequence of length T with N_i inputs, one hidden layer ($K = 2$) with $N_{o,K-1}$ hidden units and N_K output units. The i -th external input at time t is denoted by $x_i(t)$, and we let $net_{h,K-1}(t)$ and $y_h(t)$ be the *net value* (summation of the value received) and the output of unit h , respectively, in the hidden layer at time t . After unfolding the network, the forward pass of the RNN is similar to that of a MLP. The only difference being that the unit in the hidden layer receives values from both the current external inputs and the previous output of all hidden units.

Mathematically, the net value of a hidden unit is given by

$$net_{h,K-1}(t) = \sum_{i=1}^{N_i} w_{ih}x_i(t) + \sum_{h'=1}^{N_{o,K-1}} w_{h'h}y_{h'}(t-1) \quad (16.103)$$

where h' is a unit on the previous hidden layer. Then the output value of the hidden unit is calculated by applying the sigmoid activation f to the net value

$$y_h(t) = f(net_{h,K-1}(t)) \quad (16.104)$$

such that the output of hidden units at each time step can be computed recursively, starting from the first time step $t = 1$. Initial value of $y_h(0)$, $\forall h$ is commonly set to be zero, meaning that the network has not received information before the beginning of forward pass. Still, some researchers tend to have nonzero initial value by which they found that the network is more stable (see Zimmermann [2006]). The net value of the output unit, $net_{s,K}$, depends only on the output value of the hidden layer so that the output units are synchronised with the hidden units

$$net_{s,K}(t) = \sum_{h=1}^{N_{o,K-1}} w_{hs}y_h(t)$$

The error function can be defined as

$$E = \sum_{t=1}^T \sum_{s=1}^{N_K} \frac{1}{2} (y_s(t) - z_s(t))^2 \quad (16.105)$$

where $z_s(t)$ is the target value of the output unit s at time t . However, this is not the unique form of error function, as it should be based on the training algorithms of the RNN, which we will discuss in the next section.

16.4.2.2 Backward pass We explained in Section (15.1.6) how to calculate the derivatives of the error function with respect to the weights in an MLP. In the case of RNNs, there are mainly two algorithms to calculate the weight derivatives:

1. The Real Time Recurrent Learning (RTRL) (see Robinson et al. [1987], Williams et al. [1989]).
2. The Backpropagation Through Time (BPTT) (see Werbos [1990], Williams et al [1992]).

In order to describe the learning algorithm of the RNN and detail its associated problems, we focus on the BPTT, as it is simple conceptually and more efficient in computation time (see Graves [2008] [2012]). Figure 50 illustrates the scheme for updating the weights. Note that the same weights are used in every time step. The error function is defined in Equation (16.105), which include all the time steps.

For MLPs, the error signals δ_j , $j = 1, 2, \dots$, are computed in Equations (15.90), (15.92) and (15.94) according to the type of unit. However, what is common for the error signals of the hidden units is that they depend on the sum of error signals that flow back to them. Since unfolded RNNs are equivalent to MLPs, such rule can also be applied. As illustrated in Figure 50, we consider the hidden unit h_2 at time $t = 2$, which has two outgoing connections. We can

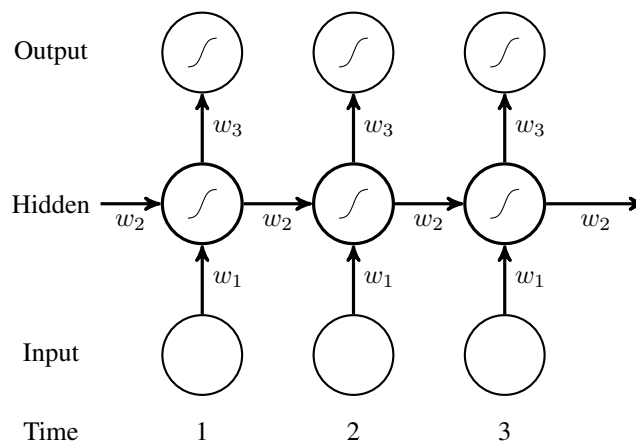


Figure 50: Weights in BPTT

see that the error signal of hidden unit h_3 at time $t = 3$, and that of the output unit o_2 at time $t = 2$ would flow back to unit h_2 during back propagation through time. The red dashed arrows in Figure 51 illustrates the flow of error signals backward to a particular hidden unit. Hence, the error signal at unit h_2 can be computed as

$$\delta_{h_2} = \delta_{h_3} + \delta_{o_2}$$

Generally, the error function depends on the output of hidden layer not only through the current output layer, but also through the hidden layer next time. Thus, the error signal at unit h and time t is given by

$$\delta_h(t) = f'(net_{h,K-1}(t)) \left(\sum_{s=1}^{N_K} \delta_s(t) w_{hs} + \sum_{h'=1}^{N_o} \delta_{h'}(t+1) w_{hh'} \right) \quad (16.106)$$

where $\delta_j(T+1) = 0, \forall j$. Hence, starting the backward pass at the last time T , we apply Equation (16.106) recursively. Having computed the error signals, we can now calculate the derivative of the error function with respect to the weight via chain rule, getting

$$\frac{\partial E}{\partial w_{ij}} = \sum_{t=1}^T \frac{\partial E}{\partial net_j(t)} \frac{\partial net_j(t)}{\partial w_{ij}} = \sum_{t=1}^T \delta_j(t) y_i(t) \quad (16.107)$$

16.4.3 The vanishing gradient problem

Even though recurrent neural networks (RNNs) are capable of processing serially correlated sequences, the length of sequence that a standard recurrent neural network can access is actually very limited. It results from the fact that the gradients would either decay or blow up when cycling around the recursive connections for too many times, which is usually referred to as *vanishing gradient problem* (see Bengio et al. [1994]). We can see in Equation (16.107) that the derivative of the error with respect to a weight depends on the error signal of the weight's destination unit, j , which itself depends on the derivative of the activation function.

Recall, the derivative of the sigmoid function can explode or vanish depending on the slope of the function. As the number of time steps get larger and larger, so does the number of layers in the unfolded RNN.

- Assuming $c = 1$ in the sigmoid function, the error signal of a hidden unit at time $t = T$ would be propagated back through $(T - 1)$ layers, until a hidden layer at time $t = 1$. That is, it would be multiplied by $(T - 1)$ sigmoid activations' derivatives, all in the range $[0, \frac{1}{4}]$. For T not too large, the vanishing problem does not have much impacts on the network, but for larger T the gradient would decay exponentially.

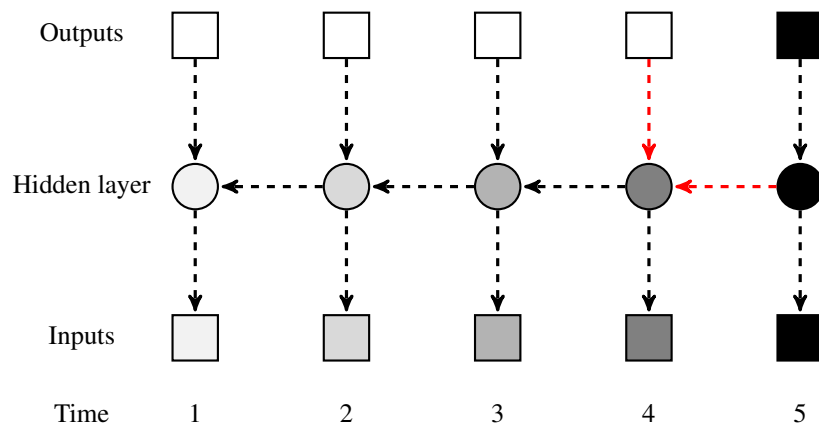


Figure 51: The flow of error signals and vanishing gradient problem in back propagation. The darkness of the shading of the nodes in the unfolded recurrent network indicates the degree of influence (sensitivity) of error signal of output unit at time $t = 5$. The darker the shade represents higher sensitivity. The sensitivity decays as the error signal is back propagated through time.

- Similarly, for large enough c , the derivative of the sigmoid function near the origin ($x = 0$) would be larger than 1, leading to gradient explosion. This property of sigmoid function is illustrated in Figure 52.

For these reasons, training RNNs with standard gradient descent algorithm is only feasible for small time steps. For longer time dependencies, the gradient vanishes as the error signal is propagated back through time, so that the network weights are never adjusted correctly when taking the events far back in the past into account (see Hochreiter et al. [2001]).

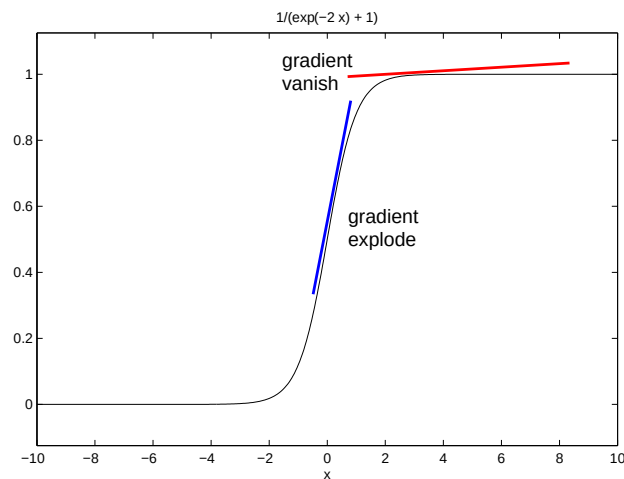


Figure 52: Plot of $\frac{1}{1 + e^{-2x}}$ and its tangent lines. The black curve is the sigmoid function ranged $[0, 1]$ with $c = 2$. The red line is the flat tangent line.

16.4.4 Dealing with long-range dependencies

We saw in Section (16.4.2) that recurrent neural networks (RNNs) were powerful models, since they combine two properties:

1. Distributed hidden state allowing them to store a lot of information about the past efficiently.
2. Non-linear dynamics allowing them to update their hidden state in complicated ways.

In RNNs, we can specify targets in several ways:

- Specify desired final activities of all the units.
- Specify desired activities of all units for the last few steps: It is good for learning attractors. Further, it is easy to add in extra error derivatives as we backpropagate.
- Specify the desired activity of a subset of the units. The other units are input or hidden units.

The differences between the forward and backward passes are as follows:

- In the forward pass we use squashing functions (like the logistic) to prevent the activity vectors from exploding.
- The backward pass, is completely linear. If we double the error derivatives at the final layer, all the error derivatives will double.
- The forward pass determines the slope of the linear function used for backpropagating through each neuron.

However, the problem of exploding or vanishing gradients, described in Section (16.4.3), results in RNNs having difficulties dealing with long-range dependencies. To go around, five effective ways of learning an RNN have been proposed:

1. Long Short Term Memory: Build the RNN out of little modules that are designed to remember values for a long time.
2. Hessian Free Optimisation: Deal with the vanishing gradients problem by using a fancy optimiser that can detect directions with a tiny gradient but even smaller curvature. For example, the HF optimiser by Sutskever et al. [2011].
3. Echo State Networks: Initialise the input, the hidden states and hidden states, the hidden states and output. The hidden state has a huge reservoir of weakly coupled oscillators which can be selectively driven by the input. ESNs only need to learn the connections from the hidden states to the output.
4. Good initialisation with momentum: Initialise like in Echo State Networks, but then learn all of the connections using momentum.
5. Gradient norm clipping strategy: Dealing with the exploding gradient problem, Mikolov [2012] proposed to rescale its norm whenever it goes over a threshold.

We will detail LSTM and ESNs in the following sections, but we first describe their basic principles.

16.4.4.1 The LSTM In LSTM, Hochreiter et al. [1997] designed a memory cell using logistic and linear units with multiplicative interactions. Information in and out of the unit is guarded by learned input and output gates. It gets into the cell whenever its "write" gate is on. The information stays in the cell so long as its "keep" gate is on. The information can be read from the cell by turning on its "read" gate. This over-parametrisation of the activation function preserve memory from the distant past. As an example, reading cursive handwriting is a natural task for an RNN. The input is a sequence of (x, y, p) coordinates of the tip of the pen, where p indicates whether the pen is up or down. The output is a sequence of characters. Graves et al. [2008] showed that RNNs with LSTM were producing good results for reading cursive writing. They used a sequence of small images as input rather than pen coordinates. However, as discussed in Section (16.4.3), the computational power of RNNs makes them very hard to train.

16.4.4.2 Hessian Free Optimisation The Hessian-Free Optimiser is used in conjunction with structural damping. The vanishing gradient problem is addressed as follows: In high dimensional spaces there is a high probability for long term components to be orthogonal to the short term ones. It allows the Hessian to rescale these components independently. In the case of exploding gradient structural damping is an enhancement forcing the Jacobian matrix to have small norm.

16.4.4.3 Reservoir computing It is easier to consider reservoir computing (Echo State Networks and Liquid State Machines) with sparse reservoir. The key idea of ESNs is as follows: A simple way for learning a feedforward network is to make the early layers random and fixed them. Then, we just learn the last layer which is a linear model that uses the transformed inputs to predict the target outputs. The equivalent idea for RNNs is to fix the input to hidden connections and the hidden to hidden connections at random values and only learn the hidden to output connections. This way we do not have to learn the hidden connections. Assuming linear output units, the learning becomes very simple. However, it is important to set the random connections very carefully so the RNN does not explode or die. Setting the random connections in Echo State Network is as follows:

- Set the hidden to hidden weights so that the length of the activity vector stays about the same after each iteration. It allows the input to echo around the network for a long time.
- Use sparse connectivity (set most of the weights to zero) to create lots of loosely coupled oscillators.
- Choose the scale of the input to hidden connections very carefully. They need to drive the loosely coupled oscillators without wiping out the information from the past that they already contain.
- Since the learning is very fast, we can try many different scales for the weights and sparsenesses to improve convergence.

Since ESNs just fit a linear model, they can be trained very quickly. Further, they demonstrate that it is very important to initialise the weights sensibly. While they can do impressive modelling of one-dimensional time-series, they experience some difficulties when modelling high-dimensional data such as pre-processed speech. Given a complex task, they require more hidden units than a RNN since the latter also learn the hidden to hidden connections. In fact, Sutskever [2012] showed that RNNs can be trained very effectively if the weights are initialised using the ESN methods.

16.4.4.4 Gradient norm clipping strategy We discussed in Section (16.4.3) the vanishing and the exploding gradient problems when training recurrent neural networks. Mikolov [2012] proposed to clip the elements of the gradient when they exceed in absolute value a fixed threshold. See algorithm (17).

Algorithm 17 Norm clipping

Require: The vector of model parameters ψ and the threshold ξ . The gradient is denoted g .

- 1: $\hat{g} \leftarrow \nabla_{\psi} g$
- 2: if $\|\hat{g}\| \geq \xi$ then
- 3: $\hat{g} \leftarrow \frac{\xi}{\|\hat{g}\|} \hat{g}$
- 4: end if

Diverging from the BPTT, Pascanu et al. [2013] followed a dynamical systems perspective (see Doya [1993]) and stated that when gradients explode we have a cliff-like structure in the error surface. Thus, similarly to Mikolov [2012], they proposed a gradient norm clipping strategy to deal with exploding gradients. They also proposed a soft constraint for the vanishing gradients problem. This regularisation term forces the Jacobian matrices to preserve the norm only in relevant directions.

16.5 The long short-term memory

The impact of RNNs in nonlinear modelling has been limited because they are difficult to train by gradient-descent methods. The gradual change of network parameters during the learning process leads the network to bifurcations, where the gradient information degenerates and may become ill-defined (see Doya [1992]). Further, many update cycles may be necessary to obtain convergence of a few parameters, leading to long training times. In addition, when dealing with long-range memory, the gradient information may dissolve exponentially over time. One remedy is to use the Long Short-Term Memory networks (LSTM), which we are now going to describe.

16.5.1 The constant error carousel

One consequence of the vanishing, or exploding, gradient problem described in Section (16.4.3) is that the traditional RNN will fail when involved with large time series. Since the vanishing gradient problem stems from the fact that the tangent line of the sigmoid function is either flat or steep, it is reasonable for some units to have constant gradient equating to 1. This kind of unit has been presented as the *Constant Error Carousel* (CEC) (see Hochreiter et al. [1997]).

In order to describe the advantageous properties of CEC, we consider a single unit, j , with a single connection to itself. According to the rule of calculation of error signals, the j th error signal satisfies

$$\delta_j(t) = f'_j(\text{net}_j(t))\delta_j(t+1)w_{jj}$$

where $f_j(\cdot)$ is the activation function of unit j . A constant error flow implies that $\delta_j(t) = \delta_j(t+1)$, leading to

$$f'_j(\text{net}_j(t))w_{jj} = 1$$

This is an ordinary differential equation (ODE)

$$\frac{\partial f_j(\text{net}_j(t))}{\partial \text{net}_j(t)} = \frac{1}{w_{jj}}$$

which we can integrate, getting

$$f_j(\text{net}_j(t)) = \frac{\text{net}_j(t)}{w_{jj}}$$

for arbitrary $\text{net}_j(t)$. Thus, the activation function f_j for the j th unit should be **linear**

$$y_j(t+1) = f_j(\text{net}_j(t+1))$$

Since unit j has only one connection to itself, we get

$$net_j(t+1) = w_{jj}f_j(t)$$

and from the equation of $f_j(net_j(t))$, we get

$$f_j(w_{jj}y_j(t)) = \frac{w_{jj}y_j(t)}{w_{jj}} = y_j(t)$$

We can therefore set the activation function to be the identity function $f_j(x) = x, \forall x$, obtaining $w_{jj} = 1$. The unit j above can be extended to CEC by adding some extra features. A multiplicative *input gate unit* and a multiplicative *output gate unit* are introduced to control the input and output of the CEC, respectively. The input gate unit can prevent the memory of unit j from being overwritten by irrelevant inputs, while the output gate unit avoid perturbation of other units by controlling the output of unit j . The CEC ensures that the error signal arriving at the memory cell would not be scaled up or down during back propagation, and thus can avoid exponential gradient.

16.5.2 Network architecture

A unit which include the input gate unit, the output gate unit and the CEC is called a *memory cell* (see Figure 53). We are now going to describe the *Long Short-Term Memory* (see Hochreiter et al. [1997]) which is a recurrent neural network consisting of set of *memory blocks*, where each block contains one or more memory cells. In fact, the LSTM network is the same as that of a standard recurrent neural network, except that the conventional hidden units are replaced by a memory block with cells and gates encapsulated in. Surely, the hidden layer of the LSTM network can be a mixture of conventional hidden units and memory blocks, but the former is not necessary. The v -th memory cell of the j -th memory block is denoted by c_j^v , the net value of the cell, the input gate and the output gate are $net_{c_j^v}$, net_{in_j} and net_{out_j} , respectively, the output value of the cell, the input gate and the output gate are $y_{c_j^v}$, y_{in_j} and y_{out_j} , respectively. Note that memory cells in the same memory block are controlled by the same input gate and output gate (see Figure 53).

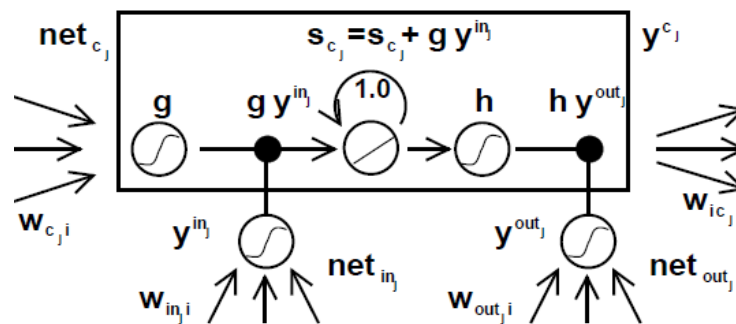


Figure 53: Architecture of a memory block with one memory cell. The CEC is a central linear unit with a self-connected weight 1.0. gate units use inputs from other units to decide whether to access or discard certain information.

As with a standard recurrent network (see Section (16.4.1)), we have

$$y_{in_j}(t) = f_{in_j}(net_{in_j}(t))$$

$$y_{out_j} = f_{out_j}(net_{out_j}(t))$$

where f_{in_j} and f_{out_j} are the activation function of the input gate and output gate of memory block i , and

$$\begin{aligned}
net_{in_j}(t) &= \sum_u w_{in_j u} y^u(t-1) + \sum_{n=1}^{N_i} w_{in_j n} x_n(t) \\
net_{out_j}(t) &= \sum_u w_{out_j u} y^u(t-1) + \sum_{n=1}^{N_i} w_{out_j n} x_n(t) \\
net_{c_j^v}(t) &= \sum_u w_{c_j^v u} y^u(t-1) + \sum_{n=1}^{N_i} w_{c_j^v n} x_n(t)
\end{aligned}$$

where the superscript u stands for memory blocks and other traditional hidden unit. The output of the memory cell is different from that of a conventional hidden layer. An additional variable $s_{c_j^v}$, called *internal state*, should be considered

$$\begin{aligned}
s_{c_j^v}(t) &= s_{c_j^v}(t-1) + y^{in_j}(t)g(net_{c_j^v}(t)), \quad t > 0 \\
s_{c_j^v}(0) &= 0,
\end{aligned} \tag{16.108}$$

Actually, the first line of Equation (16.108) can be written as

$$s_{c_j^v}(t) = 1.0 \times s_{c_j^v}(t-1) + y^{in_j}(t)g(net_{c_j^v}(t))$$

where 1.0 corresponds to the self-recursive connection with weight 1.0 in Figure 53, which is essentially w_{jj} obtained by the introduction of a simple self-connected unit j without the vanishing gradient problem. So the output of the cell can be computed as

$$y_{c_j^v}^v(t) = y^{out_j}(t)h(s_{c_j^v}(t))$$

where g and h are the squashing function of the net value of memory cell and the internal state $s_{c_j^v}$.

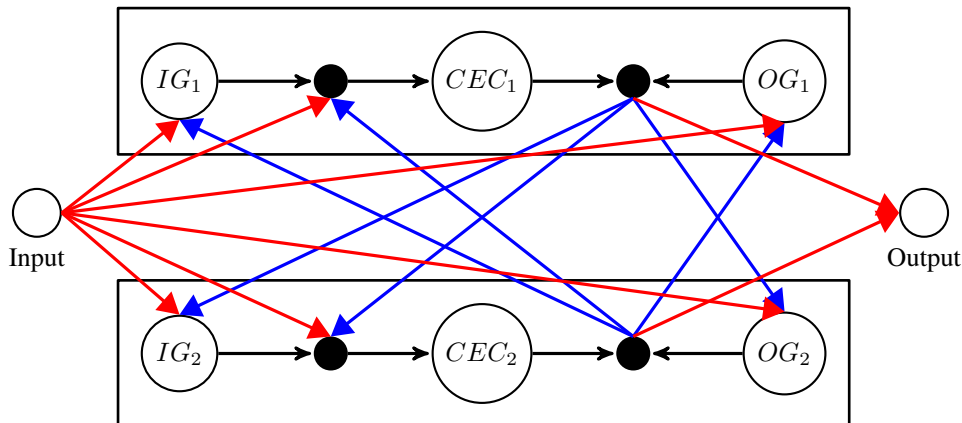


Figure 54: Connections in LSTM network. The example network consists of one input unit, a hidden layer of two single-cell LSTM memory blocks and one output units. Suppose the input is at time t . The the sources of the blue connections are from the previous one time step $t-1$. while the red connections convey information of current time step t . IG:input gate, OG: output gate, CEC:constant error carousel

It is important that the gate units and the internal states are only visible within the cell, and that only the output of the cell can connect to other blocks (including gates and cell) in the hidden layer. Therefore, the net value is the

summation of current input values and the output of memory cells $y_i^{c_j}$ from previous time step. Further, Graves et al. [2005] made some modifications to the original model of Schmidhuber and Hochreiter by modifying the rule connecting the units. Only the outputs memory cells are allowed to connect to other blocks, and the CECs and outputs of gate units are only visible within the memory blocks they are located in. As an example, Figure 54 shows the types of connections between two blocks. The gates units enable the memory cells in LSTM to store as well as to access information for a long period of time, and thus alleviate the vanishing gradient problem. For instance, if the activation function of the input gate is zero (the input gate is closed), then the memory of the cell will not be overwritten by the current input, thereby making it accessible to the network at a later time, as long as the output gate is open. However, note that the original LSTM has a weakness. When the sequence of time is long and have not been segmented to reset the network at certain time, Equation (16.108) implies that the internal state will grow infinitely and the network will collapse. To remedy this problem, Gers et al. [2000] proposed to add a *forget gate* to the self-connection with weight 1.0 in the original LSTM cell (see Figure 53). Then the revised equation for the internal state is extended from Equation (16.108) as follows:

$$\begin{aligned} s_{c_j^v}(t) &= y^{\phi_j}(t)s_{c_j^v}(t-1) + y^{in_j}(t)g(net_{c_j^v}(t)), \quad t > 0 \\ s_{c_j^v}(0) &= 0. \end{aligned} \quad (16.109)$$

where $y^{\phi_i}(t)$ is the value of the forget gate of the i -th memory block, which is squashed between zero and one. The forget gate is analogous to the reset operation of the memory cell. So far, the gate units are not entitled to control the internal state, as the gates have merely two sources of input: from the current input units and the previous output of all memory cells. In other words, the gates unit can only observe the cells' output. Once the output gate is closed, there is no way for gates to access the CEC they are supposed to control, which results in insufficient information that do harm to the performance of the network. In order to avoid the lack of information of internal states, another augmentation of the LSTM with forget gate was also introduced by Gers et al. [2002]. Weighted *peephole weights* are added in order to connect from the CEC to all gates of the same memory block. This effective remedy ensures that all the gates can "inspect" the internal state currently, even if the output gate is closed (see the dashed arrow in Figure 55).

16.5.2.1 Summary We can summarise the types of connections of the modern LSTM as follow

- **Outgoing connections**
 - outside memory block
Cells' output can feed to any types of units in any blocks in the hidden layer and the output units and they are the only output that can be connected to other blocks and output units.
 - inside memory block
The gate unit can only pass value forward to cells of the block where the gate belong to. The internal state can connect to all types of gate within the same block.
- **Incoming connections**
The memory cell's input can receive outputs from the hidden layer and the input layer;
 - The gate units can receive from the outputs from the hidden layer, input layer and the value of internal states.
 - The output unit can only receive value from the outputs of conventional hidden units and memory cells.

By including the forget gate and peephole weights, the traditional LSTM evolved to the modern LSTM and the update scheme in Hochreiter et al. [1997] needs refinement.

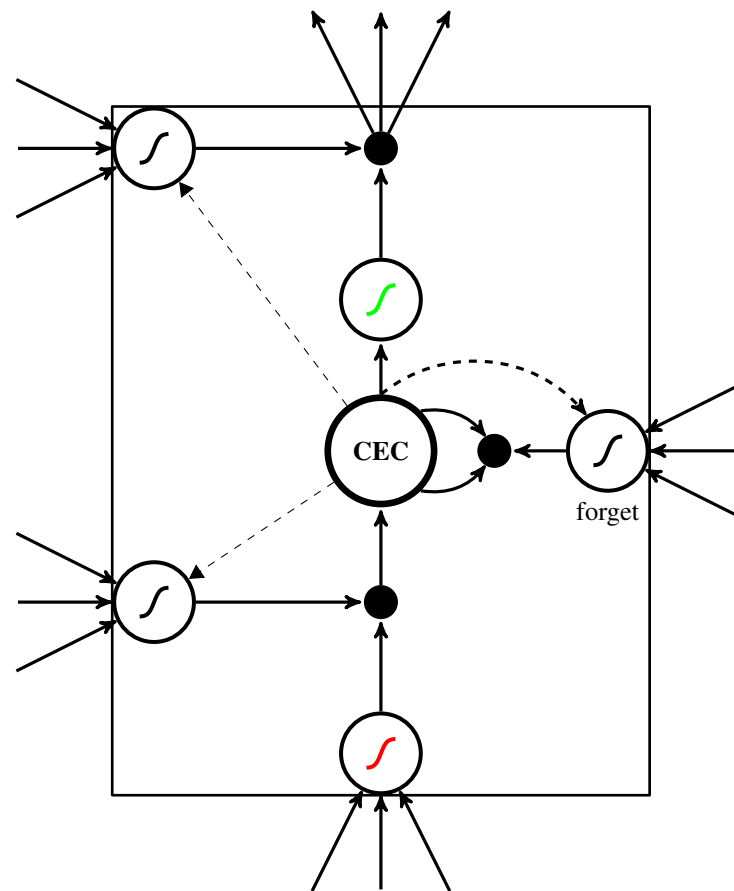


Figure 55: A modern LSTM memory block with one cell. The input, output and forget gates are the analogous of write read and reset operations for the cells. The three gates are nonlinear summation units that collect outputs from outside and inside the block. The small black dots are multiplicative units by which the activation of the cell is controlled. The gate units have sigmoid function f with range $[0, 1]$, so that 0 corresponds to the gate being closed and 1 to the gate being open. The unit with red S-shaped curve is the activation function that squashes the net value of cells' input, and the unit with green S-shaped curve is the activation function of cells' output. Usually, the functions g and h are task-specific and are encouraged to have ranges different from $[0, 1]$. The input and output gates multiply the input and output of the cell, and the forget gate multiply the previous internal state. Peephole weights from internal states to gates are shown by dashed arrow. Note, except for the self-recursive connection of the CEC, connections within the blocks have fixed weights of 1.0. The only outputs from the memory block to the rest of the network is from the upper multiplicative unit.

16.5.3 The learning algorithm

Details and proofs of the learning algorithm of the LSTM can be found in Hochreiter et al. [1997]. We display the learning algorithm in Figure 56. We emphasise here that the update scheme involves truncated derivative which are used to enhance the efficiency of the network. Note that the *training error* is the average of the $|z_t - y_t|$, $t = 1, \dots, T$, in Figure 56. The weight update scheme is illustrated in Figure 57. The updated algorithm for the backward pass is a combination of truncated BPTT and customised RTRL. The former refers to BPTT using truncated derivatives. To

```

for  $i = 1$  to  $n$  do
  Initialize. Initialize all weights  $\{w_{ij}(1)\}$  randomly.
  while stopping criteria not met do
    for  $t = 1$  to  $T - 1$  do
      Feedforward from  $x_t$  to output  $y_t$  using weights  $\{w_{ij}(t)\}$ .
      Back propagation. Calculate error signals based on  $y_t - z_t$ . Take down  $|y_t - z_t|$ .
      Update weights.  $\{w_{ij}(t)\} \rightarrow \{w_{ij}(t+1)\}$ 
    end for
    Reset network except for its weights.
  end while
end for

```

Figure 56: Pseudo-code of LSTM. The subscripts i and j standards for any units that have connections. y_t and z_t denote the actual output and target output of the network at time t . $z_t = x_{t+1}$. n is the number of simulations.

be specific, standard BPTT is applied to output units, truncated BPTT is implemented by output gates, while weights related to input gates, forget gates and memory cells use a truncated RTRL. By comparing Figure 50 with Figure 57 we can get an idea of the differences between the full BPTT and the truncated one used by LSTM. In the BPTT algorithm for standard RNN, Equation (16.106) shows that the information of all time steps has to be saved for the update of weight and the weights remain unchanged during the entire feedforward process. Whereas the LSTM network updates its weights at each time step during the forward pass, so that there is no need to store all values. To summarise, the BPTT with full gradient change the weight for one time by passing forward from time $t = 1$ to $t = T$ and passing backward from $t = T$ to $t = 1$ while the truncated BPTT passes forward from $t = 1$ to $t = T$ and the weights are updated for T times.

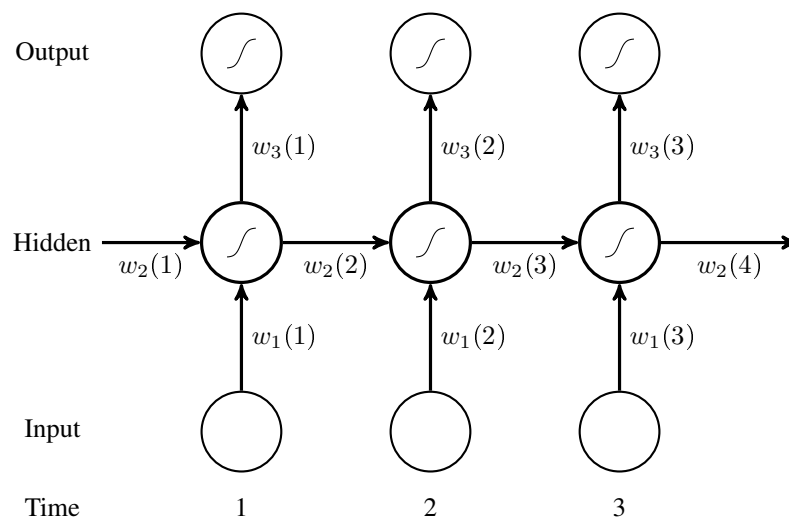


Figure 57: Algorithm of weight update of LSTM. Note that the memory cell is regarded as a normal hidden unit for simplicity. Only one hidden unit is shown. The number in bracket represents the time step. Weights are updated at each time step.

16.5.3.1 Computational complexity of LSTM The LSTM algorithm is efficient and its computational complexity is of order $O(W)$ per time step, where W is the number of weights (see Hochreiter et al. [1997]). Compared with BPTT, the LSTM is more economical in terms of space. In fact, calculating the full gradient of the LSTM also has some advantages since it is easy to debug and can be checked by numerical approximations (see Graves et al. [2005]). With the CEC, a LSTM block may be considered as a smart network unit, compared to the conventional hidden unit, to store information for arbitrary length of time. Therefore, the LSTM is well-suited to process and predict time series when there are unknown size of long time lags between important events.

Since financial time series has long term memory, predicting volatility by LSTM network may be rather promising. While the traditional RNN with weight update algorithm BPTT (Back Propagation Through Time) (see Williams et al. [1990]) and RTRL (Real-time Recurrent Learning) (see Robinson et al. [1987] and Williams et al. [1992]) and the combinations of the former two (see Schmidhuber [1992]) have been proved to occur learning failure when processing sequences with only 10 time steps (see Bengio et al. [1994], Hochreiter et al. [1997], Gers et al. [2000], Hochreiter et al. HochreiterEtAl01), the LSTM can deal with 1000 time steps and even more, outperforming those traditional RNN algorithms.

In fact, except for prediction, LSTM outperforms other RNNs in numerous aspects such as the best performance in speech recognition (see Graves et al. [2011]) and the ICDAR handwriting competition in 2009.

16.6 Reservoir computing

Reservoir Computing (RC) is a special RNN originating from Echo State Networks (ESNs) (see Jaeger [2001]) and Liquid State Machines (LSMs) (see Maass et al. [2002]), which assumes that supervised adaptation of all inter-connection weights are not necessary, and only training a memoryless supervised readout from it is sufficient to obtain good results. Thus, it avoids the shortcomings of the gradient-descent training of RNNs (see Section (16.4.3)). RC is based on the computational separation between a dynamic reservoir and a recurrence-free readout. The former is an RNN as a nonlinear temporal expansion function, randomly created and unchanged during training. The latter produces the desired output from the expansion. This is a very simple approach avoiding the problem of bifurcations encountered during the training of RNNs, and the use of complex memory cells when learning long-term dependencies, as in the LSTM described in Section (16.5). Even though several studies aimed at understanding RC and the factors affecting its performances, none have been completely satisfactory due to the complexity of the reservoir, often resulting in contradictory conclusions. Introduction to the concepts and methodologies can be found in the literature (see Lukosevicius et al. [2009], [2012]). Goudarzi et al. [2014] compared the performance of three methods, the delay line, the NARX network, and the ESN and concluded that the first two have higher memorisation capability, but fall short of the generalisation power of the latter. Thus, we are going to briefly describe the ESN.

16.6.1 Describing the Reservoir methods

Reservoir computed models are expressed in terms of non-linear state-space models comprising a set of input, output and state variables.

16.6.1.1 Description We let the time-varying input signal be an N_i -th order column vector $U(t) = [u_i(t)]$, the reservoir state is an N_x -th order column vector $X(t) = [x_j(t)]$, and the generated output is an N_o -th order column vector $Y(t) = [y_o(t)]$. A Reservoir Computer is a collection of internal nodes, whose state vector $X(t)$ evolve in discrete time t according to the nonlinear map

$$X(t) = \phi(X(t-1), U(t))$$

where ϕ is the map, and given in the form

$$X(t+1) = (1-\alpha)X(t) + \alpha f(W_{res}^\top \cdot X(t) + W_{in}^\top \cdot U(t+1) + W_{fb}^\top \cdot Y(t)) \quad (16.110)$$

for some activation function $f(\cdot)$, where the leaky integrator α is a constant controlling the speed of the dynamics. The input weight matrix is an $N_i \times N_x$ matrix $W_{in}^{in} = [w_{ij}^{in}]$ where w_{ij}^{in} is the weight of the connection from input node i to reservoir node j . The connection weights inside the reservoir are represented by an $N_x \times N_x$ matrix $W_{res} = [w_{jk}^{res}]$ where w_{jk}^{res} is weight from node j to node k in the reservoir. In presence of a bias, the output matrix is an $(N_x+1) \times N_o$ matrix $W_{out} = [w_{ko}^{out}]$, where w_{ko}^{out} is the weight of the connection from the reservoir node k to the output node o . In the case where the output nodes are also connected to the input nodes and to themselves the size of the matrix becomes $(N_x + N_i + N_o) \times N_o$. The feedback matrix is an $N_x \times N_o$ matrix $W_{fb} = [w_{ko}^{fb}]$, where w_{ko}^{fb} is the weight of the connection from the reservoir node k to the output node o . If no output feedback is needed, then W_{fb} is null. The generated output is given by

$$Y(t) = f_{out}(W_{out}^\top \cdot X(t)) \quad (16.111)$$

where $f_{out}(\cdot)$ is an output activation function, typically the identity or a sigmoid. The output weights are trained to minimise the squared output error

$$E = \|Y(t) - \hat{Y}(t)\|^2$$

given the target output $\hat{Y}(t)$. Once we know the optimum weights of the network, we can use the model to perform a forecast. To do so, we let T be a network state update, where

$$X(t+h) = T(X(t), \bar{U}^h)$$

denote the network state resulting from an iterated application of Equation (16.110) when the input sequence $\bar{U}^h = U(t+1), \dots, U(t+h)$ is fed to the network being in state $X(t)$ at time t .

The basic steps to construct an ESN are as follows:

- $\mathbf{W}_{in}$ and $\mathbf{W}_{res}$ are randomly initialised, an appropriate α is selected.
- For each t ,
 - Import input vector $\mathbf{U}(t)$ of the training set to the system and evolve the corresponding reservoir state $\mathbf{X}(t)$.
- A supervised learning algorithm is built on pairs of $\mathbf{X}(t)$ and $\hat{\mathbf{Y}}(t)$.
- For each t ,
 - Apply trained $\mathbf{W}_{out}$ on input vector $\mathbf{U}(t)$ of test set to compute predictions $\mathbf{Y}(t)$.

16.6.1.2 The parameters We present some of the major characteristics one has to consider when defining a relevant model (see Lukosevicius et al. [2009] and Lukosevicius [2012]). There are several features one has to consider when designing RC:

1. **Connectivity:** $\{W_{in}, W_{res}, W_{fb}\}$ is generated randomly. The input, bias, reservoir and feedback scaling (respectively named s_{in} , s_{bias} , s_{res} and s_{fb}) will be considered as micro parameters. For the choice of topology, the authors suggested a permutation matrix with a medium number and different lengths of connected cycles, or a general orthogonal matrix.
 - (a) N_x : The principle is that the bigger the reservoir, the better the obtainable performance, provided appropriate regularisation measures are taken against over-fitting. As ESNs are computationally cheap, we can use as big a reservoir as we can afford computationally. N_x will be considered as a macro parameter.
 - (b) Sparsity: It enables fast reservoir updates. One rule of thumb is to connect at most each node to 10 other nodes in the reservoir.
 - (c) Distribution: Uniform or Normal. W_{in} , W_{res} and W_{fb} have usually the same distribution. In general, the seed of the pseudo-random generator is fixed.
2. **Leaking Rate α** : The leaking rate can be seen as the time interval between two discrete realisations. In general, we expect its value to be close to 1.
3. **Echo State Property**: The state of the reservoir $X(t)$ should be uniquely defined by the fading history of the input $U(t) [\dots]$. The spectral radius should be greater in tasks requiring longer memory of the input. Finding clear characteristics remain a large debate among practitioners and researchers.

From these features, one can see that many meta parameters are involved, so that it is necessary to perform an additional optimisation aiming at minimising the RMSE on the validation sample. It is known as meta-optimisation (see Pedersen et al. [2008a]). The idea is to have an optimisation method act as an overlaying meta-optimiser, trying to find the best performing behavioural parameters for another optimisation method (see Pedersen et al. [2008a]). Note, parameter tuning via meta-optimiser is done in an offline manner, while adaptation of DE parameters is done online during optimisation.

According to Lukosevicius et al. [2009], the investigation shows that the error surfaces in the combined global parameter and W_{out} spaces may have very high curvature and multiple local minima. Thus, gradient descent methods are not always practical. In general, optimising reservoir is quite a hard task as we have many parameters involved. Nevertheless, the computationally cheap property of Reservoir Computing leads to the following idea: “generate k reservoirs and pick the best.”

16.6.1.3 The readout Echo State Networks (ESNs) assume that if a random RNN possesses certain algebraic properties, it suffices to train a linear readout from it to obtain good performances. Even though there are several readout methods, the most popular one is the linear regression where W_{out} solve a system of linear equations. We get

$$W_{out}^\top X_m = \hat{Y}_m$$

where both the matrix $X_m \in \mathbb{R}^{N_x \times T}$ and the matrix $\hat{Y}_m \in \mathbb{R}^{N_o \times T}$ ¹⁶, over the training period $t = 1, \dots, T$, have a column for every training time step t . The output weights can be estimated with direct pseudo-inverse calculations, or they can be estimated with the ordinary linear regression (Wiener-Hopf solution). Thus, we get

$$W_{out}^\top X_m \cdot X_m^\top = \hat{Y}_m \cdot X_m^\top$$

and the weight matrix becomes

$$W_{out}^\top = \hat{Y}_m \cdot X_m^\top (X_m \cdot X_m^\top)^{-1} = PR^{-1}$$

where $R = X_m \cdot X_m^\top$ is the correlation matrix of the reservoir states, and $P = \hat{Y}_m \cdot X_m^\top$ is the cross-correlation matrix between the states and the desired outputs. Note, $P \in \mathbb{R}^{N_o \times N_x}$ and $R \in \mathbb{R}^{N_x \times N_x}$ do not depend on the training length T and can be calculated incrementally. When R is ill-conditioned the method is numerically unstable, but computing the Moore-Penrose pseudo-inverse R^+ instead of R can improve the solution. Alternatively, we can decompose the matrix R into two triangular matrices with Cholesky or the LU decomposition (see Press et al. [1992]) and solve

$$W_{out}^\top R = P$$

by two steps of substitution to get $W_{out}^\top = PR^{-1}$.

Evolutionary search can also be used for training the linear readouts. There exists several bias to reduce the error in the validation set. One can smooth the model with regularisation functions with the Ridge regression

$$W_{out}^\top = P(R + \beta^2 I)^{-1} \quad (16.112)$$

where β controls the smoothing effect, and $I \in \mathbb{R}^{N_x \times N_x}$ is the identity matrix.

The following algorithm sums up the steps involved in the training of the ESN framework.

Algorithm 18 RC offline Training

Require: get a linear system solver (we use Cholesky Decomposition).

Require: set t_0 as a wash-out size.

Require: set β as a regularisation parameter.

- 1: $Y \leftarrow [y(t_0) \parallel \dots \parallel y(T)]$
 - 2: $Y \leftarrow f_{out}^{-1}(Y)$
 - 3: $X \leftarrow [1 \parallel U(t_0) \parallel \dots \parallel U(T) \parallel X(t_0) \parallel \dots \parallel X(T)]$
 - 4: $W_{out} \leftarrow Y \cdot X^T \cdot (X \cdot X^T + \beta^2 I)^{-1}$
 - 5: **return** W_{out}
-

Note, we can also add noise $\epsilon(t)$ (sampled from uniform or Gaussian distribution) to the reservoir states

$$X(t+1) = (1 - \alpha)X(t) + \alpha f(W_{res}^\top \cdot X(t) + W_{in}^\top \cdot U(t+1) + W_{fb}^\top \cdot Y(t)) + \epsilon(t)$$

to stabilise solutions in the model (see Jaeger [2007]). We can further adjust global control parameters to make the echo state network dynamically similar to the system we model. For instance, we can use fully connected reservoirs or sparsely connected ones.

¹⁶These matrices are transposed compared to the conventional notation.

16.6.1.4 Some comments In order to produce a reservoir with a rich enough set of dynamics the number of internal connections N_x should be large, the weight matrix W should be sparse, and the weights of the connections should be generated randomly from a uniform distribution symmetric around the zero value. Further, the network should have the echo state property (ESP), which relates asymptotic properties of the excited reservoir dynamics to the driving signal (see Jaeger [2001]). It states that the effect of a previous state $X(t)$ and a previous input $U(t)$ on a future state $X(t+h)$ should vanish gradually as $h \rightarrow \infty$, and not persist or be amplified. In reservoirs using the $\tanh$ squashing function, and for zero input, the reservoir weight matrix W^{res} must be scaled so that its spectral radius¹⁷ $\rho(W^{res})$ satisfies $\rho(W^{res}) < 1$. For any kind of inputs (including zero) and state vectors, we require $\sigma_{max}(W^{res}) < 1$ where $\sigma_{max}(W^{res})$ is the largest singular value of W^{res} . If the input comes from a stationary source, the ESP holds with probability 1 or 0. Due to the auto-feedback nature of RNNs, the reservoir states $X(t)$ reflect traces of the past input history, which can be seen as a dynamical short-term memory. Assuming a single input ESN, the short-term capacity is given by

$$C = \sum_i r^2(U(t-i), Y_i(t))$$

where $r^2(\cdot, \cdot)$ is the squared correlation coefficient between the input signal delayed by i and an output signal $Y_i(t)$ trained to memorise $U(t-i)$ on the input signal $U(t)$. For an i.i.d. input, the memory capacity C of an echo state network of size N is bounded by N . Thus, we can not train ESN on tasks requiring unbounded-time memory.

16.6.2 Choosing the parameters

Choosing the right parameters can be more beneficial than choosing the right model itself. However, it is never trivial to tune parameters of ESN. Experience will play an important role here. It is inevitable and necessary to use manual selection to get a sense of which parameters affect results more than others do, and these manually selected values could be used as the initial values for further automated selection. During the process, ensure that *only* one parameter is changed at a time. The quality of a particular parameter setting is evaluated by using root mean square error of training set and test set. Note that all the parameters ought to be optimised under a condition where adequate regularisation measure is taken to alleviate overfitting.

Practically, the input scaling, leak rate and spectral radius of reservoir weight matrix are the three most important parameters of ESN.

16.6.2.1 Input Scaling Input weight matrix W_{in} is usually dense without zero elements. The entries of the matrix usually follow a uniform distribution in a symmetric range $[-a, a]$ or classic range $[0, 1]$. Often candidate values for a is 1, 0.1, 0.01. The distribution can be Gaussian or a customised one as well, whereas the boundness is not guaranteed. The first column of the input weight matrix is usually the bias column, which corresponds to bias element '1' in input vector $U(t)$. Hence it does not share the same status as the rest of the columns. Thus, to reduce degree of freedom, all columns except the first column of the input weight matrix are randomized using one parameter a , and the first column uses another parameter b .

Small values of a will map input to a small range around zero, where $\tanh(\cdot)$ is virtually linear, hence this setting is suitable for solving linear systems. On the other hand, large values of a will map $u(n)$ to extreme values of $\tanh(\cdot)$, namely -1 or 1 , creating a non-linear, binary manner. Overall, input scaling controls non linearity of $X(t)$.

¹⁷the largest absolute eigenvalue

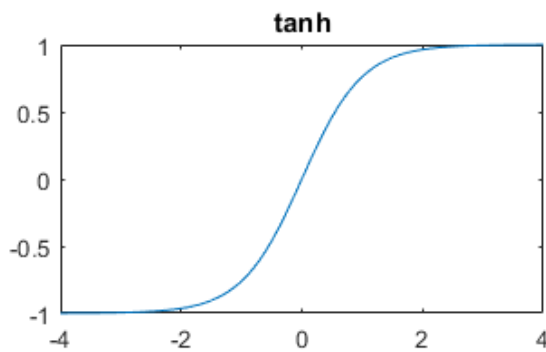


Figure 58: activation function tanh

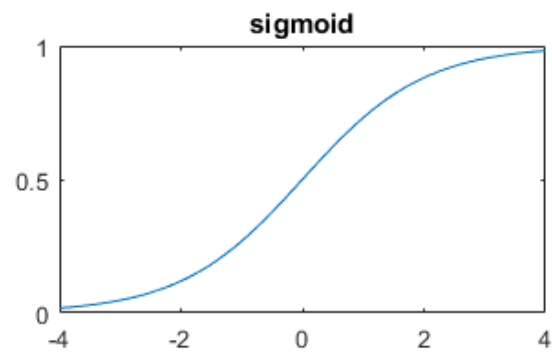


Figure 59: activation function sigmoid

The reservoir weight matrix $\mathbf{W}_{res}$ represents a structure of recurrently connected units. Similar to the case of $\mathbf{W}_{in}$, non-zero elements of $\mathbf{W}_{res}$ usually have uniform distribution in a symmetric range. However, unlike $\mathbf{W}_{in}$, $\mathbf{W}_{res}$ is sparse and has other distinct properties.

16.6.2.2 Size of the reservoir Intuitively, larger the reservoir, the better the attainable performance. the size can be as big as 20, 50, 100, 1000, even 10000 is not uncommon. There is few condition when reservoir may be oversized: one is that the task itself is pretty much trivial, and the other could be that the size approaches meaningful fraction of number of data points, or in other words, lack of data points. Normally, if sample size is n , it is required that (see Jaeger [2002])

$$N_x \leq n/2; \quad (16.113)$$

Typically, it also holds that $1 + N_u + N_x \ll n$, this constraint also reduces the risk of overfitting. At the same time, the size of reservoir should at least be the number of independent time steps that are supposed to be memorised to address the problem.

16.6.2.3 Sparsity Though the reservoir is big, the reservoir weight matrix is always set to be *sparse* in many publications. In general, higher density tends to give higher prediction performance. However, at the same time, lower sparsity means more recurrent connections, and large non-zero matrix operations increase computational costs. Generally, above a certain threshold, connectivity barely affects the final performance. Hence, one can recommend 30% connectivity, which is robust enough to be a balance between efficiency and efficacy. Compared to others, sparsity has relatively low priority to be optimised. In Matlab, there is a special representation for sparse matrix that could speed up computations.

16.6.2.4 Spectral Radius Being the central parameter of ESN, spectral radius is the maximum of absolute eigenvalues of reservoir matrix $\mathbf{W}$. Intuitively, it determines the width of the distribution of non-zero entries. If it is set too high, then the reservoir may hold periodic, even chaotic spontaneous attractor modes which deactivates echo state property. It has been proved that, in most cases, $r(\mathbf{W}) < 1$ ensures the echo state property (see Lukosevicius [2012]). However, it is also true that $\mathbf{W}$ with spectral radius greater than 1 retains echo state property. As a rule of thumb, if the task requires extensive input history, then it is recommended to set the spectral radius higher. The following steps are usually taken when specific spectral radius is assigned to reservoir weight matrix:

1. Initialize a reservoir weight matrix $\mathbf{W}_0$
2. Normalize $\mathbf{W}_0$ to $\mathbf{W}_1$ using $\mathbf{W}_1 = \mathbf{W}_0/r_1$, where r_1 is the spectral radius of $\mathbf{W}_0$. Now that $\mathbf{W}_1$ has unitary spectral radius.
3. Multiply $\mathbf{W}_1$ by r element-wise so that it has spectral radius r .

16.6.2.5 Leak Rate Leak rate is a real value between 0 and 1. Echo state network without leak rate α is called standard echo state network, the one with leak rate or leaky integrator is called LI-ESN. In this latter case, α can either be placed before application of $f(\cdot)$ or after. Introduction of leak rate makes it possible to learn very slow dynamic systems and replay them at various speeds (see Lukosevicius et al. [2009]). Small values of leak rate may induce slow dynamics of $\mathbf{X}(t)$ and extend short term memory (see Jaeger [2012]). Virtually, the prediction performance is sensitive to leak rate since a small change in leak rate may lead to a big change in final result. Hence, this parameter also has relatively high priority. Some low-prioritised parameters could be assigned default values for convenience.

16.6.3 Some improvements

Separating the reservoir from the readout training allows for two research directions to be pursued independently,

1. the generation of the reservoir, and
2. the output training.

There is no reasons why the reservoir should be randomly generated and alternative methods could be used to obtain optimal reservoir design. However, no single type of reservoir can be optimal for all types of problems (no free lunch principle). Several methods have been proposed for generating the reservoir (see Lukosevicius et al. [2009]), which can be classified as

1. generic methods for generating *RNNs* with different neuron models, connectivity patterns and dynamics.
2. unsupervised adaptation of the reservoir based on the input data $U(t)$ but not the target value $\hat{Y}(t)$.
3. supervised learning, adaptation of the reservoir using task-specific information from both $U(t)$ and $\hat{Y}(t)$.

In order to deal with different time scales simultaneously, one can divide the reservoir into decoupled sub-reservoirs and introduce inhibitory connections among all the sub-reservoirs. However, the inhibitory connections should be heuristically computed from the rest of W and W_{fb} such that they predict the activations of the sub-reservoirs one time step ahead (see Xu et al. [2007]). Alternatively, the Evolvino transfers the idea of ESNs to a LSTM type of RNNs where the LSTM RNN used for its reservoir consists of specific small memory-holding modules. In that model, the weights of the reservoir are trained using evolutionary methods (see Schmidhuber et al. [2007]). Further, ESNs like any other RNNs has only a single layer of neurons (see Figure 48a), making it unsuitable for some types of problems requiring multilayers. A solution is to use Layered ESNs (see Lukosevicius [2007]), where part of the reservoir connections are instantaneous, and the rest takes one time step for the signals to propagate as in normal ESNs. One can also add leaky integrator neurons to ESNs, getting leaky integrator ESNs (Li-ESNs) performing at least as well as the simple ESN (see Lukosevicius et al. [2006]). Since the parameters a and Δt control the speed of the reservoir dynamics, small values result in reservoirs reacting slowly to the input. Note, depending on the speed at which the input $U(t)$ changes, we can vary Δt on-the-fly, getting a warping invariant ESNs (TWIESNs). Since checking the performance of a resulting ESN is relatively inexpensive, evolutionary methods for pre-training the reservoir developed (see Ishii et al. [2004]). Generally, one separate the topology and weight sizes of W_{res} to reduce the search space (see Bush et al. [2005]). Jiang et al. [2008] showed that by only adapting the slopes of the reservoir unit activation functions $f(\cdot)$ with an evolutionary algorithm, and having W_{out} random and fixed, a very good prediction performance of an ESN could be achieved. Note, evolutionary algorithms can also be used to train the readouts. One can increase the expressiveness of the ESN by having k linear readouts trained and an online switching mechanism among them, or by averaging outputs over several instances of ESNs (see Bush et al. [2006]).

17 Towards dynamically stable reservoirs

17.1 A specific implementation

17.1.1 Overview of the topology

We let $\mathbb{T} := [1, N_T]$ be a discrete time set and define:

- $u \in \mathbb{R}^{N_T \times N_U}$ the input signal at time t and we define $U := [1, N_U]$.
- $X := (x^k)_{k \in N_K} \in \mathbb{R}^{N_K \times N_T \times N_{X_K}}$ a sequence of reservoir neuron activations (often called kernels). $\forall k \in \mathbb{K} := [1, N_K]$, x_k is a factor which can be seen as a fraction of the reservoir. Factors are usually defined with the same matrix design and can be correlated or not (in terms of dimension and jump). By allowing this third dimension, we can represent several high dimensional signal in a same reservoir and in a dependent or independent manner. They all have their own set of Equations (16.110) (16.111) and parameters coming with it. We also define $x = [x^1 \parallel \dots \parallel x^K]$.
- $y, \hat{y} \in \mathbb{R}^{N_T \times N_Y}$ are respectively the output and forecast signals. And we also define $\mathbb{Y} \equiv \hat{\mathbb{Y}} := [1, N_Y]$.
- $\forall k \in N_K$, $W_{in}^k \in \mathbb{R}^{N_{X_k} \times N_U}$, $W_{res}^k \in \mathbb{R}^{N_{X_k} \times N_{X_k}}$ and $W_{fb}^k \in \mathbb{R}^{N_{X_k} \times N_Y}$ are matrices mapping respectively u to x^k , x^k to x^k and y (or $\hat{y}$) to x^k . These matrices are randomly constructed but supported by a well chosen topology. We present our choice of topology in Appendix (17.1.3). They stand for the part of the topology learned in an unsupervised manner.
- The output matrix is slightly different as we gather all the reservoir factors in a unique vector $s = [1 \parallel u \parallel x^1 \parallel \dots \parallel x^{N_K}] \in \mathbb{R}^{N_T \times N_S}$ where $N_S = 1 + N_u + \sum_{k=1}^K N_{X_k}$. Then, the output matrix is $W_{out} \in \mathbb{R}^{N_Y \times N_S}$. This matrix is the only part of the topology learned in a supervised manner.

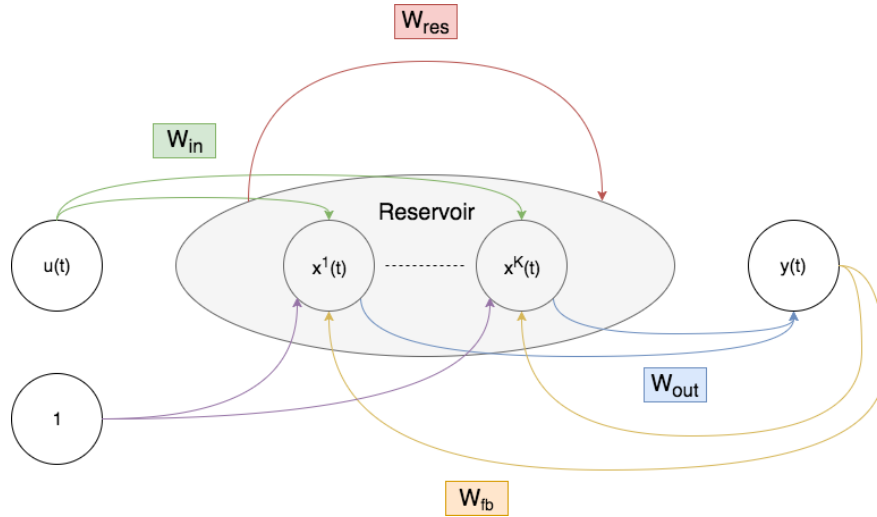


Figure 60: An Echo State Network with $N_x = K$ factors

17.1.2 Cost function

We are interested in the following temporal task: to find a model mapping u to y such that it minimises a given error measure. Mathematically, we can define a function ϕ corresponding to the network in Figure (60) as follows:

$$\begin{cases} \phi : T \times U \rightarrow Y \\ \phi : t, u \rightarrow \hat{y} = g(W_{out}[1\|u(t)\|x(t)]) \\ x : T \times U \times X \times Y \rightarrow Y \\ x : t, u, x, y \rightarrow x(t) = f(W_{in}u(t) + W_{res}x(t-1) + W_{fb}y(t-1)) \end{cases} \quad (17.114)$$

Note, x behaves as a simple network but we will introduce a more complex function later.

We recall the definition of the Root-Mean-Square Error (RMSE) between the train and target vectors:

$$E(y_{train}, y_{target}) = \sqrt{\frac{1}{T \times N_y} \sum_{t=1}^T \sum_{i=1}^{N_y} (y_{target,i}(t) - y_{train,i}(t))^2} \quad (17.115)$$

17.1.3 Random generation of the input and feedback matrices

These matrices are chosen randomly as suggested by the ESN framework. They can be initialised by using a particular random variable distribution, such as, the uniform or Gaussian distribution. In our experiments we choose to use a Gaussian distribution. We also consider a scaling parameter for each matrix in order to optimise them, denoted by s_{in} , s_{bias} , s_{res} and s_{fb} . As discussed in Section (2.2), the feedback scaling parameter is of key importance in our network as the reservoir needs to recall its last forecasting value in the best manner regarding our cost function defined in Equation (17.115).

17.1.4 Topology of the reservoir matrix

Using a completely random network leads the reservoir structure to be poorly understood. Rodan et al. [2012] demonstrated that simple cycle reservoir (SCR) shows comparable performance to the randomised one. Further, they have introduced a new model adding regular jumps to the SCR, called cycle reservoir with jumps (CRJ). The latter has shown superior performance compared to the traditional ones in non-linear system identification, more particularly in time series prediction. We consider this last model in our system. For example, if each of our reservoir factor was made of six neurons, the reservoir would be as in Figure (61).

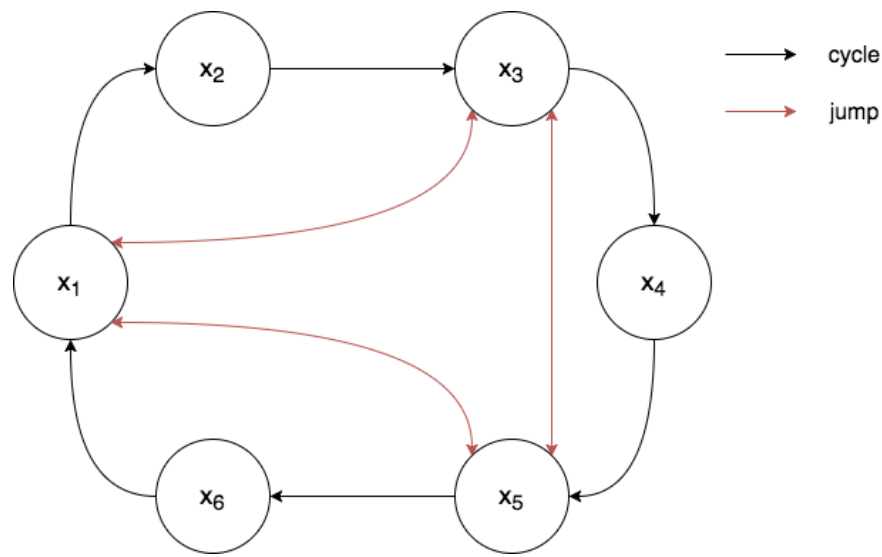


Figure 61: CRJ model for a 6 neurons reservoir factor

In the CRJ model, four parameters are considered for each factor: the cycle weight r_c , the jump weight r_j , the jump level telling at which neuron the jumps start and the jump sizing how many neurons are jumped. Given these parameters, the reservoir is entirely deterministic. Considering Figure (61) above, the reservoir matrix is constructed as follows:

$$W_{res} = \begin{bmatrix} 0 & 0 & r_j & 0 & r_j & r_c \\ r_c & 0 & 0 & 0 & 0 & 0 \\ r_j & r_c & 0 & 0 & r_j & 0 \\ 0 & 0 & r_c & 0 & 0 & 0 \\ r_j & 0 & r_j & r_c & 0 & 0 \\ 0 & 0 & 0 & 0 & r_c & 0 \end{bmatrix}$$

17.1.5 Reservoir dynamics

In a temporal task, it is important to constantly update the kernels with a function taking both new input and the previous kernels. This leads to the notion of dynamical short-term memory. The state vector x is updated at every time $t \in \mathbb{T}$ using the following recursion equations:

$$\forall k \in \mathbb{K} \quad \begin{cases} \tilde{x}^k(t) = \delta^k S_{a^k}(W_{in}^k \cdot [1; u(t)] + W_{res}^k \cdot x^k(t-1) + W_{fb}^k \cdot y(t-1) + \epsilon^k) \\ \quad + (1 - \delta^k)(W_{in}^k \cdot [1; u(t)] + W_{res}^k \cdot x^k(t-1) + W_{fb}^k \cdot y(t-1) + \epsilon^k) \\ x^k(t) = (1 - \gamma^k)x^k(t-1) + \gamma^k \tilde{x}^k(t) \end{cases} \quad (17.116)$$

where γ^k is the learning rate of the k -th kernel telling how much it learns from what has happened in the previous state as new data come in. ϵ^k is a matrix noising the k th kernel and thus preventing overfitting. It is monitored by a parameter determining the standard deviation of the Gaussian distribution. a^k is the active rate which is linked to the

parametrised sigmoid function S_{a^k} . This function is applied element wise to our multidimensional factors. A plot is given in Figure (62), illustrating its behaviour as a varies.

$$S_a : x \in \mathbb{R} \rightarrow \frac{2}{1 + e^{-\frac{x}{a}}} - 1 \quad (17.117)$$

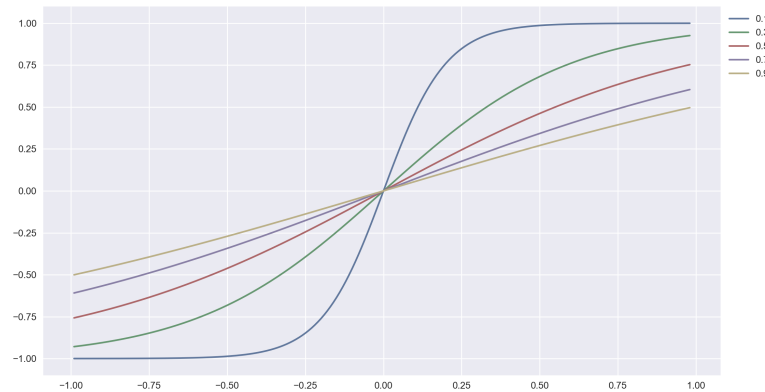


Figure 62: parametric sigmoid function

Sigmoidal functions are often used in machine learning to squash values to their limits. They are widely used in classification tasks. The reason being that this function is a way of assigning the negative value -1 or positive value $+1$ to a neuron in a smooth manner. The active rate a defines how important this squashing is. Further, δ^k monitors the importance of this squashing given the neuron's value.

For a given time t , the resulting $x(t)$ is called echoes of its input history $\{u(1) \dots u(t)\}$. We can set $\gamma = \alpha \times \Delta t$, where Δt is the time interval at which the data is coming into the system and α is a parameter. The smaller α and Δt are, the more the reservoir reacts slowly to new input. Note that from a signal processing point of view, the exponential moving average in Equations (17.116) acts as a low-pass filtering of its activations with cutoff frequency:

$$f_c = \frac{\alpha}{2\pi(1 - \alpha)\Delta t}$$

17.2 Optimisation methods

We sum up the parameter set in the following picture and table. For each parameter, we provide its location, its characteristics such as its type (micro or macro) and its RMSE variability impact.

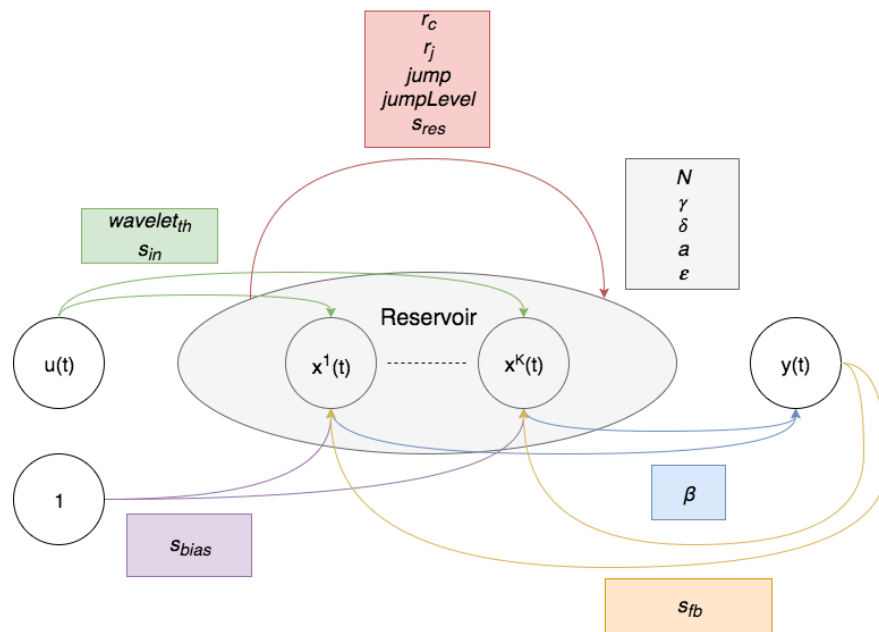


Figure 63: Parameters to be optimized.

Parameter	Type		RMSE Influence		
	Micro	Macro	Low	Med	High
r_c	×		×		
r_j	×		×		
$jump$	×		×		
$jumpLevel$	×		×		
$wavelet_{th}$		×			×
β	×		×		
N		×	×		
s_{in}	×			×	
s_{bias}	×		×		
s_{res}	×			×	
s_{fb}	×			×	
γ	×				×
δ	×		×		
a	×				×
ϵ	×		×		

Table 8: Parameter characteristics. Micro/Macro corresponds to a matrix ladder (a parameter is micro if it defines only its elements, macro if it defines its shape). Thanks to a graph of the RMSE through the optimization process, we can classify the influence of each parameter.

Note that some parameters depend on the factor's number. The number of parameters can grow very fast as we increase this number. Only β and $wavelet_{th}$ are independent of the reservoir shape. If we define m_f as the factors' number,

then we have the following number of parameters:

$$n_{param} = 13 * m_f + 2 \quad (17.118)$$

We have focused on a 2-factor reservoir in our study as it provides two high dimensional signals and at the same time keeps the optimisation time relatively low. Therefore, the number of parameters is 28. This is relatively high and we cannot rely on an exhaustive line search, as it would take too much time to run, and also the system would be over-optimised, leading to overfitting the data.

17.2.1 Random Generation: GEN

One way of solving the above problem is to consider the following functions:

- global optimisation: it generates uniformly random values inside a space *globalMax* times in a same loop and update the best parameters upon the RMSE obtained on the validation sample by running training and validation.
- local optimisation: exactly the same as the global one with *localMax* times, except that it is now restricted to a smaller space (20% of the global one).
- cross optimisation: it consists in optimizing the parameters one by one *crossMax* times and see if there is any improvement on the validation sample. It also performs two local optimisations at the end (one at 20% followed by one at 10%).
- While loop: it combines a more exhaustive global optimization (*whileMax* times) with local and cross optimisations if there is improvement in the global.

Remark 17.1 Note that at each call of any of these functions, new training and validation running are performed.

Algorithm 19 Optimisation Strategy 1: GEN

Require: get *whileMax*.

Require: set *count* $\leftarrow 0$, *best_{RMSE}* $\leftarrow \infty$.

```
1: RMSE  $\leftarrow$  global()
2: RMSE  $\leftarrow$  local()
3: RMSE  $\leftarrow$  cross()
4: while count < whileMax do
5:   p  $\leftarrow$  random()
6:   node  $\leftarrow$  Reservoir(p)
7:   prenode  $\leftarrow$  LeakyIntegrator(node)
8:   prenode.train()
9:   RMSE  $\leftarrow$  prenode.valid()
10:  if RMSE < bestRMSE then
11:    RMSE  $\leftarrow$  cross()
12:    RMSE  $\leftarrow$  local()
13:  end if
14:  count  $\leftarrow$  count + 1
15: end while
16: return RMSE
```

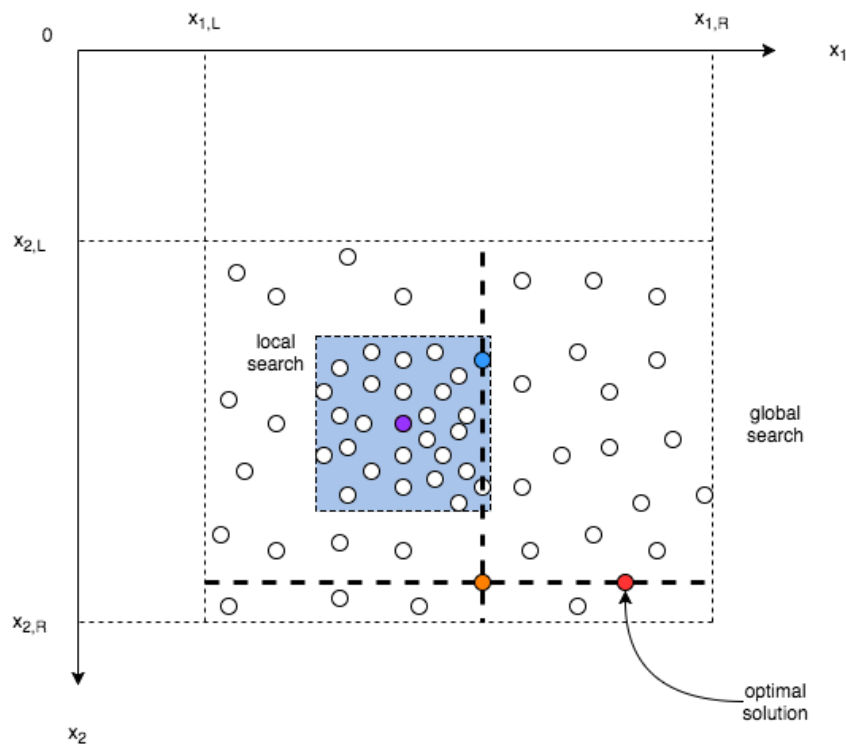


Figure 64: Random optimisation strategy with 2 parameters. The global optimisation finds an optimal point (purple). Then a local optimisation is done in the blue area where another optimum is found (blue). Then comes the cross optimisation where an optimum is firstly found along x_2 (orange) and secondly along x_1 (red). The *while* loop part is not represented here in order to lighten the figure but the reader can easily see what it consists in.

Remark 17.2 *In our experiments, we note that the while loop is often useless when the cross optimisation is long enough.*

17.2.2 Stochastic Gradient Descent: SGD

The idea of this optimisation strategy is to still perform a global search and also try to find as quickly as possible the local optimum by performing a stochastic gradient descent.

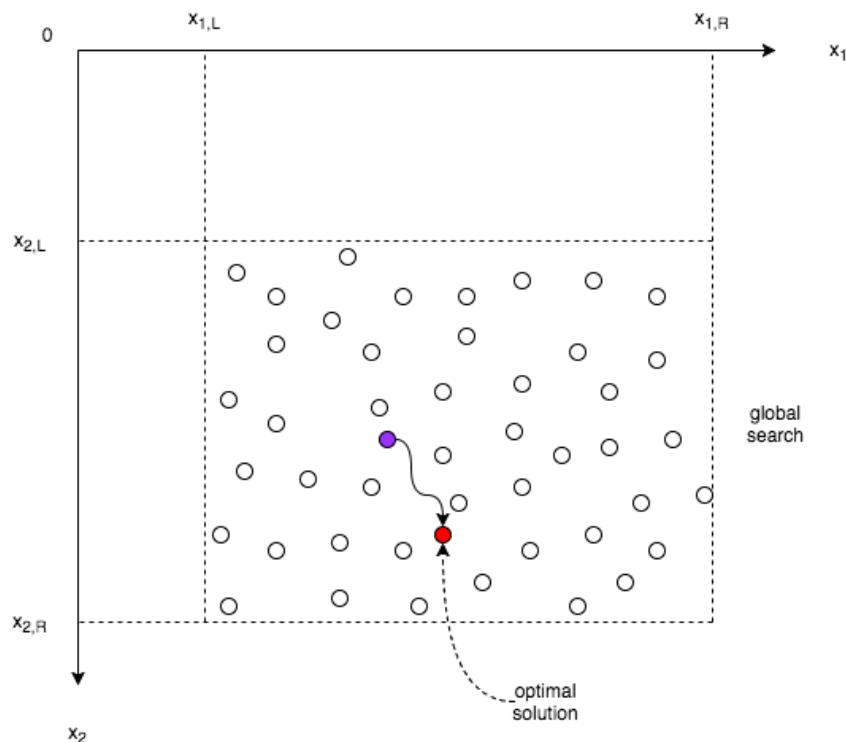


Figure 65: SGD optimisation strategy with 2 parameters. For every global generation (purple) we refine it by changing only a few parameters in order to get to a local optimum (red).

Tang et al. [2014] implemented this strategy by changing only the parameters r_c and r_j . They claimed that this method converges faster to an optimum and does not affect the accuracy. In this section, we lay out this method for the following set of parameters: r_c , r_j , δ , γ and a . The authors also define the reservoir matrix W_{res} with a cycle reservoir with jumps (CRJ). The idea of their method is to train the system with a hybrid optimisation strategy:

- Readouts: the output weights of the matrix W_{out} is trained using Equation (16.112).
- Reservoir: each parameter θ is optimised on a grid search. E is defined as in Equation (17.115).

$$\frac{\partial E}{\partial \theta} = 2 \sum_{t=t_0+1}^{t_1} (\hat{y}(t) - y(t)) W_{out} \frac{\partial x(t)}{\partial \theta} + 2 \underbrace{\sum_{t=t_0+1}^{t_1} (\hat{y}(t) - y(t)) x(t) \frac{\partial W_{out}}{\partial \theta}}_{=0} \quad (17.119)$$

where θ is the parameter we optimise. Indeed, the second term is null as we use W_{out} from Equation (16.112), which implies that $\frac{\partial E}{\partial \theta} = \sum_{t=t_0+1}^{t_1} (\hat{y}(t) - y(t)) x(t) = 0$.

Remark 17.3 Note that this fact is only true when W_{out} is constant, which is not the case in online algorithms. Thus, we will mainly use the GEN optimiser in our study in order to fairly compare the different methods.

Recall the two equations updating a factor of the reservoir at time t :

$$\begin{cases} \tilde{x}(t) = \delta S_a(W_{in} \cdot [1; u(t)] + W_{res} \cdot x(t-1) + W_{fb} \cdot y(t-1) + \epsilon) \\ \quad + (1 - \delta)(W_{in} \cdot [1; u(t)] + W_{res} \cdot x(t-1) + W_{fb} \cdot y(t-1) + \epsilon) \\ x(t) = (1 - \gamma)x(t-1) + \gamma \tilde{x}(t) \end{cases} \quad (17.120)$$

From these equations we can derive $\frac{\partial x(t)}{\partial \theta}$:

$$\begin{cases} \frac{\partial x(t)}{\partial r_c} = (1 - \gamma) \frac{\partial x(t-1)}{\partial r_c} + \gamma(1 - \delta + \frac{\delta}{a} S_a(\omega)(1 - S_a(\omega))) \odot (\frac{\partial W}{\partial r_c} x(t-1) + W \frac{\partial x(t-1)}{\partial r_c}) \\ \frac{\partial x(t)}{\partial r_j} = (1 - \gamma) \frac{\partial x(t-1)}{\partial r_j} + \gamma(1 - \delta + \frac{\delta}{a} S_a(\omega)(1 - S_a(\omega))) \odot (\frac{\partial W}{\partial r_j} x(t-1) + W \frac{\partial x(t-1)}{\partial r_j}) \\ \frac{\partial x(t)}{\partial \delta} = (1 - \gamma) \frac{\partial x(t-1)}{\partial \delta} + \gamma(S_a(\omega) - \omega) \\ \frac{\partial x(t)}{\partial \gamma} = -x(t-1) + \tilde{x}(t) \\ \frac{\partial x(t)}{\partial a} = (1 - \gamma) \frac{\partial x(t-1)}{\partial a} - \gamma \delta \frac{u}{a^2} S_a(\omega)(1 - S_a(\omega)) \end{cases} \quad (17.121)$$

where $\omega = W_{in} \cdot [1; u(t)] + W_{res} \cdot x(t-1) + W_{fb} \cdot y(t-1) + \epsilon$. Then we update parameter θ using the following equation:

$$\theta_{n+1} = \theta_n - \alpha_\theta * \frac{\partial E}{\partial \theta_n} \quad (17.122)$$

Algorithm 20 Optimisation Strategy 2: SGD

Require: get *whileMax*, *SGD_{Max}*, *V* (parameters' set).

Require: set *RMSE*, *count* $\leftarrow 0$, *whileCount* $\leftarrow 0$, *SGD_{count}* $\leftarrow 0$ **p**.

```

1: while whileCount < whileMax do
2:   p  $\leftarrow$  random()
3:   while SGDcount < SGDMax do
4:     for  $\theta$  in V do
5:        $\theta = \theta - \alpha_\theta * \frac{\partial E}{\partial \theta}$ 
6:     end for
7:     node  $\leftarrow$  Reservoir(p)
8:     prenode  $\leftarrow$  LeakyIntegrator(node)
9:     prenode.train()
10:    RMSE  $\leftarrow$  prenode.valid()
11:    SGDcount  $\leftarrow$  SGDcount + 1
12:   end while
13: end while
14: return RMSE

```

17.3 Adaptive filters for online learning

There exists several ways of defining an online training algorithm, such as gradient descent and recursive least squares (see Haykin [2000], Bishop [2006]).

17.3.1 A review of adaptive filtering theory

We consider the adaptive filtering theory and justify its ability to improve the offline model presented in the previous part. Douglas [1999] gave the following definition:

Definition 17.1 Adaptive Filter

An adaptive filter is a computational device that attempts to model the relationship between signals in real time in an iterative manner.

Later, Steil [2004] argued that an adaptive filter is able to adapt extremely fast to the main characteristics of the task in a sort of one-shot online-learning.

In general, an adaptive filter can be identified by four different features:

1. The signal being processed by the filter. For instance, it can correspond to the daily closed prices.
2. The structure/scheme/process that is used to compute the output signal from the input signal it processes. This is represented by Figure (60).
3. The parameters defined in its structure/scheme/process that can be modified in an iterative manner. The aim being to modify the input-output relationship to reduce a cost function.
4. The algorithm describing how these parameters are modified through time.

Given $\mathbb{T} = [1, N_T]$ a discrete time set, for each $t \in \mathbb{T}$, the input signal $u(t)$ is fed into the reservoir, now called adaptive filter, which computes an output signal $\hat{y}(t)$. The reservoir defines parameters that can adjust the values of $\hat{y}(t)$. At time $t + 1$ we know the real world value of the output, therefore it is natural to define the difference signal as follows:

$$\forall t \in [1, T], e(t) = y(t) - \hat{y}(t)$$

This signal, described in the diagram in Figure (66), is called the error signal. As time t is incremented, we hope that our adaptive filter learns from the environment and gets better and better at matching the response signal $y(t)$ through an adaptation process.

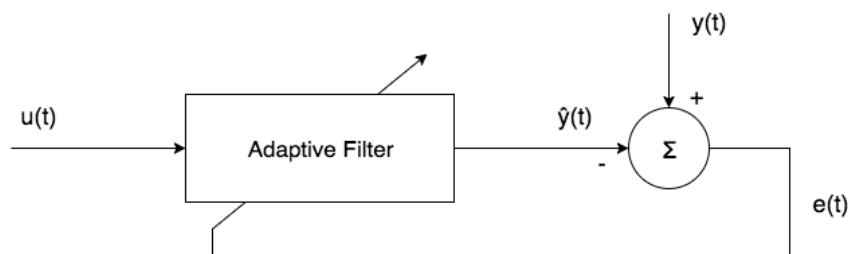


Figure 66: Scheme of the adaptive filtering purpose

There exists several types of application such as system identification, inverse modelling and linear prediction. For our task, we will use the latter and describe it in the diagram in Figure (67).

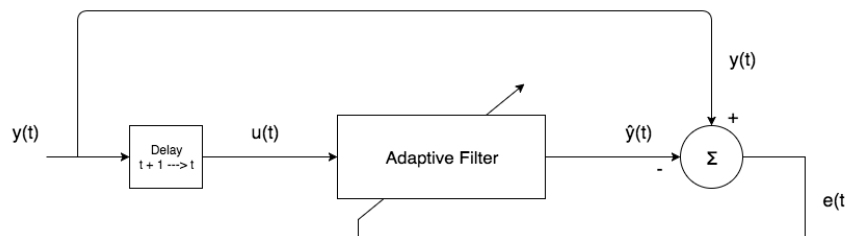


Figure 67: Linear Predictor problem

In our task, we have the following:

$$\forall t \in [1, T], u(t) = y(t - \Delta)$$

with $\Delta = 1$ when the non-filtered data are fed in the adaptive filter.

The diagram above indicates that if a certain output signal $y(t)$ is desired, we can consider it as input to compute the output of our model $\hat{y}(t)$. If the system we wish to predict is stable enough, then the next forecast should be even closer.

There are several measures to quantify the task performance such as the root mean square error (RMSE). An adaptive algorithm aims at adjusting a set of parameters of the corresponding adaptive filter in order to reduce as much as possible the RMSE. Given Definition (17.1), an adaptive signal must define some parameters which can be adapted at every point in time to achieve this goal. From RC theory, since the significant matrix in the learning process is W_{out} , we should only adapt the latter. A general equation to model an adaptive filtering algorithm in our task is as follow:

$$W_{out}(t+1) = W_{out}(t) + \mu(t)G(e(t), u(t), s(t))$$

where $G(\cdot)$ is a vector-valued non-linear function, $\mu(t)$ a parameter controlling the step size at time t . $e(t)$ and $u(t)$ are respectively the error and input signals evaluated at time t . Douglas [1999] also suggested “ $s(t)$ [as] a vector of states that store pertinent information about the characteristics of the input and error signals and/or the coefficients at previous time instants.” s stands as the reservoir states in our task. The author pointed out that the success or failure of an adaptive filtering algorithm heavily relies on the choice of the step size μ . Therefore, μ requires significant attention when optimising in the validation set.

17.3.1.1 Simple gradient descent We recall the gradient descent update equation below:

$$\omega_{i,j}(t+1) = \omega_{i,j}(t) - \mu_t \cdot \frac{\partial SE(t)}{\partial \omega_{i,j}(t)} \quad (17.123)$$

where $\omega_{i,j}(t)$ is the (i, j) th element of the output matrix W_{out} at time t . We also recall the equation computing the forecast at time t :

$$\hat{y}(t) = g(W_{out}s(t)) \quad (17.124)$$

where $g \in C^1(\mathbb{R})$ and applies to a vector in an element-wise manner and $\forall t \in \mathbb{T} \quad s(t) = [1 \| u(t) \| x(t)]$.

We derive $\frac{\partial SE(t)}{\partial \omega_{i,j}(t)}$ below using the framework defined in Appendix (15.1.4).

$$SE(t) = \sum_{i=1}^N (y_i(t) - \hat{y}_i(t))^2 \quad (17.125)$$

$$\Rightarrow \frac{\partial SE(t)}{\partial \omega_{i,j}(t)} = -2 \sum_{i=1}^N (y_i(t) - \hat{y}_i(t)) \frac{\partial \hat{y}_i(t)}{\partial \omega_{i,j}(t)} \quad (17.126)$$

$$\Rightarrow \frac{\partial SE(t)}{\partial \omega_{i,j}(t)} = -2 \sum_{i=1}^N (y_i(t) - \hat{y}_i(t)) g'(W_{out}s(t)) s_j(t) \quad (17.127)$$

17.3.1.2 Recursive least squares The Recursive Least Squares (RLS) algorithm can be used for iterative computation of the inverse of the correlation matrix R_{ij}^{-1} and the readout weights ω (see Appendix (16.6.1)). It can be summarised as follows:

$$\begin{aligned}
\kappa(t) &= \frac{\nu^{-1}\Gamma(t-1)x(t)}{1 + \nu^{-1}x^\top(t)\Gamma(t-1)x(t)} \\
\omega(t) &= \omega(t-1) + \kappa(t)(y(t) - \hat{y}(t)) \\
\Gamma(t) &= \nu^{-1}\Gamma(t-1) - \nu^{-1}\kappa(t)x^\top(t)\Gamma(t-1)
\end{aligned}$$

where $\kappa(t)$ is the gain vector and $\Gamma(t)$ is the estimate of the inverse of the correlation matrix. The forgetting factor ν is set to 1. This method ensures faster convergence of the readout weights but is much more computationally intensive than the simple gradient descent algorithm.

17.3.2 Online algorithm

We are now going to describe the online algorithm for real time learning of output matrix W_{out} . While we have considered the actual output of the financial time series y , it is more relevant to take the filtered signal as the target signal. Indeed, this signal is by definition smoother and therefore more continuous in time.

Algorithm 21 RC online Training

Require: choose a gradient descent algorithm $\mathbb{A}$

- 1: $W_{out} \leftarrow \text{Algorithm (18)}$
 - 2: set $SE \leftarrow 0$
 - 3: **for** $t \leftarrow 0$ **to** $T_{valid} - 1$ **do**
 - 4: update each reservoir factor using Equation (17.116).
 - 5: compute forecast using Equation (17.124).
 - 6: $SE \leftarrow SE + (y(t+1) - \hat{y}(t+1))^2$
 - 7: update W_{out} with Equation (17.123) using $\mathbb{A}$.
 - 8: **end for**
-

In Algorithm (21) we use $\mathbb{A}$ as the scaled conjugate gradient implemented in GNU Scientific Library [2017]. This method is very fast for a well-chosen initial guess and it makes sense to suppose that the output matrix should be close to its previous state as the signal we try to calibrate is smooth.

Remark 17.4 We can also replace y in Equation (17.116) with its last feedback $\hat{y}$. Consequently, the reservoir has more information about how accurate its last forecast was.

17.4 Associative Reservoir Computing

17.4.1 Paradigm

17.4.1.1 Definitions So far, we have taught the reservoir the input to output mapping. Equation (17.128) introduce the function ϕ defining this mapping and carrying the forward relation between the input sample U and the output sample Y without studying its properties. We are now going to understand the function ϕ in view of making better prediction.

Definition 17.2 *Ambiguous relation*

Let $\phi : U \rightarrow Y$. We say that ϕ is ambiguous if it is not injective i.e. $\exists (u_1, u_2) \in U^2 | u_1 \neq u_2 \Rightarrow \phi(u_1) = \phi(u_2)$

Definition 17.3 *State space*

A state space is the set of all possible states [the reservoir states x in our case] of a dynamical system. Each state of the system corresponds to a unique point in the state space (see Terman et al. [2008]).

Definition 17.4 *Attracting set*

Let S be a dynamical system and X its associated state space. We say of a set A to be an attracting set of S if it is a closed subset of X such that $\forall a \in A, \exists x(0) = x_0 | x(t) \rightarrow a$ as $t \rightarrow +\infty$.

We call its elements attractors.

Definition 17.5 *Basin of attraction*

Let a be an attractor of A . We define the basin of attraction of A $B(A)$ as the set of initial conditions such that $x(t) \rightarrow a$ as $t \rightarrow +\infty$.

In associative reservoir computing, the learning process aims at shaping the reservoir dynamics in a particular attractor. A major problem when learning ϕ is that it may be an ambiguous relation. Following Reinhart [2011], we develop a method for resolving the ambiguity of ϕ using a dynamical approach.

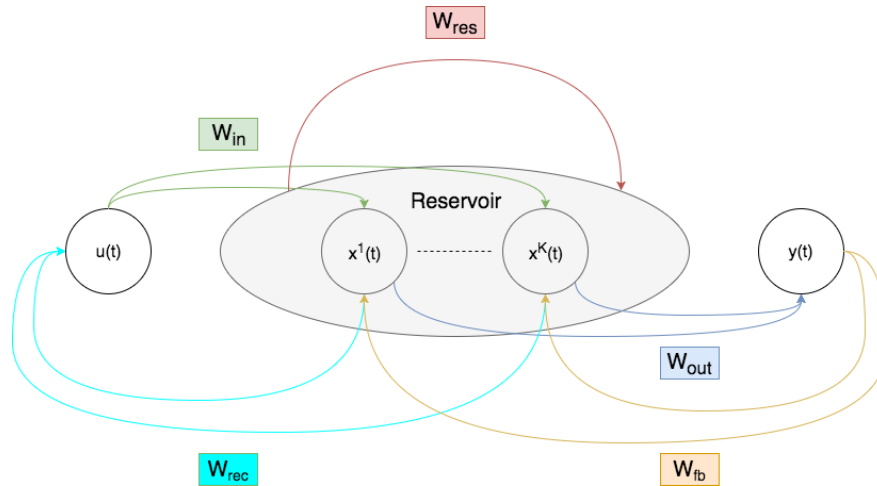
17.4.1.2 The model In the ARC model, we overwrite the reservoir forward representation by a joint representation involving input u and output y . To access the latter, we need a matrix $W_{fb} \neq 0$. Furthermore, we define a matrix mapping the reservoir states to the input layer and called it $W_{rec} (\in \mathbb{R}^{N_U \times N_S})$. We want to learn the mapping ψ from output to input. Figure (68) defines the different connections involved in this model. We have the following system:

$$\left\{ \begin{array}{l} \phi : T \times U \times X \rightarrow Y \\ \phi : t, u, x_{fwd} \rightarrow \hat{y} = g(W_{out}[1 \| u(t) \| x_{fwd}(t)]) \\ \psi : T \times Y \times X \rightarrow U \\ \psi : t, y, x_{bwd} \rightarrow \hat{u} = g(W_{rec}[1 \| y(t) \| x_{bwd}(t)]) \\ x_{fwd} : T \times U \times X \times Y \rightarrow X \\ x_{fwd} : t, u, x, y \rightarrow x_{fwd}(t) = f(W_{in}u(t) + W_{res}x(t-1) + W_{fb}\hat{y}(t-1)) \\ x_{bwd} : T \times U \times X \times Y \rightarrow X \\ x_{bwd} : t, u, x, y \rightarrow x_{bwd}(t) = f(W_{in}\hat{u}(t-1) + W_{res}x(t-1) + W_{fb}y(t)) \end{array} \right. \quad (17.128)$$

Equations (17.128) above define the associative system. The system is now composed of two functions ϕ and ψ mapping respectively input to output and output to input. We have also defined two states x_{fwd} and x_{bwd} corresponding respectively to the states obtained with forward model ϕ and backward model ψ .

Remark 17.5 In practice the backward model cannot be use in our task (at least on the testing sample) as we obviously do not know the asset price we want to predict. However this framework demonstrates that we can compute its corresponding input $\hat{u}$ by simply using the last prediction $\hat{y}(t-1)$.

Figure 68: ARC model.



Typically, it is often recommended to use the output at time t when learning on the training sample, we call this process teacher forcing. On the validation set, we can keep training the W_{out} and W_{rec} matrices in order to teach the reservoir how to stay close to the target signal. That is to say, we use equation 17.123 to update W_{out} and W_{rec} , the idea being to stay close to an attractor as time goes on. In the next subsection we lay out our algorithm for converging to an attractor at every time.

17.4.1.3 Attractor-based computation In the Figure (69) below, the system sits at time t and iterates the reservoir dynamics for a certain number of time, k_{max} , using always the same input $u(t)$. This artificial dynamics causes the reservoir $x(t)$ to evolve toward a stable state. A good forecast of the output attractor can be performed using Equations (17.128) afterwards.

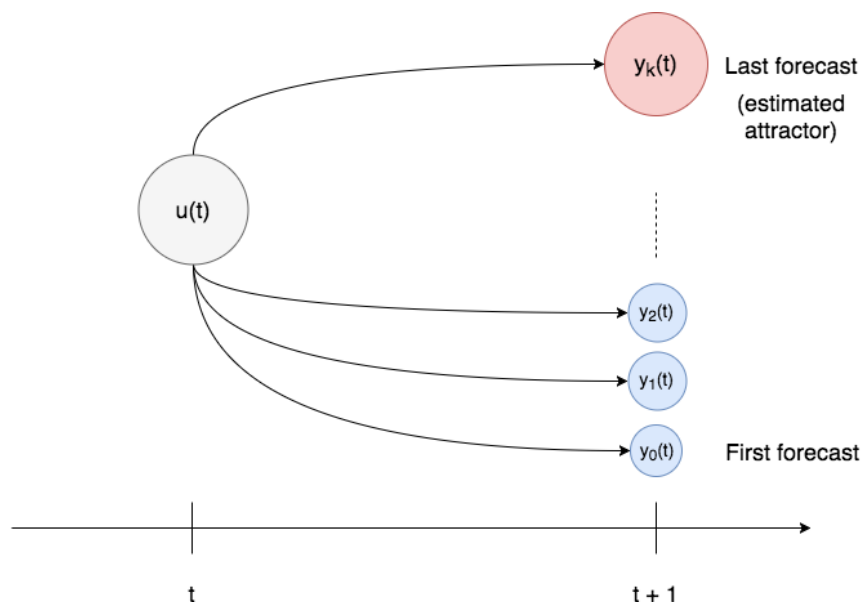


Figure 69: Dynamical system model to fit an attractor.

As explained by Reinhart [2011], the dynamical approach to model ambiguous inverse problems crucially depends on the ability to shape multi-stable attractor dynamics that reflect the multiplicity of solutions.

Remark 17.6 *The attractors we try to fit are the different filters we may use to denoise the initial signal. For instance, it can correspond to several wavelets, decomposition and thresholding levels.*

Algorithm 22 Attractor Convergence Algorithm

Require: get an input data at time $u(t)$.

Require: set k_{max} as a maximum number of iteration.

Require: set ϵ as a tolerance level.

```

1: while  $k < k_{max}$  and  $norm < \epsilon$  do
2:   compute  $x_{fwd}(t)$  using Equation ( 17.128).
3:   compute  $\hat{y}(t)$  and  $\hat{u}(t)$  using Equation ( 17.128).
4:    $norm \leftarrow ||x_{fwd}(k+1) - x_{fwd}(k)||^2$ 
5:    $k \leftarrow k+1$ 
6: end while

```

In Algorithm (22) we eliminate the transient effect by iterating the states using the same input. It is as if we would have frozen the series at time t and went on feeding the reservoir, leading to convergence to an attractor. Basically, we want to fit the reservoir dynamics to this attractor in order to teach the reservoir how to reproduce its behaviour. If it is the case, then the network should not need this trick any more and it would predict well this attractor. Ambiguity can be resolved by applying Algorithm (22), causing the reservoir to settle in a particular attractor.

17.4.2 States stabilisation techniques

A major issue of our task is that only noised data are observable in the market. However, the reservoir states need to be smooth (in the sense of its norm) in order to hope learning. Indeed, if the states are stables as time goes, then the predicted values should be stables as well. From Table (2) we have seen that there is a good learning when dealing with filtered input. However we cannot use the wavelet filter in the testing sample. To offset this problem, we could imagine teaching the reservoir how to remain consistently stable while feeding it with noisy input.

17.4.2.1 Reservoir dynamics calibration to observable stable states Suppose that we can observe stable states x^* on the training sample. We wish to find a dynamics $W_{opt} = [W_{in} \| W_{res} \| W_{fb}]$ such that it minimises the following cost function:

$$E = \sum_{t=1}^{T_{training}} ||x^*(t) - x(t)||^2 \quad (17.129)$$

Given u , x^* and y on the training sample, the problem is stated as follows:

$$\min_{W_{opt}} \sum_{t=1}^{T_{training}-1} ||x^*(t+1) - f(W_{opt}[u(t) \| x^*(t) \| y(t)])||^2 \quad (17.130)$$

Let us define

$$S^* = [[u(0) \| x^*(0) \| y(0)] \dots [u(T_{training}-1) \| x^*(T_{training}-1) \| y(T_{training}-1)]]$$

and

$$A^* = [f^{-1}(x^*(1)) \parallel \dots \parallel f^{-1}(x^*(T_{training}))]$$

Then we can solve this problem using Ridge regression:

$$\hat{W}_{opt} = A^* \cdot S^{*T} \cdot (S^* \cdot S^{*T} + \lambda I)^{-1} \quad (17.131)$$

where λ is a well chosen regularization parameter. Given these calibrated matrices, the reservoir can now learn from noisy input while staying relatively stable.

Remark 17.7 *The filter is of key importance as it will define how the system can map noisy input to smooth states.*

In practice we want the reservoir to remain stable when fed with unseen noisy input. Thus, we must consider a large enough training sample to ensure the states remains stable for exploitation.

17.4.2.2 Learning with filtered inputs Another idea would be to work with filtered inputs only and to directly teach the reservoir to predict the filtered data (coloured in green). Indeed, this filter can be seen as an indicator of the initial dataset (see Technical Indicators in Appendix (11)). One could then argue that getting a good prediction of the filtered data could lead to good real time series forecasting if one can control the forecasting error.

Suppose that we observe stable states x^* on the training and validation samples. In these samples we can use all the information at any time, so that we can fit a filtering technique such as wavelets and feed the reservoir using the filtered inputs only. The states should then be relatively stable on the training and validation, as previously discussed. Then, using Equation (17.123) (for both W_{out} and W_{res}), we teach the network to fit this filtered dynamics. At the end of the validation process, we hope that our network will be well enough trained and that we no-longer need Equation (17.123) to obtain good forecasts. That is, we should have the functions ϕ and ψ relatively constant with respect to the parameters ($W_{in}, W_{res}, W_{fb}, W_{out}, W_{rec}$).

We present in Figure (70) our risk management scheme when making a forecast. The idea being to use, under particular assumptions, the information at time t with the backward mapping function ψ , in order to have a control over the forecast error.

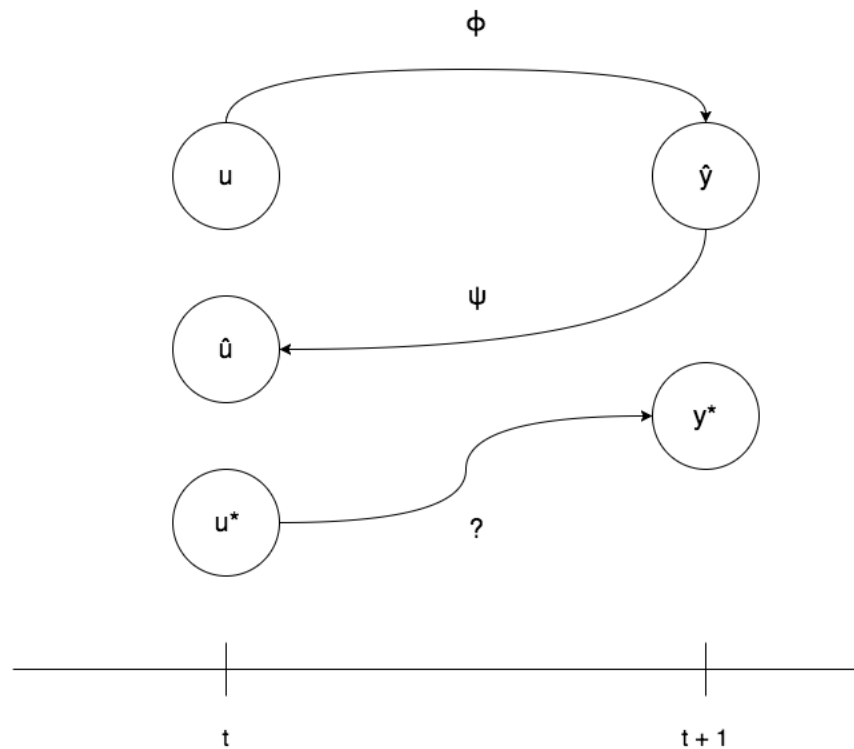


Figure 70: Risk management scheme.

Suppose that $\exists t_0 \in [0, T_{valid}]$ such that $\forall t \in \mathbb{T}_0 := [t_0, T_{valid}]$, $\phi(u(t)) = \hat{y}(t)$ and $\psi(\hat{y}(t)) = u(t)$. Then $\psi|_{\mathbb{T}_0}^{-1} = \phi|_{\mathbb{T}_0}$ (injective property is verified at time t if Algorithm (22) converges). Suppose also that $\phi|_{\mathbb{T}_0}$ is time-independent and that $\forall t \in \mathbb{T}_0$, $|u^*(t) - \hat{u}(t)| \leq K$. Then, we could set up the following strategy:

Algorithm 23 Out-of-sample forecasting risk management

Require: set $l_V = \sup\{|u^*(t) - \hat{u}(t)|, \forall t \in \mathbb{T}_0\}$

Require: set $l_T \leftarrow 0$

1: **while** $l_T < l_V$ **do**

2: update the reservoir x up to time t using Equations (16.110) and (16.111).

3: compute forecast $\hat{y}(t)$ and associative input $\hat{u}(t)$ using Equation (17.124).

4: $l_T \leftarrow |u^*(t) - \hat{u}(t)|$

5: **end while**

The ARC framework is very powerful as it allows us to evaluate the input error at time t to have an idea of the forecast accuracy at time $t + 1$.

18 SVM models

In machine learning, support vector machines (SVMs) are supervised learning models with associated learning algorithms that analyse data used for classification (see Vapnik [1995-2000]) and regression analysis (see Drucker et al. [1996]).

Vapnik et al. [1963] introduced the Generalized Portrait algorithm which was further developed by Vapnik et al.

[1964]. The algorithm implemented by support vector machines is a nonlinear generalisation of the Generalized Portrait algorithm. SVM was extended by Boser et al. [1992] who suggested a way to create nonlinear classifiers by applying the kernel trick to maximum-margin hyperplanes. Cortes et al. [1995] introduced the soft margin classifier. In the same year the algorithm was extended to the case of regression by Vapnik [1995-2000].

SVM is a learning system using a high dimensional feature space where points are classified by means of assigning them to one of the two disjoint half spaces, either in the pattern space or in a higher dimensional feature space. The main objective is to identify maximum margin hyper plane, that is, the margin of separation between positive and negative examples is maximised. SVM finds the maximum margin hyper plane as the final decision boundary.

18.1 An optimisation problem

Assume that $x_i \in \mathbb{R}^d$, $i = 1, \dots, N$ forms a set of input vectors with corresponding class labels (binary variables) $y \in \{+1, -1\}$, $i = 1, \dots, N$. We want to find the maximum margin hyperplane that divides the group of points x_i for which $y_i = 1$ from the group of points for which $y_i = -1$, which is defined so that the distance between the hyperplane and the nearest point x_i from either group is maximised.

Any hyperplane can be written as the set of points x satisfying

$$w \cdot x - b = 0$$

where w is the normal vector to the hyperplane. The parameter

$$\frac{b}{\|w\|} \tag{18.132}$$

determines the offset of the hyperplane from the origin along the normal vector w . If the training data is linearly separable, we can select two parallel hyperplanes that separate the two classes of data, so that the distance between them is as large as possible. The region bounded by these two hyperplanes is called the margin, and the maximum margin hyperplane is the hyperplane lying halfway between them. A linear classifier can be considered as a hyperplane with normal vector $w \in \mathbb{R}^d$ and offset b . The classification of x_i is determined by the hyperplane:

$$\hat{y}_i = \text{sgn}(w \cdot x_i + b)$$

where

$$\text{sgn}(x) = \begin{cases} -1 & \text{if } x < 0 \\ 0 & \text{if } x = 0 \\ 1 & \text{if } x > 0 \end{cases}$$

To prevent the data points from falling into the margin, we need to impose the following constraints: For each i either

$$w \cdot x_i - b \geq 1 \text{ if } y_i = 1$$

or

$$w \cdot x_i - b \leq -1 \text{ if } y_i = -1$$

That is, each data point must lie on the correct side of the margin. It can be rewritten as

$$y_i(w \cdot x_i - b) \geq 1, \forall 1 \leq i \leq N$$

An illustration is given in Figure (71).

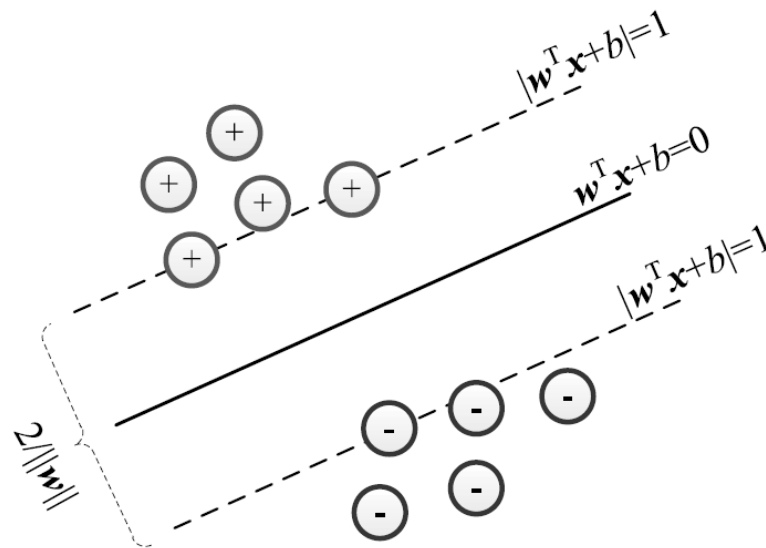


Figure 71: Illustration of SVM

The objective of a linear classifier is to find the hyperplane that optimally separates the positive samples from negative ones. This is done by maximising the margin (gain) between the two regions. This as an optimisation problem where we maximise the margin M :

$$\max_{v, M} M$$

over the normal vector v parametrising the decision boundary such that $\|v\| = 1$ and $y \cdot v^T x_i \geq M$. In that setting, the max-margin hyperplane is completely determined by those x_i which lie nearest to it. Hence, these x_i are called support vector.

We let $w = v \cdot M$ and make the substitution

$$\max_{w, M} M$$

such that $\|w\| = \frac{1}{M}$ and $\forall i : y_i \cdot w^T x_i \geq 1$.

Given the inverse relationship between the norm $\|w\|$ and the margin M , maximising the distance between the planes is equivalent to minimising $\|w\|$. We can then transform this ratio into a minimisation problem

$$\min_w \|w\|^2$$

such that $\forall i : y_i \cdot w^T x_i \geq 1$. It is a convex optimisation problem with constraints. However, this method only works if the data is linearly separable. Thus, to deal with non-separable data we need to introduce slack variables into each constraint: $\forall i : y_i \cdot w^T x_i \geq 1 - \epsilon_i$, getting soft-margin SVM. To avoid the domination of the slack variables we introduce a penalty factor to the objective function, getting

$$\min_{w, \epsilon} \|w\|^2 + C \sum_i \epsilon_i$$

such that $\forall i : y_i \cdot w^T x_i \geq 1 - \epsilon_i$ and $\epsilon_i \geq 0$ and $\lambda = \frac{1}{C}$. This is the primal offline SVM. It can be formulated with a new objective function to minimise the hinge loss as follows:

$$\min_w \|w\|^2 + C \sum_i \max(0, 1 - y_i w^\top x_i) \quad (18.133)$$

where y_i is the i th target and $w \cdot x$ is the current output. It is a trade off where the first term keep the weights small, while the second term is a sum of hinge loss functions, which are high for a poor fit. Thus, computing the (soft-margin) SVM classifier amounts to maximising the margins, or, minimising the empirical risk function

$$f(w, b) = \left[\frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i(w \cdot x_i - b)) \right] + \lambda \|w\|^2 \quad (18.134)$$

with hinge loss function

$$L(w; x_i, y_i) = \max(0, 1 - y_i(w \cdot x_i - b))$$

The parameter λ determines the tradeoff between increasing the margin size and ensuring the the x_i lie on the correct side of the margin. So, for sufficiently small values of λ , the second term of the equation becomes negligible, and we recover the hard-margin SVM, if the data is linearly separable.

Note, the SVP problem is equivalent to empirical risk minimisation (ERM) with Tikhonov regularisation, where the loss function is the hinge loss $L(x, y) = \max(0, xy)$. Thus, SVM is closely related to other fundamental classification algorithms, where the main difference is in the choice of the loss function.

18.2 The algorithms

In learning, we want to minimise that objective function. Most of the algorithms are iterative in nature and will hopefully converge to the optimal value. In order to find the break-even point between the algorithms we want to focus on the generalisation error, that is, the expected error on test set.

We can use online convex programming to optimise that problem, getting the Online SVM. Alternatively, we can consider offline algorithm and apply the Lagrange multiplier or the kernel trick.

In the former, from the representation property, at optimal solution, the weight vector w is given by $w = \sum_i \alpha_i y_i x_i$. In the latter, SVM can map the input vectors $x_i \in \mathbb{R}^d$ into a high dimensional feature space $\Phi(x_i) \in H$. A kernel function $k(x_i, x_j)$ performs the mapping $\phi(\cdot)$, that is, $k(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$. The value w , in the transformed space, is given by $w = \sum_{i=1}^N \alpha_i y_i \phi(x_i)$. Dot products with w for classification can again be computed by the kernel trick as

$$w \cdot \phi(x) = \sum_{i=1}^N \alpha_i y_i k(x, x_i)$$

We are now present more formally some online and offline algorithms to solve the optimisation problem in Equation (18.133).

18.2.1 Online algorithms

18.2.1.1 A simple algorithm We can devise a simple online algorithm based on the gradient gradient descent. Given the feasible set of the hyperplane $\mathcal{W} = \{w : \|w\|_2 \leq \lambda\}$ and the hinge loss function we get the online algorithm. See Algorithm (24) with learning rate $\eta_i = \frac{1}{\sqrt{i}}$ or $\eta_i = \frac{1}{i}$.

Algorithm 24 SVM Online Algorithm

Require: : Import input vectors and corresponding labels.

- 1: Pick $w_1 \in \mathcal{W}$ arbitrarily.
 - 2: **for** $i = 1$ to N **do**
 - 3: The incurred loss is $L(w_i) \equiv \text{hinge}(w_i; x_i, y_i)$
 - 4: Step forward on the gradient direction: $\hat{w}_{i+1} = w_i - \eta_i \nabla L(w_i)$
 - 5: Project $\hat{w}_{i+1}$ back to the feasible set: $w_{i+1} = \text{Proj}_{\mathcal{W}}(\hat{w}_{i+1})$
 - 6: **end for**
 - 7: **Return** w
-

Since the loss function is not differentiable we can use a sub-gradient to compute the gradient (see discussion below). We now briefly describe the projection. Given the feasible set $\mathcal{W}$, the projection from $\hat{w} \notin \mathcal{W}$ to its nearest point in $\mathcal{W}$ can be performed by multiplying $\hat{w}$ with a scalar

$$w_{i+1} = \text{Proj}(\hat{w}_{i+1}) = \frac{\lambda \cdot \hat{w}_{i+1}}{\|\hat{w}_{i+1}\|}$$

This is a direct consequence from the parameter determining the offset hyperplane in Equation (18.132). In the case where $\hat{w} \in \mathcal{W}$, then $\text{Proj}(\hat{w}) = \hat{w}$.

18.2.1.2 More complex algorithms Recent algorithms for finding the SVM classifier include sub-gradient descent (see Shalev-Shwartz et al. [2011]) and coordinate descent (see Hsieh et al. [2008]). Both techniques have proven to offer significant advantages over the traditional approach when dealing with large, sparse datasets. Sub-gradient methods are especially efficient when there are many training examples, and coordinate descent when the dimension of the feature space is high.

Sub-gradient descent algorithms for the SVM work directly with the expression $f(w, b)$ in Equation (18.134). Since f is a convex function, traditional gradient descent (or SGD) methods can be adapted, where instead of taking a step in the direction of the function's gradient, a step is taken in the direction of a vector selected from the function's sub-gradient.

The coordinate descent algorithms apply to the dual problem. The coefficient α_i , $i = 1, \dots, N$, is adjusted in the direction of $\frac{\partial L}{\partial \alpha_i}$. The resulting vector of coefficients $(\alpha'_1, \dots, \alpha'_N)$ is projected onto the nearest vector of coefficients that satisfies some Euclidean distances. The process is repeated until a near-optimal vector is obtained.

18.2.1.3 Parallel algorithm Zinkevich et al. [2009] proposed a parallel algorithm for Online SVM by introducing delayed updates. At the i th step the fetched gradient $\nabla L(w_{i-\tau})$ is the result at the τ -th previous step. They proved convergence. See Algorithm (25).

Algorithm 25 SVM Parallel Online Algorithm

Require: : Import input vectors and corresponding labels.

- 1: Pick $w_1 \in \mathcal{W}$ arbitrarily.
 - 2: **for** $i = 1$ to N **do**
 - 3: The incurred loss is $L(w_i) \equiv \text{hinge}(w_i; x_i, y_i)$
 - 4: Step forward on the gradient direction: $\hat{w}_{i+1} = w_i - \eta_i \nabla L(w_{i-\tau})$
 - 5: Project $\hat{w}_{i+1}$ back to the feasible set: $w_{i+1} = \text{Proj}_{\mathcal{W}}(\hat{w}_{i+1})$
 - 6: **end for**
 - 7: **Return** w
-

18.2.2 Offline algorithms

18.2.2.1 The Lagrange multiplier Recall, the primal is a constrained optimisation problem with a differentiable objective function. By solving for the Lagrangian dual of the above problem, one obtains the simplified (unconstrained) optimisation problem called the dual problem. Since minimising $\|w\|^2$ is equivalent to minimising $\frac{1}{2}\|w\|^2$ we can introduce the Lagrange multipliers α_i . We can then write the objective function given in Equation (18.133) as

$$L(w, \alpha) = \frac{1}{2}\|w\|^2 - \sum_i^N \alpha_i (y_i w^\top x_i - 1)$$

where the new objective is

$$\min_w \max_\alpha L(w, \alpha)$$

We can then compute the KKT (Karush-Kuhn-Tucker) conditions for differentiable convex programs. Applying the KKT theorem to the SVM optimisation problem we get the dual problem:

1. $\frac{\partial}{\partial w} L(w, \alpha) = 0 \rightarrow w = \sum_i \alpha_i y_i x_i$
2. $y_i w^\top x_i - 1 \geq 0$
3. $\sum_i (y_i w^\top x_i - 1) = 0$

From these conditions, we can see that

1. w can be represented as a linear combination of data points.
2. All data points are at least a normalised distance of 1 from the separating hyperplane.
3. As $\alpha_i \geq 0$ either $\alpha_i = 0$ or $y_i w^\top x_i = 1$ for all i .

Thus, the points for which $\alpha_i > 0$ are supporting the hyperplane, and as such are called support vectors. Hence, we write the set of support vectors as

$$\mathcal{S} = \{x_i : y_i w^\top x_i = 1\}$$

We can then substitute $w = \sum_i \alpha_i y_i x_i$ into the Lagrangian and get a simplified objective function

$$\begin{aligned} L(\alpha) &= \frac{1}{2} \left(\sum_i \alpha_i y_i x_i \right) \left(\sum_j \alpha_j y_j x_j \right) - \sum_i \alpha_i \left[y_i \left(\sum_j \alpha_j y_j x_j \right)^\top x_i - 1 \right] \\ &= \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^\top x_j - \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^\top x_j + \sum_i \alpha_i \\ &= \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^\top x_j \end{aligned} \tag{18.135}$$

subject to $\sum_{i=1}^N \alpha_i y_i = 0$.

If we have α , we can then solve for w , and only need to solve for

$$\alpha^* = \arg \max_\alpha L(\alpha)$$

such that $\alpha_i \geq 0, \forall i$.

Remark 18.1 The objective function only depends on inner products $x_i^\top x_j$, which is very useful for nonlinear classification tasks.

This inner product can be replaced by a kernel function $k(x_i, x_j)$ which takes the inner product to a higher dimensional space. This is the basis of the kernel trick discussed below.

18.2.2.2 The kernel trick We can also apply the kernel trick. In some cases, for the data to be linearly separable we first need to apply a nonlinear transformation to a higher dimensional space. For instance, suppose that we would like to learn a nonlinear classification rule which corresponds to a linear classification rule for the transformed data points $\phi(x_i)$. However, when the dimension of $\phi(\cdot)$ becomes very large we should not use an explicit transformation. Note, if $\phi(x)$ is all ordered monomials of degree d , then $\phi^\top(x)\phi(x') = (x^\top x')^d$ and we can implicitly work in a higher dimensional space by using a different dot product. This is called the kernel trick and

$$\phi^\top(x)\phi(x') = k(x, x') \quad (18.136)$$

is a kernel function.

As a result of the representation property (see the dual problem above), at optimal solution, the weight vector w is a linear combination of the data points given by

$$w = \sum_i \alpha_i y_i x_i \quad (18.137)$$

Suppose we are given a kernel function defined as above. Then, the classification vector w , in the transformed space, satisfies

$$w = \sum_{i=1}^N \alpha_i y_i \phi(x_i)$$

Note that w can be an infinite dimensional vector (a function). We can treat the problem as a parameter estimation problem.

From Equation (18.136), the resulting decision boundary is

$$f(x) = \text{sgn}(w \cdot \phi(x) + b) = \text{sgn}\left(\sum_{i=1}^N y_i \alpha_i \cdot k(x, x_i) + b\right)$$

Given the dual problem in Equation (18.135), in order to estimate the values α_i , $i = 1, \dots, N$ we solve the following quadratic programming problem

$$\begin{aligned} \max_{\alpha_i} & \left(\sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j \cdot y_i y_j \cdot k(x_i, x_j) \right) \\ \text{subject to} & \quad 0 \leq \alpha_i \leq c \\ & \quad \sum_{i=1}^N y_i \alpha_i = 0, \quad i = 1, \dots, N \end{aligned}$$

where the parameter c controls the tradeoff between the margin and misclassification error. In general it is set to $\frac{1}{2N\lambda}$. We can find some index i such that $0 < \alpha_i < c$ so that $\phi(x_i)$ lies on the boundary of the margin in the transformed space, and then solve

$$\begin{aligned}
b = w \cdot \phi(x_i) - y_i &= \left(\sum_{k=1}^N \alpha_k y_k \phi(x_k) \cdot \phi(x_i) \right) - y_i \\
&= \left(\sum_{k=1}^N \alpha_k y_k k(x_k, x_i) \right) - y_i
\end{aligned}$$

Then, new points can be classified by computing

$$z \rightarrow \text{sgn}(w \cdot \phi(z) - b) = \text{sgn}\left(\left(\sum_{i=1}^N \alpha_i y_i k(x_i, z)\right) - b\right)$$

We can also apply the same method to the primal optimisation problem. Replacing the weight vector w given in Equation (18.137) in the primal in Equation (18.133), the optimisation problem becomes equivalent to

$$\min_{\alpha} \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^{\top} x_j + C \sum_i \max(0, 1 - y_i \sum_j \alpha_j y_j x_j^{\top} x_i)$$

Assuming that $w \in \text{Span}\{x_i, \forall i\}$, we replace $x_i^{\top} x_j$ with $k(x_i, x_j)$. We then let $\tilde{\alpha}_i = \alpha_i y_i$ (since we are not constrained we can flip signs arbitrarily) and obtain the objective function

$$\min_{\tilde{\alpha}} \frac{1}{2} \sum_{i,j} \tilde{\alpha}_i \tilde{\alpha}_j k(x_i, x_j) + C \sum_i \max(0, 1 - y_i \sum_j \tilde{\alpha}_j k(x_i, x_j))$$

Using matrix notation, we rewrite the optimisation problem as

$$\min_{\tilde{\alpha}} \frac{1}{2} \tilde{\alpha}^{\top} K \tilde{\alpha} + C \sum_i \max(0, 1 - y_i f(x_i))$$

where K is the Gram matrix, and $f(x) = f_{\alpha}(x) = \sum_j \tilde{\alpha}_j k(x, x_j)$.

Thus, in the case of polynomial or radial basis function, the parameters of the SVM model are (d, γ, c) . In general, several levels are considered to determine these parameters efficiently.

19 Probabilistic networks

19.1 The generative and discriminative approaches

In statistical classification, including machine learning, the two main approaches are called

1. the generative approach, and
2. the discriminative approach.

These methods compute classifiers by using different approaches, differing in the degree of statistical modelling. In both cases, the notion of probabilities is introduced. Jebara [2004] distinguished three major types:

1. Given an observable variable X and a target variable Y , a generative model is a statistical model of the joint probability distribution on X and Y , denoted $P(X, Y)$.
2. A discriminative model is a model of the conditional probability of the target Y , given an observation x , symbolically, $P(Y|X = x)$.

3. Classifiers computed without using a probability model are also referred to loosely as discriminative.

These three classes are referred to as generative learning, conditional learning, and discriminative learning. However, Ng et al. [2002] only distinguished two classes, calling them generative classifiers (joint distribution) and discriminative classifiers (conditional distribution or no distribution), not distinguishing between the latter two classes. Analogously, a classifier based on a generative model is a generative classifier, while a classifier based on a discriminative model is a discriminative classifier, though this term also refers to classifiers that are not based on a model. Standard examples of each, all of which are linear classifiers, are: generative classifiers: naive Bayes classifier and linear discriminant analysis; discriminative model: logistic regression; non-model classifier: perceptron and support vector machine.

When classifying data, we want to go from an observation x to a label y (or probability distribution on labels). We can compute this directly, without using a probability distribution (distribution-free classifier), or we can estimate the probability of a label given an observation, $P(Y|X = x)$ (discriminative model), and base classification on that, or we can estimate the joint distribution $P(X, Y)$ (generative model), from that compute the conditional probability $P(Y|X = x)$, and then base classification on that. These methods are increasingly indirect, but increasingly probabilistic, allowing more domain knowledge and probability theory to be applied. In practice different approaches are used, depending on the particular problem, and hybrids can combine strengths of multiple approaches.

More generally, a generative model is a model of the conditional probability of the observable X , given a target y , symbolically, $P(X|Y = y)$, while a discriminative model is a model of the conditional probability of the target Y , given an observation x , symbolically, $P(Y|X = x)$. Regardless of the precise definition, the terminology is due to the fact that a generative model can be used to "generate" random instances (outcomes), either of an observation and target (x, y) or of an observation x given a target value y , while a discriminative model or discriminative classifier (without a model) can be used to "discriminate" the value of the target variable Y , given an observation x . The difference between "discriminate" (distinguish) and "classify" is subtle, and these are not consistently distinguished.

19.2 Generative adversarial networks

19.2.1 Overview

As discussed in Appendix (19.1), a generative model is a statistical model of the joint probability distribution of two observed random variables. In general, generative models focus on providing a parametric specification of a probability distribution function (pdf). The model can then be trained by maximising the log-likelihood. However, in these models, the pdf is generally intractable, requiring numerical approximation to compute the gradient. This is the case of the deep Boltzmann machine (see Salakhutdinov et al. [2009]). Recently, generative models developed representing the log-likelihood implicitly while generating samples from the desired distribution. For instance, generative stochastic networks can be trained with exact backpropagation, but rely on Markov chains for computation. Kingma et al. [2014] and Rezende et al. [2014] developed more general stochastic backpropagation rules, allowing one to backpropagate through Gaussian distributions with finite variance, and to backpropagate to the covariance parameter as well as the mean. They trained variational autoencoders (VAEs), which combines a differentiable generator network with a second neural network. VAEs require differentiation through the hidden units.

Schmidhuber [1992b] proposed predictability minimisation having two neural networks compete. Each hidden unit in a neural network is trained to be different from the output (a single scalar) of a second network, which predicts the value of that hidden unit given the value of all of the other hidden units. Thus, it is a regulariser encouraging the hidden units of a neural network to be statistically independent, while they accomplish some other task. It is an optimisation problem with an objective function to be minimised, and learning approaches the minimum of the objective function. Following a similar approach, Goodfellow et al. [2014] proposed GAN models which eliminate Markov chains by observing that

$$\lim_{\sigma \rightarrow 0} \nabla_x E_{\epsilon \sim N(0, \sigma^2 I)} [f(x + \epsilon)] = \nabla_x f(x)$$

GANs require differentiation through the visible units, and thus cannot model discrete data. In GANs, one network produces a rich, high dimensional vector that is used as the input to another network, and attempts to choose an input that the other network does not know how to process. Here, the competition between the networks is the only training criterion. It is based on a minimax game with a value function that one agent seeks to maximise and the other seeks to minimise. The game terminates at a saddle point corresponding to a minimum for one player and a maximum for the other player.

19.2.2 Description

A generative adversarial network (GAN) model (see Goodfellow et al. [2014]) is a novel approach for training generative models. They estimate the latter via an adversarial process where two models are simultaneously trained:

1. Generator: A generative net G captures the data distribution by learning to confuse D by generating high quality data. It is acting as a team of counterfeiters.
2. Discriminator: A discriminative net D that estimates the probability that a sample came from the training data (real) rather than G . It learns to separate the data created by G from the observed data. It acts as the police trying to catch the counterfeiters.

The training process for G is to maximise the probability that D makes a mistake (incapacity of recognising true from generated data), which corresponds to a minmax two-player game. In order for G to confuse D as much as possible, it must reduce the adversarial loss in the sense that D will not correctly discriminate the prediction. The target loss for D is equal to the adversarial loss on D (D is able to classify correctly inputs from G as fake and the dataset as true). Through competition, both networks improve their performance until the genuine data is indistinguishable from the generated one. If the pair (G, D) is made of multilayer perceptrons, the system can be trained with backpropagation.

We present the adversarial nets, where both G and D are multilayer perceptrons, and G generates samples by passing random noise as input. We let z be the input noise variables with prior $p_Z(z)$, and want to learn (implicitly) the distribution p_G over data x . The generator is the neural network $G(z, \theta_G)$ with the parameter θ_G . It maps data to the data-input space $G : z \rightarrow x$. The Discriminator is the neural network $D(x, \theta_D)$ that scores the likelihood that x was sampled from the known dataset, $p_{data}(x) - D : x \rightarrow [0, 1]$. D is trained to maximise correct labelling, while G is trained to minimise $\log(1 - D(G(z)))$. Note, there is no explicit representation for $p_G(x)$ since it is expressed via $G(\cdot, \theta_G)$. As such Markov chains are not needed.

Thus, the pair (G, D) plays a two-player minmax game with value function $V(G, D)$ given by

$$\min_G \max_D V(G, D) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_Z(z)} [\log (1 - D(G(z)))] \quad (19.138)$$

The game is implemented by using an iterative, numerical approach. See Algorithm (26). First, the input noise z is uniformly sampled, and then the mapping $x = G(z)$ imposes the non-uniform distribution p_G on the transformed samples. G contracts in regions of high density and expands in regions of low density. The gradient of D guides $G(Z)$ to flow to regions that are more likely to be classified as data. Eventually, if the pair (G, D) has enough capacity, it will reach a saddle point where $D(X) = \frac{1}{2}$, meaning that $p_G = p_{data}$.

Since optimising D in an inner loop is expensive, the authors proposed to alternate between k steps of optimising D and one step of optimising G , to maintain D near its optimal solution as long as G slowly evolves. In the early stage of the process, G is poor and D has a high accuracy at rejecting the samples created from G , leading $\log(1 - D(G(z)))$ to saturate. One solution is to train G to maximise $\log D(G(z))$, which provides stronger gradient in the early stage.

Algorithm 26 Minibatch stochastic gradient descent training of GAN.

Require: The number of steps to apply to the discriminator k is a hyperparameter. The least expensive option is $k = 1$.

Require: get the number of training iteration N .

1: **for** N **do**

2: **for** k steps **do**

3: Sample minibatch of m noise samples $\{z^1, \dots, z^m\}$ from noise prior $p_G(z)$.

4: Sample minibatch of m examples $\{x^1, \dots, x^m\}$ from data generating distribution $p_{data}(x)$.

5: Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_D} \frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log (1 - D(G(z^{(i)})))]$$

6: **end for**

7: Sample minibatch of m noise samples $\{z^1, \dots, z^m\}$ from noise prior $p_G(z)$.

8: Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_G} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)})))$$

9: **end for**

19.2.3 Analysis

From Algorithm (26) we see that the generator G implicitly defines a probability distribution p_G obtained by the mapping $G : z \rightarrow x$ for $z \sim p_Z$. We now discuss convergence to saddle point where $D(x) = \frac{1}{2}$. The authors first showed that the minmax problem has a global optimum for $p_G = p_{data}$.

Proposition 3 For G fixed, the optimal discriminator D is

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

Proof: Given a generator G , from Equation (19.138), D maximise the quantity

$$\begin{aligned} V(G, D) &= \int_x p_{data}(x) \log(D(x)) dx + \int_z p_z(z) \log(1 - D(G(z))) dz \\ &= \int_x p_{data}(x) \log(D(x)) + p_G(x) \log(1 - D(x)) dx \end{aligned}$$

For any $(a, b) \in \mathbb{R}^2 \setminus \{0, 0\}$, the function $y \rightarrow a \log(y) + b \log(1 - y)$ reaches its maximum in $[0, 1]$ at $\frac{a}{a+b}$. The discriminator does not need to be defined outside of $\text{Supp}(p_{data}) \cup \text{Supp}(p_G)$.

This is equivalent to maximising the log-likelihood for estimating the conditional probability $P(Y = y|x)$, where Y indicates whether x comes from p_{data} (with $y = 1$) or from p_G (with $y = 0$). Thus, the minmax game in Equation (19.138) can be rewritten as

$$C(G) = \max_D V(G, D) = E_{x \sim p_{data}} \left[\frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + E_{x \sim p_G} \left[\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right]$$

Theorem 19.1 The global minimum of the virtual training criterion $C(G)$ is reached if and only if $p_G = p_{data}$. At that point, $C(G)$ reaches the value $-\log 4$.

Proof: For $p_G = p_{data}$, $D_G^* = \frac{1}{2}$. Hence, from the above equation at $D_G^* = \frac{1}{2}$, we get $C(G) = \log \frac{1}{2} + \log \frac{1}{2} = -\log 4$. This is the best possible value of $C(G)$, reached only for $p_G = p_{data}$, since

$$E_{x \sim p_{data}}[-\log 2] + E_{x \sim p_G}[-\log 2] = -\log 4$$

and by subtracting this expression from $C(G) = V(G, D_G^*)$, we get

$$C(G) = -\log 4 + KL(p_{data} \parallel \frac{p_{data} + p_G}{2}) + KL(p_G \parallel \frac{p_{data} + p_G}{2})$$

where $KL(\cdot)$ is the Kullback-Leibler divergence. The above expression is the Jensen-Shannon divergence JSD between the model's distribution and the data generating process:

$$C(G) = -\log 4 + 2 \cdot JSD(p_{data} \parallel p_G)$$

Since the JSD between two distributions is always non-negative, and zero if and only if they are equal, the authors showed that $C^* = -\log 4$ is the global minimum of $C(G)$ and that the only solution if $p_G = p_{data}$.

The convergence of Algorithm (26) is given as follows:

Proposition 4 *If the pair (G, D) have enough capacity, and at each step of Algorithm (26), the discriminator is allowed to reach its optimum given G , and p_G is updated so as to improve the criterion*

$$E_{x \sim p_{data}}[\log D_G^*(x)] + E_{x \sim p_{data}}[\log (1 - D_G^*(x))]$$

then p_G converges to p_{data} .

Proof: see the authors.

Remark 19.1 *Note, in practice, adversarial nets are limited family of p_G distributions via the function $G(z, \theta_G)$, and we optimise θ_G instead of p_G directly. Hence, the proof does not apply. The authors rely on the good performance of multiplier perceptrons to suggest that they are reasonable models to use despite their lack of theoretical guarantees.*

19.2.4 Improvements

In general, GANs are good at learning an unconditional model such as image and text creation or discovery. When it comes to learning financial time series, one require a sampling process taking into account the previous state space to preserve the statistical properties of the series. As a solution, Goodfellow et al. [2014] proposed to consider Conditional GANs which can handle dependent data generation. In that setting, $p(x|c)$ can be obtained by adding c as input to the pair (G, D) . Further, they showed that one could perform learned approximate inference by training an auxiliary network to predict z given x . While similar to the inference net trained by the wake-sleep algorithm (see Hinton et al. [1995]), here the inference net may be trained for a fixed generator net after the generator net has finished training.

Over time new architectures, loss functions, training, among others, developed to improve the original model. Some of these are as follows:

- **Ensemble strategies:** Train multiple GANs with different initial conditions, slices of data and tasks to attain. The output from the generators are aggregated with some operators (summing, weighted average) from multiple checkpoints or at the end of the training. Examples of these steps are Stacked GANs (see Huang et al. [2017]), Ensembles of GANs (see Wang et al. [2016b]), AdaGANS (see Tolstikhin et al. [2017]), among others.
- **Loss function reshaping:** Reshape the original loss function to circumvent problems related to training instability.

- Adjusting architecture and training process: As an example, the Deep Convolutional GAN (see Radford et al. [2015]) has a set of constraints on the architectural topology of Convolutional GANs to make the training more stable. Another example is the Evolutionary GANs (see Wang et al. [2018]) that adds to the training loop of a GAN different metrics to jointly optimise the generator. It also employs a population of generators, created by introducing novel mutation operators.
- Bidirectional GAN (BiGAN): It aims at introducing a generator model to act as the discriminator, whereby the discriminator naturally considers the entire translation space so that the inadequate training problem can be alleviated (see Zhang et al. [2018b]). To satisfy this property, generator and discriminator are both designed to model the joint probability of sentence pairs, with the difference that, the generator decomposes the joint probability with a source language model and a source-to-target translation model, while the discriminator is formulated as a target language model and a target-to-source translation model. To further leverage the symmetry of them, an auxiliary GAN is introduced and adopts generator and discriminator models of original one as its own discriminator and generator respectively. Two GANs are alternately trained to update the parameters.

19.2.5 Conditional generative adversarial networks

Conditional GANs (cGANs) are an extension of GANs (see Mirza et al. [2014]), where the pair (G, D) is not only based in noise or generated inputs, but includes an additional information set V . It can represent a class label, a certain categorical feature, or an expected market condition. That is, cGANs try to learn an implicit conditional generative model. It can be constructed by feeding the data we wish to condition on, to both the generator and the discriminator. We let v be the conditional variable and modify the GAN model as follows: $G : z \times v \rightarrow x$ and $D : x \times v \rightarrow [0, 1]$. Now, G is trained to minimise $\log(1 - D(G(z|v)))$. Again, the pair (G, D) plays a two-player minmax game with value function $V(G, D)$ given by

$$\min_G \max_D V(G, D) = E_{x \sim p_{data}(x)} [\log D(x|v)] + E_{z \sim p_Z(z)} [\log (1 - D(G(z|v)))]$$

For example, when forecasting a time series $y_1, y_2, \dots, y_T$, the conditional set is $V = \{y_{t-1}, y_{t-2}, \dots, y_{t-p}\}$, and we want to discriminate $x = y_t$ with $p_{data}(y_t | y_{t-1}, y_{t-2}, \dots, y_{t-p})$. Here, p sets the amount of past information in the implicit conditional generative model. For $p = 0$ we recover the GAN model. For $p > 0$, the NN needs larger memory and more capacity to select and model the right past values.

20 Hybrid intelligent modelling

Data mining (see Appendix (12)) and more recent machine-learning methodologies (see Appendix (15.1.1)) provide a range of general techniques for the classification, prediction, and optimisation of structured and unstructured data. All of these methods require the use of quantitative optimisation techniques known as stochastic optimisation algorithms, such as combinatorial optimisation, simulated annealing (SA), genetic algorithms (GA), or reinforced learning. We find GA, ANN and SVM algorithms among the most popular classifiers. The artificial neural network (ANN) is the most widely used technique for classification and prediction. It is an optimisation task where the result is to find optimal weight and bias set of the network that reduce an error function. It can be formulated as the minimisation of an error function in the space of connection weights. However, ANN has many disadvantages and some of its limitations are follows:

- ANN has long trained time
- High computational cost
- Adjustment of weight is difficult
- Output quality of an ANN may be unpredictable

To overcome these disadvantages Neural Networks can be combined with another technique. One way forward is to combine ANN with another algorithm such as evolutionary computing tools that can take care of a specific problem. Hybridisation is a technique which combines two or more classifiers to improve the performance of the classifier. Many researchers proposed hybrid technique with an evolutionary algorithm (EA) to improve the performance of the classifier. For instance, GAs has been hybridised with many other classification algorithms (see Montana et al. [1989]). Alternatively, Ilonen et al. [2003] applied Differential Evolution to train the weights of neural networks, obtaining convergence to a global minimum.

20.1 ANN plus GA in finance

20.1.1 An overview

The underlying principles of genetic algorithm (GA) are to generate an initial population of chromosomes (search solutions) and then use selection and recombination operators to generate a new, more effective population which eventually will have the fittest chromosome (optimal value) among them. Some examples of combination between ANN and GA are Whitley et al. [1990] who used GA to optimise weighted connections and find a good architecture for neural network connections. Kim [2006] proposed a hybrid model of ANN with GA that performs instance selection to reduce dimensionality of data. The genetic algorithm is used to optimise the connection weights between layers of neural network and for selection of relevant instance. Due to the selected instances, the learning time is reduced and prediction performance is enhanced. Bai et al. [2006] presented an effective hybrid system on rough set and neural network. This classifier is used for managing the large document that is growing on the internet. The system creates clusters for organising the documents. The rough set is used for feature reduction and Neural Network is used for classification. Vivekanandan et al. [2010] built a rule-based classifier by using the genetic algorithm. The proposed work tries to reduce the learning time by incremental learning. To do so the classifier reduces the cost of learning and thus making it scalable for large data sets. Large researches have been performed in applying genetic algorithms for classification purpose. For instance, Bhardwaj et al. [2015] proposed a model called GONN which hybridise genetic programming with neural network for multi class classification problem. The model simultaneously deals with structure and weight of the neural network, bringing more diversity of the GP population and reaching a solution faster with more generalised solutions. GAs has been also widely used for the discovery of classification rules.

20.1.2 Some examples

20.1.2.1 Feature selection Sangwan et al. [2015] proposed an integrated ANN and GA for predictive modelling and optimization of turning parameters to minimize surface roughness. Following the same approach, Inthachot et al. [2016] used GA to solve a feature selection problem to find effective subsets of input into ANN. Considering four input variables for each technical indicator, to speed up computation time, they used GA to devise a small number of effective subsets of input variables (see Figure (72)). Accuracy is used to determine chromosome selection as well as to measure the performance of the prediction model.

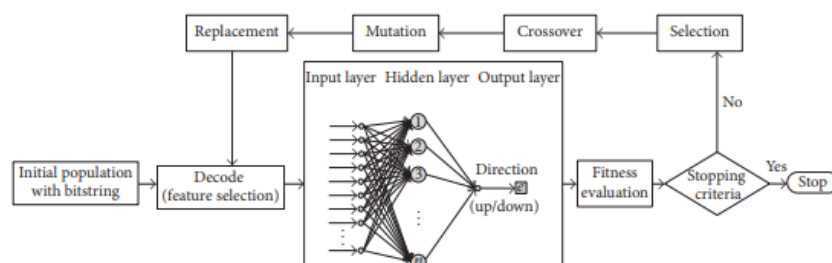


Figure 72: Steps of operation of ANN and GA hybrid intelligence

20.1.2.2 Weight selection Karimi et al. [2012] used GA to find a set of weights for connections to each node in an ANN model and determine correlation of density in nanofluids. Qiu et al. [2016] proposed to determine the optimal set of weights and biases to enhance the accuracy of an ANN model (backpropagation algorithm) by using genetic algorithms (GA). They reported improved prediction accuracy of the direction of stock price index movement with their optimised ANN model. They considered a three-layer ANN model where the output layer consists of only one neuron that represents the predicted direction of the daily stock market index. The signals are generated on a daily basis at time interval $\delta = 1$ day and the forecasting period is $d = 1$ day. GA technique is used to optimise the (initial) weights and biases of the ANN model. Then, the ANN model is trained by the BP algorithm using the determined weights and bias values. The steps are as follow:

- The data is first normalised (see Equation (9.20)), then all the weights and biases are encoded in a string and the initial population is generated. Each solution from the GA is called chromosome (or individual) describing the ANN with a certain set of weights and bias values.
- Train the ANN model using the BP algorithm and then evaluate each chromosome of the current population using a fitness function based on the MSE.
- Rank all the individuals using the fitness proportion method and select the individuals with a higher fitness value to pass on to the next generation directly.
- Apply crossover and mutation to the current population and create new chromosomes. Evaluate the fitness value of the new chromosomes and insert these new chromosomes into the population to replace the worse individuals of the current population.
- Repeat the previous steps until the stop criterion is satisfied.

20.2 ANN plus DE in finance

The training (error) of artificial neural networks (ANNs) depends on the adaptation of free network parameters, such as the weight values and the bias. Thus, it is an optimisation task where the result is to find optimal weight and bias set of the network that reduce an error function. It can be formulated as the minimisation of an error function in the space of connection weights. The methods for training ANNs are backpropagation (BP), Levenberg-Marquadt (LM), Quasi-Newton (QN). In general, the error backpropagation method (EBP), based on gradient method, is preferred. However, the objective function describing the artificial neural networks training problem is a multi-modal function, so that the algorithms based on gradient methods can easily be stuck in local extremes. To avoid this problem one can use a global search optimisation, such as simulating annealing (SA) algorithm (see Aarst et al. [1989]), Particle Swarm Optimization (PSO) and Evolutionary Algorithm (EA) (see Michalewicz [1996]). For instance, Junyou [2007] used PSO-trained neural networks to forecast the stock price.

We have seen above that when training the ANN by using GA method (GANN), the weight coefficients of neural network are encoded in chromosome, and selection, cross-over and mutation operators are used to minimise the error. However, GANN suffers from early convergence. Even though GA has diversity in its population, it lacks of convergence speed towards global optima. On the other hand, PSO has faster convergence speed than that of GA but it lacks of diversity in population.

We focus on differential evolution (DE), which has the following advantages: the possibility of finding the global minimum of a multi-modal function regardless of initial values of its parameters, quick convergence and a small number of parameters to set up at the start of the algorithm operation.

Ilonen et al. [2003] applied Differential Evolution to train the weights of neural networks, obtaining convergence to a global minimum but in a very long time. To do so, the weights of all neurons are stored in a real valued solution vector X . The algorithm is used to minimise the sum of squared errors (SSE) or a similar criterion function. The evaluation of this function requires iterating through all elements of the training set T and summing all the partial results (squared

errors in the case of SSE) obtained for all the elements of T . Note, different choices of behavioural parameters cause it to perform worse or better than BP on particular problems and the selection of good parameters is a challenge. One way forward is to try and make the DE parameters automatically adapt to new problems during optimisation, hence alleviating the need for the practitioner to select the parameters by hand. Since some meta parameters are kept fix throughout the entire evolution process, one must tune value of these control parameters. One approach is to use self-adaptive strategy for controlling these parameters (see Brest et al. [2006]). Slowik et al. [2008] applied DE technique to train a feed-forward flat ANN to classification of parity-p problem by introducing the adaptive selection of control parameters to the algorithm. The method proposed is called DE-ANNT (Differential Evolution ? Artificial Neural Network Training). As a result, only one parameter is set at the start of proposed algorithm. Similarly, Bui et al. [2015] use self-adaptive strategy to control the DE parameters. Rather than considering self-adaptive parameters, Pedersen et al. [2008b] proposed an overlaying optimisation method known as meta-optimisation.

Even though EA simultaneously process a population of problem solutions, it comes at the expense of very high computational complexity. To remedy this problem, Fan et al. [2003] introduced Trigonometric DE (TDE) algorithm and applied to train the ANN as a test case for their proposed algorithm. Bandurski et al. [2009] proposed to combine differential evolution algorithm with a gradient-based approach. This hybridisation comes in a number of ways, some of them are as follows:

1. One solution is for an EA to be used to locate a promising region of the weight space, and then use a gradient descent method to fine-tune the best solution (or all the solutions from the population) obtained by the EA (see Sherrod [2008]).
2. Another solution is to apply Lamarckian evolution (see Ross [1999]) to ANN and let the gradient descent procedure be incorporated into an EA as a new search operator. This operator is applied to the population members in each EA iteration, in addition to standard operators such as mutation and crossover (see Cortez et al. [2002]).

The authors applied the conjugate gradient (CG) algorithm (see Hestenes et al. [1952]) to each candidate solution $U_{i,G}$ before the computation of its fitness. The number of CG iterations is set by the user and remains constant throughout the entire experiment. The results obtained were significantly faster than the original version of DE.

Similarly, to keep a reasonable balance between convergence speed and the capability of global search, Mingguang et al. [2009] combined the BP and DE algorithm to optimise the weights and threshold value adjustments of ANN.

Si et al. [2012] used DE with global and local neighbourhood based mutation (DEGL) algorithm to search the synaptic weight coefficients of neural network and to minimise the learning error in the error surface. It is a modification of DE where both global and local neighbourhood-based mutation operator is combined to create donor vector.

References

- [1989] Aarst E.H.L., Korst J., Simulated annealing and Boltzmann machines. John Wiley.
- [2018] Abbas S.S., Forecasting stock index using deep learning and how it can be applied in the financial sector. Degree Project in Technology, First Cycle, Stockholm, Sweden.
- [1993] Abry P., Goncalves P., Flandrin P., Wavelet-based spectral analysis of $1/f$ processes. *IEEE International Conference on Acoustics, Speech, and Signal Processing*, **3**, pp 237–240.
- [1995] Abry P., Goncalves P., Flandrin P., Wavelets, spectrum estimation and $1/f$ processes. in A. Antoniadis and G. Oppenheim, eds, *Lecture Notes in Statistics: Wavelets and Statistics*, Springer-Verlag, pp 15–30.
- [1996] Abry P., Sellan F., The wavelet-based synthesis for fractional Brownian motion. proposed by F. Sellan and Y. Meyer, Remarks and Fast Implementation, *Applied and Computational Harmonic Analysis*, **3**, (4), pp 377–383.
- [1998] Abry P., Veitch D., Wavelet analysis of long-range-dependent traffic. *IEEE Transaction on Information Theory*, **44**, (1), pp 2–15.
- [1999] Abry P., Sellan F., A wavelet-based joint estimator of the parameters of long-range dependence. *IEEE Transaction on Information Theory*, **45**, (3), pp 878–897.
- [2000] Abry P., Flandrin P., Taqqu M.S., Veitch D., Wavelets for the analysis, estimation, and synthesis of scaling data. in *Self-similar Network Traffic and Performance Evaluation*, ed. K. Park, W. Willinger, Wiley, pp 39–87.
- [2002] Abry P., Baraniuk R., Flandrin P., Riedi R., Veitch D., Multiscale nature of network traffic. *IEEE Signal Processing Magazine*, **19**, (3), pp 28–46.
- [2001] Abu-Mostafa Y.S., Atiya A.F., Magdon-Ismael M., White H., Neural networks in financial engineering. *IEEE Transactions on Neural Networks*, **12**, (4), pp 653–656.
- [2012] Adebisi A.A., Ayo C.K., Adebisi M.O., Otokiti S.O., Stock price prediction using neural network with hybridized market indicators. *Journal of Emerging Trends in Computing and Information Science*, **3**, (1), pp 1–9.
- [1998] Adya M., Collopy F., How effective are neural networks at forecasting and prediction? A review and evaluation. *Journal of Forecasting*, **17**, pp 481–495.
- [2010] Ahmed N.K., Atiya A.F., El Gayar N., El-Shishiny H., An empirical comparison of machine learning models for time series forecasting. *Econometric Reviews*, **29**, (5), pp 594–621.
- [2010] Alpaydin E., Introduction to machine learning. MIT Press
- [1997] Amit Y., Geman D., Shape quantization and recognition with randomized trees. *Neural Computation*, **9**, (7), pp 1545–1588.
- [1986] Anderson C.W., Learning and problem solving with multilayer connectionist systems. Doctoral dissertation, Department of Computer and Information Science, University of Massachusetts, Amherst.
- [1987] Anderson C.W., Strategy learning with multilayer connectionist representations. In *Proceedings of the fourth international workshop on Machine Learning*, pp 103–114, Ann Arbor, Michigan, Morgan Kaufmann.
- [2018] Andras P., Random projection neural network approximation. 2018 International Joint Conference on Neural Networks (IJCNN), IEEE.

- [2015] Antonik P., Duport F., Smerieri A., Haelterman P., Massar S., FPGA implementation of reservoir computing with online learning. Université Libre de Bruxelles.
- [2016] Antonik P., Hermans M., Duport F., Haelterman M., Massar S., Towards pattern generation and chaotic series prediction with photonic reservoir computers. Laboratoire d'Information Quantique, Université Libre de Bruxelles, Belgium.
- [1974] Appel G., Double your money every three years. Brightwaters, NY: Windsor Books.
- [1999] Appel G., Technical analysis power tools for active investors. Financial Times Prentice Hall.
- [2008] Appel G., Appel M., A quick tutorial in MACD: Basic concepts. Working Paper.
- [2001] Armstrong J.S., Principles of forecasting: A handbook for researchers and practitioners. ed.: Norwell, MA: Kluwer Academic Publishers.
- [1991] Arneodo A., Bacry E., Muzy J-F., Wavelets and multifractal formalism for singular signals: Application to turbulence data. *Phys. Rev. Lett.*, **67**, pp 3515–3518.
- [1995] Arneodo A., Bacry E., Graves P.V., Muzy J-F., Characterizing long-range correlations in DNA sequences from wavelet analysis. *Phys. Rev. Lett.*, **74**, 3293.
- [1998] Arneodo A., Muzy J-F., Dornette D., Discrete causal cascade in the stock market. *Eur. Phys. J.*, **2**, pp 277–282.
- [2017] Arras L., Montavon G., Muller K-R., Samek W., Explaining recurrent neural network predictions in sentiment analysis. In *Proceedings of the 8th Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis*, pp. 159–168.
- [2000] Atiya A.F., Parlos A.G., New results on recurrent network training: Unifying the algorithms and accelerating convergence. *IEEE Transactions on neural networks*, **11**, (3), pp 697–709.
- [1992] BaBa N., Kozaki M., An intelligent forecasting system of stock price using neural networks. In *Proc. IJCNN*, Baltimore, Maryland, pp 652–657.
- [2001] Bacry E., Delour J., Muzy J-F., Multifractal random walk. *Physical Review E*, **64**, 026103–026106.
- [2010] Bahrammirzaee A., A comparative survey of artificial intelligence applications in finance: Artificial neural networks, expert system and hybrid intelligent systems. *Neural Computing and Applications*, **19**, (8), pp 1165–1195.
- [2006] Bai R., Wang X., An effective hybrid classifier based on rough set and neural network. *International Conference IEEE*, 0-7695-2749-3/06.
- [2018] Bai S., Kolter J.Z., Koltun V., An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271.
- [1996] Baillie R.T., Long memory processes and fractional integration in econometrics. *Journal of Econometrics*, **73**, pp 5–59.
- [1996] Baillie R.T., Bollerslev T., Mikkelsen H.O., Fractionally integrated generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, **74**, pp 3–30.
- [2015] Ballings M., Van den Poel D., Hespeels N., Gryp R., Evaluating multiple classifiers for stock price direction prediction. *Expert Systems with Applications*, **42**, pp 7046–7056.

- [2009] Bandurski K., Kwedlo W., Training neural networks with hybrid differential evolution algorithm. *Zeszyty Naukowe Politechniki Bialostockiej*.
- [2017] Bao W., Yue J., Rao Y., A deep learning framework for financial time series using stacked autoencoders and long-short term memory. *Plos One*, **12**.
- [1991] Barabasi A-L., Vicsek T., Multifractality of self-affine fractals. *Physical Review A*, **44**, (4), pp 2730–2733.
- [1983] Barto A.G., Sutton R.S., Anderson C.W., Neuronlike elements that can solve difficult learning control problems. *IEEE Trans. on Systems, Man, and Cybernetics* 19:835.
- [2012] Barunik J., Understanding the source of multifractality in financial markets. *Physica A*, **391**, (17), pp 4234–4251.
- [2013] Battula B.P., Satya Prasad R., An overview of recent machine learning strategies in data mining. *International Journal of Advanced Computer Science and Applications*, **4**, (3), pp 50–54.
- [2008] Bekiros S.D., Georgoutsos D.A., Direction-of-change forecasting using a volatility based recurrent neural network. *Journal of Forecasting*, **27**, (5), pp 407–417.
- [1999] Bellgard C., Goldschmidt P., Forecasting foreign exchange rates: Random walk hypothesis, linearity and data frequency. Working Paper, The University of Western Australia.
- [1994] Bengio Y., Simard P., Frasconi P., Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, **5**, (2), pp 157–166.
- [1994] Beran J., Statistics for long-memory processes. Monographs on Statistics and Applied Probability, Chapman & Hall, New York.
- [1998] Bessembinder H., Chan K., Market efficiency and the returns to technical analysis. *Financial Management*, **27**, (2), pp 5–27.
- [2009] Beynon-Davies P., Business information systems. Basingstoke, UK
- [2015] Bhardwaj A., Tiwari A., Breast cancer using genetically optimized neural network model. *International journal Expert System with Application*.
- [2017] Bianchi F.M., Kampffmeyer M., Maiorino E., Jensen R., Temporal overdrive recurrent neural network. Working Paper, University of Tromso, arXiv:1701.05159v1.
- [1978] Binder D.A., Bayesian cluster analysis. *Biometrika*, **65**, pp 31–38.
- [2006] Bishop C.M., Pattern recognition and machine learning. Springer Verlag.
- [2014] Bloch D., A practical guide to quantitative portfolio trading. Working Paper, University of Paris 6 Pierre et Marie Curie.
- [1986] Bollerslev T., Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, **31**, pp 307–327.
- [1999] Bollerslev T., Jubinski D., Equality trading volume and volatility: Latent information arrivals and common long-run dependencies. *Journal of Business & Economic Statistics*, **17**, pp 9–21.
- [1992] Bollinger J., Using Bollinger bands. *Technical Analysis of Stocks and Commodities*, **10**, February.
- [1992] Boser B.E., Guyon I.M., Vapnik V.N., A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory*, pp 144.

- [2017] Bouizi R., Financial time series forecasting using wavelet transform and reservoir computing paradigm. Quant Finance Ltd, MSc Thesis, Department of Mathematics, Imperial College London.
- [2012] Boulanger-Lewandowski N., Bengio Y., Vincent P., Modeling temporal dependencies in high-dimensional sequences: Application to polyphonic music generation and transcription. In *Proceedings of the Twenty-nine International Conference on Machine Learning*, (ICML 12).
- [1976] Box G.E.P., Jenkins G.M., Time series analysis. Forecasting and control. Holden-Day Inc, San Francisco, USA.
- [2000] Brabazon T., A connectivist approach to index modelling in financial markets. Department of Accountancy, University College, Dublin.
- [1998] Breidt F.J., Crato N., de Lima P., On the detection and estimation of long memory in stochastic volatility. *Journal of Econometrics*, **83**, pp 325–348.
- [1984] Breiman L., Friedman J., Stone C.J., Olshen R.A., Classification and regression trees. Chapman and Hall/CRC, Boca Raton, FL.
- [1996] Breiman L., Stacked regressions. *Machine Learning*, **24**, (1), pp 49–64.
- [1996b] Breiman L., Out-of-bag estimation. Technical report, Department of Statistics, University of California.
- [1996c] Breiman L., Bagging predictors. *Machine Learning*, **24**, (2), pp 123–140.
- [2001] Breiman L., Random forests. *Machine learning*, **45**, (1), pp 5–32.
- [2006] Brest J., Greiner S., Boskovic B., Mernik M., Zumer V., Self-adapting control parameters in differential evolution: A comparative study on numerical benchmark problems. *IEEE Transactions on Evolutionary Computation*, **10**, (6), pp 646–657.
- [1992] Brock W., Lakonishok J., LeBaron B., Simple technical trading rules and the stochastic properties of stock returns. *Journal of Finance*, **47**, (5), pp 1731–1764.
- [1999] Brown C., Technical analysis for the trading professional. 1st Edition, McGraw-Hill.
- [2015] Bush N.T., Hasegawa H., Training artificial neural network using modification of differential evolution algorithm. *International Journal of Machine Learning and Computing*, **5**, (1), pp 1–6.
- [2005] Bush K., Tsendjav B., Improving the richness of echo state features using next ascent local search. in *Proceedings of the Artificial Neural Networks in Engineering Conference*, pp 227–232, St. Louis.
- [2006] Bush K., Anderson C., Exploiting iso-error pathways in the N, k -plane to improve echo state network performance.
- [1994] Cai J., A Markov model of switching-regime ARCH. *Journal of Business*, **12**, pp 309–316.
- [2004] Cajueiro D.O., Tabak B.M., The Hurst exponent over time: Testing the assertion that emerging markets are becoming more efficient. *Physica A*, **336**, pp 521–537.
- [2007] Cajueiro D.O., Tabak B.M., Long-range dependence and multifractality in the term structure of LIBOR interest rates. *Physica A*, **373**, pp 603–614.
- [1995] Caldwell R.B., Performances metrics for neural network-based trading system development. *NeuroVet Journal*, **3**, (2), pp 22–26.
- [2001] Calvet L., Fisher A., Forecasting multifractal volatility. *Journal of Econometrics*, **105**, pp 27–58.

- [2002] Calvet L., Fisher A., Multifractality in asset returns: Theory and evidence. *The Review of Economics and Statistics*, **84**, (3), pp 381–406.
- [2004] Calvet L., Fisher A., Regime-switching and the estimation of multifractal processes. *Journal of Financial Econometrics*, **2**, pp 44–83.
- [2006] Calvet L., Fisher A., Thompson S., Volatility comovement: A multi-frequency approach. *Journal of Econometrics*, **31**, pp 179–215.
- [2013] Calvet L., Fisher A., Wu L., Staying on top of the curve: A cascade model of term structure dynamics. Working Paper.
- [1997] Campbell J.Y., Lo A.W., MacKinlay A.C., The econometrics of financial markets. Princeton University Press, New jersey.
- [2001] Cao L., Tay F.E., Financial forecasting using support vector machines. *Neural Computing and Applications*, **10**, (2), pp 184–192.
- [2008] Cao M., Qiao P., Neural network committee-based sensitivity analysis strategy for geotechnical engineering problems. *Neural Comput Appl*, **17**, pp 509–519.
- [2019] Cao H., Lin T., Li Y., Zhang H., Stock price pattern prediction based on complex network and machine learning. Hindawi, Complexity, Volume 2019, 12 pages.
- [2004] Carbone A., Castelli G., Stanley H., Time-dependent Hurst exponents in financial time series. *Physica A*, **344**, pp 267–271.
- [2007] Carbone A., Algorithm to estimate the Hurst exponent of high-dimensional fractals. Working Paper, Physics Department, Politecnico di Torino.
- [2001] Castiglione F., Bernaschi M., Market fluctuations: Simulation and forecasting. Working Paper.
- [2016] Cavalcante R.C., Brasileiro R.C., Souza V.L., Nobrega J.P., Oliveira A.L., Computational intelligence and financial markets: A survey and future directions. *Expert Systems with Applications*, **55**, pp 194–211.
- [1994] Chande T.S., Kroll S., The new technical trader. John Wiley, New York.
- [2014] Chandwani D., Saluja M.S., Stock direction forecasting techniques: An empirical study combining machine learning system with market indicators in the Indian context. *International Journal of Computer Applications*, **92**, (11), pp 8–17.
- [2009] Chang P.C., Liu C.H., Lin J.L., Fan C.Y., Ng C.S.P., A neural network with a case based dynamic window for stock trading prediction. *Expert Systems with Applications*, **36**, (3), pp 6889–6898.
- [2013] Chavan P.S., Patil S.T., Parameters for stock market prediction. *International Journal of Computer Technology and Applications*, **4**, (2), pp 337–340.
- [2009] Chavarnakul T., Enke D., A hybrid stock trading system for intelligent technical analysis-based equivolume charting. *Neurocomputing*, **72**, (16-18), pp 3517–3528.
- [2004] Chen Y., Yang B., Dong J., Nonlinear system modelling via optimal design of neural trees. *International Journal of Neural Systems*, **14**, (2), pp 125–137.
- [2009] Chen C-W., Huang C-S., Lai H-W., The impact of data snooping on the testing of technical analysis: An empirical study of Asian stock markets. *Journal of Asian Economics*, **14**, (5), pp 580–591.

- [1994] Cheng S., A neural network approach for forecasting and analyzing the price-volume relationship in the Taiwan stock market. Master thesis, National Jow-Tung University.
- [1989] Chhabra A., Jensen R.V., Direct determination of the $f(\alpha)$ singularity spectrum. *Phys. Rev. Lett.*, **62**, (12), pp 1327–1330.
- [2016] Chiang W-C., Enke D., Wu T., Wang R., An adaptive stock index trading decision support system. *Expert Syst Appl*, **59**, pp 195–207.
- [2010] Chitra A., Uma S., An ensemble model of multiple classifiers for time series prediction. *International Journal of Computer Theory and Engineering*, **2**, (3), pp 1793–8201
- [2014] Cho K., Van Merriënboer B., Gulcehre C., Bahdanau D., Bougares F., Schwenk H., Bengio Y., Learning phrase representations using rnn encoder-decoder for statistical machine translation. arXiv: 1406.1078.
- [2015] Chung J., Gulcehre C., Cho K., Bengio Y., Gated feedback recurrent neural networks. CoRR, abs:1502.02367.
- [2006] Clegg R.G., A practical guide to measuring the Hurst parameter. *International Journal of Simulation: Systems, Science and Technology*, **7**, (2), pp 3–14.
- [2015] Cogo G., Multilayer ANNs: Predicting the S&P 500 index. Master Degree Program in Economics and Finance, Università Ca'Foscari Venezia.
- [1960] Cohen J., A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, **20**, (1), pp 37–46.
- [1994] Connor J.T., Martin R.D., Atlas L.E., Recurrent neural networks and robust time series prediction. *IEEE Transactions on Neural Networks*, **5**, (2), pp 240–254.
- [2016] Contrás D., Matei O., Translation of the mutation operator from genetic algorithms to evolutionary ontologies. in *International Journal of Advanced Computer Science and Applications (IJACSA)*, **7**, (1).
- [1995] Cortes C., Vapnik V., Support-vector networks. *Machine Learning*, **20**, (3), pp 273–297.
- [2002] Cortez P., Rocha M., Neves J., A lamarckian approach for neural network training. *Neural Processing Letters*, **15**, pp 105–116.
- [2013] Cortez P., Embrechts M.J., Using sensitivity analysis and visualization techniques to open black box data mining models. *Inf Sci*, **225**, pp 1–17
- [2003] Costa R.L., Vasconcelos G.L., Long-range correlations and nonstationarity in the Brazilian stock market. *Physica A*, **329**, pp 231–248.
- [2000] Cunningham P., Carney J., Diversity versus quality in classification ensembles based on feature selection. Technical Report TCD-CS-2000-02, Department of Computer Science, Trinity College Dublin.
- [1989] Cybenko G., Approximations by superpositions of sigmoidal functions. *Mathematics of Control, Signals, and Systems*, **2**, (4), pp 303–314
- [2003] Dablemont S., Van Bellegem S., Verleysen M., Modelling and forecasting financial time series of tick data by functional analysis and neural networks. Université catholique de Louvain, Machine Learning Group, DICE, Louvain-la-Neuve, Belgium.
- [1993] Dacorogna M.M., Muller U.A., Nagler R.J., Olsen R.B., Pictet O.V., A geographical model for the daily and weekly seasonal volatility in the foreign exchange market. *Journal of International Money and Finance*, **12**, (4), pp 413–438.

- [1998] Dacorogna M.M., Muller U.A., Olsen R.B., Pictet O.V., Modelling short-term volatility with GARCH and HARCH models. in C. Dunis and B. Zhou, (eds.), *Nonlinear Modelling of High Frequency Financial Time Series*, John Wiley & Sons, pp 161–176.
- [2001] Dacorogna M.M., Gencay R., Muller U.A., Olsen R.B., Pictet O.V., An introduction to high frequency finance. Academic Press, San Diego, CA.
- [2003] Damodaran A., Investment philosophies. John Wiley & Sons, New York.
- [1998] Darbellay G.A., Predictability: An information-theoretic perspective. in *Signal Analysis and Prediction*, ed, A. Prochazka, J. Uhler, P.W.J. Rayner and N.G. Kingsbury, pp 249–262.
- [2000] Darbellay G.A., Slama M., Forecasting the short-term demand for electricity? Do neural networks stand a better chance? *International Journal of Forecasting*, **16**, pp 71–83.
- [2002] Day T.E., Wang P., Dividends, nonsynchronous prices, and the returns from trading the Dow Jones Industrial Average. *Journal of Empirical Finance*, **9**, (4), pp 431–454.
- [1994] Deboeck G.J., Trading on the edge: Neural, genetic and fuzzy systems for chaotic and financial markets. Wiley.
- [2014] De Ona J., Garrido C., Extracting the contribution of independent variables in neural network models: A new approach to handle instability. *Neural Computing and Applications*, **25**, pp 859–869.
- [1993] Ding Z., Granger C.W.J., Engle R.F., A long memory property of stock market returns and a new model. *Journal of the Empirical Finance*, **1**, pp 83–106.
- [2015] Ding X., Zhang Y., Liu T., Duan J., Deep learning for event-driven stock prediction. in *IJCAI*, pp 2327–2333.
- [2017] Dingli A., Fournier K.S., Financial time series forecasting: A machine learning approach. *Machine Learning and Applications: An International Journal (MLAIJ)*, **4**, No.1/2/3, September.
- [2017] Di Persio L., Honchar O., Recurrent neural networks approach to the financial forecast of Google assets. *International Journal of Mathematics and Computer in Simulation*, **11**, pp 7–13.
- [2000] Dominey P.F., Ramus F., Neural network processing of natural language: I. Sensitivity to serial, temporal and abstract structure of language in the infant. *Language and Cognitive Processes*, **15**, (1), pp 87–127.
- [2010] Donner R.V., Zou Y., Donges J.F., Marwan N., Kurths J., Recurrence networks, a novel paradigm for nonlinear time series analysis. *New Journal of Physics*, **12**, (3), ID 033025.
- [1999] Douglas S.C. Introduction to adaptive filters. University of Utah. Published by CRC Press LLC.
- [1992] Doya K., Bifurcations in the learning of recurrent neural networks. in *Proceedings of IEEE International Symposium on Circuits and Systems*, **6**, pp 2777–2780.
- [1993] Doya K., Bifurcations of recurrent neural networks in gradient descent learning. *IEEE Transactions on Neural Networks*, **1**, pp 78–80.
- [1996] Drucker H., Burges C.J.C., Kaufman L., Smola A.J., Vapnik V.N., Support vector regression machines. in *Advances in Neural Information Processing Systems 9, (NIPS)*, MIT Press, pp 155–161.
- [1999] Drummond C., Hearne T., The lessons. A series of 30 multi-media lessons, Drummond and Hearne Publications, Chicago.

- [1987] Durbin R., Willshaw D., An analogue approach to the Travelling Salesman. *Nature*, **326**, (16), pp 689–691.
- [1953] Eccles J.G., The Neurophysiological basis of mind. Clarendon, Oxford.
- [1993] Efron B., Tibshirani R., An introduction to the bootstrap. Chapman & Hall, New York.
- [2004] Ehlers J.F., Cybernetic analysis for stocks and futures: : Cutting-edge DSP technology to improve your trading. John Wiley & Sons, New York, USA.
- [2005] Ehlers J.F., FRAMA: Fractal adaptive moving average. Working Paper.
- [1908] Einstein A., Über die von der molekularkinetischen theorie der warme geforderte bewegung von in ruhen- den flüssigkeiten suspendierten teilchen. *Annals of Physics*, **322**.
- [1995] El Hhi S., Bengio Y., Hierarchical recurrent neural networks for long-term dependencies. In *NIPS*, **400**, pp 409.
- [1999] Embrechts P., Resnick S., Samorodnitsky G., Extreme value theory as a risk management tool. *North American Actuarial Journal*, **26**, pp 30–41.
- [2002] Embrechts P., Maejima M., Selfsimilar processes. Princeton University Press.
- [1982] Engle R.F., Autoregressive conditional heteroskedasticity with estimates of the variance of U.K. inflation. *Econometrica*, **50**, pp 987–1008.
- [1986] Engle R.F., Granger C.W.j., Rice j., Weiss A., Semiparametric estimates of the relation between weather and electricity sales. *Journal of the American Statistical Association*, **81**, pp 310–320.
- [1987] Engle R.F., Granger C.W.j., Co-integration and error correction: Representation, estimation, and testing. *Econometrica*, **55**, 2, pp 251–276.
- [1965] Fama E.F., Portfolio analysis in a stable Paretian market. *Management Science*, **11**, pp 404–419.
- [2003] Fan H.Y., Lampinen J., A trigonometric mutation operation to differential evolution. *International Journal of Global Optimization*, **27**, pp 105–129.
- [2018] Faustryjak D., Jackowska-Strumitto L., Majchrowicz M., Forward forecast of stock prices using LSTM neural networks with statistical analysis of published messages. In *Proceedings of the International Interdisciplinary PhD Workshop 2018*, Swinoujscie, Poland, pp 288–292.
- [1988] Feder J., Fractals. Plenum Press, New York.
- [2008] Fernandez-Blanco P., Technical market indicators optimization using evolutionary algorithms. In proceed- ings of the 2008 GECCO conference companion on genetic and evolutionary computation. Atlanta, USA.
- [1936] Fisher R.A., The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, **7**, pp 179–188.
- [1981] Fleiss J.L., Statistical methods for rates and proportions. John Wiley & Sons, New York, NY, 2nd edition.
- [2008] Fok W.W.T., Tam V.W.L., Computational neural network for global stock indexes prediction. *Proceedings of the World Congress on Engineering*, **2**, pp 1171–1175.
- [1986] Fox R., Taqqu M.S., Large-sample properties of parameter estimates for strongly dependent stationary Gaussian time series. *The Annals of Statistics*, **14**, (2), pp 517–532.
- [1997] Freund Y., Schapire R.E., A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, **55**, (1), pp 119–139.

- [2000] Friedman J., Hastie T., Tibshirani R., Additive logistic regression: A statistical view of boosting (with discussions). *Annals of Statistics*, **28**, (2), pp 337–407.
- [1985] Frisch U., Parisi G., Fully developed turbulence and intermittency. in M. Ghil, R. Benzi, G. Parisi, ed., *Turbulence and Predictability in Geophysical Fluid Dynamics and Climate Dynamics*, Amsterdam, pp 84–88.
- [1989] Funahashi K., On the approximate realization of continuous mappings by neural networks. *Neural Network*, **2**, (3), pp 183–192.
- [1993] Funahashi K., Nakamura Y., Approximation of dynamical systems by continuous time recurrent neural networks. *Neural Networks*, **6**, pp 801–806.
- [1995] Gately E., Neural networks for financial forecasting. Wiley, New York, USA.
- [1997] Gencay R., The predictability of security returns with simple technical trading rules. *Journal of Empirical Finance*, **5**, pp 347–359.
- [2016] Gerlein E.A., McGinnity M., Belatreche A., Coleman S., Evaluating machine learning classification for financial trading: An empirical approach. *Expert Systems with Applications*, **54**, pp 193–207.
- [2000] Gers F.A., Schmidhuber J., Cummins F., Continual prediction with LSTM. *Neural Computation*, **12**, (10), pp 2451–2471.
- [2002] Gers F.A., Schraudolph N., Schmidhuber J., Learning precise timing with LSTM recurrent networks. *Journal of Machine Learning Research*, **3**, pp 115–143.
- [2014] Goodfellow I.J., Pouget-Abadie J., Mirza M., Xu B., Warde-Farley D., Ozair S., Courville A., Bengio Y., Generative adversarial nets. in *Proceedings of the International Conference on Neural Information Processing Systems*, pp 2672–2680.
- [2014] Goudarzi A., Banda P., Lakin M.R., Teuscher C., Stefanovic D., A comparative study of reservoir computing for temporal signal processing. Working Paper, University of New Mexico and Portland State University.
- [2004] Granger C., Hyng N., Occasional structural breaks and long memory with an application to the SP500 absolute stock returns. *Journal of Empirical Finance*, **11**, pp 399–421.
- [1976] Graves J.E., Granville's new strategy of daily stock market timing for maximum profit. Prentice-Hall, Inc.
- [2005] Graves A., Schmidhber J., Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Networks*, **18**, (5-6), pp 602–610.
- [2008] Graves A., Supervised sequence labelling with recurrent neural networks. PhD thesis, Technische Univer-sitat Munchen.
- [2008] Graves A., Schmidhber J., Offline handwriting recognition with multidimensional recurrent neural networks. In *Advances in Neural Information Processing Systems*, volume 21.
- [2011] Graves A., Mohamed A-R., Hinton G., Speech recognition with deep recurrent neural networks. *International Conference on Acoustics, Speech and Signal Processing*, pp 1033–1040.
- [2012] Graves A., Supervised sequence labelling with recurrent neural networks. *Studies in Computational Intelligence*, Springer-Verlag, Berlin Heidelberg.
- [2013] Graves A., Generating sequences with recurrent neural networks. arXiv:1308.0850.

- [2004] Grech D., Mazur Z., Can one make any crash prediction in finance using the local Hurst exponent idea? *Physica A: Statistical Mechanics and its Applications*, **336**, (1), pp 133–145.
- [2005] Grech D., Mazur Z., Statistical properties of old and new techniques in detrended analysis of time series. *Acta Physica Polonica B*, **36**, (8), pp 2403–2413.
- [2015] Greff K., Srivastava R.K., Koutnik J., Steunebrink B.R., Schmidhuber J., LSTM: A search space odyssey. *Transactions On Neural Networks And Learning Systems*.
- [2017] GNU Scientific Library. <https://www.gnu.org/software/gsl/>.
- [2016] Guariglia E, Entropy and fractal antennas. *Entropy*, **18**, (3), pp 2403–2413.
- [2010] Gu G-F., Zhou W-X., Detrending moving average algorithm for multifractals. *Physical Review E*, **82**, 011136, pp 1–12.
- [2011] Guresen E, Kayakutlu G., Daim T.U., Using artificial neural network models in stock market index prediction. *Expert Syst. Appl.*, **38**, pp 10389–10397.
- [2015] Hallac D., Leskovec J., Boyd S., Network lasso: clustering and optimization in large graphs. Stanford University, arXiv:1507.00280.
- [1986] Halsey T.C., Jensen M.H., Kadanoff L.P., Procaccia I., Shraiman B.I., Fractal measures and their singularities: The characterization of strange sets. *Phys. Rev. A* **33**, (2), pp 1141.
- [2000] Hammer B., On the approximation capability of recurrent neural networks. *Neurocomputing*, **31**, (1-4), pp 107–123.
- [1983] Hampson S.E., A neural model of adaptive behavior. Doctoral dissertation, Department of Information and Computer Science, University of California, Irvine.
- [2006] Han J., Kamber M., Data mining: Concepts and techniques. Elsevier ; Morgan Kaufmann, 2nd ed. Amsterdam, Boston, San Francisco, CA.
- [2013] Han Y., Yang K., Zhou G., A new anomaly: The cross-sectional profitability of technical analysis. *Journal of Financial and Quantitative Analysis*, **48**, (5), pp 1433–1461.
- [2001] Hand D., Mannila H., Smyth P., Principles of data mining. MIT Press.
- [1990] Hansen L.K., Salamon P., Neural network ensembles. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **12**, (10), pp 993–1001.
- [2013] Hargreaves C., Hao Y., Prediction of stock performance using analytical techniques. *Journal of Emerging Technologies in Web Intelligence*, **5**, (2), pp 136–142.
- [1997] Harrauld P.G., Kamstra M., Evolving artificial neural networks to combine financial forecasts. *IEEE Transactions on Evolutionary Computation*, **1**, (1), pp 40–52.
- [2005] Harrington N., The link between bollinger bands and the commodity channel index Harrington. *Technical Analysis of Stocks and Commodities*, **23**, (10), pp 32–38.
- [1979] Harrison J.M., Kreps D., Martingale and arbitrage in multiperiods securities markets. *Journal of Economic Theory*, **20**, pp 381–408.
- [1981] Harrison J.M., Pliska S.R., Martingales and stochastic integrals in the theory of continuous trading. *Stochastic Processes Applications*, **11**, pp 215–260.

- [2002] Harvey C.R., Travers K.E., Costa M.J., Forecasting emerging market returns using neural networks. *Emerging Markets Quarterly*, , pp 1–12.
- [2009] Hastie T., Tibshirani R., Friedman J., The elements of statistical learning: Data mining, inference, and prediction. Springer Series in Statistics, Second Edition.
- [2000] Haykin S., Adaptive filter theory. Prentice-Hall, Upper Saddle River, New Jersey.
- [1949] Hebb D.O., The organization of behavior. Wiley, New York.
- [1998] Helstrom T., Holmstrom K., Predicting the stock market. Published as Opuscula ISRN HEV-BIB-OP-26-SE.
- [1983] Hentschel H.G.E., Procaccia I., The infinite number of generalised dimensions of fractals and strange attractors. *Physica 8D*, pp 435–444.
- [1952] Hestenes M.R., Stiefel E., Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, **49**, pp 409–436.
- [2000] Hill J.R., Pruitt G., Hill L., The ultimate trading guide. John Wiley & Sons, Wiley Trading Advantage.
- [1995] Hinton G.E., Dayan P., Frey B.J., Neal R.M., The wake-sleep algorithm for unsupervised neural networks. *Science*, **268**, pp 1558–1161.
- [1994] Ho T.K., Hull J.J., Srihari S.N., Decision combination in multiple classifier systems. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **16**, (1), pp 66–75.
- [1995] Ho T.K., Random decision forests. in *Proceedings of the 3rd International Conference on Document Analysis and Recognition*, Montreal, pp 14–16
- [1998] Ho T.K., The random subspace method for constructing decision forests. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **20**, (8), pp 832–844
- [2004] Ho D-S., Lee C-K., Wang C-C., Chuang M., Scaling characteristics in the Taiwan stock market. *Physica A*, **332**, pp 448–460.
- [1997] Hochreiter S., Schmidhuber J., Long short-term memory. *Neural Computation*, **9**, (8), pp 1735–1780.
- [2001] Hochreiter S., Bengio Y., Frasconi P., Schmidhuber J., Gradient flow in recurrent nets: The difficulty of learning long-term dependencies. in *A field guide to dynamical recurrent neural networks*. IEEE Press.
- [1986] Holland J.H., Escaping brittleness: The possibilities of general purpose learning algorithms applied to parallel rule-based systems. In R.S. Michalski, J.G. Carbonell, & T.M. Mitchell (Eds.), *Machine learning: An artificial intelligence approach* (Vol 2). Los Altos, CA, Morgan Kaufmann.
- [1982] Hopfield J.J., Neural networks and physical systems with emergent collective computational abilities. *Proceedings of National Academy of Sciences USA*, **79**, pp 2554–2558.
- [1985] Hopfield J.J., Tank D.W., Neural computation of decisions in optimization problems. *Biological Cybernetics*, **52**, pp 141–152.
- [1987] Hopfield J.J., Tank D.W., Computing with neural circuits: A model. *Science*, **233**, pp 625–633.
- [1989] Hornik K., Stinchcombe M., White H., Multilayer feedforward networks are universal approximators. *Neural Networks*, **2**, (5), pp 359–366.

- [1991] Hornik K., Approximation capabilities of multilayer feedforward networks. *Neural Networks*, **4**, (2), pp 251–257.
- [2008] Hsieh C-J., Chang K-W., Lin C-J., Keerthi S.S., Sundararajan S., A dual coordinate descent method for large-scale linear SVM. In *Proceedings of the 25th International Conference on Machine Learning (ICML) 08*, New York, NY, USA, pp 408–415.
- [2015] Hu Y., Liu K., Zhang X., Su L., Ngai E.W.T., Liu M., Application of evolutionary computation for rule discovery in stock algorithmic trading: A literature review. *Applied Soft Computing*, **36**, (11), pp 534–551.
- [2005] Huang W., Nakamori Y., Wang S-Y., Forecasting stock market movement direction with support vector machine. *Computers and Operations Research*, **32**, (10), pp 2513–2522.
- [2009] Huang C-L., Tsai C-Y., A hybrid sofm-svr with a filter-based feature selection for stock market forecasting. *Expert Systems with Applications*, **36**, (2), pp 1529–1539.
- [2010] Huang W-Q., Yao S., Zhuang X-T., A network dynamic model based on SSE composite index and trading volume fluctuations. *Journal of Northeastern University*, **31**, (10), pp 1516–1520.
- [2017] Huang X., Li Y., Poursaeed O., Hopcroft J.E., Belongie S.J., Stacked generative adversarial networks. in *CVPR*, **2**, pp 3.
- [2017] Hudson R., Urquhart Y., Wang P., Why do moving average rules work? Comprehensive evidence from world markets.
- [1951] Hurst H.E., Long-term storage capacity of reservoirs. *Trans. Amer. Soc. Civil Engineers*, **116**, pp 770–799.
- [1965] Hurst H.E., Black R.P., Simaika Y.M., Long-term storage: An experimental study. London, Constable.
- [2012] Ihlen E.A.F., Introduction to multifractal detrended fluctuation analysis in Matlab. *Frontiers in Physiology*, **3** (141), pp 1–18.
- [2013] Ihlen E.A.F., Multifractal analyses of response time series: A comparative study. *Behav. Res.*, **45**, pp 928–945.
- [2014] Ihlen E.A.F., Vereijken B., Detection of co-regulation of local structure and magnitude of stride time variability using a new local detrended fluctuation analysis. *Gait & Posture*, **39**, pp 466–471.
- [2000] Ilkka T., Data is more than knowledge. *Journal of Management Information Systems*, **6**, (3), pp 103–117.
- [2003] Ilonen J., Kamarainen J.K., Lampinen J., Differential evolution training algorithm for feed-forward neural networks. *Neural Processing Letters*, **17**, pp 93–105.
- [2016] Inthachot M., Boonjing V., Intakosum S., Artificial neural network and genetic algorithm hybrid intelligence for predicting Thai stock price index trend. *Computational Intelligence and Neuroscience*, **2016**, pp 1–8.
- [2004] Ishii K., van der Zant T., Becanovic V., Ploger P., Identification of motion with echo state network. in *Proceedings of the OCEANS 2004 MTS/IEEE Conference*, **3**, pp 1205–1210.
- [1991] Jacobs R.A., Jordan M.I., Nowlan S.J., Hinton G.E., Adaptive mixtures of local experts. *Neural Computation*, **3**, (1), pp 79–87.
- [2001] Jaeger H., The echo state approach to analysing and training recurrent neural networks. Technical Report GMD Report 148, German National Research Center for Information Technology.

- [2002] Jaeger H., Tutorial on training recurrent neural networks, covering BPPT, RTRL, EKF and the echo state network approach. GMD-Forschungszentrum Informationstechnik, volume 5.
- [2003] Jaeger H., Adaptive nonlinear system identification with Echo State Networks. International University Bremen.
- [2004] Jaeger H., Seminar slides. Seminar Spring.
- [2004] Jaeger H., Haas H., Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, **304**, pp 78–80.
- [2007] Jaeger H., Echo state network. *Scholarpedia*, **2**, (9), pp 2330.
- [2012] Jaeger H., Long short-term memory in echo state networks: Details of a simulation study. Technical report, Jacobs University Bremen.
- [2016] Jamous R.A., El. Seidy E., Bayoum B.I., A novel efficient forecasting of stock market using particle swarm optimization with center of mass based technique. *International Journal of Advanced Computer Science and Applications*, **7**, (4), pp 342–347.
- [1991] Jansen D.W., de Vries C.G., On the frequency of large stock market returns: Putting booms and busts into perspective. *Review of Economics and Statistics*, **73**, (1), pp 18–24.
- [2004] Jebara T., Machine learning: Discriminative and generative. The Springer International Series in Engineering and Computer Science.
- [2017] Jeon S., Hong B., Chang V., Pattern graph tracking-based stock price prediction using big data. *Future Generation Computer Systems*, **80**, (3), pp 171–187.
- [2011] Jhankal N.K., Adhyaru D., Bacterial foraging optimization algorithm: A derivative free technique. Engineering (NUICONe), Nirma University International Conference on 2011.
- [2008] Jiang F., Berry H., Schoenauer M., Supervised and evolutionary learning of echo state networks. in *Proceedings of 10th International Conference on Parallel Problem Solving from Nature*, **5199** of LNCS, pp 215–224, Springer.
- [2012] Jizba P., Korbel J., Methods and techniques for multifractal spectrum estimation in financial time series. FNSPE, Czech Technical University, Prague.
- [1986] Jolliffe T., Principal component analysis. Springer-Verlag, New York.
- [1992] Jordan M.I., Jacobs R.A., Hierarchies of adaptive experts. In J.E. Moody, S.J. Hanson, and R. Lippmann, editors, *Advances in Neural Information Processing Systems 4*, Morgan Kaufmann, San Francisco, pp 985–992.
- [1995] Jordan M.I., Xu L., Convergence results for the EM approach to mixtures of experts architectures. *Neural Networks*, **8**, (9), pp 1409–1431.
- [2013] Jordehi A., Jasni J., Parameter selection in particle swarm optimisation: A survey. *Journal of Experimental & Theoretical Artificial Intelligence*, **25**, (4), pp 527–542.
- [2007] Junyou B., Stock price forecasting using PSO-trained neural networks. in *IEEE Congress on Evolutionary Computation*, pp 2879–2885.
- [1996] Kaastra I., Boyd M., Designing a neural network for forecasting financial and economic time series. *Neurocomputing*, **10**, pp 215–236.

- [2011] Kamber H., Jaiwei P., Jian M., Data mining: Concepts and techniques. Morgan Kaufmann, (3rd ed.).
- [2002] Kantelhardt J., Zschiegner S., Koscielny-Bunde E., Bunde A., Havlin S., Stanley H.E., Multifractal detrended fluctuation analysis of nonstationary time series. *Physica A*, **316**, (1-4), pp 87–114.
- [2011] Kara Y., Boyacioglu M.A., Baykan O.K., Predicting direction of stock price index movement using artificial neural networks and support vector machines: The sample of the Istanbul Stock Exchange. *Expert Systems with Applications*, **38**, (5), pp 5311–5319.
- [2012] Karimi H., Yousefi F., Application of artificial neural network-genetic algorithm (ANN-GA) to correlation of density in nanofluids. *Fluid Phase Equilibria*, **336**, pp 79–83.
- [1960] Keltner C.W., How to make money in commodities. Keltner Statistical Service.
- [1997] Kendall S.M., Ord K., Time series. Oxford University Press, New York.
- [1995] Kennedy J., Eberhart C., Particle swarm optimization. in *Proceedings of the 1995 IEEE International Conference on Neural Networks*, Australia, pp 1942–1948.
- [2010] Khashei M., Bijari M., An artificial neural network (p, d, q) model for time series forecasting. *Expert Syst. Appl.*, **37**, pp 479–489.
- [2000] Kim K-J., Han I., Genetic algorithms approach to feature discretization in artificial neural networks for the prediction of stock price index. *Expert Systems with Applications*, **19**, (2), pp 125–132.
- [2003] Kim K-J., Financial time series forecasting using support vector machines. *Neurocomputing*, **55**, (1), pp 307–319.
- [2004] Kim K., Choi J-S., Yoon S-M., Multifractal measures for the yen-dollar exchange rate. *Journal of the Korean Physical Society*, **44**, (3), pp 643–646.
- [2004b] Kim K-J., Won Boo L., Stock market prediction using artificial neural networks with optimal feature transformation. *Neural Computing and Applications*, **13**, (3), pp 255–260.
- [2006] Kim K-J., Artificial neural networks with evolutionary instance selection for financial forecasting. *Expert Systems with Applications*, **30**, (3), pp 519–526.
- [2006] Kim M-J., Min S-H., Han I., An evolutionary approach to the combination of multiple classifiers to predict a stock price index. *Expert Systems with Applications*, **31**, (2), pp 241–247.
- [2016] Kim Y.M., Enke D., Developing a rule change trading system for the futures market using rough set analysis. *Expert Syst Appl*, **59**, pp 165–173.
- [1990] Kimoto T., Asakawa K., Yoda M., Takeoka M., Stock market prediction system with modular neural networks. in *Proceedings of the 1990 International Joint Conference on Neural Networks*, **1**, Washington, DC, USA, pp 1–6.
- [2014] Kingma D.P., Welling M., Auto-encoding variational bayes. in *Proceedings of International Conference on Learning Representations*, (ICLR).
- [1998] Kittler J., Hatef M., Duin R., Matas J., On combining classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **20**, (3), pp 226–239.
- [1997] Kohara K., Ishikawa T., Fukuhara Y., Nakamura Y., Stock price prediction using prior knowledge and neural networks. *Intelligent Systems in Accounting, Finance and Management*, **6**, (1), pp 11–22.

- [1996] Kohavi R., Wolpert D.H., Bias plus variance decomposition for zero one loss functions. In *Proceedings of the 13th International Conference on Machine Learning*, Bari, Italy, pp 275–283.
- [1990] Kohonen T., The self-organizing map. in *Proceedings of the IEEE*, **78**, (9), pp 1464–1480.
- [2019] Koshiyama A., Firoozye N., Treleaven P., Generative adversarial networks for financial trading strategies fine-tuning and combination. Working Paper, University College London.
- [2013] Kristoufek L., Vosvrda M., Measuring capital market efficiency: Global and local correlations structure. *Physica A*, **392**, (1), pp 184–193.
- [2010] Krollner B., Vanstone B., Finnie G., Financial time series forecasting with machine learning techniques: A survey. in *ESANN 2010 proceedings, European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*, Bruges, Belgium.
- [2006] Kucukemre A.U., Echo state networks for adaptive filtering. Master Thesis, University of Applied Sciences, Bohn-Rhein-Sieg, Germany.
- [2003] Kuncheva L.I., Whitaker C.J., Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine Learning*, **51**, (2), pp 181–207.
- [1998] Kuo R.J., A decision support system for the stock market through integration of fuzzy neural networks and fuzzy Delphi. *Applied Artificial Intelligence*, **6**, pp 501–520.
- [1992] Kurkova V., Kolmogorov's theorem and multilayer neural networks. *Neural Networks*, **5**, pp 501–506.
- [2005] Kwon Y.K., Choi S.S., Moon B.R., Stock prediction based on financial correlation. in *Proceedings of the Conference on Genetic Evolutionary Computation*.
- [2004] Kwong R., Intelligent web-based agent system (iWAF) for e-finance application. MPhil, The Hong Kong Polytechnique University.
- [1997] Lam L., Suen S.Y., Application of majority voting to pattern recognition: An analysis of its behavior and performance. *IEEE Transactions on Systems, Man and Cybernetics - Part A: Systems and Humans*, **27**, (5), pp 553–568.
- [2014] Lam H.K., Ekong U., Xiao B., Ouyang G., Liu H., Chan K.Y., Ling S.H., Variable weight neural networks and their applications on material surface and epilepsy seizure phase classifications. Working Paper, King's College, London.
- [1980] Lambert D.R., Commodity channel index: Tool for trading cyclic trends. *Commodities Magazine*, 219 Parkade, Cedar Falls, IA 50613.
- [1984] Lane G., Lane's stochastics. *Technical Analysis of Stocks and Commodities*, pp 87–90.
- [1990] Lang K.J., Waibel A.H., Hinton G.E., A time-delay neural network architecture for isolated word recognition. *Neural Networks*, **3**, pp 33–43.
- [1987] Lapedes A., Farber R., Nonlinear signal processing using neural networks: Prediction and modeling. Technical Report, LA-UR87-2662, Los Alamos, New Mexico.
- [1989] LeCun Y., Boser B., Denker J.S., Henderson D., Howard R.E., Hubbard W., Jackel L.D., Backpropagation applied to handwritten zip code recognition. *Neural Computation*, **1**, (4), pp 541–551.
- [1991] Lee Y., Oh S., Kim M., The effect of initial weights on premature saturation in back propagation learning. *Int Jt Conf Neural Netw*, **1**, pp 65–70.

- [2001] Lee J.W., Stock price prediction using reinforcement learning. ISIE 2001, School of Computer Science and Engineering, Sungshin Women's University, Seoul, South Korea.
- [2007] Lee C-T., Chen Y-P., The efficacy of neural networks and simple technical indicators in predicting stock markets. *International Conference on Convergence Information Technology*, pp 2292–2297.
- [2009] Lee M-C., Using support vector machine with a hybrid feature selection method to the stock trend prediction. *Expert Systems with Applications*, **36**, (8), pp 10896–10904.
- [2010] Liao Z., Wang J., Forecasting model of global stock index by stochastic time effective neural network. *Expert Systems with Applications*, **37**, (1), pp 834–841.
- [2008] Lin H-T., Li L., Support vector machinery for infinite ensemble learning. *Journal of Machine Learning Research*, **9**, pp 285–312.
- [2009] Lin X., Yang Z., Song Y., Short-term stock price prediction based on echo state networks. *Expert Systems with Applications*, **36**, (3), pp 7313–7317.
- [2007] Liu J.N.K., Kwong R.W.M., Automatic extraction and identification of chart patterns towards financial forecast. *Applied Soft Computing*, **7**, pp 1197–1208.
- [2016] Liu C.D., Wang J.H., Xiao D., Liang Q., Forecasting S&P 500 stock index using statistical learning models. *Open Journal of Statistics*, **6**, pp 1067–1075.
- [1991] Lo A.W., Long-term memory in stock market prices. *Econometrica*, **59**, (5), pp 1279–1313.
- [2002] Lo A.W., Mackinlay A.C., A non-random walk down wall street. 5th edition, Princeton University Press.
- [2008] Lo A.W., Hedge funds, systemic risk, and the financial crisis of 2007-2008. Written Testimony of A.W. Lo, Prepared for the US House of Representative.
- [2008] Lo A.W., Patel P.N., 130/30: The new long-only. *The Journal of Portfolio Management*, pp 12–38.
- [2010] Lo A.W., Hasanhodzic J., The evolution of technical analysis: Financial prediction from Babylonian tablets to Bloomberg terminals. John Wiley and Sons, USA.
- [1995a] Longerstaey J., Zangari P., Five questions about RiskMetrics. Morgan Guaranty Trust Company, Market Risk Research, JPMorgan.
- [1995b] Longerstaey J., More L., Introduction to RiskMetrics. 4th edition, Morgan Guaranty Trust Company, New York.
- [1996] Longerstaey J., Spencer M., RiskMetrics: Technical document. Fourth Edition, Morgan Guaranty Trust Company, New York.
- [1996] Longin F., The asymptotic distribution of extreme stock market returns. *Journal of Business*, **69**, pp 383–408.
- [1994] Loretan M., Phillips P., Testing the covariance stationarity of heavy-tailed time series. *Journal of Empirical Finance*, **1**, pp 211–248.
- [2009] Lu C-J., Lee T-S., Chiu C-C., Financial time series forecasting using independent component analysis and support vector regression. *Decision Support Systems*, **47**, (2), pp 115–125.
- [2006] Lu Dang Khoa N., Sakakibara K., Nishikawa I., Stock price forecasting using back propagation neural networks with time and profit based adjusted weight factors. *SICE-ICASE International Joint Conference*, Oct., Bexco, Busan, Korea, pp 5484–5488.

- [1988] Lukac L., Brorsen B., Irwin S., A test of futures market disequilibrium using twelve different technical trading systems. *Applied Economics*, **20**, (5), pp 623–639.
- [2006] Lukosevicius M., Popovici D., Jaeger H., Siewert U., Time warping invariant echo state networks Technical Report No. 2, Jacobs University Bremen.
- [2007] Lukosevicius M., Echo state networks with trained feedbacks. Technical Report No. 4, Jacobs University Bremen.
- [2009] Lukosevicius M., Jaeger H., Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, **3**, (3), pp 127–149.
- [2012] Lukosevicius M., A practical guide to applying echo state networks. Jacobs University Bremen, Germany.
- [2012] Lukosevicius M., Jaeger H., Schrauwen B., Reservoir computing trends. *Kunstliche Intelligenz*, Springer, **26**, (4), pp 365–371.
- [1996] Lux T., The stable Paretian hypothesis and the frequency of large returns: An examination of major German stocks. *Applied Economics Letters*, **6**, pp 463–475.
- [2003] Lux T., The multi-fractal model of asset returns: Its estimation via GMM and its use for volatility forecasting. Economics Working Paper No. 2003-13, Christian-Albrechts-Universitat Kiel.
- [2012] Lye C-T., Hooy C-W., Multifractality and efficiency: Evidence from Malaysian sectoral indices. *Int. Journal of Economics and Management*, **6**, (2), pp 278–294.
- [1998] Maass W., Orponen P., On the effect of analog noise in discrete-time analog computations. *Neural Computation*, **10**, pp 1071–1095.
- [2002] Maass W., Natschlager T., Markram H., Real-time computing without stable states: a new framework for neural computation based on perturbations. *Neural Computation*, **14**, (11), pp 2531–2560.
- [2005a] Maass W., Joshi P., Sontag E.D., Computational aspects of feedback in neural circuits. Working Paper, It was published in 2007 in *PLoS Computational Biology*.
- [2005b] Maass W., Joshi P., Sontag E.D., Principles of real-time computing with feedback applied to cortical microcircuit models. Conference Paper in *Advances in neural information processing systems*, January.
- [1977] Mackey M.C., Glass L., Oscillation and chaos in physiological control systems. *Science*, **197**, (4300), pp 287–289.
- [1993] Maheswaran S., Sims C., Empirical implications of arbitrage-free asset markets. in P.C.B. Phillips, ed., *Models, Methods and Applications of Econometrics*, Cambridge, Basil Blackwell, pp 301–316.
- [2011] Maknickiene N., Vytautas Rutkauskas A., Maknickas A., Investigation of financial market prediction by recurrent neural network. *Innovative Infotechnologies for Science, Business and Education*, **2**, (11), pp 3–8.
- [1982] Makridakis S., Andersen A., Carbon R., Fildes R., Hibon M., Lewandowski R., Newton J., Parzen R., Winkler R., The accuracy of extrapolation (time series) methods: Results of a forecasting competition. *Journal of Forecasting*, **1**, pp 111–153.
- [2018] Makridakis S., Spiliotis E., Assimakopoulos V., Statistical and machine learning forecasting methods: Concerns and ways forward. *PLoS ONE*, **13**, (3).
- [1989] Mallat S.G., A theory for multiresolution signal decomposition: The wavelet representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **11**, (7), pp 674–693.

- [1990] Mallat S.G., Hwang W.L., Singularity detection and processing with wavelets. Technical Report No. 549, Computer Science Department, New York University.
- [1992] Mallat S.G., Hwang W.L., Singularity detection and processing with wavelets. *IEEE Trans. Inform. Theory*, **38**, pp 617–643.
- [1992b] Mallat S.G., Zhong S., Complete signal representation with multiscale edges. *IEEE Trans.*, PAMI **14**, pp 710–732.
- [1960] Mandelbrot B.B., The Pareto-Levy law and the distribution of income. *International Economic Review*, **1**.
- [1963a] Mandelbrot B.B., New methods in statistical economics. *The Journal of Political Economy*, **71**.
- [1963] Mandelbrot B.B., The variation of certain speculative prices. *Journal of Business*, **36**, (4), pp 394–419.
- [1968] Mandelbrot B.B., van Ness J., Fractional Brownian motions, fractional noises and applications. *SIAM Review*, **10**, (4), pp 422–437.
- [1971] Mandelbrot B.B., When can price be arbitrated efficiently? A limit to the validity of the random walk and martingale models. *Re. Econom. Statist.*, **53**, pp 225–236.
- [1974] Mandelbrot B.B., Intermittent turbulence in self similar cascades: Divergence of high moments and dimension of the carrier. *Journal of Fluid Mechanics*, **62**, (2), pp 331–358.
- [1975] Mandelbrot B.B., Les objets fractals: forme, hasard et dimension. Paris, Flammarion.
- [1979] Mandelbrot B.B., Taqqu M., Robust R/S analysis of long-run serial correlation. in *Proceedings of the 42nd Session of the International Statistical Institute*, , pp 69–104, Manila, Bulletin of the I.S.I..
- [1982] Mandelbrot B.B., The fractal geometry of nature. W.H. Freeman and Company, New York.
- [1997] Mandelbrot B.B., Fisher A., Calvet L., A multifractal model of asset returns. Cowles Foundation Discussion Paper No. 1164.
- [2004] Mandelbrot B.B., The (mis)behavior of markets, a fractal view of risk, ruin and reward. Basic Books.
- [2005] Manimaran P., Panigrahi P.K., Parikh J.C., Wavelet analysis and scaling properties of time series. *Phys. Rev. E*, **72**, 046120, pp 1–5.
- [2009] Manimaran P., Panigrahi P.K., Parikh J.C., Multiresolution analysis of fluctuations in non-stationary time series through discrete wavelets. *Physica A*, **388**, pp 2306–2314.
- [2000] Mantegna R.N., Stanley H.E., An introduction to econophysics: Correlation and complexity in finance. Cambridge University Press, Cambridge.
- [2009] Marwan N., Donges J.F., Zou Y., Donner R.V., Kurths J., Complex network approach for recurrence analysis of time series. *Physics Letters A*, **373**, (46), pp 4246–4254.
- [2003] Matia K., Ashkenazy Y., Stanley H.E., Multifractal properties of price fluctuations of stocks and commodities. *Europhysics Letters*, **61**, (3), pp 422–428.
- [2004] Matos J.A.O., Gama S.M.A., Ruskin H., Duarte J., An econophysics approach to the Portuguese stock index-psi-20. *Physica A*, **342**, (3-4), pp 665–676.
- [2008] Matos J.A.O., Gama S.M.A., Ruskin H.J., Al Sharkasi A., Crane M., Time and scale Hurst exponent analysis for financial markets. *Physica A*, **387**, pp 3910–3915.

- [1943] McCulloch W., Pitts W., A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, **5**, pp 115–133.
- [1996] McCulloch J.H., Financial applications of stable distributions. in G.S. Maddala, C.R. Rao, eds., *Handbook of Statistics*, **14**, Elsevier, pp 393–425.
- [1997] McCulloch J.H., Measuring tail thickness to estimate the stable index α : A critique. *Journal of Business & Economic Statistics*, **15**, (1), pp 74–81.
- [2010] Menkhoff L., The use of technical analysis by fund managers: International evidence. *Journal of Banking & Finance*, **34**, (11), pp 2573–2586.
- [1981] Merton R.C., On market timing and investment performance: An equilibrium theory of value for market forecasts. *Journal of Business*, **54** (3), pp 363–406.
- [2007] Metghalchi M., Glasure Y., Garza-Gomez X., Chen C., Profitable technical trading rules for the Austrian stock market. *International Business & Economics Research Journal*, **6**, (9), pp 49–58.
- [2012] Metghalchi M., Marcucci J., Chang Y-H., Are moving average trading rules profitable? Evidence from the European stock markets. *Applied Economics*, **44**, (12), pp 1239–1559.
- [2015] Metghalchi M., Chen C., Hayes L.A., History of share prices and market efficiency of the Madrid general stock index. *International Review of Financial Analysis*, **40**, pp 178–184
- [2018] Metghalchi M., Hajilee M., Hayes L.A., Return predictability and efficient market hypothesis: Evidence from Iceland. *The Journal of Alternative Investments*, **21**, (1), pp 68–78.
- [1996] Michalewicz Z., Genetic algorithms + data structures = evolution programs. Springer Verlag.
- [2012] Mikolov T., Statistical language models based on neural networks. PhD thesis, Brno University of Technology.
- [2003] Mikosch T., Starica C., Long-range dependence effects and ARCH modeling. In *Theory and Applications of Long-range Dependence*, Birkhauser Boston, Boston, pp 439–459.
- [2018] Miller J., Recurrent models don't need to be recurrent. Berkeley Artificial Intelligence Research.
- [2019] Miller J., Hardt M., Stable recurrent models. arXiv:1805.10369.
- [2009] Mingguang L., Gaoyang L., Artificial neural network co-optimisation algorithm based on differential evolution. In *Second International Symposium on Computational Intelligence and Design*, pp 256–559.
- [2014] Mirza M., Osindero S., Conditional generative adversarial networks. Working Paper, arXiv:1411.1784.
- [1997] Mitchell T.M., Machine learning. McGraw-Hill.
- [2009] Mitra S.K., Optimal combination of trading rules using neural networks. *International Journal of Business Research*, **2**, (1), pp 86–99.
- [2016] Moghaddam A.H., Moghaddam M.H., Esfandiyari M., Stock market index prediction using artificial neural network. *Journal of Economics, Finance and Administrative Science*, **21**, (41), pp 89–93.
- [1989] Montana D.J., Davis L., Training feedforward neural networks using genetic algorithms. *Int Jt Conf Artif Intell*, **89**, pp 762–767.
- [2001] Moody J., Saffell M., Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, **12**, (4), pp 875–889.

- [2006] Moyano L.G., de Souza J., Duarte Queiros S.M., Multi-fractal structure of traded volume in financial markets. *Physica A*, **371**, pp 118–121.
- [1990] Muller U.A., Dacorogna M.M., Olsen R.B., Pictet O.V., Schwarz M., Morgenegg C., Statistical study of foreign exchange rates, empirical evidence of a price change scaling law, and intraday analysis. *Journal of Banking and Finance*, **14**, pp 1189–1208.
- [1997] Muller U.A., Dacorogna M.M., Dave R.D., Olsen R.B., Pictet O.V., von Weizsacker J.E., Volatilities of different time resolutions: Analyzing the dynamics of market components. *Journal of Empirical Finance*, **4**, pp 213–239.
- [2018] Murdoch W.J., Liu P.J., Yu B., Beyond word importance: Contextual decomposition to extract interactions from lstms. International Conference on Learning Representations.
- [2009] Murguia J.S., Perez-Terrazas J.E., Rosu H.C., Multifractal properties of elementary cellular automata in a discret wavelet approach of MF-DFA. *EPL Journal*, **87**, pp 2803–2808.
- [1999] Murphy J.J., Technical analysis of the financial markets: A comprehensive guide to trading methods and applications. Prentice Hall Press.
- [2003] Murtagh F., Stark J.L., Renaud O., On neuro-wavelet modeling. Working Paper, School of Computer Science, Queen's University Belfast.
- [1991] Muzy J.F., Bacry E., Arneodo A., Wavelets and multifractal formalism for singular signals: Application to turbulence data. *Phys. Rev. Lett.*, **67**, (25), pp 3515–3518.
- [1993] Muzy J.F., Bacry E., Arneodo A., Multifractal formalism for fractal signals: The structure-function approach versus the wavelet-transform modulus-maxima method. *Phys. Rev. E*, **47**, (2), pp 875–884.
- [2011] Nair B.B., Sai S.G., Naveen A., Lakshmi A., Venkatesh G., Mohandas V., A GA-artificial neural network hybrid system for financial time series forecasting. *Inf Techno Mob Commun.*, **147**, pp 499–506.
- [1990] Nelson D.B., ARCH models as diffusion approximations. *Journal of Econometrics*, **45**, pp 7–38.
- [1991] Nelson D.B., Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, **59**, pp 347–370.
- [2002] Ng A.Y., Jordan M.I., On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. Working Paper, University of California, Berkeley.
- [2013] Niere H.M., A multifractality measure of stock market efficiency in Asean region. *European Journal of Business and Management*, **5**, (22), pp 13–19.
- [2005] Norouzzadeh P., Jafari G.R., Application of multifractal measures to Tehran price index. *Physica A*, **356**, pp 609–627.
- [2006] Norouzzadeh P., Rahmani B., A multifractal detrended fluctuation description of Iranian rial-US dollar exchange rate. *Physica A*, **367**, pp 328–336.
- [2012] Oh G., Eom C., Havlin S., Jung W-S., Wang F., Stanley H.E., Kim S., A multifractal analysis of Asian foreign exchange markets. *European Physical Journal B*, pp 85–214.
- [2003] Olson D., Mossman C., Neural network forecasts of Canadian stock returns using accounting ratios. *International Journal of Forecasting*, **19**, pp 453–465.

- [2013] Fernando F.E.B., Freitas A.A., Rice j., Johnson C.G., New sequential covering strategy for inducing classification rules with ant colony algorithms. *IEEE Transactions on Evolutionary Computation*, **17**, (1), pp 64–76.
- [2002] Palmer A., Montano J.J., Redes neuronales artificiales aplicadas al analisis de datos. Doctoral Dissertation. University of Palma de Mallorca.
- [2014] Paluch M., Jackowska-Strumitto L., The influence of using fractal analysis in hybrid MLP model for short-term forecast of close prices on Warsaw stock exchange. In *Proceedings of the Federated Conference on Computer Science and Information Systems 2014*, Warsaw, Poland, pp 111–118.
- [2015] Paluch M., Jackowska-Strumitto L., Intelligent information system for stock exchange data processing and presentation. In *Proceedings of the 8th International Conference on Human System Interaction*, HSI 2015, Warsaw, Poland, pp 238–243.
- [2016] Paluch M., Jackowska-Strumitto L., Decision system for stock data forecasting based on Hopfield artificial neural network. *Inform. Control Measure. Econ. Environ. Protect.*, **6**, pp 28–33.
- [2018a] Paluch M., Jackowska-Strumitto L., Intelligent decision system for stock exchange data processing and presentation. In *Human-Computer Systems Interaction. Backgrounds and Applications 4; Series: Advances in Intelligent Systems and Computing*, Z.S. Hippe, J.L. Kulikowski, T. Mroczek, Eds. Springer International Publishing, Switzerland, **551**, pp 80–90.
- [2018b] Paluch M., Jackowska-Strumitto L., Hybrid models combining technical and fractal analysis with ANN for short-term prediction of close values on Warsaw stock exchange. *Applied Sciences*, **8**, 2473.
- [2001] Panas E., Estimating fractal dimension using stable distributions and exploring long memory through ARFIMA models in Athens stock exchange. *Applied Financial Economics*, **11**.
- [1985] Parker D.B., Learning logic report TR-47. MIT Press
- [1997] Partridge D., Krzanowski W.J., Software diversity: Practical statistics for its measurement and exploitation. *Information & Software Technology*, **39**, (10), pp 707–717.
- [2013] Pascanu R., Mikolov T., Bengio Y., On the difficulty of training recurrent neural networks. in *Proceedings of the 30th International Conference on Machine Learning*.
- [2000] Pasquini M., Serva M., Clustering of volatility as a multiscale phenomenon. *European Physical Journal B*, **16**, (1), pp 195–201.
- [2015] Patel J., Shah S., Thakkar P., Kotecha K., Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques. *Expert Systems with Applications*, **42**, pp 259–268.
- [2008a] Pedersen M.E.H., Chipper
eld A.J., Parameter tuning versus adaptation: Proof of principle study on differential evolution. Technical Report HL0802, Hvass Laboratories.
- [2008b] Pedersen M.E.H., Chipper
eld A.J., Tuning differential evolution for artificial neural networks. Hvass Laboratories Technical Report no. HL0803.
- [1994] Peng C.K., Buldyrev S.V., Havlin S., Simons M., Stanley H.E., Goldberger A.L., Mosaic organization of DNA nucleotides. *Physical Review E*, **49**, (2), pp 1685–1689.

- [1991-96] Peters E.E., Chaos and order in the capital markets. second edition, John Wiley & Sons.
- [1994] Peters E.E., Fractal market analysis: Applying chaos theory to investment and economics. John Wiley & Sons.
- [1993] Plummer T., Forecasting financial markets. Kogan Page Ltd.
- [2015] Polydoros A.S., Nalpantidis L., Kruger V., Advantages and limitations of reservoir computing on model learning for robot control. IROS Workshop on Machine Learning in Planning and Control of Robot Motion
- [1993] Pomerleau D.A., Knowledge-based training of artificial neural networks for autonomous robot driving. in J. Connell and S. Mahadevan, (eds.), *Robot Learning*, pp 19–43, Kluwe Academic Publishers.
- [1992] Press W.H., Teukolsky S.A., Vetterling W.T., Flannery B.P., Numerical recipes. 2nd ed. Cambridge, Cambridge University Press.
- [2001] Qi M., Predicting US recessions with leading indicators via neural network models. *International Journal of Forecasting*, **17**, pp 383–401.
- [2016] Qiu M., Song Y., Predicting the direction of stock market index movement using an optimized artificial neural network model. *PLoS ONE* **11**, (5): e0155133. doi:10.1371/journal.
- [1993] Quinlan J.R., C4.5: Programs for machine Learning. Morgan Kaufmann, San Francisco, CA.
- [1998] Quinlan J.R., Induction of decision trees. *Machine Learning*, **1**, (1), pp 81–106.
- [1993] Rabemananjara R., Zakoian J., Threshold ARCH models and asymmetries in volatility. *Journal of Applied Econometrics*, **8**, pp 31–49.
- [2015] Radford A., Metz L., Chintala S., Unsupervised representation learning with deep convolutional generative adversarial networks. Working Paper, arXiv:1511.06434.
- [1999] Ramsey J.B., The contribution of wavelets to the analysis of economic and financial data. *Phil. Trans. R. Soc.*, **357**, pp 2593–2606.
- [2017] Rani T., Kumar N., *International Journal of Advanced Research in Computer and Communication Engineering*, **6**, (5), pp 616–623.
- [1995] Refenes A.N., Neural networks in capital markets. Wiley.
- [1994] Refenes A.N., Zaprani A.D., Francis G., Stock performance modeling using neural network: A comparative study with regression models. *Neural Network*, **5**, pp 961–970.
- [1997] Refenes A.N., Bentz Y., Bunn D.W., Burgess A.N., Zaprani A.D., Financial time series modeling with discounted least squares backpropagation. *Neurocomputing*, **14**, (2), pp 123–138.
- [2008] Reinhart R.F., Steil J.J., Recurrent neural associative learning of forward and inverse kinematics for movement generation of the redundant PA-10 robot. In *Learning and Adaptive Behaviors for Robotic Systems (LAB-RS)*, pp 35–40.
- [2009] Reinhart R.F., Steil J.J., Attractor-based computation with reservoirs for online learning of inverse kinematics. Research Institute for Cognition and Robotics (CoR-Lab), Bielefeld University.
- [2009b] Reinhart R.F., Steil J.J., Reaching movement generation with a recurrent neural network based on learning inverse kinematics for the humanoid robot icub. In *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, pp 323–330.

- [2011] Reinhart R.F., Reservoir computing with output feedback. Technische Fakultät, Universität Bielefeld.
- [2011] Reinhart R.F., Steil J.J., A constrained regularization approach for input-driven recurrent neural networks. *Differential Equations and Dynamical Systems*, **19**, pp 27–46.
- [1999] Resnick S., Samorodnitsky G., Xue F., How misleading can sample ACFs of stable MAs be? (Very!). *Ann. Appl. Prob.*, **9**, pp 797–817.
- [2000] Resnick S., Van Den Berg E., Sample correlation behavior for the heavy tailed general bilinear process. *Comm. Statist. Stochastic Models*, **16**, pp 233–258.
- [2014] Rezende D.J., Mohamed S., Wierstra D., Stochastic backpropagation and approximate inference in deep generative models. Technical Report, arXiv:1401.4082.
- [2016] Ribeiro M.T., Singh S., Guestrin C., Why should I trust you?: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp 1135–1144.
- [1995] Rice J.A., Mathematical statistics and data analysis. Thomson Information, Second Edition.
- [1987] Robinson A.J., Fallside F., The utility driven dynamic error propagation network. Cambridge University Engineering Department
- [2012] Rodan A., Tiño P., Simple deterministically constructed cycle reservoirs with regular jumps. The University of Birmingham.
- [2010] Rokach L., Pattern classification using ensemble methods. World Scientific, Singapore.
- [1962] Rosenblatt F., Principles of perceptrons. Spartan, Washington, DC.
- [1999] Ross B.J., A lamarckian evolution strategy for genetic algorithms. In Lance D. Chambers, editor, *Practical Handbook of Genetic Algorithms: Complex Coding Systems*, **3**, pp 1–16. CRC Press, Boca Raton, Florida.
- [2017] Ross A.S., Hughes M.C., Doshi-Velez F., Right for the right reasons: Training differentiable models by constraining their explanations. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17*, pp 2662–2670.
- [2012] Ruiz E.J., Hristidis V., Castillo C., Gionis A., Jaimes A., Correlating financial time series with microblogging activity. In *Proceedings of the fifth ACM international conference on web search and data mining*, ACM, pp 513–522.
- [1986] Rumelhart D., McClelland J., Parallel distributed processing. MIT Press, Cambridge, Mass.
- [1986b] Rumelhart D.E., Hinton G.E., Williams R.J., Learning internal representations by error propagation. in Parallel distributed processing: explorations in the microstructure of cognition, **1**, MIT Press, Cambridge, Mass, pp 318–362.
- [1994] Rumelhart D., Widrow B., Lehr M., The basic ideas in neural networks *Communications of the ACM*, **37**, (3), pp 87–92.
- [2004] Samaras G.D., Matsatsinis N.F., An intelligent decision support system for portfolio management. *Operational Research*, **4**, (3), pp 357–371.
- [2019] Ryll L., Seidens S., Evaluating the performance of machine learning algorithms in financial market forecasting: A comprehensive survey. Working Paper.

- [2009] Salakhutdinov R., Hinton G.E., Deep Boltzmann machine. in *AISTATS 2009*, pp 448–455.
- [1959] Samuel A.L., Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, **3**, (3), pp 535–554.
- [2015] Sangwan K.S., Saxena S., Kant G., Optimization of machining parameters to minimize surface roughness using integrated ANN-GA approach. in *Proceedings of the 22nd CIRP Conference on Life Cycle Engineering*, **29**, Sydney, Australia, pp 305–310.
- [1990] Schapire R.E., The strength of weak learnability. *Machine Learning*, **5**, (2), pp 197–227.
- [1992] Schmidhuber J., A fixed size storage $O(n^3)$ time complexity learning algorithm for fully recurrent continually running network. *Neural Computation*, **4**, (2), pp 243–248.
- [1992b] Schmidhuber J., Learning factorial codes by predictability minimization. *Neural Computation*, **4**, (6), pp 863–879.
- [2007] Schmidhuber J., Wierstra D., Gagliolo M., Gomez F.J., Training recurrent networks by Evolino. *Neural Computation*, **19**, (3), pp 757–779.
- [1999] Schmitt F., Schertzer D., Lovejoy S., Multifractal analysis of foreign exchange data. *Applied Stochastic Models and Data Analysis*, **15**, pp 29–53.
- [1990] Schoneburg E., Stock price prediction using neural networks: A project report. *Neurocomputing*, **2**, (1), pp 17–27.
- [2005] Schulmeister S., Components of the profitability of technical currency trading. WIFO Working Papers, No. 263, December.
- [2009] Schumaker R.P., Chen H-C., Textual analysis of stock market prediction using breaking financial news: The AZFin text system. *ACM Transactions on Information Systems*, **27**, (2), Article No 12.
- [1996] Schwager W.F., Technical analysis. Wiley.
- [2008] Senol D., Ozturan M., Stock price direction prediction using artificial neural network approach: The case of Turkey. *Journal Artif Intell*, **1**, (2), pp 70–77.
- [2019] Shah D., Isah H., Zulkernine F., Stock market analysis: A review and taxonomy of prediction techniques. *International Journal of Financial Studies*.
- [2011] Shalev-Shwartz S., Singer Y., Srebro N., Cotter A., Pegasos: primal estimated sub-gradient solver for SVM. *Mathematical Programming*, **127**, (1), pp 3–30.
- [2012] Shen S., Jiang H., Zhang T., Stock market forecasting using machine learning algorithms. Working Paper, Stanford University.
- [2008] Sherrod P.H., Dtree: predictive modeling software.
- [1991] Shin Y., Ghosh J., The pi-sigma network: An efficient higher-order neural network for pattern classification and function approximation. *International Joint Conference on Neural Networks*.
- [1998] Shin K-S., Kim K-J., Han I., Financial data mining using genetic algorithms technique: Application to KOSPI 200. Korea Advanced Institute of Science and Technology
- [2000] Shin T., Han I., Optimal signal multi-resolution by genetic algorithms to support financial neural networks for exchange-rate forecasting. *Expert Syst. Appl.*, **18**, pp 257–269.

- [2002] Shipp C.A., Kuncheva L.I., Relationships between combination methods and measures of diversity in combining classifiers. *Information Fusion*, **3**, (2), pp 135–148.
- [2012] Shynkevich A., Performance of technical analysis in growth and small cap segments of the US equity market. *Journal of Banking & Finance*, **36**, pp 193–208.
- [2012] Si T., Hazra S., Jana N.D., Artificial neural network training using differential evolutionary algorithm for classification. Department of Information Technology National Institute of Technology, Durgapur West Bengal, India.
- [1991] Siegelmann H.T., Sontag E.D., Turing computability with neural nets. *Applied Mathematics Letters*, **4**, (6), pp 77–80.
- [2007] Siewert U., Wustlich W., Echo-state networks with bandpass neurons: Towards generic time-scale-independent reservoir structures. Internal Status Report, *PLANET Intelligent Systems GmbH*.
- [1996] Skalak D.B., The sources of increased accuracy for two proposed boosting algorithms. In *Working Notes of the AAAI'96 Workshop on Integrating Multiple Learned Models*, Portland, OR.
- [2008] Slowik A., Bialko M., Training of artificial neural networks using differential evolution algorithm. Department of Electronics and Computer Science, Koszalin University of Technology, Koszalin, Poland.
- [1973] Sneath P.H.A., Sokal R.R., Numerical taxonomy: The principles and practice of numerical classification. W.H. Freeman, San Francisco, CA.
- [2015] Sondwale P., Overview of predictive and descriptive data mining Techniques. *International Journal of Advanced Research in Computer Science and Software*, **5**, (4), pp. 262–265.
- [2008] Sorokina D., Caruana R., Riedewald M., Fink D., Detecting statistical interactions with additive groves of trees. In *Proceedings of the 25th international conference on Machine learning*, pp 1000–1007
- [2004] Steil J.J., Backpropagation-decorrelation: Online recurrent learning with $O(N)$ complexity. Neuroinformatics Group, Faculty of Technology University of Bielefeld. In *Proceedings of the IEEE International Joint Conference on Artificial Neural Networks*, **2**, pp 843–848.
- [2005] Steil J.J., Memory in backpropagation-decorrelation $O(N)$ efficient online recurrent learning. In *Proceedings of the 15th International Conference on Artificial Neural Networks*, **3697** of LNCS, pp 649–654.
- [1997] Steiner M., Wittkemper H.G., Portfolio optimization with a neural network implementation of the coherent market hypothesis. *European journal of operational research*, **100**, (1), pp 27–40.
- [2012] Stepanek J., Stovicek J., Cimler R., Application of genetic algorithms in stock market simulation. Cyprus International Conference on Educational Research (CY-ICER-2012) North Cyprus, US08-10.
- [2014] Stosic D., Stosic D., Stosic T., Stanley H.E., Multifractal analysis of managed and independent float exchange rates. Department of Physics, Boston University. Published in 2015 in *Physica A: Statistical Mechanics and its Applications*, **428**, pp 13–18.
- [1998] Struzik Z.R., Removing divergence in the negative moments of the multi-fractal partition function with the wavelet transformation. Working Paper INS-R9803, Centrum voor Wiskunde en Informatica, Amsterdam, The Netherlands.
- [1999] Struzik Z.R., Local effective Holder exponent estimation on the wavelet transform maxima tree. in *Fractals: Theory and Applications in Engineering*, eds., M. Dekking, J. Levy Vehel, E. Lutton, C. Tricot, Springer Verlag, pp 93–112.

- [2000] Struzik Z.R., Determining local singularity strengths and their spectra with the wavelet transform. *Fractals*, **8**, (2), pp 163–179.
- [2002] Struzik Z.R., Siebes A., Wavelet transform based multifractal formalism in outlier detection and localisation for financial time series. *Physica A*, **309**, pp 388–402.
- [2003] Struzik Z.R., Econophysics vs cardiophysics: The dual face of multifractality. Working Paper, Centrum voor Wiskunde en Informatica, Amsterdam, The Netherlands.
- [1999] Sun X., The Lasso and its implementation for neural networks. Department of Statistics, University of Toronto.
- [2009] Sussillo D., Abbott L.F., Generating coherent patterns of activity from chaotic neural networks. *Neuron*, **63**, (4), pp 544–557.
- [2008] Sutskever I., Hinton G., Taylor G.W., The recurrent temporal restricted boltzmann machine. In *Advances in Neural Information Processing Systems 21*, (NIPS 2008), pp 1601–1608.
- [2011] Sutskever I., Martens J., Hinton G., Generating text with recurrent neural networks. In *ICML*, 2011.
- [2012] Sutskever I., Training recurrent neural networks. PhD thesis, University of Toronto.
- [2014] Sutskever I., Vinyals O., Le Q.V., Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pp 3104–3112.
- [1984] Sutton R.S., Temporal credit assignment in reinforcement learning. Ph.D. thesis, University of Massachusetts, Amherst.
- [1988] Sutton R.S., Learning to predict by the methods of temporal differences. *Machine Learning*, **3**, pp 9–44.
- [1986] Sweeney R., Beating the foreign exchange market. *The Journal of Finance*, **41**, (1), pp 163–182.
- [1995] Tan H., Neural network model for stock forecasting. Master's Thesis, Texas Tech University.
- [2009] Tan C., Financial time series forecasting using improved wavelet neural network. Master's Thesis, Aarhus Universitet.
- [2007] Tanaka-Yamawaki M., Tokuoka S., Adaptive use of technical indicators for the prediction of intra-day stock prices. *Physica A*, **383**, pp 125–133.
- [2014] Tang F., Tiño P., Chen H., Learning the deterministically constructed echo state networks. International Joint Conference on Neural Networks.
- [1981] Taqqu M.S., Self-similar processes and related ultraviolet and infrared catastrophes. In *Random Fields : Rigorous Results in Statistical Mechanics and Quantum Field Theory, Colloquia Mathematica Societatis Janos Bolya*, Vol. 27, Book 2, pp 1027–1096.
- [1995] Taqqu M.S., Teverovsky V., Willinger W., Estimators for long-range dependence: An empirical study. *Fractals*, **3**, (4), pp 785–798.
- [1986] Taylor S.J., Modelling financial time series. New York, John Wiley & Sons.
- [2007] Taylor G.W., Hinton G.E., Roweis S., Modeling human motion using binary latent variables. in *Advances in Neural Information Processing Systems*, **19**, pp 1345–1352, MIT Press, Cambridge.
- [2010] Teixeira L.A., De Oliveira A.L.I., A method for automatic stock trading combining technical analysis and nearest neighbor classification. *Expert Systems with Applications*, **37**, (10), pp 6885–6890.

- [2008] Terman D.H., Izhikevich E.M., State space. Scholarpedia, [http://www.scholarpedia.org/article/Phase space](http://www.scholarpedia.org/article/Phase%20space), Ohio State University.
- [2000] Thomas N.B., Encyclopedia of chart patterns. John Wiley & Sons.
- [2004] Timmermann A., Granger C.W.J., Efficient market hypothesis and forecasting. *International Journal of Forecasting*, **20**, pp 15–27.
- [2017] Tolstikhin I.O., Gelly S., Bousquet O., Simon-Gabriel C-J., Scholkopf B., Adagan: Boosting generative models. in *Advances in Neural Information Processing Systems*, pp 5424–5433.
- [2018] Tsang M., Cheng D., Liu Y., Detecting statistical interactions from neural network weights. Conference Paper at ICLR 2018, Department of Computer Science, University of Southern California.
- [2002] Tsay R.S., Analysis of financial time series. John Wiley & Sons, Hoboken, New Jersey.
- [1995] Tsibouris G., Zeidenberg M., Testing the efficient markets hypothesis with gradient descent algorithms. in *Neural networks in the capital markets*, **8**, Wiley, pp 127–136.
- [2006] Turiel A., Perez-Vicente C.J., Grazzini J., Numerical methods for the estimation of multifractal singularity spectra on sampled data: A comparative study. *Journal of Computational Physics*, **216**, pp 362–390.
- [2016] Van den Poel D., Chesterman C., Koppen M., Ballings M., Equity price direction prediction for day trading: Ensemble classification using technical analysis indicators with interaction effects. in *2016 IEEE Congress on Evolutionary Computation (CEC)*.
- [1997] Vandewalle N., Ausloos M., Coherent and random sequences in financial fluctuations. *Physica A*, **246**, (3), pp 454–459.
- [1998] Vandewalle N., Ausloos M., Crossing of two mobile averages: A method for measuring the robustness exponent. *Phys. Rev.*, **58**, pp 177–188.
- [1998b] Vandewalle N., Ausloos M., Multi-affine analysis of typical currency exchange rates. *European Physical Journal B*, **4**, (2), pp 257–261.
- [1998c] Vandewalle N., Ausloos M., Boveroux PH., Detrended fluctuation analysis of the foreign exchange market. Working Paper.
- [1963] Vapnik V., Lerner A., Pattern recognition using generalized portrait method. *Automation and Remote Control*, **24**, pp 774–780.
- [1964] Vapnik V., Chervonenkis A., A note on one class of perceptrons. *Automation and Remote Control*, **25**.
- [1982] Vapnik V., Estimation of dependencies based on empirical data. Springer-Verlag, New York.
- [1989] Vapnik V., Chervonenkis A., Necessary and sufficient conditions for consistency of the method of empirical risk minimization. in *Yearbook of the Academy of Sciences of the USSR on Recognition, Classification, and Forecasting*, 2, Nauka (Moscow), pp 217–249.
- [1992] Vapnik V., Principles of risk minimization for learning theory. AT&T Bell Laboratories.
- [1995-2000] Vapnik V., The nature of statistical learning theory. Information Science and Statistics. Springer-Verlag. First published in 1995.
- [2010] Vivekanandan P., Nedunchezian R., A fast genetic algorithm for mining classification rules in large datasets. *International Journal on Soft Computing*, **1**, (1), pp 10–20.

- [2008] Wang W., Nie S., The performance of several combining forecasts for stock index. in *2008 International Seminar on Future Information Technology and Management Engineering*, pp 450–455.
- [2009] Wang Y., Liu L., Gu R., Analysis of efficiency for Shenzhen stock market based on multifractal detrended fluctuation analysis. *International Review of Financial Analysis*, **18**, pp 271–276.
- [2011] Wang Y., Wei Y., Wu C., Analysis of the efficiency and multifractality of gold markets based on multifractal detrended fluctuation analysis. *Physica A: Statistical Mechanics and its Applications*, **390**, pp 817–827.
- [2011b] Wang Y., Wu C., Pan Z., Multifractal detrending moving average analysis on the US Dollar exchange rates. *Physica A*, **390**, pp 3512–3523.
- [2016] Wang M., Chen Y., Tian L., Jiang S., Tian Z., Du R., Fluctuation behavior analysis of international crude oil and gasoline price based on complex network perspective. *Applied Energy*, **175**, pp 109–127.
- [2016b] Wang Y., Zhang L., van de Weijer J., Ensembles of generative adversarial networks. Working Paper, arXiv:1612.00991.
- [2017] Wang M., Vilela A.L.M., Tian L., Xu H., Du R., A new time series prediction method based on complex network theory. in *Proceedings of the 5th IEEE International Conference on Big Data*, pp 4170–4175.
- [2018] Wang C., Xu C., Yao X., Tao D., Evolutionary generative adversarial networks. Working Paper, arXiv:1803.00657.
- [2008] Wendt H., Contributions of wavelet leaders and bootstrap to multifractal analysis: Images, estimation performance, dependence structure and vanishing moments. Confidence intervals and hypothesis tests. Docteur de l'Université de Lyon, Ecole Normale Supérieure de Lyon, Traitement du Signal - Physique.
- [1974] Werbos P., Beyond regression: New tools for prediction and analysis in the behavioral sciences. PhD thesis, Harvard University.
- [1990] Werbos P., Backpropagation through time: What it does and how to do it. in *Proceedings of the IEEE*, **78**, (10), pp 1550–1560.
- [2000] White H., A reality check for data snooping. *Econometrica*, **68**, pp 1097–1127.
- [1990] Whitley D., Starkweather T., Bogart C., Genetic algorithms and neural networks: optimizing connections and connectivity. *Parallel Computing*, **14**, (3), pp 347–361.
- [1960] Widrow B., Hoff M.E., Adaptive switching circuits. In *WESCON Convention Record*, Part IV, pp 96–104.
- [2005] Wiestra D., Gomez F., Schmidhuber J., Modeling systems with internal state Evolino. In *GECCO*, pp 1795–1802.
- [1978] Wilder W., New concepts in technical trading systems. Trend Research, Greensboro, NC.
- [1989] Williams R.J., Zipser D., A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, **1**, pp 270–280.
- [1990] Williams R.J., Peng J., An efficient gradient-based algorithm for online training of recurrent network trajectories. *Neural Computation*, **2**, (4), pp 490–501.
- [1992] Williams R.J., Zipser D., Gradient-based learning algorithms for recurrent networks and their computational complexity. in *Back-propagation: Theory, architectures and applications*. NJ:Erlbaum, ed. Y.Chauvin and D.E.Rumelhart, chapter 13, pp 433–486.

- [1992b] Williams R.J., Simple statistical gradient following algorithms for connectionist reinforcement learning. *Machine Learning*, **8**, Issue 3-4, pp 229–256.
- [2005] Witkowska D., Marcinkiewicz E., Construction and evaluation of trading systems: Warsaw index futures. *Int. Adv. Econ. Res.*, **11**, pp 83–92.
- [2005] Witten J.H., Frank E., Data mining: Practical machine learning tools and techniques. Morgan Kaufmann, 2 edition.
- [2018] Wojcik P.I., Random projection in deep neural networks. AGH University of Science and Technology in Krakow.
- [1992] Wolpert D.H., Original contribution: Stacked generalization. *Neural Networks*, **5**, (2), pp 241–259.
- [2014] Wu X., Zhu X., Wu G-Q., Ding W., Data mining with big data. *IEEE Transactions on Knowledge and Data Engineering*, **26**, (2), pp 97–107.
- [2008] Wyffels F., Schrauwen B., Verstraeten D., Stroobandt D., Band-pass reservoir computing. In 2008 *IEEE International Joint Conference on Neural Networks*, pp 3204–3209.
- [2009] Wyffels F., Schrauwen B., Design of a central pattern generator using reservoir computing for learning human motion. Electronics and Information Systems Department, Ghent University, Belgium.
- [1995] Xu L., Jordan M.I., Hinton G.E., An alternative model for mixtures of experts. In G. Tesauro, D.S. Touretzky, and T.K. Leen, editors, *Advances in Neural Information Processing Systems 7*, MIT Press, pp 633–640. Cambridge, MA, 1995.
- [1996] Xu L., Jordan M.I., On convergence properties of the EM algorithm for Gaussian mixtures. *Neural Computation*, **8**, (1), pp 129–151.
- [2007] Xu Y., Yang L., Haykin S., Decoupled echo state networks with lateral inhibition. *Neural Networks*, **20**, (3), pp 365–376.
- [2017] Yang B., Gong Z-J., Yang W., Stock markets index prediction using deep neural network ensemble. Paper presented at the 36th Chinese Control Conference, Dalian, China.
- [1996] Yao J.T., Poh H.L., Equity forecasting: A case study on the KLSE index. *Neural Networks in Financial Engineering, Proceedings of 3rd International Conference on Neural Networks in the Capital Markets*, Oct 1995, London, A-P.N. Refenes, Y. Abu-Mostafa, J. Moody and A. Weigend, (eds.), World Scientific, pp 341–353.
- [1998] Yao J.T., Tan C.L., A study on training criteria for financial time series forecasting. Working Paper.
- [2000] Yao J.T., Tan C.L., Time dependent directional profit model for financial time series forecasting. *Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks*, July 2000, Como, Italy, **5**, pp 291–296.
- [2006] Yasuda T., Nakamura K., Kawahara A., Tanaka K., Neural network with variable type connection weights for autonomous obstacle avoidance on a prototype of six-wheel type intelligent wheelchair. *International Journal of Innovative Computing, Information and Control*, **2**, (5), pp 1165–1177.
- [2015] Yosinski J., Clune J., Nguyen A., Fuchs T., Lipson H., Understanding neural networks through deep visualization. arXiv preprint arXiv:1506.06579.
- [2009] Yuan Y., Zhuang X-T., Jin X., Measuring multifractality of stock price fluctuation using multifractal detrended fluctuation analysis. *Physica A*, **388**, (11), pp 2189–2197.

- [1900] Yule G., On the association of attributes in statistics. *Philosophical Transactions of the Royal Society of London*, **194**, pp 257–319.
- [1994] Zadeh L.A., Fuzzy logic, neural networks, and soft computing. *Communications of the ACM*, **37**, (3), pp 77–84.
- [2006] Zeng F., Zhang Y., Stock index prediction based on the analytical center of version space. *Advances in Neural Networks*, **3973**, pp 458–463.
- [2003] Zhang G.P., Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing*, **50**, pp 159–175.
- [2007] Zhang X., Chen Y., Yang J.Y., Stock index forecasting using pso based selective neural network ensemble. in *International Conference on Artificial Intelligence*, pp 260–264.
- [2018] Zhang J., Cui S., Xu Y., Li Q., Li T., A novel data-driven stock price trend prediction system. *Expert Systems with Applications*, **97**, pp 60–69.
- [2018b] Zhang Z., Lui S., Li M., Zhou M., Chen E., Bidirectional generative adversarial networks for neural machine translation. in *Proceedings of the 22nd Conference on Computational Natural Language Learning*, pp 190–199.
- [2019] Zhang K., Zhong G., Dong J., Wang S., Wang Y., Stock market prediction based on generative adversarial network. *Procedia Computer Science*, **147**, pp 400–406.
- [2017a] Zhong X., Enke D., Forecasting daily stock market return using dimensionality reduction. *Expert Syst Appl*, **67**, pp 126–139.
- [2017b] Zhong X., Enke D., A comprehensive cluster and classification mining procedure for daily stock market return forecasting. *Neurocomputing*, **267**, pp 152–168.
- [2019] Zhong X., Enke D., Predicting the daily return direction of the stock market using hybrid machine learning algorithms. Springer Open.
- [2012] Zhou Z-H., Ensemble methods: Foundations and algorithms. Chapman & Hall, Machine Learning & Pattern Recognition Series.
- [2018] Zhou X., Pan Z., Hu G., Tang S., Zhao C., Stock market prediction on high-frequency data using generative adversarial nets. *Mathematical Problems in Engineering*, **2018**, ID 4907423.
- [2008] Zhu X., Wang H., Xu L., Li H., Predicting stock index increments by neural networks: The role of trading volume under different horizons. *Expert Syst. Appl.*, **34**, (4), pp 3043–3054.
- [2006] Zimmermann H.G., Grothmann R., Schaefer A.M., Tietz C., Identification and forecasting of large dynamical systems by dynamical consistent neural networks. in *New Directions in Statistical Signal Processing: From systems to brain*. MIT Press, ed. S. Haykin, J. Principe, T. Sejnowski, J. McWhirter, pp 203–242.
- [2009] Zinkevich M., Smola A., Langford J., Slow learners are fast. In Y. Bengio, D. Schuurmans, J. Lafferty, C.K.I. Williams, and A. Culotta, editors, *Advances in Neural Information Processing Systems 22*, pp 2331–2339.
- [2007] Zunino L., Tabak B.M., Perez D.G., Garavaglia M., Rosso O.A., Inefficiency in Latin-American market indices. *The European Physical Journal B*, **60**, (1), pp 111–121.
- [2008] Zunino L., Tabak B.M., Figlio A., Perez D.G., Garavaglia M., Rosso O.A., A multifractal approach for stock market inefficiency. *Physica A*, **387**, pp 6558–6566.